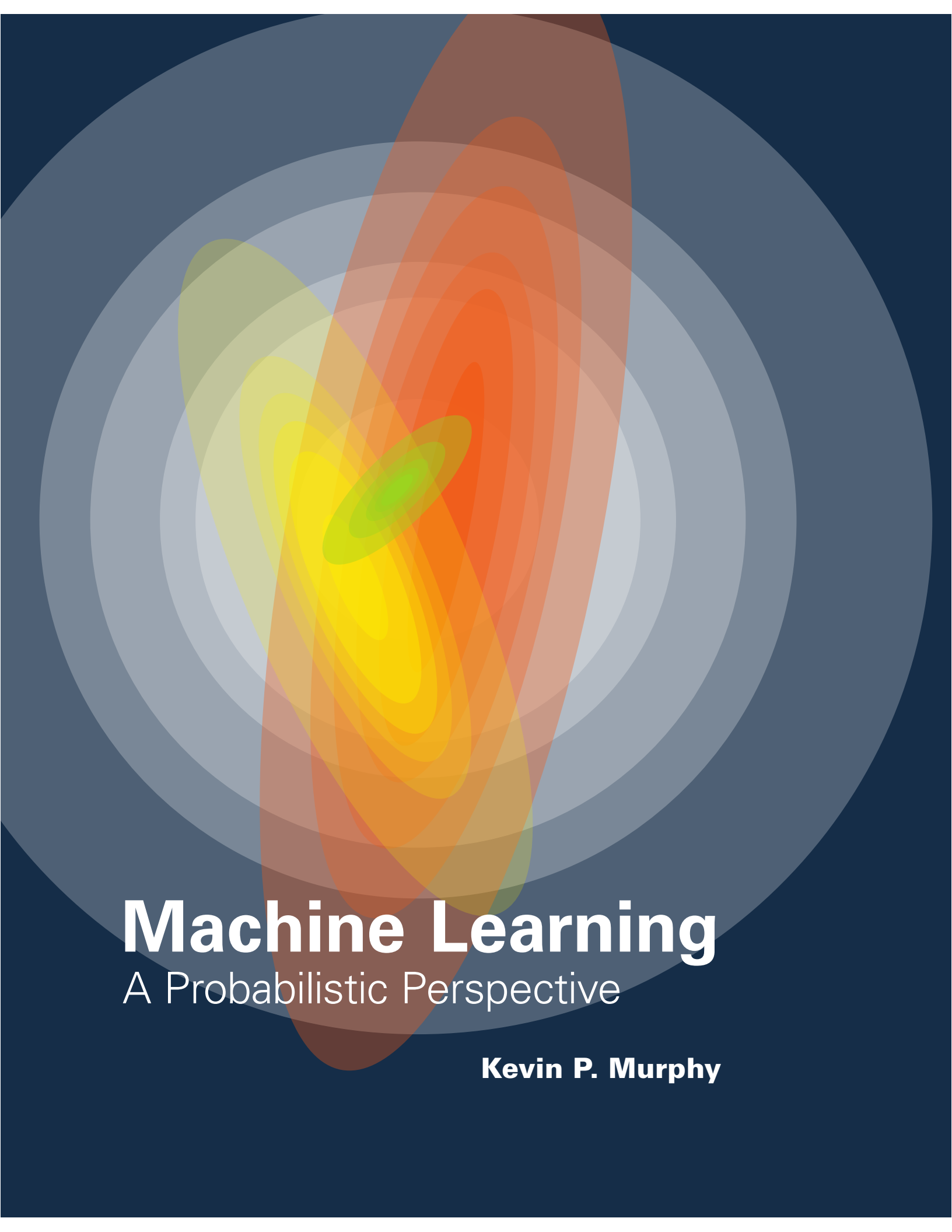




Machine Learning

A Probabilistic Perspective

Kevin P. Murphy

Machine Learning: A Probabilistic Perspective

Machine Learning

A Probabilistic Perspective

Kevin P. Murphy

The MIT Press
Cambridge, Massachusetts
London, England

© 2012 Massachusetts Institute of Technology

All rights reserved. No part of this book may be reproduced in any form by any electronic or mechanical means (including photocopying, recording, or information storage and retrieval) without permission in writing from the publisher.

For information about special quantity discounts, please email special_sales@mitpress.mit.edu

This book was set in the $\text{\LaTeX}$ programming language by the author. Printed and bound in the United States of America.

Library of Congress Cataloging-in-Publication Information

Murphy, Kevin P.
Machine learning : a probabilistic perspective / Kevin P. Murphy.
p. cm. — (Adaptive computation and machine learning series)
Includes bibliographical references and index.
ISBN 978-0-262-01802-9 (hardcover : alk. paper)
1. Machine learning. 2. Probabilities. I. Title.
Q325.5.M87 2012
006.3'1—dc23
2012004558

10 9 8 7 6 5 4 3 2 1

This book is dedicated to Alessandro, Michael and Stefano,
and to the memory of Gerard Joseph Murphy.

Contents

Preface **xxvii**

1 Introduction **1**

1.1	Machine learning: what and why?	1
1.1.1	Types of machine learning	2
1.2	Supervised learning	3
1.2.1	Classification	3
1.2.2	Regression	8
1.3	Unsupervised learning	9
1.3.1	Discovering clusters	10
1.3.2	Discovering latent factors	11
1.3.3	Discovering graph structure	13
1.3.4	Matrix completion	14
1.4	Some basic concepts in machine learning	16
1.4.1	Parametric vs non-parametric models	16
1.4.2	A simple non-parametric classifier: K -nearest neighbors	16
1.4.3	The curse of dimensionality	18
1.4.4	Parametric models for classification and regression	19
1.4.5	Linear regression	19
1.4.6	Logistic regression	21
1.4.7	Overfitting	22
1.4.8	Model selection	22
1.4.9	No free lunch theorem	24

2 Probability **27**

2.1	Introduction	27
2.2	A brief review of probability theory	28
2.2.1	Discrete random variables	28
2.2.2	Fundamental rules	28
2.2.3	Bayes rule	29
2.2.4	Independence and conditional independence	30
2.2.5	Continuous random variables	32

2.2.6	Quantiles	33
2.2.7	Mean and variance	33
2.3	Some common discrete distributions	34
2.3.1	The binomial and Bernoulli distributions	34
2.3.2	The multinomial and multinoulli distributions	35
2.3.3	The Poisson distribution	37
2.3.4	The empirical distribution	37
2.4	Some common continuous distributions	38
2.4.1	Gaussian (normal) distribution	38
2.4.2	Degenerate pdf	39
2.4.3	The Laplace distribution	41
2.4.4	The gamma distribution	41
2.4.5	The beta distribution	42
2.4.6	Pareto distribution	43
2.5	Joint probability distributions	44
2.5.1	Covariance and correlation	44
2.5.2	The multivariate Gaussian	46
2.5.3	Multivariate Student t distribution	46
2.5.4	Dirichlet distribution	47
2.6	Transformations of random variables	49
2.6.1	Linear transformations	49
2.6.2	General transformations	50
2.6.3	Central limit theorem	51
2.7	Monte Carlo approximation	52
2.7.1	Example: change of variables, the MC way	53
2.7.2	Example: estimating π by Monte Carlo integration	54
2.7.3	Accuracy of Monte Carlo approximation	54
2.8	Information theory	56
2.8.1	Entropy	56
2.8.2	KL divergence	57
2.8.3	Mutual information	59
3	<i>Generative models for discrete data</i>	65
3.1	Introduction	65
3.2	Bayesian concept learning	65
3.2.1	Likelihood	67
3.2.2	Prior	67
3.2.3	Posterior	68
3.2.4	Posterior predictive distribution	71
3.2.5	A more complex prior	72
3.3	The beta-binomial model	72
3.3.1	Likelihood	73
3.3.2	Prior	74
3.3.3	Posterior	75
3.3.4	Posterior predictive distribution	77

3.4	The Dirichlet-multinomial model	78
3.4.1	Likelihood	79
3.4.2	Prior	79
3.4.3	Posterior	79
3.4.4	Posterior predictive	81
3.5	Naive Bayes classifiers	82
3.5.1	Model fitting	83
3.5.2	Using the model for prediction	85
3.5.3	The log-sum-exp trick	86
3.5.4	Feature selection using mutual information	86
3.5.5	Classifying documents using bag of words	87
4	Gaussian models	97
4.1	Introduction	97
4.1.1	Notation	97
4.1.2	Basics	97
4.1.3	MLE for an MVN	99
4.1.4	Maximum entropy derivation of the Gaussian *	101
4.2	Gaussian discriminant analysis	101
4.2.1	Quadratic discriminant analysis (QDA)	102
4.2.2	Linear discriminant analysis (LDA)	103
4.2.3	Two-class LDA	104
4.2.4	MLE for discriminant analysis	106
4.2.5	Strategies for preventing overfitting	106
4.2.6	Regularized LDA *	107
4.2.7	Diagonal LDA	108
4.2.8	Nearest shrunken centroids classifier *	109
4.3	Inference in jointly Gaussian distributions	110
4.3.1	Statement of the result	111
4.3.2	Examples	111
4.3.3	Information form	115
4.3.4	Proof of the result *	116
4.4	Linear Gaussian systems	119
4.4.1	Statement of the result	119
4.4.2	Examples	120
4.4.3	Proof of the result *	124
4.5	Digression: The Wishart distribution *	125
4.5.1	Inverse Wishart distribution	126
4.5.2	Visualizing the Wishart distribution *	127
4.6	Inferring the parameters of an MVN	127
4.6.1	Posterior distribution of μ	128
4.6.2	Posterior distribution of Σ *	128
4.6.3	Posterior distribution of μ and Σ *	132
4.6.4	Sensor fusion with unknown precisions *	138

5	<i>Bayesian statistics</i>	149
5.1	Introduction	149
5.2	Summarizing posterior distributions	149
5.2.1	MAP estimation	149
5.2.2	Credible intervals	152
5.2.3	Inference for a difference in proportions	154
5.3	Bayesian model selection	155
5.3.1	Bayesian Occam's razor	156
5.3.2	Computing the marginal likelihood (evidence)	158
5.3.3	Bayes factors	163
5.3.4	Jeffreys-Lindley paradox *	164
5.4	Priors	165
5.4.1	Uninformative priors	165
5.4.2	Jeffreys priors *	166
5.4.3	Robust priors	168
5.4.4	Mixtures of conjugate priors	168
5.5	Hierarchical Bayes	171
5.5.1	Example: modeling related cancer rates	171
5.6	Empirical Bayes	172
5.6.1	Example: beta-binomial model	173
5.6.2	Example: Gaussian-Gaussian model	173
5.7	Bayesian decision theory	176
5.7.1	Bayes estimators for common loss functions	177
5.7.2	The false positive vs false negative tradeoff	180
5.7.3	Other topics *	184
6	<i>Frequentist statistics</i>	191
6.1	Introduction	191
6.2	Sampling distribution of an estimator	191
6.2.1	Bootstrap	192
6.2.2	Large sample theory for the MLE *	193
6.3	Frequentist decision theory	194
6.3.1	Bayes risk	195
6.3.2	Minimax risk	196
6.3.3	Admissible estimators	197
6.4	Desirable properties of estimators	200
6.4.1	Consistent estimators	200
6.4.2	Unbiased estimators	200
6.4.3	Minimum variance estimators	201
6.4.4	The bias-variance tradeoff	202
6.5	Empirical risk minimization	204
6.5.1	Regularized risk minimization	205
6.5.2	Structural risk minimization	206
6.5.3	Estimating the risk using cross validation	206
6.5.4	Upper bounding the risk using statistical learning theory *	209

6.5.5	Surrogate loss functions	210
6.6	Pathologies of frequentist statistics *	211
6.6.1	Counter-intuitive behavior of confidence intervals	212
6.6.2	p-values considered harmful	213
6.6.3	The likelihood principle	214
6.6.4	Why isn't everyone a Bayesian?	215
7	Linear regression	217
7.1	Introduction	217
7.2	Model specification	217
7.3	Maximum likelihood estimation (least squares)	217
7.3.1	Derivation of the MLE	219
7.3.2	Geometric interpretation	220
7.3.3	Convexity	221
7.4	Robust linear regression *	223
7.5	Ridge regression	225
7.5.1	Basic idea	225
7.5.2	Numerically stable computation *	227
7.5.3	Connection with PCA *	228
7.5.4	Regularization effects of big data	230
7.6	Bayesian linear regression	231
7.6.1	Computing the posterior	232
7.6.2	Computing the posterior predictive	233
7.6.3	Bayesian inference when σ^2 is unknown *	234
7.6.4	EB for linear regression (evidence procedure)	238
8	Logistic regression	245
8.1	Introduction	245
8.2	Model specification	245
8.3	Model fitting	245
8.3.1	MLE	246
8.3.2	Steepest descent	247
8.3.3	Newton's method	249
8.3.4	Iteratively reweighted least squares (IRLS)	250
8.3.5	Quasi-Newton (variable metric) methods	251
8.3.6	ℓ_2 regularization	252
8.3.7	Multi-class logistic regression	252
8.4	Bayesian logistic regression	254
8.4.1	Laplace approximation	255
8.4.2	Derivation of the BIC	255
8.4.3	Gaussian approximation for logistic regression	256
8.4.4	Approximating the posterior predictive	256
8.4.5	Residual analysis (outlier detection) *	260
8.5	Online learning and stochastic optimization	261
8.5.1	Online learning and regret minimization	262

8.5.2	Stochastic optimization and risk minimization	262
8.5.3	The LMS algorithm	264
8.5.4	The perceptron algorithm	265
8.5.5	A Bayesian view	266
8.6	Generative vs discriminative classifiers	267
8.6.1	Pros and cons of each approach	268
8.6.2	Dealing with missing data	269
8.6.3	Fisher's linear discriminant analysis (FLDA) *	271
9	Generalized linear models and the exponential family	281
9.1	Introduction	281
9.2	The exponential family	281
9.2.1	Definition	282
9.2.2	Examples	282
9.2.3	Log partition function	284
9.2.4	MLE for the exponential family	286
9.2.5	Bayes for the exponential family *	287
9.2.6	Maximum entropy derivation of the exponential family *	289
9.3	Generalized linear models (GLMs)	290
9.3.1	Basics	290
9.3.2	ML and MAP estimation	292
9.3.3	Bayesian inference	293
9.4	Probit regression	293
9.4.1	ML/MAP estimation using gradient-based optimization	294
9.4.2	Latent variable interpretation	294
9.4.3	Ordinal probit regression *	295
9.4.4	Multinomial probit models *	295
9.5	Multi-task learning	296
9.5.1	Hierarchical Bayes for multi-task learning	296
9.5.2	Application to personalized email spam filtering	296
9.5.3	Application to domain adaptation	297
9.5.4	Other kinds of prior	297
9.6	Generalized linear mixed models *	298
9.6.1	Example: semi-parametric GLMMs for medical data	298
9.6.2	Computational issues	300
9.7	Learning to rank *	300
9.7.1	The pointwise approach	301
9.7.2	The pairwise approach	301
9.7.3	The listwise approach	302
9.7.4	Loss functions for ranking	303
10	Directed graphical models (Bayes nets)	307
10.1	Introduction	307
10.1.1	Chain rule	307
10.1.2	Conditional independence	308

10.1.3	Graphical models	308
10.1.4	Graph terminology	309
10.1.5	Directed graphical models	310
10.2	Examples	311
10.2.1	Naive Bayes classifiers	311
10.2.2	Markov and hidden Markov models	312
10.2.3	Medical diagnosis	313
10.2.4	Genetic linkage analysis *	315
10.2.5	Directed Gaussian graphical models *	318
10.3	Inference	319
10.4	Learning	320
10.4.1	Plate notation	320
10.4.2	Learning from complete data	322
10.4.3	Learning with missing and/or latent variables	323
10.5	Conditional independence properties of DGMs	324
10.5.1	d-separation and the Bayes Ball algorithm (global Markov properties)	324
10.5.2	Other Markov properties of DGMs	327
10.5.3	Markov blanket and full conditionals	327
10.6	Influence (decision) diagrams *	328
11	Mixture models and the EM algorithm	337
11.1	Latent variable models	337
11.2	Mixture models	337
11.2.1	Mixtures of Gaussians	339
11.2.2	Mixture of multinoullis	340
11.2.3	Using mixture models for clustering	340
11.2.4	Mixtures of experts	342
11.3	Parameter estimation for mixture models	345
11.3.1	Unidentifiability	346
11.3.2	Computing a MAP estimate is non-convex	347
11.4	The EM algorithm	348
11.4.1	Basic idea	349
11.4.2	EM for GMMs	350
11.4.3	EM for mixture of experts	357
11.4.4	EM for DGMs with hidden variables	358
11.4.5	EM for the Student distribution *	359
11.4.6	EM for probit regression *	362
11.4.7	Theoretical basis for EM *	363
11.4.8	Online EM	365
11.4.9	Other EM variants *	367
11.5	Model selection for latent variable models	370
11.5.1	Model selection for probabilistic models	370
11.5.2	Model selection for non-probabilistic methods	370
11.6	Fitting models with missing data	372

11.6.1	EM for the MLE of an MVN with missing data	373
12	<i>Latent linear models</i>	381
12.1	Factor analysis	381
12.1.1	FA is a low rank parameterization of an MVN	381
12.1.2	Inference of the latent factors	382
12.1.3	Unidentifiability	383
12.1.4	Mixtures of factor analysers	385
12.1.5	EM for factor analysis models	386
12.1.6	Fitting FA models with missing data	387
12.2	Principal components analysis (PCA)	387
12.2.1	Classical PCA: statement of the theorem	387
12.2.2	Proof *	389
12.2.3	Singular value decomposition (SVD)	392
12.2.4	Probabilistic PCA	395
12.2.5	EM algorithm for PCA	396
12.3	Choosing the number of latent dimensions	398
12.3.1	Model selection for FA/PPCA	398
12.3.2	Model selection for PCA	399
12.4	PCA for categorical data	402
12.5	PCA for paired and multi-view data	404
12.5.1	Supervised PCA (latent factor regression)	405
12.5.2	Partial least squares	406
12.5.3	Canonical correlation analysis	407
12.6	Independent Component Analysis (ICA)	407
12.6.1	Maximum likelihood estimation	410
12.6.2	The FastICA algorithm	411
12.6.3	Using EM	414
12.6.4	Other estimation principles *	415
13	<i>Sparse linear models</i>	421
13.1	Introduction	421
13.2	Bayesian variable selection	422
13.2.1	The spike and slab model	424
13.2.2	From the Bernoulli-Gaussian model to ℓ_0 regularization	425
13.2.3	Algorithms	426
13.3	ℓ_1 regularization: basics	429
13.3.1	Why does ℓ_1 regularization yield sparse solutions?	430
13.3.2	Optimality conditions for lasso	431
13.3.3	Comparison of least squares, lasso, ridge and subset selection	435
13.3.4	Regularization path	436
13.3.5	Model selection	439
13.3.6	Bayesian inference for linear models with Laplace priors	440
13.4	ℓ_1 regularization: algorithms	441
13.4.1	Coordinate descent	441

13.4.2	LARS and other homotopy methods	441
13.4.3	Proximal and gradient projection methods	442
13.4.4	EM for lasso	447
13.5	ℓ_1 regularization: extensions	449
13.5.1	Group Lasso	449
13.5.2	Fused lasso	454
13.5.3	Elastic net (ridge and lasso combined)	455
13.6	Non-convex regularizers	457
13.6.1	Bridge regression	458
13.6.2	Hierarchical adaptive lasso	458
13.6.3	Other hierarchical priors	462
13.7	Automatic relevance determination (ARD)/sparse Bayesian learning (SBL)	463
13.7.1	ARD for linear regression	463
13.7.2	Whence sparsity?	465
13.7.3	Connection to MAP estimation	465
13.7.4	Algorithms for ARD *	466
13.7.5	ARD for logistic regression	468
13.8	Sparse coding *	468
13.8.1	Learning a sparse coding dictionary	469
13.8.2	Results of dictionary learning from image patches	470
13.8.3	Compressed sensing	472
13.8.4	Image inpainting and denoising	472
14	Kernels	479
14.1	Introduction	479
14.2	Kernel functions	479
14.2.1	RBF kernels	480
14.2.2	Kernels for comparing documents	480
14.2.3	Mercer (positive definite) kernels	481
14.2.4	Linear kernels	482
14.2.5	Matern kernels	482
14.2.6	String kernels	483
14.2.7	Pyramid match kernels	484
14.2.8	Kernels derived from probabilistic generative models	485
14.3	Using kernels inside GLMs	486
14.3.1	Kernel machines	486
14.3.2	LIVMs, RVMs, and other sparse vector machines	487
14.4	The kernel trick	488
14.4.1	Kernelized nearest neighbor classification	489
14.4.2	Kernelized K-medoids clustering	489
14.4.3	Kernelized ridge regression	492
14.4.4	Kernel PCA	493
14.5	Support vector machines (SVMs)	496
14.5.1	SVMs for regression	497
14.5.2	SVMs for classification	498

14.5.3	Choosing C	504
14.5.4	Summary of key points	504
14.5.5	A probabilistic interpretation of SVMs	505
14.6	Comparison of discriminative kernel methods	505
14.7	Kernels for building generative models	507
14.7.1	Smoothing kernels	507
14.7.2	Kernel density estimation (KDE)	508
14.7.3	From KDE to KNN	509
14.7.4	Kernel regression	510
14.7.5	Locally weighted regression	512
15	<i>Gaussian processes</i>	515
15.1	Introduction	515
15.2	GPs for regression	516
15.2.1	Predictions using noise-free observations	517
15.2.2	Predictions using noisy observations	518
15.2.3	Effect of the kernel parameters	519
15.2.4	Estimating the kernel parameters	521
15.2.5	Computational and numerical issues *	524
15.2.6	Semi-parametric GPs *	524
15.3	GPs meet GLMs	525
15.3.1	Binary classification	525
15.3.2	Multi-class classification	528
15.3.3	GPs for Poisson regression	531
15.4	Connection with other methods	532
15.4.1	Linear models compared to GPs	532
15.4.2	Linear smoothers compared to GPs	533
15.4.3	SVMs compared to GPs	534
15.4.4	LIVM and RVMs compared to GPs	534
15.4.5	Neural networks compared to GPs	535
15.4.6	Smoothing splines compared to GPs *	536
15.4.7	RKHS methods compared to GPs *	538
15.5	GP latent variable model	540
15.6	Approximation methods for large datasets	542
16	<i>Adaptive basis function models</i>	543
16.1	Introduction	543
16.2	Classification and regression trees (CART)	544
16.2.1	Basics	544
16.2.2	Growing a tree	545
16.2.3	Pruning a tree	549
16.2.4	Pros and cons of trees	550
16.2.5	Random forests	550
16.2.6	CART compared to hierarchical mixture of experts *	551
16.3	Generalized additive models	552

16.3.1	Backfitting	552	
16.3.2	Computational efficiency	553	
16.3.3	Multivariate adaptive regression splines (MARS)	553	
16.4	Boosting	554	
16.4.1	Forward stagewise additive modeling	555	
16.4.2	L2boosting	557	
16.4.3	AdaBoost	558	
16.4.4	LogitBoost	559	
16.4.5	Boosting as functional gradient descent	560	
16.4.6	Sparse boosting	561	
16.4.7	Multivariate adaptive regression trees (MART)	562	
16.4.8	Why does boosting work so well?	562	
16.4.9	A Bayesian view	563	
16.5	Feedforward neural networks (multilayer perceptrons)	563	
16.5.1	Convolutional neural networks	564	
16.5.2	Other kinds of neural networks	568	
16.5.3	A brief history of the field	568	
16.5.4	The backpropagation algorithm	569	
16.5.5	Identifiability	572	
16.5.6	Regularization	572	
16.5.7	Bayesian inference *	576	
16.6	Ensemble learning	580	
16.6.1	Stacking	580	
16.6.2	Error-correcting output codes	581	
16.6.3	Ensemble learning is not equivalent to Bayes model averaging	581	
16.7	Experimental comparison	582	
16.7.1	Low-dimensional features	582	
16.7.2	High-dimensional features	583	
16.8	Interpreting black-box models	585	
17	Markov and hidden Markov models	589	
17.1	Introduction	589	
17.2	Markov models	589	
17.2.1	Transition matrix	589	
17.2.2	Application: Language modeling	591	
17.2.3	Stationary distribution of a Markov chain *	596	
17.2.4	Application: Google's PageRank algorithm for web page ranking *	600	
17.3	Hidden Markov models	603	
17.3.1	Applications of HMMs	604	
17.4	Inference in HMMs	606	
17.4.1	Types of inference problems for temporal models	606	
17.4.2	The forwards algorithm	609	
17.4.3	The forwards-backwards algorithm	610	
17.4.4	The Viterbi algorithm	612	
17.4.5	Forwards filtering, backwards sampling	616	

17.5	Learning for HMMs	617	
17.5.1	Training with fully observed data	617	
17.5.2	EM for HMMs (the Baum-Welch algorithm)	618	
17.5.3	Bayesian methods for “fitting” HMMs *	620	
17.5.4	Discriminative training	620	
17.5.5	Model selection	621	
17.6	Generalizations of HMMs	621	
17.6.1	Variable duration (semi-Markov) HMMs	622	
17.6.2	Hierarchical HMMs	624	
17.6.3	Input-output HMMs	625	
17.6.4	Auto-regressive and buried HMMs	626	
17.6.5	Factorial HMM	627	
17.6.6	Coupled HMM and the influence model	628	
17.6.7	Dynamic Bayesian networks (DBNs)	628	
18	State space models	631	
18.1	Introduction	631	
18.2	Applications of SSMs	632	
18.2.1	SSMs for object tracking	632	
18.2.2	Robotic SLAM	633	
18.2.3	Online parameter learning using recursive least squares	636	
18.2.4	SSM for time series forecasting *	637	
18.3	Inference in LG-SSM	640	
18.3.1	The Kalman filtering algorithm	640	
18.3.2	The Kalman smoothing algorithm	643	
18.4	Learning for LG-SSM	646	
18.4.1	Identifiability and numerical stability	646	
18.4.2	Training with fully observed data	647	
18.4.3	EM for LG-SSM	647	
18.4.4	Subspace methods	647	
18.4.5	Bayesian methods for “fitting” LG-SSMs	647	
18.5	Approximate online inference for non-linear, non-Gaussian SSMs	647	
18.5.1	Extended Kalman filter (EKF)	648	
18.5.2	Unscented Kalman filter (UKF)	650	
18.5.3	Assumed density filtering (ADF)	652	
18.6	Hybrid discrete/continuous SSMs	655	
18.6.1	Inference	656	
18.6.2	Application: data association and multi-target tracking	658	
18.6.3	Application: fault diagnosis	659	
18.6.4	Application: econometric forecasting	660	
19	Undirected graphical models (Markov random fields)	661	
19.1	Introduction	661	
19.2	Conditional independence properties of UGMs	661	
19.2.1	Key properties	661	

19.2.2	An undirected alternative to d-separation	663
19.2.3	Comparing directed and undirected graphical models	664
19.3	Parameterization of MRFs	665
19.3.1	The Hammersley-Clifford theorem	665
19.3.2	Representing potential functions	667
19.4	Examples of MRFs	668
19.4.1	Ising model	668
19.4.2	Hopfield networks	669
19.4.3	Potts model	671
19.4.4	Gaussian MRFs	672
19.4.5	Markov logic networks *	674
19.5	Learning	676
19.5.1	Training maxent models using gradient methods	676
19.5.2	Training partially observed maxent models	677
19.5.3	Approximate methods for computing the MLEs of MRFs	678
19.5.4	Pseudo likelihood	678
19.5.5	Stochastic maximum likelihood	679
19.5.6	Feature induction for maxent models *	680
19.5.7	Iterative proportional fitting (IPF) *	681
19.6	Conditional random fields (CRFs)	684
19.6.1	Chain-structured CRFs, MEMMs and the label-bias problem	684
19.6.2	Applications of CRFs	686
19.6.3	CRF training	692
19.7	Structural SVMs	693
19.7.1	SSVMs: a probabilistic view	693
19.7.2	SSVMs: a non-probabilistic view	695
19.7.3	Cutting plane methods for fitting SSVMs	698
19.7.4	Online algorithms for fitting SSVMs	700
19.7.5	Latent structural SVMs	701
20	Exact inference for graphical models	707
20.1	Introduction	707
20.2	Belief propagation for trees	707
20.2.1	Serial protocol	707
20.2.2	Parallel protocol	709
20.2.3	Gaussian BP *	710
20.2.4	Other BP variants *	712
20.3	The variable elimination algorithm	714
20.3.1	The generalized distributive law *	717
20.3.2	Computational complexity of VE	717
20.3.3	A weakness of VE	720
20.4	The junction tree algorithm *	720
20.4.1	Creating a junction tree	720
20.4.2	Message passing on a junction tree	722
20.4.3	Computational complexity of JTA	725

20.4.4	JTA generalizations *	726
20.5	Computational intractability of exact inference in the worst case	726
20.5.1	Approximate inference	727
21	Variational inference	731
21.1	Introduction	731
21.2	Variational inference	732
21.2.1	Alternative interpretations of the variational objective	733
21.2.2	Forward or reverse KL? *	733
21.3	The mean field method	735
21.3.1	Derivation of the mean field update equations	736
21.3.2	Example: mean field for the Ising model	737
21.4	Structured mean field *	739
21.4.1	Example: factorial HMM	740
21.5	Variational Bayes	742
21.5.1	Example: VB for a univariate Gaussian	742
21.5.2	Example: VB for linear regression	746
21.6	Variational Bayes EM	749
21.6.1	Example: VBEM for mixtures of Gaussians *	750
21.7	Variational message passing and VIBES	756
21.8	Local variational bounds *	756
21.8.1	Motivating applications	756
21.8.2	Bohning's quadratic bound to the log-sum-exp function	758
21.8.3	Bounds for the sigmoid function	760
21.8.4	Other bounds and approximations to the log-sum-exp function *	762
21.8.5	Variational inference based on upper bounds	763
22	More variational inference	767
22.1	Introduction	767
22.2	Loopy belief propagation: algorithmic issues	767
22.2.1	A brief history	767
22.2.2	LBP on pairwise models	768
22.2.3	LBP on a factor graph	769
22.2.4	Convergence	771
22.2.5	Accuracy of LBP	774
22.2.6	Other speedup tricks for LBP *	775
22.3	Loopy belief propagation: theoretical issues *	776
22.3.1	UGMs represented in exponential family form	776
22.3.2	The marginal polytope	777
22.3.3	Exact inference as a variational optimization problem	778
22.3.4	Mean field as a variational optimization problem	779
22.3.5	LBP as a variational optimization problem	779
22.3.6	Loopy BP vs mean field	783
22.4	Extensions of belief propagation *	783
22.4.1	Generalized belief propagation	783

22.4.2	Convex belief propagation	785	
22.5	Expectation propagation	787	
22.5.1	EP as a variational inference problem	788	
22.5.2	Optimizing the EP objective using moment matching	789	
22.5.3	EP for the clutter problem	791	
22.5.4	LBP is a special case of EP	792	
22.5.5	Ranking players using TrueSkill	793	
22.5.6	Other applications of EP	799	
22.6	MAP state estimation	799	
22.6.1	Linear programming relaxation	799	
22.6.2	Max-product belief propagation	800	
22.6.3	Graphcuts	801	
22.6.4	Experimental comparison of graphcuts and BP	804	
22.6.5	Dual decomposition	806	
23	Monte Carlo inference	815	
23.1	Introduction	815	
23.2	Sampling from standard distributions	815	
23.2.1	Using the cdf	815	
23.2.2	Sampling from a Gaussian (Box-Muller method)	817	
23.3	Rejection sampling	817	
23.3.1	Basic idea	817	
23.3.2	Example	818	
23.3.3	Application to Bayesian statistics	819	
23.3.4	Adaptive rejection sampling	819	
23.3.5	Rejection sampling in high dimensions	820	
23.4	Importance sampling	820	
23.4.1	Basic idea	820	
23.4.2	Handling unnormalized distributions	821	
23.4.3	Importance sampling for a DGM: likelihood weighting	822	
23.4.4	Sampling importance resampling (SIR)	822	
23.5	Particle filtering	823	
23.5.1	Sequential importance sampling	824	
23.5.2	The degeneracy problem	825	
23.5.3	The resampling step	825	
23.5.4	The proposal distribution	827	
23.5.5	Application: robot localization	828	
23.5.6	Application: visual object tracking	828	
23.5.7	Application: time series forecasting	831	
23.6	Rao-Blackwellised particle filtering (RBPF)	831	
23.6.1	RBPF for switching LG-SSMs	831	
23.6.2	Application: tracking a maneuvering target	832	
23.6.3	Application: Fast SLAM	834	
24	Markov chain Monte Carlo (MCMC) inference	837	

24.1	Introduction	837	
24.2	Gibbs sampling	838	
24.2.1	Basic idea	838	
24.2.2	Example: Gibbs sampling for the Ising model	838	
24.2.3	Example: Gibbs sampling for inferring the parameters of a GMM	840	
24.2.4	Collapsed Gibbs sampling *	841	
24.2.5	Gibbs sampling for hierarchical GLMs	844	
24.2.6	BUGS and JAGS	846	
24.2.7	The Imputation Posterior (IP) algorithm	847	
24.2.8	Blocking Gibbs sampling	847	
24.3	Metropolis Hastings algorithm	848	
24.3.1	Basic idea	848	
24.3.2	Gibbs sampling is a special case of MH	849	
24.3.3	Proposal distributions	850	
24.3.4	Adaptive MCMC	853	
24.3.5	Initialization and mode hopping	854	
24.3.6	Why MH works *	854	
24.3.7	Reversible jump (trans-dimensional) MCMC *	855	
24.4	Speed and accuracy of MCMC	856	
24.4.1	The burn-in phase	856	
24.4.2	Mixing rates of Markov chains *	857	
24.4.3	Practical convergence diagnostics	858	
24.4.4	Accuracy of MCMC	860	
24.4.5	How many chains?	862	
24.5	Auxiliary variable MCMC *	863	
24.5.1	Auxiliary variable sampling for logistic regression	863	
24.5.2	Slice sampling	864	
24.5.3	Swendsen Wang	866	
24.5.4	Hybrid/Hamiltonian MCMC *	868	
24.6	Annealing methods	868	
24.6.1	Simulated annealing	869	
24.6.2	Annealed importance sampling	871	
24.6.3	Parallel tempering	871	
24.7	Approximating the marginal likelihood	872	
24.7.1	The candidate method	872	
24.7.2	Harmonic mean estimate	872	
24.7.3	Annealed importance sampling	873	
25	Clustering	875	
25.1	Introduction	875	
25.1.1	Measuring (dis)similarity	875	
25.1.2	Evaluating the output of clustering methods *	876	
25.2	Dirichlet process mixture models	879	
25.2.1	From finite to infinite mixture models	879	
25.2.2	The Dirichlet process	882	

25.2.3	Applying Dirichlet processes to mixture modeling	885
25.2.4	Fitting a DP mixture model	886
25.3	Affinity propagation	887
25.4	Spectral clustering	890
25.4.1	Graph Laplacian	891
25.4.2	Normalized graph Laplacian	892
25.4.3	Example	893
25.5	Hierarchical clustering	893
25.5.1	Agglomerative clustering	895
25.5.2	Divisive clustering	898
25.5.3	Choosing the number of clusters	899
25.5.4	Bayesian hierarchical clustering	899
25.6	Clustering datapoints and features	901
25.6.1	Biclustering	903
25.6.2	Multi-view clustering	903
26	Graphical model structure learning	907
26.1	Introduction	907
26.2	Structure learning for knowledge discovery	908
26.2.1	Relevance networks	908
26.2.2	Dependency networks	909
26.3	Learning tree structures	910
26.3.1	Directed or undirected tree?	911
26.3.2	Chow-Liu algorithm for finding the ML tree structure	912
26.3.3	Finding the MAP forest	912
26.3.4	Mixtures of trees	914
26.4	Learning DAG structures	914
26.4.1	Markov equivalence	914
26.4.2	Exact structural inference	916
26.4.3	Scaling up to larger graphs	920
26.5	Learning DAG structure with latent variables	922
26.5.1	Approximating the marginal likelihood when we have missing data	922
26.5.2	Structural EM	925
26.5.3	Discovering hidden variables	926
26.5.4	Case study: Google's Rephil	928
26.5.5	Structural equation models *	929
26.6	Learning causal DAGs	931
26.6.1	Causal interpretation of DAGs	931
26.6.2	Using causal DAGs to resolve Simpson's paradox	933
26.6.3	Learning causal DAG structures	935
26.7	Learning undirected Gaussian graphical models	938
26.7.1	MLE for a GGM	938
26.7.2	Graphical lasso	939
26.7.3	Bayesian inference for GGM structure *	941
26.7.4	Handling non-Gaussian data using copulas *	942

26.8	Learning undirected discrete graphical models	942
26.8.1	Graphical lasso for MRFs/CRFs	942
26.8.2	Thin junction trees	944
27	<i>Latent variable models for discrete data</i>	945
27.1	Introduction	945
27.2	Distributed state LVMs for discrete data	946
27.2.1	Mixture models	946
27.2.2	Exponential family PCA	947
27.2.3	LDA and mPCA	948
27.2.4	GaP model and non-negative matrix factorization	949
27.3	Latent Dirichlet allocation (LDA)	950
27.3.1	Basics	950
27.3.2	Unsupervised discovery of topics	953
27.3.3	Quantitatively evaluating LDA as a language model	953
27.3.4	Fitting using (collapsed) Gibbs sampling	955
27.3.5	Example	956
27.3.6	Fitting using batch variational inference	957
27.3.7	Fitting using online variational inference	959
27.3.8	Determining the number of topics	960
27.4	Extensions of LDA	961
27.4.1	Correlated topic model	961
27.4.2	Dynamic topic model	962
27.4.3	LDA-HMM	963
27.4.4	Supervised LDA	967
27.5	LVMs for graph-structured data	970
27.5.1	Stochastic block model	971
27.5.2	Mixed membership stochastic block model	973
27.5.3	Relational topic model	974
27.6	LVMs for relational data	975
27.6.1	Infinite relational model	976
27.6.2	Probabilistic matrix factorization for collaborative filtering	979
27.7	Restricted Boltzmann machines (RBMs)	983
27.7.1	Varieties of RBMs	985
27.7.2	Learning RBMs	987
27.7.3	Applications of RBMs	991
28	<i>Deep learning</i>	995
28.1	Introduction	995
28.2	Deep generative models	995
28.2.1	Deep directed networks	996
28.2.2	Deep Boltzmann machines	996
28.2.3	Deep belief networks	997
28.2.4	Greedy layer-wise learning of DBNs	998
28.3	Deep neural networks	999

28.3.1	Deep multi-layer perceptrons	999	
28.3.2	Deep auto-encoders	1000	
28.3.3	Stacked denoising auto-encoders	1001	
28.4	Applications of deep networks	1001	
28.4.1	Handwritten digit classification using DBNs	1001	
28.4.2	Data visualization and feature discovery using deep auto-encoders		1002
28.4.3	Information retrieval using deep auto-encoders (semantic hashing)		1003
28.4.4	Learning audio features using 1d convolutional DBNs	1004	
28.4.5	Learning image features using 2d convolutional DBNs	1005	
28.5	Discussion	1005	
Notation		1009	
Bibliography		1015	
Indexes		1047	
	Index to code	1047	
	Index to keywords	1050	

Preface

Introduction

With the ever increasing amounts of data in electronic form, the need for automated methods for data analysis continues to grow. The goal of machine learning is to develop methods that can automatically detect patterns in data, and then to use the uncovered patterns to predict future data or other outcomes of interest. Machine learning is thus closely related to the fields of statistics and data mining, but differs slightly in terms of its emphasis and terminology. This book provides a detailed introduction to the field, and includes worked examples drawn from application domains such as molecular biology, text processing, computer vision, and robotics.

Target audience

This book is suitable for upper-level undergraduate students and beginning graduate students in computer science, statistics, electrical engineering, econometrics, or any one else who has the appropriate mathematical background. Specifically, the reader is assumed to already be familiar with basic multivariate calculus, probability, linear algebra, and computer programming. Prior exposure to statistics is helpful but not necessary.

A probabilistic approach

This book adopts the view that the best way to make machines that can learn from data is to use the tools of probability theory, which has been the mainstay of statistics and engineering for centuries. Probability theory can be applied to any problem involving uncertainty. In machine learning, uncertainty comes in many forms: what is the best prediction (or decision) given some data? what is the best model given some data? what measurement should I perform next? etc.

The systematic application of probabilistic reasoning to all inferential problems, including inferring parameters of statistical models, is sometimes called a Bayesian approach. However, this term tends to elicit very strong reactions (either positive or negative, depending on who you ask), so we prefer the more neutral term “probabilistic approach”. Besides, we will often use techniques such as maximum likelihood estimation, which are not Bayesian methods, but certainly fall within the probabilistic paradigm.

Rather than describing a cookbook of different heuristic methods, this book stresses a principled model-based approach to machine learning. For any given model, a variety of algorithms

can often be applied. Conversely, any given algorithm can often be applied to a variety of models. This kind of modularity, where we distinguish model from algorithm, is good pedagogy and good engineering.

We will often use the language of graphical models to specify our models in a concise and intuitive way. In addition to aiding comprehension, the graph structure aids in developing efficient algorithms, as we will see. However, this book is not primarily about graphical models; it is about probabilistic modeling in general.

A practical approach

Nearly all of the methods described in this book have been implemented in a MATLAB software package called **PMTK**, which stands for probabilistic modeling toolkit. This is freely available from pmtk3.googlecode.com (the digit 3 refers to the third edition of the toolkit, which is the one used in this version of the book). There are also a variety of supporting files, written by other people, available at pmtksupport.googlecode.com. These will be downloaded automatically, if you follow the setup instructions described on the PMTK website.

MATLAB is a high-level, interactive scripting language ideally suited to numerical computation and data visualization, and can be purchased from www.mathworks.com. Some of the code requires the Statistics toolbox, which needs to be purchased separately. There is also a free version of Matlab called **Octave**, available at <http://www.gnu.org/software/octave/>, which supports most of the functionality of MATLAB. Some (but not all) of the code in this book also works in Octave. See the PMTK website for details.

PMTK was used to generate many of the figures in this book; the source code for these figures is included on the PMTK website, allowing the reader to easily see the effects of changing the data or algorithm or parameter settings. The book refers to files by name, e.g., `naiveBayesFit`. In order to find the corresponding file, you can use two methods: within Matlab you can type `which naiveBayesFit` and it will return the full path to the file; or, if you do not have Matlab but want to read the source code anyway, you can use your favorite search engine, which should return the corresponding file from the pmtk3.googlecode.com website.

Details on *how to use* PMTK can be found on the website, which will be updated over time. Details on the *underlying theory* behind these methods can be found in this book.

Acknowledgments

A book this large is obviously a team effort. I would especially like to thank the following people: my wife Margaret, for keeping the home fires burning as I toiled away in my office for the last six years; Matt Dunham, who created many of the figures in this book, and who wrote much of the code in PMTK; Baback Moghaddam, who gave extremely detailed feedback on every page of an earlier draft of the book; Chris Williams, who also gave very detailed feedback; Cody Severinski and Wei-Lwun Lu, who assisted with figures; generations of UBC students, who gave helpful comments on earlier drafts; Daphne Koller, Nir Friedman, and Chris Manning, for letting me use their latex style files; Stanford University, Google Research and Skyline College for hosting me during part of my sabbatical; and various Canadian funding agencies (NSERC, CRC and CIFAR) who have supported me financially over the years.

In addition, I would like to thank the following people for giving me helpful feedback on

parts of the book, and/or for sharing figures, code, exercises or even (in some cases) text: David Blei, Hannes Bretschneider, Greg Corrado, Arnaud Doucet, Mario Figueiredo, Nando de Freitas, Mark Girolami, Gabriel Goh, Tom Griffiths, Katherine Heller, Geoff Hinton, Aapo Hyvarinen, Tommi Jaakkola, Mike Jordan, Charles Kemp, Emtiyaz Khan, Bonnie Kirkpatrick, Daphne Koller, Zico Kolter, Honglak Lee, Julien Mairal, Andrew McPherson, Tom Minka, Ian Nabney, Arthur Pope, Carl Rasmussen, Ryan Rifkin, Ruslan Salakhutdinov, Mark Schmidt, Daniel Selsam, David Sontag, Erik Sudderth, Josh Tenenbaum, Kai Yu, Martin Wainwright, Yair Weiss.

Kevin Patrick Murphy
Palo Alto, California
June 2012

1 *Introduction*

1.1 Machine learning: what and why?

We are drowning in information and starving for knowledge. — John Naisbitt.

We are entering the era of **big data**. For example, there are about 1 trillion web pages¹; one hour of video is uploaded to YouTube every second, amounting to 10 years of content every day²; the genomes of 1000s of people, each of which has a length of 3.8×10^9 base pairs, have been sequenced by various labs; Walmart handles more than 1M transactions per hour and has databases containing more than 2.5 petabytes (2.5×10^{15}) of information (Cukier 2010); and so on.

This deluge of data calls for automated methods of data analysis, which is what **machine learning** provides. In particular, we define machine learning as a set of methods that can automatically detect patterns in data, and then use the uncovered patterns to predict future data, or to perform other kinds of decision making under uncertainty (such as planning how to collect more data!).

This book adopts the view that the best way to solve such problems is to use the tools of probability theory. Probability theory can be applied to any problem involving uncertainty. In machine learning, uncertainty comes in many forms: what is the best prediction about the future given some past data? what is the best model to explain some data? what measurement should I perform next? etc. The probabilistic approach to machine learning is closely related to the field of statistics, but differs slightly in terms of its emphasis and terminology³.

We will describe a wide variety of probabilistic models, suitable for a wide variety of data and tasks. We will also describe a wide variety of algorithms for learning and using such models. The goal is not to develop a cook book of ad hoc techniques, but instead to present a unified view of the field through the lens of probabilistic modeling and inference. Although we will pay attention to computational efficiency, details on how to scale these methods to truly massive datasets are better described in other books, such as (Rajaraman and Ullman 2011; Bekkerman et al. 2011).

1. <http://googleblog.blogspot.com/2008/07/we-knew-web-was-big.html>

2. Source: http://www.youtube.com/t/press_statistics.

3. Rob Tibshirani, a statistician at Stanford university, has created an amusing comparison between machine learning and statistics, available at <http://www-stat.stanford.edu/~tibs/stat315a/glossary.pdf>.

It should be noted, however, that even when one has an apparently massive data set, the effective number of data points for certain cases of interest might be quite small. In fact, data across a variety of domains exhibits a property known as the **long tail**, which means that a few things (e.g., words) are very common, but most things are quite rare (see Section 2.4.6 for details). For example, 20% of Google searches each day have never been seen before⁴. This means that the core statistical issues that we discuss in this book, concerning generalizing from relatively small samples sizes, are still very relevant even in the big data era.

1.1.1 Types of machine learning

Machine learning is usually divided into two main types. In the **predictive** or **supervised learning** approach, the goal is to learn a mapping from inputs $\mathbf{x}$ to outputs y , given a labeled set of input-output pairs $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$. Here $\mathcal{D}$ is called the **training set**, and N is the number of training examples.

In the simplest setting, each training input $\mathbf{x}_i$ is a D -dimensional vector of numbers, representing, say, the height and weight of a person. These are called **features**, **attributes** or **covariates**. In general, however, $\mathbf{x}_i$ could be a complex structured object, such as an image, a sentence, an email message, a time series, a molecular shape, a graph, etc.

Similarly the form of the output or **response variable** can in principle be anything, but most methods assume that y_i is a **categorical** or **nominal** variable from some finite set, $y_i \in \{1, \dots, C\}$ (such as male or female), or that y_i is a real-valued scalar (such as income level). When y_i is categorical, the problem is known as **classification** or **pattern recognition**, and when y_i is real-valued, the problem is known as **regression**. Another variant, known as **ordinal regression**, occurs where label space $\mathcal{Y}$ has some natural ordering, such as grades A–F.

The second main type of machine learning is the **descriptive** or **unsupervised learning** approach. Here we are only given inputs, $\mathcal{D} = \{\mathbf{x}_i\}_{i=1}^N$, and the goal is to find “interesting patterns” in the data. This is sometimes called **knowledge discovery**. This is a much less well-defined problem, since we are not told what kinds of patterns to look for, and there is no obvious error metric to use (unlike supervised learning, where we can compare our prediction of y for a given $\mathbf{x}$ to the observed value).

There is a third type of machine learning, known as **reinforcement learning**, which is somewhat less commonly used. This is useful for learning how to act or behave when given occasional reward or punishment signals. (For example, consider how a baby learns to walk.) Unfortunately, RL is beyond the scope of this book, although we do discuss decision theory in Section 5.7, which is the basis of RL. See e.g., (Kaelbling et al. 1996; Sutton and Barto 1998; Russell and Norvig 2010; Szepesvari 2010; Wiering and van Otterlo 2012) for more information on RL.

4.

<http://certifiedknowledge.org/blog/are-search-queries-becoming-even-more-unique-statistics-from-google>.

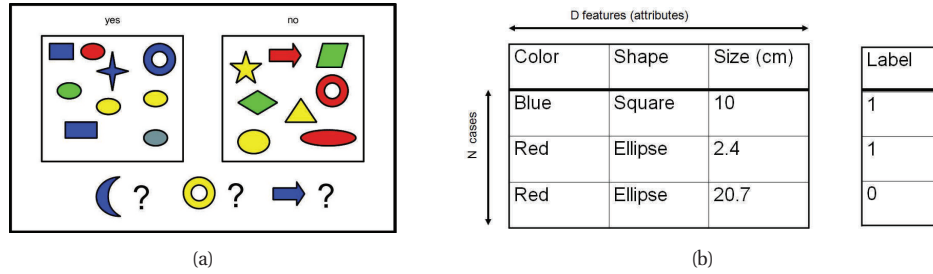


Figure 1.1 Left: Some labeled training examples of colored shapes, along with 3 unlabeled test cases. Right: Representing the training data as an $N \times D$ design matrix. Row i represents the feature vector $\mathbf{x}_i$. The last column is the label, $y_i \in \{0, 1\}$. Based on a figure by Leslie Kaelbling.

1.2 Supervised learning

We begin our investigation of machine learning by discussing supervised learning, which is the form of ML most widely used in practice.

1.2.1 Classification

In this section, we discuss classification. Here the goal is to learn a mapping from inputs $\mathbf{x}$ to outputs y , where $y \in \{1, \dots, C\}$, with C being the number of classes. If $C = 2$, this is called **binary classification** (in which case we often assume $y \in \{0, 1\}$); if $C > 2$, this is called **multiclass classification**. If the class labels are not mutually exclusive (e.g., somebody may be classified as tall and strong), we call it **multi-label classification**, but this is best viewed as predicting multiple related binary class labels (a so-called **multiple output model**). When we use the term “classification”, we will mean multiclass classification with a single output, unless we state otherwise.

One way to formalize the problem is as **function approximation**. We assume $y = f(\mathbf{x})$ for some unknown function f , and the goal of learning is to estimate the function f given a labeled training set, and then to make predictions using $\hat{y} = \hat{f}(\mathbf{x})$. (We use the hat symbol to denote an estimate.) Our main goal is to make predictions on novel inputs, meaning ones that we have not seen before (this is called **generalization**), since predicting the response on the training set is easy (we can just look up the answer).

1.2.1.1 Example

As a simple toy example of classification, consider the problem illustrated in Figure 1.1(a). We have two classes of object which correspond to labels 0 and 1. The inputs are colored shapes. These have been described by a set of D features or attributes, which are stored in an $N \times D$ design matrix $\mathbf{X}$, shown in Figure 1.1(b). The input features $\mathbf{x}$ can be discrete, continuous or a combination of the two. In addition to the inputs, we have a vector of training labels $\mathbf{y}$.

In Figure 1.1, the test cases are a blue crescent, a yellow circle and a blue arrow. None of these have been seen before. Thus we are required to **generalize** beyond the training set. A

reasonable guess is that blue crescent should be $y = 1$, since all blue shapes are labeled 1 in the training set. The yellow circle is harder to classify, since some yellow things are labeled $y = 1$ and some are labeled $y = 0$, and some circles are labeled $y = 1$ and some $y = 0$. Consequently it is not clear what the right label should be in the case of the yellow circle. Similarly, the correct label for the blue arrow is unclear.

1.2.1.2 The need for probabilistic predictions

To handle ambiguous cases, such as the yellow circle above, it is desirable to return a probability. The reader is assumed to already have some familiarity with basic concepts in probability. If not, please consult Chapter 2 for a refresher, if necessary.

We will denote the probability distribution over possible labels, given the input vector $\mathbf{x}$ and training set $\mathcal{D}$ by $p(y|\mathbf{x}, \mathcal{D})$. In general, this represents a vector of length C . (If there are just two classes, it is sufficient to return the single number $p(y = 1|\mathbf{x}, \mathcal{D})$, since $p(y = 1|\mathbf{x}, \mathcal{D}) + p(y = 0|\mathbf{x}, \mathcal{D}) = 1$.) In our notation, we make explicit that the probability is conditional on the test input $\mathbf{x}$, as well as the training set $\mathcal{D}$, by putting these terms on the right hand side of the conditioning bar $|$. We are also implicitly conditioning on the form of model that we use to make predictions. When choosing between different models, we will make this assumption explicit by writing $p(y|\mathbf{x}, \mathcal{D}, M)$, where M denotes the model. However, if the model is clear from context, we will drop M from our notation for brevity.

Given a probabilistic output, we can always compute our “best guess” as to the “true label” using

$$\hat{y} = \hat{f}(\mathbf{x}) = \arg\max_{c=1}^C p(y = c|\mathbf{x}, \mathcal{D}) \quad (1.1)$$

This corresponds to the most probable class label, and is called the **mode** of the distribution $p(y|\mathbf{x}, \mathcal{D})$; it is also known as a **MAP estimate** (MAP stands for **maximum a posteriori**). Using the most probable label makes intuitive sense, but we will give a more formal justification for this procedure in Section 5.7.

Now consider a case such as the yellow circle, where $p(\hat{y}|\mathbf{x}, \mathcal{D})$ is far from 1.0. In such a case we are not very confident of our answer, so it might be better to say “I don’t know” instead of returning an answer that we don’t really trust. This is particularly important in domains such as medicine and finance where we may be risk averse, as we explain in Section 5.7. Another application where it is important to assess risk is when playing TV game shows, such as Jeopardy. In this game, contestants have to solve various word puzzles and answer a variety of trivia questions, but if they answer incorrectly, they lose money. In 2011, IBM unveiled a computer system called Watson which beat the top human Jeopardy champion. Watson uses a variety of interesting techniques (Ferrucci et al. 2010), but the most pertinent one for our present purposes is that it contains a module that estimates how confident it is of its answer. The system only chooses to “buzz in” its answer if sufficiently confident it is correct. Similarly, Google has a system known as SmartASS (ad selection system) that predicts the probability you will click on an ad based on your search history and other user and ad-specific features (Metz 2010). This probability is known as the **click-through rate** or **CTR**, and can be used to maximize expected profit. We will discuss some of the basic principles behind systems such as SmartASS later in this book.

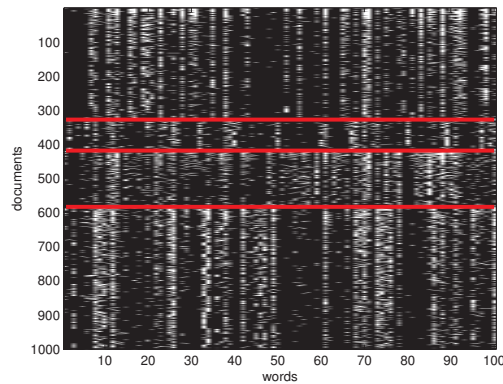


Figure 1.2 Subset of size 16242 x 100 of the 20-newsgroups data. We only show 1000 rows, for clarity. Each row is a document (represented as a bag-of-words bit vector), each column is a word. The red lines separate the 4 classes, which are (in descending order) comp, rec, sci, talk (these are the titles of USENET groups). We can see that there are subsets of words whose presence or absence is indicative of the class. The data is available from <http://cs.nyu.edu/~roweis/data.html>. Figure generated by newsgroupsVisualize.

1.2.1.3 Real-world applications

Classification is probably the most widely used form of machine learning, and has been used to solve many interesting and often difficult real-world problems. We have already mentioned some important applications. We give a few more examples below.

Document classification and email spam filtering

In **document classification**, the goal is to classify a document, such as a web page or email message, into one of C classes, that is, to compute $p(y = c | \mathbf{x}, \mathcal{D})$, where $\mathbf{x}$ is some representation of the text. A special case of this is **email spam filtering**, where the classes are spam $y = 1$ or ham $y = 0$.

Most classifiers assume that the input vector $\mathbf{x}$ has a fixed size. A common way to represent variable-length documents in feature-vector format is to use a **bag of words** representation. This is explained in detail in Section 3.4.4.1, but the basic idea is to define $x_{ij} = 1$ iff word j occurs in document i . If we apply this transformation to every document in our data set, we get a binary document $\times$ word co-occurrence matrix: see Figure 1.2 for an example. Essentially the document classification problem has been reduced to one that looks for subtle changes in the pattern of bits. For example, we may notice that most spam messages have a high probability of containing the words “buy”, “cheap”, “viagra”, etc. In Exercise 8.1 and Exercise 8.2, you will get hands-on experience applying various classification techniques to the spam filtering problem.

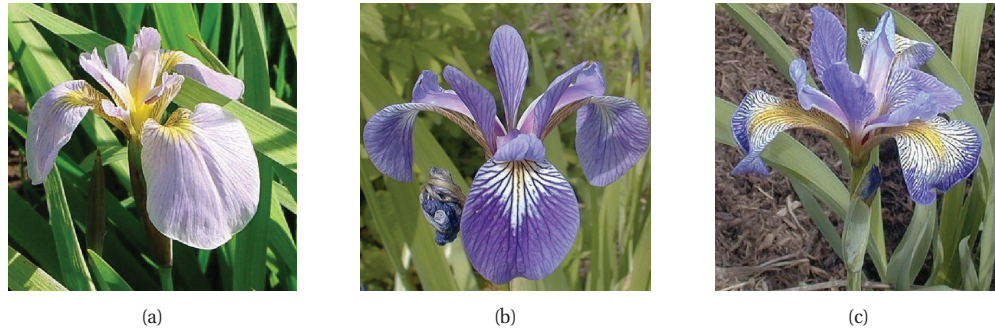


Figure 1.3 Three types of iris flowers: setosa, versicolor and virginica. Source: <http://www.statlab.uni-heidelberg.de/data/iris/>. Used with kind permission of Dennis Kramb and SIGNA.

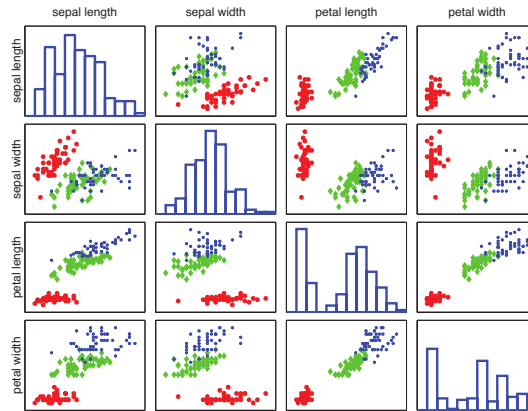


Figure 1.4 Visualization of the Iris data as a pairwise scatter plot. The diagonal plots the marginal histograms of the 4 features. The off diagonals contain scatterplots of all possible pairs of features. Red circle = setosa, green diamond = versicolor, blue star = virginica. Figure generated by `fisheririsDemo`.

Classifying flowers

Figure 1.3 gives another example of classification, due to the statistician Ronald Fisher. The goal is to learn to distinguish three different kinds of iris flower, called setosa, versicolor and virginica. Fortunately, rather than working directly with images, a botanist has already extracted 4 useful features or characteristics: sepal length and width, and petal length and width. (Such **feature extraction** is an important, but difficult, task. Most machine learning methods use features chosen by some human. Later we will discuss some methods that can learn good features from the data.) If we make a **scatter plot** of the iris data, as in Figure 1.4, we see that it is easy to distinguish setosas (red circles) from the other two classes by just checking if their petal length

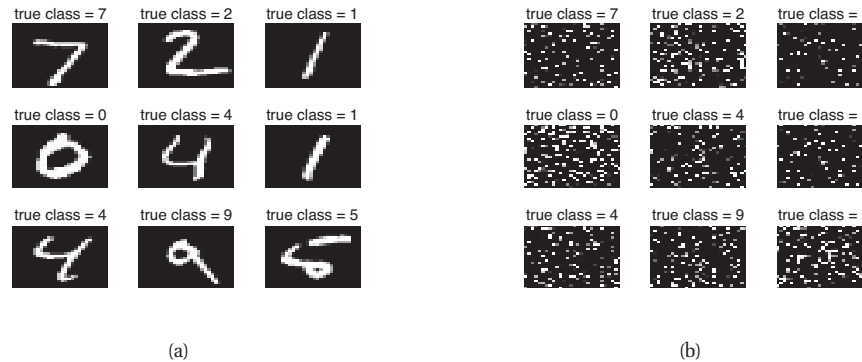


Figure 1.5 (a) First 9 test MNIST gray-scale images. (b) Same as (a), but with the features permuted randomly. Classification performance is identical on both versions of the data (assuming the training data is permuted in an identical way). Figure generated by `shuffledDigitsDemo`.

or width is below some threshold. However, distinguishing *versicolor* from *virginica* is slightly harder; any decision will need to be based on at least two features. (It is always a good idea to perform **exploratory data analysis**, such as plotting the data, before applying a machine learning method.)

Image classification and handwriting recognition

Now consider the harder problem of classifying images directly, where a human has not pre-processed the data. We might want to classify the image as a whole, e.g., is it an indoors or outdoors scene? is it a horizontal or vertical photo? does it contain a dog or not? This is called **image classification**.

In the special case that the images consist of isolated handwritten letters and digits, for example, in a postal or ZIP code on a letter, we can use classification to perform **handwriting recognition**. A standard dataset used in this area is known as **MNIST**, which stands for “Modified National Institute of Standards”⁵. (The term “modified” is used because the images have been preprocessed to ensure the digits are mostly in the center of the image.) This dataset contains 60,000 training images and 10,000 test images of the digits 0 to 9, as written by various people. The images are size 28×28 and have grayscale values in the range 0 : 255. See Figure 1.5(a) for some example images.

Many generic classification methods ignore any structure in the input features, such as spatial layout. Consequently, they can also just as easily handle data that looks like Figure 1.5(b), which is the same data except we have randomly permuted the order of all the features. (You will verify this in Exercise 1.1.) This flexibility is both a blessing (since the methods are general purpose) and a curse (since the methods ignore an obviously useful source of information). We will discuss methods for exploiting structure in the input features later in the book.

⁵. Available from <http://yann.lecun.com/exdb/mnist/>.



Figure 1.6 Example of face detection. (a) Input image (Murphy family, photo taken 5 August 2010). Used with kind permission of Bernard Diedrich of Sherwood Studios. (b) Output of classifier, which detected 5 faces at different poses. This was produced using the online demo at <http://demo.pittpatt.com/>. The classifier was trained on 1000s of manually labeled images of faces and non-faces, and then was applied to a dense set of overlapping patches in the test image. Only the patches whose probability of containing a face was sufficiently high were returned. Used with kind permission of Pittpatt.com

Face detection and recognition

A harder problem is to find objects within an image; this is called **object detection** or **object localization**. An important special case of this is **face detection**. One approach to this problem is to divide the image into many small overlapping patches at different locations, scales and orientations, and to classify each such patch based on whether it contains face-like texture or not. This is called a **sliding window detector**. The system then returns those locations where the probability of face is sufficiently high. See Figure 1.6 for an example. Such face detection systems are built-in to most modern digital cameras; the locations of the detected faces are used to determine the center of the auto-focus. Another application is automatically blurring out faces in Google's StreetView system.

Having found the faces, one can then proceed to perform **face recognition**, which means estimating the identity of the person (see Figure 1.10(a)). In this case, the number of class labels might be very large. Also, the features one should use are likely to be different than in the face detection problem: for recognition, subtle differences between faces such as hairstyle may be important for determining identity, but for detection, it is important to be **invariant** to such details, and to just focus on the differences between faces and non-faces. For more information about visual object detection, see e.g., (Szeliski 2010).

1.2.2 Regression

Regression is just like classification except the response variable is continuous. Figure 1.7 shows a simple example: we have a single real-valued input $x_i \in \mathbb{R}$, and a single real-valued response $y_i \in \mathbb{R}$. We consider fitting two models to the data: a straight line and a quadratic function. (We explain how to fit such models below.) Various extensions of this basic problem can arise, such as having high-dimensional inputs, outliers, non-smooth responses, etc. We will discuss ways to handle such problems later in the book.

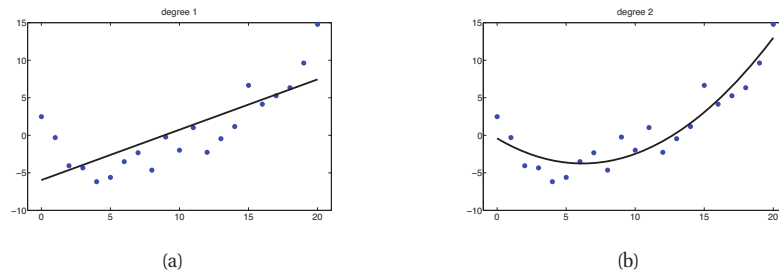


Figure 1.7 (a) Linear regression on some 1d data. (b) Same data with polynomial regression (degree 2). Figure generated by `linregPolyVsDegree`.

Here are some examples of real-world regression problems.

- Predict tomorrow's stock market price given current market conditions and other possible side information.
- Predict the age of a viewer watching a given video on YouTube.
- Predict the location in 3d space of a robot arm end effector, given control signals (torques) sent to its various motors.
- Predict the amount of prostate specific antigen (PSA) in the body as a function of a number of different clinical measurements.
- Predict the temperature at any location inside a building using weather data, time, door sensors, etc.

1.3 Unsupervised learning

We now consider **unsupervised learning**, where we are just given output data, without any inputs. The goal is to discover “interesting structure” in the data; this is sometimes called **knowledge discovery**. Unlike supervised learning, we are not told what the desired output is for each input. Instead, we will formalize our task as one of **density estimation**, that is, we want to build models of the form $p(\mathbf{x}_i|\boldsymbol{\theta})$. There are two differences from the supervised case. First, we have written $p(\mathbf{x}_i|\boldsymbol{\theta})$ instead of $p(y_i|\mathbf{x}_i, \boldsymbol{\theta})$; that is, supervised learning is conditional density estimation, whereas unsupervised learning is unconditional density estimation. Second, $\mathbf{x}_i$ is a vector of features, so we need to create multivariate probability models. By contrast, in supervised learning, y_i is usually just a single variable that we are trying to predict. This means that for most supervised learning problems, we can use univariate probability models (with input-dependent parameters), which significantly simplifies the problem. (We will discuss multi-output classification in Chapter 19, where we will see that it also involves multivariate probability models.)

Unsupervised learning is arguably more typical of human and animal learning. It is also more widely applicable than supervised learning, since it does not require a human expert to

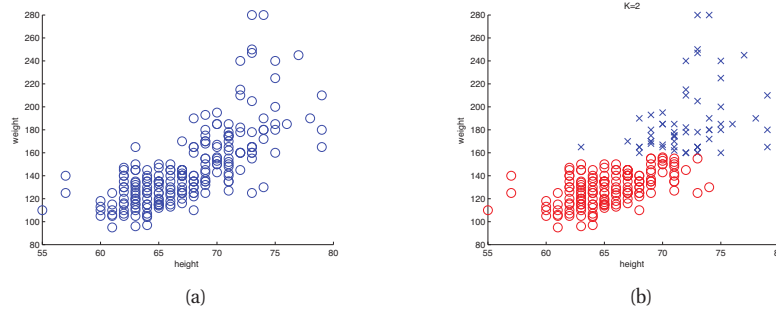


Figure 1.8 (a) The height and weight of some people. (b) A possible clustering using $K = 2$ clusters. Figure generated by `kmeansHeightWeight`.

manually label the data. Labeled data is not only expensive to acquire⁶, but it also contains relatively little information, certainly not enough to reliably estimate the parameters of complex models. Geoff Hinton, who is a famous professor of ML at the University of Toronto, has said:

When we're learning to see, nobody's telling us what the right answers are — we just look. Every so often, your mother says “that's a dog”, but that's very little information. You'd be lucky if you got a few bits of information — even one bit per second — that way. The brain's visual system has 10^{14} neural connections. And you only live for 10^9 seconds. So it's no use learning one bit per second. You need more like 10^5 bits per second. And there's only one place you can get that much information: from the input itself. — Geoffrey Hinton, 1996 (quoted in (Gorder 2006)).

Below we describe some canonical examples of unsupervised learning.

1.3.1 Discovering clusters

As a canonical example of unsupervised learning, consider the problem of **clustering** data into groups. For example, Figure 1.8(a) plots some 2d data, representing the height and weight of a group of 210 people. It seems that there might be various clusters, or subgroups, although it is not clear how many. Let K denote the number of clusters. Our first goal is to estimate the distribution over the number of clusters, $p(K|\mathcal{D})$; this tells us if there are subpopulations within the data. For simplicity, we often approximate the distribution $p(K|\mathcal{D})$ by its mode, $K^* = \arg \max_K p(K|\mathcal{D})$. In the supervised case, we were told that there are two classes (male and female), but in the unsupervised case, we are free to choose as many or few clusters as we like. Picking a model of the “right” complexity is called model selection, and will be discussed in detail below.

Our second goal is to estimate which cluster each point belongs to. Let $z_i \in \{1, \dots, K\}$ represent the cluster to which data point i is assigned. (z_i is an example of a **hidden** or

6. The advent of **crowd sourcing** web sites such as Mechanical Turk, (<https://www.mturk.com/mturk/welcome>), which outsource data processing tasks to humans all over the world, has reduced the cost of labeling data. Nevertheless, the amount of unlabeled data is still orders of magnitude larger than the amount of labeled data.

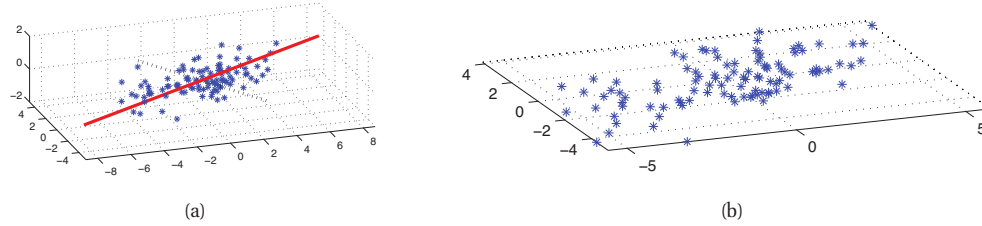


Figure 1.9 (a) A set of points that live on a 2d linear subspace embedded in 3d. The solid red line is the first principal component direction. The dotted black line is the second PC direction. (b) 2D representation of the data. Figure generated by `pcaDemo3d`.

latent variable, since it is never observed in the training set.) We can infer which cluster each data point belongs to by computing $z_i^* = \operatorname{argmax}_k p(z_i = k | \mathbf{x}_i, \mathcal{D})$. This is illustrated in Figure 1.8(b), where we use different colors to indicate the assignments, assuming $K = 2$.

In this book, we focus on **model based clustering**, which means we fit a probabilistic model to the data, rather than running some ad hoc algorithm. The advantages of the model-based approach are that one can compare different kinds of models in an objective way (in terms of the likelihood they assign to the data), we can combine them together into larger systems, etc.

Here are some real world applications of clustering.

- In astronomy, the **autoclass** system (Cheeseman et al. 1988) discovered a new type of star, based on clustering astrophysical measurements.
- In e-commerce, it is common to cluster users into groups, based on their purchasing or web-surfing behavior, and then to send customized targeted advertising to each group (see e.g., (Berkhin 2006)).
- In biology, it is common to cluster flow-cytometry data into groups, to discover different sub-populations of cells (see e.g., (Lo et al. 2009)).

1.3.2 Discovering latent factors

When dealing with high dimensional data, it is often useful to reduce the dimensionality by projecting the data to a lower dimensional subspace which captures the “essence” of the data. This is called **dimensionality reduction**. A simple example is shown in Figure 1.9, where we project some 3d data down to a 2d plane. The 2d approximation is quite good, since most points lie close to this subspace. Reducing to 1d would involve projecting points onto the red line in Figure 1.9(a); this would be a rather poor approximation. (We will make this notion precise in Chapter 12.)

The motivation behind this technique is that although the data may appear high dimensional, there may only be a small number of degrees of variability, corresponding to **latent factors**. For example, when modeling the appearance of face images, there may only be a few underlying latent factors which describe most of the variability, such as lighting, pose, identity, etc, as illustrated in Figure 1.10.



Figure 1.10 a) 25 randomly chosen 64×64 pixel images from the Olivetti face database. (b) The mean and the first three principal component basis vectors (eigenfaces). Figure generated by `pcaImageDemo`.

When used as input to other statistical models, such low dimensional representations often result in better predictive accuracy, because they focus on the “essence” of the object, filtering out inessential features. Also, low dimensional representations are useful for enabling fast nearest neighbor searches and two dimensional projections are very useful for **visualizing** high dimensional data.

The most common approach to dimensionality reduction is called **principal components analysis** or **PCA**. This can be thought of as an unsupervised version of (multi-output) linear regression, where we observe the high-dimensional response $\mathbf{y}$, but not the low-dimensional “cause” $\mathbf{z}$. Thus the model has the form $\mathbf{z} \rightarrow \mathbf{y}$; we have to “invert the arrow”, and infer the latent low-dimensional $\mathbf{z}$ from the observed high-dimensional $\mathbf{y}$. See Section 12.1 for details.

Dimensionality reduction, and PCA in particular, has been applied in many different areas. Some examples include the following:

- In biology, it is common to use PCA to interpret gene microarray data, to account for the fact that each measurement is usually the result of many genes which are correlated in their behavior by the fact that they belong to different biological pathways.
- In natural language processing, it is common to use a variant of PCA called latent semantic analysis for document retrieval (see Section 27.2.2).
- In signal processing (e.g., of acoustic or neural signals), it is common to use ICA (which is a variant of PCA) to separate signals into their different sources (see Section 12.6).
- In computer graphics, it is common to project motion capture data to a low dimensional space, and use it to create animations. See Section 15.5 for one way to tackle such problems.

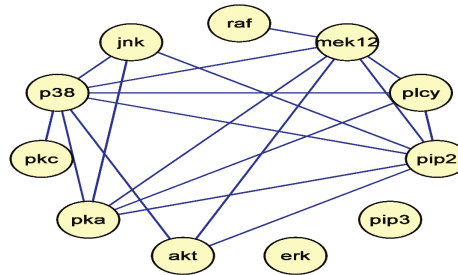


Figure 1.11 A sparse undirected Gaussian graphical model learned using graphical lasso (Section 26.7.2) applied to some flow cytometry data (from (Sachs et al. 2005)), which measures the phosphorylation status of 11 proteins. Figure generated by `ggmLassoDemo`.

1.3.3 Discovering graph structure

Sometimes we measure a set of correlated variables, and we would like to discover which ones are most correlated with which others. This can be represented by a graph G , in which nodes represent variables, and edges represent direct dependence between variables (we will make this precise in Chapter 10, when we discuss graphical models). We can then learn this graph structure from data, i.e., we compute $\hat{G} = \operatorname{argmax}_G p(G|\mathcal{D})$.

As with unsupervised learning in general, there are two main applications for learning sparse graphs: to discover new knowledge, and to get better joint probability density estimators. We now give some examples of each.

- Much of the motivation for learning sparse graphical models comes from the systems biology community. For example, suppose we measure the phosphorylation status of some proteins in a cell (Sachs et al. 2005). Figure 1.11 gives an example of a graph structure that was learned from this data (using methods discussed in Section 26.7.2). As another example, Smith et al. (2006) showed that one can recover the neural “wiring diagram” of a certain kind of bird from time-series EEG data. The recovered structure closely matched the known functional connectivity of this part of the bird brain.
- In some cases, we are not interested in interpreting the graph structure, we just want to use it to model correlations and to make predictions. One example of this is in financial portfolio management, where accurate models of the covariance between large numbers of different stocks is important. Carvalho and West (2007) show that by learning a sparse graph, and then using this as the basis of a trading strategy, it is possible to outperform (i.e., make more money than) methods that do not exploit sparse graphs. Another example is predicting traffic jams on the freeway. Horvitz et al. (2005) describe a deployed system called JamBayes for predicting traffic flow in the Seattle area; predictions are made using a graphical model whose structure was learned from data.

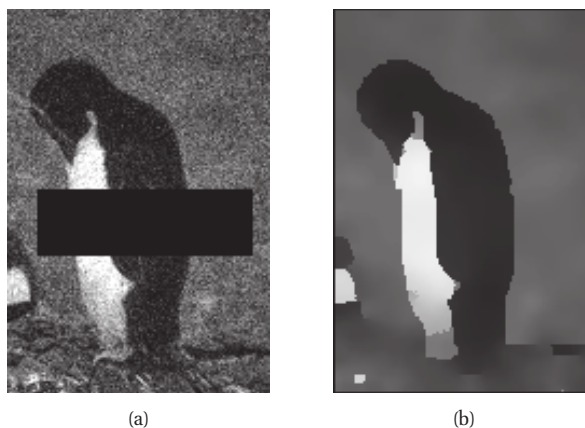


Figure 1.12 (a) A noisy image with an occluder. (b) An estimate of the underlying pixel intensities, based on a pairwise MRF model. Source: Figure 8 of (Felzenszwalb and Huttenlocher 2006). Used with kind permission of Pedro Felzenszwalb.

1.3.4 Matrix completion

Sometimes we have missing data, that is, variables whose values are unknown. For example, we might have conducted a survey, and some people might not have answered certain questions. Or we might have various sensors, some of which fail. The corresponding design matrix will then have “holes” in it; these missing entries are often represented by **NaN**, which stands for “not a number”. The goal of **imputation** is to infer plausible values for the missing entries. This is sometimes called **matrix completion**. Below we give some example applications.

1.3.4.1 Image inpainting

An interesting example of an imputation-like task is known as **image inpainting**. The goal is to “fill in” holes (e.g., due to scratches or occlusions) in an image with realistic texture. This is illustrated in Figure 1.12, where we denoise the image, as well as impute the pixels hidden behind the occlusion. This can be tackled by building a joint probability model of the pixels, given a set of clean images, and then inferring the unknown variables (pixels) given the known variables (pixels). This is somewhat like market basket analysis, except the data is real-valued and spatially structured, so the kinds of probability models we use are quite different. See Sections 19.6.2.7 and 13.8.4 for some possible choices.

1.3.4.2 Collaborative filtering

Another interesting example of an imputation-like task is known as **collaborative filtering**. A common example of this concerns predicting which movies people will want to watch based on how they, and other people, have rated movies which they have already seen. The key idea is that the prediction is not based on features of the movie or user (although it could be), but merely on a ratings matrix. More precisely, we have a matrix $\mathbf{X}$ where $X(m, u)$ is the rating

	1		?	3	5	?
	?	1				2
		4		4	5	?

Figure 1.13 Example of movie-rating data. Training data is in red, test data is denoted by ?, empty cells are unknown.

(say an integer between 1 and 5, where 1 is dislike and 5 is like) by user u of movie m . Note that most of the entries in $\mathbf{X}$ will be missing or unknown, since most users will not have rated most movies. Hence we only observe a tiny subset of the $\mathbf{X}$ matrix, and we want to predict a different subset. In particular, for any given user u , we might want to predict which of the unrated movies he/she is most likely to want to watch.

In order to encourage research in this area, the DVD rental company Netflix created a competition, launched in 2006, with a \$1M USD prize (see <http://netflixprize.com/>). In particular, they provided a large matrix of ratings, on a scale of 1 to 5, for $\sim 18k$ movies created by $\sim 500k$ users. The full matrix would have $\sim 9 \times 10^9$ entries, but only about 1% of the entries are observed, so the matrix is extremely **sparse**. A subset of these are used for training, and the rest for testing, as shown in Figure 1.13. The goal of the competition was to predict more accurately than Netflix's existing system. On 21 September 2009, the prize was awarded to a team of researchers known as "BellKor's Pragmatic Chaos". Section 27.6.2 discusses some of their methodology. Further details on the teams and their methods can be found at <http://www.netflixprize.com/community/viewtopic.php?id=1537>.

1.3.4.3 Market basket analysis

In commercial data mining, there is much interest in a task called **market basket analysis**. The data consists of a (typically very large but sparse) binary matrix, where each column represents an item or product, and each row represents a transaction. We set $x_{ij} = 1$ if item j was purchased on the i 'th transaction. Many items are purchased together (e.g., bread and butter), so there will be correlations amongst the bits. Given a new partially observed bit vector, representing a subset of items that the consumer has bought, the goal is to predict which other bits are likely to turn on, representing other items the consumer might be likely to buy. (Unlike collaborative filtering, we often assume there is no missing data in the training data, since we know the past shopping behavior of each customer.)

This task arises in other domains besides modeling purchasing patterns. For example, similar techniques can be used to model dependencies between files in complex software systems. In this case, the task is to predict, given a subset of files that have been changed, which other ones need to be updated to ensure consistency (see e.g., (Hu et al. 2010)).

It is common to solve such tasks using **frequent itemset mining**, which create association rules (see e.g., (Hastie et al. 2009, sec 14.2) for details). Alternatively, we can adopt a probabilistic approach, and fit a joint density model $p(x_1, \dots, x_D)$ to the bit vectors, see e.g., (Hu et al.

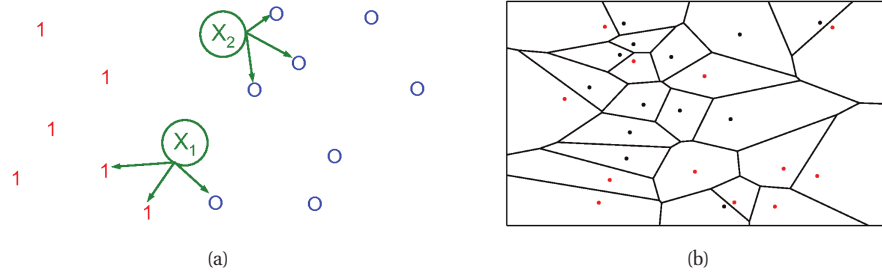


Figure 1.14 (a) Illustration of a K -nearest neighbors classifier in 2d for $K = 3$. The 3 nearest neighbors of test point x_1 have labels 1, 1 and 0, so we predict $p(y = 1|x_1, \mathcal{D}, K = 3) = 2/3$. The 3 nearest neighbors of test point x_2 have labels 0, 0, and 0, so we predict $p(y = 1|x_2, \mathcal{D}, K = 3) = 0/3$. (b) Illustration of the Voronoi tessellation induced by 1-NN. Based on Figure 4.13 of (Duda et al. 2001). Figure generated by `knnVoronoi`.

2010). Such models often have better predictive accuracy than association rules, although they may be less interpretable. This is typical of the difference between data mining and machine learning: in data mining, there is more emphasis on interpretable models, whereas in machine learning, there is more emphasis on accurate models.

1.4 Some basic concepts in machine learning

In this Section, we provide an introduction to some key ideas in machine learning. We will expand on these concepts later in the book, but we introduce them briefly here, to give a flavor of things to come.

1.4.1 Parametric vs non-parametric models

In this book, we will be focussing on probabilistic models of the form $p(y|x)$ or $p(x)$, depending on whether we are interested in supervised or unsupervised learning respectively. There are many ways to define such models, but the most important distinction is this: does the model have a fixed number of parameters, or does the number of parameters grow with the amount of training data? The former is called a **parametric model**, and the latter is called a **non-parametric model**. Parametric models have the advantage of often being faster to use, but the disadvantage of making stronger assumptions about the nature of the data distributions. Non-parametric models are more flexible, but often computationally intractable for large datasets. We will give examples of both kinds of models in the sections below. We focus on supervised learning for simplicity, although much of our discussion also applies to unsupervised learning.

1.4.2 A simple non-parametric classifier: K -nearest neighbors

A simple example of a non-parametric classifier is the K **nearest neighbor (KNN)** classifier. This simply “looks at” the K points in the training set that are nearest to the test input x ,

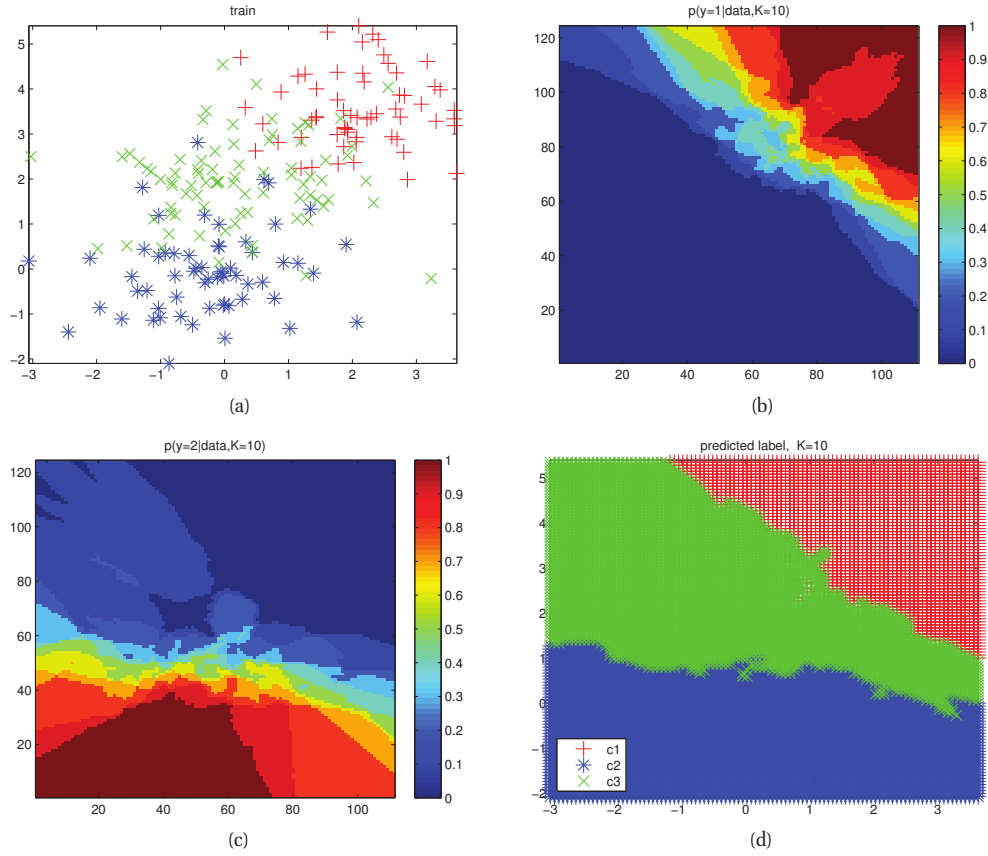


Figure 1.15 (a) Some synthetic 3-class training data in 2d. (b) Probability of class 1 for KNN with $K = 10$. (c) Probability of class 2. (d) MAP estimate of class label. Figure generated by `knnClassifyDemo`.

counts how many members of each class are in this set, and returns that empirical fraction as the estimate, as illustrated in Figure 1.14. More formally,

$$p(y = c | \mathbf{x}, \mathcal{D}, K) = \frac{1}{K} \sum_{i \in N_K(\mathbf{x}, \mathcal{D})} \mathbb{I}(y_i = c) \quad (1.2)$$

where $N_K(\mathbf{x}, \mathcal{D})$ are the (indices of the) K nearest points to $\mathbf{x}$ in $\mathcal{D}$ and $\mathbb{I}(e)$ is the **indicator function** defined as follows:

$$\mathbb{I}(e) = \begin{cases} 1 & \text{if } e \text{ is true} \\ 0 & \text{if } e \text{ is false} \end{cases} \quad (1.3)$$

This method is an example of **memory-based learning** or **instance-based learning**. It can be derived from a probabilistic framework as explained in Section 14.7.3. The most common

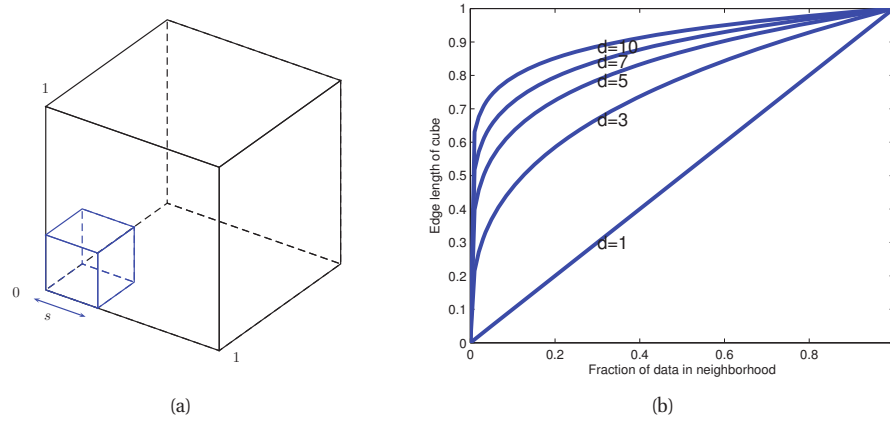


Figure 1.16 Illustration of the curse of dimensionality. (a) We embed a small cube of side s inside a larger unit cube. (b) We plot the edge length of a cube needed to cover a given volume of the unit cube as a function of the number of dimensions. Based on Figure 2.6 from (Hastie et al. 2009). Figure generated by `curseDimensionality`.

distance metric to use is Euclidean distance (which limits the applicability of the technique to data which is real-valued), although other metrics can be used.

Figure 1.15 gives an example of the method in action, where the input is two dimensional, we have three classes, and $K = 10$. (We discuss the effect of K below.) Panel (a) plots the training data. Panel (b) plots $p(y = 1|\mathbf{x}, \mathcal{D})$ where $\mathbf{x}$ is evaluated on a grid of points. Panel (c) plots $p(y = 2|\mathbf{x}, \mathcal{D})$. We do not need to plot $p(y = 3|\mathbf{x}, \mathcal{D})$, since probabilities sum to one. Panel (d) plots the MAP estimate $\hat{y}(\mathbf{x}) = \operatorname{argmax}_c (y = c|\mathbf{x}, \mathcal{D})$.

A KNN classifier with $K = 1$ induces a **Voronoi tessellation** of the points (see Figure 1.14(b)). This is a partition of space which associates a region $V(\mathbf{x}_i)$ with each point $\mathbf{x}_i$ in such a way that all points in $V(\mathbf{x}_i)$ are closer to $\mathbf{x}_i$ than to any other point. Within each cell, the predicted label is the label of the corresponding training point.

1.4.3 The curse of dimensionality

The KNN classifier is simple and can work quite well, provided it is given a good distance metric and has enough labeled training data. In fact, it can be shown that the KNN classifier can come within a factor of 2 of the best possible performance if $N \rightarrow \infty$ (Cover and Hart 1967).

However, the main problem with KNN classifiers is that they do not work well with high dimensional inputs. The poor performance in high dimensional settings is due to the **curse of dimensionality**.

To explain the curse, we give some examples from (Hastie et al. 2009, p22). Consider applying a KNN classifier to data where the inputs are uniformly distributed in the D -dimensional unit cube. Suppose we estimate the density of class labels around a test point $\mathbf{x}$ by “growing” a hyper-cube around $\mathbf{x}$ until it contains a desired fraction f of the data points. The expected edge length of this cube will be $e_D(f) = f^{1/D}$. If $D = 10$, and we want to base our estimate on 10%

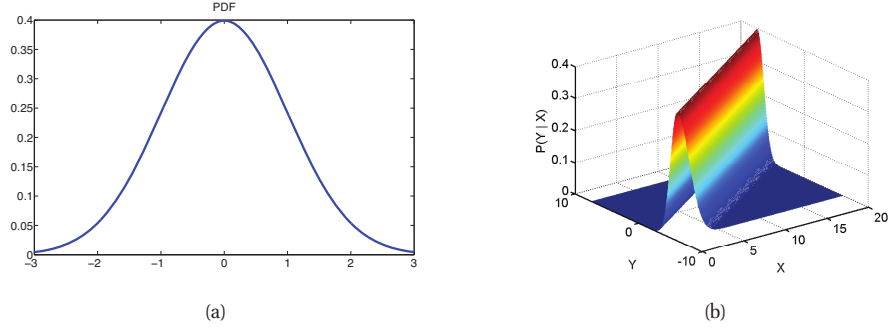


Figure 1.17 (a) A Gaussian pdf with mean 0 and variance 1. Figure generated by `gaussPlotDemo`. (b) Visualization of the conditional density model $p(y|x, \theta) = \mathcal{N}(y|w_0 + w_1x, \sigma^2)$. The density falls off exponentially fast as we move away from the regression line. Figure generated by `linregWedgeDemo2`.

of the data, we have $e_{10}(0.1) = 0.8$, so we need to extend the cube 80% along each dimension around $\mathbf{x}$. Even if we only use 1% of the data, we find $e_{10}(0.01) = 0.63$: see Figure 1.16. Since the entire range of the data is only 1 along each dimension, we see that the method is no longer very local, despite the name “nearest neighbor”. The trouble with looking at neighbors that are so far away is that they may not be good predictors about the behavior of the input-output function at a given point.

1.4.4 Parametric models for classification and regression

The main way to combat the curse of dimensionality is to make some assumptions about the nature of the data distribution (either $p(y|\mathbf{x})$ for a supervised problem or $p(\mathbf{x})$ for an unsupervised problem). These assumptions, known as **inductive bias**, are often embodied in the form of a **parametric model**, which is a statistical model with a fixed number of parameters. Below we briefly describe two widely used examples; we will revisit these and other models in much greater depth later in the book.

1.4.5 Linear regression

One of the most widely used models for regression is known as **linear regression**. This asserts that the response is a linear function of the inputs. This can be written as follows:

$$y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + \epsilon = \sum_{j=1}^D w_j x_j + \epsilon \quad (1.4)$$

where $\mathbf{w}^T \mathbf{x}$ represents the inner or **scalar product** between the input vector $\mathbf{x}$ and the model's **weight vector** $\mathbf{w}^T$, and ϵ is the **residual error** between our linear predictions and the true response.

7. In statistics, it is more common to denote the regression weights by β .

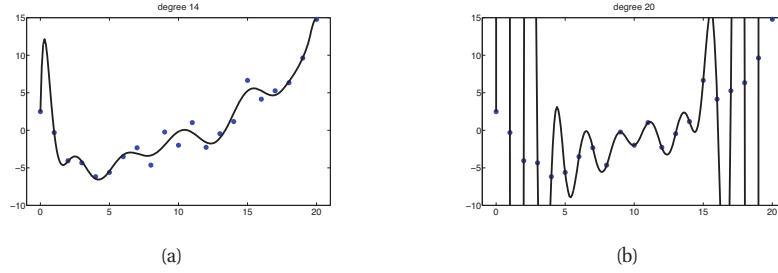


Figure 1.18 Polynomial of degrees 14 and 20 fit by least squares to 21 data points. Figure generated by `linregPolyVsDegree`.

We often assume that ϵ has a **Gaussian**⁸ or **normal** distribution. We denote this by $\epsilon \sim \mathcal{N}(\mu, \sigma^2)$, where μ is the mean and σ^2 is the variance (see Chapter 2 for details). When we plot this distribution, we get the well-known **bell curve** shown in Figure 1.17(a).

To make the connection between linear regression and Gaussians more explicit, we can rewrite the model in the following form:

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y|\mu(\mathbf{x}), \sigma^2(\mathbf{x})) \quad (1.5)$$

This makes it clear that the model is a conditional probability density. In the simplest case, we assume μ is a linear function of $\mathbf{x}$, so $\mu = \mathbf{w}^T \mathbf{x}$, and that the noise is fixed, $\sigma^2(x) = \sigma^2$. In this case, $\boldsymbol{\theta} = (\mathbf{w}, \sigma^2)$ are the parameters of the model.

For example, suppose the input is 1 dimensional. We can represent the expected response as follows:

$$\mu(\mathbf{x}) = w_0 + w_1 x = \mathbf{w}^T \mathbf{x} \quad (1.6)$$

where w_0 is the intercept or **bias** term, w_1 is the slope, and where we have defined the vector $\mathbf{x} = (1, x)$. (Prepending a constant 1 term to an input vector is a common notational trick which allows us to combine the intercept term with the other terms in the model.) If w_1 is positive, it means we expect the output to increase as the input increases. This is illustrated in 1d in Figure 1.17(b); a more conventional plot, of the mean response vs x , is shown in Figure 1.7(a).

Linear regression can be made to model non-linear relationships by replacing $\mathbf{x}$ with some non-linear function of the inputs, $\phi(\mathbf{x})$. That is, we use

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y|\mathbf{w}^T \phi(\mathbf{x}), \sigma^2) \quad (1.7)$$

This is known as **basis function expansion**. For example, Figure 1.18 illustrates the case where $\phi(\mathbf{x}) = [1, x, x^2, \dots, x^d]$, for $d = 14$ and $d = 20$; this is known as **polynomial regression**. We will consider other kinds of basis functions later in the book. In fact, many popular machine learning methods — such as support vector machines, neural networks, classification and regression trees, etc. — can be seen as just different ways of estimating basis functions from data, as we discuss in Chapters 14 and 16.

8. Carl Friedrich Gauss (1777–1855) was a German mathematician and physicist.

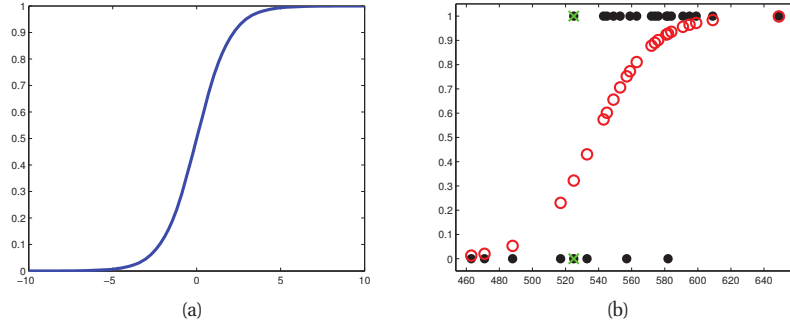


Figure 1.19 (a) The sigmoid or logistic function. We have $\text{sigm}(-\infty) = 0$, $\text{sigm}(0) = 0.5$, and $\text{sigm}(\infty) = 1$. Figure generated by `sigmoidPlot`. (b) Logistic regression for SAT scores. Solid black dots are the data. The open red circles are the predicted probabilities. The green crosses denote two students with the same SAT score of 525 (and hence same input representation x) but with different training labels (one student passed, $y = 1$, the other failed, $y = 0$). Hence this data is not perfectly separable using just the SAT feature. Figure generated by `logregSATdemo`.

1.4.6 Logistic regression

We can generalize linear regression to the (binary) classification setting by making two changes. First we replace the Gaussian distribution for y with a **Bernoulli** distribution⁹, which is more appropriate for the case when the response is binary, $y \in \{0, 1\}$. That is, we use

$$p(y|\mathbf{x}, \mathbf{w}) = \text{Ber}(y|\mu(\mathbf{x})) \quad (1.8)$$

where $\mu(\mathbf{x}) = \mathbb{E}[y|\mathbf{x}] = p(y = 1|\mathbf{x})$. Second, we compute a linear combination of the inputs, as before, but then we pass this through a function that ensures $0 \leq \mu(\mathbf{x}) \leq 1$ by defining

$$\mu(\mathbf{x}) = \text{sigm}(\mathbf{w}^T \mathbf{x}) \quad (1.9)$$

where $\text{sigm}(\eta)$ refers to the **sigmoid** function, also known as the **logistic** or **logit** function. This is defined as

$$\text{sigm}(\eta) \triangleq \frac{1}{1 + \exp(-\eta)} = \frac{e^\eta}{e^\eta + 1} \quad (1.10)$$

The term “sigmoid” means S-shaped: see Figure 1.19(a) for a plot. It is also known as a **squashing function**, since it maps the whole real line to $[0, 1]$, which is necessary for the output to be interpreted as a probability.

Putting these two steps together we get

$$p(y|\mathbf{x}, \mathbf{w}) = \text{Ber}(y|\text{sigm}(\mathbf{w}^T \mathbf{x})) \quad (1.11)$$

This is called **logistic regression** due to its similarity to linear regression (although it is a form of classification, not regression!).

9. Daniel Bernoulli (1700–1782) was a Dutch-Swiss mathematician and physicist.

A simple example of logistic regression is shown in Figure 1.19(b), where we plot

$$p(y_i = 1|x_i, \mathbf{w}) = \text{sigm}(w_0 + w_1 x_i) \quad (1.12)$$

where x_i is the SAT¹⁰ score of student i and y_i is whether they passed or failed a class. The solid black dots show the training data, and the red circles plot $p(y = 1|\mathbf{x}_i, \hat{\mathbf{w}})$, where $\hat{\mathbf{w}}$ are the parameters estimated from the training data (we discuss how to compute these estimates in Section 8.3.4).

If we threshold the output probability at 0.5, we can induce a **decision rule** of the form

$$\hat{y}(x) = 1 \iff p(y = 1|\mathbf{x}) > 0.5 \quad (1.13)$$

By looking at Figure 1.19(b), we see that $\text{sigm}(w_0 + w_1 x) = 0.5$ for $x \approx 545 = x^*$. We can imagine drawing a vertical line at $x = x^*$; this is known as a decision boundary. Everything to the left of this line is classified as a 0, and everything to the right of the line is classified as a 1.

We notice that this decision rule has a non-zero error rate even on the training set. This is because the data is not **linearly separable**, i.e., there is no straight line we can draw to separate the 0s from the 1s. We can create models with non-linear decision boundaries using basis function expansion, just as we did with non-linear regression. We will see many examples of this later in the book.

1.4.7 Overfitting

When we fit highly flexible models, we need to be careful that we do not **overfit** the data, that is, we should avoid trying to model every minor variation in the input, since this is more likely to be noise than true signal. This is illustrated in Figure 1.18(b), where we see that using a high degree polynomial results in a curve that is very “wiggly”. It is unlikely that the true function has such extreme oscillations. Thus using such a model might result in accurate predictions of future outputs.

As another example, consider the KNN classifier. The value of K can have a large effect on the behavior of this model. When $K = 1$, the method makes no errors on the training set (since we just return the labels of the original training points), but the resulting prediction surface is very “wiggly” (see Figure 1.20(a)). Therefore the method may not work well at predicting future data. In Figure 1.20(b), we see that using $K = 5$ results in a smoother prediction surface, because we are averaging over a larger neighborhood. As K increases, the predictions becomes smoother until, in the limit of $K = N$, we end up predicting the majority label of the whole data set. Below we discuss how to pick the “right” value of K .

1.4.8 Model selection

When we have a variety of models of different complexity (e.g., linear or logistic regression models with different degree polynomials, or KNN classifiers with different values of K), how should we pick the right one? A natural approach is to compute the **misclassification rate** on

10. SAT stands for “Scholastic Aptitude Test”. This is a standardized test for college admissions used in the United States (the data in this example is from (Johnson and Albert 1999, p87)).

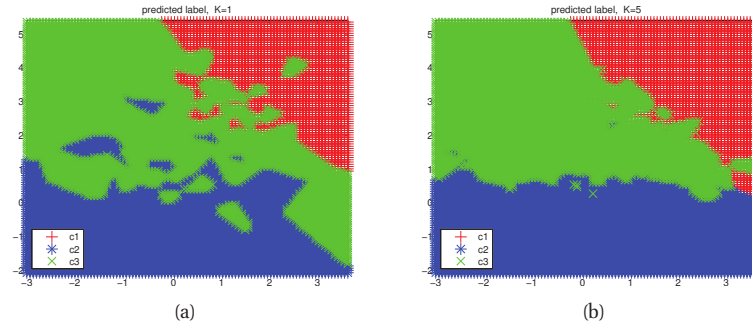


Figure 1.20 Prediction surface for KNN on the data in Figure 1.15(a). (a) $K=1$. (b) $K=5$. Figure generated by `knnClassifyDemo`.

the training set for each method. This is defined as follows:

$$\text{err}(f, \mathcal{D}) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(f(\mathbf{x}_i) \neq y_i) \quad (1.14)$$

where $f(\mathbf{x})$ is our classifier. In Figure 1.21(a), we plot this error rate vs K for a KNN classifier (dotted blue line). We see that increasing K *increases* our error rate on the training set, because we are over-smoothing. As we said above, we can get minimal error on the training set by using $K = 1$, since this model is just memorizing the data.

However, what we care about is **generalization error**, which is the expected value of the misclassification rate when averaged over future data (see Section 6.3 for details). This can be approximated by computing the misclassification rate on a large independent *test set*, not used during model training. We plot the test error vs K in Figure 1.21(a) in solid red (upper curve). Now we see a **U-shaped curve**: for complex models (small K), the method overfits, and for simple models (big K), the method **underfits**. Therefore, an obvious way to pick K is to pick the value with the minimum error on the test set (in this example, any value between 10 and 100 should be fine).

Unfortunately, when training the model, we don't have access to the test set (by assumption), so we cannot use the test set to pick the model of the right complexity.¹¹ However, we can create a test set by partitioning the training set into two: the part used for training the model, and a second part, called the **validation set**, used for selecting the model complexity. We then fit all the models on the training set, and evaluate their performance on the validation set, and pick the best. Once we have picked the best, we can refit it to all the available data. If we have a separate test set, we can evaluate performance on this, in order to estimate the accuracy of our method. (We discuss this in more detail in Section 6.5.3.)

Often we use about 80% of the data for the training set, and 20% for the validation set. But if the number of training cases is small, this technique runs into problems, because the model

¹¹ In academic settings, we usually do have access to the test set, but we should not use it for model fitting or model selection, otherwise we will get an unrealistically optimistic estimate of performance of our method. This is one of the “golden rules” of machine learning research.

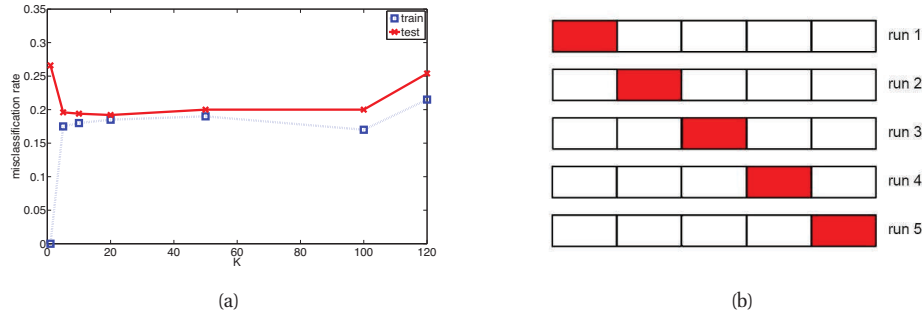


Figure 1.21 (a) Misclassification rate vs K in a K-nearest neighbor classifier. On the left, where K is small, the model is complex and hence we overfit. On the right, where K is large, the model is simple and we underfit. Dotted blue line: training set (size 200). Solid red line: test set (size 500). (b) Schematic of 5-fold cross validation. Figure generated by `knnClassifyDemo`.

won't have enough data to train on, and we won't have enough data to make a reliable estimate of the future performance.

A simple but popular solution to this is to use **cross validation (CV)**. The idea is simple: we split the training data into K **folds**; then, for each fold $k \in \{1, \dots, K\}$, we train on all the folds but the k 'th, and test on the k 'th, in a round-robin fashion, as sketched in Figure 1.21(b). We then compute the error averaged over all the folds, and use this as a proxy for the test error. (Note that each point gets predicted only once, although it will be used for training $K - 1$ times.) It is common to use $K = 5$; this is called 5-fold CV. If we set $K = N$, then we get a method called **leave-one out cross validation**, or **LOOCV**, since in fold i , we train on all the data cases except for i , and then test on i . Exercise 1.3 asks you to compute the 5-fold CV estimate of the test error vs K , and to compare it to the empirical test error in Figure 1.21(a).

Choosing K for a KNN classifier is a special case of a more general problem known as **model selection**, where we have to choose between models with different degrees of flexibility. Cross-validation is widely used for solving such problems, although we will discuss other approaches later in the book.

1.4.9 No free lunch theorem

All models are wrong, but some models are useful. — George Box (Box and Draper 1987, p424).¹²

Much of machine learning is concerned with devising different models, and different algorithms to fit them. We can use methods such as cross validation to empirically choose the best method for our particular problem. However, there is no universally best model — this is sometimes called the **no free lunch theorem** (Wolpert 1996). The reason for this is that a set of assumptions that works well in one domain may work poorly in another.

¹² George Box is a retired statistics professor at the University of Wisconsin.

As a consequence of the no free lunch theorem, we need to develop many different types of models, to cover the wide variety of data that occurs in the real world. And for each model, there may be many different algorithms we can use to train the model, which make different speed-accuracy-complexity tradeoffs. It is this combination of data, models and algorithms that we will be studying in the subsequent chapters.

Exercises

Exercise 1.1 KNN classifier on shuffled MNIST data

Run `mnist1NNDemo` and verify that the misclassification rate (on the first 1000 test cases) of MNIST of a 1-NN classifier is 3.8%. (If you run it all on all 10,000 test cases, the error rate is 3.09%.) Modify the code so that you first randomly permute the features (columns of the training and test design matrices), as in `shuffledDigitsDemo`, and then apply the classifier. Verify that the error rate is not changed.

Exercise 1.2 Approximate KNN classifiers

Use the Matlab/C++ code at <http://people.cs.ubc.ca/~mariusm/index.php/FLANN/FLANN> to perform approximate nearest neighbor search, and combine it with `mnist1NNDemo` to classify the MNIST data set. How much speedup do you get, and what is the drop (if any) in accuracy?

Exercise 1.3 CV for KNN

Use `knnClassifyDemo` to plot the CV estimate of the misclassification rate on the test set. Compare this to Figure 1.21(a). Discuss the similarities and differences to the test error rate.

2 *Probability*

2.1 Introduction

Probability theory is nothing but common sense reduced to calculation. — Pierre Laplace, 1812

In the previous chapter, we saw how probability can play a useful role in machine learning. In this chapter, we discuss probability theory in more detail. We do not have to space to go into great detail — for that, you are better off consulting some of the excellent textbooks available on this topic, such as (Jaynes 2003; Bertsekas and Tsitsiklis 2008; Wasserman 2004). But we will briefly review many of the key ideas you will need in later chapters.

Before we start with the more technical material, let us pause and ask: what is probability? We are all familiar with the phrase “the probability that a coin will land heads is 0.5”. But what does this mean? There are actually at least two different interpretations of probability. One is called the **frequentist** interpretation. In this view, probabilities represent long run frequencies of events. For example, the above statement means that, if we flip the coin many times, we expect it to land heads about half the time.¹

The other interpretation is called the **Bayesian** interpretation of probability. In this view, probability is used to quantify our **uncertainty** about something; hence it is fundamentally related to information rather than repeated trials (Jaynes 2003). In the Bayesian view, the above statement means we believe the coin is equally likely to land heads or tails on the next toss.

One big advantage of the Bayesian interpretation is that it can be used to model our uncertainty about events that do not have long term frequencies. For example, we might want to compute the probability that the polar ice cap will melt by 2020 CE. This event will happen zero or one times, but cannot happen repeatedly. Nevertheless, we ought to be able to quantify our uncertainty about this event; based on how probable we think this event is, we will (hopefully!) take appropriate actions (see Section 5.7 for a discussion of optimal decision making under uncertainty). To give some more machine learning oriented examples, we might have received a specific email message, and want to compute the probability it is spam. Or we might have observed a “blip” on our radar screen, and want to compute the probability distribution over the location of the corresponding target (be it a bird, plane, or missile). In all these cases, the idea of repeated trials does not make sense, but the Bayesian interpretation is valid and indeed

1. Actually, the Stanford statistician (and former professional magician) Persi Diaconis has shown that a coin is about 51% likely to land facing the same way up as it started, due to the physics of the problem (Diaconis et al. 2007).

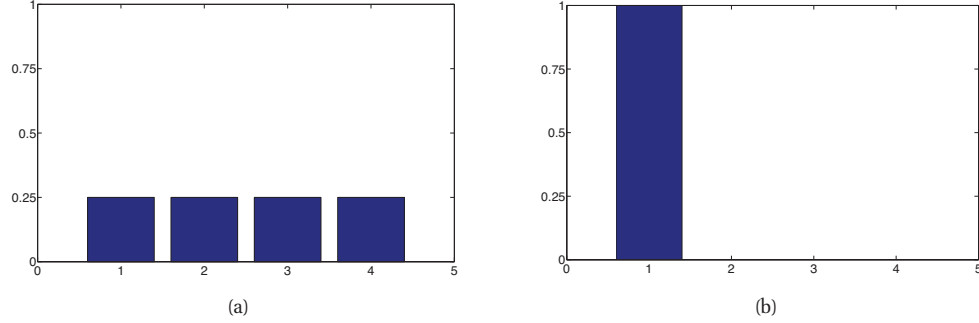


Figure 2.1 (A) a uniform distribution on $\{1, 2, 3, 4\}$, with $p(x = k) = 1/4$. (b) a degenerate distribution $p(x) = 1$ if $x = 1$ and $p(x) = 0$ if $x \in \{2, 3, 4\}$. Figure generated by `discreteProbDistFig`.

quite natural. We shall therefore adopt the Bayesian interpretation in this book. Fortunately, the basic rules of probability theory are the same, no matter which interpretation is adopted.

2.2 A brief review of probability theory

This section is a very brief review of the basics of probability theory, and is merely meant as a refresher for readers who may be “rusty”. Readers who are already familiar with these basics may safely skip this section.

2.2.1 Discrete random variables

The expression $p(A)$ denotes the probability that the event A is true. For example, A might be the logical expression “it will rain tomorrow”. We require that $0 \leq p(A) \leq 1$, where $p(A) = 0$ means the event definitely will not happen, and $p(A) = 1$ means the event definitely will happen. We write $p(\bar{A})$ to denote the probability of the event not A ; this is defined to $p(\bar{A}) = 1 - p(A)$. We will often write $A = 1$ to mean the event A is true, and $A = 0$ to mean the event A is false.

We can extend the notion of binary events by defining a **discrete random variable** X , which can take on any value from a finite or countably infinite set $\mathcal{X}$. We denote the probability of the event that $X = x$ by $p(X = x)$, or just $p(x)$ for short. Here $p(\cdot)$ is called a **probability mass function** or **pmf**. This satisfies the properties $0 \leq p(x) \leq 1$ and $\sum_{x \in \mathcal{X}} p(x) = 1$. Figure 2.1 shows two pmf’s defined on the finite **state space** $\mathcal{X} = \{1, 2, 3, 4, 5\}$. On the left we have a uniform distribution, $p(x) = 1/5$, and on the right, we have a degenerate distribution, $p(x) = \mathbb{I}(x = 1)$, where $\mathbb{I}(\cdot)$ is the binary **indicator function**. This distribution represents the fact that X is always equal to the value 1, in other words, it is a constant.

2.2.2 Fundamental rules

In this section, we review the basic rules of probability.

2.2.2.1 Probability of a union of two events

Given two events, A and B , we define the probability of A or B as follows:

$$p(A \vee B) = p(A) + p(B) - p(A \wedge B) \quad (2.1)$$

$$= p(A) + p(B) \text{ if } A \text{ and } B \text{ are mutually exclusive} \quad (2.2)$$

2.2.2.2 Joint probabilities

We define the probability of the joint event A and B as follows:

$$p(A, B) = p(A \wedge B) = p(A|B)p(B) \quad (2.3)$$

This is sometimes called the **product rule**. Given a **joint distribution** on two events $p(A, B)$, we define the **marginal distribution** as follows:

$$p(A) = \sum_b p(A, B) = \sum_b p(A|B = b)p(B = b) \quad (2.4)$$

where we are summing over all possible states of B . We can define $p(B)$ similarly. This is sometimes called the **sum rule** or the **rule of total probability**.

The product rule can be applied multiple times to yield the **chain rule** of probability:

$$p(X_{1:D}) = p(X_1)p(X_2|X_1)p(X_3|X_2, X_1)p(X_4|X_1, X_2, X_3) \dots p(X_D|X_{1:D-1}) \quad (2.5)$$

where we introduce the Matlab-like notation $1 : D$ to denote the set $\{1, 2, \dots, D\}$.

2.2.2.3 Conditional probability

We define the **conditional probability** of event A , given that event B is true, as follows:

$$p(A|B) = \frac{p(A, B)}{p(B)} \text{ if } p(B) > 0 \quad (2.6)$$

2.2.3 Bayes rule

Combining the definition of conditional probability with the product and sum rules yields **Bayes rule**, also called **Bayes Theorem**²:

$$p(X = x|Y = y) = \frac{p(X = x, Y = y)}{p(Y = y)} = \frac{p(X = x)p(Y = y|X = x)}{\sum_{x'} p(X = x')p(Y = y|X = x')} \quad (2.7)$$

2.2.3.1 Example: medical diagnosis

As an example of how to use this rule, consider the following medical diagnosis problem. Suppose you are a woman in your 40s, and you decide to have a medical test for breast cancer called a **mammogram**. If the test is positive, what is the probability you have cancer? That obviously depends on how reliable the test is. Suppose you are told the test has a **sensitivity**

2. Thomas Bayes (1702–1761) was an English mathematician and Presbyterian minister.

of 80%, which means, if you have cancer, the test will be positive with probability 0.8. In other words,

$$p(x = 1|y = 1) = 0.8 \quad (2.8)$$

where $x = 1$ is the event the mammogram is positive, and $y = 1$ is the event you have breast cancer. Many people conclude they are therefore 80% likely to have cancer. But this is false! It ignores the prior probability of having breast cancer, which fortunately is quite low:

$$p(y = 1) = 0.004 \quad (2.9)$$

Ignoring this prior is called the **base rate fallacy**. We also need to take into account the fact that the test may be a **false positive** or **false alarm**. Unfortunately, such false positives are quite likely (with current screening technology):

$$p(x = 1|y = 0) = 0.1 \quad (2.10)$$

Combining these three terms using Bayes rule, we can compute the correct answer as follows:

$$p(y = 1|x = 1) = \frac{p(x = 1|y = 1)p(y = 1)}{p(x = 1|y = 1)p(y = 1) + p(x = 1|y = 0)p(y = 0)} \quad (2.11)$$

$$= \frac{0.8 \times 0.004}{0.8 \times 0.004 + 0.1 \times 0.996} = 0.031 \quad (2.12)$$

where $p(y = 0) = 1 - p(y = 1) = 0.996$. In other words, if you test positive, you only have about a 3% chance of actually having breast cancer!³

2.2.3.2 Example: Generative classifiers

We can generalize the medical diagnosis example to classify feature vectors $\mathbf{x}$ of arbitrary type as follows:

$$p(y = c|\mathbf{x}, \boldsymbol{\theta}) = \frac{p(y = c|\boldsymbol{\theta})p(\mathbf{x}|y = c, \boldsymbol{\theta})}{\sum_{c'} p(y = c'|\boldsymbol{\theta})p(\mathbf{x}|y = c', \boldsymbol{\theta})} \quad (2.13)$$

This is called a **generative classifier**, since it specifies how to generate the data using the **class-conditional density** $p(\mathbf{x}|y = c)$ and the class prior $p(y = c)$. We discuss such models in detail in Chapters 3 and 4. An alternative approach is to directly fit the class posterior, $p(y = c|\mathbf{x})$; this is known as a discriminative classifier. We discuss the pros and cons of the two approaches in Section 8.6.

2.2.4 Independence and conditional independence

We say X and Y are **unconditionally independent** or **marginally independent**, denoted $X \perp Y$, if we can represent the joint as the product of the two marginals (see Figure 2.2), i.e.,

$$X \perp Y \iff p(X, Y) = p(X)p(Y) \quad (2.14)$$

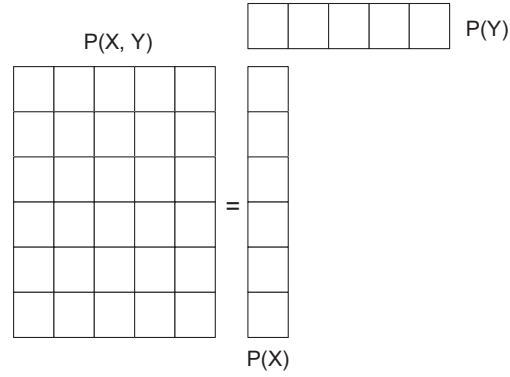


Figure 2.2 Computing $p(x, y) = p(x)p(y)$, where $X \perp Y$. Here X and Y are discrete random variables; X has 6 possible states (values) and Y has 5 possible states. A general joint distribution on two such variables would require $(6 \times 5) - 1 = 29$ parameters to define it (we subtract 1 because of the sum-to-one constraint). By assuming (unconditional) independence, we only need $(6 - 1) + (5 - 1) = 9$ parameters to define $p(x, y)$.

In general, we say a set of variables is mutually independent if the joint can be written as a product of marginals.

Unfortunately, unconditional independence is rare, because most variables can influence most other variables. However, usually this influence is mediated via other variables rather than being direct. We therefore say X and Y are **conditionally independent (CI)** given Z iff the conditional joint can be written as a product of conditional marginals:

$$X \perp Y | Z \iff p(X, Y | Z) = p(X | Z)p(Y | Z) \quad (2.15)$$

When we discuss graphical models in Chapter 10, we will see that we can write this assumption as a graph $X - Z - Y$, which captures the intuition that all the dependencies between X and Y are mediated via Z . For example, the probability it will rain tomorrow (event X) is independent of whether the ground is wet today (event Y), given knowledge of whether it is raining today (event Z). Intuitively, this is because Z “causes” both X and Y , so if we know Z , we do not need to know about Y in order to predict X or vice versa. We shall expand on this concept in Chapter 10.

Another characterization of CI is this:

Theorem 2.2.1. $X \perp Y | Z$ iff there exist function g and h such that

$$p(x, y | z) = g(x, z)h(y, z) \quad (2.16)$$

for all x, y, z such that $p(z) > 0$.

3. These numbers are from (McGrayne 2011, p257). Based on this analysis, the US government decided not to recommend annual mammogram screening to women in their 40s: the number of false alarms would cause needless worry and stress amongst women, and result in unnecessary, expensive, and potentially harmful followup tests. See Section 5.7 for the optimal way to trade off risk reverse reward in the face of uncertainty.

See Exercise 2.8 for the proof.

CI assumptions allow us to build large probabilistic models from small pieces. We will see many examples of this throughout the book. In particular, in Section 3.5, we discuss naive Bayes classifiers, in Section 17.2, we discuss Markov models, and in Chapter 10 we discuss graphical models; all of these models heavily exploit CI properties.

2.2.5 Continuous random variables

So far, we have only considered reasoning about uncertain discrete quantities. We will now show (following (Jaynes 2003, p107)) how to extend probability to reason about uncertain continuous quantities.

Suppose X is some uncertain continuous quantity. The probability that X lies in any interval $a \leq X \leq b$ can be computed as follows. Define the events $A = (X \leq a)$, $B = (X \leq b)$ and $W = (a < X \leq b)$. We have that $B = A \vee W$, and since A and W are mutually exclusive, the sum rules gives

$$p(B) = p(A) + p(W) \quad (2.17)$$

and hence

$$p(W) = p(B) - p(A) \quad (2.18)$$

Define the function $F(q) \triangleq p(X \leq q)$. This is called the **cumulative distribution function** or **cdf** of X . This is obviously a monotonically increasing function. See Figure 2.3(a) for an example. Using this notation we have

$$p(a < X \leq b) = F(b) - F(a) \quad (2.19)$$

Now define $f(x) = \frac{d}{dx}F(x)$ (we assume this derivative exists); this is called the **probability density function** or **pdf**. See Figure 2.3(b) for an example. Given a pdf, we can compute the probability of a continuous variable being in a finite interval as follows:

$$P(a < X \leq b) = \int_a^b f(x)dx \quad (2.20)$$

As the size of the interval gets smaller, we can write

$$P(x \leq X \leq x + dx) \approx p(x)dx \quad (2.21)$$

We require $p(x) \geq 0$, but it is possible for $p(x) > 1$ for any given x , so long as the density integrates to 1. As an example, consider the **uniform distribution** $\text{Unif}(a, b)$:

$$\text{Unif}(x|a, b) = \frac{1}{b-a} \mathbb{I}(a \leq x \leq b) \quad (2.22)$$

If we set $a = 0$ and $b = \frac{1}{2}$, we have $p(x) = 2$ for any $x \in [0, \frac{1}{2}]$.

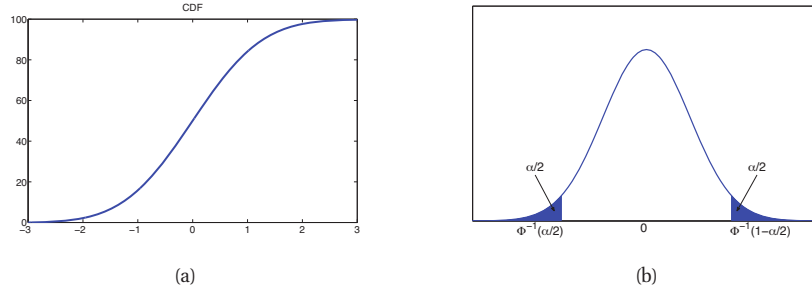


Figure 2.3 (a) Plot of the cdf for the standard normal, $\mathcal{N}(0, 1)$. (b) Corresponding pdf. The shaded regions each contain $\alpha/2$ of the probability mass. Therefore the nonshaded region contains $1 - \alpha$ of the probability mass. If the distribution is Gaussian $\mathcal{N}(0, 1)$, then the leftmost cutoff point is $\Phi^{-1}(\alpha/2)$, where Φ is the cdf of the Gaussian. By symmetry, the rightmost cutoff point is $\Phi^{-1}(1 - \alpha/2) = -\Phi^{-1}(\alpha/2)$. If $\alpha = 0.05$, the central interval is 95%, and the left cutoff is -1.96 and the right is 1.96. Figure generated by `quantileDemo`.

2.2.6 Quantiles

Since the cdf F is a monotonically increasing function, it has an inverse; let us denote this by F^{-1} . If F is the cdf of X , then $F^{-1}(\alpha)$ is the value of x_α such that $P(X \leq x_\alpha) = \alpha$; this is called the α **quantile** of F . The value $F^{-1}(0.5)$ is the **median** of the distribution, with half of the probability mass on the left, and half on the right. The values $F^{-1}(0.25)$ and $F^{-1}(0.75)$ are the lower and upper **quantiles**.

We can also use the inverse cdf to compute **tail area probabilities**. For example, if Φ is the cdf of the Gaussian distribution $\mathcal{N}(0, 1)$, then points to the left of $\Phi^{-1}(\alpha/2)$ contain $\alpha/2$ probability mass, as illustrated in Figure 2.3(b). By symmetry, points to the right of $\Phi^{-1}(1 - \alpha/2)$ also contain $\alpha/2$ of the mass. Hence the central interval $(\Phi^{-1}(\alpha/2), \Phi^{-1}(1 - \alpha/2))$ contains $1 - \alpha$ of the mass. If we set $\alpha = 0.05$, the central 95% interval is covered by the range

$$(\Phi^{-1}(0.025), \Phi^{-1}(0.975)) = (-1.96, 1.96) \quad (2.23)$$

If the distribution is $\mathcal{N}(\mu, \sigma^2)$, then the 95% interval becomes $(\mu - 1.96\sigma, \mu + 1.96\sigma)$. This is sometimes approximated by writing $\mu \pm 2\sigma$.

2.2.7 Mean and variance

The most familiar property of a distribution is its **mean**, or **expected value**, denoted by μ . For discrete rv's, it is defined as $\mathbb{E}[X] \triangleq \sum_{x \in \mathcal{X}} x p(x)$, and for continuous rv's, it is defined as $\mathbb{E}[X] \triangleq \int_{\mathcal{X}} x p(x) dx$. If this integral is not finite, the mean is not defined (we will see some examples of this later).

The **variance** is a measure of the “spread” of a distribution, denoted by σ^2 . This is defined

as follows:

$$\text{var}[X] \triangleq \mathbb{E}[(X - \mu)^2] = \int (x - \mu)^2 p(x) dx \quad (2.24)$$

$$= \int x^2 p(x) dx + \mu^2 \int p(x) dx - 2\mu \int x p(x) dx = \mathbb{E}[X^2] - \mu^2 \quad (2.25)$$

from which we derive the useful result

$$\mathbb{E}[X^2] = \mu^2 + \sigma^2 \quad (2.26)$$

The **standard deviation** is defined as

$$\text{std}[X] \triangleq \sqrt{\text{var}[X]} \quad (2.27)$$

This is useful since it has the same units as X itself.

2.3 Some common discrete distributions

In this section, we review some commonly used parametric distributions defined on discrete state spaces, both finite and countably infinite.

2.3.1 The binomial and Bernoulli distributions

Suppose we toss a coin n times. Let $X \in \{0, \dots, n\}$ be the number of heads. If the probability of heads is θ , then we say X has a **binomial** distribution, written as $X \sim \text{Bin}(n, \theta)$. The pmf is given by

$$\text{Bin}(k|n, \theta) \triangleq \binom{n}{k} \theta^k (1 - \theta)^{n-k} \quad (2.28)$$

where

$$\binom{n}{k} \triangleq \frac{n!}{(n-k)!k!} \quad (2.29)$$

is the number of ways to choose k items from n (this is known as the **binomial coefficient**, and is pronounced “n choose k”). See Figure 2.4 for some examples of the binomial distribution. This distribution has the following mean and variance:

$$\text{mean} = \theta, \quad \text{var} = n\theta(1 - \theta) \quad (2.30)$$

Now suppose we toss a coin only once. Let $X \in \{0, 1\}$ be a binary random variable, with probability of “success” or “heads” of θ . We say that X has a **Bernoulli** distribution. This is written as $X \sim \text{Ber}(\theta)$, where the pmf is defined as

$$\text{Ber}(x|\theta) = \theta^{\mathbb{I}(x=1)} (1 - \theta)^{\mathbb{I}(x=0)} \quad (2.31)$$

In other words,

$$\text{Ber}(x|\theta) = \begin{cases} \theta & \text{if } x = 1 \\ 1 - \theta & \text{if } x = 0 \end{cases} \quad (2.32)$$

This is obviously just a special case of a Binomial distribution with $n = 1$.

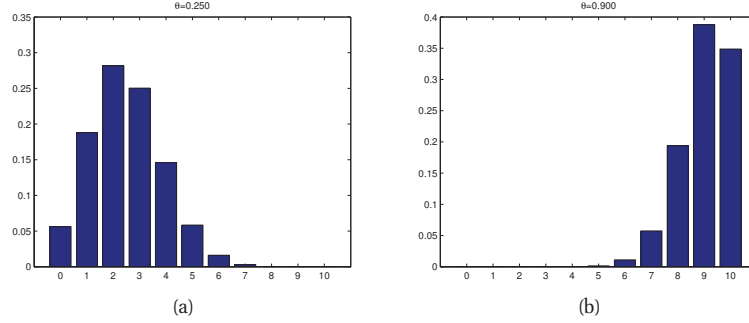


Figure 2.4 Illustration of the binomial distribution with $n = 10$ and $\theta \in \{0.25, 0.9\}$. Figure generated by `binomDistPlot`.

2.3.2 The multinomial and multinoulli distributions

The binomial distribution can be used to model the outcomes of coin tosses. To model the outcomes of tossing a K -sided die, we can use the **multinomial** distribution. This is defined as follows: let $\mathbf{x} = (x_1, \dots, x_K)$ be a random vector, where x_j is the number of times side j of the die occurs. Then $\mathbf{x}$ has the following pmf:

$$\text{Mu}(\mathbf{x}|n, \boldsymbol{\theta}) \triangleq \binom{n}{x_1 \dots x_K} \prod_{j=1}^K \theta_j^{x_j} \quad (2.33)$$

where θ_j is the probability that side j shows up, and

$$\binom{n}{x_1 \dots x_K} \triangleq \frac{n!}{x_1! x_2! \dots x_K!} \quad (2.34)$$

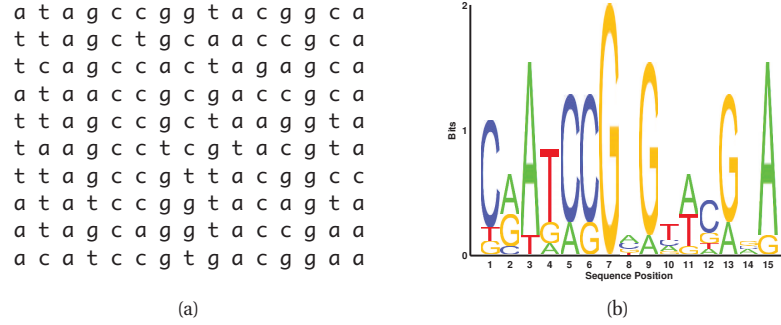
is the **multinomial coefficient** (the number of ways to divide a set of size $n = \sum_{k=1}^K x_k$ into subsets with sizes x_1 up to x_K).

Now suppose $n = 1$. This is like rolling a K -sided dice once, so $\mathbf{x}$ will be a vector of 0s and 1s (a bit vector), in which only one bit can be turned on. Specifically, if the dice shows up as face k , then the k 'th bit will be on. In this case, we can think of x as being a scalar categorical random variable with K states (values), and $\mathbf{x}$ is its **dummy encoding**, that is, $\mathbf{x} = [\mathbb{I}(x = 1), \dots, \mathbb{I}(x = K)]$. For example, if $K = 3$, we encode the states 1, 2 and 3 as $(1, 0, 0)$, $(0, 1, 0)$, and $(0, 0, 1)$. This is also called a **one-hot encoding**, since we imagine that only one of the K “wires” is “hot” or on. In this case, the pmf becomes

$$\text{Mu}(\mathbf{x}|1, \boldsymbol{\theta}) = \prod_{j=1}^K \theta_j^{\mathbb{I}(x_j=1)} \quad (2.35)$$

See Figure 2.1(b-c) for an example. This very common special case is known as a **categorical** or **discrete** distribution. (Gustavo Lacerda suggested we call it the **multinoulli distribution**, by analogy with the Binomial/ Bernoulli distinction, a term which we shall adopt in this book.) We

Name	n	K	x
Multinomial	-	-	$\mathbf{x} \in \{0, 1, \dots, n\}^K, \sum_{k=1}^K x_k = n$
Multinoulli	1	-	$\mathbf{x} \in \{0, 1\}^K, \sum_{k=1}^K x_k = 1$ (1-of- K encoding)
Binomial	-	1	$x \in \{0, 1, \dots, n\}$
Bernoulli	1	1	$x \in \{0, 1\}$

Table 2.1 Summary of the multinomial and related distributions.**Figure 2.5** (a) Some aligned DNA sequences. (b) The corresponding sequence logo. Figure generated by seqlogoDemo.

will use the following notation for this case:

$$\text{Cat}(x|\boldsymbol{\theta}) \triangleq \text{Mu}(\mathbf{x}|1, \boldsymbol{\theta}) \quad (2.36)$$

In otherwords, if $x \sim \text{Cat}(\boldsymbol{\theta})$, then $p(x = j|\boldsymbol{\theta}) = \theta_j$. See Table 2.1 for a summary.

2.3.2.1 Application: DNA sequence motifs

An interesting application of multinomial models arises in **biosequence analysis**. Suppose we have a set of (aligned) DNA sequences, such as in Figure 2.5(a), where there are 10 rows (sequences) and 15 columns (locations along the genome). We see that several locations are conserved by evolution (e.g., because they are part of a gene coding region), since the corresponding columns tend to be “pure”. For example, column 7 is all G’s.

One way to visually summarize the data is by using a **sequence logo**: see Figure 2.5(b). We plot the letters A, C, G and T with a fontsize proportional to their empirical probability, and with the most probable letter on the top. The empirical probability distribution at location t , $\hat{\boldsymbol{\theta}}_t$, is gotten by normalizing the vector of counts (see Equation 3.48):

$$\mathbf{N}_t = \left(\sum_{i=1}^N \mathbb{I}(X_{it} = 1), \sum_{i=1}^N \mathbb{I}(X_{it} = 2), \sum_{i=1}^N \mathbb{I}(X_{it} = 3), \sum_{i=1}^N \mathbb{I}(X_{it} = 4) \right) \quad (2.37)$$

$$\hat{\boldsymbol{\theta}}_t = \mathbf{N}_t / N \quad (2.38)$$

This distribution is known as a **motif**. We can also compute the most probable letter in each location; this is called the **consensus sequence**.

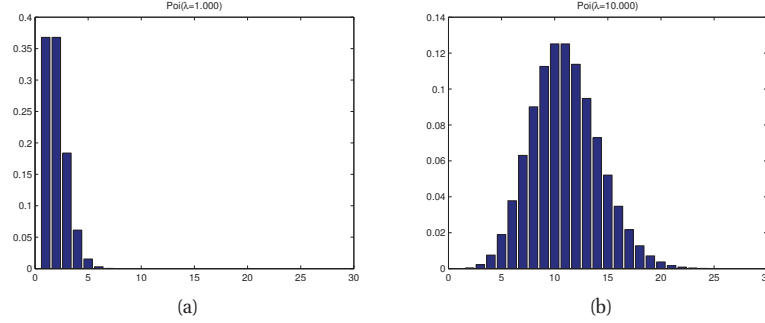


Figure 2.6 Illustration of some Poisson distributions for $\lambda \in \{1, 10\}$. We have truncated the x-axis to 25 for clarity, but the support of the distribution is over all the non-negative integers. Figure generated by `poissonPlotDemo`.

2.3.3 The Poisson distribution

We say that $X \in \{0, 1, 2, \dots\}$ has a **Poisson** distribution with parameter $\lambda > 0$, written $X \sim \text{Poi}(\lambda)$, if its pmf is

$$\text{Poi}(x|\lambda) = e^{-\lambda} \frac{\lambda^x}{x!} \quad (2.39)$$

The first term is just the normalization constant, required to ensure the distribution sums to 1.

The Poisson distribution is often used as a model for counts of rare events like radioactive decay and traffic accidents. See Figure 2.6 for some plots.

2.3.4 The empirical distribution

Given a set of data, $\mathcal{D} = \{x_1, \dots, x_N\}$, we define the **empirical distribution**, also called the **empirical measure**, as follows:

$$p_{\text{emp}}(A) \triangleq \frac{1}{N} \sum_{i=1}^N \delta_{x_i}(A) \quad (2.40)$$

where $\delta_x(A)$ is the **Dirac measure**, defined by

$$\delta_x(A) = \begin{cases} 0 & \text{if } x \notin A \\ 1 & \text{if } x \in A \end{cases} \quad (2.41)$$

In general, we can associate “weights” with each sample:

$$p(x) = \sum_{i=1}^N w_i \delta_{x_i}(x) \quad (2.42)$$

where we require $0 \leq w_i \leq 1$ and $\sum_{i=1}^N w_i = 1$. We can think of this as a histogram, with “spikes” at the data points x_i , where w_i determines the height of spike i . This distribution assigns 0 probability to any point not in the data set.

2.4 Some common continuous distributions

In this section we present some commonly used univariate (one-dimensional) continuous probability distributions.

2.4.1 Gaussian (normal) distribution

The most widely used distribution in statistics and machine learning is the Gaussian or normal distribution. Its pdf is given by

$$\mathcal{N}(x|\mu, \sigma^2) \triangleq \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(x-\mu)^2} \quad (2.43)$$

Here $\mu = \mathbb{E}[X]$ is the mean (and mode), and $\sigma^2 = \text{var}[X]$ is the variance. $\sqrt{2\pi\sigma^2}$ is the normalization constant needed to ensure the density integrates to 1 (see Exercise 2.11).

We write $X \sim \mathcal{N}(\mu, \sigma^2)$ to denote that $p(X = x) = \mathcal{N}(x|\mu, \sigma^2)$. If $X \sim \mathcal{N}(0, 1)$, we say X follows a **standard normal** distribution. See Figure 2.3(b) for a plot of this pdf; this is sometimes called the **bell curve**.

We will often talk about the **precision** of a Gaussian, by which we mean the inverse variance: $\lambda = 1/\sigma^2$. A high precision means a narrow distribution (low variance) centered on μ .⁴

Note that, since this is a pdf, we can have $p(x) > 1$. To see this, consider evaluating the density at its center, $x = \mu$. We have $\mathcal{N}(\mu|\mu, \sigma^2) = (\sigma\sqrt{2\pi})^{-1}e^0$, so if $\sigma < 1/\sqrt{2\pi}$, we have $p(x) > 1$.

The cumulative distribution function or cdf of the Gaussian is defined as

$$\Phi(x; \mu, \sigma^2) \triangleq \int_{-\infty}^x \mathcal{N}(z|\mu, \sigma^2) dz \quad (2.44)$$

See Figure 2.3(a) for a plot of this cdf when $\mu = 0$, $\sigma^2 = 1$. This integral has no closed form expression, but is built in to most software packages. In particular, we can compute it in terms of the **error function (erf)**:

$$\Phi(x; \mu, \sigma) = \frac{1}{2}[1 + \text{erf}(z/\sqrt{2})] \quad (2.45)$$

where $z = (x - \mu)/\sigma$ and

$$\text{erf}(x) \triangleq \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt \quad (2.46)$$

The Gaussian distribution is the most widely used distribution in statistics. There are several reasons for this. First, it has two parameters which are easy to interpret, and which capture some of the most basic properties of a distribution, namely its mean and variance. Second, the central limit theorem (Section 2.6.3) tells us that sums of independent random variables have an approximately Gaussian distribution, making it a good choice for modeling residual errors or “noise”. Third, the Gaussian distribution makes the least number of assumptions (has

4. The symbol λ will have many different meanings in this book, in order to be consistent with the rest of the literature. The intended meaning should be clear from context.

maximum entropy), subject to the constraint of having a specified mean and variance, as we show in Section 9.2.6; this makes it a good default choice in many cases. Finally, it has a simple mathematical form, which results in easy to implement, but often highly effective, methods, as we will see. See (Jaynes 2003, ch 7) for a more extensive discussion of why Gaussians are so widely used.

2.4.2 Degenerate pdf

In the limit that $\sigma^2 \rightarrow 0$, the Gaussian becomes an infinitely tall and infinitely thin “spike” centered at μ :

$$\lim_{\sigma^2 \rightarrow 0} \mathcal{N}(x|\mu, \sigma^2) = \delta(x - \mu) \quad (2.47)$$

where δ is called a **Dirac delta function**, and is defined as

$$\delta(x) = \begin{cases} \infty & \text{if } x = 0 \\ 0 & \text{if } x \neq 0 \end{cases} \quad (2.48)$$

such that

$$\int_{-\infty}^{\infty} \delta(x) dx = 1 \quad (2.49)$$

A useful property of delta functions is the **sifting property**, which selects out a single term from a sum or integral:

$$\int_{-\infty}^{\infty} f(x) \delta(x - \mu) dx = f(\mu) \quad (2.50)$$

since the integrand is only non-zero if $x - \mu = 0$.

One problem with the Gaussian distribution is that it is sensitive to outliers, since the log-probability only decays quadratically with distance from the center. A more robust distribution is the **Student t distribution**⁵ Its pdf is as follows:

$$\mathcal{T}(x|\mu, \sigma^2, \nu) \propto \left[1 + \frac{1}{\nu} \left(\frac{x - \mu}{\sigma} \right)^2 \right]^{-\left(\frac{\nu+1}{2}\right)} \quad (2.51)$$

where μ is the mean, $\sigma^2 > 0$ is the scale parameter, and $\nu > 0$ is called the **degrees of freedom**. See Figure 2.7 for some plots. For later reference, we note that the distribution has the following properties:

$$\text{mean} = \mu, \text{ mode} = \mu, \text{ var} = \frac{\nu\sigma^2}{(\nu - 2)} \quad (2.52)$$

5. This distribution has a colourful etymology. It was first published in 1908 by William Sealy Gosset, who worked at the Guinness brewery in Dublin. Since his employer would not allow him to use his own name, he called it the “Student” distribution. The origin of the term t seems to have arisen in the context of Tables of the Student distribution, used by Fisher when developing the basis of classical statistical inference. See <http://jeff560.tripod.com/s.html> for more historical details.

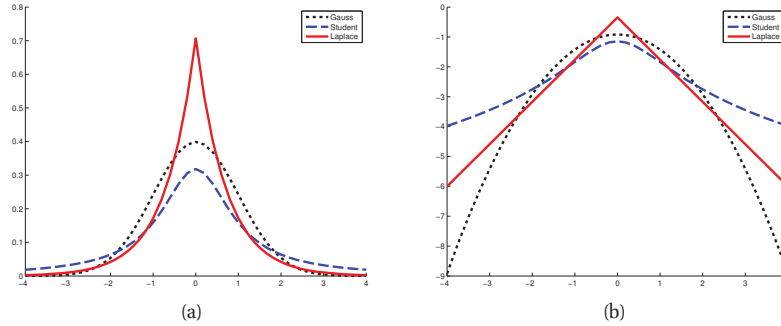


Figure 2.7 (a) The pdf's for a $\mathcal{N}(0, 1)$, $\mathcal{T}(0, 1, 1)$ and $\text{Lap}(0, 1/\sqrt{2})$. The mean is 0 and the variance is 1 for both the Gaussian and Laplace. The mean and variance of the Student is undefined when $\nu = 1$. (b) Log of these pdf's. Note that the Student distribution is not log-concave for any parameter value, unlike the Laplace distribution, which is always log-concave (and log-convex...). Nevertheless, both are unimodal. Figure generated by `studentLaplacePdfPlot`.

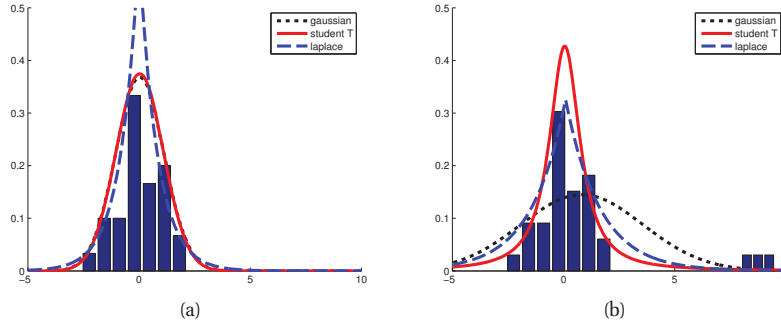


Figure 2.8 Illustration of the effect of outliers on fitting Gaussian, Student and Laplace distributions. (a) No outliers (the Gaussian and Student curves are on top of each other). (b) With outliers. We see that the Gaussian is more affected by outliers than the Student and Laplace distributions. Based on Figure 2.16 of (Bishop 2006a). Figure generated by `robustDemo`.

The variance is only defined if $\nu > 2$. The mean is only defined if $\nu > 1$.

As an illustration of the robustness of the Student distribution, consider Figure 2.8. On the left, we show a Gaussian and a Student fit to some data with no outliers. On the right, we add some outliers. We see that the Gaussian is affected a lot, whereas the Student distribution hardly changes. This is because the Student has heavier tails, at least for small ν (see Figure 2.7).

If $\nu = 1$, this distribution is known as the **Cauchy** or **Lorentz** distribution. This is notable for having such heavy tails that the integral that defines the mean does not converge.

To ensure finite variance, we require $\nu > 2$. It is common to use $\nu = 4$, which gives good performance in a range of problems (Lange et al. 1989). For $\nu \gg 5$, the Student distribution rapidly approaches a Gaussian distribution and loses its robustness properties.

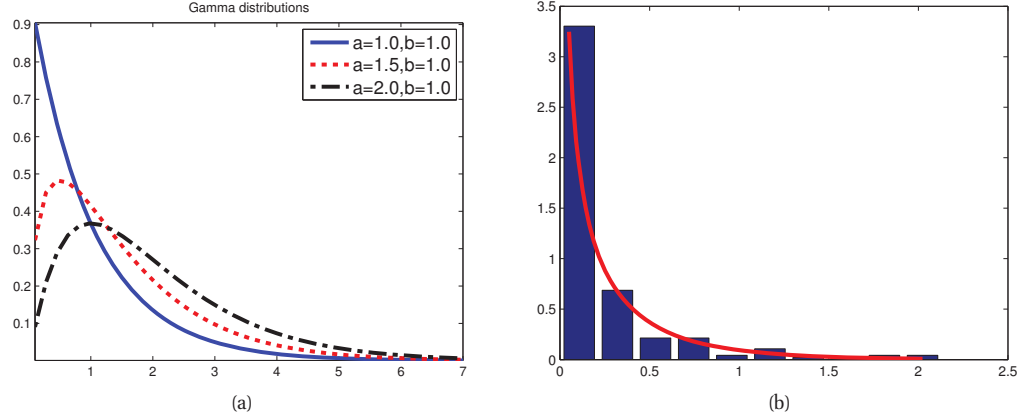


Figure 2.9 (a) Some $\text{Ga}(a, b = 1)$ distributions. If $a \leq 1$, the mode is at 0, otherwise it is > 0 . As we increase the rate b , we reduce the horizontal scale, thus squeezing everything leftwards and upwards. Figure generated by `gammaPlotDemo`. (b) An empirical pdf of some rainfall data, with a fitted Gamma distribution superimposed. Figure generated by `gammaRainfallDemo`.

2.4.3 The Laplace distribution

Another distribution with heavy tails is the **Laplace distribution**⁶, also known as the **double sided exponential** distribution. This has the following pdf:

$$\text{Lap}(x|\mu, b) \triangleq \frac{1}{2b} \exp\left(-\frac{|x - \mu|}{b}\right) \quad (2.53)$$

Here μ is a location parameter and $b > 0$ is a scale parameter. See Figure 2.7 for a plot. This distribution has the following properties:

$$\text{mean} = \mu, \text{ mode} = \mu, \text{ var} = 2b^2 \quad (2.54)$$

Its robustness to outliers is illustrated in Figure 2.8. It also puts more probability density at 0 than the Gaussian. This property is a useful way to encourage sparsity in a model, as we will see in Section 13.3.

2.4.4 The gamma distribution

The **gamma distribution** is a flexible distribution for positive real valued rv's, $x > 0$. It is defined in terms of two parameters, called the shape $a > 0$ and the rate $b > 0$:⁷

$$\text{Ga}(T|\text{shape} = a, \text{rate} = b) \triangleq \frac{b^a}{\Gamma(a)} T^{a-1} e^{-Tb} \quad (2.55)$$

6. Pierre-Simon Laplace (1749–1827) was a French mathematician, who played a key role in creating the field of Bayesian statistics.

7. There is an alternative parameterization, where we use the scale parameter instead of the rate: $\text{Ga}_s(T|a, b) \triangleq \text{Ga}(T|a, 1/b)$. This version is the one used by Matlab's `gampdf`, although in this book will use the rate parameterization unless otherwise specified.

where $\Gamma(a)$ is the gamma function:

$$\Gamma(x) \triangleq \int_0^\infty u^{x-1} e^{-u} du \quad (2.56)$$

See Figure 2.9 for some plots. For later reference, we note that the distribution has the following properties:

$$\text{mean} = \frac{a}{b}, \quad \text{mode} = \frac{a-1}{b}, \quad \text{var} = \frac{a}{b^2} \quad (2.57)$$

There are several distributions which are just special cases of the Gamma, which we discuss below.

- **Exponential distribution** This is defined by $\text{Expon}(x|\lambda) \triangleq \text{Ga}(x|1, \lambda)$, where λ is the rate parameter. This distribution describes the times between events in a Poisson process, i.e. a process in which events occur continuously and independently at a constant average rate λ .
- **Erlang distribution** This is the same as the Gamma distribution where a is an integer. It is common to fix $a = 2$, yielding the one-parameter Erlang distribution, $\text{Erlang}(x|\lambda) = \text{Ga}(x|2, \lambda)$, where λ is the rate parameter.
- **Chi-squared distribution** This is defined by $\chi^2(x|\nu) \triangleq \text{Ga}(x|\frac{\nu}{2}, \frac{1}{2})$. This is the distribution of the sum of squared Gaussian random variables. More precisely, if $Z_i \sim \mathcal{N}(0, 1)$, and $S = \sum_{i=1}^\nu Z_i^2$, then $S \sim \chi_\nu^2$.

Another useful result is the following: If $X \sim \text{Ga}(a, b)$, then one can show (Exercise 2.10) that $\frac{1}{X} \sim \text{IG}(a, b)$, where **IG** is the **inverse gamma** distribution defined by

$$\text{IG}(x|\text{shape} = a, \text{scale} = b) \triangleq \frac{b^a}{\Gamma(a)} x^{-(a+1)} e^{-b/x} \quad (2.58)$$

The distribution has these properties

$$\text{mean} = \frac{b}{a-1}, \quad \text{mode} = \frac{b}{a+1}, \quad \text{var} = \frac{b^2}{(a-1)^2(a-2)}, \quad (2.59)$$

The mean only exists if $a > 1$. The variance only exists if $a > 2$.

We will see applications of these distributions later on.

2.4.5 The beta distribution

The **beta distribution** has support over the interval $[0, 1]$ and is defined as follows:

$$\text{Beta}(x|a, b) = \frac{1}{B(a, b)} x^{a-1} (1-x)^{b-1} \quad (2.60)$$

Here $B(p, q)$ is the beta function,

$$B(a, b) \triangleq \frac{\Gamma(a)\Gamma(b)}{\Gamma(a+b)} \quad (2.61)$$

See Figure 2.10 for plots of some beta distributions. We require $a, b > 0$ to ensure the distribution is integrable (i.e., to ensure $B(a, b)$ exists). If $a = b = 1$, we get the uniform distribution. If

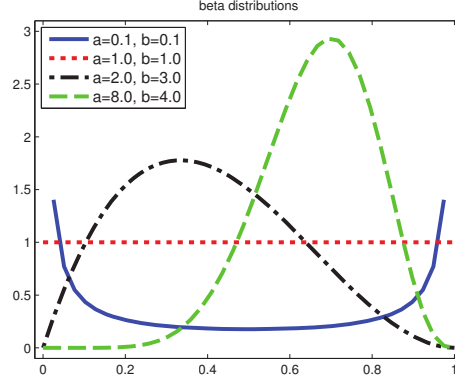


Figure 2.10 Some beta distributions. Figure generated by betaPlotDemo.

a and b are both less than 1, we get a bimodal distribution with “spikes” at 0 and 1; if a and b are both greater than 1, the distribution is unimodal. For later reference, we note that the distribution has the following properties (Exercise 2.16):

$$\text{mean} = \frac{a}{a+b}, \quad \text{mode} = \frac{a-1}{a+b-2}, \quad \text{var} = \frac{ab}{(a+b)^2(a+b+1)} \quad (2.62)$$

2.4.6 Pareto distribution

The **Pareto distribution** is used to model the distribution of quantities that exhibit **long tails**, also called **heavy tails**. For example, it has been observed that the most frequent word in English (“the”) occurs approximately twice as often as the second most frequent word (“of”), which occurs twice as often as the fourth most frequent word, etc. If we plot the frequency of words vs their rank, we will get a power law; this is known as **Zipf’s law**. Wealth has a similarly skewed distribution, especially in plutocracies such as the USA.⁸

The Pareto pdf is defined as follow:

$$\text{Pareto}(x|k, m) = km^k x^{-(k+1)} \mathbb{I}(x \geq m) \quad (2.63)$$

This density asserts that x must be greater than some constant m , but not too much greater, where k controls what is “too much”. As $k \rightarrow \infty$, the distribution approaches $\delta(x - m)$. See Figure 2.11(a) for some plots. If we plot the distribution on a log-log scale, it forms a straight line, of the form $\log p(x) = a \log x + c$ for some constants a and c . See Figure 2.11(b) for an illustration (this is known as a **power law**). This distribution has the following properties

$$\text{mean} = \frac{km}{k-1} \text{ if } k > 1, \quad \text{mode} = m, \quad \text{var} = \frac{m^2 k}{(k-1)^2(k-2)} \text{ if } k > 2 \quad (2.64)$$

8. In the USA, 400 Americans have more wealth than half of all Americans combined. (Source: <http://www.politifact.com/wisconsin/statements/2011/mar/10/michael-moore/michael-moore-says-400-americans-have-more-wealth->.) See (Hacker and Pierson 2010) for a political analysis of how such an extreme distribution of income has arisen in a democratic country.

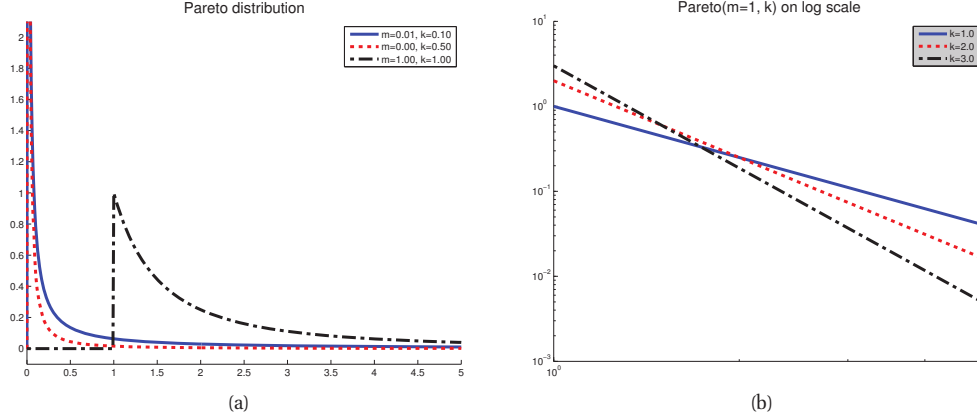


Figure 2.11 (a) The Pareto distribution $\text{Pareto}(x|m, k)$ for $m = 1$. (b) The pdf on a log-log scale. Figure generated by `paretoPlot`.

2.5 Joint probability distributions

So far, we have been mostly focusing on modeling univariate probability distributions. In this section, we start our discussion of the more challenging problem of building joint probability distributions on multiple related random variables; this will be a central topic in this book.

A **joint probability distribution** has the form $p(x_1, \dots, x_D)$ for a set of $D > 1$ variables, and models the (stochastic) relationships between the variables. If all the variables are discrete, we can represent the joint distribution as a big multi-dimensional array, with one variable per dimension. However, the number of parameters needed to define such a model is $O(K^D)$, where K is the number of states for each variable.

We can define high dimensional joint distributions using fewer parameters by making conditional independence assumptions, as we explain in Chapter 10. In the case of continuous distributions, an alternative approach is to restrict the form of the pdf to certain functional forms, some of which we will examine below.

2.5.1 Covariance and correlation

The **covariance** between two rv's X and Y measures the degree to which X and Y are (linearly) related. Covariance is defined as

$$\text{cov}[X, Y] \triangleq \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y] \quad (2.65)$$

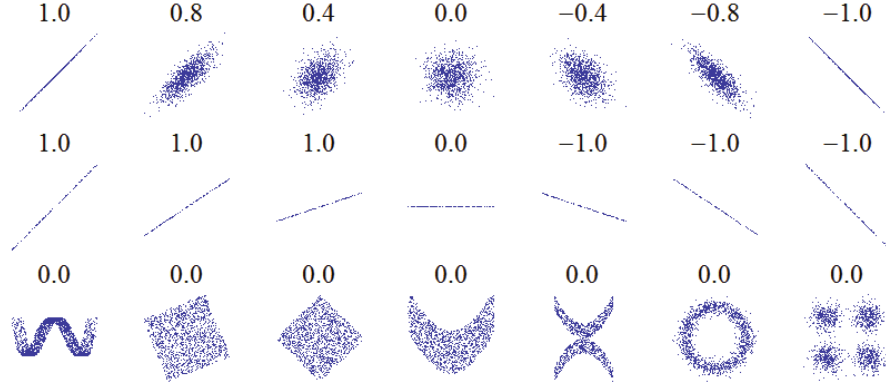


Figure 2.12 Several sets of (x, y) points, with the correlation coefficient of x and y for each set. Note that the correlation reflects the noisiness and direction of a linear relationship (top row), but not the slope of that relationship (middle), nor many aspects of nonlinear relationships (bottom). N.B.: the figure in the center has a slope of 0 but in that case the correlation coefficient is undefined because the variance of Y is zero. Source: http://en.wikipedia.org/wiki/File:Correlation_examples.png

If $\mathbf{x}$ is a d -dimensional random vector, its **covariance matrix** is defined to be the following symmetric, positive definite matrix:

$$\text{cov}[\mathbf{x}] \triangleq \mathbb{E}[(\mathbf{x} - \mathbb{E}[\mathbf{x}])(\mathbf{x} - \mathbb{E}[\mathbf{x}])^T] \quad (2.66)$$

$$= \begin{pmatrix} \text{var}[X_1] & \text{cov}[X_1, X_2] & \cdots & \text{cov}[X_1, X_d] \\ \text{cov}[X_2, X_1] & \text{var}[X_2] & \cdots & \text{cov}[X_2, X_d] \\ \vdots & \vdots & \ddots & \vdots \\ \text{cov}[X_d, X_1] & \text{cov}[X_d, X_2] & \cdots & \text{var}[X_d] \end{pmatrix} \quad (2.67)$$

Covariances can be between 0 and infinity. Sometimes it is more convenient to work with a normalized measure, with a finite upper bound. The (Pearson) **correlation coefficient** between X and Y is defined as

$$\text{corr}[X, Y] \triangleq \frac{\text{cov}[X, Y]}{\sqrt{\text{var}[X] \text{var}[Y]}} \quad (2.68)$$

A **correlation matrix** has the form

$$\mathbf{R} = \begin{pmatrix} \text{corr}[X_1, X_1] & \text{corr}[X_1, X_2] & \cdots & \text{corr}[X_1, X_d] \\ \vdots & \vdots & \ddots & \vdots \\ \text{corr}[X_d, X_1] & \text{corr}[X_d, X_2] & \cdots & \text{corr}[X_d, X_d] \end{pmatrix} \quad (2.69)$$

One can show (Exercise 4.3) that $-1 \leq \text{corr}[X, Y] \leq 1$. Hence in a correlation matrix, each entry on the diagonal is 1, and the other entries are between -1 and 1.

One can also show that $\text{corr}[X, Y] = 1$ if and only if $Y = aX + b$ for some parameters a and b , i.e., if there is a *linear* relationship between X and Y (see Exercise 4.4). Intuitively one

might expect the correlation coefficient to be related to the slope of the regression line, i.e., the coefficient a in the expression $Y = aX + b$. However, as we show in Equation 7.99 later, the regression coefficient is in fact given by $a = \text{cov}[X, Y] / \text{var}[X]$. A better way to think of the correlation coefficient is as a degree of linearity: see Figure 2.12.

If X and Y are independent, meaning $p(X, Y) = p(X)p(Y)$ (see Section 2.2.4), then $\text{cov}[X, Y] = 0$, and hence $\text{corr}[X, Y] = 0$ so they are uncorrelated. However, the converse is not true: *uncorrelated does not imply independent*. For example, let $X \sim U(-1, 1)$ and $Y = X^2$. Clearly Y is dependent on X (in fact, Y is uniquely determined by X), yet one can show (Exercise 4.1) that $\text{corr}[X, Y] = 0$. Some striking examples of this fact are shown in Figure 2.12. This shows several data sets where there is clear dependence between X and Y , and yet the correlation coefficient is 0. A more general measure of dependence between random variables is mutual information, discussed in Section 2.8.3. This is only zero if the variables truly are independent.

2.5.2 The multivariate Gaussian

The **multivariate Gaussian** or **multivariate normal** (MVN) is the most widely used joint probability density function for continuous variables. We discuss MVNs in detail in Chapter 4; here we just give some definitions and plots.

The pdf of the MVN in D dimensions is defined by the following:

$$\mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) \triangleq \frac{1}{(2\pi)^{D/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right] \quad (2.70)$$

where $\boldsymbol{\mu} = \mathbb{E}[\mathbf{x}] \in \mathbb{R}^D$ is the mean vector, and $\boldsymbol{\Sigma} = \text{cov}[\mathbf{x}]$ is the $D \times D$ covariance matrix. Sometimes we will work in terms of the **precision matrix** or **concentration matrix** instead. This is just the inverse covariance matrix, $\boldsymbol{\Lambda} = \boldsymbol{\Sigma}^{-1}$. The normalization constant $(2\pi)^{-D/2}|\boldsymbol{\Lambda}|^{1/2}$ just ensures that the pdf integrates to 1 (see Exercise 4.5).

Figure 2.13 plots some MVN densities in 2d for three different kinds of covariance matrices. A full covariance matrix has $D(D+1)/2$ parameters (we divide by 2 since $\boldsymbol{\Sigma}$ is symmetric). A diagonal covariance matrix has D parameters, and has 0s in the off-diagonal terms. A **spherical** or **isotropic** covariance, $\boldsymbol{\Sigma} = \sigma^2 \mathbf{I}_D$, has one free parameter.

2.5.3 Multivariate Student t distribution

A more robust alternative to the MVN is the **multivariate Student t** distribution, whose pdf is given by

$$\mathcal{T}(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}, \nu) = \frac{\Gamma(\nu/2 + D/2)}{\Gamma(\nu/2)} \frac{|\boldsymbol{\Sigma}|^{-1/2}}{\nu^{D/2} \pi^{D/2}} \times \left[1 + \frac{1}{\nu}(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right]^{-\left(\frac{\nu+D}{2}\right)} \quad (2.71)$$

$$= \frac{\Gamma(\nu/2 + D/2)}{\Gamma(\nu/2)} |\pi \mathbf{V}|^{-1/2} \times \left[1 + (\mathbf{x} - \boldsymbol{\mu})^T \mathbf{V}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right]^{-\left(\frac{\nu+D}{2}\right)} \quad (2.72)$$

where $\boldsymbol{\Sigma}$ is called the scale matrix (since it is not exactly the covariance matrix) and $\mathbf{V} = \nu \boldsymbol{\Sigma}$. This has fatter tails than a Gaussian. The smaller ν is, the fatter the tails. As $\nu \rightarrow \infty$, the

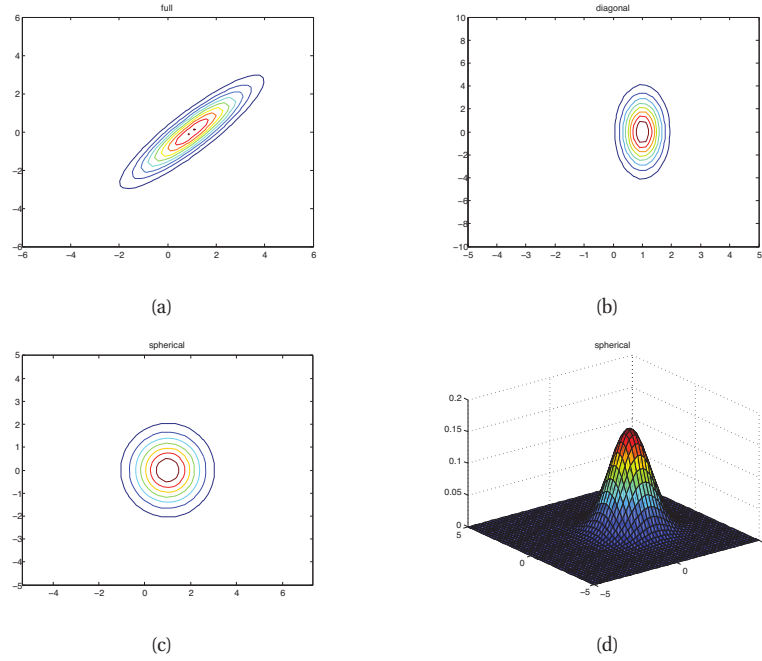


Figure 2.13 We show the level sets for 2d Gaussians. (a) A full covariance matrix has elliptical contours. (b) A diagonal covariance matrix is an **axis aligned** ellipse. (c) A spherical covariance matrix has a circular shape. (d) Surface plot for the spherical Gaussian in (c). Figure generated by `gaussPlot2Ddemo`.

distribution tends towards a Gaussian. The distribution has the following properties

$$\text{mean} = \boldsymbol{\mu}, \quad \text{mode} = \boldsymbol{\mu}, \quad \text{Cov} = \frac{\nu}{\nu - 2} \boldsymbol{\Sigma} \quad (2.73)$$

2.5.4 Dirichlet distribution

A multivariate generalization of the beta distribution is the **Dirichlet distribution**⁹, which has support over the **probability simplex**, defined by

$$S_K = \{\mathbf{x} : 0 \leq x_k \leq 1, \sum_{k=1}^K x_k = 1\} \quad (2.74)$$

The pdf is defined as follows:

$$\text{Dir}(\mathbf{x}|\boldsymbol{\alpha}) \triangleq \frac{1}{B(\boldsymbol{\alpha})} \prod_{k=1}^K x_k^{\alpha_k - 1} \mathbb{I}(\mathbf{x} \in S_K) \quad (2.75)$$

9. Johann Dirichlet was a German mathematician, 1805–1859.

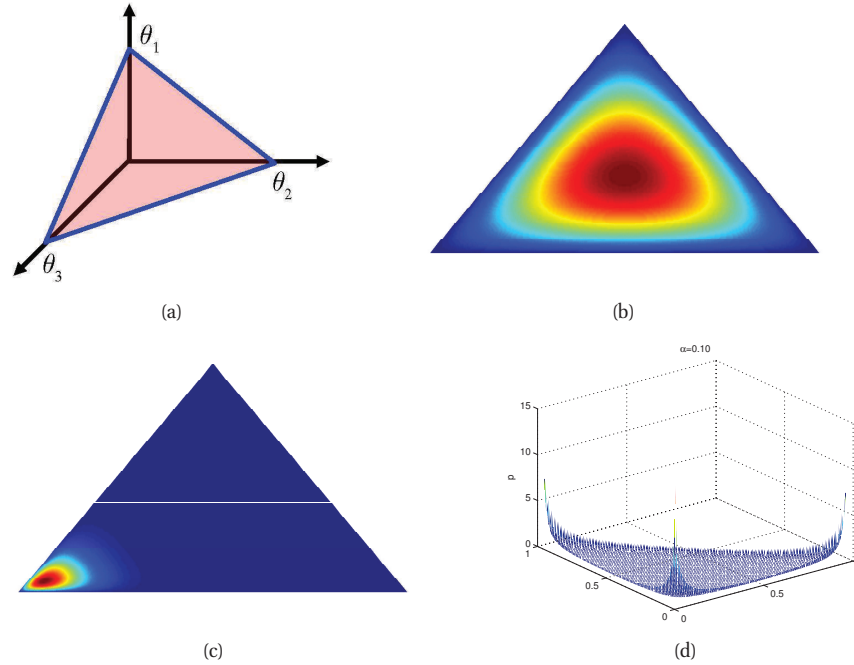


Figure 2.14 (a) The Dirichlet distribution when $K = 3$ defines a distribution over the simplex, which can be represented by the triangular surface. Points on this surface satisfy $0 \leq \theta_k \leq 1$ and $\sum_{k=1}^3 \theta_k = 1$. (b) Plot of the Dirichlet density when $\alpha = (2, 2, 2)$. (c) $\alpha = (20, 2, 2)$. Figure generated by `visDirichletGui`, by Jonathan Huang. (d) $\alpha = (0.1, 0.1, 0.1)$. (The comb-like structure on the edges is a plotting artifact.) Figure generated by `dirichlet3dPlot`.

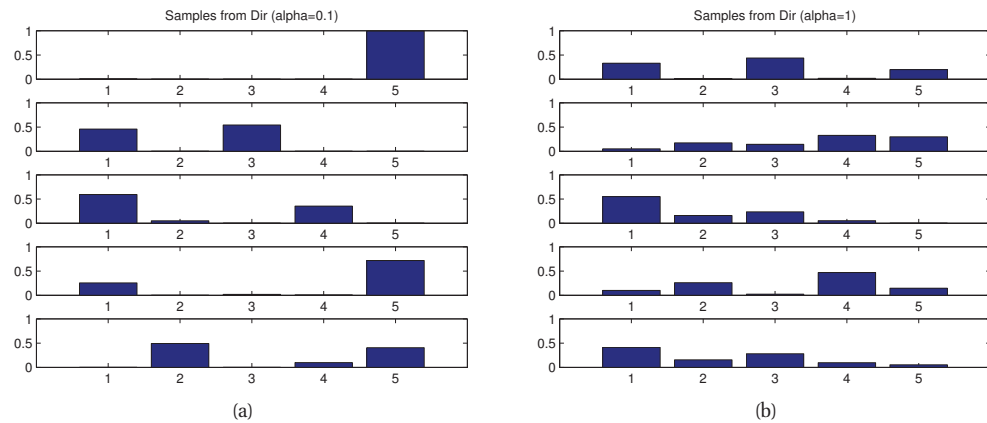


Figure 2.15 Samples from a 5-dimensional symmetric Dirichlet distribution for different parameter values. (a) $\alpha = (0.1, \dots, 0.1)$. This results in very sparse distributions, with many 0s. (b) $\alpha = (1, \dots, 1)$. This results in more uniform (and dense) distributions. Figure generated by `dirichletHistogramDemo`.

where $B(\alpha_1, \dots, \alpha_K)$ is the natural generalization of the beta function to K variables:

$$B(\boldsymbol{\alpha}) \triangleq \frac{\prod_{k=1}^K \Gamma(\alpha_k)}{\Gamma(\alpha_0)} \quad (2.76)$$

where $\alpha_0 \triangleq \sum_{k=1}^K \alpha_k$.

Figure 2.14 shows some plots of the Dirichlet when $K = 3$, and Figure 2.15 for some sampled probability vectors. We see that $\alpha_0 = \sum_{k=1}^K \alpha_k$ controls the strength of the distribution (how peaked it is), and the α_k control where the peak occurs. For example, $\text{Dir}(1, 1, 1)$ is a uniform distribution, $\text{Dir}(2, 2, 2)$ is a broad distribution centered at $(1/3, 1/3, 1/3)$, and $\text{Dir}(20, 20, 20)$ is a narrow distribution centered at $(1/3, 1/3, 1/3)$. If $\alpha_k < 1$ for all k , we get “spikes” at the corner of the simplex.

For future reference, the distribution has these properties

$$\mathbb{E}[x_k] = \frac{\alpha_k}{\alpha_0}, \quad \text{mode}[x_k] = \frac{\alpha_k - 1}{\alpha_0 - K}, \quad \text{var}[x_k] = \frac{\alpha_k(\alpha_0 - \alpha_k)}{\alpha_0^2(\alpha_0 + 1)} \quad (2.77)$$

where $\alpha_0 = \sum_k \alpha_k$. Often we use a symmetric Dirichlet prior of the form $\alpha_k = \alpha/K$. In this case, the mean becomes $1/K$, and the variance becomes $\text{var}[x_k] = \frac{K-1}{K^2(\alpha+1)}$. So increasing α increases the precision (decreases the variance) of the distribution.

2.6 Transformations of random variables

If $\mathbf{x} \sim p()$ is some random variable, and $\mathbf{y} = f(\mathbf{x})$, what is the distribution of $\mathbf{y}$? This is the question we address in this section.

2.6.1 Linear transformations

Suppose $f()$ is a linear function:

$$\mathbf{y} = f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b} \quad (2.78)$$

In this case, we can easily derive the mean and covariance of $\mathbf{y}$ as follows. First, for the mean, we have

$$\mathbb{E}[\mathbf{y}] = \mathbb{E}[\mathbf{A}\mathbf{x} + \mathbf{b}] = \mathbf{A}\boldsymbol{\mu} + \mathbf{b} \quad (2.79)$$

where $\boldsymbol{\mu} = \mathbb{E}[\mathbf{x}]$. This is called the **linearity of expectation**. If $f()$ is a scalar-valued function, $f(\mathbf{x}) = \mathbf{a}^T \mathbf{x} + b$, the corresponding result is

$$\mathbb{E}[\mathbf{a}^T \mathbf{x} + b] = \mathbf{a}^T \boldsymbol{\mu} + b \quad (2.80)$$

For the covariance, we have

$$\text{cov}[\mathbf{y}] = \text{cov}[\mathbf{A}\mathbf{x} + \mathbf{b}] = \mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^T \quad (2.81)$$

where $\boldsymbol{\Sigma} = \text{cov}[\mathbf{x}]$. We leave the proof of this as an exercise. If $f()$ is scalar valued, the result becomes

$$\text{var}[y] = \text{var}[\mathbf{a}^T \mathbf{x} + b] = \mathbf{a}^T \boldsymbol{\Sigma} \mathbf{a} \quad (2.82)$$

We will use both of these results extensively in later chapters. Note, however, that the mean and covariance only completely define the distribution of $\mathbf{y}$ if $\mathbf{x}$ is Gaussian. In general we must use the techniques described below to derive the full distribution of $\mathbf{y}$, as opposed to just its first two moments.

2.6.2 General transformations

If X is a discrete rv, we can derive the pmf for y by simply summing up the probability mass for all the x 's such that $f(x) = y$:

$$p_y(y) = \sum_{x: f(x)=y} p_x(x) \quad (2.83)$$

For example, if $f(X) = 1$ if X is even and $f(X) = 0$ otherwise, and $p_x(X)$ is uniform on the set $\{1, \dots, 10\}$, then $p_y(1) = \sum_{x \in \{2, 4, 6, 8, 10\}} p_x(x) = 0.5$, and $p_y(0) = 0.5$ similarly. Note that in this example, f is a many-to-one function.

If X is continuous, we cannot use Equation 2.83 since $p_x(x)$ is a density, not a pmf, and we cannot sum up densities. Instead, we work with cdf's, and write

$$P_y(y) \triangleq P(Y \leq y) = P(f(X) \leq y) = P(X \in \{x | f(x) \leq y\}) \quad (2.84)$$

We can derive the pdf of y by differentiating the cdf.

In the case of monotonic and hence invertible functions, we can write

$$P_y(y) = P(f(X) \leq y) = P(X \leq f^{-1}(y)) = P_x(f^{-1}(y)) \quad (2.85)$$

Taking derivatives we get

$$p_y(y) \triangleq \frac{d}{dy} P_y(y) = \frac{d}{dy} P_x(f^{-1}(y)) = \frac{dx}{dy} \frac{d}{dx} P_x(x) = \frac{dx}{dy} p_x(x) \quad (2.86)$$

where $x = f^{-1}(y)$. We can think of dx as a measure of volume in the x -space; similarly dy measures volume in y space. Thus $\frac{dx}{dy}$ measures the change in volume. Since the sign of this change is not important, we take the absolute value to get the general expression:

$$p_y(y) = p_x(x) \left| \frac{dx}{dy} \right| \quad (2.87)$$

This is called **change of variables** formula. We can understand this result more intuitively as follows. Observations falling in the range $(x, x + \delta x)$ will get transformed into $(y, y + \delta y)$, where $p_x(x)\delta x \approx p_y(y)\delta y$. Hence $p_y(y) \approx p_x(x) \left| \frac{\delta x}{\delta y} \right|$. For example, suppose $X \sim U(-1, 1)$, and $Y = X^2$. Then $p_y(y) = \frac{1}{2}y^{-\frac{1}{2}}$. See also Exercise 2.10.

2.6.2.1 Multivariate change of variables *

We can extend the previous results to multivariate distributions as follows. Let f be a function that maps $\mathbb{R}^n$ to $\mathbb{R}^n$, and let $\mathbf{y} = f(\mathbf{x})$. Then its **Jacobian matrix** $\mathbf{J}$ is given by

$$\mathbf{J}_{\mathbf{x} \rightarrow \mathbf{y}} \triangleq \frac{\partial(y_1, \dots, y_n)}{\partial(x_1, \dots, x_n)} \triangleq \begin{pmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_n}{\partial x_1} & \dots & \frac{\partial y_n}{\partial x_n} \end{pmatrix} \quad (2.88)$$

$|\det \mathbf{J}|$ measures how much a unit cube changes in volume when we apply f .

If f is an invertible mapping, we can define the pdf of the transformed variables using the Jacobian of the inverse mapping $\mathbf{y} \rightarrow \mathbf{x}$:

$$p_{\mathbf{y}}(\mathbf{y}) = p_{\mathbf{x}}(\mathbf{x}) \left| \det \left(\frac{\partial \mathbf{x}}{\partial \mathbf{y}} \right) \right| = p_{\mathbf{x}}(\mathbf{x}) |\det \mathbf{J}_{\mathbf{y} \rightarrow \mathbf{x}}| \quad (2.89)$$

In Exercise 4.5 you will use this formula to derive the normalization constant for a multivariate Gaussian.

As a simple example, consider transforming a density from Cartesian coordinates $\mathbf{x} = (x_1, x_2)$ to polar coordinates $\mathbf{y} = (r, \theta)$, where $x_1 = r \cos \theta$ and $x_2 = r \sin \theta$. Then

$$\mathbf{J}_{\mathbf{y} \rightarrow \mathbf{x}} = \begin{pmatrix} \frac{\partial x_1}{\partial r} & \frac{\partial x_1}{\partial \theta} \\ \frac{\partial x_2}{\partial r} & \frac{\partial x_2}{\partial \theta} \end{pmatrix} = \begin{pmatrix} \cos \theta & -r \sin \theta \\ \sin \theta & r \cos \theta \end{pmatrix} \quad (2.90)$$

and

$$|\det \mathbf{J}| = |r \cos^2 \theta + r \sin^2 \theta| = |r| \quad (2.91)$$

Hence

$$p_{\mathbf{y}}(\mathbf{y}) = p_{\mathbf{x}}(\mathbf{x}) |\det \mathbf{J}| \quad (2.92)$$

$$p_{r,\theta}(r, \theta) = p_{x_1, x_2}(x_1, x_2) r = p_{x_1, x_2}(r \cos \theta, r \sin \theta) r \quad (2.93)$$

To see this geometrically, notice that the area of the shaded patch in Figure 2.16 is given by

$$P(r \leq R \leq r + dr, \theta \leq \Theta \leq \theta + d\theta) = p_{r,\theta}(r, \theta) dr d\theta \quad (2.94)$$

In the limit, this is equal to the density at the center of the patch, $p(r, \theta)$, times the size of the patch, $r dr d\theta$. Hence

$$p_{r,\theta}(r, \theta) dr d\theta = p_{x_1, x_2}(r \cos \theta, r \sin \theta) r dr d\theta \quad (2.95)$$

2.6.3 Central limit theorem

Now consider N random variables with pdf's (not necessarily Gaussian) $p(x_i)$, each with mean μ and variance σ^2 . We assume each variable is **independent and identically distributed** or **iid** for short. Let $S_N = \sum_{i=1}^N X_i$ be the sum of the rv's. This is a simple but widely used transformation of rv's. One can show that, as N increases, the distribution of this sum approaches

$$p(S_N = s) = \frac{1}{\sqrt{2\pi N \sigma^2}} \exp \left(-\frac{(s - N\mu)^2}{2N\sigma^2} \right) \quad (2.96)$$

Hence the distribution of the quantity

$$Z_N \triangleq \frac{S_N - N\mu}{\sigma\sqrt{N}} = \frac{\bar{X} - \mu}{\sigma/\sqrt{N}} \quad (2.97)$$

converges to the standard normal, where $\bar{X} = \frac{1}{N} \sum_{i=1}^N x_i$ is the sample mean. This is called the **central limit theorem**. See e.g., (Jaynes 2003, p222) or (Rice 1995, p169) for a proof.

In Figure 2.17 we give an example in which we compute the mean of rv's drawn from a beta distribution. We see that the sampling distribution of the mean value rapidly converges to a Gaussian distribution.

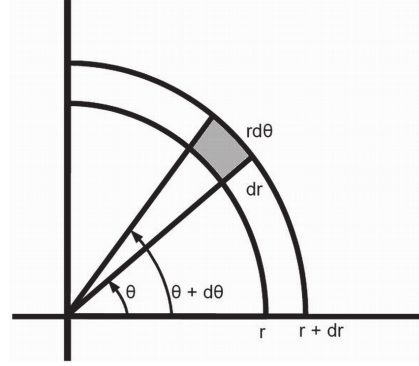


Figure 2.16 Change of variables from polar to Cartesian. The area of the shaded patch is $r dr d\theta$. Based on (Rice 1995) Figure 3.16.

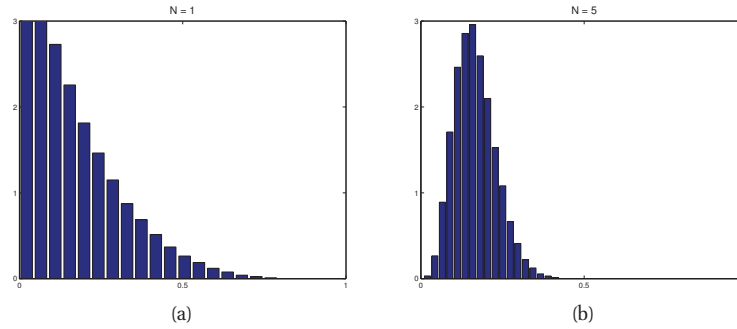


Figure 2.17 The central limit theorem in pictures. We plot a histogram of $\frac{1}{N} \sum_{i=1}^N x_{ij}$, where $x_{ij} \sim \text{Beta}(1, 5)$, for $j = 1 : 10000$. As $N \rightarrow \infty$, the distribution tends towards a Gaussian. (a) $N = 1$. (b) $N = 5$. Based on Figure 2.6 of (Bishop 2006a). Figure generated by `centralLimitDemo`.

2.7 Monte Carlo approximation

In general, computing the distribution of a function of an rv using the change of variables formula can be difficult. One simple but powerful alternative is as follows. First we generate S samples from the distribution, call them $x_1, \dots, x_S$. (There are many ways to generate such samples; one popular method, for high dimensional distributions, is called Markov chain Monte Carlo or MCMC; this will be explained in Chapter 24.) Given the samples, we can approximate the distribution of $f(X)$ by using the empirical distribution of $\{f(x_s)\}_{s=1}^S$. This is called a **Monte Carlo** approximation, named after a city in Europe known for its plush gambling casinos. Monte Carlo techniques were first developed in the area of statistical physics — in particular, during development of the atomic bomb — but are now widely used in statistics and machine learning as well.

We can use Monte Carlo to approximate the expected value of any function of a random

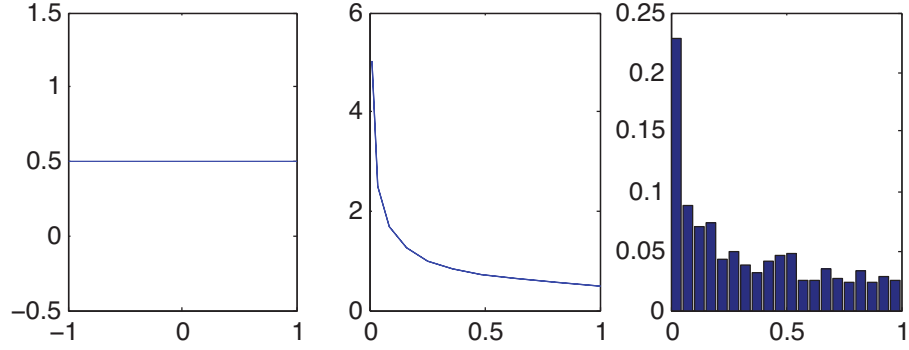


Figure 2.18 Computing the distribution of $y = x^2$, where $p(x)$ is uniform (left). The analytic result is shown in the middle, and the Monte Carlo approximation is shown on the right. Figure generated by `changeOfVarsDemo1d`.

variable. We simply draw samples, and then compute the arithmetic mean of the function applied to the samples. This can be written as follows:

$$\mathbb{E}[f(X)] = \int f(x)p(x)dx \approx \frac{1}{S} \sum_{s=1}^S f(x_s) \quad (2.98)$$

where $x_s \sim p(X)$. This is called Monte Carlo integration, and has the advantage over numerical integration (which is based on evaluating the function at a fixed grid of points) that the function is only evaluated in places where there is non-negligible probability.

By varying the function $f()$, we can approximate many quantities of interest, such as

- $\bar{x} = \frac{1}{S} \sum_{s=1}^S x_s \rightarrow \mathbb{E}[X]$
- $\frac{1}{S} \sum_{s=1}^S (x_s - \bar{x})^2 \rightarrow \text{var}[X]$
- $\frac{1}{S} \#\{x_s \leq c\} \rightarrow P(X \leq c)$
- $\text{median}\{x_1, \dots, x_S\} \rightarrow \text{median}(X)$

We give some examples below, and will see many more in later chapters.

2.7.1 Example: change of variables, the MC way

In Section 2.6.2, we discussed how to analytically compute the distribution of a function of a random variable, $y = f(x)$. A much simpler approach is to use a Monte Carlo approximation. For example, suppose $x \sim \text{Unif}(-1, 1)$ and $y = x^2$. We can approximate $p(y)$ by drawing many samples from $p(x)$, squaring them, and computing the resulting empirical distribution. See Figure 2.18 for an illustration. We will use this technique extensively in later chapters. See also Figure 5.2.

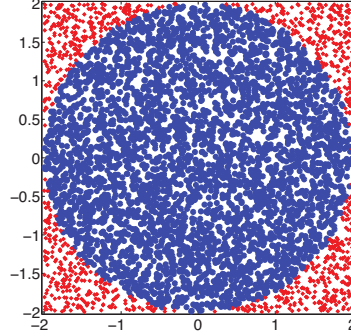


Figure 2.19 Estimating π by Monte Carlo integration. Blue points are inside the circle, red crosses are outside. Figure generated by `mcEstimatePi`.

2.7.2 Example: estimating π by Monte Carlo integration

MC approximation can be used for many applications, not just statistical ones. Suppose we want to estimate π . We know that the area of a circle with radius r is πr^2 , but it is also equal to the following definite integral:

$$I = \int_{-r}^r \int_{-r}^r \mathbb{I}(x^2 + y^2 \leq r^2) dx dy \quad (2.99)$$

Hence $\pi = I/(r^2)$. Let us approximate this by Monte Carlo integration. Let $f(x, y) = \mathbb{I}(x^2 + y^2 \leq r^2)$ be an indicator function that is 1 for points inside the circle, and 0 outside, and let $p(x)$ and $p(y)$ be uniform distributions on $[-r, r]$, so $p(x) = p(y) = 1/(2r)$. Then

$$I = (2r)(2r) \int \int f(x, y) p(x) p(y) dx dy \quad (2.100)$$

$$= 4r^2 \int \int f(x, y) p(x) p(y) dx dy \quad (2.101)$$

$$\approx 4r^2 \frac{1}{S} \sum_{s=1}^S f(x_s, y_s) \quad (2.102)$$

We find $\hat{\pi} = 3.1416$ with standard error 0.09 (see Section 2.7.3 for a discussion of standard errors). We can plot the points that are accepted/ rejected as in Figure 2.19.

2.7.3 Accuracy of Monte Carlo approximation

The accuracy of an MC approximation increases with sample size. This is illustrated in Figure 2.20. On the top line, we plot a histogram of samples from a Gaussian distribution. On the bottom line, we plot a smoothed version of these samples, created using a kernel density estimate (Section 14.7.2). This smoothed distribution is then evaluated on a dense grid of points

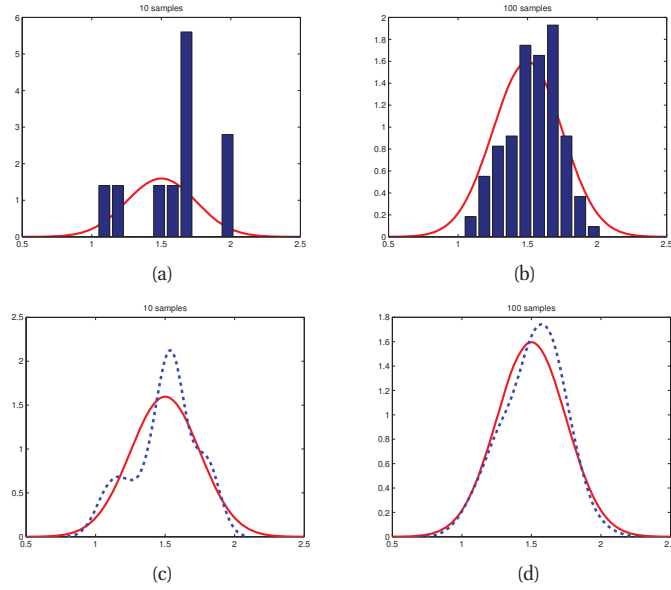


Figure 2.20 10 and 100 samples from a Gaussian distribution, $\mathcal{N}(\mu = 1.5, \sigma^2 = 0.25)$. Solid red line is true pdf. Top line: histogram of samples. Bottom line: kernel density estimate derived from samples in dotted blue, solid red line is true pdf. Based on Figure 4.1 of (Hoff 2009). Figure generated by mcAccuracyDemo.

and plotted. Note that this smoothing is just for the purposes of plotting, it is not used for the Monte Carlo estimate itself.

If we denote the exact mean by $\mu = \mathbb{E}[f(X)]$, and the MC approximation by $\hat{\mu}$, one can show that, with independent samples,

$$(\hat{\mu} - \mu) \rightarrow \mathcal{N}\left(0, \frac{\sigma^2}{S}\right) \quad (2.103)$$

where

$$\sigma^2 = \text{var}[f(X)] = \mathbb{E}[f(X)^2] - \mathbb{E}[f(X)]^2 \quad (2.104)$$

This is a consequence of the central-limit theorem. Of course, σ^2 is unknown in the above expression, but it can also be estimated by MC:

$$\hat{\sigma}^2 = \frac{1}{S} \sum_{s=1}^S (f(x_s) - \hat{\mu})^2 \quad (2.105)$$

Then we have

$$P\left\{\mu - 1.96 \frac{\hat{\sigma}}{\sqrt{S}} \leq \hat{\mu} \leq \mu + 1.96 \frac{\hat{\sigma}}{\sqrt{S}}\right\} \approx 0.95 \quad (2.106)$$

The term $\sqrt{\frac{\hat{\sigma}^2}{S}}$ is called the (numerical or empirical) **standard error**, and is an estimate of our uncertainty about our estimate of μ . (See Section 6.2 for more discussion on standard errors.)

If we want to report an answer which is accurate to within $\pm\epsilon$ with probability at least 95%, we need to use a number of samples S which satisfies $1.96\sqrt{\hat{\sigma}^2/S} \leq \epsilon$. We can approximate the 1.96 factor by 2, yielding $S \geq \frac{4\hat{\sigma}^2}{\epsilon^2}$.

2.8 Information theory

information theory is concerned with representing data in a compact fashion (a task known as **data compression** or **source coding**), as well as with transmitting and storing it in a way that is robust to errors (a task known as **error correction** or **channel coding**). At first, this seems far removed from the concerns of probability theory and machine learning, but in fact there is an intimate connection. To see this, note that compactly representing data requires allocating short codewords to highly probable bit strings, and reserving longer codewords to less probable bit strings. This is similar to the situation in natural language, where common words (such as “a”, “the”, “and”) are generally much shorter than rare words. Also, decoding messages sent over noisy channels requires having a good probability model of the kinds of messages that people tend to send. In both cases, we need a model that can predict which kinds of data are likely and which unlikely, which is also a central problem in machine learning (see (MacKay 2003) for more details on the connection between information theory and machine learning).

Obviously we cannot go into the details of information theory here (see e.g., (Cover and Thomas 2006) if you are interested to learn more). However, we will introduce a few basic concepts that we will need later in the book.

2.8.1 Entropy

The **entropy** of a random variable X with distribution p , denoted by $\mathbb{H}(X)$ or sometimes $\mathbb{H}(p)$, is a measure of its uncertainty. In particular, for a discrete variable with K states, it is defined by

$$\mathbb{H}(X) \triangleq - \sum_{k=1}^K p(X=k) \log_2 p(X=k) \quad (2.107)$$

Usually we use log base 2, in which case the units are called **bits** (short for binary digits). If we use log base e , the units are called **nats**. For example, if $X \in \{1, \dots, 5\}$ with histogram distribution $p = [0.25, 0.25, 0.2, 0.15, 0.15]$, we find $H = 2.2855$. The discrete distribution with maximum entropy is the uniform distribution (see Section 9.2.6 for a proof). Hence for a K -ary random variable, the entropy is maximized if $p(x=k) = 1/K$; in this case, $\mathbb{H}(X) = \log_2 K$. Conversely, the distribution with minimum entropy (which is zero) is any delta-function that puts all its mass on one state. Such a distribution has no uncertainty. In Figure 2.5(b), where we plotted a DNA sequence logo, the height of each bar is defined to be $2 - H$, where H is the entropy of that distribution, and 2 is the maximum possible entropy. Thus a bar of height 0 corresponds to a uniform distribution, whereas a bar of height 2 corresponds to a deterministic distribution.

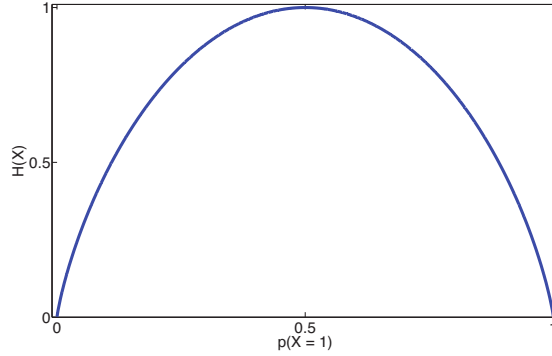


Figure 2.21 Entropy of a Bernoulli random variable as a function of θ . The maximum entropy is $\log_2 2 = 1$. Figure generated by `bernoulliEntropyFig`.

For the special case of binary random variables, $X \in \{0, 1\}$, we can write $p(X = 1) = \theta$ and $p(X = 0) = 1 - \theta$. Hence the entropy becomes

$$\mathbb{H}(X) = -[p(X = 1) \log_2 p(X = 1) + p(X = 0) \log_2 p(X = 0)] \quad (2.108)$$

$$= -[\theta \log_2 \theta + (1 - \theta) \log_2 (1 - \theta)] \quad (2.109)$$

This is called the **binary entropy function**, and is also written $\mathbb{H}(\theta)$. We plot this in Figure 2.21. We see that the maximum value of 1 occurs when the distribution is uniform, $\theta = 0.5$.

2.8.2 KL divergence

One way to measure the dissimilarity of two probability distributions, p and q , is known as the **Kullback-Leibler divergence (KL divergence)** or **relative entropy**. This is defined as follows:

$$\mathbb{KL}(p||q) \triangleq \sum_{k=1}^K p_k \log \frac{p_k}{q_k} \quad (2.110)$$

where the sum gets replaced by an integral for pdfs.¹⁰ We can rewrite this as

$$\mathbb{KL}(p||q) = \sum_k p_k \log p_k - \sum_k p_k \log q_k = -\mathbb{H}(p) + \mathbb{H}(p, q) \quad (2.111)$$

where $\mathbb{H}(p, q)$ is called the **cross entropy**,

$$\mathbb{H}(p, q) \triangleq - \sum_k p_k \log q_k \quad (2.112)$$

One can show (Cover and Thomas 2006) that the cross entropy is the average number of bits needed to encode data coming from a source with distribution p when we use model q to

10. The KL divergence is not a distance, since it is asymmetric. One symmetric version of the KL divergence is the **Jensen-Shannon divergence**, defined as $JS(p_1, p_2) = 0.5\mathbb{KL}(p_1||q) + 0.5\mathbb{KL}(p_2||q)$, where $q = 0.5p_1 + 0.5p_2$.

define our codebook. Hence the “regular” entropy $\mathbb{H}(p) = \mathbb{H}(p, p)$, defined in Section 2.8.1, is the expected number of bits if we use the true model, so the KL divergence is the difference between these. In other words, the KL divergence is the average number of *extra* bits needed to encode the data, due to the fact that we used distribution q to encode the data instead of the true distribution p .

The “extra number of bits” interpretation should make it clear that $\mathbb{KL}(p||q) \geq 0$, and that the KL is only equal to zero iff $q = p$. We now give a proof of this important result.

Theorem 2.8.1. (Information inequality) $\mathbb{KL}(p||q) \geq 0$ with equality iff $p = q$.

Proof. To prove the theorem, we need to use **Jensen’s inequality**. This states that, for any convex function f , we have that

$$f\left(\sum_{i=1}^n \lambda_i \mathbf{x}_i\right) \leq \sum_{i=1}^n \lambda_i f(\mathbf{x}_i) \quad (2.113)$$

where $\lambda_i \geq 0$ and $\sum_{i=1}^n \lambda_i = 1$. This is clearly true for $n = 2$ (by definition of convexity), and can be proved by induction for $n > 2$.

Let us now prove the main theorem, following (Cover and Thomas 2006, p28). Let $A = \{x : p(x) > 0\}$ be the support of $p(x)$. Then

$$-\mathbb{KL}(p||q) = -\sum_{x \in A} p(x) \log \frac{p(x)}{q(x)} = \sum_{x \in A} p(x) \log \frac{q(x)}{p(x)} \quad (2.114)$$

$$\leq \log \sum_{x \in A} p(x) \frac{q(x)}{p(x)} = \log \sum_{x \in A} q(x) \quad (2.115)$$

$$\leq \log \sum_{x \in \mathcal{X}} q(x) = \log 1 = 0 \quad (2.116)$$

where the first inequality follows from Jensen’s. Since $\log(x)$ is a strictly concave function, we have equality in Equation 2.115 iff $p(x) = cq(x)$ for some c . We have equality in Equation 2.116 iff $\sum_{x \in A} q(x) = \sum_{x \in \mathcal{X}} q(x) = 1$, which implies $c = 1$. Hence $\mathbb{KL}(p||q) = 0$ iff $p(x) = q(x)$ for all x . $\square$

One important consequence of this result is that the *discrete distribution with the maximum entropy is the uniform distribution*. More precisely, $\mathbb{H}(X) \leq \log |\mathcal{X}|$, where $|\mathcal{X}|$ is the number of states for X , with equality iff $p(x)$ is uniform. To see this, let $u(x) = 1/|\mathcal{X}|$. Then

$$0 \leq \mathbb{KL}(p||u) = \sum_x p(x) \log \frac{p(x)}{u(x)} \quad (2.117)$$

$$= \sum_x p(x) \log p(x) - \sum_x p(x) \log u(x) = -\mathbb{H}(X) + \log |\mathcal{X}| \quad (2.118)$$

This is a formulation of Laplace’s **principle of insufficient reason**, which argues in favor of using uniform distributions when there are no other reasons to favor one distribution over another. See Section 9.2.6 for a discussion of how to create distributions that satisfy certain constraints, but otherwise are as least-committal as possible. (For example, the Gaussian satisfies first and second moment constraints, but otherwise has maximum entropy.)

2.8.3 Mutual information

Consider two random variables, X and Y . Suppose we want to know how much knowing one variable tells us about the other. We could compute the correlation coefficient, but this is only defined for real-valued random variables, and furthermore, this is a very limited measure of dependence, as we saw in Figure 2.12. A more general approach is to determine how similar the joint distribution $p(X, Y)$ is to the factored distribution $p(X)p(Y)$. This is called the **mutual information** or **MI**, and is defined as follows:

$$\mathbb{I}(X; Y) \triangleq \mathbb{KL}(p(X, Y) || p(X)p(Y)) = \sum_x \sum_y p(x, y) \log \frac{p(x, y)}{p(x)p(y)} \quad (2.119)$$

We have $\mathbb{I}(X; Y) \geq 0$ with equality iff $p(X, Y) = p(X)p(Y)$. That is, the MI is zero iff the variables are independent.

To gain insight into the meaning of MI, it helps to re-express it in terms of joint and conditional entropies. One can show (Exercise 2.12) that the above expression is equivalent to the following:

$$\mathbb{I}(X; Y) = \mathbb{H}(X) - \mathbb{H}(X|Y) = \mathbb{H}(Y) - \mathbb{H}(Y|X) \quad (2.120)$$

where $\mathbb{H}(Y|X)$ is the **conditional entropy**, defined as $\mathbb{H}(Y|X) = \sum_x p(x) \mathbb{H}(Y|X = x)$. Thus we can interpret the MI between X and Y as the reduction in uncertainty about X after observing Y , or, by symmetry, the reduction in uncertainty about Y after observing X . We will encounter several applications of MI later in the book. See also Exercises 2.13 and 2.14 for the connection between MI and correlation coefficients.

A quantity which is closely related to MI is the **pointwise mutual information** or PMI. For two events (not random variables) x and y , this is defined as

$$\text{PMI}(x, y) \triangleq \log \frac{p(x, y)}{p(x)p(y)} = \log \frac{p(x|y)}{p(x)} = \log \frac{p(y|x)}{p(y)} \quad (2.121)$$

This measures the discrepancy between these events occurring together compared to what would be expected by chance. Clearly the MI of X and Y is just the expected value of the PMI. Interestingly, we can rewrite the PMI as follows:

$$\text{PMI}(x, y) = \log \frac{p(x|y)}{p(x)} = \log \frac{p(y|x)}{p(y)} \quad (2.122)$$

This is the amount we learn from updating the prior $p(x)$ into the posterior $p(x|y)$, or equivalently, updating the prior $p(y)$ into the posterior $p(y|x)$.

2.8.3.1 Mutual information for continuous random variables *

The above formula for MI is defined for discrete random variables. For continuous random variables, it is common to first **discretize** or **quantize** them, by dividing the ranges of each variable into bins, and computing how many values fall in each histogram bin (Scott 1979). We can then easily compute the MI using the formula above (see `mutualInfoAllPairsMixed` for some code, and `miMixedDemo` for a demo).

Unfortunately, the number of bins used, and the location of the bin boundaries, can have a significant effect on the results. One way around this is to try to estimate the MI directly,

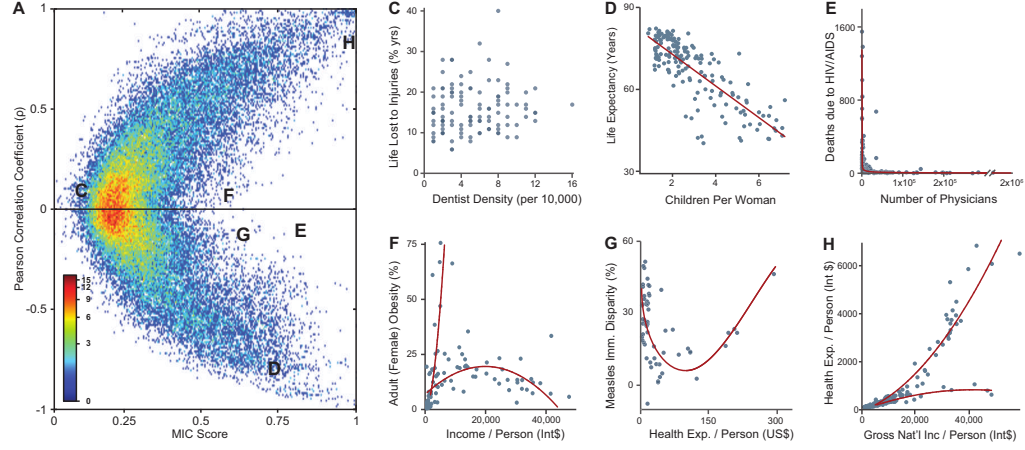


Figure 2.22 Left: Correlation coefficient vs maximal information criterion (MIC) for all pairwise relationships in the WHO data. Right: scatter plots of certain pairs of variables. The red lines are non-parametric smoothing regressions (Section 15.4.6) fit separately to each trend. Source: Figure 4 of (Reshed et al. 2011). Used with kind permission of David Reshef and the American Association for the Advancement of Science.

without first performing density estimation (Learned-Miller 2004). Another approach is to try many different bin sizes and locations, and to compute the maximum MI achieved. This statistic, appropriately normalized, is known as the **maximal information coefficient** (MIC) (Reshed et al. 2011). More precisely, define

$$m(x, y) = \frac{\max_{G \in \mathcal{G}(x, y)} \mathbb{I}(X(G); Y(G))}{\log \min(x, y)} \quad (2.123)$$

where $\mathcal{G}(x, y)$ is the set of 2d grids of size $x \times y$, and $X(G)$, $Y(G)$ represents a discretization of the variables onto this grid. (The maximization over bin locations can be performed efficiently using dynamic programming (Reshed et al. 2011).) Now define the MIC as

$$\text{MIC} \triangleq \max_{x, y: xy < B} m(x, y) \quad (2.124)$$

where B is some sample-size dependent bound on the number of bins we can use and still reliably estimate the distribution ((Reshed et al. 2011) suggest $B = N^{0.6}$). It can be shown that the MIC lies in the range $[0, 1]$, where 0 represents no relationship between the variables, and 1 represents a noise-free relationship of any form, not just linear.

Figure 2.22 gives an example of this statistic in action. The data consists of 357 variables measuring a variety of social, economic, health and political indicators, collected by the World Health Organization (WHO). On the left of the figure, we see the correlation coefficient (CC) plotted against the MIC for all 63,566 variable pairs. On the right of the figure, we see scatter plots for particular pairs of variables, which we now discuss:

- The point marked C has a low CC and a low MIC. The corresponding scatter plot makes it

clear that there is no relationship between these two variables (percentage of lives lost to injury and density of dentists in the population).

- The points marked D and H have high CC (in absolute value) and high MIC, because they represent nearly linear relationships.
- The points marked E, F, and G have low CC but high MIC. This is because they correspond to non-linear (and sometimes, as in the case of E and F, non-functional, i.e., one-to-many) relationships between the variables.

In summary, we see that statistics (such as MIC) based on mutual information can be used to discover interesting relationships between variables in a way that simpler measures, such as correlation coefficients, cannot. For this reason, the MIC has been called “a correlation for the 21st century” (Speed 2011).

Exercises

Exercise 2.1 Probabilities are sensitive to the form of the question that was used to generate the answer

(Source: Minka.) My neighbor has two children. Assuming that the gender of a child is like a coin flip, it is most likely, a priori, that my neighbor has one boy and one girl, with probability 1/2. The other possibilities—two boys or two girls—have probabilities 1/4 and 1/4.

- Suppose I ask him whether he has any boys, and he says yes. What is the probability that one child is a girl?
- Suppose instead that I happen to see one of his children run by, and it is a boy. What is the probability that the other child is a girl?

Exercise 2.2 Legal reasoning

(Source: Peter Lee.) Suppose a crime has been committed. Blood is found at the scene for which there is no innocent explanation. It is of a type which is present in 1% of the population.

- The prosecutor claims: “There is a 1% chance that the defendant would have the crime blood type if he were innocent. Thus there is a 99% chance that he guilty”. This is known as the **prosecutor’s fallacy**. What is wrong with this argument?
- The defender claims: “The crime occurred in a city of 800,000 people. The blood type would be found in approximately 8000 people. The evidence has provided a probability of just 1 in 8000 that the defendant is guilty, and thus has no relevance.” This is known as the **defender’s fallacy**. What is wrong with this argument?

Exercise 2.3 Variance of a sum

Show that the variance of a sum is $\text{var}[X + Y] = \text{var}[X] + \text{var}[Y] + 2\text{cov}[X, Y]$, where $\text{cov}[X, Y]$ is the covariance between X and Y

Exercise 2.4 Bayes rule for medical diagnosis

(Source: Koller.) After your yearly checkup, the doctor has bad news and good news. The bad news is that you tested positive for a serious disease, and that the test is 99% accurate (i.e., the probability of testing positive given that you have the disease is 0.99, as is the probability of testing negative given that you don’t have the disease). The good news is that this is a rare disease, striking only one in 10,000 people. What are the chances that you actually have the disease? (Show your calculations as well as giving the final result.)

Exercise 2.5 The Monty Hall problem

(Source: Mackay.) On a game show, a contestant is told the rules as follows:

There are three doors, labelled 1, 2, 3. A single prize has been hidden behind one of them. You get to select one door. Initially your chosen door will *not* be opened. Instead, the gameshow host will open one of the other two doors, and *he will do so in such a way as not to reveal the prize*. For example, if you first choose door 1, he will then open one of doors 2 and 3, and it is guaranteed that he will choose which one to open so that the prize will not be revealed.

At this point, you will be given a fresh choice of door: you can either stick with your first choice, or you can switch to the other closed door. All the doors will then be opened and you will receive whatever is behind your final choice of door.

Imagine that the contestant chooses door 1 first; then the gameshow host opens door 3, revealing nothing behind the door, as promised. Should the contestant (a) stick with door 1, or (b) switch to door 2, or (c) does it make no difference? You may assume that initially, the prize is equally likely to be behind any of the 3 doors. Hint: use Bayes rule.

Exercise 2.6 Conditional independence

(Source: Koller.)

- a. Let $H \in \{1, \dots, K\}$ be a discrete random variable, and let e_1 and e_2 be the observed values of two other random variables E_1 and E_2 . Suppose we wish to calculate the vector

$$\vec{P}(H|e_1, e_2) = (P(H = 1|e_1, e_2), \dots, P(H = K|e_1, e_2))$$

Which of the following sets of numbers are sufficient for the calculation?

- i. $P(e_1, e_2), P(H), P(e_1|H), P(e_2|H)$
 - ii. $P(e_1, e_2), P(H), P(e_1, e_2|H)$
 - iii. $P(e_1|H), P(e_2|H), P(H)$
- b. Now suppose we now assume $E_1 \perp E_2|H$ (i.e., E_1 and E_2 are conditionally independent given H). Which of the above 3 sets are sufficient now?

Show your calculations as well as giving the final result. Hint: use Bayes rule.

Exercise 2.7 Pairwise independence does not imply mutual independence

We say that two random variables are pairwise independent if

$$p(X_2|X_1) = p(X_2) \tag{2.125}$$

and hence

$$p(X_2, X_1) = p(X_1)p(X_2|X_1) = p(X_1)p(X_2) \tag{2.126}$$

We say that n random variables are mutually independent if

$$p(X_i|X_S) = p(X_i) \quad \forall S \subseteq \{1, \dots, n\} \setminus \{i\} \tag{2.127}$$

and hence

$$p(X_{1:n}) = \prod_{i=1}^n p(X_i) \tag{2.128}$$

Show that pairwise independence between all pairs of variables does not necessarily imply mutual independence. It suffices to give a counter example.

Exercise 2.8 Conditional independence iff joint factorizes

In the text we said $X \perp Y|Z$ iff

$$p(x, y|z) = p(x|z)p(y|z) \quad (2.129)$$

for all x, y, z such that $p(z) > 0$. Now prove the following alternative definition: $X \perp Y|Z$ iff there exist function g and h such that

$$p(x, y|z) = g(x, z)h(y, z) \quad (2.130)$$

for all x, y, z such that $p(z) > 0$.

Exercise 2.9 Conditional independence

(Source: Koller.) Are the following properties true? Prove or disprove. Note that we are not restricting attention to distributions that can be represented by a graphical model.

- True or false? $(X \perp W|Z, Y) \wedge (X \perp Y|Z) \Rightarrow (X \perp Y, W|Z)$
- True or false? $(X \perp Y|Z) \wedge (X \perp Y|W) \Rightarrow (X \perp Y|Z, W)$

Exercise 2.10 Deriving the inverse gamma density

Let $X \sim \text{Ga}(a, b)$, i.e.

$$\text{Ga}(x|a, b) = \frac{b^a}{\Gamma(a)} x^{a-1} e^{-xb} \quad (2.131)$$

Let $Y = 1/X$. Show that $Y \sim \text{IG}(a, b)$, i.e.,

$$\text{IG}(x|\text{shape} = a, \text{scale} = b) = \frac{b^a}{\Gamma(a)} x^{-(a+1)} e^{-b/x} \quad (2.132)$$

Hint: use the change of variables formula.

Exercise 2.11 Normalization constant for a 1D Gaussian

The normalization constant for a zero-mean Gaussian is given by

$$Z = \int_a^b \exp\left(-\frac{x^2}{2\sigma^2}\right) dx \quad (2.133)$$

where $a = -\infty$ and $b = \infty$. To compute this, consider its square

$$Z^2 = \int_a^b \int_a^b \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) dx dy \quad (2.134)$$

Let us change variables from cartesian (x, y) to polar (r, θ) using $x = r \cos \theta$ and $y = r \sin \theta$. Since $dx dy = r dr d\theta$, and $\cos^2 \theta + \sin^2 \theta = 1$, we have

$$Z^2 = \int_0^{2\pi} \int_0^\infty r \exp\left(-\frac{r^2}{2\sigma^2}\right) dr d\theta \quad (2.135)$$

Evaluate this integral and hence show $Z = \sigma\sqrt{(2\pi)}$. Hint 1: separate the integral into a product of two terms, the first of which (involving $d\theta$) is constant, so is easy. Hint 2: if $u = e^{-r^2/2\sigma^2}$ then $du/dr = -\frac{1}{\sigma^2} r e^{-r^2/2\sigma^2}$, so the second integral is also easy (since $\int u'(r) dr = u(r)$).

Exercise 2.12 Expressing mutual information in terms of entropies

Show that

$$I(X, Y) = H(X) - H(X|Y) = H(Y) - H(Y|X) \quad (2.136)$$

Exercise 2.13 Mutual information for correlated normals

(Source: (Cover and Thomas 1991, Q9.3).) Find the mutual information $I(X_1, X_2)$ where $\mathbf{X}$ has a bivariate normal distribution:

$$\begin{pmatrix} X_1 \\ X_2 \end{pmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{pmatrix} \sigma^2 & \rho\sigma^2 \\ \rho\sigma^2 & \sigma^2 \end{pmatrix}\right) \quad (2.137)$$

Evaluate $I(X_1, X_2)$ for $\rho = 1$, $\rho = 0$ and $\rho = -1$ and comment. Hint: The (differential) entropy of a d -dimensional Gaussian is

$$h(\mathbf{X}) = \frac{1}{2} \log_2 \left[(2\pi e)^d \det \Sigma \right] \quad (2.138)$$

In the 1d case, this becomes

$$h(X) = \frac{1}{2} \log_2 [2\pi e \sigma^2] \quad (2.139)$$

Hint: $\log(0) = \infty$.

Exercise 2.14 A measure of correlation (normalized mutual information)

(Source: (Cover and Thomas 1991, Q2.20).) Let X and Y be discrete random variables which are identically distributed (so $H(X) = H(Y)$) but not necessarily independent. Define

$$r = 1 - \frac{H(Y|X)}{H(X)} \quad (2.140)$$

- Show $r = \frac{I(X, Y)}{H(X)}$
- Show $0 \leq r \leq 1$
- When is $r = 0$?
- When is $r = 1$?

Exercise 2.15 MLE minimizes KL divergence to the empirical distribution

Let $p_{\text{emp}}(x)$ be the empirical distribution, and let $q(x|\boldsymbol{\theta})$ be some model. Show that $\arg\min_q \mathbb{KL}(p_{\text{emp}}||q)$ is obtained by $q(x) = q(x; \hat{\boldsymbol{\theta}})$, where $\hat{\boldsymbol{\theta}}$ is the MLE. Hint: use non-negativity of the KL divergence.

Exercise 2.16 Mean, mode, variance for the beta distribution

Suppose $\theta \sim \text{Beta}(a, b)$. Derive the mean, mode and variance.

Exercise 2.17 Expected value of the minimum

Suppose X, Y are two points sampled independently and uniformly at random from the interval $[0, 1]$. What is the expected location of the left most point?

3

Generative models for discrete data

3.1 Introduction

In Section 2.2.3.2, we discussed how to classify a feature vector $\mathbf{x}$ by applying Bayes rule to a generative classifier of the form

$$p(y = c|\mathbf{x}, \boldsymbol{\theta}) \propto p(\mathbf{x}|y = c, \boldsymbol{\theta})p(y = c|\boldsymbol{\theta}) \quad (3.1)$$

The key to using such models is specifying a suitable form for the class-conditional density $p(\mathbf{x}|y = c, \boldsymbol{\theta})$, which defines what kind of data we expect to see in each class. In this chapter, we focus on the case where the observed data are discrete symbols. We also discuss how to infer the unknown parameters $\boldsymbol{\theta}$ of such models.

3.2 Bayesian concept learning

Consider how a child learns to understand the meaning of a word, such as “dog”. Presumably the child’s parents point out positive examples of this concept, saying such things as, “look at the cute dog!”, or “mind the doggy”, etc. However, it is very unlikely that they provide negative examples, by saying “look at that non-dog”. Certainly, negative examples may be obtained during an active learning process — the child says “look at the dog” and the parent says “that’s a cat, dear, not a dog” — but psychological research has shown that people can learn concepts from positive examples alone (Xu and Tenenbaum 2007).

We can think of learning the meaning of a word as equivalent to **concept learning**, which in turn is equivalent to binary classification. To see this, define $f(x) = 1$ if x is an example of the concept C , and $f(x) = 0$ otherwise. Then the goal is to learn the indicator function f , which just defines which elements are in the set C . By allowing for uncertainty about the definition of f , or equivalently the elements of C , we can emulate **fuzzy set theory**, but using standard probability calculus. Note that standard binary classification techniques require positive and negative examples. By contrast, we will devise a way to learn from positive examples alone.

For pedagogical purposes, we will consider a very simple example of concept learning called the **number game**, based on part of Josh Tenenbaum’s PhD thesis (Tenenbaum 1999). The game proceeds as follows. I choose some simple arithmetical concept C , such as “prime number” or “a number between 1 and 10”. I then give you a series of randomly chosen positive examples $\mathcal{D} = \{x_1, \dots, x_N\}$ drawn from C , and ask you whether some new test case $\tilde{x}$ belongs to C , i.e., I ask you to classify $\tilde{x}$.

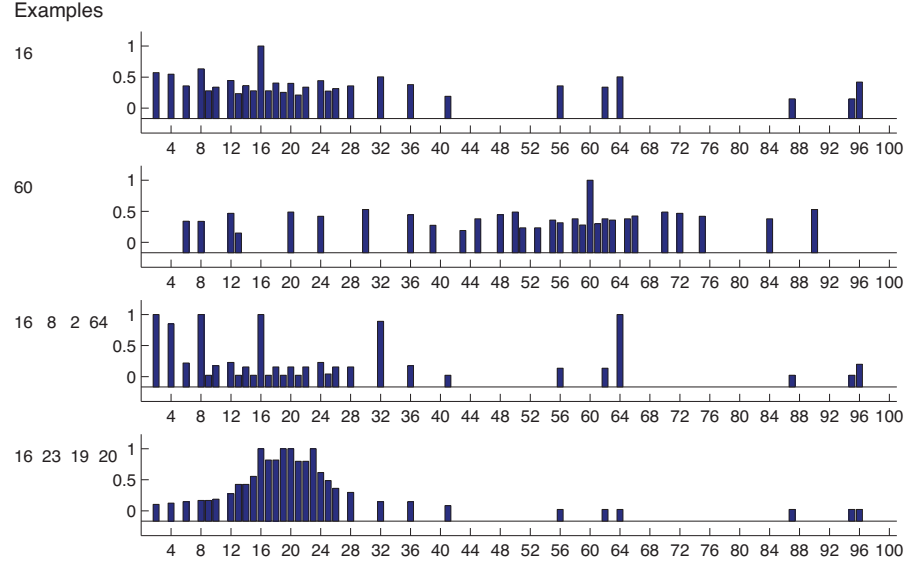


Figure 3.1 Empirical predictive distribution averaged over 8 humans in the number game. First two rows: after seeing $\mathcal{D} = \{16\}$ and $\mathcal{D} = \{60\}$. This illustrates diffuse similarity. Third row: after seeing $\mathcal{D} = \{16, 8, 2, 64\}$. This illustrates rule-like behavior (powers of 2). Bottom row: after seeing $\mathcal{D} = \{16, 23, 19, 20\}$. This illustrates focussed similarity (numbers near 20). Source: Figure 5.5 of (Tenenbaum 1999). Used with kind permission of Josh Tenenbaum.

Suppose, for simplicity, that all numbers are integers between 1 and 100. Now suppose I tell you “16” is a positive example of the concept. What other numbers do you think are positive? 17? 6? 32? 99? It’s hard to tell with only one example, so your predictions will be quite vague. Presumably numbers that are similar in some sense to 16 are more likely. But similar in what way? 17 is similar, because it is “close by”, 6 is similar because it has a digit in common, 32 is similar because it is also even and a power of 2, but 99 does not seem similar. Thus some numbers are more likely than others. We can represent this as a probability distribution, $p(\tilde{x}|\mathcal{D})$, which is the probability that $\tilde{x} \in C$ given the data $\mathcal{D}$ for any $\tilde{x} \in \{1, \dots, 100\}$. This is called the **posterior predictive distribution**. Figure 3.1(top) shows the predictive distribution of people derived from a lab experiment. We see that people predict numbers that are similar to 16, under a variety of kinds of similarity.

Now suppose I tell you that 8, 2 and 64 are *also* positive examples. Now you may guess that the hidden concept is “powers of two”. This is an example of **induction**. Given this hypothesis, the predictive distribution is quite specific, and puts most of its mass on powers of 2, as shown in Figure 3.1(third row). If instead I tell you the data is $\mathcal{D} = \{16, 23, 19, 20\}$, you will get a different kind of **generalization gradient**, as shown in Figure 3.1(bottom).

How can we explain this behavior and emulate it in a machine? The classic approach to induction is to suppose we have a **hypothesis space** of concepts, $\mathcal{H}$, such as: odd numbers, even numbers, all numbers between 1 and 100, powers of two, all numbers ending in j (for

$0 \leq j \leq 9$), etc. The subset of $\mathcal{H}$ that is consistent with the data D is called the **version space**. As we see more examples, the version space shrinks and we become increasingly certain about the concept (Mitchell 1997).

However, the version space is not the whole story. After seeing $\mathcal{D} = \{16\}$, there are many consistent rules; how do you combine them to predict if $\tilde{x} \in C$? Also, after seeing $\mathcal{D} = \{16, 8, 2, 64\}$, why did you choose the rule “powers of two” and not, say, “all even numbers”, or “powers of two except for 32”, both of which are equally consistent with the evidence? We will now provide a Bayesian explanation for this.

3.2.1 Likelihood

We must explain why we chose $h_{\text{two}} \triangleq$ “powers of two”, and not, say, $h_{\text{even}} \triangleq$ “even numbers” after seeing $\mathcal{D} = \{16, 8, 2, 64\}$, given that both hypotheses are consistent with the evidence. The key intuition is that we want to avoid **suspicious coincidences**. If the true concept was even numbers, how come we only saw numbers that happened to be powers of two?

To formalize this, let us assume that examples are sampled uniformly at random from the **extension** of a concept. (The extension of a concept is just the set of numbers that belong to it, e.g., the extension of h_{even} is $\{2, 4, 6, \dots, 98, 100\}$; the extension of “numbers ending in 9” is $\{9, 19, \dots, 99\}$.) Tenenbaum calls this the **strong sampling assumption**. Given this assumption, the probability of independently sampling N items (with replacement) from h is given by

$$p(\mathcal{D}|h) = \left[\frac{1}{\text{size}(h)} \right]^N = \left[\frac{1}{|h|} \right]^N \quad (3.2)$$

This crucial equation embodies what Tenenbaum calls the **size principle**, which means the model favors the simplest (smallest) hypothesis consistent with the data. This is more commonly known as **Occam’s razor**.¹

To see how it works, let $\mathcal{D} = \{16\}$. Then $p(\mathcal{D}|h_{\text{two}}) = 1/6$, since there are only 6 powers of two less than 100, but $p(\mathcal{D}|h_{\text{even}}) = 1/50$, since there are 50 even numbers. So the likelihood that $h = h_{\text{two}}$ is higher than if $h = h_{\text{even}}$. After 4 examples, the likelihood of h_{two} is $(1/6)^4 = 7.7 \times 10^{-4}$, whereas the likelihood of h_{even} is $(1/50)^4 = 1.6 \times 10^{-7}$. This is a **likelihood ratio** of almost 5000:1 in favor of h_{two} . This quantifies our earlier intuition that $D = \{16, 8, 2, 64\}$ would be a very suspicious coincidence if generated by h_{even} .

3.2.2 Prior

Suppose $D = \{16, 8, 2, 64\}$. Given this data, the concept $h' =$ “powers of two except 32” is more likely than $h =$ “powers of two”, since h' does not need to explain the coincidence that 32 is missing from the set of examples.

However, the hypothesis $h' =$ “powers of two except 32” seems “conceptually unnatural”. We can capture such intuition by assigning low prior probability to unnatural concepts. Of course, your prior might be different than mine. This **subjective** aspect of Bayesian reasoning is a source of much controversy, since it means, for example, that a child and a math professor

1. William of Occam (also spelt Ockham) was an English monk and philosopher, 1288–1348.

will reach different answers. In fact, they presumably not only have different priors, but also different hypothesis spaces. However, we can finesse that by defining the hypothesis space of the child and the math professor to be the same, and then setting the child's prior weight to be zero on certain "advanced" concepts. Thus there is no sharp distinction between the prior and the hypothesis space.

Although the subjectivity of the prior is controversial, it is actually quite useful. If you are told the numbers are from some arithmetic rule, then given 1200, 1500, 900 and 1400, you may think 400 is likely but 1183 is unlikely. But if you are told that the numbers are examples of healthy cholesterol levels, you would probably think 400 is unlikely and 1183 is likely. Thus we see that the prior is the mechanism by which background knowledge can be brought to bear on a problem. Without this, rapid learning (i.e., from small samples sizes) is impossible.

So, what prior should we use? For illustration purposes, let us use a simple prior which puts uniform probability on 30 simple arithmetical concepts, such as "even numbers", "odd numbers", "prime numbers", "numbers ending in 9", etc. To make things more interesting, we make the concepts even and odd more likely apriori. We also include two "unnatural" concepts, namely "powers of 2, plus 37" and "powers of 2, except 32", but give them low prior weight. See Figure 3.2(a) for a plot of this prior. We will consider a slightly more sophisticated prior later on.

3.2.3 Posterior

The posterior is simply the likelihood times the prior, normalized. In this context we have

$$p(h|\mathcal{D}) = \frac{p(\mathcal{D}|h)p(h)}{\sum_{h' \in \mathcal{H}} p(\mathcal{D}, h')} = \frac{p(h)\mathbb{I}(\mathcal{D} \in h)/|h|^N}{\sum_{h' \in \mathcal{H}} p(h')\mathbb{I}(\mathcal{D} \in h')/|h'|^N} \quad (3.3)$$

where $\mathbb{I}(\mathcal{D} \in h)$ is 1 **iff** (iff and only if) all the data are in the extension of the hypothesis h . Figure 3.2 plots the prior, likelihood and posterior after seeing $\mathcal{D} = \{16\}$. We see that the posterior is a combination of prior and likelihood. In the case of most of the concepts, the prior is uniform, so the posterior is proportional to the likelihood. However, the "unnatural" concepts of "powers of 2, plus 37" and "powers of 2, except 32" have low posterior support, despite having high likelihood, due to the low prior. Conversely, the concept of odd numbers has low posterior support, despite having a high prior, due to the low likelihood.

Figure 3.3 plots the prior, likelihood and posterior after seeing $\mathcal{D} = \{16, 8, 2, 64\}$. Now the likelihood is much more peaked on the powers of two concept, so this dominates the posterior. Essentially the learner has an **aha** moment, and figures out the true concept. (Here we see the need for the low prior on the unnatural concepts, otherwise we would have overfit the data and picked "powers of 2, except for 32".)

In general, when we have enough data, the posterior $p(h|\mathcal{D})$ becomes peaked on a single concept, namely the MAP estimate, i.e.,

$$p(h|\mathcal{D}) \rightarrow \delta_{\hat{h}_{MAP}}(h) \quad (3.4)$$

where $\hat{h}^{MAP} = \operatorname{argmax}_h p(h|\mathcal{D})$ is the posterior mode, and where δ is the **Dirac measure** defined by

$$\delta_x(A) = \begin{cases} 1 & \text{if } x \in A \\ 0 & \text{if } x \notin A \end{cases} \quad (3.5)$$

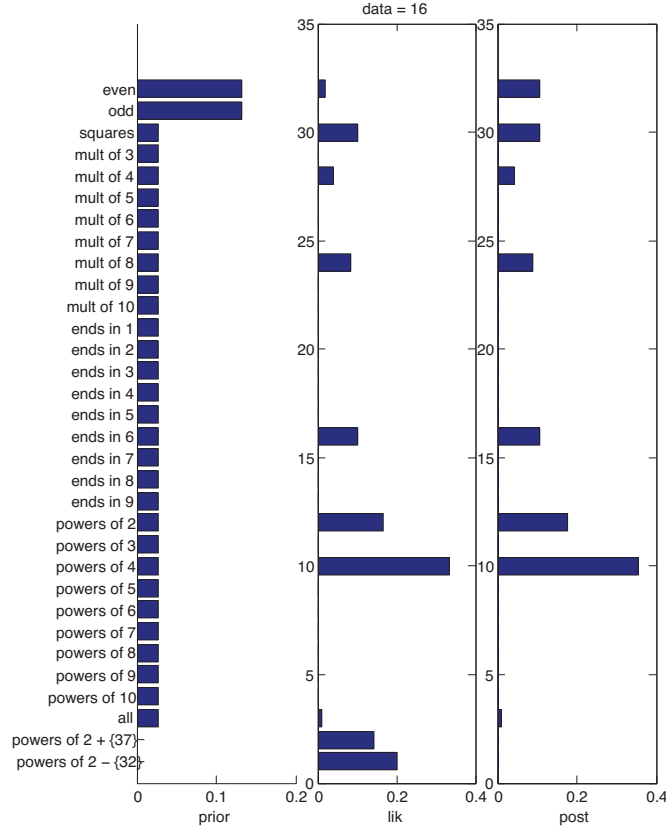


Figure 3.2 Prior, likelihood and posterior for $\mathcal{D} = \{16\}$. Based on (Tenenbaum 1999). Figure generated by `numbersGame`.

Note that the MAP estimate can be written as

$$\hat{h}^{MAP} = \operatorname{argmax}_h p(\mathcal{D}|h)p(h) = \operatorname{argmax}_h [\log p(\mathcal{D}|h) + \log p(h)] \quad (3.6)$$

Since the likelihood term depends exponentially on N , and the prior stays constant, as we get more and more data, the MAP estimate converges towards the **maximum likelihood estimate** or **MLE**:

$$\hat{h}^{mle} \triangleq \operatorname{argmax}_h p(\mathcal{D}|h) = \operatorname{argmax}_h \log p(\mathcal{D}|h) \quad (3.7)$$

In other words, if we have enough data, we see that the **data overwhelms the prior**. In this

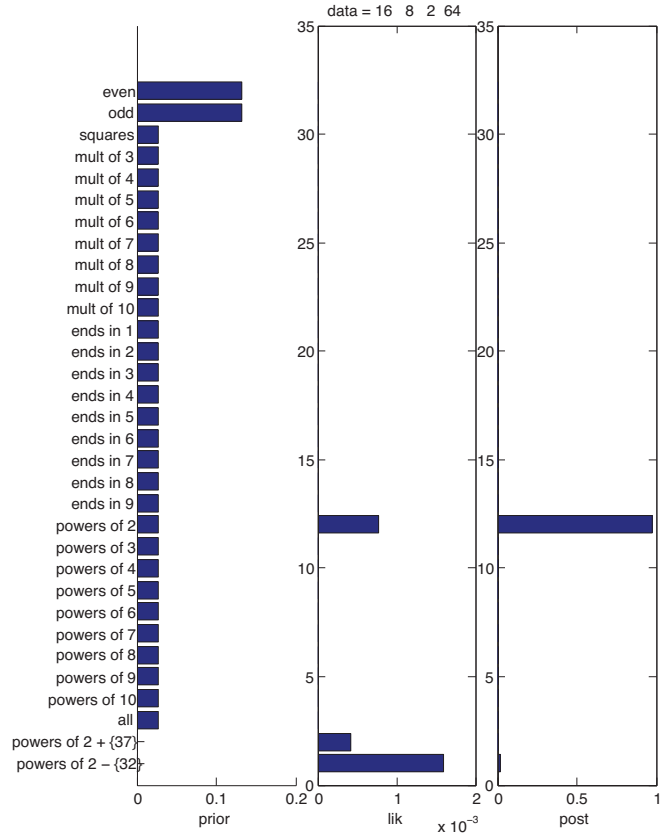


Figure 3.3 Prior, likelihood and posterior for $\mathcal{D} = \{16, 8, 2, 64\}$. Based on (Tenenbaum 1999). Figure generated by numbersGame.

case, the MAP estimate converges towards the MLE.

If the true hypothesis is in the hypothesis space, then the MAP/ ML estimate will converge upon this hypothesis. Thus we say that Bayesian inference (and ML estimation) are consistent estimators (see Section 6.4.1 for details). We also say that the hypothesis space is **identifiable in the limit**, meaning we can recover the truth in the limit of infinite data. If our hypothesis class is not rich enough to represent the “truth” (which will usually be the case), we will converge on the hypothesis that is as close as possible to the truth. However, formalizing this notion of “closeness” is beyond the scope of this chapter.

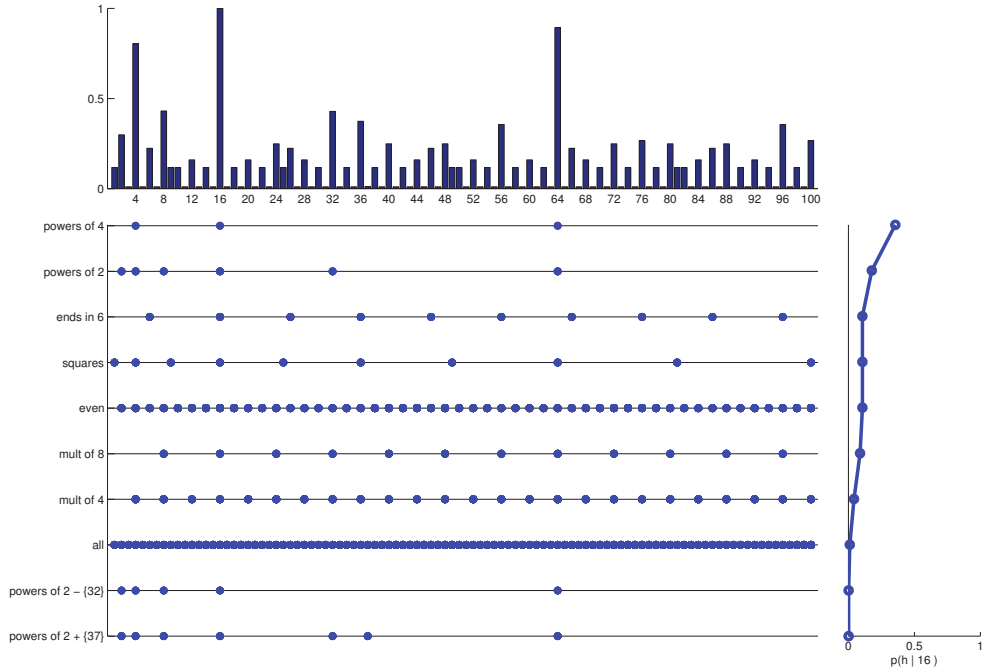


Figure 3.4 Posterior over hypotheses and the corresponding predictive distribution after seeing one example, $\mathcal{D} = \{16\}$. A dot means this number is consistent with this hypothesis. The graph $p(h|\mathcal{D})$ on the right is the weight given to hypothesis h . By taking a weighed sum of dots, we get $p(\tilde{x} \in C|\mathcal{D})$ (top). Based on Figure 2.9 of (Tenenbaum 1999). Figure generated by `numbersGame`.

3.2.4 Posterior predictive distribution

The posterior is our internal **belief state** about the world. The way to test if our beliefs are justified is to use them to predict objectively observable quantities (this is the basis of the scientific method). Specifically, the posterior predictive distribution in this context is given by

$$p(\tilde{x} \in C|\mathcal{D}) = \sum_h p(y = 1|\tilde{x}, h)p(h|\mathcal{D}) \quad (3.8)$$

This is just a weighted average of the predictions of each individual hypothesis and is called **Bayes model averaging** (Hoeting et al. 1999). This is illustrated in Figure 3.4. The dots at the bottom show the predictions from each hypothesis; the vertical curve on the right shows the weight associated with each hypothesis. If we multiply each row by its weight and add up, we get the distribution at the top.

When we have a small and/or ambiguous dataset, the posterior $p(h|\mathcal{D})$ is vague, which induces a broad predictive distribution. However, once we have “figured things out”, the posterior becomes a delta function centered at the MAP estimate. In this case, the predictive distribution

becomes

$$p(\tilde{x} \in C|\mathcal{D}) = \sum_h p(\tilde{x}|h)\delta_{\hat{h}}(h) = p(\tilde{x}|\hat{h}) \quad (3.9)$$

This is called a **plug-in approximation** to the predictive density and is very widely used, due to its simplicity. However, in general, this under-represents our uncertainty, and our predictions will not be as “smooth” as when using BMA. We will see more examples of this later in the book.

Although MAP learning is simple, it cannot explain the gradual shift from similarity-based reasoning (with uncertain posteriors) to rule-based reasoning (with certain posteriors). For example, suppose we observe $\mathcal{D} = \{16\}$. If we use the simple prior above, the minimal consistent hypothesis is “all powers of 4”, so only 4 and 16 get a non-zero probability of being predicted. This is of course an example of overfitting. Given $\mathcal{D} = \{16, 8, 2, 64\}$, the MAP hypothesis is “all powers of two”. Thus the plug-in predictive distribution gets broader (or stays the same) as we see more data: it starts narrow, but is forced to broaden as it seems more data. In contrast, in the Bayesian approach, we start broad and then narrow down as we learn more, which makes more intuitive sense. In particular, given $\mathcal{D} = \{16\}$, there are many hypotheses with non-negligible posterior support, so the predictive distribution is broad. However, when we see $\mathcal{D} = \{16, 8, 2, 64\}$, the posterior concentrates its mass on one hypothesis, so the predictive distribution becomes narrower. So the predictions made by a plug-in approach and a Bayesian approach are quite different in the small sample regime, although they converge to the same answer as we see more data.

3.2.5 A more complex prior

To model human behavior, Tenenbaum used a slightly more sophisticated prior which was derived by analysing some experimental data of how people measure similarity between numbers; see (Tenenbaum 1999, p208) for details. The result is a set of arithmetical concepts similar to those mentioned above, plus all intervals between n and m for $1 \leq n, m \leq 100$. (Note that these hypotheses are not mutually exclusive.) Thus the prior is a **mixture** of two priors, one over arithmetical rules, and one over intervals:

$$p(h) = \pi_0 p_{\text{rules}}(h) + (1 - \pi_0) p_{\text{interval}}(h) \quad (3.10)$$

The only free parameter in the model is the relative weight, π_0 , given to these two parts of the prior. The results are not very sensitive to this value, so long as $\pi_0 > 0.5$, reflecting the fact that people are more likely to think of concepts defined by rules. The predictive distribution of the model, using this larger hypothesis space, is shown in Figure 3.5. It is strikingly similar to the human predictive distribution, shown in Figure 3.1, even though it was not fit to human data (modulo the choice of hypothesis space).

3.3 The beta-binomial model

The number game involved inferring a distribution over a discrete variable drawn from a finite hypothesis space, $h \in \mathcal{H}$, given a series of discrete observations. This made the computations particularly simple: we just needed to sum, multiply and divide. However, in many applications, the unknown parameters are continuous, so the hypothesis space is (some subset) of $\mathbb{R}^K$, where

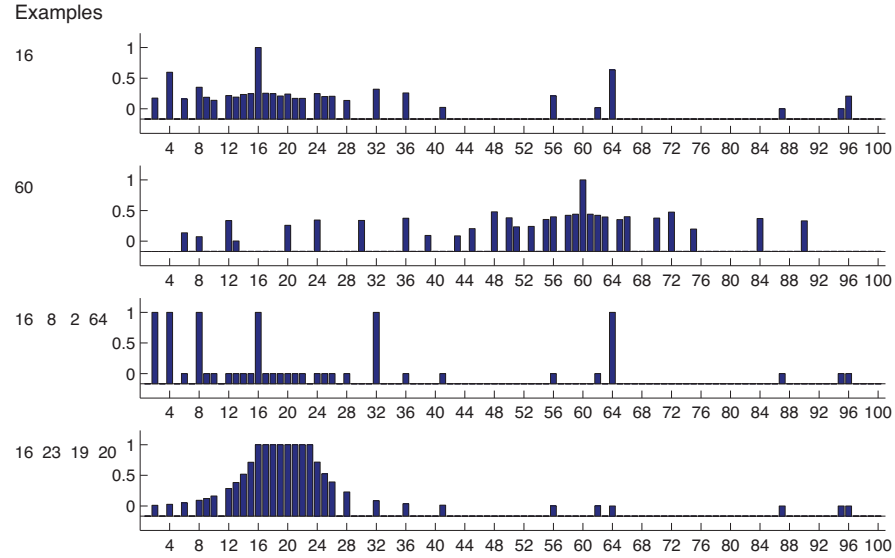


Figure 3.5 Predictive distributions for the model using the full hypothesis space. Compare to Figure 3.1. The predictions of the Bayesian model are only plotted for those values of $\tilde{x}$ for which human data is available; this is why the top line looks sparser than Figure 3.4. Source: Figure 5.6 of (Tenenbaum 1999). Used with kind permission of Josh Tenenbaum.

K is the number of parameters. This complicates the mathematics, since we have to replace sums with integrals. However, the basic ideas are the same.

We will illustrate this by considering the problem of inferring the probability that a coin shows up heads, given a series of observed coin tosses. Although this might seem trivial, it turns out that this model forms the basis of many of the methods we will consider later in this book, including naive Bayes classifiers, Markov models, etc. It is historically important, since it was the example which was analyzed in Bayes’ original paper of 1763. (Bayes’ analysis was subsequently generalized by Pierre-Simon Laplace, creating what we now call “Bayes rule” — see (Stigler 1986) for further historical details.)

We will follow our now-familiar recipe of specifying the likelihood and prior, and deriving the posterior and posterior predictive.

3.3.1 Likelihood

Suppose $X_i \sim \text{Ber}(\theta)$, where $X_i = 1$ represents “heads”, $X_i = 0$ represents “tails”, and $\theta \in [0, 1]$ is the rate parameter (probability of heads). If the data are iid, the likelihood has the form

$$p(\mathcal{D}|\theta) = \theta^{N_1} (1 - \theta)^{N_0} \quad (3.11)$$

where we have $N_1 = \sum_{i=1}^N \mathbb{I}(x_i = 1)$ heads and $N_0 = \sum_{i=1}^N \mathbb{I}(x_i = 0)$ tails. These two counts are called the **sufficient statistics** of the data, since this is all we need to know about $\mathcal{D}$ to infer θ . (An alternative set of sufficient statistics are N_1 and $N = N_0 + N_1$.)

More formally, we say $\mathbf{s}(\mathcal{D})$ is a sufficient statistic for data $\mathcal{D}$ if $p(\theta|\mathcal{D}) = p(\theta|\mathbf{s}(\mathcal{D}))$. If we use a uniform prior, this is equivalent to saying $p(\mathcal{D}|\theta) \propto p(\mathbf{s}(\mathcal{D})|\theta)$. Consequently, if we have two datasets with the same sufficient statistics, we will infer the same value for θ .

Now suppose the data consists of the count of the number of heads N_1 observed in a fixed number $N = N_1 + N_0$ of trials. In this case, we have $N_1 \sim \text{Bin}(N, \theta)$, where Bin represents the binomial distribution, which has the following pmf:

$$\text{Bin}(k|n, \theta) \triangleq \binom{n}{k} \theta^k (1 - \theta)^{n-k} \quad (3.12)$$

Since $\binom{n}{k}$ is a constant independent of θ , the likelihood for the binomial sampling model is the same as the likelihood for the Bernoulli model. So any inferences we make about θ will be the same whether we observe the counts, $\mathcal{D} = (N_1, N)$, or a sequence of trials, $\mathcal{D} = \{x_1, \dots, x_N\}$.

3.3.2 Prior

We need a prior which has support over the interval $[0, 1]$. To make the math easier, it would be convenient if the prior had the same form as the likelihood, i.e., if the prior looked like

$$p(\theta) \propto \theta^{\gamma_1} (1 - \theta)^{\gamma_2} \quad (3.13)$$

for some prior parameters γ_1 and γ_2 . If this were the case, then we could easily evaluate the posterior by simply adding up the exponents:

$$p(\theta) \propto p(\mathcal{D}|\theta)p(\theta) = \theta^{N_1} (1 - \theta)^{N_0} \theta^{\gamma_1} (1 - \theta)^{\gamma_2} = \theta^{N_1 + \gamma_1} (1 - \theta)^{N_0 + \gamma_2} \quad (3.14)$$

When the prior and the posterior have the same form, we say that the prior is a **conjugate prior** for the corresponding likelihood. Conjugate priors are widely used because they simplify computation, and are easy to interpret, as we see below.

In the case of the Bernoulli, the conjugate prior is the beta distribution, which we encountered in Section 2.4.5:

$$\text{Beta}(\theta|a, b) \propto \theta^{a-1} (1 - \theta)^{b-1} \quad (3.15)$$

The parameters of the prior are called **hyper-parameters**. We can set them in order to encode our prior beliefs. For example, to encode our beliefs that θ has mean 0.7 and standard deviation 0.2, we set $a = 2.975$ and $b = 1.275$ (Exercise 3.15). Or to encode our beliefs that θ has mean 0.15 and that we think it lives in the interval $(0.05, 0.30)$ with probability, then we find $a = 4.5$ and $b = 25.5$ (Exercise 3.16).

If we know “nothing” about θ , except that it lies in the interval $[0, 1]$, we can use a uniform prior, which is a kind of uninformative prior (see Section 5.4.2 for details). The uniform distribution can be represented by a beta distribution with $a = b = 1$.

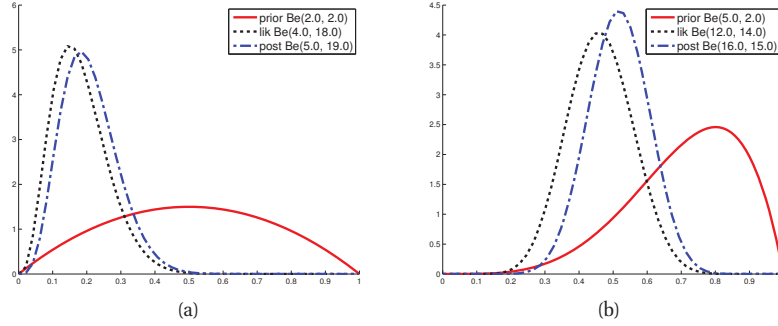


Figure 3.6 (a) Updating a Beta(2, 2) prior with a Binomial likelihood with sufficient statistics $N_1 = 3, N_0 = 17$ to yield a Beta(5,19) posterior. (b) Updating a Beta(5, 2) prior with a Binomial likelihood with sufficient statistics $N_1 = 11, N_0 = 13$ to yield a Beta(16, 15) posterior. Figure generated by binomialBetaPosteriorDemo.

3.3.3 Posterior

If we multiply the likelihood by the beta prior we get the following posterior (following Equation 3.14):

$$p(\theta|\mathcal{D}) \propto \text{Bin}(N_1|\theta, N_0 + N_1)\text{Beta}(\theta|a, b)\text{Beta}(\theta|N_1 + a, N_0 + b) \quad (3.16)$$

In particular, the posterior is obtained by adding the prior hyper-parameters to the empirical counts. For this reason, the hyper-parameters are known as **pseudo counts**. The strength of the prior, also known as the **effective sample size** of the prior, is the sum of the pseudo counts, $a + b$; this plays a role analogous to the data set size, $N_1 + N_0 = N$.

Figure 3.6(a) gives an example where we update a weak Beta(2,2) prior with a peaked likelihood function, corresponding to a large sample size; we see that the posterior is essentially identical to the likelihood: since the data has overwhelmed the prior. Figure 3.6(b) gives an example where we update a strong Beta(5,2) prior with a peaked likelihood function; now we see that the posterior is a “compromise” between the prior and likelihood.

Note that updating the posterior sequentially is equivalent to updating in a single batch. To see this, suppose we have two data sets $\mathcal{D}_a$ and $\mathcal{D}_b$ with sufficient statistics N_1^a, N_0^a and N_1^b, N_0^b . Let $N_1 = N_1^a + N_1^b$ and $N_0 = N_0^a + N_0^b$ be the sufficient statistics of the combined datasets. In batch mode we have

$$p(\theta|\mathcal{D}_a, \mathcal{D}_b) \propto \text{Bin}(N_1|\theta, N_1 + N_0)\text{Beta}(\theta|a, b) \propto \text{Beta}(\theta|N_1 + a, N_0 + b) \quad (3.17)$$

In sequential mode, we have

$$p(\theta|\mathcal{D}_a, \mathcal{D}_b) \propto p(\mathcal{D}_b|\theta)p(\theta|\mathcal{D}_a) \quad (3.18)$$

$$\propto \text{Bin}(N_1^b|\theta, N_1^b + N_0^b)\text{Beta}(\theta|N_1^a + a, N_0^a + b) \quad (3.19)$$

$$\propto \text{Beta}(\theta|N_1^a + N_1^b + a, N_0^a + N_0^b + b) \quad (3.20)$$

This makes Bayesian inference particularly well-suited to **online learning**, as we will see later.

3.3.3.1 Posterior mean and mode

From Equation 2.62, the MAP estimate is given by

$$\hat{\theta}_{MAP} = \frac{a + N_1 - 1}{a + b + N - 2} \quad (3.21)$$

If we use a uniform prior, then the MAP estimate reduces to the MLE, which is just the empirical fraction of heads:

$$\hat{\theta}_{MLE} = \frac{N_1}{N} \quad (3.22)$$

This makes intuitive sense, but it can also be derived by applying elementary calculus to maximize the likelihood function in Equation 3.11. (Exercise 3.1).

By contrast, the posterior mean is given by,

$$\bar{\theta} = \frac{a + N_1}{a + b + N} \quad (3.23)$$

This difference between the mode and the mean will prove important later.

We will now show that the posterior mean is convex combination of the prior mean and the MLE, which captures the notion that the posterior is a compromise between what we previously believed and what the data is telling us.

Let $\alpha_0 = a + b$ be the **equivalent sample size** of the prior, which controls its strength, and let the prior mean be $m_1 = a/\alpha_0$. Then the posterior mean is given by

$$\mathbb{E}[\theta|\mathcal{D}] = \frac{\alpha_0 m_1 + N_1}{N + \alpha_0} = \frac{\alpha_0}{N + \alpha_0} m_1 + \frac{N}{N + \alpha_0} \frac{N_1}{N} = \lambda m_1 + (1 - \lambda) \hat{\theta}_{MLE} \quad (3.24)$$

where $\lambda = \frac{\alpha_0}{N + \alpha_0}$ is the ratio of the prior to posterior equivalent sample size. So the weaker the prior, the smaller is λ , and hence the closer the posterior mean is to the MLE. One can show similarly that the posterior mode is a convex combination of the prior mode and the MLE, and that it too converges to the MLE.

3.3.3.2 Posterior variance

The mean and mode are point estimates, but it is useful to know how much we can trust them. The variance of the posterior is one way to measure this. The variance of the Beta posterior is given by

$$\text{var}[\theta|\mathcal{D}] = \frac{(a + N_1)(b + N_0)}{(a + N_1 + b + N_0)^2(a + N_1 + b + N_0 + 1)} \quad (3.25)$$

We can simplify this formidable expression in the case that $N \gg a, b$, to get

$$\text{var}[\theta|\mathcal{D}] \approx \frac{N_1 N_0}{N N N} = \frac{\hat{\theta}(1 - \hat{\theta})}{N} \quad (3.26)$$

where $\hat{\theta}$ is the MLE. Hence the “**error bar**” in our estimate (i.e., the posterior standard deviation), is given by

$$\sigma = \sqrt{\text{var}[\theta|\mathcal{D}]} \approx \sqrt{\frac{\hat{\theta}(1 - \hat{\theta})}{N}} \quad (3.27)$$

We see that the uncertainty goes down at a rate of $1/\sqrt{N}$. Note, however, that the uncertainty (variance) is maximized when $\hat{\theta} = 0.5$, and is minimized when $\hat{\theta}$ is close to 0 or 1. This means it is easier to be sure that a coin is biased than to be sure that it is fair.

3.3.4 Posterior predictive distribution

So far, we have been focusing on inference of the unknown parameter(s). Let us now turn our attention to prediction of future observable data.

Consider predicting the probability of heads in a single future trial under a $\text{Beta}(a, b)$ posterior. We have

$$p(\tilde{x} = 1|\mathcal{D}) = \int_0^1 p(x = 1|\theta)p(\theta|\mathcal{D})d\theta \quad (3.28)$$

$$= \int_0^1 \theta \text{Beta}(\theta|a, b)d\theta = \mathbb{E}[\theta|\mathcal{D}] = \frac{a}{a+b} \quad (3.29)$$

Thus we see that the mean of the posterior predictive distribution is equivalent (in this case) to plugging in the posterior mean parameters: $p(\tilde{x}|\mathcal{D}) = \text{Ber}(\tilde{x}|\mathbb{E}[\theta|\mathcal{D}])$.

3.3.4.1 Overfitting and the black swan paradox

Suppose instead that we plug-in the MLE, i.e., we use $p(\tilde{x}|\mathcal{D}) \approx \text{Ber}(\tilde{x}|\hat{\theta}_{MLE})$. Unfortunately, this approximation can perform quite poorly when the sample size is small. For example, suppose we have seen $N = 3$ tails in a row. The MLE is $\hat{\theta} = 0/3 = 0$, since this makes the observed data as probable as possible. However, using this estimate, we predict that heads are impossible. This is called the **zero count problem** or the **sparse data problem**, and frequently occurs when estimating counts from small amounts of data. One might think that in the era of “big data”, such concerns are irrelevant, but note that once we partition the data based on certain criteria — such as the number of times a *specific person* has engaged in a *specific activity* — the sample sizes can become much smaller. This problem arises, for example, when trying to perform personalized recommendation of web pages. Thus Bayesian methods are still useful, even in the big data regime (Jordan 2011).

The zero-count problem is analogous to a problem in philosophy called the **black swan paradox**. This is based on the ancient Western conception that all swans were white. In that context, a black swan was a metaphor for something that could not exist. (Black swans were discovered in Australia by European explorers in the 17th Century.) The term “black swan paradox” was first coined by the famous philosopher of science Karl Popper; the term has also been used as the title of a recent popular book (Taleb 2007). This paradox was used to illustrate the problem of **induction**, which is the problem of how to draw general conclusions about the future from specific observations from the past.

Let us now derive a simple Bayesian solution to the problem. We will use a uniform prior, so $a = b = 1$. In this case, plugging in the posterior mean gives **Laplace’s rule of succession**

$$p(\tilde{x} = 1|\mathcal{D}) = \frac{N_1 + 1}{N_1 + N_0 + 2} \quad (3.30)$$

This justifies the common practice of adding 1 to the empirical counts, normalizing and then plugging them in, a technique known as **add-one smoothing**. (Note that plugging in the MAP

parameters would not have this smoothing effect, since the mode has the form $\hat{\theta} = \frac{N_1 + a - 1}{N + a + b - 2}$, which becomes the MLE if $a = b = 1$.)

3.3.4.2 Predicting the outcome of multiple future trials

Suppose now we were interested in predicting the number of heads, x , in M future trials. This is given by

$$p(x|\mathcal{D}, M) = \int_0^1 \text{Bin}(x|\theta, M) \text{Beta}(\theta|a, b) d\theta \quad (3.31)$$

$$= \binom{M}{x} \frac{1}{B(a, b)} \int_0^1 \theta^x (1 - \theta)^{M-x} \theta^{a-1} (1 - \theta)^{b-1} d\theta \quad (3.32)$$

We recognize the integral as the normalization constant for a $\text{Beta}(a+x, M-x+b)$ distribution. Hence

$$\int_0^1 \theta^x (1 - \theta)^{M-x} \theta^{a-1} (1 - \theta)^{b-1} d\theta = B(x+a, M-x+b) \quad (3.33)$$

Thus we find that the posterior predictive is given by the following, known as the (compound) **beta-binomial** distribution:

$$Bb(x|a, b, M) \triangleq \binom{M}{x} \frac{B(x+a, M-x+b)}{B(a, b)} \quad (3.34)$$

This distribution has the following mean and variance

$$\mathbb{E}[x] = M \frac{a}{a+b}, \quad \text{var}[x] = \frac{Mab}{(a+b)^2} \frac{(a+b+M)}{a+b+1} \quad (3.35)$$

If $M = 1$, and hence $x \in \{0, 1\}$, we see that the mean becomes $\mathbb{E}[x|\mathcal{D}] = p(x=1|\mathcal{D}) = \frac{a}{a+b}$, which is consistent with Equation 3.29.

This process is illustrated in Figure 3.7(a). We start with a $\text{Beta}(2,2)$ prior, and plot the posterior predictive density after seeing $N_1 = 3$ heads and $N_0 = 17$ tails. Figure 3.7(b) plots a plug-in approximation using a MAP estimate. We see that the Bayesian prediction has longer tails, spreading its probability mass more widely, and is therefore less prone to overfitting and blackswan type paradoxes.

3.4 The Dirichlet-multinomial model

In the previous section, we discussed how to infer the probability that a coin comes up heads. In this section, we generalize these results to infer the probability that a dice with K sides comes up as face k . This might seem like another toy exercise, but the methods we will study are widely used to analyse text data, biosequence data, etc., as we will see later.

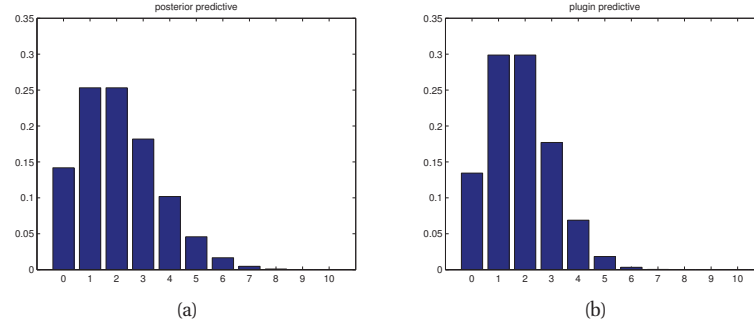


Figure 3.7 (a) Posterior predictive distributions after seeing $N_1 = 3, N_0 = 17$. (b) Plugin approximation. Figure generated by `betaBinomPostPredDemo`.

3.4.1 Likelihood

Suppose we observe N dice rolls, $\mathcal{D} = \{x_1, \dots, x_N\}$, where $x_i \in \{1, \dots, K\}$. If we assume the data is iid, the likelihood has the form

$$p(\mathcal{D}|\boldsymbol{\theta}) = \prod_{k=1}^K \theta_k^{N_k} \quad (3.36)$$

where $N_k = \sum_{i=1}^N \mathbb{I}(y_i = k)$ is the number of times event k occurred (these are the sufficient statistics for this model). The likelihood for the multinomial model has the same form, up to an irrelevant constant factor.

3.4.2 Prior

Since the parameter vector lives in the K -dimensional probability simplex, we need a prior that has support over this simplex. Ideally it would also be conjugate. Fortunately, the Dirichlet distribution (Section 2.5.4) satisfies both criteria. So we will use the following prior:

$$\text{Dir}(\boldsymbol{\theta}|\boldsymbol{\alpha}) = \frac{1}{B(\boldsymbol{\alpha})} \prod_{k=1}^K \theta_k^{\alpha_k - 1} \mathbb{I}(\mathbf{x} \in S_K) \quad (3.37)$$

3.4.3 Posterior

Multiplying the likelihood by the prior, we find that the posterior is also Dirichlet:

$$p(\boldsymbol{\theta}|\mathcal{D}) \propto p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}) \quad (3.38)$$

$$\propto \prod_{k=1}^K \theta_k^{N_k} \theta_k^{\alpha_k - 1} = \prod_{k=1}^K \theta_k^{\alpha_k + N_k - 1} \quad (3.39)$$

$$= \text{Dir}(\boldsymbol{\theta}|\alpha_1 + N_1, \dots, \alpha_K + N_K) \quad (3.40)$$

We see that the posterior is obtained by adding the prior hyper-parameters (pseudo-counts) α_k to the empirical counts N_k .

We can derive the mode of this posterior (i.e., the MAP estimate) by using calculus. However, we must enforce the constraint that $\sum_k \theta_k = 1$.² We can do this by using a **Lagrange multiplier**. The constrained objective function, or **Lagrangian**, is given by the log likelihood plus log prior plus the constraint:

$$\ell(\boldsymbol{\theta}, \lambda) = \sum_k N_k \log \theta_k + \sum_k (\alpha_k - 1) \log \theta_k + \lambda \left(1 - \sum_k \theta_k \right) \quad (3.41)$$

To simplify notation, we define $N'_k \triangleq N_k + \alpha_k - 1$. Taking derivatives with respect to λ yields the original constraint:

$$\frac{\partial \ell}{\partial \lambda} = \left(1 - \sum_k \theta_k \right) = 0 \quad (3.42)$$

Taking derivatives with respect to θ_k yields

$$\frac{\partial \ell}{\partial \theta_k} = \frac{N'_k}{\theta_k} - \lambda = 0 \quad (3.43)$$

$$N'_k = \lambda \theta_k \quad (3.44)$$

We can solve for λ using the sum-to-one constraint:

$$\sum_k N'_k = \lambda \sum_k \theta_k \quad (3.45)$$

$$N + \alpha_0 - K = \lambda \quad (3.46)$$

where $\alpha_0 \triangleq \sum_{k=1}^K \alpha_k$ is the equivalent sample size of the prior. Thus the MAP estimate is given by

$$\hat{\theta}_k = \frac{N_k + \alpha_k - 1}{N + \alpha_0 - K} \quad (3.47)$$

which is consistent with Equation 2.77. If we use a uniform prior, $\alpha_k = 1$, we recover the MLE:

$$\hat{\theta}_k = N_k / N \quad (3.48)$$

This is just the empirical fraction of times face k shows up.

2. We do not need to explicitly enforce the constraint that $\theta_k \geq 0$ since the gradient of the objective has the form $N_k/\theta_k - \lambda$; so negative values would reduce the objective, rather than maximize it. (Of course, this does not preclude setting $\theta_k = 0$, and indeed this is the optimal solution if $N_k = 0$ and $\alpha_k = 1$.)

3.4.4 Posterior predictive

The posterior predictive distribution for a single multinoulli trial is given by the following expression:

$$p(X = j|\mathcal{D}) = \int p(X = j|\boldsymbol{\theta})p(\boldsymbol{\theta}|\mathcal{D})d\boldsymbol{\theta} \quad (3.49)$$

$$= \int p(X = j|\theta_j) \left[\int p(\boldsymbol{\theta}_{-j}, \theta_j|\mathcal{D})d\boldsymbol{\theta}_{-j} \right] d\theta_j \quad (3.50)$$

$$= \int \theta_j p(\theta_j|\mathcal{D})d\theta_j = \mathbb{E}[\theta_j|\mathcal{D}] = \frac{\alpha_j + N_j}{\sum_k (\alpha_k + N_k)} = \frac{\alpha_j + N_j}{\alpha_0 + N} \quad (3.51)$$

where $\boldsymbol{\theta}_{-j}$ are all the components of $\boldsymbol{\theta}$ except θ_j . See also Exercise 3.13.

The above expression avoids the zero-count problem, just as we saw in Section 3.3.4.1. In fact, this form of Bayesian smoothing is even more important in the multinomial case than the binary case, since the likelihood of data sparsity increases once we start partitioning the data into many categories.

3.4.4.1 Worked example: language models using bag of words

One application of Bayesian smoothing using the Dirichlet-multinomial model is to **language modeling**, which means predicting which words might occur next in a sequence. Here we will take a very simple-minded approach, and assume that the i 'th word, $X_i \in \{1, \dots, K\}$, is sampled independently from all the other words using a $\text{Cat}(\boldsymbol{\theta})$ distribution. This is called the **bag of words** model. Given a past sequence of words, how can we predict which one is likely to come next?

For example, suppose we observe the following sequence (part of a children's nursery rhyme):

Mary had a little lamb, little lamb, little lamb,
Mary had a little lamb, its fleece as white as snow

Furthermore, suppose our vocabulary consists of the following words:

mary	lamb	little	big	fleece	white	black	snow	rain	unk
1	2	3	4	5	6	7	8	9	10

Here **unk** stands for unknown, and represents all other words that do not appear elsewhere on the list. To encode each line of the nursery rhyme, we first strip off punctuation, and remove any **stop words** such as “a”, “as”, “the”, etc. We can also perform **stemming**, which means reducing words to their base form, such as stripping off the final *s* in plural words, or the *ing* from verbs (e.g., *running* becomes *run*). In this example, no words need stemming. Finally, we replace each word by its index into the vocabulary to get:

1	10	3	2	3	2	3	2	
1	10	3	2	10	5	10	6	8

We now ignore the word order, and count how often each word occurred, resulting in a histogram of word counts:

Token	1	2	3	4	5	6	7	8	9	10
Word	mary	lamb	little	big	fleece	white	black	snow	rain	unk
Count	2	4	4	0	1	1	0	1	0	4

Denote the above counts by N_j . If we use a $\text{Dir}(\alpha)$ prior for θ , the posterior predictive is just

$$p(\tilde{X} = j|D) = E[\theta_j|D] = \frac{\alpha_j + N_j}{\sum_{j'} \alpha_{j'} + N_{j'}} = \frac{1 + N_j}{10 + 17} \quad (3.52)$$

If we set $\alpha_j = 1$, we get

$$p(\tilde{X} = j|D) = (3/27, 5/27, 5/27, 1/27, 2/27, 2/27, 1/27, 2/27, 1/27, 5/27) \quad (3.53)$$

The modes of the predictive distribution are $X = 2$ (“lamb”) and $X = 10$ (“unk”). Note that the words “big”, “black” and “rain” are predicted to occur with non-zero probability in the future, even though they have never been seen before. Later on we will see more sophisticated language models.

3.5 Naive Bayes classifiers

In this section, we discuss how to classify vectors of discrete-valued features, $\mathbf{x} \in \{1, \dots, K\}^D$, where K is the number of values for each feature, and D is the number of features. We will use a generative approach. This requires us to specify the class conditional distribution, $p(\mathbf{x}|y = c)$. The simplest approach is to assume the features are **conditionally independent** given the class label. This allows us to write the class conditional density as a product of one dimensional densities:

$$p(\mathbf{x}|y = c, \theta) = \prod_{j=1}^D p(x_j|y = c, \theta_{jc}) \quad (3.54)$$

The resulting model is called a **naive Bayes classifier** (NBC).

The model is called “naive” since we do not expect the features to be independent, even conditional on the class label. However, even if the naive Bayes assumption is not true, it often results in classifiers that work well (Domingos and Pazzani 1997). One reason for this is that the model is quite simple (it only has $O(CD)$ parameters, for C classes and D features), and hence it is relatively immune to overfitting.

The form of the class-conditional density depends on the type of each feature. We give some possibilities below:

- In the case of real-valued features, we can use the Gaussian distribution: $p(\mathbf{x}|y = c, \theta) = \prod_{j=1}^D \mathcal{N}(x_j|\mu_{jc}, \sigma_{jc}^2)$, where μ_{jc} is the mean of feature j in objects of class c , and σ_{jc}^2 is its variance.
- In the case of binary features, $x_j \in \{0, 1\}$, we can use the Bernoulli distribution: $p(\mathbf{x}|y = c, \theta) = \prod_{j=1}^D \text{Ber}(x_j|\mu_{jc})$, where μ_{jc} is the probability that feature j occurs in class c . This is sometimes called the **multivariate Bernoulli naive Bayes** model. We will see an application of this below.

- In the case of categorical features, $x_j \in \{1, \dots, K\}$, we can model use the multinoulli distribution: $p(\mathbf{x}|y = c, \boldsymbol{\theta}) = \prod_{j=1}^D \text{Cat}(x_j|\boldsymbol{\mu}_{jc})$, where $\boldsymbol{\mu}_{jc}$ is a histogram over the K possible values for x_j in class c .

Obviously we can handle other kinds of features, or use different distributional assumptions. Also, it is easy to mix and match features of different types.

3.5.1 Model fitting

We now discuss how to “train” a naive Bayes classifier. This usually means computing the MLE or the MAP estimate for the parameters. However, we will also discuss how to compute the full posterior, $p(\boldsymbol{\theta}|\mathcal{D})$.

3.5.1.1 MLE for NBC

The probability for a single data case is given by

$$p(\mathbf{x}_i, y_i|\boldsymbol{\theta}) = p(y_i|\boldsymbol{\pi}) \prod_j p(x_{ij}|\boldsymbol{\theta}_j) = \prod_c \pi_c^{\mathbb{I}(y_i=c)} \prod_j \prod_c p(x_{ij}|\boldsymbol{\theta}_{jc})^{\mathbb{I}(y_i=c)} \quad (3.55)$$

Hence the log-likelihood is given by

$$\log p(\mathcal{D}|\boldsymbol{\theta}) = \sum_{c=1}^C N_c \log \pi_c + \sum_{j=1}^D \sum_{c=1}^C \sum_{i:y_i=c} \log p(x_{ij}|\boldsymbol{\theta}_{jc}) \quad (3.56)$$

We see that this expression decomposes into a series of terms, one concerning $\boldsymbol{\pi}$, and DC terms containing the $\boldsymbol{\theta}_{jc}$'s. Hence we can optimize all these parameters separately.

From Equation 3.48, the MLE for the class prior is given by

$$\hat{\pi}_c = \frac{N_c}{N} \quad (3.57)$$

where $N_c \triangleq \sum_i \mathbb{I}(y_i = c)$ is the number of examples in class c .

The MLE for the likelihood depends on the type of distribution we choose to use for each feature. For simplicity, let us suppose all features are binary, so $x_j|y = c \sim \text{Ber}(\theta_{jc})$. In this case, the MLE becomes

$$\hat{\theta}_{jc} = \frac{N_{jc}}{N_c} \quad (3.58)$$

It is extremely simple to implement this model fitting procedure: See Algorithm 8 for some pseudo-code (and `naiveBayesFit` for some Matlab code). This algorithm obviously takes $O(ND)$ time. The method is easily generalized to handle features of mixed type. This simplicity is one reason the method is so widely used.

Figure 3.8 gives an example where we have 2 classes and 600 binary features, representing the presence or absence of words in a bag-of-words model. The plot visualizes the $\boldsymbol{\theta}_c$ vectors for the two classes. The big spike at index 107 corresponds to the word “subject”, which occurs in both classes with probability 1. (In Section 3.5.4, we discuss how to “filter out” such uninformative features.)

Algorithm 3.1: Fitting a naive Bayes classifier to binary features

```

1  $N_c = 0, N_{jc} = 0;$ 
2 for  $i = 1 : N$  do
3    $c = y_i$  // Class label of  $i$ 'th example;
4    $N_c := N_c + 1$  ;
5   for  $j = 1 : D$  do
6     if  $x_{ij} = 1$  then
7        $N_{jc} := N_{jc} + 1$ 
8  $\hat{\pi}_c = \frac{N_c}{N}, \hat{\theta}_{jc} = \frac{N_{jc}}{N}$ 

```

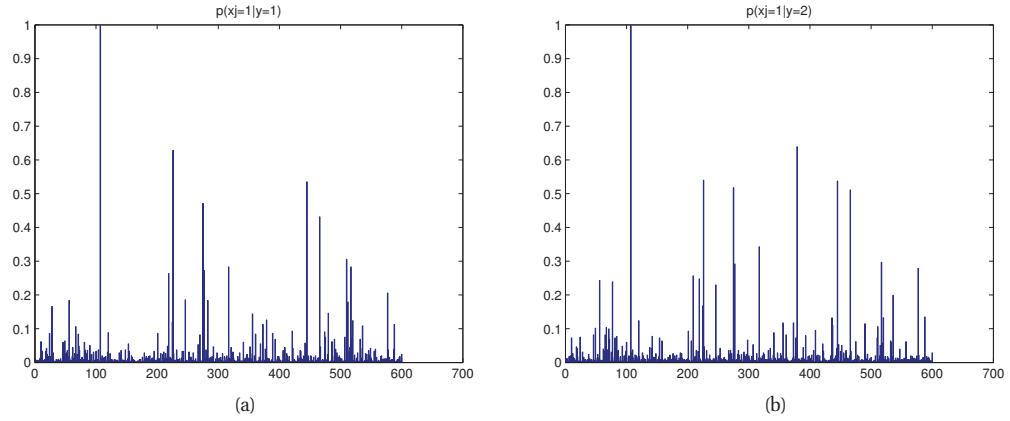


Figure 3.8 Class conditional densities $p(x_j = 1 | y = c)$ for two document classes, corresponding to “X windows” and “MS windows”. Figure generated by `naiveBayesBowDemo`.

3.5.1.2 Bayesian naive Bayes

The trouble with maximum likelihood is that it can overfit. For example, consider the example in Figure 3.8: the feature corresponding to the word “subject” (call it feature j) always occurs in both classes, so we estimate $\theta_{jc} = 1$. What will happen if we encounter a new email which does not have this word in it? Our algorithm will crash and burn, since we will find that $p(y = c | \mathbf{x}, \hat{\theta}) = 0$ for both classes! This is another manifestation of the black swan paradox discussed in Section 3.3.4.1.

A simple solution to overfitting is to be Bayesian. For simplicity, we will use a factored prior:

$$p(\theta) = p(\pi) \prod_{j=1}^D \prod_{c=1}^C p(\theta_{jc}) \quad (3.59)$$

We will use a $\text{Dir}(\alpha)$ prior for π and a $\text{Beta}(\beta_0, \beta_1)$ prior for each θ_{jc} . Often we just take $\alpha = \mathbf{1}$ and $\beta = \mathbf{1}$, corresponding to add-one or Laplace smoothing.

Combining the factored likelihood in Equation 3.56 with the factored prior above gives the following factored posterior:

$$p(\boldsymbol{\theta}|\mathcal{D}) = p(\boldsymbol{\pi}|\mathcal{D}) \prod_{j=1}^D \prod_{c=1}^C p(\theta_{jc}|\mathcal{D}) \quad (3.60)$$

$$p(\boldsymbol{\pi}|\mathcal{D}) = \text{Dir}(N_1 + \alpha_1, \dots, N_C + \alpha_C) \quad (3.61)$$

$$p(\theta_{jc}|\mathcal{D}) = \text{Beta}((N_c - N_{jc}) + \beta_0, N_{jc} + \beta_1) \quad (3.62)$$

In other words, to compute the posterior, we just update the prior counts with the empirical counts from the likelihood. It is straightforward to modify algorithm 8 to handle this version of model “fitting”.

3.5.2 Using the model for prediction

At test time, the goal is to compute

$$p(y = c|\mathbf{x}, \mathcal{D}) \propto p(y = c|\mathcal{D}) \prod_{j=1}^D p(x_j|y = c, \mathcal{D}) \quad (3.63)$$

The correct Bayesian procedure is to integrate out the unknown parameters:

$$p(y = c|\mathbf{x}, \mathcal{D}) \propto \left[\int \text{Cat}(y = c|\boldsymbol{\pi}) p(\boldsymbol{\pi}|\mathcal{D}) d\boldsymbol{\pi} \right] \quad (3.64)$$

$$\prod_{j=1}^D \left[\int \text{Ber}(x_j|y = c, \theta_{jc}) p(\theta_{jc}|\mathcal{D}) \right] \quad (3.65)$$

Fortunately, this is easy to do, at least if the posterior is Dirichlet. In particular, from Equation 3.51, we know the posterior predictive density can be obtained by simply plugging in the posterior mean parameters $\bar{\boldsymbol{\theta}}$. Hence

$$p(y = c|\mathbf{x}, \mathcal{D}) \propto \bar{\pi}_c \prod_{j=1}^D (\bar{\theta}_{jc})^{\mathbb{I}(x_j=1)} (1 - \bar{\theta}_{jc})^{\mathbb{I}(x_j=0)} \quad (3.66)$$

$$\bar{\theta}_{jk} = \frac{N_{jc} + \beta_1}{N_c + \beta_0 + \beta_1} \quad (3.67)$$

$$\bar{\pi}_c = \frac{N_c + \alpha_c}{N + \alpha_0} \quad (3.68)$$

where $\alpha_0 = \sum_c \alpha_c$.

If we have approximated the posterior by a single point, $p(\boldsymbol{\theta}|\mathcal{D}) \approx \delta_{\hat{\boldsymbol{\theta}}}(\boldsymbol{\theta})$, where $\hat{\boldsymbol{\theta}}$ may be the ML or MAP estimate, then the posterior predictive density is obtained by simply plugging in the parameters, to yield a virtually identical rule:

$$p(y = c|\mathbf{x}, \mathcal{D}) \propto \hat{\pi}_c \prod_{j=1}^D (\hat{\theta}_{jc})^{\mathbb{I}(x_j=1)} (1 - \hat{\theta}_{jc})^{\mathbb{I}(x_j=0)} \quad (3.69)$$

The only difference is we replaced the posterior mean $\bar{\theta}$ with the posterior mode or MLE $\hat{\theta}$. However, this small difference can be important in practice, since the posterior mean will result in less overfitting (see Section 3.4.4.1).

3.5.3 The log-sum-exp trick

We now discuss one important practical detail that arises when using generative classifiers of any kind. We can compute the posterior over class labels using Equation 2.13, using the appropriate class-conditional density (and a plug-in approximation). Unfortunately a naive implementation of Equation 2.13 can fail due to **numerical underflow**. The problem is that $p(\mathbf{x}|y = c)$ is often a very small number, especially if $\mathbf{x}$ is a high-dimensional vector. This is because we require that $\sum_{\mathbf{x}} p(\mathbf{x}|y) = 1$, so the probability of observing any particular high-dimensional vector is small. The obvious solution is to take logs when applying Bayes rule, as follows:

$$\log p(y = c|\mathbf{x}) = b_c - \log \left[\sum_{c'=1}^C e^{b_{c'}} \right] \quad (3.70)$$

$$b_c \triangleq \log p(\mathbf{x}|y = c) + \log p(y = c) \quad (3.71)$$

However, this requires evaluating the following expression

$$\log \left[\sum_{c'} e^{b_{c'}} \right] = \log \sum_{c'} p(y = c', \mathbf{x}) = \log p(\mathbf{x}) \quad (3.72)$$

and we can't add up in the log domain. Fortunately, we can factor out the largest term, and just represent the remaining numbers relative to that. For example,

$$\log(e^{-120} + e^{-121}) = \log(e^{-120}(e^0 + e^{-1})) = \log(e^0 + e^{-1}) - 120 \quad (3.73)$$

In general, we have

$$\log \sum_c e^{b_c} = \log \left[\left(\sum_c e^{b_c - B} \right) e^B \right] = \left[\log \left(\sum_c e^{b_c - B} \right) \right] + B \quad (3.74)$$

where $B = \max_c b_c$. This is called the **log-sum-exp** trick, and is widely used. (See the function `logsumexp` for an implementation.)

This trick is used in Algorithm 1 which gives pseudo-code for using an NBC to compute $p(y_i|\mathbf{x}_i, \hat{\theta})$. See `naiveBayesPredict` for the Matlab code. Note that we do not need the log-sum-exp trick if we only want to compute $\hat{y}_i$, since we can just maximize the unnormalized quantity $\log p(y_i = c) + \log p(\mathbf{x}_i|y = c)$.

3.5.4 Feature selection using mutual information

Since an NBC is fitting a joint distribution over potentially many features, it can suffer from overfitting. In addition, the run-time cost is $O(D)$, which may be too high for some applications.

One common approach to tackling both of these problems is to perform **feature selection**, to remove “irrelevant” features that do not help much with the classification problem. The simplest approach to feature selection is to evaluate the relevance of each feature separately, and then

Algorithm 3.2: Predicting with a naive bayes classifier for binary features

```

1 for  $i = 1 : N$  do
2   for  $c = 1 : C$  do
3      $L_{ic} = \log \hat{\pi}_c$ ;
4     for  $j = 1 : D$  do
5       if  $x_{ij} = 1$  then  $L_{ic} := L_{ic} + \log \hat{\theta}_{jc}$  else  $L_{ic} := L_{ic} + \log(1 - \hat{\theta}_{jc})$ 
6    $p_{ic} = \exp(L_{ic} - \text{logsumexp}(L_{i,:}))$ ;
7    $\hat{y}_i = \text{argmax}_c p_{ic}$ ;

```

take the top K , where K is chosen based on some tradeoff between accuracy and complexity. This approach is known as variable **ranking**, **filtering**, or **screening**.

One way to measure relevance is to use mutual information (Section 2.8.3) between feature X_j and the class label Y :

$$I(X, Y) = \sum_{x_j} \sum_y p(x_j, y) \log \frac{p(x_j, y)}{p(x_j)p(y)} \quad (3.75)$$

The mutual information can be thought of as the reduction in entropy on the label distribution once we observe the value of feature j . If the features are binary, it is easy to show (Exercise 3.21) that the MI can be computed as follows

$$I_j = \sum_c \left[\theta_{jc} \pi_c \log \frac{\theta_{jc}}{\theta_j} + (1 - \theta_{jc}) \pi_c \log \frac{1 - \theta_{jc}}{1 - \theta_j} \right] \quad (3.76)$$

where $\pi_c = p(y = c)$, $\theta_{jc} = p(x_j = 1 | y = c)$, and $\theta_j = p(x_j = 1) = \sum_c \pi_c \theta_{jc}$. (All of these quantities can be computed as a by-product of fitting a naive Bayes classifier.)

Figure 3.1 illustrates what happens if we apply this to the binary bag of words dataset used in Figure 3.8. We see that the words with highest mutual information are much more discriminative than the words which are most probable. For example, the most probable word in both classes is “subject”, which always occurs because this is newsgroup data, which always has a subject line. But obviously this is not very discriminative. The words with highest MI with the class label are (in decreasing order) “windows”, “microsoft”, “DOS” and “motif”, which makes sense, since the classes correspond to Microsoft Windows and X Windows.

3.5.5 Classifying documents using bag of words

Document classification is the problem of classifying text documents into different categories. One simple approach is to represent each document as a binary vector, which records whether each word is present or not, so $x_{ij} = 1$ iff word j occurs in document i , otherwise $x_{ij} = 0$. We can then use the following class conditional density:

$$p(\mathbf{x}_i | y_i = c, \boldsymbol{\theta}) = \prod_{j=1}^D \text{Ber}(x_{ij} | \theta_{jc}) = \prod_{j=1}^D \theta_{jc}^{\mathbb{I}(x_{ij})} (1 - \theta_{jc})^{\mathbb{I}(1-x_{ij})} \quad (3.77)$$

class 1	prob	class 2	prob	highest MI	MI
subject	0.998	subject	0.998	windows	0.215
this	0.628	windows	0.639	microsoft	0.095
with	0.535	this	0.540	dos	0.092
but	0.471	with	0.538	motif	0.078
you	0.431	but	0.518	window	0.067

Table 3.1 We list the 5 most likely words for class 1 (X windows) and class 2 (MS windows). We also show the 5 words with highest mutual information with class label. Produced by `naiveBayesBowDemo`

This is called the **Bernoulli product model**, or the **binary independence model**.

However, ignoring the number of times each word occurs in a document loses some information (McCallum and Nigam 1998). A more accurate representation counts the number of occurrences of each word. Specifically, let $\mathbf{x}_i$ be a vector of counts for document i , so $x_{ij} \in \{0, 1, \dots, N_i\}$, where N_i is the number of terms in document i (so $\sum_{j=1}^D x_{ij} = N_i$). For the class conditional densities, we can use a multinomial distribution:

$$p(\mathbf{x}_i | y_i = c, \boldsymbol{\theta}) = \text{Mu}(\mathbf{x}_i | N_i, \boldsymbol{\theta}_c) = \frac{N_i!}{\prod_{j=1}^D x_{ij}!} \prod_{j=1}^D \theta_{jc}^{x_{ij}} \quad (3.78)$$

where we have implicitly assumed that the document length N_i is independent of the class. Here θ_{jc} is the probability of generating word j in documents of class c ; these parameters satisfy the constraint that $\sum_{j=1}^D \theta_{jc} = 1$ for each class c .³

Although the multinomial classifier is easy to train and easy to use at test time, it does not work particularly well for document classification. One reason for this is that it does not take into account the **burstiness** of word usage. This refers to the phenomenon that most words never appear in any given document, but if they do appear once, they are likely to appear more than once, i.e., words occur in bursts.

The multinomial model cannot capture the burstiness phenomenon. To see why, note that Equation 3.78 has the form $\theta_{jc}^{N_{ij}}$, and since $\theta_{jc} \ll 1$ for rare words, it becomes increasingly unlikely to generate many of them. For more frequent words, the decay rate is not as fast. To see why intuitively, note that the most frequent words are function words which are not specific to the class, such as “and”, “the”, and “but”; the chance of the word “and” occurring is pretty much the same no matter how many time it has previously occurred (modulo document length), so the independence assumption is more reasonable for common words. However, since rare words are the ones that matter most for classification purposes, these are the ones we want to model the most carefully.

Various ad hoc heuristics have been proposed to improve the performance of the multinomial document classifier (Rennie et al. 2003). We now present an alternative class conditional density that performs as well as these ad hoc methods, yet is probabilistically sound (Madsen et al. 2005).

3. Since Equation 3.78 models each word independently, this model is often called a naive Bayes classifier, although technically the features x_{ij} are not independent, because of the constraint $\sum_j x_{ij} = N_i$.

Suppose we simply replace the multinomial class conditional density with the **Dirichlet Compound Multinomial** or **DCM** density, defined as follows:

$$p(\mathbf{x}_i | y_i = c, \boldsymbol{\alpha}) = \int \text{Mu}(\mathbf{x}_i | N_i, \boldsymbol{\theta}_c) \text{Dir}(\boldsymbol{\theta}_c | \boldsymbol{\alpha}_c) d\boldsymbol{\theta}_c = \frac{N_i!}{\prod_{j=1}^D x_{ij}!} \frac{B(\mathbf{x}_i + \boldsymbol{\alpha}_c)}{B(\boldsymbol{\alpha}_c)} \quad (3.79)$$

(This equation is derived in Equation 5.24.) Surprisingly this simple change is all that is needed to capture the burstiness phenomenon. The intuitive reason for this is as follows: After seeing one occurrence of a word, say word j , the posterior counts on θ_j gets updated, making another occurrence of word j more likely. By contrast, if θ_j is fixed, then the occurrences of each word are independent. The multinomial model corresponds to drawing a ball from an urn with K colors of ball, recording its color, and then replacing it. By contrast, the DCM model corresponds to drawing a ball, recording its color, and then replacing it with one additional copy; this is called the **Polya urn**.

Using the DCM as the class conditional density gives much better results than using the multinomial, and has performance comparable to state of the art methods, as described in (Madsen et al. 2005). The only disadvantage is that fitting the DCM model is more complex; see (Minka 2000e; Elkan 2006) for the details.

Exercises

Exercise 3.1 MLE for the Bernoulli/ binomial model

Derive Equation 3.22 by optimizing the log of the likelihood in Equation 3.11.

Exercise 3.2 Marginal likelihood for the Beta-Bernoulli model

In Equation 5.23, we showed that the marginal likelihood is the ratio of the normalizing constants:

$$p(D) = \frac{Z(\alpha_1 + N_1, \alpha_0 + N_0)}{Z(\alpha_1, \alpha_0)} = \frac{\Gamma(\alpha_1 + N_1) \Gamma(\alpha_0 + N_0)}{\Gamma(\alpha_1 + \alpha_0 + N)} \frac{\Gamma(\alpha_1 + \alpha_0)}{\Gamma(\alpha_1) \Gamma(\alpha_0)} \quad (3.80)$$

We will now derive an alternative derivation of this fact. By the chain rule of probability,

$$p(x_{1:N}) = p(x_1) p(x_2 | x_1) p(x_3 | x_{1:2}) \dots \quad (3.81)$$

In Section 3.3.4, we showed that the posterior predictive distribution is

$$p(X = k | D_{1:N}) = \frac{N_k + \alpha_k}{\sum_i N_i + \alpha_i} \triangleq \frac{N_k + \alpha_k}{N + \alpha} \quad (3.82)$$

where $k \in \{0, 1\}$ and $D_{1:N}$ is the data seen so far. Now suppose $D = H, T, T, H, H$ or $D = 1, 0, 0, 1, 1$. Then

$$p(D) = \frac{\alpha_1}{\alpha} \cdot \frac{\alpha_0}{\alpha + 1} \cdot \frac{\alpha_0 + 1}{\alpha + 2} \cdot \frac{\alpha_1 + 1}{\alpha + 3} \cdot \frac{\alpha_1 + 2}{\alpha + 4} \quad (3.83)$$

$$= \frac{[\alpha_1(\alpha_1 + 1)(\alpha_1 + 2)] [\alpha_0(\alpha_0 + 1)]}{\alpha(\alpha + 1) \dots (\alpha + 4)} \quad (3.84)$$

$$= \frac{[(\alpha_1) \dots (\alpha_1 + N_1 - 1)] [(\alpha_0) \dots (\alpha_0 + N_0 - 1)]}{(\alpha) \dots (\alpha + N - 1)} \quad (3.85)$$

Show how this reduces to Equation 3.80 by using the fact that, for integers, $(\alpha - 1)! = \Gamma(\alpha)$.

Exercise 3.3 Posterior predictive for Beta-Binomial model

Recall from Equation 3.32 that the posterior predictive for the Beta-Binomial is given by

$$p(x|n, D) = Bb(x|\alpha'_0, \alpha'_1, n) \quad (3.86)$$

$$= \frac{B(x + \alpha'_1, n - x + \alpha'_0)}{B(\alpha'_1, \alpha'_0)} \binom{n}{x} \quad (3.87)$$

Prove that this reduces to

$$p(\tilde{x} = 1|D) = \frac{\alpha'_1}{\alpha'_0 + \alpha'_1} \quad (3.88)$$

when $n = 1$ (and hence $x \in \{0, 1\}$). i.e., show that

$$Bb(1|\alpha'_1, \alpha'_0, 1) = \frac{\alpha'_1}{\alpha'_1 + \alpha'_0} \quad (3.89)$$

Hint: use the fact that

$$\Gamma(\alpha_0 + \alpha_1 + 1) = (\alpha_0 + \alpha_1 + 1)\Gamma(\alpha_0 + \alpha_1) \quad (3.90)$$

Exercise 3.4 Beta updating from censored likelihood

(Source: Gelman.) Suppose we toss a coin $n = 5$ times. Let X be the number of heads. We observe that there are fewer than 3 heads, but we don't know exactly how many. Let the prior probability of heads be $p(\theta) = \text{Beta}(\theta|1, 1)$. Compute the posterior $p(\theta|X < 3)$ up to normalization constants, i.e., derive an expression proportional to $p(\theta, X < 3)$. Hint: the answer is a mixture distribution.

Exercise 3.5 Uninformative prior for log-odds ratio

Let

$$\phi = \text{logit}(\theta) = \log \frac{\theta}{1 - \theta} \quad (3.91)$$

Show that if $p(\phi) \propto 1$, then $p(\theta) \propto \text{Beta}(\theta|0, 0)$. Hint: use the change of variables formula.

Exercise 3.6 MLE for the Poisson distribution

The Poisson pmf is defined as $\text{Poi}(x|\lambda) = e^{-\lambda} \frac{\lambda^x}{x!}$, for $x \in \{0, 1, 2, \dots\}$ where $\lambda > 0$ is the rate parameter. Derive the MLE.

Exercise 3.7 Bayesian analysis of the Poisson distribution

In Exercise 3.6, we defined the Poisson distribution with rate λ and derived its MLE. Here we perform a conjugate Bayesian analysis.

- Derive the posterior $p(\lambda|D)$ assuming a conjugate prior $p(\lambda) = \text{Ga}(\lambda|a, b) \propto \lambda^{a-1} e^{-\lambda b}$. Hint: the posterior is also a Gamma distribution.
- What does the posterior mean tend to as $a \rightarrow 0$ and $b \rightarrow 0$? (Recall that the mean of a $\text{Ga}(a, b)$ distribution is a/b .)

Exercise 3.8 MLE for the uniform distribution

(Source: Kaelbling.) Consider a uniform distribution centered on 0 with width $2a$. The density function is given by

$$p(x) = \frac{1}{2a} I(x \in [-a, a]) \quad (3.92)$$

- Given a data set $x_1, \dots, x_n$, what is the maximum likelihood estimate of a (call it $\hat{a}$)?
- What probability would the model assign to a new data point x_{n+1} using $\hat{a}$?
- Do you see any problem with the above approach? Briefly suggest (in words) a better approach.

Exercise 3.9 Bayesian analysis of the uniform distribution

Consider the uniform distribution $\text{Unif}(0, \theta)$. The maximum likelihood estimate is $\hat{\theta} = \max(\mathcal{D})$, as we saw in Exercise 3.8, but this is unsuitable for predicting future data since it puts zero probability mass outside the training data. In this exercise, we will perform a Bayesian analysis of the uniform distribution (following (Minka 2001a)). The conjugate prior is the Pareto distribution, $p(\theta) = \text{Pareto}(\theta|b, K)$, defined in Section 2.4.6. Given a Pareto prior, the joint distribution of θ and $\mathcal{D} = (x_1, \dots, x_N)$ is

$$p(\mathcal{D}, \theta) = \frac{Kb^K}{\theta^{N+K+1}} \mathbb{I}(\theta \geq \max(\mathcal{D})) \quad (3.93)$$

Let $m = \max(\mathcal{D})$. The evidence (the probability that all N samples came from the same uniform distribution) is

$$p(\mathcal{D}) = \int_m^\infty \frac{Kb^K}{\theta^{N+K+1}} d\theta \quad (3.94)$$

$$= \begin{cases} \frac{K}{(N+K)b^N} & \text{if } m \leq b \\ \frac{Kb^K}{(N+K)m^{N+K}} & \text{if } m > b \end{cases} \quad (3.95)$$

Derive the posterior $p(\theta|\mathcal{D})$, and show that it can be expressed as a Pareto distribution.

Exercise 3.10 Taxicab (tramcar) problem

Suppose you arrive in a new city and see a taxi numbered 100. How many taxis are there in this city? Let us assume taxis are numbered sequentially as integers starting from 0, up to some unknown upper bound θ . (We number taxis from 0 for simplicity; we can also count from 1 without changing the analysis.) Hence the likelihood function is $p(x) = U(0, \theta)$, the uniform distribution. The goal is to estimate θ . We will use the Bayesian analysis from Exercise 3.9.

- Suppose we see one taxi numbered 100, so $\mathcal{D} = \{100\}$, $m = 100$, $N = 1$. Using an (improper) non-informative prior on θ of the form $p(\theta) = Pa(\theta|0, 0) \propto 1/\theta$, what is the posterior $p(\theta|\mathcal{D})$?
- Compute the posterior mean, mode and median number of taxis in the city, if such quantities exist.
- Rather than trying to compute a point estimate of the number of taxis, we can compute the predictive density over the next taxicab number using

$$p(D'|D, \alpha) = \int p(D'|\theta)p(\theta|D, \alpha)d\theta = p(D'|\beta) \quad (3.96)$$

where $\alpha = (b, K)$ are the hyper-parameters, $\beta = (c, N + K)$ are the updated hyper-parameters. Now consider the case $D = \{m\}$, and $D' = \{x\}$. Using Equation 3.95, write down an expression for

$$p(x|D, \alpha) \quad (3.97)$$

As above, use a non-informative prior $b = K = 0$.

- Use the predictive density formula to compute the probability that the next taxi you will see (say, the next day) has number 100, 50 or 150, i.e., compute $p(x = 100|D, \alpha)$, $p(x = 50|D, \alpha)$, $p(x = 150|D, \alpha)$.
- Briefly describe (1-2 sentences) some ways we might make the model more accurate at prediction.

Exercise 3.11 Bayesian analysis of the exponential distribution

A lifetime X of a machine is modeled by an exponential distribution with unknown parameter θ . The likelihood is $p(x|\theta) = \theta e^{-\theta x}$ for $x \geq 0$, $\theta > 0$.

- Show that the MLE is $\hat{\theta} = 1/\bar{x}$, where $\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$.
- Suppose we observe $X_1 = 5, X_2 = 6, X_3 = 4$ (the lifetimes (in years) of 3 different iid machines). What is the MLE given this data?
- Assume that an expert believes θ should have a prior distribution that is also exponential

$$p(\theta) = \text{Expon}(\theta|\lambda) \quad (3.98)$$

Choose the prior parameter, call it $\hat{\lambda}$, such that $\mathbb{E}[\theta] = 1/3$. Hint: recall that the Gamma distribution has the form

$$\text{Ga}(\theta|a, b) \propto \theta^{a-1} e^{-\theta b} \quad (3.99)$$

and its mean is a/b .

- What is the posterior, $p(\theta|\mathcal{D}, \hat{\lambda})$?
- Is the exponential prior conjugate to the exponential likelihood?
- What is the posterior mean, $\mathbb{E}[\theta|\mathcal{D}, \hat{\lambda}]$?
- Explain why the MLE and posterior mean differ. Which is more reasonable in this example?

Exercise 3.12 MAP estimation for the Bernoulli with non-conjugate priors

(Source: Jaakkola.) In the book, we discussed Bayesian inference of a Bernoulli rate parameter with the prior $p(\theta) = \text{Beta}(\theta|\alpha, \beta)$. We know that, with this prior, the MAP estimate is given by

$$\hat{\theta} = \frac{N_1 + \alpha - 1}{N + \alpha + \beta - 2} \quad (3.100)$$

where N_1 is the number of heads, N_0 is the number of tails, and $N = N_0 + N_1$ is the total number of trials.

- Now consider the following prior, that believes the coin is fair, or is slightly biased towards tails:

$$p(\theta) = \begin{cases} 0.5 & \text{if } \theta = 0.5 \\ 0.5 & \text{if } \theta = 0.4 \\ 0 & \text{otherwise} \end{cases} \quad (3.101)$$

Derive the MAP estimate under this prior as a function of N_1 and N .

- Suppose the true parameter is $\theta = 0.41$. Which prior leads to a better estimate when N is small? Which prior leads to a better estimate when N is large?

Exercise 3.13 Posterior predictive distribution for a batch of data with the dirichlet-multinomial model

In Equation 3.51, we gave the the posterior predictive distribution for a single multinomial trial using a dirichlet prior. Now consider predicting a *batch* of new data, $\tilde{\mathcal{D}} = (X_1, \dots, X_m)$, consisting of m single multinomial trials (think of predicting the next m words in a sentence, assuming they are drawn iid). Derive an expression for

$$p(\tilde{\mathcal{D}}|\mathcal{D}, \alpha) \quad (3.102)$$

Your answer should be a function of α , and the old and new counts (sufficient statistics), defined as

$$N_k^{old} = \sum_{i \in \mathcal{D}} I(x_i = k) \quad (3.103)$$

$$N_k^{new} = \sum_{i \in \tilde{\mathcal{D}}} I(x_i = k) \quad (3.104)$$

Hint: recall that, for a vector of counts, $N_{1:K}$, the marginal likelihood (evidence) is given by

$$p(\mathcal{D}|\alpha) = \frac{\Gamma(\alpha)}{\Gamma(N + \alpha)} \prod_k \frac{\Gamma(N_k + \alpha_k)}{\Gamma(\alpha_k)} \quad (3.105)$$

where $\alpha = \sum_k \alpha_k$ and $N = \sum_k N_k$.

Exercise 3.14 Posterior predictive for Dirichlet-multinomial

(Source: Koller.).

- Suppose we compute the empirical distribution over letters of the Roman alphabet plus the space character (a distribution over 27 values) from 2000 samples. Suppose we see the letter “e” 260 times. What is $p(x_{2001} = e|\mathcal{D})$, if we assume $\theta \sim \text{Dir}(\alpha_1, \dots, \alpha_{27})$, where $\alpha_k = 10$ for all k ?
- Suppose, in the 2000 samples, we saw “e” 260 times, “a” 100 times, and “p” 87 times. What is $p(x_{2001} = p, x_{2002} = a|\mathcal{D})$, if we assume $\theta \sim \text{Dir}(\alpha_1, \dots, \alpha_{27})$, where $\alpha_k = 10$ for all k ? Show your work.

Exercise 3.15 Setting the beta hyper-parameters

Suppose $\theta \sim \beta(\alpha_1, \alpha_2)$ and we believe that $\mathbb{E}[\theta] = m$ and $\text{var}[\theta] = v$. Using Equation 2.62, solve for α_1 and α_2 in terms of m and v . What values do you get if $m = 0.7$ and $v = 0.2^2$?

Exercise 3.16 Setting the beta hyper-parameters II

(Source: Draper.) Suppose $\theta \sim \beta(\alpha_1, \alpha_2)$ and we believe that $\mathbb{E}[\theta] = m$ and $p(\ell < \theta < u) = 0.95$. Write a program that can solve for α_1 and α_2 in terms of m , ℓ and u . Hint: write α_2 as a function of α_1 and m , so the pdf only has one unknown; then write down the probability mass contained in the interval as an integral, and minimize its squared discrepancy from 0.95. What values do you get if $m = 0.15$, $\ell = 0.05$ and $u = 0.3$? What is the equivalent sample size of this prior?

Exercise 3.17 Marginal likelihood for beta-binomial under uniform prior

Suppose we toss a coin N times and observe N_1 heads. Let $N_1 \sim \text{Bin}(N, \theta)$ and $\theta \sim \text{Beta}(1, 1)$. Show that the marginal likelihood is $p(N_1|N) = 1/(N + 1)$. Hint: $\Gamma(x + 1) = x!$ if x is an integer.

Exercise 3.18 Bayes factor for coin tossing

Suppose we toss a coin $N = 10$ times and observe $N_1 = 9$ heads. Let the null hypothesis be that the coin is fair, and the alternative be that the coin can have any bias, so $p(\theta) = \text{Unif}(0, 1)$. Derive the Bayes factor $BF_{1,0}$ in favor of the biased coin hypothesis. What if $N = 100$ and $N_1 = 90$? Hint: see Exercise 3.17.

Exercise 3.19 Irrelevant features with naive Bayes

(Source: Jaakkola.) Let $x_{iw} = 1$ if word w occurs in document i and $x_{iw} = 0$ otherwise. Let θ_{cw} be the estimated probability that word w occurs in documents of class c . Then the log-likelihood that document

$\mathbf{x}$ belongs to class c is

$$\log p(\mathbf{x}_i|c, \theta) = \log \prod_{w=1}^W \theta_{cw}^{x_{iw}} (1 - \theta_{cw})^{1-x_{iw}} \quad (3.106)$$

$$= \sum_{w=1}^W x_{iw} \log \theta_{cw} + (1 - x_{iw}) \log(1 - \theta_{cw}) \quad (3.107)$$

$$= \sum_{w=1}^W x_{iw} \log \frac{\theta_{cw}}{1 - \theta_{cw}} + \sum_w \log(1 - \theta_{cw}) \quad (3.108)$$

where W is the number of words in the vocabulary. We can write this more succinctly as

$$\log p(\mathbf{x}_i|c, \theta) = \phi(\mathbf{x}_i)^T \beta_c \quad (3.109)$$

where $\mathbf{x}_i = (x_{i1}, \dots, x_{iW})$ is a bit vector, $\phi(\mathbf{x}_i) = (\mathbf{x}_i, 1)$, and

$$\beta_c = (\log \frac{\theta_{c1}}{1 - \theta_{c1}}, \dots, \log \frac{\theta_{cW}}{1 - \theta_{cW}}, \sum_w \log(1 - \theta_{cw}))^T \quad (3.110)$$

We see that this is a linear classifier, since the class-conditional density is a linear function (an inner product) of the parameters β_c .

- Assuming $p(C = 1) = p(C = 2) = 0.5$, write down an expression for the log posterior odds ratio, $\log_2 \frac{p(c=1|\mathbf{x}_i)}{p(c=2|\mathbf{x}_i)}$, in terms of the features $\phi(\mathbf{x}_i)$ and the parameters β_1 and β_2 .
- Intuitively, words that occur in both classes are not very “discriminative”, and therefore should not affect our beliefs about the class label. Consider a particular word w . State the conditions on $\theta_{1,w}$ and $\theta_{2,w}$ (or equivalently the conditions on $\beta_{1,w}, \beta_{2,w}$) under which the presence or absence of w in a test document will have no effect on the class posterior (such a word will be ignored by the classifier). Hint: using your previous result, figure out when the posterior odds ratio is 0.5/0.5.
- The posterior mean estimate of θ , using a Beta(1,1) prior, is given by

$$\hat{\theta}_{cw} = \frac{1 + \sum_{i \in c} x_{iw}}{2 + n_c} \quad (3.111)$$

where the sum is over the n_c documents in class c . Consider a particular word w , and suppose it always occurs in every document (regardless of class). Let there be n_1 documents of class 1 and n_2 be the number of documents in class 2, where $n_1 \neq n_2$ (since e.g., we get much more non-spam than spam; this is an example of class imbalance). If we use the above estimate for θ_{cw} , will word w be ignored by our classifier? Explain why or why not.

- What other ways can you think of which encourage “irrelevant” words to be ignored?

Exercise 3.20 Class conditional densities for binary data

Consider a generative classifier for C classes with class conditional density $p(\mathbf{x}|y)$ and uniform class prior $p(y)$. Suppose all the D features are binary, $x_j \in \{0, 1\}$. If we assume all the features are conditionally independent (the naive Bayes assumption), we can write

$$p(\mathbf{x}|y = c) = \prod_{j=1}^D \text{Ber}(x_j|\theta_{jc}) \quad (3.112)$$

This requires DC parameters.

- a. Now consider a different model, which we will call the “full” model, in which all the features are fully dependent (i.e., we make no factorization assumptions). How might we represent $p(\mathbf{x}|y = c)$ in this case? How many parameters are needed to represent $p(\mathbf{x}|y = c)$?
- b. Assume the number of features D is fixed. Let there be N training cases. If the sample size N is very small, which model (naive Bayes or full) is likely to give lower test set error, and why?
- c. If the sample size N is very large, which model (naive Bayes or full) is likely to give lower test set error, and why?
- d. What is the computational complexity of fitting the full and naive Bayes models as a function of N and D ? Use big-Oh notation. (Fitting the model here means computing the MLE or MAP parameter estimates. You may assume you can convert a D -bit vector to an array index in $O(D)$ time.)
- e. What is the computational complexity of applying the full and naive Bayes models at test time to a single test case?
- f. Suppose the test case has missing data. Let $\mathbf{x}_v$ be the visible features of size v , and $\mathbf{x}_h$ be the hidden (missing) features of size h , where $v + h = D$. What is the computational complexity of computing $p(y|\mathbf{x}_v, \hat{\theta})$ for the full and naive Bayes models, as a function of v and h ?

Exercise 3.21 Mutual information for naive Bayes classifiers with binary features

Derive Equation 3.76.

Exercise 3.22 Fitting a naive bayes spam filter by hand

(Source: Daphne Koller.). Consider a Naive Bayes model (multivariate Bernoulli version) for spam classification with the vocabulary $V = \{\text{"secret", "offer", "low", "price", "valued", "customer", "today", "dollar", "million", "sports", "is", "for", "play", "healthy", "pizza"}\}$. We have the following example spam messages "million dollar offer", "secret offer today", "secret is secret" and normal messages, "low price for valued customer", "play secret sports today", "sports is healthy", "low price pizza". Give the MLEs for the following parameters: θ_{spam} , $\theta_{\text{secret}|\text{spam}}$, $\theta_{\text{secret}|\text{non-spam}}$, $\theta_{\text{sports}|\text{non-spam}}$, $\theta_{\text{dollar}|\text{spam}}$.

4 *Gaussian models*

4.1 Introduction

In this chapter, we discuss the **multivariate Gaussian** or **multivariate normal (MVN)**, which is the most widely used joint probability density function for continuous variables. It will form the basis for many of the models we will encounter in later chapters.

Unfortunately, the level of mathematics in this chapter is higher than in many other chapters. In particular, we rely heavily on linear algebra and matrix calculus. This is the price one must pay in order to deal with high-dimensional data. Beginners may choose to skip sections marked with a *. In addition, since there are so many equations in this chapter, we have put a box around those that are particularly important.

4.1.1 Notation

Let us briefly say a few words about notation. We denote vectors by boldface lower case letters, such as $\mathbf{x}$. We denote matrices by boldface upper case letters, such as $\mathbf{X}$. We denote entries in a matrix by non-bold upper case letters, such as X_{ij} .

All vectors are assumed to be column vectors unless noted otherwise. We use $[x_1, \dots, x_D]$ to denote a column vector created by stacking D scalars. Similarly, if we write $\mathbf{x} = [\mathbf{x}_1, \dots, \mathbf{x}_D]$, where the left hand side is a tall column vector, we mean to stack the $\mathbf{x}_i$ along the rows; this is usually written as $\mathbf{x} = (\mathbf{x}_1^T, \dots, \mathbf{x}_D^T)^T$, but that is rather ugly. If we write $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_D]$, where the left hand side is a matrix, we mean to stack the $\mathbf{x}_i$ along the columns, creating a matrix.

4.1.2 Basics

Recall from Section 2.5.2 that the pdf for an MVN in D dimensions is defined by the following:

$$\mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) \triangleq \frac{1}{(2\pi)^{D/2}|\boldsymbol{\Sigma}|^{1/2}} \exp \left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}) \right] \quad (4.1)$$

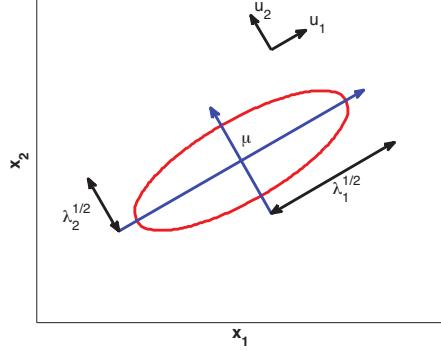


Figure 4.1 Visualization of a 2 dimensional Gaussian density. The major and minor axes of the ellipse are defined by the first two eigenvectors of the covariance matrix, namely $\mathbf{u}_1$ and $\mathbf{u}_2$. Based on Figure 2.7 of (Bishop 2006a).

The expression inside the exponent is the Mahalanobis distance between a data vector $\mathbf{x}$ and the mean vector $\boldsymbol{\mu}$. We can gain a better understanding of this quantity by performing an **eigendecomposition** of $\boldsymbol{\Sigma}$. That is, we write $\boldsymbol{\Sigma} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^T$, where $\mathbf{U}$ is an orthonormal matrix of eigenvectors satisfying $\mathbf{U}^T\mathbf{U} = \mathbf{I}$, and $\boldsymbol{\Lambda}$ is a diagonal matrix of eigenvalues.

Using the eigendecomposition, we have that

$$\boldsymbol{\Sigma}^{-1} = \mathbf{U}^{-T}\boldsymbol{\Lambda}^{-1}\mathbf{U}^{-1} = \mathbf{U}\boldsymbol{\Lambda}^{-1}\mathbf{U}^T = \sum_{i=1}^D \frac{1}{\lambda_i} \mathbf{u}_i \mathbf{u}_i^T \quad (4.2)$$

where $\mathbf{u}_i$ is the i 'th column of $\mathbf{U}$, containing the i 'th eigenvector. Hence we can rewrite the Mahalanobis distance as follows:

$$(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) = (\mathbf{x} - \boldsymbol{\mu})^T \left(\sum_{i=1}^D \frac{1}{\lambda_i} \mathbf{u}_i \mathbf{u}_i^T \right) (\mathbf{x} - \boldsymbol{\mu}) \quad (4.3)$$

$$= \sum_{i=1}^D \frac{1}{\lambda_i} (\mathbf{x} - \boldsymbol{\mu})^T \mathbf{u}_i \mathbf{u}_i^T (\mathbf{x} - \boldsymbol{\mu}) = \sum_{i=1}^D \frac{y_i^2}{\lambda_i} \quad (4.4)$$

where $y_i \triangleq \mathbf{u}_i^T (\mathbf{x} - \boldsymbol{\mu})$. Recall that the equation for an ellipse in 2d is

$$\frac{y_1^2}{\lambda_1} + \frac{y_2^2}{\lambda_2} = 1 \quad (4.5)$$

Hence we see that the contours of equal probability density of a Gaussian lie along ellipses. This is illustrated in Figure 4.1. The eigenvectors determine the orientation of the ellipse, and the eigenvalues determine how elongated it is.

In general, we see that the Mahalanobis distance corresponds to Euclidean distance in a transformed coordinate system, where we shift by $\boldsymbol{\mu}$ and rotate by $\mathbf{U}$.

4.1.3 MLE for an MVN

We now describe one way to estimate the parameters of an MVN, using MLE. In later sections, we will discuss Bayesian inference for the parameters, which can mitigate overfitting, and can provide a measure of confidence in our estimates.

Theorem 4.1.1 (MLE for a Gaussian). *If we have N iid samples $\mathbf{x}_i \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, then the MLE for the parameters is given by*

$$\hat{\boldsymbol{\mu}}_{mle} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i \triangleq \bar{\mathbf{x}} \quad (4.6)$$

$$\hat{\boldsymbol{\Sigma}}_{mle} = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T = \frac{1}{N} \left(\sum_{i=1}^N \mathbf{x}_i \mathbf{x}_i^T \right) - \bar{\mathbf{x}} \bar{\mathbf{x}}^T \quad (4.7)$$

That is, the MLE is just the empirical mean and empirical covariance. In the univariate case, we get the following familiar results:

$$\hat{\mu} = \frac{1}{N} \sum_i x_i = \bar{x} \quad (4.8)$$

$$\hat{\sigma}^2 = \frac{1}{N} \sum_i (x_i - \bar{x})^2 = \left(\frac{1}{N} \sum_i x_i^2 \right) - (\bar{x})^2 \quad (4.9)$$

4.1.3.1 Proof *

To prove this result, we will need several results from matrix algebra, which we summarize below. In the equations, $\mathbf{a}$ and $\mathbf{b}$ are vectors, and $\mathbf{A}$ and $\mathbf{B}$ are matrices. Also, the notation $\text{tr}(\mathbf{A})$ refers to the **trace** of a matrix, which is the sum of its diagonals: $\text{tr}(\mathbf{A}) = \sum_i A_{ii}$.

$$\begin{aligned} \frac{\partial(\mathbf{b}^T \mathbf{a})}{\partial \mathbf{a}} &= \mathbf{b} \\ \frac{\partial(\mathbf{a}^T \mathbf{A} \mathbf{a})}{\partial \mathbf{a}} &= (\mathbf{A} + \mathbf{A}^T) \mathbf{a} \\ \frac{\partial}{\partial \mathbf{A}} \text{tr}(\mathbf{B} \mathbf{A}) &= \mathbf{B}^T \\ \frac{\partial}{\partial \mathbf{A}} \log |\mathbf{A}| &= \mathbf{A}^{-T} \triangleq (\mathbf{A}^{-1})^T \\ \text{tr}(\mathbf{A} \mathbf{B} \mathbf{C}) &= \text{tr}(\mathbf{C} \mathbf{A} \mathbf{B}) = \text{tr}(\mathbf{B} \mathbf{C} \mathbf{A}) \end{aligned} \quad (4.10)$$

The last equation is called the **cyclic permutation property** of the trace operator. Using this, we can derive the widely used **trace trick**, which reorders the scalar inner product $\mathbf{x}^T \mathbf{A} \mathbf{x}$ as follows

$$\mathbf{x}^T \mathbf{A} \mathbf{x} = \text{tr}(\mathbf{x}^T \mathbf{A} \mathbf{x}) = \text{tr}(\mathbf{x} \mathbf{x}^T \mathbf{A}) = \text{tr}(\mathbf{A} \mathbf{x} \mathbf{x}^T) \quad (4.11)$$

Proof. We can now begin with the proof. The log-likelihood is

$$\ell(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \log p(\mathcal{D}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{N}{2} \log |\boldsymbol{\Lambda}| - \frac{1}{2} \sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu})^T \boldsymbol{\Lambda} (\mathbf{x}_i - \boldsymbol{\mu}) \quad (4.12)$$

where $\boldsymbol{\Lambda} = \boldsymbol{\Sigma}^{-1}$ is the precision matrix.

Using the substitution $\mathbf{y}_i = \mathbf{x}_i - \boldsymbol{\mu}$ and the chain rule of calculus, we have

$$\frac{\partial}{\partial \boldsymbol{\mu}} (\mathbf{x}_i - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x}_i - \boldsymbol{\mu}) = \frac{\partial}{\partial \mathbf{y}_i} \mathbf{y}_i^T \boldsymbol{\Sigma}^{-1} \mathbf{y}_i \frac{\partial \mathbf{y}_i}{\partial \boldsymbol{\mu}} \quad (4.13)$$

$$= -1(\boldsymbol{\Sigma}^{-1} + \boldsymbol{\Sigma}^{-T}) \mathbf{y}_i \quad (4.14)$$

Hence

$$\frac{\partial}{\partial \boldsymbol{\mu}} \ell(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = -\frac{1}{2} \sum_{i=1}^N -2 \boldsymbol{\Sigma}^{-1} (\mathbf{x}_i - \boldsymbol{\mu}) = \boldsymbol{\Sigma}^{-1} \sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu}) = 0 \quad (4.15)$$

$$\hat{\boldsymbol{\mu}} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i = \bar{\mathbf{x}} \quad (4.16)$$

So the MLE of $\boldsymbol{\mu}$ is just the empirical mean.

Now we can use the trace-trick to rewrite the log-likelihood for $\boldsymbol{\Lambda}$ as follows:

$$\ell(\boldsymbol{\Lambda}) = \frac{N}{2} \log |\boldsymbol{\Lambda}| - \frac{1}{2} \sum_i \text{tr}[(\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^T \boldsymbol{\Lambda}] \quad (4.17)$$

$$= \frac{N}{2} \log |\boldsymbol{\Lambda}| - \frac{1}{2} \text{tr}[\mathbf{S}_\mu \boldsymbol{\Lambda}] \quad (4.18)$$

$$(4.19)$$

where

$$\mathbf{S}_\mu \triangleq \sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^T \quad (4.20)$$

is the scatter matrix centered on $\boldsymbol{\mu}$. Taking derivatives of this expression with respect to $\boldsymbol{\Lambda}$ yields

$$\frac{\partial \ell(\boldsymbol{\Lambda})}{\partial \boldsymbol{\Lambda}} = \frac{N}{2} \boldsymbol{\Lambda}^{-T} - \frac{1}{2} \mathbf{S}_\mu^T = 0 \quad (4.21)$$

$$\boldsymbol{\Lambda}^{-T} = \boldsymbol{\Lambda}^{-1} = \boldsymbol{\Sigma} = \frac{1}{N} \mathbf{S}_\mu \quad (4.22)$$

so

$$\hat{\boldsymbol{\Sigma}} = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^T \quad (4.23)$$

which is just the empirical covariance matrix centered on $\boldsymbol{\mu}$. If we plug-in the MLE $\boldsymbol{\mu} = \bar{\mathbf{x}}$ (since both parameters must be simultaneously optimized), we get the standard equation for the MLE of a covariance matrix. $\square$

4.1.4 Maximum entropy derivation of the Gaussian *

In this section, we show that the multivariate Gaussian is the distribution with maximum entropy subject to having a specified mean and covariance (see also Section 9.2.6). This is one reason the Gaussian is so widely used: the first two moments are usually all that we can reliably estimate from data, so we want a distribution that captures these properties, but otherwise makes as few additional assumptions as possible.

To simplify notation, we will assume the mean is zero. The pdf has the form

$$p(\mathbf{x}) = \frac{1}{Z} \exp\left(-\frac{1}{2} \mathbf{x}^T \boldsymbol{\Sigma}^{-1} \mathbf{x}\right) \quad (4.24)$$

If we define $f_{ij}(\mathbf{x}) = x_i x_j$ and $\lambda_{ij} = \frac{1}{2}(\boldsymbol{\Sigma}^{-1})_{ij}$, for $i, j \in \{1, \dots, D\}$, we see that this is in the same form as Equation 9.74. The (differential) entropy of this distribution (using log base e) is given by

$$h(\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})) = \frac{1}{2} \ln [(2\pi e)^D |\boldsymbol{\Sigma}|] \quad (4.25)$$

We now show the MVN has maximum entropy amongst all distributions with a specified covariance $\boldsymbol{\Sigma}$.

Theorem 4.1.2. *Let $q(\mathbf{x})$ be any density satisfying $\int q(\mathbf{x}) x_i x_j = \Sigma_{ij}$. Let $p = \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$. Then $h(q) \leq h(p)$.*

Proof. (From (Cover and Thomas 1991, p234).) We have

$$0 \leq \mathbb{KL}(q||p) = \int q(\mathbf{x}) \log \frac{q(\mathbf{x})}{p(\mathbf{x})} d\mathbf{x} \quad (4.26)$$

$$= -h(q) - \int q(\mathbf{x}) \log p(\mathbf{x}) d\mathbf{x} \quad (4.27)$$

$$=^* -h(q) - \int p(\mathbf{x}) \log p(\mathbf{x}) d\mathbf{x} \quad (4.28)$$

$$= -h(q) + h(p) \quad (4.29)$$

where the key step in Equation 4.28 (marked with a *) follows since q and p yield the same moments for the quadratic form encoded by $\log p(\mathbf{x})$. $\square$

4.2 Gaussian discriminant analysis

One important application of MVNs is to define the the class conditional densities in a generative classifier, i.e.,

$$p(\mathbf{x}|y = c, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c) \quad (4.30)$$

The resulting technique is called (Gaussian) **discriminant analysis** or GDA (even though it is a generative, not discriminative, classifier — see Section 8.6 for more on this distinction). If $\boldsymbol{\Sigma}_c$ is diagonal, this is equivalent to naive Bayes.

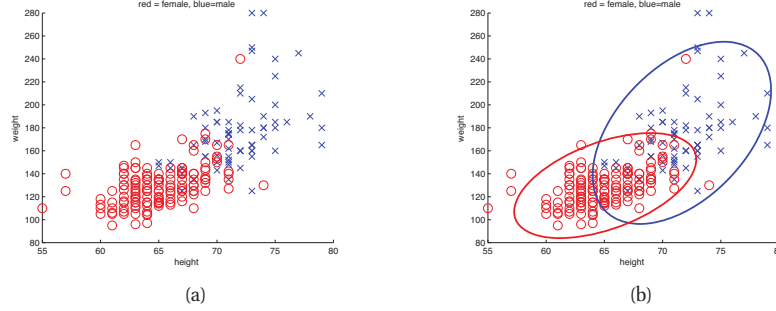


Figure 4.2 (a) Height/weight data. (b) Visualization of 2d Gaussians fit to each class. 95% of the probability mass is inside the ellipse. Figure generated by `gaussHeightWeight`.

We can classify a feature vector using the following decision rule, derived from Equation 2.13:

$$\hat{y}(\mathbf{x}) = \underset{c}{\operatorname{argmax}} [\log p(y = c|\boldsymbol{\pi}) + \log p(\mathbf{x}|\boldsymbol{\theta}_c)] \quad (4.31)$$

When we compute the probability of $\mathbf{x}$ under each class conditional density, we are measuring the distance from $\mathbf{x}$ to the center of each class, $\boldsymbol{\mu}_c$, using Mahalanobis distance. This can be thought of as a **nearest centroids classifier**.

As an example, Figure 4.2 shows two Gaussian class-conditional densities in 2d, representing the height and weight of men and women. We can see that the features are correlated, as is to be expected (tall people tend to weigh more). The ellipses for each class contain 95% of the probability mass. If we have a uniform prior over classes, we can classify a new test vector as follows:

$$\hat{y}(\mathbf{x}) = \underset{c}{\operatorname{argmin}} (\mathbf{x} - \boldsymbol{\mu}_c)^T \boldsymbol{\Sigma}_c^{-1} (\mathbf{x} - \boldsymbol{\mu}_c) \quad (4.32)$$

4.2.1 Quadratic discriminant analysis (QDA)

The posterior over class labels is given by Equation 2.13. We can gain further insight into this model by plugging in the definition of the Gaussian density, as follows:

$$p(y = c|\mathbf{x}, \boldsymbol{\theta}) = \frac{\pi_c |2\pi \boldsymbol{\Sigma}_c|^{-\frac{1}{2}} \exp \left[-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_c)^T \boldsymbol{\Sigma}_c^{-1} (\mathbf{x} - \boldsymbol{\mu}_c) \right]}{\sum_{c'} \pi_{c'} |2\pi \boldsymbol{\Sigma}_{c'}|^{-\frac{1}{2}} \exp \left[-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_{c'})^T \boldsymbol{\Sigma}_{c'}^{-1} (\mathbf{x} - \boldsymbol{\mu}_{c'}) \right]} \quad (4.33)$$

Thresholding this results in a quadratic function of $\mathbf{x}$. The result is known as **quadratic discriminant analysis** (QDA). Figure 4.3 gives some examples of what the decision boundaries look like in 2D.

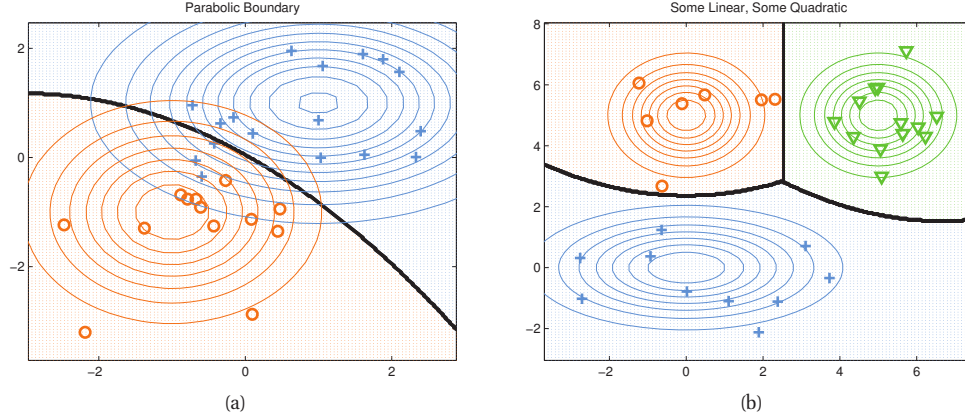


Figure 4.3 Quadratic decision boundaries in 2D for the 2 and 3 class case. Figure generated by `discrimAnalysisDboundariesDemo`.

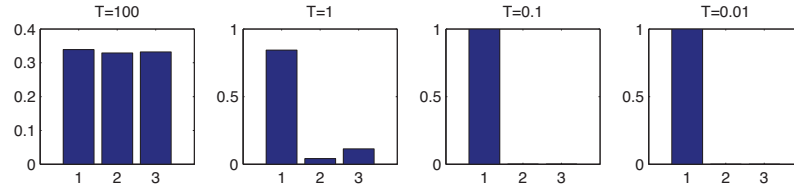


Figure 4.4 Softmax distribution $\mathcal{S}(\boldsymbol{\eta}/T)$, where $\boldsymbol{\eta} = (3, 0, 1)$, at different temperatures T . When the temperature is high (left), the distribution is uniform, whereas when the temperature is low (right), the distribution is “spiky”, with all its mass on the largest element. Figure generated by `softmaxDemo2`.

4.2.2 Linear discriminant analysis (LDA)

We now consider a special case in which the covariance matrices are **tied** or **shared** across classes, $\Sigma_c = \Sigma$. In this case, we can simplify Equation 4.33 as follows:

$$p(y = c | \mathbf{x}, \boldsymbol{\theta}) \propto \pi_c \exp \left[\boldsymbol{\mu}_c^T \Sigma^{-1} \mathbf{x} - \frac{1}{2} \mathbf{x}^T \Sigma^{-1} \mathbf{x} - \frac{1}{2} \boldsymbol{\mu}_c^T \Sigma^{-1} \boldsymbol{\mu}_c \right] \quad (4.34)$$

$$= \exp \left[\boldsymbol{\mu}_c^T \Sigma^{-1} \mathbf{x} - \frac{1}{2} \boldsymbol{\mu}_c^T \Sigma^{-1} \boldsymbol{\mu}_c + \log \pi_c \right] \exp \left[-\frac{1}{2} \mathbf{x}^T \Sigma^{-1} \mathbf{x} \right] \quad (4.35)$$

Since the quadratic term $\mathbf{x}^T \Sigma^{-1} \mathbf{x}$ is independent of c , it will cancel out in the numerator and denominator. If we define

$$\gamma_c = -\frac{1}{2} \boldsymbol{\mu}_c^T \Sigma^{-1} \boldsymbol{\mu}_c + \log \pi_c \quad (4.36)$$

$$\boldsymbol{\beta}_c = \Sigma^{-1} \boldsymbol{\mu}_c \quad (4.37)$$

then we can write

$$p(y = c | \mathbf{x}, \boldsymbol{\theta}) = \frac{e^{\boldsymbol{\beta}_c^T \mathbf{x} + \gamma_c}}{\sum_{c'} e^{\boldsymbol{\beta}_{c'}^T \mathbf{x} + \gamma_{c'}}} = \mathcal{S}(\boldsymbol{\eta})_c \quad (4.38)$$

where $\boldsymbol{\eta} = [\boldsymbol{\beta}_1^T \mathbf{x} + \gamma_1, \dots, \boldsymbol{\beta}_C^T \mathbf{x} + \gamma_C]$, and $\mathcal{S}$ is the **softmax** function, defined as follows:

$$\mathcal{S}(\boldsymbol{\eta})_c = \frac{e^{\eta_c}}{\sum_{c'=1}^C e^{\eta_{c'}}} \quad (4.39)$$

The softmax function is so-called since it acts a bit like the max function. To see this, let us divide each η_c by a constant T called the **temperature**. Then as $T \rightarrow 0$, we find

$$\mathcal{S}(\boldsymbol{\eta}/T)_c = \begin{cases} 1.0 & \text{if } c = \operatorname{argmax}_{c'} \eta_{c'} \\ 0.0 & \text{otherwise} \end{cases} \quad (4.40)$$

In other words, at low temperatures, the distribution spends essentially all of its time in the most probable state, whereas at high temperatures, it visits all states uniformly. See Figure 4.4 for an illustration. Note that this terminology comes from the area of statistical physics, where it is common to use the **Boltzmann distribution**, which has the same form as the softmax function.

An interesting property of Equation 4.38 is that, if we take logs, we end up with a linear function of $\mathbf{x}$. (The reason it is linear is because the $\mathbf{x}^T \boldsymbol{\Sigma}^{-1} \mathbf{x}$ cancels from the numerator and denominator.) Thus the decision boundary between any two classes, say c and c' , will be a straight line. Hence this technique is called **linear discriminant analysis** or **LDA**.¹ We can derive the form of this line as follows:

$$p(y = c | \mathbf{x}, \boldsymbol{\theta}) = p(y = c' | \mathbf{x}, \boldsymbol{\theta}) \quad (4.41)$$

$$\boldsymbol{\beta}_c^T \mathbf{x} + \gamma_c = \boldsymbol{\beta}_{c'}^T \mathbf{x} + \gamma_{c'} \quad (4.42)$$

$$\mathbf{x}^T (\boldsymbol{\beta}_{c'} - \boldsymbol{\beta}_c) = \gamma_{c'} - \gamma_c \quad (4.43)$$

See Figure 4.5 for some examples.

An alternative to fitting an LDA model and then deriving the class posterior is to directly fit $p(y | \mathbf{x}, \mathbf{W}) = \text{Cat}(y | \mathbf{W}\mathbf{x})$ for some $C \times D$ weight matrix $\mathbf{W}$. This is called **multi-class logistic regression**, or **multinomial logistic regression**.² We will discuss this model in detail in Section 8.2. The difference between the two approaches is explained in Section 8.6.

4.2.3 Two-class LDA

To gain further insight into the meaning of these equations, let us consider the binary case. In this case, the posterior is given by

$$p(y = 1 | \mathbf{x}, \boldsymbol{\theta}) = \frac{e^{\boldsymbol{\beta}_1^T \mathbf{x} + \gamma_1}}{e^{\boldsymbol{\beta}_1^T \mathbf{x} + \gamma_1} + e^{\boldsymbol{\beta}_0^T \mathbf{x} + \gamma_0}} \quad (4.44)$$

$$= \frac{1}{1 + e^{(\boldsymbol{\beta}_0 - \boldsymbol{\beta}_1)^T \mathbf{x} + (\gamma_0 - \gamma_1)}} = \text{sigm}((\boldsymbol{\beta}_1 - \boldsymbol{\beta}_0)^T \mathbf{x} + (\gamma_1 - \gamma_0)) \quad (4.45)$$

1. The abbreviation “LDA”, could either stand for “linear discriminant analysis” or “latent Dirichlet allocation” (Section 27.3). We hope the meaning is clear from text.

2. In the language modeling community, this model is called a **maximum entropy** model, for reasons explained in Section 9.2.6.

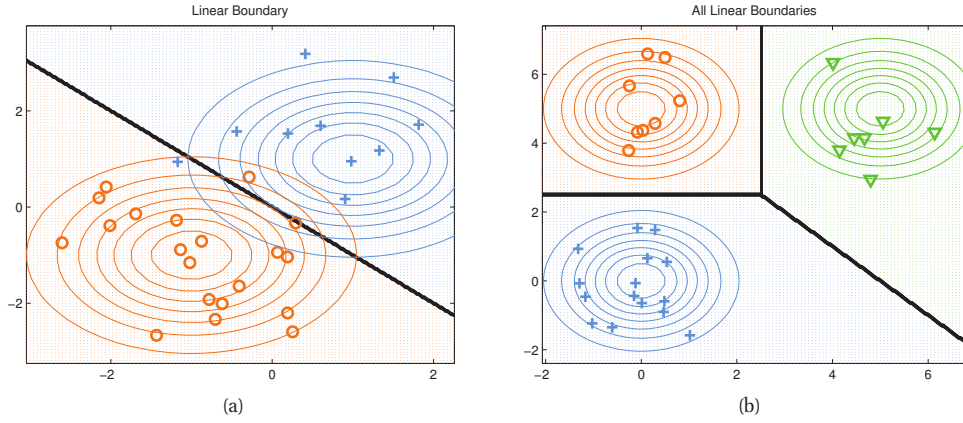


Figure 4.5 Linear decision boundaries in 2D for the 2 and 3 class case. Figure generated by `discrimAnalysisDBoundariesDemo`.

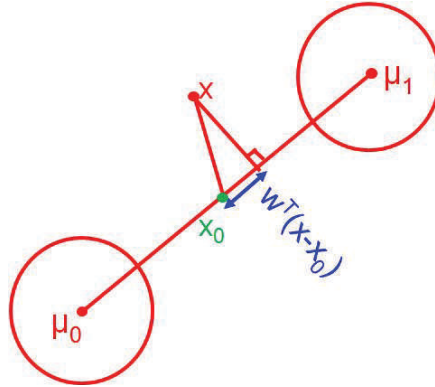


Figure 4.6 Geometry of LDA in the 2 class case where $\Sigma_1 = \Sigma_2 = \mathbf{I}$.

where $\text{sigm}(\eta)$ refers to the sigmoid function (Equation 1.10).

Now

$$\gamma_1 - \gamma_0 = -\frac{1}{2}\mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2}\mu_0^T \Sigma^{-1} \mu_0 + \log(\pi_1/\pi_0) \quad (4.46)$$

$$= -\frac{1}{2}(\mu_1 - \mu_0)^T \Sigma^{-1} (\mu_1 + \mu_0) + \log(\pi_1/\pi_0) \quad (4.47)$$

So if we define

$$\mathbf{w} = \beta_1 - \beta_0 = \Sigma^{-1}(\mu_1 - \mu_0) \quad (4.48)$$

$$\mathbf{x}_0 = \frac{1}{2}(\mu_1 + \mu_0) - (\mu_1 - \mu_0) \frac{\log(\pi_1/\pi_0)}{(\mu_1 - \mu_0)^T \Sigma^{-1} (\mu_1 - \mu_0)} \quad (4.49)$$

then we have $\mathbf{w}^T \mathbf{x}_0 = -(\gamma_1 - \gamma_0)$, and hence

$$p(y = 1 | \mathbf{x}, \boldsymbol{\theta}) = \text{sigm}(\mathbf{w}^T (\mathbf{x} - \mathbf{x}_0)) \quad (4.50)$$

(This is closely related to logistic regression, which we will discuss in Section 8.2.) So the final decision rule is as follows: shift $\mathbf{x}$ by $\mathbf{x}_0$, project onto the line $\mathbf{w}$, and see if the result is positive or negative.

If $\Sigma = \sigma^2 \mathbf{I}$, then $\mathbf{w}$ is in the direction of $\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0$. So we classify the point based on whether its projection is closer to $\boldsymbol{\mu}_0$ or $\boldsymbol{\mu}_1$. This is illustrated in Figure 4.6. Furthermore, if $\pi_1 = \pi_0$, then $\mathbf{x}_0 = \frac{1}{2}(\boldsymbol{\mu}_1 + \boldsymbol{\mu}_0)$, which is half way between the means. If we make $\pi_1 > \pi_0$, then $\mathbf{x}_0$ gets closer to $\boldsymbol{\mu}_0$, so more of the line belongs to class 1 *a priori*. Conversely if $\pi_1 < \pi_0$, the boundary shifts right. Thus we see that the class prior, π_c , just changes the decision threshold, and not the overall geometry, as we claimed above. (A similar argument applies in the multi-class case.)

The magnitude of $\mathbf{w}$ determines the steepness of the logistic function, and depends on how well-separated the means are, relative to the variance. In psychology and signal detection theory, it is common to define the **discriminability** of a signal from the background noise using a quantity called **d-prime**:

$$d' \triangleq \frac{\mu_1 - \mu_0}{\sigma} \quad (4.51)$$

where μ_1 is the mean of the signal and μ_0 is the mean of the noise, and σ is the standard deviation of the noise. If d' is large, the signal will be easier to discriminate from the noise.

4.2.4 MLE for discriminant analysis

We now discuss how to fit a discriminant analysis model. The simplest way is to use maximum likelihood. The log-likelihood function is as follows:

$$\log p(\mathcal{D} | \boldsymbol{\theta}) = \left[\sum_{i=1}^N \sum_{c=1}^C \mathbb{I}(y_i = c) \log \pi_c \right] + \sum_{c=1}^C \left[\sum_{i: y_i = c} \log \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c) \right] \quad (4.52)$$

We see that this factorizes into a term for $\boldsymbol{\pi}$, and C terms for each $\boldsymbol{\mu}_c$ and $\boldsymbol{\Sigma}_c$. Hence we can estimate these parameters separately. For the class prior, we have $\hat{\pi}_c = \frac{N_c}{N}$, as with naive Bayes. For the class-conditional densities, we just partition the data based on its class label, and compute the MLE for each Gaussian:

$$\hat{\boldsymbol{\mu}}_c = \frac{1}{N_c} \sum_{i: y_i = c} \mathbf{x}_i, \quad \hat{\boldsymbol{\Sigma}}_c = \frac{1}{N_c} \sum_{i: y_i = c} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_c)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_c)^T \quad (4.53)$$

See `discrimAnalysisFit` for a Matlab implementation. Once the model has been fit, you can make predictions using `discrimAnalysisPredict`, which uses a plug-in approximation.

4.2.5 Strategies for preventing overfitting

The speed and simplicity of the MLE method is one of its greatest appeals. However, the MLE can badly overfit in high dimensions. In particular, the MLE for a full covariance matrix is singular if $N_c < D$. And even when $N_c > D$, the MLE can be ill-conditioned, meaning it is close to singular. There are several possible solutions to this problem:

- Use a diagonal covariance matrix for each class, which assumes the features are conditionally independent; this is equivalent to using a naive Bayes classifier (Section 3.5).
- Use a full covariance matrix, but force it to be the same for all classes, $\Sigma_c = \Sigma$. This is an example of **parameter tying** or **parameter sharing**, and is equivalent to LDA (Section 4.2.2).
- Use a diagonal covariance matrix *and* forced it to be shared. This is called diagonal covariance LDA, and is discussed in Section 4.2.7.
- Use a full covariance matrix, but impose a prior and then integrate it out. If we use a conjugate prior, this can be done in closed form, using the results from Section 4.6.3; this is analogous to the “Bayesian naive Bayes” method in Section 3.5.1.2. See (Minka 2000f) for details.
- Fit a full or diagonal covariance matrix by MAP estimation. We discuss two different kinds of prior below.
- Project the data into a low dimensional subspace and fit the Gaussians there. See Section 8.6.3.3 for a way to find the best (most discriminative) linear projection.

We discuss some of these options below.

4.2.6 Regularized LDA *

Suppose we tie the covariance matrices, so $\Sigma_c = \Sigma$, as in LDA, and furthermore we perform MAP estimation of Σ using an inverse Wishart prior of the form $IW(\text{diag}(\hat{\Sigma}_{mle}), \nu_0)$ (see Section 4.5.1). Then we have

$$\hat{\Sigma} = \lambda \text{diag}(\hat{\Sigma}_{mle}) + (1 - \lambda) \hat{\Sigma}_{mle} \quad (4.54)$$

where λ controls the amount of regularization, which is related to the strength of the prior, ν_0 (see Section 4.6.2.1 for details). This technique is known as **regularized discriminant analysis** or RDA (Hastie et al. 2009, p656).

When we evaluate the class conditional densities, we need to compute $\hat{\Sigma}^{-1}$, and hence $\hat{\Sigma}_{mle}^{-1}$, which is impossible to compute if $D > N$. However, we can use the SVD of $\mathbf{X}$ (Section 12.2.3) to get around this, as we show below. (Note that this trick cannot be applied to QDA, which is a nonlinear function of $\mathbf{x}$.)

Let $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^T$ be the SVD of the design matrix, where $\mathbf{V}$ is $D \times N$, $\mathbf{U}$ is an $N \times N$ orthogonal matrix, and $\mathbf{D}$ is a diagonal matrix of size N . Furthermore, define the $N \times N$ matrix $\mathbf{Z} = \mathbf{U}\mathbf{D}$; this is like a design matrix in a lower dimensional space (since we assume $N < D$). Also, define $\boldsymbol{\mu}_z = \mathbf{V}^T \boldsymbol{\mu}$ as the mean of the data in this reduced space; we can recover the original mean using $\boldsymbol{\mu} = \mathbf{V}\boldsymbol{\mu}_z$, since $\mathbf{V}^T \mathbf{V} = \mathbf{V}\mathbf{V}^T = \mathbf{I}$. With these definitions, we can

rewrite the MLE as follows:

$$\hat{\Sigma}_{mle} = \frac{1}{N} \mathbf{X}^T \mathbf{X} - \boldsymbol{\mu} \boldsymbol{\mu}^T \quad (4.55)$$

$$= \frac{1}{N} (\mathbf{Z} \mathbf{V}^T)^T (\mathbf{Z} \mathbf{V}^T) - (\mathbf{V} \boldsymbol{\mu}_z)(\mathbf{V} \boldsymbol{\mu}_z)^T \quad (4.56)$$

$$= \frac{1}{N} \mathbf{V} \mathbf{Z}^T \mathbf{Z} \mathbf{V}^T - \mathbf{V} \boldsymbol{\mu}_z \boldsymbol{\mu}_z^T \mathbf{V}^T \quad (4.57)$$

$$= \mathbf{V} \left(\frac{1}{N} \mathbf{Z}^T \mathbf{Z} - \boldsymbol{\mu}_z \boldsymbol{\mu}_z^T \right) \mathbf{V}^T \quad (4.58)$$

$$= \mathbf{V} \hat{\Sigma}_z \mathbf{V}^T \quad (4.59)$$

where $\hat{\Sigma}_z$ is the empirical covariance of $\mathbf{Z}$. Hence we can rewrite the MAP estimate as

$$\hat{\Sigma}_{map} = \mathbf{V} \tilde{\Sigma}_z \mathbf{V}^T \quad (4.60)$$

$$\tilde{\Sigma}_z = \lambda \text{diag}(\hat{\Sigma}_z) + (1 - \lambda) \hat{\Sigma}_z \quad (4.61)$$

Note, however, that we never need to actually compute the $D \times D$ matrix $\hat{\Sigma}_{map}$. This is because Equation 4.38 tells us that to classify using LDA, all we need to compute is $p(y = c | \mathbf{x}, \boldsymbol{\theta}) \propto \exp(\delta_c)$, where

$$\delta_c = -\mathbf{x}^T \boldsymbol{\beta}_c + \gamma_c, \quad \boldsymbol{\beta}_c = \hat{\Sigma}^{-1} \boldsymbol{\mu}_c, \quad \gamma_c = \frac{1}{2} \boldsymbol{\mu}_c^T \boldsymbol{\beta}_c + \log \pi_c \quad (4.62)$$

We can compute the crucial $\boldsymbol{\beta}_c$ term for RDA without inverting the $D \times D$ matrix as follows:

$$\boldsymbol{\beta}_c = \hat{\Sigma}_{map}^{-1} \boldsymbol{\mu}_c = (\mathbf{V} \tilde{\Sigma}_z \mathbf{V}^T)^{-1} \boldsymbol{\mu}_c = \mathbf{V} \tilde{\Sigma}_z^{-1} \mathbf{V}^T \boldsymbol{\mu}_c = \mathbf{V} \tilde{\Sigma}_z^{-1} \boldsymbol{\mu}_{z,c} \quad (4.63)$$

where $\boldsymbol{\mu}_{z,c} = \mathbf{V}^T \boldsymbol{\mu}_c$ is the mean of the $\mathbf{Z}$ matrix for data belonging to class c . See `rdaFit` for the code.

4.2.7 Diagonal LDA

A simple alternative to RDA is to tie the covariance matrices, so $\Sigma_c = \Sigma$ as in LDA, and then to use a diagonal covariance matrix for each class. This is called the **diagonal LDA** model, and is equivalent to RDA with $\lambda = 1$. The corresponding discriminant function is as follows (compare to Equation 4.33):

$$\delta_c(\mathbf{x}) = \log p(\mathbf{x}, y = c | \boldsymbol{\theta}) = - \sum_{j=1}^D \frac{(x_j - \mu_{cj})^2}{2\sigma_j^2} + \log \pi_c \quad (4.64)$$

Typically we set $\hat{\mu}_{cj} = \bar{x}_{cj}$ and $\hat{\sigma}_j^2 = s_j^2$, which is the **pooled empirical variance** of feature j (pooled across classes) defined by

$$s_j^2 = \frac{\sum_{c=1}^C \sum_{i: y_i=c} (x_{ij} - \bar{x}_{cj})^2}{N - C} \quad (4.65)$$

In high dimensional settings, this model can work much better than LDA and RDA (Bickel and Levina 2004).

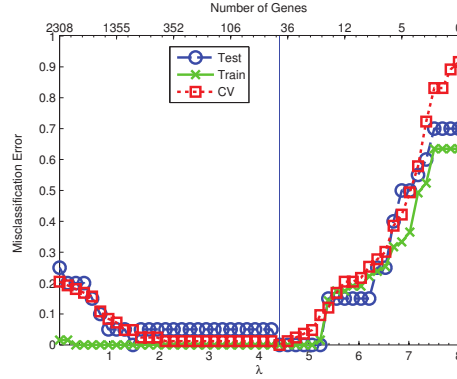


Figure 4.7 Error versus amount of shrinkage for nearest shrunken centroid classifier applied to the SRBCT gene expression data. Based on Figure 18.4 of (Hastie et al. 2009). Figure generated by `shrunkCentroidsSRBCTdemo`.

4.2.8 Nearest shrunken centroids classifier *

One drawback of diagonal LDA is that it depends on all of the features. In high dimensional problems, we might prefer a method that only depends on a subset of the features, for reasons of accuracy and interpretability. One approach is to use a screening method, perhaps based on mutual information, as in Section 3.5.4. We now discuss another approach to this problem known as the **nearest shrunken centroids** classifier (Hastie et al. 2009, p652).

The basic idea is to perform MAP estimation for diagonal LDA with a sparsity-promoting (Laplace) prior (see Section 13.3). More precisely, define the class-specific feature mean, μ_{cj} , in terms of the class-independent feature mean, m_j , and a class-specific offset, Δ_{cj} . Thus we have

$$\mu_{cj} = m_j + \Delta_{cj} \quad (4.66)$$

We will then put a prior on the Δ_{cj} terms to encourage them to be strictly zero and compute a MAP estimate. If, for feature j , we find that $\Delta_{cj} = 0$ for all c , then feature j will play no role in the classification decision (since μ_{cj} will be independent of c). Thus features that are not discriminative are automatically ignored. The details can be found in (Hastie et al. 2009, p652) and (Greenshtein and Park 2009). See `shrunkCentroidsFit` for some code.

Let us give an example of the method in action, based on (Hastie et al. 2009, p652). Consider the problem of classifying a gene expression dataset, which 2308 genes, 4 classes, 63 training samples and 20 test samples. Using a diagonal LDA classifier produces 5 errors on the test set. Using the nearest shrunken centroids classifier produced 0 errors on the test set, for a range of λ values: see Figure 4.7. More importantly, the model is sparse and hence more interpretable: Figure 4.8 plots an unpenalized estimate of the difference, d_{cj} , in gray, as well as the shrunken estimates Δ_{cj} in blue. (These estimates are computed using the value of λ estimated by CV.) We see that only 39 genes are used, out of the original 2308.

Now consider an even harder problem, with 16,603 genes, a training set of 144 patients, a test set of 54 patients, and 14 different types of cancer (Ramaswamy et al. 2001). Hastie et al. (Hastie et al. 2009, p656) report that nearest shrunken centroids produced 17 errors on the test

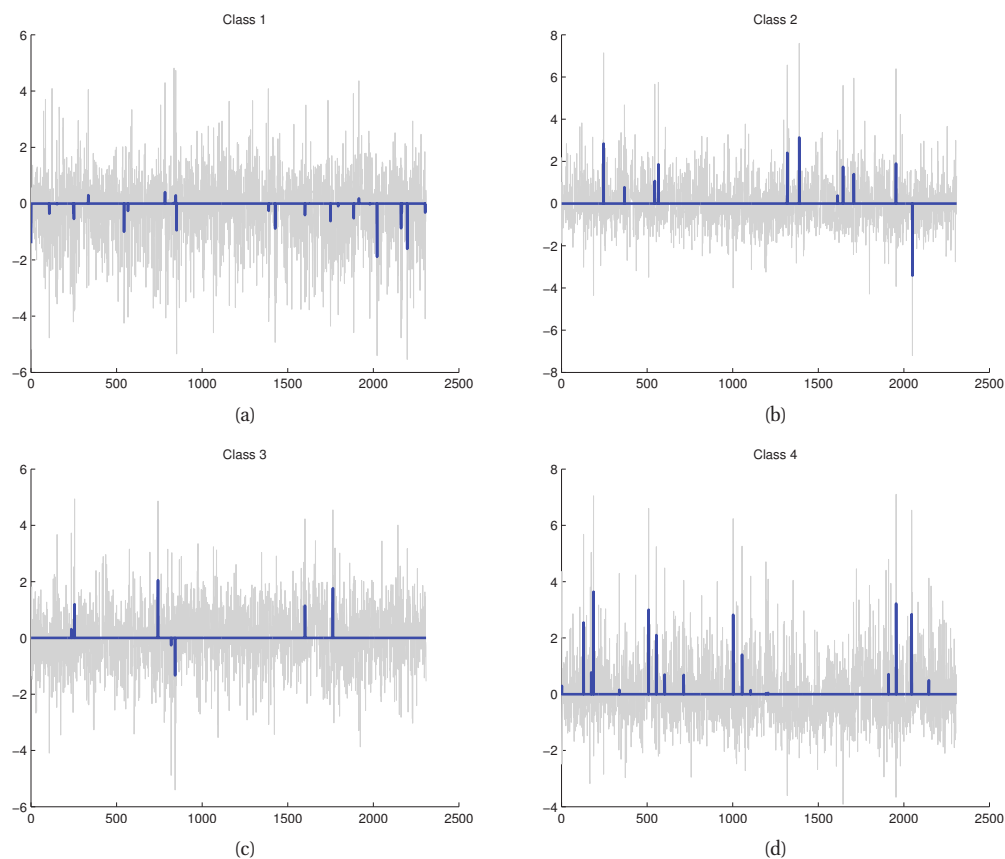


Figure 4.8 Profile of the shrunken centroids corresponding to $\lambda = 4.4$ (CV optimal in Figure 4.7). This selects 39 genes. Based on Figure 18.4 of (Hastie et al. 2009). Figure generated by `shrunkCentroidsSRBCTdemo`.

set, using 6,520 genes, and that RDA (Section 4.2.6) produced 12 errors on the test set, using all 16,603 genes. The PMTK function `cancerHighDimClassifDemo` can be used to reproduce these numbers.

4.3 Inference in jointly Gaussian distributions

Given a joint distribution, $p(\mathbf{x}_1, \mathbf{x}_2)$, it is useful to be able to compute marginals $p(\mathbf{x}_1)$ and conditionals $p(\mathbf{x}_1|\mathbf{x}_2)$. We discuss how to do this below, and then give some applications. These operations take $O(D^3)$ time in the worst case. See Section 20.4.3 for faster methods.

4.3.1 Statement of the result

Theorem 4.3.1 (Marginals and conditionals of an MVN). *Suppose $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2)$ is jointly Gaussian with parameters*

$$\boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{\mu}_1 \\ \boldsymbol{\mu}_2 \end{pmatrix}, \quad \boldsymbol{\Sigma} = \begin{pmatrix} \boldsymbol{\Sigma}_{11} & \boldsymbol{\Sigma}_{12} \\ \boldsymbol{\Sigma}_{21} & \boldsymbol{\Sigma}_{22} \end{pmatrix}, \quad \boldsymbol{\Lambda} = \boldsymbol{\Sigma}^{-1} = \begin{pmatrix} \boldsymbol{\Lambda}_{11} & \boldsymbol{\Lambda}_{12} \\ \boldsymbol{\Lambda}_{21} & \boldsymbol{\Lambda}_{22} \end{pmatrix} \quad (4.67)$$

Then the marginals are given by

$$\begin{aligned} p(\mathbf{x}_1) &= \mathcal{N}(\mathbf{x}_1 | \boldsymbol{\mu}_1, \boldsymbol{\Sigma}_{11}) \\ p(\mathbf{x}_2) &= \mathcal{N}(\mathbf{x}_2 | \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_{22}) \end{aligned} \quad (4.68)$$

and the posterior conditional is given by

$$\begin{aligned} p(\mathbf{x}_1 | \mathbf{x}_2) &= \mathcal{N}(\mathbf{x}_1 | \boldsymbol{\mu}_{1|2}, \boldsymbol{\Sigma}_{1|2}) \\ \boldsymbol{\mu}_{1|2} &= \boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_{12} \boldsymbol{\Sigma}_{22}^{-1} (\mathbf{x}_2 - \boldsymbol{\mu}_2) \\ &= \boldsymbol{\mu}_1 - \boldsymbol{\Lambda}_{11}^{-1} \boldsymbol{\Lambda}_{12} (\mathbf{x}_2 - \boldsymbol{\mu}_2) \\ &= \boldsymbol{\Sigma}_{1|2} (\boldsymbol{\Lambda}_{11} \boldsymbol{\mu}_1 - \boldsymbol{\Lambda}_{12} (\mathbf{x}_2 - \boldsymbol{\mu}_2)) \\ \boldsymbol{\Sigma}_{1|2} &= \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12} \boldsymbol{\Sigma}_{22}^{-1} \boldsymbol{\Sigma}_{21} = \boldsymbol{\Lambda}_{11}^{-1} \end{aligned} \quad (4.69)$$

Equation 4.69 is of such crucial importance in this book that we have put a box around it, so you can easily find it. For the proof, see Section 4.3.4.

We see that both the marginal and conditional distributions are themselves Gaussian. For the marginals, we just extract the rows and columns corresponding to $\mathbf{x}_1$ or $\mathbf{x}_2$. For the conditional, we have to do a bit more work. However, it is not that complicated: the conditional mean is just a linear function of $\mathbf{x}_2$, and the conditional covariance is just a constant matrix that is independent of $\mathbf{x}_2$. We give three different (but equivalent) expressions for the posterior mean, and two different (but equivalent) expressions for the posterior covariance; each one is useful in different circumstances.

4.3.2 Examples

Below we give some examples of these equations in action, which will make them seem more intuitive.

4.3.2.1 Marginals and conditionals of a 2d Gaussian

Let us consider a 2d example. The covariance matrix is

$$\boldsymbol{\Sigma} = \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix} \quad (4.70)$$

The marginal $p(x_1)$ is a 1D Gaussian, obtained by projecting the joint distribution onto the x_1 line:

$$p(x_1) = \mathcal{N}(x_1 | \mu_1, \sigma_1^2) \quad (4.71)$$

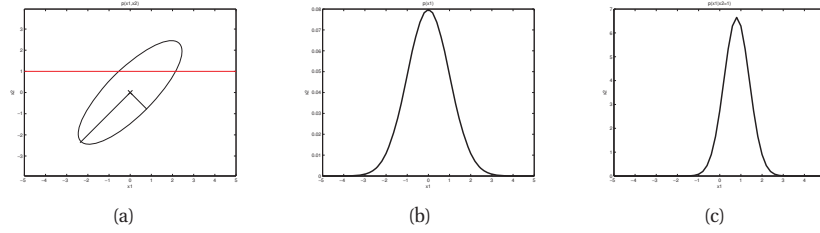


Figure 4.9 (a) A joint Gaussian distribution $p(x_1, x_2)$ with a correlation coefficient of 0.8. We plot the 95% contour and the principal axes. (b) The unconditional marginal $p(x_1)$. (c) The conditional $p(x_1|x_2) = \mathcal{N}(x_1|0.8, 0.36)$, obtained by slicing (a) at height $x_2 = 1$. Figure generated by `gaussCondition2Ddemo2`.

Suppose we observe $X_2 = x_2$; the conditional $p(x_1|x_2)$ is obtained by “slicing” the joint distribution through the $X_2 = x_2$ line (see Figure 4.9):

$$p(x_1|x_2) = \mathcal{N}\left(x_1|\mu_1 + \frac{\rho\sigma_1\sigma_2}{\sigma_2^2}(x_2 - \mu_2), \sigma_1^2 - \frac{(\rho\sigma_1\sigma_2)^2}{\sigma_2^2}\right) \quad (4.72)$$

If $\sigma_1 = \sigma_2 = \sigma$, we get

$$p(x_1|x_2) = \mathcal{N}(x_1|\mu_1 + \rho(x_2 - \mu_2), \sigma^2(1 - \rho^2)) \quad (4.73)$$

In Figure 4.9 we show an example where $\rho = 0.8$, $\sigma_1 = \sigma_2 = 1$, $\boldsymbol{\mu} = \mathbf{0}$ and $x_2 = 1$. We see that $\mathbb{E}[x_1|x_2 = 1] = 0.8$, which makes sense, since $\rho = 0.8$ means that we believe that if x_2 increases by 1 (beyond its mean), then x_1 increases by 0.8. We also see $\text{var}[x_1|x_2 = 1] = 1 - 0.8^2 = 0.36$. This also makes sense: our uncertainty about x_1 has gone down, since we have learned something about x_1 (indirectly) by observing x_2 . If $\rho = 0$, we get $p(x_1|x_2) = \mathcal{N}(x_1|\mu_1, \sigma_1^2)$, since x_2 conveys no information about x_1 if they are uncorrelated (and hence independent).

4.3.2.2 Interpolating noise-free data

Suppose we want to estimate a 1d function, defined on the interval $[0, T]$, such that $y_i = f(t_i)$ for N observed points t_i . We assume for now that the data is noise-free, so we want to **interpolate** it, that is, fit a function that goes exactly through the data. (See Section 4.4.2.3 for the noisy data case.) The question is: how does the function behave in between the observed data points? It is often reasonable to assume that the unknown function is smooth. In Chapter 15, we shall see how to encode *priors over functions*, and how to update such a prior with observed values to get a posterior over functions. But in this section, we take a simpler approach, which is adequate for MAP estimation of functions defined on 1d inputs. We follow the presentation of (Calvetti and Somersalo 2007, p135).

We start by discretizing the problem. First we divide the support of the function into D equal subintervals. We then define

$$x_j = f(s_j), \quad s_j = jh, \quad h = \frac{T}{D}, \quad 1 \leq j \leq D \quad (4.74)$$

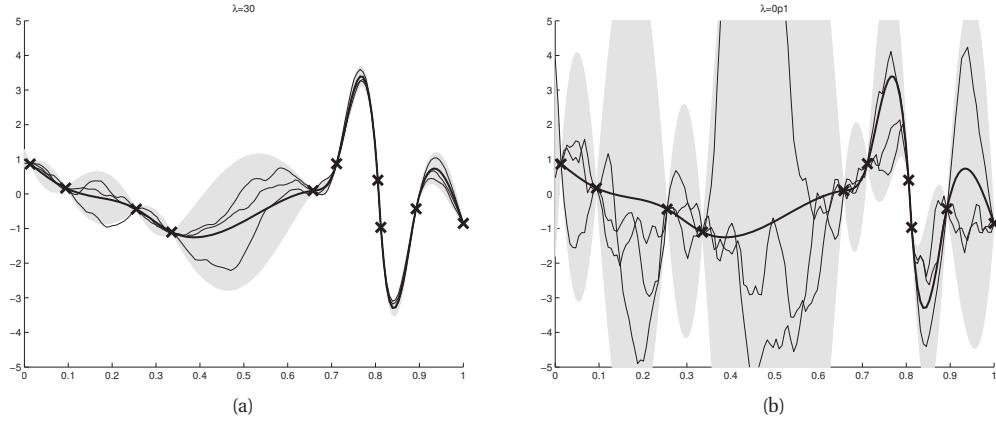


Figure 4.10 Interpolating noise-free data using a Gaussian with prior precision λ . (a) $\lambda = 30$. (b) $\lambda = 0.01$. See also Figure 4.15. Based on Figure 7.1 of (Calvetti and Somersalo 2007). Figure generated by `gaussInterpDemo`.

We can encode our smoothness prior by assuming that x_j is an average of its neighbors, x_{j-1} and x_{j+1} , plus some Gaussian noise:

$$x_j = \frac{1}{2}(x_{j-1} + x_{j+1}) + \epsilon_j, \quad 2 \leq j \leq D-2 \quad (4.75)$$

where $\epsilon \sim \mathcal{N}(\mathbf{0}, (1/\lambda)\mathbf{I})$. The precision term λ controls how much we think the function will vary: a large λ corresponds to a belief that the function is very smooth, a small λ corresponds to a belief that the function is quite “wiggly”. In vector form, the above equation can be written as follows:

$$\mathbf{L}\mathbf{x} = \boldsymbol{\epsilon} \quad (4.76)$$

where $\mathbf{L}$ is the $(D-2) \times D$ second order **finite difference matrix**

$$\mathbf{L} = \frac{1}{2} \begin{pmatrix} -1 & 2 & -1 & & \\ & -1 & 2 & -1 & \\ & & \ddots & \ddots & \\ & & & -1 & 2 & -1 \end{pmatrix} \quad (4.77)$$

The corresponding prior has the form

$$p(\mathbf{x}) = \mathcal{N}(\mathbf{x}|\mathbf{0}, (\lambda^2 \mathbf{L}^T \mathbf{L})^{-1}) \propto \exp\left(-\frac{\lambda^2}{2} \|\mathbf{L}\mathbf{x}\|_2^2\right) \quad (4.78)$$

We will henceforth assume we have scaled $\mathbf{L}$ by λ so we can ignore the λ term, and just write $\boldsymbol{\Lambda} = \mathbf{L}^T \mathbf{L}$ for the precision matrix.

Note that although $\mathbf{x}$ is D -dimensional, the precision matrix $\boldsymbol{\Lambda}$ only has rank $D-2$. Thus this is an improper prior, known as an intrinsic Gaussian random field (see Section 19.4.4 for

more information). However, providing we observe $N \geq 2$ data points, the posterior will be proper.

Now let $\mathbf{x}_2$ be the N noise-free observations of the function, and $\mathbf{x}_1$ be the $D - N$ unknown function values. Without loss of generality, assume that the unknown variables are ordered first, then the known variables. Then we can partition the $\mathbf{L}$ matrix as follows:

$$\mathbf{L} = [\mathbf{L}_1, \mathbf{L}_2], \quad \mathbf{L}_1 \in \mathbb{R}^{(D-2) \times (D-N)}, \quad \mathbf{L}_2 \in \mathbb{R}^{(D-2) \times (N)} \quad (4.79)$$

We can also partition the precision matrix of the joint distribution:

$$\mathbf{\Lambda} = \mathbf{L}^T \mathbf{L} = \begin{pmatrix} \mathbf{\Lambda}_{11} & \mathbf{\Lambda}_{12} \\ \mathbf{\Lambda}_{21} & \mathbf{\Lambda}_{22} \end{pmatrix} = \begin{pmatrix} \mathbf{L}_1^T \mathbf{L}_1 & \mathbf{L}_1^T \mathbf{L}_2 \\ \mathbf{L}_2^T \mathbf{L}_1 & \mathbf{L}_2^T \mathbf{L}_2 \end{pmatrix} \quad (4.80)$$

Using Equation 4.69, we can write the conditional distribution as follows:

$$p(\mathbf{x}_1 | \mathbf{x}_2) = \mathcal{N}(\boldsymbol{\mu}_{1|2}, \boldsymbol{\Sigma}_{1|2}) \quad (4.81)$$

$$\boldsymbol{\mu}_{1|2} = -\mathbf{\Lambda}_{11}^{-1} \mathbf{\Lambda}_{12} \mathbf{x}_2 = -\mathbf{L}_1^T \mathbf{L}_2 \mathbf{x}_2 \quad (4.82)$$

$$\boldsymbol{\Sigma}_{1|2} = \mathbf{\Lambda}_{11}^{-1} \quad (4.83)$$

Note that we can compute the mean by solving the following system of linear equations:

$$\mathbf{L}_1 \boldsymbol{\mu}_{1|2} = -\mathbf{L}_2 \mathbf{x}_2 \quad (4.84)$$

This is efficient since $\mathbf{L}_1$ is tridiagonal. Figure 4.10 gives an illustration of these equations. We see that the posterior mean $\boldsymbol{\mu}_{1|2}$ equals the observed data at the specified points, and smoothly interpolates in between, as desired.

It is also interesting to plot the 95% **pointwise marginal credibility intervals**, $\mu_j \pm 2\sqrt{\Sigma_{1|2,jj}}$, shown in grey. We see that the variance goes up as we move away from the data. We also see that the variance goes up as we decrease the precision of the prior, λ . Interestingly, λ has no effect on the posterior mean, since it cancels out when multiplying $\mathbf{\Lambda}_{11}$ and $\mathbf{\Lambda}_{12}$. By contrast, when we consider noisy data in Section 4.4.2.3, we will see that the prior precision affects the smoothness of posterior mean estimate.

The marginal credibility intervals do not capture the fact that neighboring locations are correlated. We can represent that by drawing complete functions (i.e., vectors $\mathbf{x}$) from the posterior, and plotting them. These are shown by the thin lines in Figure 4.10. These are not quite as smooth as the posterior mean itself. This is because the prior only penalizes first-order differences. See Section 4.4.2.3 for further discussion of this point.

4.3.2.3 Data imputation

Suppose we are missing some entries in a design matrix. If the columns are correlated, we can use the observed entries to predict the missing entries. Figure 4.11 shows a simple example. We sampled some data from a 20 dimensional Gaussian, and then deliberately “hid” 50% of the data in each row. We then inferred the missing entries given the observed entries, using the true (generating) model. More precisely, for each row i , we compute $p(\mathbf{x}_{\mathbf{h}_i} | \mathbf{x}_{\mathbf{v}_i}, \boldsymbol{\theta})$, where $\mathbf{h}_i$ and $\mathbf{v}_i$ are the indices of the hidden and visible entries in case i . From this, we compute the marginal distribution of each missing variable, $p(x_{h_{ij}} | \mathbf{x}_{\mathbf{v}_i}, \boldsymbol{\theta})$. We then plot the mean of this distribution, $\hat{x}_{ij} = \mathbb{E}[x_j | \mathbf{x}_{\mathbf{v}_i}, \boldsymbol{\theta}]$; this represents our “best guess” about the true value of that entry, in the

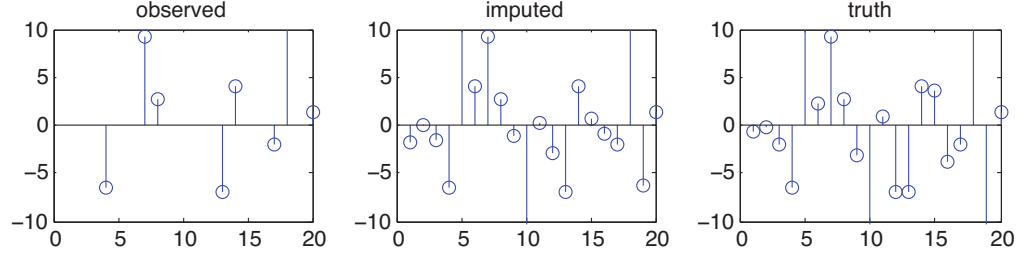


Figure 4.11 Illustration of data imputation. Left column: visualization of three rows of the data matrix with missing entries. Middle column: mean of the posterior predictive, based on partially observed data in that row, but the true model parameters. Right column: true values. Figure generated by `gaussImputationDemo`.

sense that it minimizes our expected squared error (see Section 5.7 for details). Figure 4.11 shows that the estimates are quite close to the truth. (Of course, if $j \in \mathbf{v}_i$, the expected value is equal to the observed value, $\hat{x}_{ij} = x_{ij}$.)

We can use $\text{var}[x_{h_{ij}}|\mathbf{x}_{\mathbf{v}_i}, \boldsymbol{\theta}]$ as a measure of confidence in this guess, although this is not shown. Alternatively, we could draw multiple samples from $p(\mathbf{x}_{\mathbf{h}_i}|\mathbf{x}_{\mathbf{v}_i}, \boldsymbol{\theta})$; this is called **multiple imputation**.

In addition to imputing the missing entries, we may be interested in computing the likelihood of each partially observed row in the table, $p(\mathbf{x}_{\mathbf{v}_i}|\boldsymbol{\theta})$, which can be computed using Equation 4.68. This is useful for detecting outliers (atypical observations).

4.3.3 Information form

Suppose $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$. One can show that $\mathbb{E}[\mathbf{x}] = \boldsymbol{\mu}$ is the mean vector, and $\text{cov}[\mathbf{x}] = \boldsymbol{\Sigma}$ is the covariance matrix. These are called the **moment parameters** of the distribution. However, it is sometimes useful to use the **canonical parameters** or **natural parameters**, defined as

$$\boldsymbol{\Lambda} \triangleq \boldsymbol{\Sigma}^{-1}, \quad \boldsymbol{\xi} \triangleq \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} \quad (4.85)$$

We can convert back to the moment parameters using

$$\boldsymbol{\mu} = \boldsymbol{\Lambda}^{-1}\boldsymbol{\xi}, \quad \boldsymbol{\Sigma} = \boldsymbol{\Lambda}^{-1} \quad (4.86)$$

Using the canonical parameters, we can write the MVN in **information form** (i.e., in exponential family form, defined in Section 9.2):

$$\mathcal{N}_c(\mathbf{x}|\boldsymbol{\xi}, \boldsymbol{\Lambda}) = (2\pi)^{-D/2} |\boldsymbol{\Lambda}|^{\frac{1}{2}} \exp \left[-\frac{1}{2} (\mathbf{x}^T \boldsymbol{\Lambda} \mathbf{x} + \boldsymbol{\xi}^T \boldsymbol{\Lambda}^{-1} \boldsymbol{\xi} - 2\mathbf{x}^T \boldsymbol{\xi}) \right] \quad (4.87)$$

where we use the notation $\mathcal{N}_c()$ to distinguish from the moment parameterization $\mathcal{N}()$.

It is also possible to derive the marginalization and conditioning formulas in information form. We find

$$p(\mathbf{x}_2) = \mathcal{N}_c(\mathbf{x}_2|\boldsymbol{\xi}_2 - \boldsymbol{\Lambda}_{21}\boldsymbol{\Lambda}_{11}^{-1}\boldsymbol{\xi}_1, \boldsymbol{\Lambda}_{22} - \boldsymbol{\Lambda}_{21}\boldsymbol{\Lambda}_{11}^{-1}\boldsymbol{\Lambda}_{12}) \quad (4.88)$$

$$p(\mathbf{x}_1|\mathbf{x}_2) = \mathcal{N}_c(\mathbf{x}_1|\boldsymbol{\xi}_1 - \boldsymbol{\Lambda}_{12}\mathbf{x}_2, \boldsymbol{\Lambda}_{11}) \quad (4.89)$$

Thus we see that marginalization is easier in moment form, and conditioning is easier in information form.

Another operation that is significantly easier in information form is multiplying two Gaussians. One can show that

$$\mathcal{N}_c(\xi_f, \lambda_f) \mathcal{N}_c(\xi_g, \lambda_g) = \mathcal{N}_c(\xi_f + \xi_g, \lambda_f + \lambda_g) \quad (4.90)$$

However, in moment form, things are much messier:

$$\mathcal{N}(\mu_f, \sigma_f^2) \mathcal{N}(\mu_g, \sigma_g^2) = \mathcal{N}\left(\frac{\mu_f \sigma_g^2 + \mu_g \sigma_f^2}{\sigma_g^2 + \sigma_f^2}, \frac{\sigma_f^2 \sigma_g^2}{\sigma_g^2 + \sigma_f^2}\right) \quad (4.91)$$

4.3.4 Proof of the result *

We now prove Theorem 4.3.1. Readers who are intimidated by heavy matrix algebra can safely skip this section. We first derive some results that we will need here and elsewhere in the book. We will return to the proof at the end.

4.3.4.1 Inverse of a partitioned matrix using Schur complements

The key tool we need is a way to invert a partitioned matrix. This can be done using the following result.

Theorem 4.3.2 (Inverse of a partitioned matrix). *Consider a general partitioned matrix*

$$\mathbf{M} = \begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix} \quad (4.92)$$

where we assume $\mathbf{E}$ and $\mathbf{H}$ are invertible. We have

$$\mathbf{M}^{-1} = \begin{pmatrix} (\mathbf{M}/\mathbf{H})^{-1} & -(\mathbf{M}/\mathbf{H})^{-1} \mathbf{F} \mathbf{H}^{-1} \\ -\mathbf{H}^{-1} \mathbf{G} (\mathbf{M}/\mathbf{H})^{-1} & \mathbf{H}^{-1} + \mathbf{H}^{-1} \mathbf{G} (\mathbf{M}/\mathbf{H})^{-1} \mathbf{F} \mathbf{H}^{-1} \end{pmatrix} \quad (4.93)$$

$$= \begin{pmatrix} \mathbf{E}^{-1} + \mathbf{E}^{-1} \mathbf{F} (\mathbf{M}/\mathbf{E})^{-1} \mathbf{G} \mathbf{E}^{-1} & -\mathbf{E}^{-1} \mathbf{F} (\mathbf{M}/\mathbf{E})^{-1} \\ -(\mathbf{M}/\mathbf{E})^{-1} \mathbf{G} \mathbf{E}^{-1} & (\mathbf{M}/\mathbf{E})^{-1} \end{pmatrix} \quad (4.94)$$

where

$$\mathbf{M}/\mathbf{H} \triangleq \mathbf{E} - \mathbf{F} \mathbf{H}^{-1} \mathbf{G} \quad (4.95)$$

$$\mathbf{M}/\mathbf{E} \triangleq \mathbf{H} - \mathbf{G} \mathbf{E}^{-1} \mathbf{F} \quad (4.96)$$

We say that $\mathbf{M}/\mathbf{H}$ is the **Schur complement** of $\mathbf{M}$ wrt $\mathbf{H}$. Equation 4.93 is called the **partitioned inverse formula**.

Proof. If we could block diagonalize $\mathbf{M}$, it would be easier to invert. To zero out the top right block of $\mathbf{M}$ we can pre-multiply as follows

$$\begin{pmatrix} \mathbf{I} & -\mathbf{F} \mathbf{H}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix} = \begin{pmatrix} \mathbf{E} - \mathbf{F} \mathbf{H}^{-1} \mathbf{G} & \mathbf{0} \\ \mathbf{G} & \mathbf{H} \end{pmatrix} \quad (4.97)$$

Similarly, to zero out the bottom left we can post-multiply as follows

$$\begin{pmatrix} \mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G} & \mathbf{0} \\ \mathbf{G} & \mathbf{H} \end{pmatrix} \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ -\mathbf{H}^{-1}\mathbf{G} & \mathbf{I} \end{pmatrix} = \begin{pmatrix} \mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G} & \mathbf{0} \\ \mathbf{0} & \mathbf{H} \end{pmatrix} \quad (4.98)$$

Putting it all together we get

$$\underbrace{\begin{pmatrix} \mathbf{I} & -\mathbf{F}\mathbf{H}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix}}_{\mathbf{X}} \underbrace{\begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix}}_{\mathbf{M}} \underbrace{\begin{pmatrix} \mathbf{I} & \mathbf{0} \\ -\mathbf{H}^{-1}\mathbf{G} & \mathbf{I} \end{pmatrix}}_{\mathbf{Z}} = \underbrace{\begin{pmatrix} \mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G} & \mathbf{0} \\ \mathbf{0} & \mathbf{H} \end{pmatrix}}_{\mathbf{W}} \quad (4.99)$$

Taking the inverse of both sides yields

$$\mathbf{Z}^{-1}\mathbf{M}^{-1}\mathbf{X}^{-1} = \mathbf{W}^{-1} \quad (4.100)$$

and hence

$$\mathbf{M}^{-1} = \mathbf{Z}\mathbf{W}^{-1}\mathbf{X} \quad (4.101)$$

Substituting in the definitions we get

$$\begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix}^{-1} = \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ -\mathbf{H}^{-1}\mathbf{G} & \mathbf{I} \end{pmatrix} \begin{pmatrix} (\mathbf{M}/\mathbf{H})^{-1} & \mathbf{0} \\ \mathbf{0} & \mathbf{H}^{-1} \end{pmatrix} \begin{pmatrix} \mathbf{I} & -\mathbf{F}\mathbf{H}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \quad (4.102)$$

$$= \begin{pmatrix} (\mathbf{M}/\mathbf{H})^{-1} & \mathbf{0} \\ -\mathbf{H}^{-1}\mathbf{G}(\mathbf{M}/\mathbf{H})^{-1} & \mathbf{H}^{-1} \end{pmatrix} \begin{pmatrix} \mathbf{I} & -\mathbf{F}\mathbf{H}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \quad (4.103)$$

$$= \begin{pmatrix} (\mathbf{M}/\mathbf{H})^{-1} & -(\mathbf{M}/\mathbf{H})^{-1}\mathbf{F}\mathbf{H}^{-1} \\ -\mathbf{H}^{-1}\mathbf{G}(\mathbf{M}/\mathbf{H})^{-1} & \mathbf{H}^{-1} + \mathbf{H}^{-1}\mathbf{G}(\mathbf{M}/\mathbf{H})^{-1}\mathbf{F}\mathbf{H}^{-1} \end{pmatrix} \quad (4.104)$$

Alternatively, we could have decomposed the matrix $\mathbf{M}$ in terms of $\mathbf{E}$ and $\mathbf{M}/\mathbf{E} = (\mathbf{H} - \mathbf{G}\mathbf{E}^{-1}\mathbf{F})$, yielding

$$\begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix}^{-1} = \begin{pmatrix} \mathbf{E}^{-1} + \mathbf{E}^{-1}\mathbf{F}(\mathbf{M}/\mathbf{E})^{-1}\mathbf{G}\mathbf{E}^{-1} & -\mathbf{E}^{-1}\mathbf{F}(\mathbf{M}/\mathbf{E})^{-1} \\ -(\mathbf{M}/\mathbf{E})^{-1}\mathbf{G}\mathbf{E}^{-1} & (\mathbf{M}/\mathbf{E})^{-1} \end{pmatrix} \quad (4.105)$$

□

4.3.4.2 The matrix inversion lemma

We now derive some useful corollaries of the above result.

Corollary 4.3.1 (Matrix inversion lemma). *Consider a general partitioned matrix $\mathbf{M} = \begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix}$, where we assume $\mathbf{E}$ and $\mathbf{H}$ are invertible. We have*

$$(\mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G})^{-1} = \mathbf{E}^{-1} + \mathbf{E}^{-1}\mathbf{F}(\mathbf{H} - \mathbf{G}\mathbf{E}^{-1}\mathbf{F})^{-1}\mathbf{G}\mathbf{E}^{-1} \quad (4.106)$$

$$(\mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G})^{-1}\mathbf{F}\mathbf{H}^{-1} = \mathbf{E}^{-1}\mathbf{F}(\mathbf{H} - \mathbf{G}\mathbf{E}^{-1}\mathbf{F})^{-1} \quad (4.107)$$

$$|\mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G}| = |\mathbf{H} - \mathbf{G}\mathbf{E}^{-1}\mathbf{F}||\mathbf{H}^{-1}||\mathbf{E}| \quad (4.108)$$

The first two equations are known as the **matrix inversion lemma** or the **Sherman-Morrison-Woodbury formula**. The third equation is known as the **matrix determinant lemma**. A typical application in machine learning/ statistics is the following. Let $\mathbf{E} = \Sigma$ be a $N \times N$ diagonal matrix, let $\mathbf{F} = \mathbf{G}^T = \mathbf{X}$ of size $N \times D$, where $N \gg D$, and let $\mathbf{H}^{-1} = -\mathbf{I}$. Then we have

$$(\Sigma + \mathbf{X}\mathbf{X}^T)^{-1} = \Sigma^{-1} - \Sigma^{-1}\mathbf{X}(\mathbf{I} + \mathbf{X}^T\Sigma^{-1}\mathbf{X})^{-1}\mathbf{X}^T\Sigma^{-1} \quad (4.109)$$

The LHS takes $O(N^3)$ time to compute, the RHS takes time $O(D^3)$ to compute.

Another application concerns computing a **rank one update** of an inverse matrix. Let $H = -1$ (a scalar), $\mathbf{F} = \mathbf{u}$ (a column vector), and $\mathbf{G} = \mathbf{v}^T$ (a row vector). Then we have

$$(\mathbf{E} + \mathbf{u}\mathbf{v}^T)^{-1} = \mathbf{E}^{-1} + \mathbf{E}^{-1}\mathbf{u}(-1 - \mathbf{v}^T\mathbf{E}^{-1}\mathbf{u})^{-1}\mathbf{v}^T\mathbf{E}^{-1} \quad (4.110)$$

$$= \mathbf{E}^{-1} - \frac{\mathbf{E}^{-1}\mathbf{u}\mathbf{v}^T\mathbf{E}^{-1}}{1 + \mathbf{v}^T\mathbf{E}^{-1}\mathbf{u}} \quad (4.111)$$

This is useful when we incrementally add a data vector to a design matrix, and want to update our sufficient statistics. (One can derive an analogous formula for removing a data vector.)

Proof. To prove Equation 4.106, we simply equate the top left block of Equation 4.93 and Equation 4.94. To prove Equation 4.107, we simply equate the top right blocks of Equations 4.93 and 4.94. The proof of Equation 4.108 is left as an exercise. $\square$

4.3.4.3 Proof of Gaussian conditioning formulas

We can now return to our original goal, which is to derive Equation 4.69. Let us factor the joint $p(\mathbf{x}_1, \mathbf{x}_2)$ as $p(\mathbf{x}_2)p(\mathbf{x}_1|\mathbf{x}_2)$ as follows:

$$E = \exp \left\{ -\frac{1}{2} \begin{pmatrix} \mathbf{x}_1 - \boldsymbol{\mu}_1 \\ \mathbf{x}_2 - \boldsymbol{\mu}_2 \end{pmatrix}^T \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix}^{-1} \begin{pmatrix} \mathbf{x}_1 - \boldsymbol{\mu}_1 \\ \mathbf{x}_2 - \boldsymbol{\mu}_2 \end{pmatrix} \right\} \quad (4.112)$$

Using Equation 4.102 the above exponent becomes

$$E = \exp \left\{ -\frac{1}{2} \begin{pmatrix} \mathbf{x}_1 - \boldsymbol{\mu}_1 \\ \mathbf{x}_2 - \boldsymbol{\mu}_2 \end{pmatrix}^T \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ -\Sigma_{22}^{-1}\Sigma_{21} & \mathbf{I} \end{pmatrix} \begin{pmatrix} (\Sigma/\Sigma_{22})^{-1} & \mathbf{0} \\ \mathbf{0} & \Sigma_{22}^{-1} \end{pmatrix} \right\} \quad (4.113)$$

$$\times \begin{pmatrix} \mathbf{I} & -\Sigma_{12}\Sigma_{22}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \begin{pmatrix} \mathbf{x}_1 - \boldsymbol{\mu}_1 \\ \mathbf{x}_2 - \boldsymbol{\mu}_2 \end{pmatrix} \right\} \quad (4.114)$$

$$= \exp \left\{ -\frac{1}{2} (\mathbf{x}_1 - \boldsymbol{\mu}_1 - \Sigma_{12}\Sigma_{22}^{-1}(\mathbf{x}_2 - \boldsymbol{\mu}_2))^T (\Sigma/\Sigma_{22})^{-1} \right. \quad (4.115)$$

$$\left. (\mathbf{x}_1 - \boldsymbol{\mu}_1 - \Sigma_{12}\Sigma_{22}^{-1}(\mathbf{x}_2 - \boldsymbol{\mu}_2)) \right\} \times \exp \left\{ -\frac{1}{2} (\mathbf{x}_2 - \boldsymbol{\mu}_2)^T \Sigma_{22}^{-1} (\mathbf{x}_2 - \boldsymbol{\mu}_2) \right\} \quad (4.116)$$

This is of the form

$$\exp(\text{quadratic form in } \mathbf{x}_1, \mathbf{x}_2) \times \exp(\text{quadratic form in } \mathbf{x}_2) \quad (4.117)$$

Hence we have successfully factorized the joint as

$$p(\mathbf{x}_1, \mathbf{x}_2) = p(\mathbf{x}_1|\mathbf{x}_2)p(\mathbf{x}_2) \quad (4.118)$$

$$= \mathcal{N}(\mathbf{x}_1|\boldsymbol{\mu}_{1|2}, \boldsymbol{\Sigma}_{1|2})\mathcal{N}(\mathbf{x}_2|\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_{22}) \quad (4.119)$$

where the parameters of the conditional distribution can be read off from the above equations using

$$\boldsymbol{\mu}_{1|2} = \boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}(\mathbf{x}_2 - \boldsymbol{\mu}_2) \quad (4.120)$$

$$\boldsymbol{\Sigma}_{1|2} = \boldsymbol{\Sigma}/\boldsymbol{\Sigma}_{22} = \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21} \quad (4.121)$$

We can also use the fact that $|\mathbf{M}| = |\mathbf{M}/\mathbf{H}||\mathbf{H}|$ to check the normalization constants are correct:

$$(2\pi)^{(d_1+d_2)/2}|\boldsymbol{\Sigma}|^{\frac{1}{2}} = (2\pi)^{(d_1+d_2)/2}(|\boldsymbol{\Sigma}/\boldsymbol{\Sigma}_{22}| |\boldsymbol{\Sigma}_{22}|)^{\frac{1}{2}} \quad (4.122)$$

$$= (2\pi)^{d_1/2}|\boldsymbol{\Sigma}/\boldsymbol{\Sigma}_{22}|^{\frac{1}{2}} (2\pi)^{d_2/2}|\boldsymbol{\Sigma}_{22}|^{\frac{1}{2}} \quad (4.123)$$

where $d_1 = \dim(\mathbf{x}_1)$ and $d_2 = \dim(\mathbf{x}_2)$.

We leave the proof of the other forms of the result in Equation 4.69 as an exercise.

4.4 Linear Gaussian systems

Suppose we have two variables, $\mathbf{x}$ and $\mathbf{y}$. Let $\mathbf{x} \in \mathbb{R}^{D_x}$ be a hidden variable, and $\mathbf{y} \in \mathbb{R}^{D_y}$ be a noisy observation of $\mathbf{x}$. Let us assume we have the following prior and likelihood:

$$\begin{aligned} p(\mathbf{x}) &= \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_x, \boldsymbol{\Sigma}_x) \\ p(\mathbf{y}|\mathbf{x}) &= \mathcal{N}(\mathbf{y}|\mathbf{A}\mathbf{x} + \mathbf{b}, \boldsymbol{\Sigma}_y) \end{aligned} \quad (4.124)$$

where $\mathbf{A}$ is a matrix of size $D_y \times D_x$. This is an example of a **linear Gaussian system**. We can represent this schematically as $\mathbf{x} \rightarrow \mathbf{y}$, meaning $\mathbf{x}$ generates $\mathbf{y}$. In this section, we show how to “invert the arrow”, that is, how to infer $\mathbf{x}$ from $\mathbf{y}$. We state the result below, then give several examples, and finally we derive the result. We will see many more applications of these results in later chapters.

4.4.1 Statement of the result

Theorem 4.4.1 (Bayes rule for linear Gaussian systems). *Given a linear Gaussian system, as in Equation 4.124, the posterior $p(\mathbf{x}|\mathbf{y})$ is given by the following:*

$$\begin{aligned} p(\mathbf{x}|\mathbf{y}) &= \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_{x|y}, \boldsymbol{\Sigma}_{x|y}) \\ \boldsymbol{\Sigma}_{x|y}^{-1} &= \boldsymbol{\Sigma}_x^{-1} + \mathbf{A}^T \boldsymbol{\Sigma}_y^{-1} \mathbf{A} \\ \boldsymbol{\mu}_{x|y} &= \boldsymbol{\Sigma}_{x|y} [\mathbf{A}^T \boldsymbol{\Sigma}_y^{-1} (\mathbf{y} - \mathbf{b}) + \boldsymbol{\Sigma}_x^{-1} \boldsymbol{\mu}_x] \end{aligned} \quad (4.125)$$

In addition, the normalization constant $p(\mathbf{y})$ is given by

$$p(\mathbf{y}) = \mathcal{N}(\mathbf{y} | \mathbf{A}\boldsymbol{\mu}_x + \mathbf{b}, \boldsymbol{\Sigma}_y + \mathbf{A}\boldsymbol{\Sigma}_x\mathbf{A}^T) \quad (4.126)$$

For the proof, see Section 4.4.3.

4.4.2 Examples

In this section, we give some example applications of the above result.

4.4.2.1 Inferring an unknown scalar from noisy measurements

Suppose we make N noisy measurements y_i of some underlying quantity x ; let us assume the measurement noise has fixed precision $\lambda_y = 1/\sigma^2$, so the likelihood is

$$p(y_i | x) = \mathcal{N}(y_i | x, \lambda_y^{-1}) \quad (4.127)$$

Now let us use a Gaussian prior for the value of the unknown source:

$$p(x) = \mathcal{N}(x | \mu_0, \lambda_0^{-1}) \quad (4.128)$$

We want to compute $p(x | y_1, \dots, y_N, \sigma^2)$. We can convert this to a form that lets us apply Bayes rule for Gaussians by defining $\mathbf{y} = (y_1, \dots, y_N)$, $\mathbf{A} = \mathbf{1}_N^T$ (an $1 \times N$ row vector of 1's), and $\boldsymbol{\Sigma}_y^{-1} = \text{diag}(\lambda_y \mathbf{I})$. Then we get

$$p(x | \mathbf{y}) = \mathcal{N}(x | \mu_N, \lambda_N^{-1}) \quad (4.129)$$

$$\lambda_N = \lambda_0 + N\lambda_y \quad (4.130)$$

$$\mu_N = \frac{N\lambda_y \bar{y} + \lambda_0 \mu_0}{\lambda_N} = \frac{N\lambda_y}{N\lambda_y + \lambda_0} \bar{y} + \frac{\lambda_0}{N\lambda_y + \lambda_0} \mu_0 \quad (4.131)$$

These equations are quite intuitive: the posterior precision λ_N is the prior precision λ_0 plus N units of measurement precision λ_y . Also, the posterior mean μ_N is a convex combination of the MLE $\bar{y}$ and the prior mean μ_0 . This makes it clear that the posterior mean is a compromise between the MLE and the prior. If the prior is weak relative to the signal strength (λ_0 is small relative to λ_y), we put more weight on the MLE. If the prior is strong relative to the signal strength (λ_0 is large relative to λ_y), we put more weight on the prior. This is illustrated in Figure 4.12, which is very similar to the analogous results for the beta-binomial model in Figure 3.6.

Note that the posterior mean is written in terms of $N\lambda_y \bar{y}$, so having N measurements each of precision λ_y is like having one measurement with value $\bar{y}$ and precision $N\lambda_y$.

We can rewrite the results in terms of the posterior variance, rather than posterior precision,

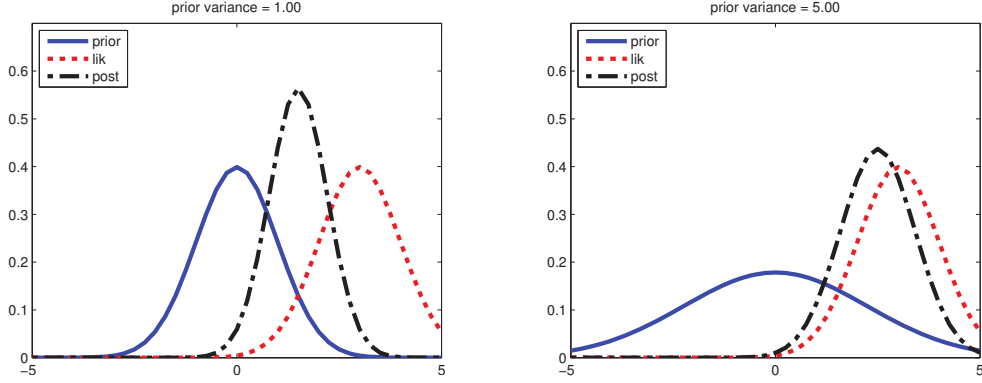


Figure 4.12 Inference about x given a noisy observation $y = 3$. (a) Strong prior $\mathcal{N}(0, 1)$. The posterior mean is “shrunk” towards the prior mean, which is 0. (a) Weak prior $\mathcal{N}(0, 5)$. The posterior mean is similar to the MLE. Figure generated by `gaussInferParamsMean1d`.

as follows:

$$p(x|\mathcal{D}, \sigma^2) = \mathcal{N}(x|\mu_N, \tau_N^2) \quad (4.132)$$

$$\tau_N^2 = \frac{1}{\frac{N}{\sigma^2} + \frac{1}{\tau_0^2}} = \frac{\sigma^2 \tau_0^2}{N\tau_0^2 + \sigma^2} \quad (4.133)$$

$$\mu_N = \tau_N^2 \left(\frac{\mu_0}{\tau_0^2} + \frac{N\bar{y}}{\sigma^2} \right) = \frac{\sigma^2}{N\tau_0^2 + \sigma^2} \mu_0 + \frac{N\tau_0^2}{N\tau_0^2 + \sigma^2} \bar{y} \quad (4.134)$$

where $\tau_0^2 = 1/\lambda_0$ is the prior variance and $\tau_N^2 = 1/\lambda_N$ is the posterior variance.

We can also compute the posterior sequentially, by updating after each observation. If $N = 1$, we can rewrite the posterior after seeing a single observation as follows (where we define $\Sigma_y = \sigma^2$, $\Sigma_0 = \tau_0^2$ and $\Sigma_1 = \tau_1^2$ to be the variances of the likelihood, prior and posterior):

$$p(x|y) = \mathcal{N}(x|\mu_1, \Sigma_1) \quad (4.135)$$

$$\Sigma_1 = \left(\frac{1}{\Sigma_0} + \frac{1}{\Sigma_y} \right)^{-1} = \frac{\Sigma_y \Sigma_0}{\Sigma_0 + \Sigma_y} \quad (4.136)$$

$$\mu_1 = \Sigma_1 \left(\frac{\mu_0}{\Sigma_0} + \frac{y}{\Sigma_y} \right) \quad (4.137)$$

We can rewrite the posterior mean in 3 different ways:

$$\mu_1 = \frac{\Sigma_y}{\Sigma_y + \Sigma_0} \mu_0 + \frac{\Sigma_0}{\Sigma_y + \Sigma_0} y \quad (4.138)$$

$$= \mu_0 + (y - \mu_0) \frac{\Sigma_0}{\Sigma_y + \Sigma_0} \quad (4.139)$$

$$= y - (y - \mu_0) \frac{\Sigma_y}{\Sigma_y + \Sigma_0} \quad (4.140)$$

The first equation is a convex combination of the prior and the data. The second equation is the prior mean adjusted towards the data. The third equation is the data adjusted towards the prior mean; this is called **shrinkage**. These are all equivalent ways of expressing the tradeoff between likelihood and prior. If Σ_0 is small relative to Σ_y , corresponding to a strong prior, the amount of shrinkage is large (see Figure 4.12(a)), whereas if Σ_0 is large relative to Σ_y , corresponding to a weak prior, the amount of shrinkage is small (see Figure 4.12(b)).

Another way to quantify the amount of shrinkage is in terms of the **signal-to-noise ratio**, which is defined as follows:

$$\text{SNR} \triangleq \frac{\mathbb{E}[X^2]}{\mathbb{E}[\epsilon^2]} = \frac{\Sigma_0 + \mu_0^2}{\Sigma_y} \quad (4.141)$$

where $x \sim \mathcal{N}(\mu_0, \Sigma_0)$ is the true signal, $y = x + \epsilon$ is the observed signal, and $\epsilon \sim \mathcal{N}(0, \Sigma_y)$ is the noise term.

4.4.2.2 Inferring an unknown vector from noisy measurements

Now consider N vector-valued observations, $\mathbf{y}_i \sim \mathcal{N}(\mathbf{x}, \Sigma_y)$, and a Gaussian prior, $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}_0, \Sigma_0)$. Setting $\mathbf{A} = \mathbf{I}$, $\mathbf{b} = \mathbf{0}$, and using $\bar{\mathbf{y}}$ for the effective observation with precision $N\Sigma_y^{-1}$, we have

$$p(\mathbf{x}|\mathbf{y}_1, \dots, \mathbf{y}_N) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_N, \Sigma_N) \quad (4.142)$$

$$\Sigma_N^{-1} = \Sigma_0^{-1} + N\Sigma_y^{-1} \quad (4.143)$$

$$\boldsymbol{\mu}_N = \Sigma_N(\Sigma_y^{-1}(N\bar{\mathbf{y}}) + \Sigma_0^{-1}\boldsymbol{\mu}_0) \quad (4.144)$$

See Figure 4.13 for a 2d example. We can think of $\mathbf{x}$ as representing the true, but unknown, location of an object in 2d space, such as a missile or airplane, and the $\mathbf{y}_i$ as being noisy observations, such as radar “blips”. As we receive more blips, we are better able to localize the source. In Section 18.3.1, we will see how to extend this example to track moving objects using the famous Kalman filter algorithm.

Now suppose we have multiple measuring devices, and we want to combine them together; this is known as **sensor fusion**. If we have multiple observations with different covariances (corresponding to sensors with different reliabilities), the posterior will be an appropriate weighted average of the data. Consider the example in Figure 4.14. We use an uninformative prior on $\mathbf{x}$, namely $p(\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_0, \Sigma_0) = \mathcal{N}(\mathbf{0}, 10^{10}\mathbf{I}_2)$. We get 2 noisy observations, $\mathbf{y}_1 \sim \mathcal{N}(\mathbf{x}, \Sigma_{y,1})$ and $\mathbf{y}_2 \sim \mathcal{N}(\mathbf{x}, \Sigma_{y,2})$. We then compute $p(\mathbf{x}|\mathbf{y}_1, \mathbf{y}_2)$.

In Figure 4.14(a), we set $\Sigma_{y,1} = \Sigma_{y,2} = 0.01\mathbf{I}_2$, so both sensors are equally reliable. In this case, the posterior mean is half way between the two observations, $\mathbf{y}_1$ and $\mathbf{y}_2$. In Figure 4.14(b), we set $\Sigma_{y,1} = 0.05\mathbf{I}_2$ and $\Sigma_{y,2} = 0.01\mathbf{I}_2$, so sensor 2 is more reliable than sensor 1. In this case, the posterior mean is closer to $\mathbf{y}_2$. In Figure 4.14(c), we set

$$\Sigma_{y,1} = 0.01 \begin{pmatrix} 10 & 1 \\ 1 & 1 \end{pmatrix}, \quad \Sigma_{y,2} = 0.01 \begin{pmatrix} 1 & 1 \\ 1 & 10 \end{pmatrix} \quad (4.145)$$

so sensor 1 is more reliable in the y_2 component (vertical direction), and sensor 2 is more reliable in the y_1 component (horizontal direction). In this case, the posterior mean uses $\mathbf{y}_1$'s vertical component and $\mathbf{y}_2$'s horizontal component.

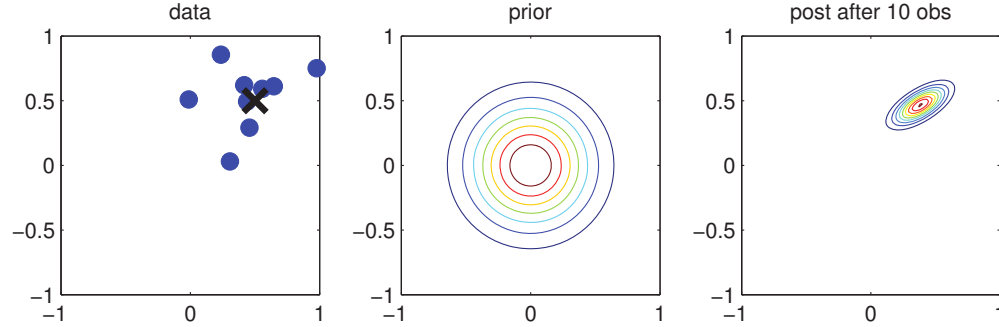


Figure 4.13 Illustration of Bayesian inference for the mean of a 2d Gaussian. (a) The data is generated from $\mathbf{y}_i \sim \mathcal{N}(\mathbf{x}, \Sigma_y)$, where $\mathbf{x} = [0.5, 0.5]^T$ and $\Sigma_y = 0.1[2, 1; 1, 1]$. We assume the sensor noise covariance Σ_y is known but $\mathbf{x}$ is unknown. The black cross represents $\mathbf{x}$. (b) The prior is $p(\mathbf{x}) = \mathcal{N}(\mathbf{x}|\mathbf{0}, 0.1\mathbf{I}_2)$. (c) We show the posterior after 10 data points have been observed. Figure generated by `gaussInferParamsMean2d`.

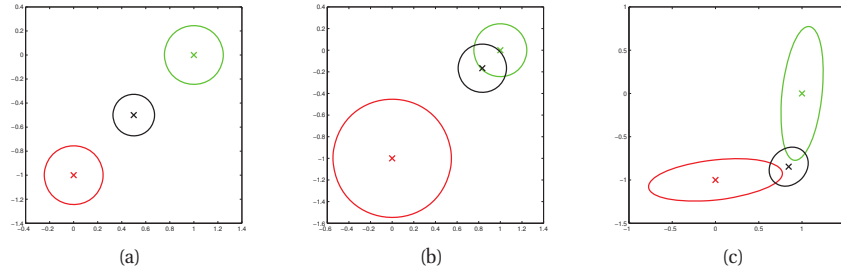


Figure 4.14 We observe $\mathbf{y}_1 = (0, -1)$ (red cross) and $\mathbf{y}_2 = (1, 0)$ (green cross) and infer $E(\boldsymbol{\mu}|\mathbf{y}_1, \mathbf{y}_2, \boldsymbol{\theta})$ (black cross). (a) Equally reliable sensors, so the posterior mean estimate is in between the two circles. (b) Sensor 2 is more reliable, so the estimate shifts more towards the green circle. (c) Sensor 1 is more reliable in the vertical direction, Sensor 2 is more reliable in the horizontal direction. The estimate is an appropriate combination of the two measurements. Figure generated by `sensorFusion2d`.

Note that this technique crucially relies on modeling our uncertainty of each sensor; computing an unweighted average would give the wrong result. However, we have assumed the sensor precisions are known. When they are not, we should model out uncertainty about Σ_1 and Σ_2 as well. See Section 4.6.4 for details.

4.4.2.3 Interpolating noisy data

We now revisit the example of Section 4.3.2.2. This time we no longer assume noise-free observations. Instead, let us assume that we obtain N noisy observations y_i ; without loss of generality, assume these correspond to $x_1, \dots, x_N$. We can model this setup as a linear

Gaussian system:

$$\mathbf{y} = \mathbf{A}\mathbf{x} + \epsilon \quad (4.146)$$

where $\epsilon \sim \mathcal{N}(\mathbf{0}, \Sigma_y)$, $\Sigma_y = \sigma^2 \mathbf{I}$, σ^2 is the observation noise, and $\mathbf{A}$ is a $N \times D$ projection matrix that selects out the observed elements. For example, if $N = 2$ and $D = 4$ we have

$$\mathbf{A} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \quad (4.147)$$

Using the same improper prior as before, $\Sigma_x = (\mathbf{L}^T \mathbf{L})^{-1}$, we can easily compute the posterior mean and variance. In Figure 4.15, we plot the posterior mean, posterior variance, and some posterior samples. Now we see that the prior precision λ effects the posterior mean as well as the posterior variance. In particular, for a strong prior (large λ), the estimate is very smooth, and the uncertainty is low. but for a weak prior (small λ), the estimate is wiggly, and the uncertainty (away from the data) is high.

The posterior mean can also be computed by solving the following optimization problem:

$$\min_{\mathbf{x}} \frac{1}{2\sigma^2} \sum_{i=1}^N (x_i - y_i)^2 + \frac{\lambda}{2} \sum_{j=1}^D [(x_j - x_{j-1})^2 + (x_j - x_{j+1})^2] \quad (4.148)$$

where we have defined $x_0 = x_1$ and $x_{D+1} = x_D$ for notational simplicity. We recognize this as a discrete approximation to the following problem:

$$\min_f \frac{1}{2\sigma^2} \int (f(t) - y(t))^2 dt + \frac{\lambda}{2} \int [f'(t)]^2 dt \quad (4.149)$$

where $f'(t)$ is the first derivative of f . The first term measures fit to the data, and the second term penalizes functions that are “too wiggly”. This is an example of **Tikhonov regularization**, which is a popular approach to **functional data analysis**. See Chapter 15 for more sophisticated approaches, which enforce higher order smoothness (so the resulting samples look less “jagged”).

4.4.3 Proof of the result *

We now derive Equation 4.125. The basic idea is to derive the joint distribution, $p(\mathbf{x}, \mathbf{y}) = p(\mathbf{x})p(\mathbf{y}|\mathbf{x})$, and then to use the results from Section 4.3.1 for computing $p(\mathbf{x}|\mathbf{y})$.

In more detail, we proceed as follows. The log of the joint distribution is as follows (dropping irrelevant constants):

$$\log p(\mathbf{x}, \mathbf{y}) = -\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_x)^T \Sigma_x^{-1} (\mathbf{x} - \boldsymbol{\mu}_x) - \frac{1}{2}(\mathbf{y} - \mathbf{A}\mathbf{x} - \mathbf{b})^T \Sigma_y^{-1} (\mathbf{y} - \mathbf{A}\mathbf{x} - \mathbf{b}) \quad (4.150)$$

This is clearly a joint Gaussian distribution, since it is the exponential of a quadratic form.

Expanding out the quadratic terms involving $\mathbf{x}$ and $\mathbf{y}$, and ignoring linear and constant terms, we have

$$Q = -\frac{1}{2}\mathbf{x}^T \Sigma_x^{-1} \mathbf{x} - \frac{1}{2}\mathbf{y}^T \Sigma_y^{-1} \mathbf{y} - \frac{1}{2}(\mathbf{A}\mathbf{x})^T \Sigma_y^{-1} (\mathbf{A}\mathbf{x}) + \mathbf{y}^T \Sigma_y^{-1} \mathbf{A}\mathbf{x} \quad (4.151)$$

$$= -\frac{1}{2} \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix}^T \begin{pmatrix} \Sigma_x^{-1} + \mathbf{A}^T \Sigma_y^{-1} \mathbf{A} & -\mathbf{A}^T \Sigma_y^{-1} \\ -\Sigma_y^{-1} \mathbf{A} & \Sigma_y^{-1} \end{pmatrix} \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix} \quad (4.152)$$

$$= -\frac{1}{2} \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix}^T \Sigma^{-1} \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix} \quad (4.153)$$

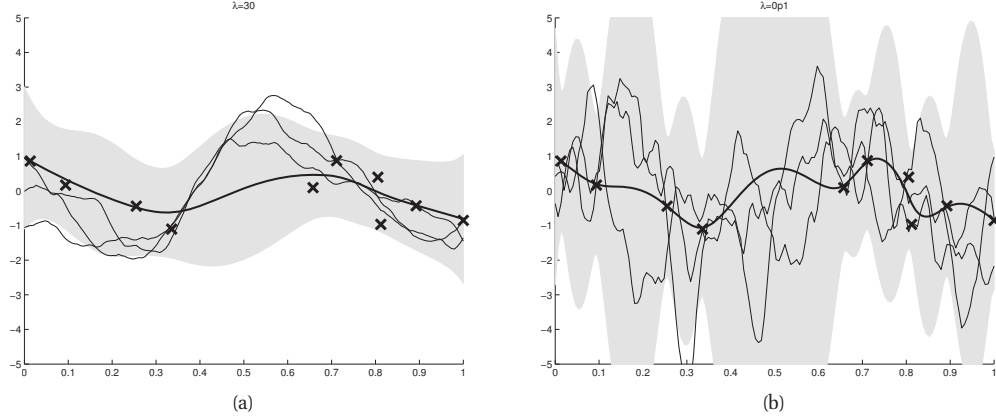


Figure 4.15 Interpolating noisy data (noise variance $\sigma^2 = 1$) using a Gaussian with prior precision λ . (a) $\lambda = 30$. (b) $\lambda = 0.01$. See also Figure 4.10. Based on Figure 7.1 of (Calvetti and Somersalo 2007). Figure generated by `gaussInterpNoisyDemo`. See also `splineBasisDemo`.

where the precision matrix of the joint is defined as

$$\Sigma^{-1} = \begin{pmatrix} \Sigma_x^{-1} + \mathbf{A}^T \Sigma_y^{-1} \mathbf{A} & -\mathbf{A}^T \Sigma_y^{-1} \\ -\Sigma_y^{-1} \mathbf{A} & \Sigma_y^{-1} \end{pmatrix} \triangleq \Lambda = \begin{pmatrix} \Lambda_{xx} & \Lambda_{xy} \\ \Lambda_{yx} & \Lambda_{yy} \end{pmatrix} \quad (4.154)$$

From Equation 4.69, and using the fact that $\mu_y = \mathbf{A}\mu_x + \mathbf{b}$, we have

$$p(\mathbf{x}|\mathbf{y}) = \mathcal{N}(\mu_{x|y}, \Sigma_{x|y}) \quad (4.155)$$

$$\Sigma_{x|y} = \Lambda_{xx}^{-1} = (\Sigma_x^{-1} + \mathbf{A}^T \Sigma_y^{-1} \mathbf{A})^{-1} \quad (4.156)$$

$$\mu_{x|y} = \Sigma_{x|y} (\Lambda_{xx} \mu_x - \Lambda_{xy} (\mathbf{y} - \mu_y)) \quad (4.157)$$

$$= \Sigma_{x|y} (\Sigma_x^{-1} \mu + \mathbf{A}^T \Sigma_y^{-1} (\mathbf{y} - \mathbf{b})) \quad (4.158)$$

4.5 Digression: The Wishart distribution *

The **Wishart** distribution is the generalization of the Gamma distribution to positive definite matrices. Press (Press 2005, p107) has said “The Wishart distribution ranks next to the (multivariate) normal distribution in order of importance and usefulness in multivariate statistics”. We will mostly use it to model our uncertainty in covariance matrices, Σ , or their inverses, $\Lambda = \Sigma^{-1}$.

The pdf of the Wishart is defined as follows:

$$\text{Wi}(\Lambda|\mathbf{S}, \nu) = \frac{1}{Z_{\text{Wi}}} |\Lambda|^{(\nu-D-1)/2} \exp\left(-\frac{1}{2} \text{tr}(\Lambda \mathbf{S})\right) \quad (4.159)$$

Here ν is called the “degrees of freedom” and $\mathbf{S}$ is the “scale matrix”. (We shall get more intuition for these parameters shortly.) The normalization constant for this distribution (which

requires integrating over all symmetric pd matrices) is the following formidable expression

$$Z_{\text{Wi}} = 2^{\nu D/2} \Gamma_D(\nu/2) |\mathbf{S}|^{\nu/2} \quad (4.160)$$

where $\Gamma_D(a)$ is the **multivariate gamma function**:

$$\Gamma_D(x) = \pi^{D(D-1)/4} \prod_{i=1}^D \Gamma(x + (1-i)/2) \quad (4.161)$$

Hence $\Gamma_1(a) = \Gamma(a)$ and

$$\Gamma_D(\nu_0/2) = \prod_{i=1}^D \Gamma\left(\frac{\nu_0 + 1 - i}{2}\right) \quad (4.162)$$

The normalization constant only exists (and hence the pdf is only well defined) if $\nu > D - 1$.

There is a connection between the Wishart distribution and the Gaussian. In particular, let $\mathbf{x}_i \sim \mathcal{N}(0, \mathbf{\Sigma})$. Then the scatter matrix $\mathbf{S} = \sum_{i=1}^N \mathbf{x}_i \mathbf{x}_i^T$ has a Wishart distribution: $\mathbf{S} \sim \text{Wi}(\mathbf{\Sigma}, 1)$. Hence $\mathbb{E}[\mathbf{S}] = N\mathbf{\Sigma}$. More generally, one can show that the mean and mode of $\text{Wi}(\mathbf{S}, \nu)$ are given by

$$\text{mean} = \nu \mathbf{S}, \quad \text{mode} = (\nu - D - 1) \mathbf{S} \quad (4.163)$$

where the mode only exists if $\nu > D + 1$.

If $D = 1$, the Wishart reduces to the Gamma distribution:

$$\text{Wi}(\lambda | s^{-1}, \nu) = \text{Ga}(\lambda | \frac{\nu}{2}, \frac{s}{2}) \quad (4.164)$$

4.5.1 Inverse Wishart distribution

Recall that we showed (Exercise 2.10) that if $\lambda \sim \text{Ga}(a, b)$, then that $\frac{1}{\lambda} \sim \text{IG}(a, b)$. Similarly, if $\mathbf{\Sigma}^{-1} \sim \text{Wi}(\mathbf{S}, \nu)$ then $\mathbf{\Sigma} \sim \text{IW}(\mathbf{S}^{-1}, \nu + D + 1)$, where IW is the **inverse Wishart**, the multidimensional generalization of the inverse Gamma. It is defined as follows, for $\nu > D - 1$ and $\mathbf{S} \succ 0$:

$$\text{IW}(\mathbf{\Sigma} | \mathbf{S}, \nu) = \frac{1}{Z_{\text{IW}}} |\mathbf{\Sigma}|^{-(\nu+D+1)/2} \exp\left(-\frac{1}{2} \text{tr}(\mathbf{S}^{-1} \mathbf{\Sigma}^{-1})\right) \quad (4.165)$$

$$Z_{\text{IW}} = |\mathbf{S}|^{-\nu/2} 2^{\nu D/2} \Gamma_D(\nu/2) \quad (4.166)$$

One can show that the distribution has these properties

$$\text{mean} = \frac{\mathbf{S}^{-1}}{\nu - D - 1}, \quad \text{mode} = \frac{\mathbf{S}^{-1}}{\nu + D + 1} \quad (4.167)$$

If $D = 1$, this reduces to the inverse Gamma:

$$\text{IW}(\sigma^2 | S^{-1}, \nu) = \text{IG}(\sigma^2 | \nu/2, S/2) \quad (4.168)$$

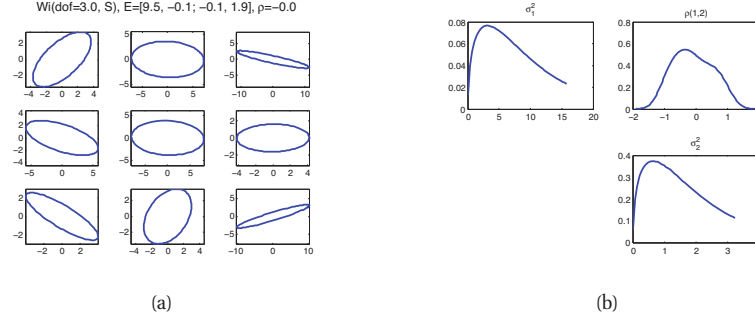


Figure 4.16 Visualization of the Wishart distribution. Left: Some samples from the Wishart distribution, $\Sigma \sim \text{Wi}(\mathbf{S}, \nu)$, where $\mathbf{S} = [3.1653, -0.0262; -0.0262, 0.6477]$ and $\nu = 3$. Right: Plots of the marginals (which are Gamma), and the approximate (sample-based) marginal on the correlation coefficient. If $\nu = 3$ there is a lot of uncertainty about the value of the correlation coefficient ρ (see the almost uniform distribution on $[-1, 1]$). The sampled matrices are highly variable, and some are nearly singular. As ν increases, the sampled matrices are more concentrated on the prior $\mathbf{S}$. Figure generated by `wiPlotDemo`.

4.5.2 Visualizing the Wishart distribution *

Since the Wishart is a distribution over matrices, it is hard to plot as a density function. However, we can easily sample from it, and in the 2d case, we can use the eigenvectors of the resulting matrix to define an ellipse, as explained in Section 4.1.2. See Figure 4.16 for some examples.

For higher dimensional matrices, we can plot marginals of the distribution. The diagonals of a Wishart distributed matrix have Gamma distributions, so are easy to plot. It is hard in general to work out the distribution of the off-diagonal elements, but we can sample matrices from the distribution, and then compute the distribution empirically. In particular, we can convert each sampled matrix to a correlation matrix, and thus compute a Monte Carlo approximation (Section 2.7) to the expected correlation coefficients:

$$\mathbb{E}[R_{ij}] \approx \frac{1}{S} \sum_{s=1}^S \mathbf{R}(\Sigma^{(s)})_{ij} \quad (4.169)$$

where $\Sigma^{(s)} \sim \text{Wi}(\Sigma, \nu)$ and $\mathbf{R}(\Sigma)$ converts matrix Σ into a correlation matrix:

$$R_{ij} = \frac{\Sigma_{ij}}{\sqrt{\Sigma_{ii} \Sigma_{jj}}} \quad (4.170)$$

We can then use kernel density estimation (Section 14.7.2) to produce a smooth approximation to the univariate density $\mathbb{E}[R_{ij}]$ for plotting purposes. See Figure 4.16 for some examples.

4.6 Inferring the parameters of an MVN

So far, we have discussed inference in a Gaussian assuming the parameters $\theta = (\mu, \Sigma)$ are known. We now discuss how to infer the parameters themselves. We will assume the data has

the form $\mathbf{x}_i \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ for $i = 1 : N$ and is fully observed, so we have no missing data (see Section 11.6.1 for how to estimate parameters of an MVN in the presence of missing values). To simplify the presentation, we derive the posterior in three parts: first we compute $p(\boldsymbol{\mu}|\mathcal{D}, \boldsymbol{\Sigma})$; then we compute $p(\boldsymbol{\Sigma}|\mathcal{D}, \boldsymbol{\mu})$; finally we compute the joint $p(\boldsymbol{\mu}, \boldsymbol{\Sigma}|\mathcal{D})$.

4.6.1 Posterior distribution of $\boldsymbol{\mu}$

We have discussed how to compute the MLE for $\boldsymbol{\mu}$; we now discuss how to compute its posterior, which is useful for modeling our uncertainty about its value.

The likelihood has the form

$$p(\mathcal{D}|\boldsymbol{\mu}) = \mathcal{N}(\bar{\mathbf{x}}|\boldsymbol{\mu}, \frac{1}{N}\boldsymbol{\Sigma}) \quad (4.171)$$

For simplicity, we will use a conjugate prior, which in this case is a Gaussian. In particular, if $p(\boldsymbol{\mu}) = \mathcal{N}(\boldsymbol{\mu}|\mathbf{m}_0, \mathbf{V}_0)$ then we can derive a Gaussian posterior for $\boldsymbol{\mu}$ based on the results in Section 4.4.2.2. We get

$$p(\boldsymbol{\mu}|\mathcal{D}, \boldsymbol{\Sigma}) = \mathcal{N}(\boldsymbol{\mu}|\mathbf{m}_N, \mathbf{V}_N) \quad (4.172)$$

$$\mathbf{V}_N^{-1} = \mathbf{V}_0^{-1} + N\boldsymbol{\Sigma}^{-1} \quad (4.173)$$

$$\mathbf{m}_N = \mathbf{V}_N(\boldsymbol{\Sigma}^{-1}(N\bar{\mathbf{x}}) + \mathbf{V}_0^{-1}\mathbf{m}_0) \quad (4.174)$$

This is exactly the same process as inferring the location of an object based on noisy radar “blips”, except now we are inferring the mean of a distribution based on noisy samples. (To a Bayesian, there is no difference between uncertainty about parameters and uncertainty about anything else.)

We can model an uninformative prior by setting $\mathbf{V}_0 = \infty\mathbf{I}$. In this case we have $p(\boldsymbol{\mu}|\mathcal{D}, \boldsymbol{\Sigma}) = \mathcal{N}(\bar{\mathbf{x}}, \frac{1}{N}\boldsymbol{\Sigma})$, so the posterior mean is equal to the MLE. We also see that the posterior variance goes down as $1/N$, which is a standard result from frequentist statistics.

4.6.2 Posterior distribution of $\boldsymbol{\Sigma}$

We now discuss how to compute $p(\boldsymbol{\Sigma}|\mathcal{D}, \boldsymbol{\mu})$. The likelihood has the form

$$p(\mathcal{D}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) \propto |\boldsymbol{\Sigma}|^{-\frac{N}{2}} \exp\left(-\frac{1}{2}\text{tr}(\mathbf{S}_\mu \boldsymbol{\Sigma}^{-1})\right) \quad (4.175)$$

The corresponding conjugate prior is known as the inverse Wishart distribution (Section 4.5.1). Recall that this has the following pdf:

$$\text{IW}(\boldsymbol{\Sigma}|\mathbf{S}_0^{-1}, \nu_0) \propto |\boldsymbol{\Sigma}|^{-(\nu_0 + D + 1)/2} \exp\left(-\frac{1}{2}\text{tr}(\mathbf{S}_0 \boldsymbol{\Sigma}^{-1})\right) \quad (4.176)$$

Here $\nu_0 > D - 1$ is the degrees of freedom (dof), and $\mathbf{S}_0$ is a symmetric pd matrix. We see that $\mathbf{S}_0^{-1}$ plays the role of the prior scatter matrix, and $N_0 \triangleq \nu_0 + D + 1$ controls the strength of the prior, and hence plays a role analogous to the sample size N .

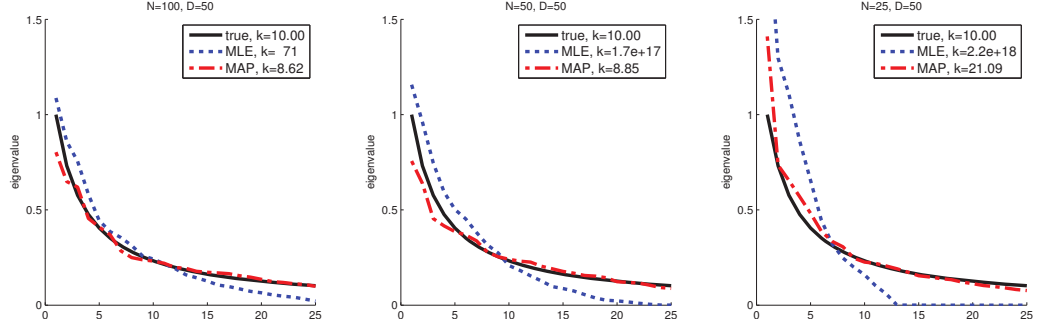


Figure 4.17 Estimating a covariance matrix in $D = 50$ dimensions using $N \in \{100, 50, 25\}$ samples. We plot the eigenvalues in descending order for the true covariance matrix (solid black), the MLE (dotted blue) and the MAP estimate (dashed red), using Equation 4.184 with $\lambda = 0.9$. We also list the condition number of each matrix in the legend. Based on Figure 1 of (Schaefer and Strimmer 2005). Figure generated by `shrinkcovDemo`.

Multiplying the likelihood and prior we find that the posterior is also inverse Wishart:

$$p(\Sigma|\mathcal{D}, \mu) \propto |\Sigma|^{-\frac{N}{2}} \exp\left(-\frac{1}{2}\text{tr}(\Sigma^{-1}\mathbf{S}_\mu)\right) |\Sigma|^{-(\nu_0+D+1)/2} \exp\left(-\frac{1}{2}\text{tr}(\Sigma^{-1}\mathbf{S}_0)\right) \quad (4.177)$$

$$= |\Sigma|^{-\frac{N+(\nu_0+D+1)}{2}} \exp\left(-\frac{1}{2}\text{tr}[\Sigma^{-1}(\mathbf{S}_\mu + \mathbf{S}_0)]\right) \quad (4.178)$$

$$= \text{IW}(\Sigma|\mathbf{S}_N, \nu_N) \quad (4.179)$$

$$\nu_N = \nu_0 + N \quad (4.180)$$

$$\mathbf{S}_N^{-1} = \mathbf{S}_0 + \mathbf{S}_\mu \quad (4.181)$$

In words, this says that the posterior strength ν_N is the prior strength ν_0 plus the number of observations N , and the posterior scatter matrix $\mathbf{S}_N$ is the prior scatter matrix $\mathbf{S}_0$ plus the data scatter matrix $\mathbf{S}_\mu$.

4.6.2.1 MAP estimation

We see from Equation 4.7 that $\hat{\Sigma}_{mle}$ is a rank $\min(N, D)$ matrix. If $N < D$, this is not full rank, and hence will be uninvertible. And even if $N > D$, it may be the case that $\hat{\Sigma}$ is ill-conditioned (meaning it is nearly singular).

To solve these problems, we can use the posterior mode (or mean). One can show (using techniques analogous to the derivation of the MLE) that the MAP estimate is given by

$$\hat{\Sigma}_{map} = \frac{\mathbf{S}_N}{\nu_N + D + 1} = \frac{\mathbf{S}_0 + \mathbf{S}_\mu}{N_0 + N} \quad (4.182)$$

If we use an improper uniform prior, corresponding to $N_0 = 0$ and $\mathbf{S}_0 = \mathbf{0}$, we recover the MLE.

Let us now consider the use of a proper informative prior, which is necessary whenever D/N is large (say bigger than 0.1). Let $\boldsymbol{\mu} = \bar{\mathbf{x}}$, so $\mathbf{S}_\mu = \mathbf{S}_{\bar{\mathbf{x}}}$. Then we can rewrite the MAP estimate as a convex combination of the prior mode and the MLE. To see this, let $\boldsymbol{\Sigma}_0 \triangleq \frac{\mathbf{S}_0}{N_0}$ be the prior mode. Then the posterior mode can be rewritten as

$$\hat{\boldsymbol{\Sigma}}_{map} = \frac{\mathbf{S}_0 + \mathbf{S}_{\bar{\mathbf{x}}}}{N_0 + N} = \frac{N_0}{N_0 + N} \frac{\mathbf{S}_0}{N_0} + \frac{N}{N_0 + N} \frac{\mathbf{S}}{N} = \lambda \boldsymbol{\Sigma}_0 + (1 - \lambda) \hat{\boldsymbol{\Sigma}}_{mle} \quad (4.183)$$

where $\lambda = \frac{N_0}{N_0 + N}$, controls the amount of shrinkage towards the prior.

This begs the question: where do the parameters of the prior come from? It is common to set λ by cross validation. Alternatively, we can use the closed-form formula provided in (Ledoit and Wolf 2004b,a; Schaefer and Strimmer 2005), which is the optimal frequentist estimate if we use squared loss. This is arguably not the most natural loss function for covariance matrices (because it ignores the positive definite constraint), but it results in a simple estimator, which is implemented in the PMTK function `shrinkcov`. We discuss Bayesian ways of estimating λ later.

As for the prior covariance matrix, $\mathbf{S}_0$, it is common to use the following (data dependent) prior: $\mathbf{S}_0 = \text{diag}(\hat{\boldsymbol{\Sigma}}_{mle})$. In this case, the MAP estimate is given by

$$\hat{\boldsymbol{\Sigma}}_{map}(i, j) = \begin{cases} \hat{\boldsymbol{\Sigma}}_{mle}(i, j) & \text{if } i = j \\ (1 - \lambda) \hat{\boldsymbol{\Sigma}}_{mle}(i, j) & \text{otherwise} \end{cases} \quad (4.184)$$

Thus we see that the diagonal entries are equal to their ML estimates, and the off diagonal elements are “shrunk” somewhat towards 0. This technique is therefore called **shrinkage estimation**, or **regularized estimation**.

The benefits of MAP estimation are illustrated in Figure 4.17. We consider fitting a 50 dimensional Gaussian to $N = 100$, $N = 50$ and $N = 25$ data points. We see that the MAP estimate is always well-conditioned, unlike the MLE. In particular, we see that the **eigenvalue spectrum** of the MAP estimate is much closer to that of the true matrix than the MLE’s. The eigenvectors, however, are unaffected.

The importance of regularizing the estimate of $\boldsymbol{\Sigma}$ will become apparent in later chapters, when we consider fitting covariance matrices to high dimensional data.

4.6.2.2 Univariate posterior

In the 1d case, the likelihood has the form

$$p(\mathcal{D}|\sigma^2) \propto (\sigma^2)^{-N/2} \exp\left(-\frac{1}{2\sigma^2} \sum_{i=1}^N (x_i - \mu)^2\right) \quad (4.185)$$

The standard conjugate prior is the inverse Gamma distribution, which is just the scalar version of the inverse Wishart:

$$\text{IG}(\sigma^2|a_0, b_0) \propto (\sigma^2)^{-(a_0+1)} \exp\left(-\frac{b_0}{\sigma^2}\right) \quad (4.186)$$

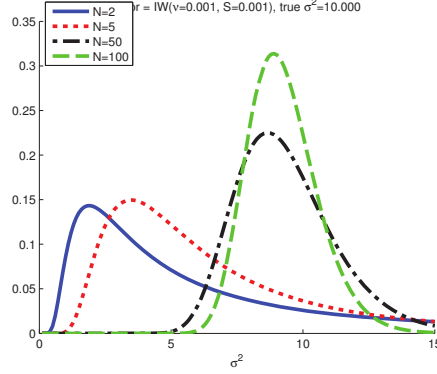


Figure 4.18 Sequential updating of the posterior for σ^2 starting from an uninformative prior. The data was generated from a Gaussian with known mean $\mu = 5$ and unknown variance $\sigma^2 = 10$. Figure generated by `gaussSeqUpdateSigma1D`.

Multiplying the likelihood and the prior, we see that the posterior is also IG:

$$p(\sigma^2|\mathcal{D}) = \text{IG}(\sigma^2|a_N, b_N) \quad (4.187)$$

$$a_N = a_0 + N/2 \quad (4.188)$$

$$b_N = b_0 + \frac{1}{2} \sum_{i=1}^N (x_i - \mu)^2 \quad (4.189)$$

See Figure 4.18 for an illustration.

The form of the posterior is not quite as pretty as the multivariate case, because of the factors of $\frac{1}{2}$. This arises because $\text{IW}(\sigma^2|s_0, \nu_0) = \text{IG}(\sigma^2|\frac{s_0}{2}, \frac{\nu_0}{2})$. Another problem with using the $\text{IG}(a_0, b_0)$ distribution is that the strength of the prior is encoded in both a_0 and b_0 . To avoid both of these problems, it is common (in the statistics literature) to use an alternative parameterization of the IG distribution, known as the (scaled) **inverse chi-squared distribution**. This is defined as follows:

$$\chi^{-2}(\sigma^2|\nu_0, \sigma_0^2) = \text{IG}(\sigma^2|\frac{\nu_0}{2}, \frac{\nu_0 \sigma_0^2}{2}) \propto (\sigma^2)^{-\nu_0/2-1} \exp(-\frac{\nu_0 \sigma_0^2}{2\sigma^2}) \quad (4.190)$$

Here ν_0 controls the strength of the prior, and σ_0^2 encodes the value of the prior. With this prior, the posterior becomes

$$p(\sigma^2|\mathcal{D}, \mu) = \chi^{-2}(\sigma^2|\nu_N, \sigma_N^2) \quad (4.191)$$

$$\nu_N = \nu_0 + N \quad (4.192)$$

$$\sigma_N^2 = \frac{\nu_0 \sigma_0^2 + \sum_{i=1}^N (x_i - \mu)^2}{\nu_N} \quad (4.193)$$

We see that the posterior dof ν_N is the prior dof ν_0 plus N , and the posterior sum of squares $\nu_N \sigma_N^2$ is the prior sum of squares $\nu_0 \sigma_0^2$ plus the data sum of squares.

We can emulate an uninformative prior, $p(\sigma^2) \propto \sigma^{-2}$, by setting $\nu_0 = 0$, which makes intuitive sense (since it corresponds to a zero virtual sample size).

4.6.3 Posterior distribution of $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ *

We now discuss how to compute $p(\boldsymbol{\mu}, \boldsymbol{\Sigma} | \mathcal{D})$. These results are a bit complex, but will prove useful later on in this book. Feel free to skip this section on a first reading.

4.6.3.1 Likelihood

The likelihood is given by

$$p(\mathcal{D} | \boldsymbol{\mu}, \boldsymbol{\Sigma}) = (2\pi)^{-ND/2} |\boldsymbol{\Sigma}|^{-\frac{N}{2}} \exp \left(-\frac{1}{2} \sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x}_i - \boldsymbol{\mu}) \right) \quad (4.194)$$

Now one can show that

$$\sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x}_i - \boldsymbol{\mu}) = \text{tr}(\boldsymbol{\Sigma}^{-1} \mathbf{S}_{\bar{x}}) + N(\bar{\mathbf{x}} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\bar{\mathbf{x}} - \boldsymbol{\mu}) \quad (4.195)$$

Hence we can rewrite the likelihood as follows:

$$p(\mathcal{D} | \boldsymbol{\mu}, \boldsymbol{\Sigma}) = (2\pi)^{-ND/2} |\boldsymbol{\Sigma}|^{-\frac{N}{2}} \exp \left(-\frac{N}{2} (\boldsymbol{\mu} - \bar{\mathbf{x}})^T \boldsymbol{\Sigma}^{-1} (\boldsymbol{\mu} - \bar{\mathbf{x}}) \right) \quad (4.196)$$

$$\exp \left(-\frac{N}{2} \text{tr}(\boldsymbol{\Sigma}^{-1} \mathbf{S}_{\bar{x}}) \right) \quad (4.197)$$

We will use this form below.

4.6.3.2 Prior

The obvious prior to use is the following

$$p(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \mathcal{N}(\boldsymbol{\mu} | \mathbf{m}_0, \mathbf{V}_0) \text{IW}(\boldsymbol{\Sigma} | \mathbf{S}_0, \nu_0) \quad (4.198)$$

Unfortunately, this is not conjugate to the likelihood. To see why, note that $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ appear together in a non-factorized way in the likelihood; hence they will also be coupled together in the posterior.

The above prior is sometimes called **semi-conjugate** or **conditionally conjugate**, since both conditionals, $p(\boldsymbol{\mu} | \boldsymbol{\Sigma})$ and $p(\boldsymbol{\Sigma} | \boldsymbol{\mu})$, are individually conjugate. To create a full conjugate prior, we need to use a prior where $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ are dependent on each other. We will use a joint distribution of the form

$$p(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = p(\boldsymbol{\Sigma}) p(\boldsymbol{\mu} | \boldsymbol{\Sigma}) \quad (4.199)$$

Looking at the form of the likelihood equation, Equation 4.197, we see that a natural conjugate

prior has the form of a **Normal-inverse-wishart** or **NIW** distribution, defined as follows:

$$\text{NIW}(\boldsymbol{\mu}, \boldsymbol{\Sigma} | \mathbf{m}_0, \kappa_0, \nu_0, \mathbf{S}_0) \triangleq \quad (4.200)$$

$$\mathcal{N}(\boldsymbol{\mu} | \mathbf{m}_0, \frac{1}{\kappa_0} \boldsymbol{\Sigma}) \times \text{IW}(\boldsymbol{\Sigma} | \mathbf{S}_0, \nu_0) \quad (4.201)$$

$$= \frac{1}{Z_{\text{NIW}}} |\boldsymbol{\Sigma}|^{-\frac{1}{2}} \exp\left(-\frac{\kappa_0}{2} (\boldsymbol{\mu} - \mathbf{m}_0)^T \boldsymbol{\Sigma}^{-1} (\boldsymbol{\mu} - \mathbf{m}_0)\right) \quad (4.202)$$

$$\times |\boldsymbol{\Sigma}|^{-\frac{\nu_0 + D + 1}{2}} \exp\left(-\frac{1}{2} \text{tr}(\boldsymbol{\Sigma}^{-1} \mathbf{S}_0)\right) \quad (4.203)$$

$$= \frac{1}{Z_{\text{NIW}}} |\boldsymbol{\Sigma}|^{-\frac{\nu_0 + D + 2}{2}} \quad (4.204)$$

$$\times \exp\left(-\frac{\kappa_0}{2} (\boldsymbol{\mu} - \mathbf{m}_0)^T \boldsymbol{\Sigma}^{-1} (\boldsymbol{\mu} - \mathbf{m}_0) - \frac{1}{2} \text{tr}(\boldsymbol{\Sigma}^{-1} \mathbf{S}_0)\right) \quad (4.205)$$

$$Z_{\text{NIW}} = 2^{\nu_0 D / 2} \Gamma_D(\nu_0 / 2) (2\pi / \kappa_0)^{D/2} |\mathbf{S}_0|^{-\nu_0 / 2} \quad (4.206)$$

where $\Gamma_D(a)$ is the multivariate Gamma function.

The parameters of the NIW can be interpreted as follows: $\mathbf{m}_0$ is our prior mean for $\boldsymbol{\mu}$, and κ_0 is how strongly we believe this prior; and $\mathbf{S}_0$ is (proportional to) our prior mean for $\boldsymbol{\Sigma}$, and ν_0 is how strongly we believe this prior.³

One can show (Minka 2000f) that the (improper) uninformative prior has the form

$$\lim_{k \rightarrow 0} \mathcal{N}(\boldsymbol{\mu} | \mathbf{m}_0, \boldsymbol{\Sigma}/k) \text{IW}(\boldsymbol{\Sigma} | \mathbf{S}_0, k) \propto |2\pi \boldsymbol{\Sigma}|^{-\frac{1}{2}} |\boldsymbol{\Sigma}|^{-(D+1)/2} \quad (4.207)$$

$$\propto |\boldsymbol{\Sigma}|^{-(\frac{D}{2} + 1)} \propto \text{NIW}(\boldsymbol{\mu}, \boldsymbol{\Sigma} | \mathbf{0}, 0, 0, 0\mathbf{I}) \quad (4.208)$$

In practice, it is often better to use a weakly informative data-dependent prior. A common choice (see e.g., (Chipman et al. 2001, p81), (Fraley and Raftery 2007, p6)) is to use $\mathbf{S}_0 = \text{diag}(\mathbf{S}_{\bar{x}})/N$, and $\nu_0 = D + 2$, to ensure $\mathbb{E}[\boldsymbol{\Sigma}] = \mathbf{S}_0$, and to set $\boldsymbol{\mu}_0 = \bar{\mathbf{x}}$ and κ_0 to some small number, such as 0.01.

3. Although this prior has four parameters, there are really only three free parameters, since our uncertainty in the mean is proportional to the variance. In particular, if we believe that the variance is large, then our uncertainty in $\boldsymbol{\mu}$ must be large too. This makes sense intuitively, since if the data has large spread, it may be hard to pin down its mean. See also Exercise 9.1, where we will see the three free parameters more explicitly. If we want separate “control” over our confidence in $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$, we must use a semi-conjugate prior.

4.6.3.3 Posterior

The posterior can be shown (Exercise 4.11) to be NIW with updated parameters:

$$p(\boldsymbol{\mu}, \boldsymbol{\Sigma} | \mathcal{D}) = \text{NIW}(\boldsymbol{\mu}, \boldsymbol{\Sigma} | \mathbf{m}_N, \kappa_N, \nu_N, \mathbf{S}_N) \quad (4.209)$$

$$\mathbf{m}_N = \frac{\kappa_0 \mathbf{m}_0 + N \bar{\mathbf{x}}}{\kappa_N} = \frac{\kappa_0}{\kappa_0 + N} \mathbf{m}_0 + \frac{N}{\kappa_0 + N} \bar{\mathbf{x}} \quad (4.210)$$

$$\kappa_N = \kappa_0 + N \quad (4.211)$$

$$\nu_N = \nu_0 + N \quad (4.212)$$

$$\mathbf{S}_N = \mathbf{S}_0 + \mathbf{S}_{\bar{x}} + \frac{\kappa_0 N}{\kappa_0 + N} (\bar{\mathbf{x}} - \mathbf{m}_0)(\bar{\mathbf{x}} - \mathbf{m}_0)^T \quad (4.213)$$

$$= \mathbf{S}_0 + \mathbf{S} + \kappa_0 \mathbf{m}_0 \mathbf{m}_0^T - \kappa_N \mathbf{m}_N \mathbf{m}_N^T \quad (4.214)$$

where we have defined $\mathbf{S} \triangleq \sum_{i=1}^N \mathbf{x}_i \mathbf{x}_i^T$ as the uncentered sum-of-squares matrix (this is easier to update incrementally than the centered version).

This result is actually quite intuitive: the posterior mean is a convex combination of the prior mean and the MLE, with “strength” $\kappa_0 + N$; and the posterior scatter matrix $\mathbf{S}_N$ is the prior scatter matrix $\mathbf{S}_0$ plus the empirical scatter matrix $\mathbf{S}_{\bar{x}}$ plus an extra term due to the uncertainty in the mean (which creates its own virtual scatter matrix).

4.6.3.4 Posterior mode

The mode of the joint distribution has the following form:

$$\text{argmax}_{\boldsymbol{\mu}, \boldsymbol{\Sigma}} p(\boldsymbol{\mu}, \boldsymbol{\Sigma} | \mathcal{D}) = \left(\mathbf{m}_N, \frac{\mathbf{S}_N}{\nu_N + D + 2} \right) \quad (4.215)$$

If we set $\kappa_0 = 0$, this reduces to

$$\text{argmax}_{\boldsymbol{\mu}, \boldsymbol{\Sigma}} p(\boldsymbol{\mu}, \boldsymbol{\Sigma} | \mathcal{D}) = \left(\bar{\mathbf{x}}, \frac{\mathbf{S}_0 + \mathbf{S}_{\bar{x}}}{\nu_0 + N + D + 2} \right) \quad (4.216)$$

The corresponding estimate $\hat{\boldsymbol{\Sigma}}$ is almost the same as Equation 4.183, but differs by 1 in the denominator, because this is the mode of the joint, not the mode of the marginal.

4.6.3.5 Posterior marginals

The posterior marginal for $\boldsymbol{\Sigma}$ is simply

$$p(\boldsymbol{\Sigma} | \mathcal{D}) = \int p(\boldsymbol{\mu}, \boldsymbol{\Sigma} | \mathcal{D}) d\boldsymbol{\mu} = \text{IW}(\boldsymbol{\Sigma} | \mathbf{S}_N, \nu_N) \quad (4.217)$$

The mode and mean of this marginal are given by

$$\hat{\boldsymbol{\Sigma}}_{\text{map}} = \frac{\mathbf{S}_N}{\nu_N + D + 1}, \quad \mathbb{E}[\boldsymbol{\Sigma}] = \frac{\mathbf{S}_N}{\nu_N - D - 1} \quad (4.218)$$

One can show that the posterior marginal for $\boldsymbol{\mu}$ has a multivariate Student T distribution:

$$p(\boldsymbol{\mu} | \mathcal{D}) = \int p(\boldsymbol{\mu}, \boldsymbol{\Sigma} | \mathcal{D}) d\boldsymbol{\Sigma} = \mathcal{T}(\boldsymbol{\mu} | \mathbf{m}_N, \frac{1}{\kappa_N(\nu_N - D + 1)} \mathbf{S}_N, \nu_N - D + 1) \quad (4.219)$$

This follows from the fact that the Student distribution can be represented as a scaled mixture of Gaussians (see Equation 11.61).

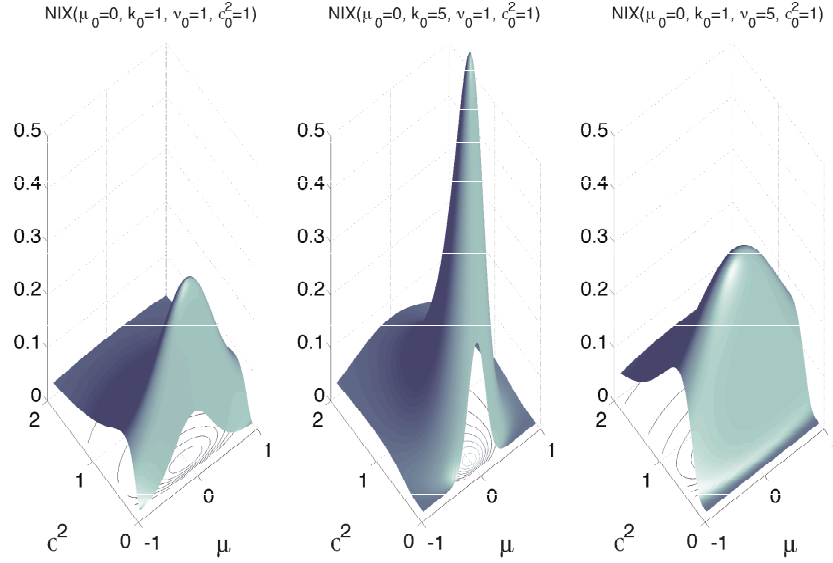


Figure 4.19 The $NI\chi^2(m_0, \kappa_0, \nu_0, \sigma_0^2)$ distribution. m_0 is the prior mean and κ_0 is how strongly we believe this; σ_0^2 is the prior variance and ν_0 is how strongly we believe this. (a) $m_0 = 0, \kappa_0 = 1, \nu_0 = 1, \sigma_0^2 = 1$. Notice that the contour plot (underneath the surface) is shaped like a “squashed egg”. (b) We increase the strength of our belief in the mean, so it gets narrower: $m_0 = 0, \kappa_0 = 5, \nu_0 = 1, \sigma_0^2 = 1$. (c) We increase the strength of our belief in the variance, so it gets narrower: $m_0 = 0, \kappa_0 = 1, \nu_0 = 5, \sigma_0^2 = 1$. Figure generated by NIXdemo2.

4.6.3.6 Posterior predictive

The posterior predictive is given by

$$p(\mathbf{x}|\mathcal{D}) = \frac{p(\mathbf{x}, \mathcal{D})}{p(\mathcal{D})} \quad (4.220)$$

so it can be easily evaluated in terms of a ratio of marginal likelihoods.

It turns out that this ratio has the form of a multivariate Student-T distribution:

$$p(\mathbf{x}|\mathcal{D}) = \int \int \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) \text{NIW}(\boldsymbol{\mu}, \boldsymbol{\Sigma}|\mathbf{m}_N, \kappa_N, \nu_N, \mathbf{S}_N) d\boldsymbol{\mu} d\boldsymbol{\Sigma} \quad (4.221)$$

$$= \mathcal{T}(\mathbf{x}|\mathbf{m}_N, \frac{\kappa_N + 1}{\kappa_N(\nu_N - D + 1)} \mathbf{S}_N, \nu_N - D + 1) \quad (4.222)$$

The Student-T has wider tails than a Gaussian, which takes into account the fact that $\boldsymbol{\Sigma}$ is unknown. However, this rapidly becomes Gaussian-like.

4.6.3.7 Posterior for scalar data

We now specialise the above results to the case where x_i is 1d. These results are widely used in the statistics literature. As in Section 4.6.2.2, it is conventional not to use the normal inverse

Wishart, but to use the **normal inverse chi-squared** or **NIX** distribution, defined by

$$NI\chi^2(\mu, \sigma^2 | m_0, \kappa_0, \nu_0, \sigma_0^2) \triangleq \mathcal{N}(\mu | m_0, \sigma^2 / \kappa_0) \chi^{-2}(\sigma^2 | \nu_0, \sigma_0^2) \quad (4.223)$$

$$\propto \left(\frac{1}{\sigma^2}\right)^{(\nu_0+3)/2} \exp\left(-\frac{\nu_0\sigma_0^2 + \kappa_0(\mu - m_0)^2}{2\sigma^2}\right) \quad (4.224)$$

See Figure 4.19 for some plots. Along the μ axis, the distribution is shaped like a Gaussian, and along the σ^2 axis, the distribution is shaped like a χ^{-2} ; the contours of the joint density have a “squashed egg” appearance. Interestingly, we see that the contours for μ are more peaked for small values of σ^2 , which makes sense, since if the data is low variance, we will be able to estimate its mean more reliably.

One can show that the posterior is given by

$$p(\mu, \sigma^2 | \mathcal{D}) = NI\chi^2(\mu, \sigma^2 | m_N, \kappa_N, \nu_N, \sigma_N^2) \quad (4.225)$$

$$m_N = \frac{\kappa_0 m_0 + N\bar{x}}{\kappa_N} \quad (4.226)$$

$$\kappa_N = \kappa_0 + N \quad (4.227)$$

$$\nu_N = \nu_0 + N \quad (4.228)$$

$$\nu_N \sigma_N^2 = \nu_0 \sigma_0^2 + \sum_{i=1}^N (x_i - \bar{x})^2 + \frac{N\kappa_0}{\kappa_0 + N} (m_0 - \bar{x})^2 \quad (4.229)$$

The posterior marginal for σ^2 is just

$$p(\sigma^2 | \mathcal{D}) = \int p(\mu, \sigma^2 | \mathcal{D}) d\mu = \chi^{-2}(\sigma^2 | \nu_N, \sigma_N^2) \quad (4.230)$$

with the posterior mean given by $\mathbb{E}[\sigma^2 | \mathcal{D}] = \frac{\nu_N}{\nu_N - 2} \sigma_N^2$.

The posterior marginal for μ has a Student T distribution, which follows from the scale mixture representation of the student:

$$p(\mu | \mathcal{D}) = \int p(\mu, \sigma^2 | \mathcal{D}) d\sigma^2 = \mathcal{T}(\mu | m_N, \sigma_N^2 / \kappa_N, \nu_N) \quad (4.231)$$

with the posterior mean given by $\mathbb{E}[\mu | \mathcal{D}] = m_N$.

Let us see how these results look if we use the following uninformative prior:

$$p(\mu, \sigma^2) \propto p(\mu)p(\sigma^2) \propto \sigma^{-2} \propto NI\chi^2(\mu, \sigma^2 | \mu_0 = 0, \kappa_0 = 0, \nu_0 = -1, \sigma_0^2 = 0) \quad (4.232)$$

With this prior, the posterior has the form

$$p(\mu, \sigma^2 | \mathcal{D}) = NI\chi^2(\mu, \sigma^2 | m_N = \bar{x}, \kappa_N = N, \nu_N = N - 1, \sigma_N^2 = s^2) \quad (4.233)$$

where

$$s^2 \triangleq \frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2 = \frac{N}{N-1} \hat{\sigma}_{mle}^2 \quad (4.234)$$

is the the **sample standard deviation**. (In Section 6.4.2, we show that this is an unbiased estimate of the variance.) Hence the marginal posterior for the mean is given by

$$p(\mu | \mathcal{D}) = \mathcal{T}(\mu | \bar{x}, \frac{s^2}{N}, N - 1) \quad (4.235)$$

and the posterior variance of μ is

$$\text{var}[\mu|\mathcal{D}] = \frac{\nu_N}{\nu_N - 2} \sigma_N^2 = \frac{N-1}{N-3} \frac{s^2}{N} \rightarrow \frac{s^2}{N} \quad (4.236)$$

The square root of this is called the **standard error of the mean**:

$$\sqrt{\text{var}[\mu|\mathcal{D}]} \approx \frac{s}{\sqrt{N}} \quad (4.237)$$

Thus an approximate 95% posterior **credible interval** for the mean is

$$I_{.95}(\mu|\mathcal{D}) = \bar{x} \pm 2 \frac{s}{\sqrt{N}} \quad (4.238)$$

(Bayesian credible intervals are discussed in more detail in Section 5.2.2; they are contrasted with frequentist confidence intervals in Section 6.6.1.)

4.6.3.8 Bayesian t-test

Suppose we want to test the hypothesis that $\mu \neq \mu_0$ for some known value μ_0 (often 0), given values $x_i \sim \mathcal{N}(\mu, \sigma^2)$. This is called a two-sided, one-sample **t-test**. A simple way to perform such a test is just to check if $\mu_0 \in I_{0.95}(\mu|\mathcal{D})$. If it is not, then we can be 95% sure that $\mu \neq \mu_0$.⁴ A more common scenario is when we want to test if two paired samples have the same mean. More precisely, suppose $y_i \sim \mathcal{N}(\mu_1, \sigma^2)$ and $z_i \sim \mathcal{N}(\mu_2, \sigma^2)$. We want to determine if $\mu = \mu_1 - \mu_2 > 0$, using $x_i = y_i - z_i$ as our data. We can evaluate this quantity as follows:

$$p(\mu > \mu_0|\mathcal{D}) = \int_{\mu_0}^{\infty} p(\mu|\mathcal{D}) d\mu \quad (4.239)$$

This is called a one-sided, **paired t-test**. (For a similar approach to unpaired tests, comparing the difference in binomial proportions, see Section 5.2.3.)

To calculate the posterior, we must specify a prior. Suppose we use an uninformative prior. As we showed above, we find that the posterior marginal on μ has the form

$$p(\mu|\mathcal{D}) = \mathcal{T}(\mu|\bar{x}, \frac{s^2}{N}, N-1) \quad (4.240)$$

Now let us define the following **t statistic**:

$$t \triangleq \frac{\bar{x} - \mu_0}{s/\sqrt{N}} \quad (4.241)$$

where the denominator is the standard error of the mean. We see that

$$p(\mu|\mathcal{D}) = 1 - F_{N-1}(t) \quad (4.242)$$

where $F_{\nu}(t)$ is the cdf of the standard Student t distribution $\mathcal{T}(0, 1, \nu)$.

4. A more complex approach is to perform Bayesian model comparison. That is, we compute the Bayes factor (described in Section 5.3.3) $p(\mathcal{D}|H_0)/p(\mathcal{D}|H_1)$, where H_0 is the point null hypothesis that $\mu = \mu_0$, and H_1 is the alternative hypothesis that $\mu \neq \mu_0$. See (Gonen et al. 2005; Rouder et al. 2009) for details.

4.6.3.9 Connection with frequentist statistics *

If we use an uninformative prior, it turns out that the above Bayesian analysis gives the same result as derived using frequentist methods. (We discuss frequentist statistics in Chapter 6.) Specifically, from the above results, we see that

$$\frac{\mu - \bar{x}}{\sqrt{s/N}} | \mathcal{D} \sim t_{N-1} \quad (4.243)$$

This has the same form as the sampling distribution of the MLE:

$$\frac{\mu - \bar{X}}{\sqrt{s/N}} | \mu \sim t_{N-1} \quad (4.244)$$

The reason is that the Student distribution is symmetric in its first two arguments, so $\mathcal{T}(\bar{x} | \mu, \sigma^2, \nu) = \mathcal{T}(\mu | \bar{x}, \sigma^2, \nu)$; hence statements about the posterior for μ have the same form as statements about the sampling distribution of $\bar{x}$. Consequently, the (one-sided) p-value (defined in Section 6.6.2) returned by a frequentist test is the same as $p(\mu > \mu_0 | \mathcal{D})$ returned by the Bayesian method. See `bayesTtestDemo` for an example.

Despite the superficial similarity, these two results have a different interpretation: in the Bayesian approach, μ is unknown and $\bar{x}$ is fixed, whereas in the frequentist approach, $\bar{X}$ is unknown and μ is fixed. More equivalences between frequentist and Bayesian inference in simple models using uninformative priors can be found in (Box and Tiao 1973). See also Section 7.6.3.3.

4.6.4 Sensor fusion with unknown precisions *

In this section, we apply the results in Section 4.6.3 to the problem of sensor fusion in the case where the precision of each measurement device is unknown. This generalizes the results of Section 4.4.2.2, where the measurement model was assumed to be Gaussian with known precision. The unknown precision case turns out to give qualitatively different results, yielding a potentially multi-modal posterior as we will see. Our presentation is based on (Minka 2001e).

Suppose we want to pool data from multiple sources to estimate some quantity $\mu \in \mathbb{R}$, but the reliability of the sources is unknown. Specifically, suppose we have two different measurement devices, x and y , with different precisions: $x_i | \mu \sim \mathcal{N}(\mu, \lambda_x^{-1})$ and $y_i | \mu \sim \mathcal{N}(\mu, \lambda_y^{-1})$. We make two independent measurements with each device, which turn out to be

$$x_1 = 1.1, x_2 = 1.9, y_1 = 2.9, y_2 = 4.1 \quad (4.245)$$

We will use a non-informative prior for μ , $p(\mu) \propto 1$, which we can emulate using an infinitely broad Gaussian, $p(\mu) = \mathcal{N}(\mu | m_0 = 0, \lambda_0^{-1} = \infty)$. If the λ_x and λ_y terms were known, then the posterior would be Gaussian:

$$p(\mu | \mathcal{D}, \lambda_x, \lambda_y) = \mathcal{N}(\mu | m_N, \lambda_N^{-1}) \quad (4.246)$$

$$\lambda_N = \lambda_0 + N_x \lambda_x + N_y \lambda_y \quad (4.247)$$

$$m_N = \frac{\lambda_x N_x \bar{x} + \lambda_y N_y \bar{y}}{N_x \lambda_x + N_y \lambda_y} \quad (4.248)$$

where $N_x = 2$ is the number of x measurements, $N_y = 2$ is the number of y measurements, $\bar{x} = \frac{1}{N_x} \sum_{i=1}^{N_x} x_i = 1.5$ and $\bar{y} = \frac{1}{N_y} \sum_{i=1}^{N_y} y_i = 3.5$. This result follows because the posterior precision is the sum of the measurement precisions, and the posterior mean is a weighted sum of the prior mean (which is 0) and the data means.

However, the measurement precisions are not known. Initially we will estimate them by maximum likelihood. The log-likelihood is given by

$$\ell(\mu, \lambda_x, \lambda_y) = \log \lambda_x - \frac{\lambda_x}{2} \sum_i (x_i - \mu)^2 + \log \lambda_y - \frac{\lambda_y}{2} \sum_i (y_i - \mu)^2 \quad (4.249)$$

The MLE is obtained by solving the following simultaneous equations:

$$\frac{\partial \ell}{\partial \mu} = \lambda_x N_x (\bar{x} - \mu) + \lambda_y N_y (\bar{y} - \mu) = 0 \quad (4.250)$$

$$\frac{\partial \ell}{\partial \lambda_x} = \frac{1}{\lambda_x} - \frac{1}{N_x} \sum_{i=1}^{N_x} (x_i - \mu)^2 = 0 \quad (4.251)$$

$$\frac{\partial \ell}{\partial \lambda_y} = \frac{1}{\lambda_y} - \frac{1}{N_y} \sum_{i=1}^{N_y} (y_i - \mu)^2 = 0 \quad (4.252)$$

This gives

$$\hat{\mu} = \frac{N_x \hat{\lambda}_x \bar{x} + N_y \hat{\lambda}_y \bar{y}}{N_x \hat{\lambda}_x + N_y \hat{\lambda}_y} \quad (4.253)$$

$$1/\hat{\lambda}_x = \frac{1}{N_x} \sum_i (x_i - \hat{\mu})^2 \quad (4.254)$$

$$1/\hat{\lambda}_y = \frac{1}{N_y} \sum_i (y_i - \hat{\mu})^2 \quad (4.255)$$

We notice that the MLE for μ has the same form as the posterior mean, m_N .

We can solve these equations by fixed point iteration. Let us initialize by estimating $\lambda_x = 1/s_x^2$ and $\lambda_y = 1/s_y^2$, where $s_x^2 = \frac{1}{N_x} \sum_{i=1}^{N_x} (x_i - \bar{x})^2 = 0.16$ and $s_y^2 = \frac{1}{N_y} \sum_{i=1}^{N_y} (y_i - \bar{y})^2 = 0.36$. Using this, we get $\hat{\mu} = 2.1154$, so $p(\mu|\mathcal{D}, \hat{\lambda}_x, \hat{\lambda}_y) = \mathcal{N}(\mu|2.1154, 0.0554)$. If we now iterate, we converge to $\hat{\lambda}_x = 1/0.1662$, $\hat{\lambda}_y = 1/4.0509$, $p(\mu|\mathcal{D}, \hat{\lambda}_x, \hat{\lambda}_y) = \mathcal{N}(\mu|1.5788, 0.0798)$.

The plug-in approximation to the posterior is plotted in Figure 4.20(a). This weights each sensor according to its estimated precision. Since sensor y was estimated to be much less reliable than sensor x , we have $\mathbb{E}[\mu|\mathcal{D}, \hat{\lambda}_x, \hat{\lambda}_y] \approx \bar{x}$, so we effectively ignore the y sensor.

Now we will adopt a Bayesian approach and integrate out the unknown precisions, rather than trying to estimate them. That is, we compute

$$p(\mu|\mathcal{D}) \propto p(\mu) \left[\int p(\mathcal{D}_x|\mu, \lambda_x) p(\lambda_x|\mu) d\lambda_x \right] \left[\int p(\mathcal{D}_y|\mu, \lambda_y) p(\lambda_y|\mu) d\lambda_y \right] \quad (4.256)$$

We will use uninformative Jeffrey's priors, $p(\mu) \propto 1$, $p(\lambda_x|\mu) \propto 1/\lambda_x$ and $p(\lambda_y|\mu) \propto 1/\lambda_y$.

Since the x and y terms are symmetric, we will just focus on one of them. The key integral is

$$I = \int p(\mathcal{D}_x | \mu, \lambda_x) p(\lambda_x | \mu) d\lambda_x \propto \int \lambda_x^{-1} (N_x \lambda_x)^{N_x/2} \quad (4.257)$$

$$\exp\left(-\frac{N_x}{2} \lambda_x (\bar{x} - \mu)^2 - \frac{N_x}{2} s_x^2 \lambda_x\right) d\lambda_x \quad (4.258)$$

Exploiting the fact that $N_x = 2$ this simplifies to

$$I = \int \lambda_x^{-1} \lambda_x^1 \exp(-\lambda_x [(\bar{x} - \mu)^2 + s_x^2]) d\lambda_x \quad (4.259)$$

We recognize this as proportional to the integral of an unnormalized Gamma density

$$\text{Ga}(\lambda | a, b) \propto \lambda^{a-1} e^{-\lambda b} \quad (4.260)$$

where $a = 1$ and $b = (\bar{x} - \mu)^2 + s_x^2$. Hence the integral is proportional to the normalizing constant of the Gamma distribution, $\Gamma(a)b^{-a}$, so we get

$$I \propto \int p(\mathcal{D}_x | \mu, \lambda_x) p(\lambda_x | \mu) d\lambda_x \propto (\bar{x} - \mu)^2 + s_x^2)^{-1} \quad (4.261)$$

and the posterior becomes

$$p(\mu | \mathcal{D}) \propto \frac{1}{(\bar{x} - \mu)^2 + s_x^2} \frac{1}{(\bar{y} - \mu)^2 + s_y^2} \quad (4.262)$$

The exact posterior is plotted in Figure 4.20(b). We see that it has two modes, one near $\bar{x} = 1.5$ and one near $\bar{y} = 3.5$. These correspond to the beliefs that the x sensor is more reliable than the y one, and vice versa. The weight of the first mode is larger, since the data from the x sensor agree more with each other, so it seems slightly more likely that the x sensor is the reliable one. (They obviously cannot both be reliable, since they disagree on the values that they are reporting.) However, the Bayesian solution keeps open the possibility that the y sensor is the more reliable one; from two measurements, we cannot tell, and choosing just the x sensor, as the plug-in approximation does, results in over confidence (a posterior that is too narrow).

Exercises

Exercise 4.1 Uncorrelated does not imply independent

Let $X \sim U(-1, 1)$ and $Y = X^2$. Clearly Y is dependent on X (in fact, Y is uniquely determined by X). However, show that $\rho(X, Y) = 0$. Hint: if $X \sim U(a, b)$ then $E[X] = (a + b)/2$ and $\text{var}[X] = (b - a)^2/12$.

Exercise 4.2 Uncorrelated and Gaussian does not imply independent unless *jointly* Gaussian

Let $X \sim \mathcal{N}(0, 1)$ and $Y = WX$, where $p(W = -1) = p(W = 1) = 0.5$. It is clear that X and Y are not independent, since Y is a function of X .

a. Show $Y \sim \mathcal{N}(0, 1)$.

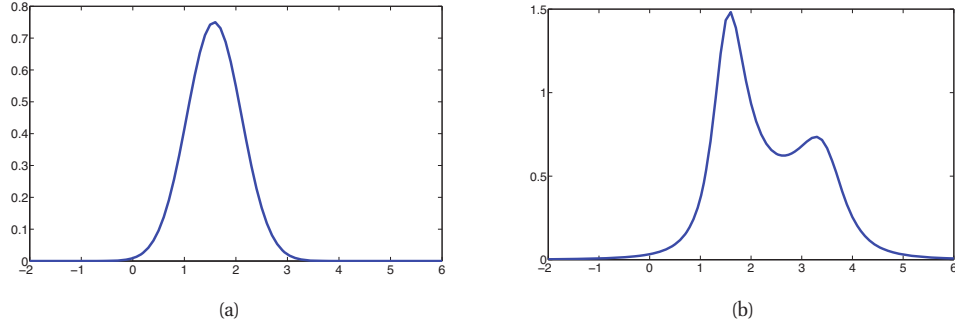


Figure 4.20 Posterior for μ . (a) Plug-in approximation. (b) Exact posterior. Figure generated by `sensorFusionUnknownPrec`.

- b. Show $\text{cov}[X, Y] = 0$. Thus X and Y are uncorrelated but dependent, even though they are Gaussian. Hint: use the definition of covariance

$$\text{cov}[X, Y] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y] \quad (4.263)$$

and the **rule of iterated expectation**

$$\mathbb{E}[XY] = \mathbb{E}[\mathbb{E}[XY|W]] \quad (4.264)$$

Exercise 4.3 Correlation coefficient is between -1 and +1

Prove that $-1 \leq \rho(X, Y) \leq 1$

Exercise 4.4 Correlation coefficient for linearly related variables is ± 1

Show that, if $Y = aX + b$ for some parameters $a > 0$ and b , then $\rho(X, Y) = 1$. Similarly show that if $a < 0$, then $\rho(X, Y) = -1$.

Exercise 4.5 Normalization constant for a multidimensional Gaussian

Prove that the normalization constant for a d -dimensional Gaussian is given by

$$(2\pi)^{d/2} |\Sigma|^{\frac{1}{2}} = \int \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^T \Sigma^{-1}(\mathbf{x} - \mu)\right) d\mathbf{x} \quad (4.265)$$

Hint: diagonalize Σ and use the fact that $|\Sigma| = \prod_i \lambda_i$ to write the joint pdf as a product of d one-dimensional Gaussians in a transformed coordinate system. (You will need the change of variables formula.) Finally, use the normalization constant for univariate Gaussians.

Exercise 4.6 Bivariate Gaussian

Let $\mathbf{x} \sim \mathcal{N}(\mu, \Sigma)$ where $\mathbf{x} \in \mathbb{R}^2$ and

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix} \quad (4.266)$$

where ρ is the correlation coefficient. Show that the pdf is given by

$$p(x_1, x_2) = \frac{1}{2\pi\sigma_1\sigma_2\sqrt{1-\rho^2}} \quad (4.267)$$

$$\exp\left(-\frac{1}{2(1-\rho^2)}\left(\frac{(x_1 - \mu_1)^2}{\sigma_1^2} + \frac{(x_2 - \mu_2)^2}{\sigma_2^2} - 2\rho\frac{(x_1 - \mu_1)}{\sigma_1}\frac{(x_2 - \mu_2)}{\sigma_2}\right)\right) \quad (4.268)$$

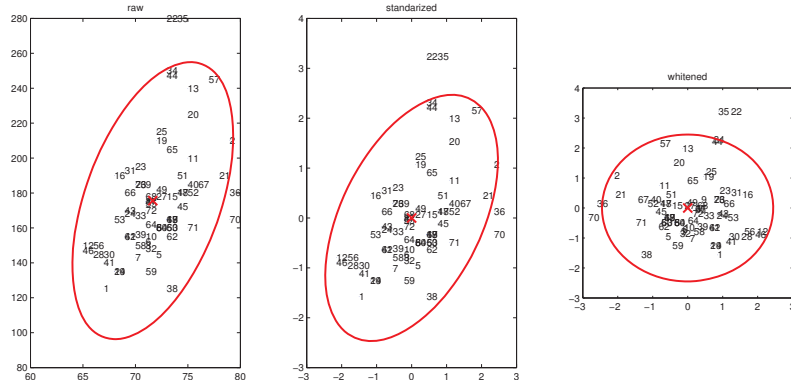


Figure 4.21 (a) Height/weight data for the men. (b) Standardized. (c) Whitened.

Exercise 4.7 Conditioning a bivariate Gaussian

Consider a bivariate Gaussian distribution $p(x_1, x_2) = \mathcal{N}(x|\mu, \Sigma)$ where

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{21} & \sigma_2^2 \end{pmatrix} = \sigma_1 \sigma_2 \begin{pmatrix} \frac{\sigma_1}{\sigma_2} & \rho \\ \rho & \frac{\sigma_2}{\sigma_1} \end{pmatrix} \quad (4.269)$$

where the correlation coefficient is given by

$$\rho \triangleq \frac{\sigma_{12}}{\sigma_1 \sigma_2} \quad (4.270)$$

- What is $P(X_2|x_1)$? Simplify your answer by expressing it in terms of ρ , σ_2 , σ_1 , μ_1, μ_2 and x_1 .
- Assume $\sigma_1 = \sigma_2 = 1$. What is $P(X_2|x_1)$ now?

Exercise 4.8 Whitening vs standardizing

- Load the height/weight data using `rawdata = dlmread('heightWeightData.txt')`. The first column is the class label (1=male, 2=female), the second column is height, the third weight. Extract the height/weight data corresponding to the males. Fit a 2d Gaussian to the male data, using the empirical mean and covariance. Plot your Gaussian as an ellipse (use `gaussPlot2d`), superimposing on your scatter plot. It should look like Figure 4.21(a), where we have labeled each datapoint by its index. Turn in your figure and code.
- Standardizing** the data means ensuring the empirical variance along each dimension is 1. This can be done by computing $\frac{x_{ij} - \bar{x}_j}{\sigma_j}$, where σ_j is the empirical std of dimension j . Standardize the data and replot. It should look like Figure 4.21(b). (Use `axis('equal')`.) Turn in your figure and code.
- Whitening** or **sphereing** the data means ensuring its empirical covariance matrix is proportional to $\mathbf{I}$, so the data is uncorrelated and of equal variance along each dimension. This can be done by computing $\Lambda^{-\frac{1}{2}} \mathbf{U}^T \mathbf{x}$ for each data vector $\mathbf{x}$, where $\mathbf{U}$ are the eigenvectors and Λ the eigenvalues of $\mathbf{X}$. Whiten the data and replot. It should look like Figure 4.21(c). Note that whitening rotates the data, so people move to counter-intuitive locations in the new coordinate system (see e.g., person 2, who moves from the right hand side to the left).

Exercise 4.9 Sensor fusion with known variances in 1d

Suppose we have two sensors with known (and different) variances v_1 and v_2 , but unknown (and the same) mean μ . Suppose we observe n_1 observations $y_i^{(1)} \sim \mathcal{N}(\mu, v_1)$ from the first sensor and n_2 observations

$y_i^{(2)} \sim \mathcal{N}(\mu, v_2)$ from the second sensor. (For example, suppose μ is the true temperature outside, and sensor 1 is a precise (low variance) digital thermosensing device, and sensor 2 is an imprecise (high variance) mercury thermometer.) Let $\mathcal{D}$ represent all the data from both sensors. What is the posterior $p(\mu|\mathcal{D})$, assuming a non-informative prior for μ (which we can simulate using a Gaussian with a precision of 0)? Give an explicit expression for the posterior mean and variance.

Exercise 4.10 Derivation of information form formulae for marginalizing and conditioning

Derive the information form results of Section 4.3.1.

Exercise 4.11 Derivation of the NIW posterior

Derive Equation 4.209. Hint: one can show that

$$N(\bar{\mathbf{x}} - \boldsymbol{\mu})(\bar{\mathbf{x}} - \boldsymbol{\mu})^T + \kappa_0(\boldsymbol{\mu} - \mathbf{m}_0)(\boldsymbol{\mu} - \mathbf{m}_0)^T \quad (4.271)$$

$$= \kappa_N(\boldsymbol{\mu} - \mathbf{m}_N)(\boldsymbol{\mu} - \mathbf{m}_N)^T + \frac{\kappa_0 N}{\kappa_N}(\bar{\mathbf{x}} - \mathbf{m}_0)(\bar{\mathbf{x}} - \mathbf{m}_0)^T \quad (4.272)$$

This is a matrix generalization of an operation called **completing the square**.⁵

Derive the corresponding result for the normal-Wishart model.

Exercise 4.12 BIC for Gaussians

(Source: Jaakkola.)

The Bayesian information criterion (BIC) is a penalized log-likelihood function that can be used for model selection (see Section 5.3.2.4). It is defined as

$$BIC = \log p(\mathcal{D}|\hat{\boldsymbol{\theta}}_{ML}) - \frac{d}{2} \log(N) \quad (4.273)$$

where d is the number of free parameters in the model and N is the number of samples. In this question, we will see how to use this to choose between a full covariance Gaussian and a Gaussian with a diagonal covariance. Obviously a full covariance Gaussian has higher likelihood, but it may not be “worth” the extra parameters if the improvement over a diagonal covariance matrix is too small. So we use the BIC score to choose the model.

Following Section 4.1.3, we can write

$$\log p(\mathcal{D}|\hat{\boldsymbol{\Sigma}}, \hat{\boldsymbol{\mu}}) = -\frac{N}{2} \text{tr}(\hat{\boldsymbol{\Sigma}}^{-1} \hat{\mathbf{S}}) - \frac{N}{2} \log(|\hat{\boldsymbol{\Sigma}}|) \quad (4.274)$$

$$\hat{\mathbf{S}} = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T \quad (4.275)$$

where $\hat{\mathbf{S}}$ is the scatter matrix (empirical covariance), the trace of a matrix is the sum of its diagonals, and we have used the trace trick.

- Derive the BIC score for a Gaussian in D dimensions with full covariance matrix. Simplify your answer as much as possible, exploiting the form of the MLE. Be sure to specify the number of free parameters d .
- Derive the BIC score for a Gaussian in D dimensions with a *diagonal* covariance matrix. Be sure to specify the number of free parameters d . Hint: for the diagonal case, the ML estimate of $\boldsymbol{\Sigma}$ is the same as $\hat{\boldsymbol{\Sigma}}_{ML}$ except the off-diagonal terms are zero:

$$\hat{\boldsymbol{\Sigma}}_{diag} = \text{diag}(\hat{\boldsymbol{\Sigma}}_{ML}(1, 1), \dots, \hat{\boldsymbol{\Sigma}}_{ML}(D, D)) \quad (4.276)$$

5. In the scalar case, completing the square means rewriting $c_2 x^2 + c_1 x + c_0$ as $-a(x - b)^2 + w$ where $a = -c_2$, $b = \frac{c_1}{2c_2}$ and $w = \frac{c_1^2}{4c_2} + c_0$.

Exercise 4.13 Gaussian posterior credible interval

(Source: DeGroot.)

Let $X \sim \mathcal{N}(\mu, \sigma^2 = 4)$ where μ is unknown but has prior $\mu \sim \mathcal{N}(\mu_0, \sigma_0^2 = 9)$. The posterior after seeing n samples is $\mu \sim \mathcal{N}(\mu_n, \sigma_n^2)$. (This is called a credible interval, and is the Bayesian analog of a confidence interval.) How big does n have to be to ensure

$$p(\ell \leq \mu_n \leq u | D) \geq 0.95 \quad (4.277)$$

where (ℓ, u) is an interval (centered on μ_n) of width 1 and D is the data. Hint: recall that 95% of the probability mass of a Gaussian is within $\pm 1.96\sigma$ of the mean.

Exercise 4.14 MAP estimation for 1D Gaussians

(Source: Jaakkola.)

Consider samples $x_1, \dots, x_n$ from a Gaussian random variable with known variance σ^2 and unknown mean μ . We further assume a prior distribution (also Gaussian) over the mean, $\mu \sim \mathcal{N}(m, s^2)$, with fixed mean m and fixed variance s^2 . Thus the only unknown is μ .

- Calculate the MAP estimate $\hat{\mu}_{MAP}$. You can state the result without proof. Alternatively, with a lot more work, you can compute derivatives of the log posterior, set to zero and solve.
- Show that as the number of samples n increase, the MAP estimate converges to the maximum likelihood estimate.
- Suppose n is small and fixed. What does the MAP estimator converge to if we increase the prior variance s^2 ?
- Suppose n is small and fixed. What does the MAP estimator converge to if we decrease the prior variance s^2 ?

Exercise 4.15 Sequential (recursive) updating of $\hat{\Sigma}$

(Source: (Duda et al. 2001, Q3.35,3.36).)

The unbiased estimates for the covariance of a d -dimensional Gaussian based on n samples is given by

$$\hat{\Sigma} = \mathbf{C}_n = \frac{1}{n-1} \sum_{i=1}^n (\mathbf{x}_i - \mathbf{m}_n)(\mathbf{x}_i - \mathbf{m}_n)^T \quad (4.278)$$

It is clear that it takes $O(nd^2)$ time to compute $\mathbf{C}_n$. If the data points arrive one at a time, it is more efficient to incrementally update these estimates than to recompute from scratch.

- Show that the covariance can be sequentially updated as follows

$$\mathbf{C}_{n+1} = \frac{n-1}{n} \mathbf{C}_n + \frac{1}{n+1} (\mathbf{x}_{n+1} - \mathbf{m}_n)(\mathbf{x}_{n+1} - \mathbf{m}_n)^T \quad (4.279)$$

- How much time does it take per sequential update? (Use big-O notation.)
- Show that we can sequentially update the precision matrix using

$$\mathbf{C}_{n+1}^{-1} = \frac{n}{n-1} \left[\mathbf{C}_n^{-1} - \frac{\mathbf{C}_n^{-1}(\mathbf{x}_{n+1} - \mathbf{m}_n)(\mathbf{x}_{n+1} - \mathbf{m}_n)^T \mathbf{C}_n^{-1}}{\frac{n^2-1}{n} + (\mathbf{x}_{n+1} - \mathbf{m}_n)^T \mathbf{C}_n^{-1}(\mathbf{x}_{n+1} - \mathbf{m}_n)} \right] \quad (4.280)$$

Hint: notice that the update to $\mathbf{C}_{n+1}$ consists of adding a rank-one matrix, namely $\mathbf{u}\mathbf{u}^T$, where $\mathbf{u} = \mathbf{x}_{n+1} - \mathbf{m}_n$. Use the matrix inversion lemma for rank-one updates (Equation 4.111), which we repeat here for convenience:

$$(\mathbf{E} + \mathbf{u}\mathbf{u}^T)^{-1} = \mathbf{E}^{-1} - \frac{\mathbf{E}^{-1}\mathbf{u}\mathbf{v}^T\mathbf{E}^{-1}}{1 + \mathbf{v}^T\mathbf{E}^{-1}\mathbf{u}} \quad (4.281)$$

d. What is the time complexity per update?

Exercise 4.16 Likelihood ratio for Gaussians

Source: Source: Alpaydin p103 ex 4. Consider a binary classifier where the K class conditional densities are MVN $p(x|y = j) = \mathcal{N}(x|\mu_j, \Sigma_j)$. By Bayes rule, we have

$$\log \frac{p(y = 1|x)}{p(y = 0|x)} = \log \frac{p(x|y = 1)}{p(x|y = 0)} + \log \frac{p(y = 1)}{p(y = 0)} \quad (4.282)$$

In other words, the log posterior ratio is the log likelihood ratio plus the log prior ratio. For each of the 4 cases in the table below, derive an expression for the log likelihood ratio $\log \frac{p(x|y=1)}{p(x|y=0)}$, simplifying as much as possible.

Form of Σ_j	Cov	Num parameters
Arbitrary	Σ_j	$Kd(d+1)/2$
Shared	$\Sigma_j = \Sigma$	$d(d+1)/2$
Shared, axis-aligned	$\Sigma_j = \Sigma$ with $\Sigma_{ij} = 0$ for $i \neq j$	d
Shared, spherical	$\Sigma_j = \sigma^2 I$	1

Exercise 4.17 LDA/QDA on height/weight data

The function `discrimAnalysisHeightWeightDemo` fits an LDA and QDA model to the height/weight data. Compute the misclassification rate of both of these models on the training set. Turn in your numbers and code.

Exercise 4.18 Naive Bayes with mixed features

Consider a 3 class naive Bayes classifier with one binary feature and one Gaussian feature:

$$y \sim \text{Mu}(y|\boldsymbol{\pi}, 1), \quad x_1|y = c \sim \text{Ber}(x_1|\theta_c), \quad x_2|y = c \sim \mathcal{N}(x_2|\mu_c, \sigma_c^2) \quad (4.283)$$

Let the parameter vectors be as follows:

$$\boldsymbol{\pi} = (0.5, 0.25, 0.25), \quad \boldsymbol{\theta} = (0.5, 0.5, 0.5), \quad \boldsymbol{\mu} = (-1, 0, 1), \quad \boldsymbol{\sigma}^2 = (1, 1, 1) \quad (4.284)$$

- Compute $p(y|x_1 = 0, x_2 = 0)$ (the result should be a vector of 3 numbers that sums to 1).
- Compute $p(y|x_1 = 0)$.
- Compute $p(y|x_2 = 0)$.
- Explain any interesting patterns you see in your results. Hint: look at the parameter vector $\boldsymbol{\theta}$.

Exercise 4.19 Decision boundary for LDA with semi tied covariances

Consider a generative classifier with class conditional densities of the form $\mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$. In LDA, we assume $\boldsymbol{\Sigma}_c = \boldsymbol{\Sigma}$, and in QDA, each $\boldsymbol{\Sigma}_c$ is arbitrary. Here we consider the 2 class case in which $\boldsymbol{\Sigma}_1 = k\boldsymbol{\Sigma}_0$, for $k > 1$. That is, the Gaussian ellipsoids have the same “shape”, but the one for class 1 is “wider”. Derive an expression for $p(y = 1|\mathbf{x}, \boldsymbol{\theta})$, simplifying as much as possible. Give a geometric interpretation of your result, if possible.

Exercise 4.20 Logistic regression vs LDA/QDA

(Source: Jaakkola.) Suppose we train the following binary classifiers via maximum likelihood.

- GaussI: A generative classifier, where the class conditional densities are Gaussian, with both covariance matrices set to $\mathbf{I}$ (identity matrix), i.e., $p(\mathbf{x}|y = c) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_c, \mathbf{I})$. We assume $p(y)$ is uniform.
- GaussX: as for GaussI, but the covariance matrices are unconstrained, i.e., $p(\mathbf{x}|y = c) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$.

- c. LinLog: A logistic regression model with linear features.
- d. QuadLog: A logistic regression model, using linear and quadratic features (i.e., polynomial basis function expansion of degree 2).

After training we compute the performance of each model M on the training set as follows:

$$L(M) = \frac{1}{n} \sum_{i=1}^n \log p(y_i | \mathbf{x}_i, \hat{\boldsymbol{\theta}}, M) \quad (4.285)$$

(Note that this is the *conditional* log-likelihood $p(y|\mathbf{x}, \hat{\boldsymbol{\theta}})$ and not the joint log-likelihood $p(y, \mathbf{x}|\hat{\boldsymbol{\theta}})$.) We now want to compare the performance of each model. We will write $L(M) \leq L(M')$ if model M *must* have lower (or equal) log likelihood (on the training set) than M' , for any training set (in other words, M is worse than M' , at least as far as training set logprob is concerned). For each of the following model pairs, state whether $L(M) \leq L(M')$, $L(M) \geq L(M')$, or whether no such statement can be made (i.e., M might sometimes be better than M' and sometimes worse); also, for each question, briefly (1-2 sentences) explain why.

- a. GaussI, LinLog.
- b. GaussX, QuadLog.
- c. LinLog, QuadLog.
- d. GaussI, QuadLog.
- e. Now suppose we measure performance in terms of the average misclassification rate on the training set:

$$R(M) = \frac{1}{n} \sum_{i=1}^n I(y_i \neq \hat{y}(\mathbf{x}_i)) \quad (4.286)$$

Is it true in general that $L(M) > L(M')$ implies that $R(M) < R(M')$? Explain why or why not.

Exercise 4.21 Gaussian decision boundaries

(Source: (Duda et al. 2001, Q3.7).) Let $p(x|y = j) = \mathcal{N}(x|\mu_j, \sigma_j)$ where $j = 1, 2$ and $\mu_1 = 0, \sigma_1^2 = 1, \mu_2 = 1, \sigma_2^2 = 10^6$. Let the class priors be equal, $p(y = 1) = p(y = 2) = 0.5$.

- a. Find the decision region

$$R_1 = \{x : p(x|\mu_1, \sigma_1) \geq p(x|\mu_2, \sigma_2)\} \quad (4.287)$$

Sketch the result. Hint: draw the curves and find where they intersect. Find *both* solutions of the equation

$$p(x|\mu_1, \sigma_1) = p(x|\mu_2, \sigma_2) \quad (4.288)$$

Hint: recall that to solve a quadratic equation $ax^2 + bx + c = 0$, we use

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad (4.289)$$

- b. Now suppose $\sigma_2 = 1$ (and all other parameters remain the same). What is R_1 in this case?

Exercise 4.22 QDA with 3 classes

Consider a three category classification problem. Let the prior probabilities:

$$P(Y = 1) = P(Y = 2) = P(Y = 3) = 1/3 \quad (4.290)$$

The class-conditional densities are multivariate normal densities with parameters:

$$\mu_1 = [0, 0]^T, \mu_2 = [1, 1]^T, \mu_3 = [-1, 1]^T \quad (4.291)$$

$$\Sigma_1 = \begin{bmatrix} 0.7 & 0 \\ 0 & 0.7 \end{bmatrix}, \Sigma_2 = \begin{bmatrix} 0.8 & 0.2 \\ 0.2 & 0.8 \end{bmatrix}, \Sigma_3 = \begin{bmatrix} 0.8 & 0.2 \\ 0.2 & 0.8 \end{bmatrix} \quad (4.292)$$

Classify the following points:

- a. $\mathbf{x} = [-0.5, 0.5]$
- b. $\mathbf{x} = [0.5, 0.5]$

Exercise 4.23 Scalar QDA

[Note: you can solve this exercise by hand or using a computer (matlab, R, whatever). In either case, show your work.] Consider the following training set of heights x (in inches) and gender y (male/female) of some US college students: $\mathbf{x} = (67, 79, 71, 68, 67, 60)$, $\mathbf{y} = (m, m, m, f, f, f)$.

- a. Fit a Bayes classifier to this data, using maximum likelihood estimation, i.e., estimate the parameters of the class conditional likelihoods

$$p(x|y = c) = \mathcal{N}(x; \mu_c, \sigma_c) \quad (4.293)$$

and the class prior

$$p(y = c) = \pi_c \quad (4.294)$$

What are your values of μ_c , σ_c , π_c for $c = m, f$? Show your work (so you can get partial credit if you make an arithmetic error).

- b. Compute $p(y = m|x, \hat{\theta})$, where $x = 72$, and $\hat{\theta}$ are the MLE parameters. (This is called a plug-in prediction.)
- c. What would be a simple way to extend this technique if you had multiple attributes per person, such as height and weight? Write down your proposed model as an equation.

5

Bayesian statistics

5.1 Introduction

We have now seen a variety of different probability models, and we have discussed how to fit them to data, i.e., we have discussed how to compute MAP parameter estimates $\hat{\theta} = \operatorname{argmax}_{\theta} p(\theta|\mathcal{D})$, using a variety of different priors. We have also discussed how to compute the full posterior $p(\theta|\mathcal{D})$, as well as the posterior predictive density, $p(\mathbf{x}|\mathcal{D})$, for certain special cases (and in later chapters, we will discuss algorithms for the general case).

Using the posterior distribution to summarize everything we know about a set of unknown variables is at the core of **Bayesian statistics**. In this chapter, we discuss this approach to statistics in more detail. In Chapter 6, we discuss an alternative approach to statistics known as frequentist or classical statistics.

5.2 Summarizing posterior distributions

The posterior $p(\theta|\mathcal{D})$ summarizes everything we know about the unknown quantities θ . In this section, we discuss some simple quantities that can be derived from a probability distribution, such as a posterior. These summary statistics are often easier to understand and visualize than the full joint.

5.2.1 MAP estimation

We can easily compute a **point estimate** of an unknown quantity by computing the posterior mean, median or mode. In Section 5.7, we discuss how to use decision theory to choose between these methods. Typically the posterior mean or median is the most appropriate choice for a real-valued quantity, and the vector of posterior marginals is the best choice for a discrete quantity. However, the posterior mode, aka the MAP estimate, is the most popular choice because it reduces to an optimization problem, for which efficient algorithms often exist. Furthermore, MAP estimation can be interpreted in non-Bayesian terms, by thinking of the log prior as a regularizer (see Section 6.5 for more details).

Although this approach is computationally appealing, it is important to point out that there are various drawbacks to MAP estimation, which we briefly discuss below. This will provide motivation for the more thoroughly Bayesian approach which we will study later in this chapter (and elsewhere in this book).

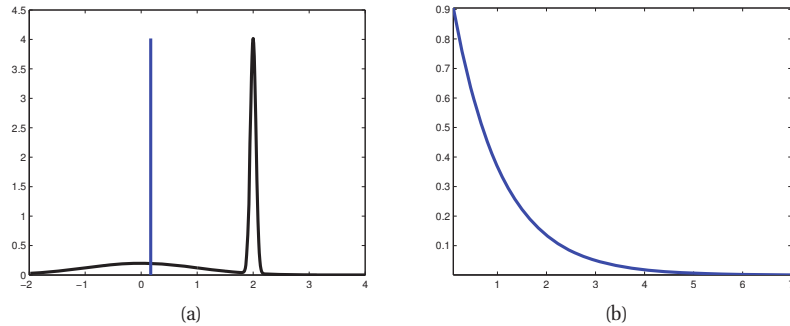


Figure 5.1 (a) A bimodal distribution in which the mode is very untypical of the distribution. The thin blue vertical line is the mean, which is arguably a better summary of the distribution, since it is near the majority of the probability mass. Figure generated by `bimodalDemo`. (b) A skewed distribution in which the mode is quite different from the mean. Figure generated by `gammaPlotDemo`.

5.2.1.1 No measure of uncertainty

The most obvious drawback of MAP estimation, and indeed of any other **point estimate** such as the posterior mean or median, is that it does not provide any measure of uncertainty. In many applications, it is important to know how much one can trust a given estimate. We can derive such confidence measures from the posterior, as we discuss in Section 5.2.2.

5.2.1.2 Plugging in the MAP estimate can result in overfitting

In machine learning, we often care more about predictive accuracy than in interpreting the parameters of our models. However, if we don't model the uncertainty in our parameters, then our predictive distribution will be overconfident. We saw several examples of this in Chapter 3, and we will see more examples later. Overconfidence in predictions is particularly problematic in situations where we may be risk averse; see Section 5.7 for details.

5.2.1.3 The mode is an untypical point

Choosing the mode as a summary of a posterior distribution is often a very poor choice, since the mode is usually quite untypical of the distribution, unlike the mean or median. This is illustrated in Figure 5.1(a) for a 1d continuous space. The basic problem is that the mode is a point of measure zero, whereas the mean and median take the volume of the space into account. Another example is shown in Figure 5.1(b): here the mode is 0, but the mean is non-zero. Such skewed distributions often arise when inferring variance parameters, especially in hierarchical models. In such cases the MAP estimate (and hence the MLE) is obviously a very bad estimate.

How should we summarize a posterior if the mode is not a good choice? The answer is to use decision theory, which we discuss in Section 5.7. The basic idea is to specify a loss function, where $L(\theta, \hat{\theta})$ is the loss you incur if the truth is θ and your estimate is $\hat{\theta}$. If we use 0-1 loss, $L(\theta, \hat{\theta}) = \mathbb{I}(\theta \neq \hat{\theta})$, then the optimal estimate is the posterior mode. 0-1 loss means you only get “points” if you make no errors, otherwise you get nothing; there is no “partial credit” under

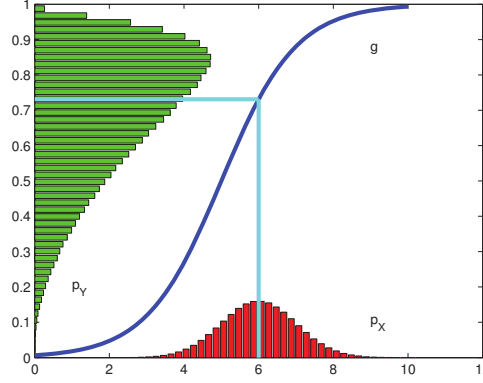


Figure 5.2 Example of the transformation of a density under a nonlinear transform. Note how the mode of the transformed distribution is not the transform of the original mode. Based on Exercise 1.4 of (Bishop 2006b). Figure generated by `bayesChangeOfVar`.

this loss function! For continuous-valued quantities, we often prefer to use squared error loss, $L(\theta, \hat{\theta}) = (\theta - \hat{\theta})^2$; the corresponding optimal estimator is then the posterior mean, as we show in Section 5.7. Or we can use a more robust loss function, $L(\theta, \hat{\theta}) = |\theta - \hat{\theta}|$, which gives rise to the posterior median.

5.2.1.4 MAP estimation is not invariant to reparameterization *

A more subtle problem with MAP estimation is that the result we get depends on how we parameterize the probability distribution. Changing from one representation to another equivalent representation changes the result, which is not very desirable, since the units of measurement are arbitrary (e.g., when measuring distance, we can use centimetres or inches).

To understand the problem, suppose we compute the posterior for x . If we define $y = f(x)$, the distribution for y is given by Equation 2.87, which we repeat here for convenience:

$$p_y(y) = p_x(x) \left| \frac{dx}{dy} \right| \quad (5.1)$$

The $\left| \frac{dx}{dy} \right|$ term is called the Jacobian, and it measures the change in size of a unit volume passed through f . Let $\hat{x} = \operatorname{argmax}_x p_x(x)$ be the MAP estimate for x . In general it is not the case that $\hat{y} = \operatorname{argmax}_y p_y(y)$ is given by $f(\hat{x})$. For example, let $x \sim \mathcal{N}(6, 1)$ and $y = f(x)$, where

$$f(x) = \frac{1}{1 + \exp(-x + 5)} \quad (5.2)$$

We can derive the distribution of y using Monte Carlo simulation (see Section 2.7.1). The result is shown in Figure 5.2. We see that the original Gaussian has become “squashed” by the sigmoid nonlinearity. In particular, we see that the mode of the transformed distribution is not equal to the transform of the original mode.

To see how this problem arises in the context of MAP estimation, consider the following example, due to Michael Jordan. The Bernoulli distribution is typically parameterized by its mean μ , so $p(y = 1|\mu) = \mu$, where $y \in \{0, 1\}$. Suppose we have a uniform prior on the unit interval: $p_\mu(\mu) = 1 \mathbb{I}(0 \leq \mu \leq 1)$. If there is no data, the MAP estimate is just the mode of the prior, which can be anywhere between 0 and 1. We will now show that different parameterizations can pick different points in this interval arbitrarily.

First let $\theta = \sqrt{\mu}$ so $\mu = \theta^2$. The new prior is

$$p_\theta(\theta) = p_\mu(\mu) \left| \frac{d\mu}{d\theta} \right| = 2\theta \quad (5.3)$$

for $\theta \in [0, 1]$ so the new mode is

$$\hat{\theta}_{MAP} = \arg \max_{\theta \in [0, 1]} 2\theta = 1 \quad (5.4)$$

Now let $\phi = 1 - \sqrt{1 - \mu}$. The new prior is

$$p_\phi(\phi) = p_\mu(\mu) \left| \frac{d\mu}{d\phi} \right| = 2(1 - \phi) \quad (5.5)$$

for $\phi \in [0, 1]$, so the new mode is

$$\hat{\phi}_{MAP} = \arg \max_{\phi \in [0, 1]} 2 - 2\phi = 0 \quad (5.6)$$

Thus the MAP estimate depends on the parameterization. The MLE does not suffer from this since the likelihood is a function, not a probability density. Bayesian inference does not suffer from this problem either, since the change of measure is taken into account when integrating over the parameter space.

One solution to the problem is to optimize the following objective function:

$$\hat{\theta} = \operatorname{argmax}_{\theta} p(\mathcal{D}|\theta)p(\theta)|\mathbf{I}(\theta)|^{-\frac{1}{2}} \quad (5.7)$$

Here $\mathbf{I}(\theta)$ is the Fisher information matrix associated with $p(\mathbf{x}|\theta)$ (see Section 6.2.2). This estimate is parameterization independent, for reasons explained in (Jermyn 2005; Druilhet and Marin 2007). Unfortunately, optimizing Equation 5.7 is often difficult, which minimizes the appeal of the whole approach.

5.2.2 Credible intervals

In addition to point estimates, we often want a measure of confidence. A standard measure of confidence in some (scalar) quantity θ is the “width” of its posterior distribution. This can be measured using a $100(1 - \alpha)\%$ **credible interval**, which is a (contiguous) region $C = (\ell, u)$ (standing for lower and upper) which contains $1 - \alpha$ of the posterior probability mass, i.e.,

$$C_\alpha(\mathcal{D}) = (\ell, u) : P(\ell \leq \theta \leq u|\mathcal{D}) = 1 - \alpha \quad (5.8)$$

There may be many such intervals, so we choose one such that there is $(1 - \alpha)/2$ mass in each tail; this is called a **central interval**.

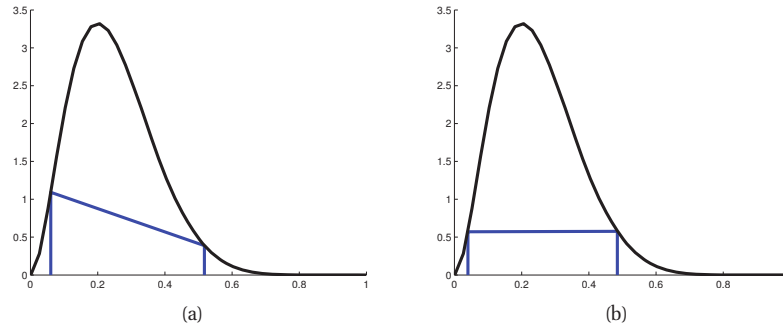


Figure 5.3 (a) Central interval and (b) HPD region for a Beta(3,9) posterior. The CI is (0.06, 0.52) and the HPD is (0.04, 0.48). Based on Figure 3.6 of (Hoff 2009). Figure generated by `betaHPD`.

If the posterior has a known functional form, we can compute the posterior central interval using $\ell = F^{-1}(\alpha/2)$ and $u = F^{-1}(1-\alpha/2)$, where F is the cdf of the posterior. For example, if the posterior is Gaussian, $p(\theta|\mathcal{D}) = \mathcal{N}(0, 1)$, and $\alpha = 0.05$, then we have $\ell = \Phi(\alpha/2) = -1.96$, and $u = \Phi(1 - \alpha/2) = 1.96$, where Φ denotes the cdf of the Gaussian. This is illustrated in Figure 2.3(c). This justifies the common practice of quoting a credible interval in the form of $\mu \pm 2\sigma$, where μ represents the posterior mean, σ represents the posterior standard deviation, and 2 is a good approximation to 1.96.

Of course, the posterior is not always Gaussian. For example, in our coin example, if we use a uniform prior and we observe $N_1 = 47$ heads out of $N = 100$ trials, then the posterior is a beta distribution, $p(\theta|\mathcal{D}) = \text{Beta}(48, 54)$. We find the 95% posterior credible interval is (0.3749, 0.5673) (see `betaCredibleInt` for the one line of Matlab code we used to compute this).

If we don't know the functional form, but we can draw samples from the posterior, then we can use a Monte Carlo approximation to the posterior quantiles: we simply sort the S samples, and find the one that occurs at location α/S along the sorted list. As $S \rightarrow \infty$, this converges to the true quantile. See `mcQuantileDemo` for a demo.

People often confuse Bayesian credible intervals with frequentist confidence intervals. However, they are not the same thing, as we discuss in Section 6.6.1. In general, credible intervals are usually what people want to compute, but confidence intervals are usually what they actually compute, because most people are taught frequentist statistics but not Bayesian statistics. Fortunately, the mechanics of computing a credible interval is just as easy as computing a confidence interval (see e.g., `betaCredibleInt` for how to do it in Matlab).

5.2.2.1 Highest posterior density regions *

A problem with central intervals is that there might be points outside the CI which have higher probability density. This is illustrated in Figure 5.3(a), where we see that points outside the left-most CI boundary have higher density than those just inside the right-most CI boundary.

This motivates an alternative quantity known as the **highest posterior density** or **HPD** region. This is defined as the (set of) most probable points that in total constitute $100(1 - \alpha)\%$ of the

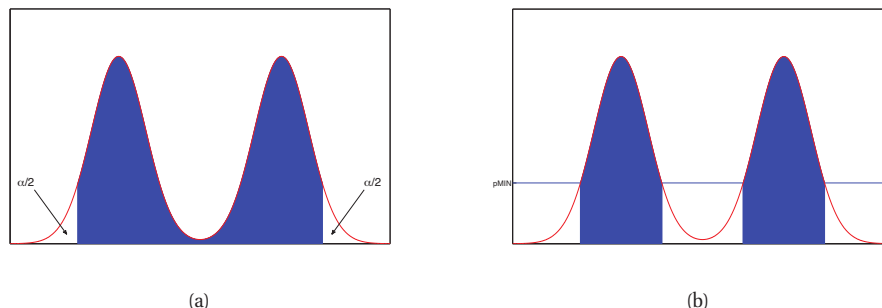


Figure 5.4 (a) Central interval and (b) HPD region for a hypothetical multimodal posterior. Based on Figure 2.2 of (Gelman et al. 2004). Figure generated by `postDensityIntervals`.

probability mass. More formally, we find the threshold p^* on the pdf such that

$$1 - \alpha = \int_{\theta: p(\theta|\mathcal{D}) > p^*} p(\theta|\mathcal{D}) d\theta \quad (5.9)$$

and then define the HPD as

$$C_\alpha(\mathcal{D}) = \{\theta : p(\theta|\mathcal{D}) \geq p^*\} \quad (5.10)$$

In 1d, the HPD region is sometimes called a **highest density interval** or **HDI**. For example, Figure 5.3(b) shows the 95% HDI of a $\text{Beta}(3, 9)$ distribution, which is $(0.04, 0.48)$. We see that this is narrower than the CI, even though it still contains 95% of the mass; furthermore, every point inside of it has higher density than every point outside of it.

For a unimodal distribution, the HDI will be the narrowest interval around the mode containing 95% of the mass. To see this, imagine “water filling” in reverse, where we lower the level until 95% of the mass is revealed, and only 5% is submerged. This gives a simple algorithm for computing HDIs in the 1d case: simply search over points such that the interval contains 95% of the mass and has minimal width. This can be done by 1d numerical optimization if we know the inverse CDF of the distribution, or by search over the sorted data points if we have a bag of samples (see `betaHPD` for a demo).

If the posterior is multimodal, the HDI may not even be a connected region: see Figure 5.4(b) for an example. However, summarizing multimodal posteriors is always difficult.

5.2.3 Inference for a difference in proportions

Sometimes we have multiple parameters, and we are interested in computing the posterior distribution of some function of these parameters. For example, suppose you are about to buy something from Amazon.com, and there are two sellers offering it for the same price. Seller 1 has 90 positive reviews and 10 negative reviews. Seller 2 has 2 positive reviews and 0 negative reviews. Who should you buy from?¹

1. This example is from www.johndcook.com/blog/2011/09/27/bayesian-amazon. See also lingpipe-blog.com/2009/10/13/bayesian-counterpart-to-fisher-exact-test-on-contingency-tables.

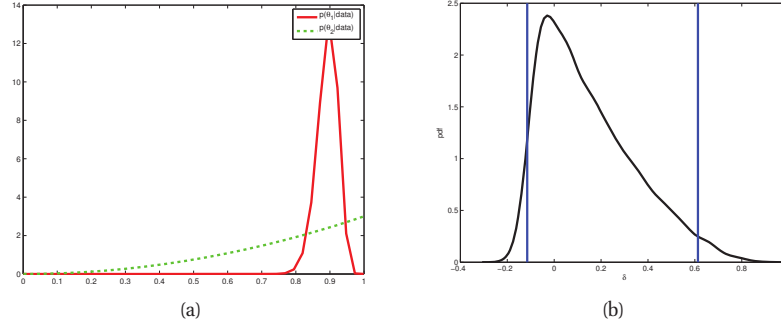


Figure 5.5 (a) Exact posteriors $p(\theta_i | \mathcal{D}_i)$. (b) Monte Carlo approximation to $p(\delta | \mathcal{D})$. We use kernel density estimation to get a smooth plot. The vertical lines enclose the 95% central interval. Figure generated by `amazonSellerDemo`,

On the face of it, you should pick seller 2, but we cannot be very confident that seller 2 is better since it has had so few reviews. In this section, we sketch a Bayesian analysis of this problem. Similar methodology can be used to compare rates or proportions across groups for a variety of other settings.

Let θ_1 and θ_2 be the unknown reliabilities of the two sellers. Since we don't know much about them, we'll endow them both with uniform priors, $\theta_i \sim \text{Beta}(1, 1)$. The posteriors are $p(\theta_1 | \mathcal{D}_1) = \text{Beta}(91, 11)$ and $p(\theta_2 | \mathcal{D}_2) = \text{Beta}(3, 1)$.

We want to compute $p(\theta_1 > \theta_2 | \mathcal{D})$. For convenience, let us define $\delta = \theta_1 - \theta_2$ as the difference in the rates. (Alternatively we might want to work in terms of the log-odds ratio.) We can compute the desired quantity using numerical integration:

$$p(\delta > 0 | \mathcal{D}) = \int_0^1 \int_0^1 \mathbb{I}(\theta_1 > \theta_2) \text{Beta}(\theta_1 | y_1 + 1, N_1 - y_1 + 1) \text{Beta}(\theta_2 | y_2 + 1, N_2 - y_2 + 1) d\theta_1 d\theta_2 \quad (5.11)$$

We find $p(\delta > 0 | \mathcal{D}) = 0.710$, which means you are better off buying from seller 1! See `amazonSellerDemo` for the code. (It is also possible to solve the integral analytically (Cook 2005).)

A simpler way to solve the problem is to approximate the posterior $p(\delta | \mathcal{D})$ by Monte Carlo sampling. This is easy, since θ_1 and θ_2 are independent in the posterior, and both have beta distributions, which can be sampled from using standard methods. The distributions $p(\theta_i | \mathcal{D}_i)$ are shown in Figure 5.5(a), and a MC approximation to $p(\delta | \mathcal{D})$, together with a 95% HPD, is shown Figure 5.5(b). An MC approximation to $p(\delta > 0 | \mathcal{D})$ is obtained by counting the fraction of samples where $\theta_1 > \theta_2$; this turns out to be 0.718, which is very close to the exact value. (See `amazonSellerDemo` for the code.)

5.3 Bayesian model selection

In Figure 1.18, we saw that using too high a degree polynomial results in overfitting, and using too low a degree results in underfitting. Similarly, in Figure 7.8(a), we saw that using too small

a regularization parameter results in overfitting, and too large a value results in underfitting. In general, when faced with a set of models (i.e., families of parametric distributions) of different complexity, how should we choose the best one? This is called the **model selection** problem.

One approach is to use cross-validation to estimate the generalization error of all the candidate models, and then to pick the model that seems the best. However, this requires fitting each model K times, where K is the number of CV folds. A more efficient approach is to compute the posterior over models,

$$p(m|\mathcal{D}) = \frac{p(\mathcal{D}|m)p(m)}{\sum_{m \in \mathcal{M}} p(m, \mathcal{D})} \quad (5.12)$$

From this, we can easily compute the MAP model, $\hat{m} = \operatorname{argmax}_m p(m|\mathcal{D})$. This is called **Bayesian model selection**.

If we use a uniform prior over models, $p(m) \propto 1$, this amounts to picking the model which maximizes

$$p(\mathcal{D}|m) = \int p(\mathcal{D}|\theta)p(\theta|m)d\theta \quad (5.13)$$

This quantity is called the **marginal likelihood**, the **integrated likelihood**, or the **evidence** for model m . The details on how to perform this integral will be discussed in Section 5.3.2. But first we give an intuitive interpretation of what this quantity means.

5.3.1 Bayesian Occam's razor

One might think that using $p(\mathcal{D}|m)$ to select models would always favor the model with the most parameters. This is true if we use $p(\mathcal{D}|\hat{\theta}_m)$ to select models, where $\hat{\theta}_m$ is the MLE or MAP estimate of the parameters for model m , because models with more parameters will fit the data better, and hence achieve higher likelihood. However, if we integrate out the parameters, rather than maximizing them, we are automatically protected from overfitting: models with more parameters do not necessarily have higher *marginal* likelihood. This is called the **Bayesian Occam's razor** effect (MacKay 1995b; Murray and Ghahramani 2005), named after the principle known as **Occam's razor**, which says one should pick the simplest model that adequately explains the data.

One way to understand the Bayesian Occam's razor is to notice that the marginal likelihood can be rewritten as follows, based on the chain rule of probability (Equation 2.5):

$$p(\mathcal{D}) = p(y_1)p(y_2|y_1)p(y_3|y_{1:2}) \dots p(y_N|y_{1:N-1}) \quad (5.14)$$

where we have dropped the conditioning on $\mathbf{x}$ for brevity. This is similar to a leave-one-out cross-validation estimate (Section 1.4.8) of the likelihood, since we predict each future point given all the previous ones. (Of course, the order of the data does not matter in the above expression.) If a model is too complex, it will overfit the “early” examples and will then predict the remaining ones poorly.

Another way to understand the Bayesian Occam's razor effect is to note that probabilities must sum to one. Hence $\sum_{\mathcal{D}'} p(\mathcal{D}'|m) = 1$, where the sum is over all possible data sets. Complex models, which can predict many things, must spread their probability mass thinly, and hence will not obtain as large a probability for any given data set as simpler models. This is sometimes

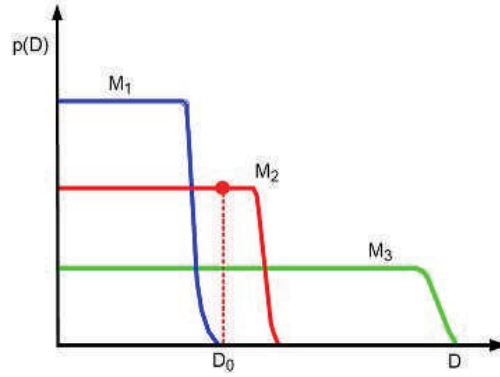


Figure 5.6 A schematic illustration of the Bayesian Occam's razor. The broad (green) curve corresponds to a complex model, the narrow (blue) curve to a simple model, and the middle (red) curve is just right. Based on Figure 3.13 of (Bishop 2006a). See also (Murray and Ghahramani 2005, Figure 2) for a similar plot produced on real data.

called the **conservation of probability mass** principle, and is illustrated in Figure 5.6. On the horizontal axis we plot all possible data sets in order of increasing complexity (measured in some abstract sense). On the vertical axis we plot the predictions of 3 possible models: a simple one, M_1 ; a medium one, M_2 ; and a complex one, M_3 . We also indicate the actually observed data $\mathcal{D}_0$ by a vertical line. Model 1 is too simple and assigns low probability to $\mathcal{D}_0$. Model 3 also assigns $\mathcal{D}_0$ relatively low probability, because it can predict many data sets, and hence it spreads its probability quite widely and thinly. Model 2 is “just right”: it predicts the observed data with a reasonable degree of confidence, but does not predict too many other things. Hence model 2 is the most probable model.

As a concrete example of the Bayesian Occam's razor, consider the data in Figure 5.7. We plot polynomials of degrees 1, 2 and 3 fit to $N = 5$ data points. It also shows the posterior over models, where we use a Gaussian prior (see Section 7.6 for details). There is not enough data to justify a complex model, so the MAP model is $d = 1$. Figure 5.8 shows what happens when $N = 30$. Now it is clear that $d = 2$ is the right model (the data was in fact generated from a quadratic).

As another example, Figure 7.8(c) plots $\log p(\mathcal{D}|\lambda)$ vs $\log(\lambda)$, for the polynomial ridge regression model, where λ ranges over the same set of values used in the CV experiment. We see that the maximum evidence occurs at roughly the same point as the minimum of the test MSE, which also corresponds to the point chosen by CV.

When using the Bayesian approach, we are not restricted to evaluating the evidence at a finite grid of values. Instead, we can use numerical optimization to find $\lambda^* = \operatorname{argmax}_{\lambda} p(\mathcal{D}|\lambda)$. This technique is called **empirical Bayes** or **type II maximum likelihood** (see Section 5.6 for details). An example is shown in Figure 7.8(b): we see that the curve has a similar shape to the CV estimate, but it can be computed more efficiently.

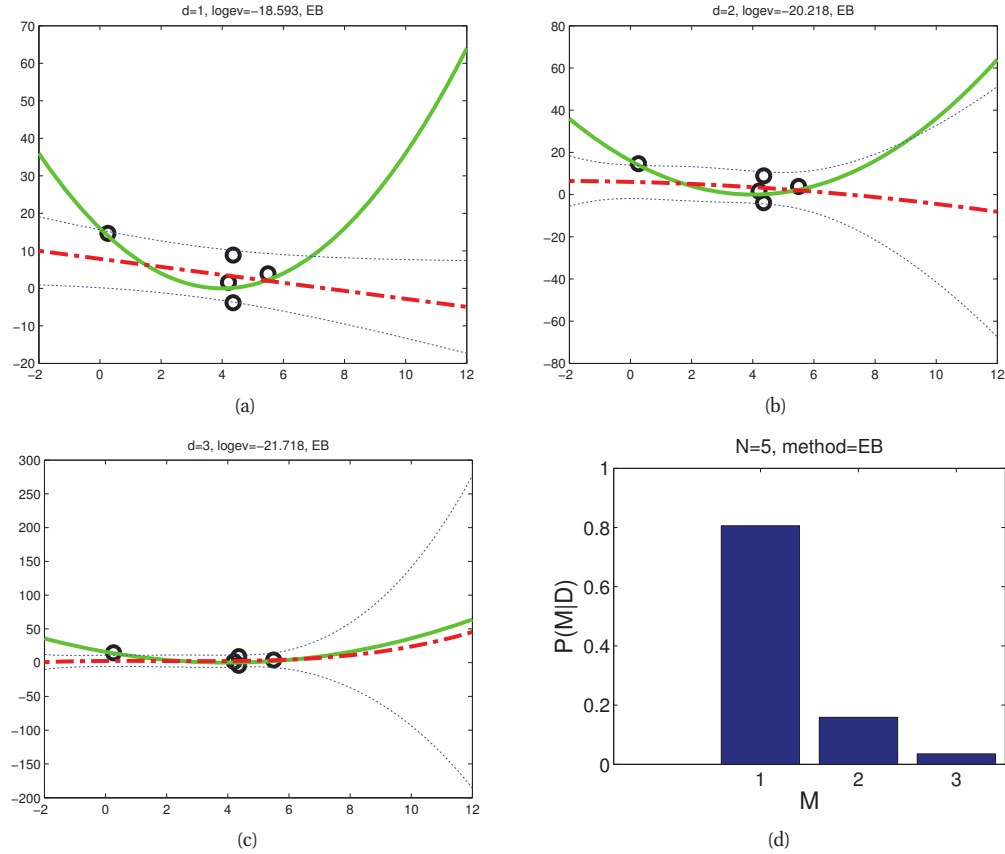


Figure 5.7 (a-c) We plot polynomials of degrees 1, 2 and 3 fit to $N = 5$ data points using empirical Bayes. The solid green curve is the true function, the dashed red curve is the prediction (dotted blue lines represent $\pm\sigma$ around the mean). (d) We plot the posterior over models, $p(d|\mathcal{D})$, assuming a uniform prior $p(d) \propto 1$. Based on a figure by Zoubin Ghahramani. Figure generated by `linregEbModelSelVsN`.

5.3.2 Computing the marginal likelihood (evidence)

When discussing parameter inference for a fixed model, we often wrote

$$p(\theta|\mathcal{D}, m) \propto p(\theta|m)p(\mathcal{D}|\theta, m) \quad (5.15)$$

thus ignoring the normalization constant $p(\mathcal{D}|m)$. This is valid since $p(\mathcal{D}|m)$ is constant wrt θ . However, when comparing models, we need to know how to compute the marginal likelihood, $p(\mathcal{D}|m)$. In general, this can be quite hard, since we have to integrate over all possible parameter values, but when we have a conjugate prior, it is easy to compute, as we now show.

Let $p(\theta) = q(\theta)/Z_0$ be our prior, where $q(\theta)$ is an unnormalized distribution, and Z_0 is the normalization constant of the prior. Let $p(\mathcal{D}|\theta) = q(\mathcal{D}|\theta)/Z_\ell$ be the likelihood, where Z_ℓ contains any constant factors in the likelihood. Finally let $p(\theta|\mathcal{D}) = q(\theta|\mathcal{D})/Z_N$ be our poste-

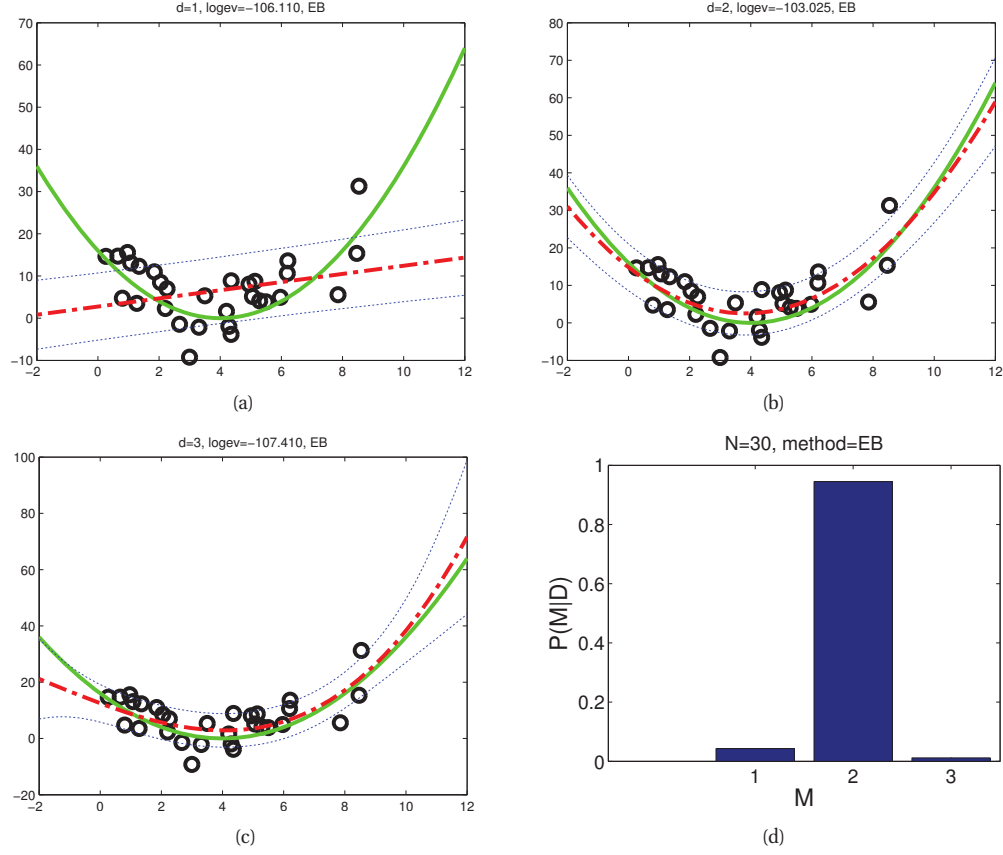


Figure 5.8 Same as Figure 5.7 except now $N = 30$. Figure generated by `linregEbModelSelVsN`.

rior, where $q(\boldsymbol{\theta}|\mathcal{D}) = q(\mathcal{D}|\boldsymbol{\theta})q(\boldsymbol{\theta})$ is the unnormalized posterior, and Z_N is the normalization constant of the posterior. We have

$$p(\boldsymbol{\theta}|\mathcal{D}) = \frac{p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathcal{D})} \quad (5.16)$$

$$\frac{q(\boldsymbol{\theta}|\mathcal{D})}{Z_N} = \frac{q(\mathcal{D}|\boldsymbol{\theta})q(\boldsymbol{\theta})}{Z_\ell Z_0 p(\mathcal{D})} \quad (5.17)$$

$$p(\mathcal{D}) = \frac{Z_N}{Z_0 Z_\ell} \quad (5.18)$$

So assuming the relevant normalization constants are tractable, we have an easy way to compute the marginal likelihood. We give some examples below.

5.3.2.1 Beta-binomial model

Let us apply the above result to the Beta-binomial model. Since we know $p(\theta|\mathcal{D}) = \text{Beta}(\theta|a', b')$, where $a' = a + N_1$ and $b' = b + N_0$, we know the normalization constant of the posterior is $B(a', b')$. Hence

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)p(\theta)}{p(\mathcal{D})} \quad (5.19)$$

$$= \frac{1}{p(\mathcal{D})} \left[\frac{1}{B(a, b)} \theta^{a-1} (1-\theta)^{b-1} \right] \left[\binom{N}{N_1} \theta^{N_1} (1-\theta)^{N_0} \right] \quad (5.20)$$

$$= \binom{N}{N_1} \frac{1}{p(\mathcal{D})} \frac{1}{B(a, b)} [\theta^{a+N_1-1} (1-\theta)^{b+N_0-1}] \quad (5.21)$$

So

$$\frac{1}{B(a + N_1, b + N_0)} = \binom{N}{N_1} \frac{1}{p(\mathcal{D})} \frac{1}{B(a, b)} \quad (5.22)$$

$$p(\mathcal{D}) = \binom{N}{N_1} \frac{B(a + N_1, b + N_0)}{B(a, b)} \quad (5.23)$$

The marginal likelihood for the Beta-Bernoulli model is the same as above, except it is missing the $\binom{N}{N_1}$ term.

5.3.2.2 Dirichlet-multinoulli model

By the same reasoning as the Beta-Bernoulli case, one can show that the marginal likelihood for the Dirichlet-multinoulli model is given by

$$p(\mathcal{D}) = \frac{B(\mathbf{N} + \boldsymbol{\alpha})}{B(\boldsymbol{\alpha})} \quad (5.24)$$

where

$$B(\boldsymbol{\alpha}) = \frac{\prod_{k=1}^K \Gamma(\alpha_k)}{\Gamma(\sum_k \alpha_k)} \quad (5.25)$$

Hence we can rewrite the above result in the following form, which is what is usually presented in the literature:

$$p(\mathcal{D}) = \frac{\Gamma(\sum_k \alpha_k)}{\Gamma(N + \sum_k \alpha_k)} \prod_k \frac{\Gamma(N_k + \alpha_k)}{\Gamma(\alpha_k)} \quad (5.26)$$

We will see many applications of this equation later.

5.3.2.3 Gaussian-Gaussian-Wishart model

Consider the case of an MVN with a conjugate NIW prior. Let Z_0 be the normalizer for the prior, Z_N be normalizer for the posterior, and let $Z_l = (2\pi)^{ND/2}$ be the normalizer for the

likelihood. Then it is easy to see that

$$p(\mathcal{D}) = \frac{Z_N}{Z_0 Z_l} \quad (5.27)$$

$$= \frac{1}{\pi^{ND/2}} \frac{1}{2^{ND/2}} \frac{\left(\frac{2\pi}{\kappa_N}\right)^{D/2} |\mathbf{S}_N|^{-\nu_N/2} 2^{(\nu_0+N)D/2} \Gamma_D(\nu_N/2)}{\left(\frac{2\pi}{\kappa_0}\right)^{D/2} |\mathbf{S}_0|^{-\nu_0/2} 2^{\nu_0 D/2} \Gamma_D(\nu_0/2)} \quad (5.28)$$

$$= \frac{1}{\pi^{ND/2}} \left(\frac{\kappa_0}{\kappa_N}\right)^{D/2} \frac{|\mathbf{S}_0|^{\nu_0/2} \Gamma_D(\nu_N/2)}{|\mathbf{S}_N|^{\nu_N/2} \Gamma_D(\nu_0/2)} \quad (5.29)$$

This equation will prove useful later.

5.3.2.4 BIC approximation to log marginal likelihood

In general, computing the integral in Equation 5.13 can be quite difficult. One simple but popular approximation is known as the **Bayesian information criterion** or **BIC**, which has the following form (Schwarz 1978):

$$\text{BIC} \triangleq \log p(\mathcal{D}|\hat{\boldsymbol{\theta}}) - \frac{\text{dof}(\hat{\boldsymbol{\theta}})}{2} \log N \approx \log p(\mathcal{D}) \quad (5.30)$$

where $\text{dof}(\hat{\boldsymbol{\theta}})$ is the number of **degrees of freedom** in the model, and $\hat{\boldsymbol{\theta}}$ is the MLE for the model.² We see that this has the form of a **penalized log likelihood**, where the penalty term depends on the model's complexity. See Section 8.4.2 for the derivation of the BIC score.

As an example, consider linear regression. As we show in Section 7.3, the MLE is given by $\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ and $\hat{\sigma}^2 = \text{RSS}/N$, where $\text{RSS} = \sum_{i=1}^N (y_i - \hat{\mathbf{w}}_{mle}^T \mathbf{x}_i)^2$. The corresponding log likelihood is given by

$$\log p(\mathcal{D}|\hat{\boldsymbol{\theta}}) = -\frac{N}{2} \log(2\pi\hat{\sigma}^2) - \frac{N}{2} \quad (5.31)$$

Hence the BIC score is as follows (dropping constant terms)

$$\text{BIC} = -\frac{N}{2} \log(\hat{\sigma}^2) - \frac{D}{2} \log(N) \quad (5.32)$$

where D is the number of variables in the model. In the statistics literature, it is common to use an alternative definition of BIC, which we call the BIC *cost* (since we want to minimize it):

$$\text{BIC-cost} \triangleq -2 \log p(\mathcal{D}|\hat{\boldsymbol{\theta}}) + \text{dof}(\hat{\boldsymbol{\theta}}) \log N \approx -2 \log p(\mathcal{D}) \quad (5.33)$$

In the context of linear regression, this becomes

$$\text{BIC-cost} = N \log(\hat{\sigma}^2) + D \log(N) \quad (5.34)$$

2. Traditionally the BIC score is defined using the ML estimate $\hat{\boldsymbol{\theta}}$, so it is independent of the prior. However, for models such as mixtures of Gaussians, the ML estimate can be poorly behaved, so it is better to evaluate the BIC score using the MAP estimate, as in (Fraley and Raftery 2007).

The BIC method is very closely related to the **minimum description length** or **MDL** principle, which characterizes the score for a model in terms of how well it fits the data, minus how complex the model is to define. See (Hansen and Yu 2001) for details.

There is a very similar expression to BIC/ MDL called the **Akaike information criterion** or **AIC**, defined as

$$\text{AIC}(m, \mathcal{D}) \triangleq \log p(\mathcal{D} | \hat{\theta}_{MLE}) - \text{dof}(m) \quad (5.35)$$

This is derived from a frequentist framework, and cannot be interpreted as an approximation to the marginal likelihood. Nevertheless, the form of this expression is very similar to BIC. We see that the penalty for AIC is less than for BIC. This causes AIC to pick more complex models. However, this can result in better predictive accuracy. See e.g., (Clarke et al. 2009, sec 10.2) for further discussion on such information criteria.

5.3.2.5 Effect of the prior

Sometimes it is not clear how to set the prior. When we are performing posterior inference, the details of the prior may not matter too much, since the likelihood often overwhelms the prior anyway. But when computing the marginal likelihood, the prior plays a much more important role, since we are averaging the likelihood over all possible parameter settings, as weighted by the prior.

In Figures 5.7 and 5.8, where we demonstrated model selection for linear regression, we used a prior of the form $p(\mathbf{w}) = \mathcal{N}(\mathbf{0}, \alpha^{-1}\mathbf{I})$. Here α is a tuning parameter that controls how strong the prior is. This parameter can have a large effect, as we discuss in Section 7.5. Intuitively, if α is large, the weights are “forced” to be small, so we need to use a complex model with many small parameters (e.g., a high degree polynomial) to fit the data. Conversely, if α is small, we will favor simpler models, since each parameter is “allowed” to vary in magnitude by a lot.

If the prior is unknown, the correct Bayesian procedure is to put a prior on the prior. That is, we should put a prior on the hyper-parameter α as well as the parameters $\mathbf{w}$. To compute the marginal likelihood, we should integrate out all unknowns, i.e., we should compute

$$p(\mathcal{D} | m) = \int \int p(\mathcal{D} | \mathbf{w}) p(\mathbf{w} | \alpha, m) p(\alpha | m) d\mathbf{w} d\alpha \quad (5.36)$$

Of course, this requires specifying the hyper-prior. Fortunately, the higher up we go in the Bayesian hierarchy, the less sensitive are the results to the prior settings. So we can usually make the hyper-prior uninformative.

A computational shortcut is to optimize α rather than integrating it out. That is, we use

$$p(\mathcal{D} | m) \approx \int p(\mathcal{D} | \mathbf{w}) p(\mathbf{w} | \hat{\alpha}, m) d\mathbf{w} \quad (5.37)$$

where

$$\hat{\alpha} = \underset{\alpha}{\text{argmax}} p(\mathcal{D} | \alpha, m) = \underset{\alpha}{\text{argmax}} \int p(\mathcal{D} | \mathbf{w}) p(\mathbf{w} | \alpha, m) d\mathbf{w} \quad (5.38)$$

This approach is called empirical Bayes (EB), and is discussed in more detail in Section 5.6. This is the method used in Figures 5.7 and 5.8.

Bayes factor $BF(1, 0)$	Interpretation
$BF < \frac{1}{100}$	Decisive evidence for M_0
$BF < \frac{1}{10}$	Strong evidence for M_0
$\frac{1}{10} < BF < \frac{1}{3}$	Moderate evidence for M_0
$\frac{1}{3} < BF < 1$	Weak evidence for M_0
$1 < BF < 3$	Weak evidence for M_1
$3 < BF < 10$	Moderate evidence for M_1
$BF > 10$	Strong evidence for M_1
$BF > 100$	Decisive evidence for M_1

Table 5.1 Jeffreys' scale of evidence for interpreting Bayes factors.

5.3.3 Bayes factors

Suppose our prior on models is uniform, $p(m) \propto 1$. Then model selection is equivalent to picking the model with the highest marginal likelihood. Now suppose we just have two models we are considering, call them the **null hypothesis**, M_0 , and the **alternative hypothesis**, M_1 . Define the **Bayes factor** as the ratio of marginal likelihoods:

$$BF_{1,0} \triangleq \frac{p(\mathcal{D}|M_1)}{p(\mathcal{D}|M_0)} = \frac{p(M_1|\mathcal{D})}{p(M_0|\mathcal{D})} \frac{p(M_1)}{p(M_0)} \quad (5.39)$$

(This is like a **likelihood ratio**, except we integrate out the parameters, which allows us to compare models of different complexity.) If $BF_{1,0} > 1$ then we prefer model 1, otherwise we prefer model 0.

Of course, it might be that $BF_{1,0}$ is only slightly greater than 1. In that case, we are not very confident that model 1 is better. Jeffreys (1961) proposed a scale of evidence for interpreting the magnitude of a Bayes factor, which is shown in Table 5.1. This is a Bayesian alternative to the frequentist concept of a p-value.³ Alternatively, we can just convert the Bayes factor to a posterior over models. If $p(M_1) = p(M_0) = 0.5$, we have

$$p(M_0|\mathcal{D}) = \frac{BF_{0,1}}{1 + BF_{0,1}} = \frac{1}{BF_{1,0} + 1} \quad (5.40)$$

5.3.3.1 Example: Testing if a coin is fair

Suppose we observe some coin tosses, and want to decide if the data was generated by a fair coin, $\theta = 0.5$, or a potentially biased coin, where θ could be any value in $[0, 1]$. Let us denote the first model by M_0 and the second model by M_1 . The marginal likelihood under M_0 is simply

$$p(\mathcal{D}|M_0) = \left(\frac{1}{2}\right)^N \quad (5.41)$$

3. A **p-value**, is defined as the probability (under the null hypothesis) of observing some **test statistic** $f(\mathcal{D})$ (such as the **chi-squared statistic**) that is as large or larger than that actually observed, i.e., $\text{pvalue}(\mathcal{D}) \triangleq P(f(\tilde{\mathcal{D}}) \geq f(\mathcal{D}) | \tilde{\mathcal{D}} \sim H_0)$. Note that has almost nothing to do with what we really want to know, which is $p(H_0|\mathcal{D})$.

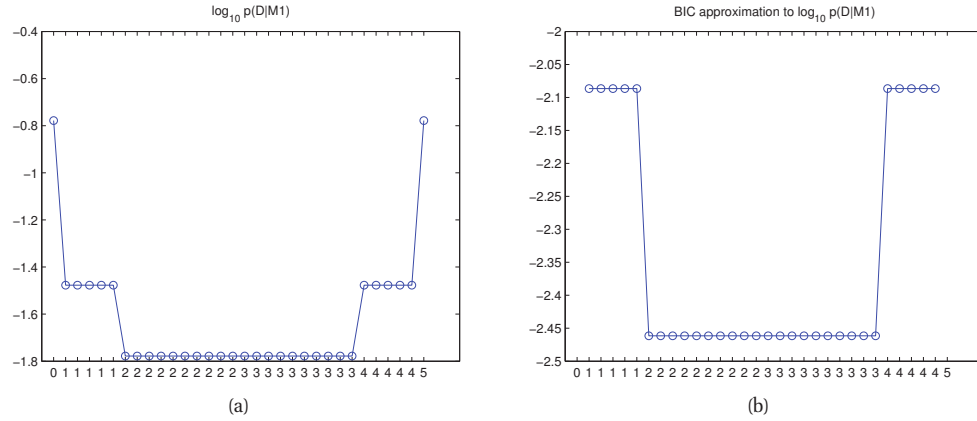


Figure 5.9 (a) Log marginal likelihood for the coins example. (b) BIC approximation. Figure generated by `coinsModelSelDemo`.

where N is the number of coin tosses. The marginal likelihood under M_1 , using a Beta prior, is

$$p(\mathcal{D}|M_1) = \int p(\mathcal{D}|\theta)p(\theta)d\theta = \frac{B(\alpha_1 + N_1, \alpha_0 + N_0)}{B(\alpha_1, \alpha_0)} \quad (5.42)$$

We plot $\log p(\mathcal{D}|M_1)$ vs the number of heads N_1 in Figure 5.9(a), assuming $N = 5$ and $\alpha_1 = \alpha_0 = 1$. (The shape of the curve is not very sensitive to α_1 and α_0 , as long as $\alpha_0 = \alpha_1$.) If we observe 2 or 3 heads, the unbiased coin hypothesis M_0 is more likely than M_1 , since M_0 is a simpler model (it has no free parameters) — it would be a suspicious coincidence if the coin were biased but happened to produce almost exactly 50/50 heads/tails. However, as the counts become more extreme, we favor the biased coin hypothesis. Note that, if we plot the log Bayes factor, $\log BF_{1,0}$, it will have exactly the same shape, since $\log p(\mathcal{D}|M_0)$ is a constant. See also Exercise 3.18.

In Figure 5.9(b) shows the BIC approximation to $\log p(\mathcal{D}|M_1)$ for our biased coin example from Section 5.3.3.1. We see that the curve has approximately the same shape as the exact log marginal likelihood, which is all that matters for model selection purposes, since the absolute scale is irrelevant. In particular, it favors the simpler model unless the data is overwhelmingly in support of the more complex model.

5.3.4 Jeffreys-Lindley paradox *

Problems can arise when we use improper priors (i.e., priors that do not integrate to 1) for model selection/ hypothesis testing, even though such priors may be acceptable for other purposes. For example, consider testing the hypotheses $M_0 : \theta \in \Theta_0$ vs $M_1 : \theta \in \Theta_1$. To define the marginal density on θ , we use the following mixture model

$$p(\theta) = p(\theta|M_0)p(M_0) + p(\theta|M_1)p(M_1) \quad (5.43)$$

This is only meaningful if $p(\theta|M_0)$ and $p(\theta|M_1)$ are proper (normalized) density functions. In this case, the posterior is given by

$$p(M_0|\mathcal{D}) = \frac{p(M_0)p(\mathcal{D}|M_0)}{p(M_0)p(\mathcal{D}|M_0) + p(M_1)p(\mathcal{D}|M_1)} \quad (5.44)$$

$$= \frac{p(M_0) \int_{\Theta_0} p(\mathcal{D}|\theta)p(\theta|M_0)d\theta}{p(M_0) \int_{\Theta_0} p(\mathcal{D}|\theta)p(\theta|M_0)d\theta + p(M_1) \int_{\Theta_1} p(\mathcal{D}|\theta)p(\theta|M_1)d\theta} \quad (5.45)$$

Now suppose we use improper priors, $p(\theta|M_0) \propto c_0$ and $p(\theta|M_1) \propto c_1$. Then

$$p(M_0|\mathcal{D}) = \frac{p(M_0)c_0 \int_{\Theta_0} p(\mathcal{D}|\theta)d\theta}{p(M_0)c_0 \int_{\Theta_0} p(\mathcal{D}|\theta)d\theta + p(M_1)c_1 \int_{\Theta_1} p(\mathcal{D}|\theta)d\theta} \quad (5.46)$$

$$= \frac{p(M_0)c_0\ell_0}{p(M_0)c_0\ell_0 + p(M_1)c_1\ell_1} \quad (5.47)$$

where $\ell_i = \int_{\Theta_i} p(\mathcal{D}|\theta)d\theta$ is the integrated or marginal likelihood for model i . Now let $p(M_0) = p(M_1) = \frac{1}{2}$. Hence

$$p(M_0|\mathcal{D}) = \frac{c_0\ell_0}{c_0\ell_0 + c_1\ell_1} = \frac{\ell_0}{\ell_0 + (c_1/c_0)\ell_1} \quad (5.48)$$

Thus we can change the posterior arbitrarily by choosing c_1 and c_0 as we please. Note that using proper, but very vague, priors can cause similar problems. In particular, the Bayes factor will always favor the simpler model, since the probability of the observed data under a complex model with a very diffuse prior will be very small. This is called the **Jeffreys-Lindley paradox**.

Thus it is important to use proper priors when performing model selection. Note, however, that, if M_0 and M_1 share the same prior over a subset of the parameters, this part of the prior can be improper, since the corresponding normalization constant will cancel out.

5.4 Priors

The most controversial aspect of Bayesian statistics is its reliance on priors. Bayesians argue this is unavoidable, since nobody is a **tabula rasa** or **blank slate**: all inference must be done conditional on certain assumptions about the world. Nevertheless, one might be interested in minimizing the impact of one's prior assumptions. We briefly discuss some ways to do this below.

5.4.1 Uninformative priors

If we don't have strong beliefs about what θ should be, it is common to use an **uninformative** or **non-informative** prior, and to "let the data speak for itself".

The issue of designing uninformative priors is actually somewhat tricky. As an example of the difficulty, consider a Bernoulli parameter, $\theta \in [0, 1]$. One might think that the most uninformative prior would be the uniform distribution, $\text{Beta}(1, 1)$. But the posterior mean in this case is $\mathbb{E}[\theta|\mathcal{D}] = \frac{N_1+1}{N_1+N_0+2}$, whereas the MLE is $\frac{N_1}{N_1+N_0}$. Hence one could argue that the prior wasn't completely uninformative after all.

Clearly by decreasing the magnitude of the pseudo counts, we can lessen the impact of the prior. By the above argument, the most non-informative prior is

$$\lim_{c \rightarrow 0} \text{Beta}(c, c) = \text{Beta}(0, 0) \quad (5.49)$$

which is a mixture of two equal point masses at 0 and 1 (see (Zhu and Lu 2004)). This is also called the **Haldane prior**. Note that the Haldane prior is an improper prior, meaning it does not integrate to 1. However, as long as we see at least one head and at least one tail, the posterior will be proper.

In Section 5.4.2.1 we will argue that the “right” uninformative prior is in fact $\text{Beta}(\frac{1}{2}, \frac{1}{2})$. Clearly the difference in practice between these three priors is very likely negligible. In general, it is advisable to perform some kind of **sensitivity analysis**, in which one checks how much one’s conclusions or predictions change in response to change in the modeling assumptions, which includes the choice of prior, but also the choice of likelihood and any kind of data pre-processing. If the conclusions are relatively insensitive to the modeling assumptions, one can have more confidence in the results.

5.4.2 Jeffreys priors *

Harold Jeffreys⁴ designed a general purpose technique for creating non-informative priors. The result is known as the **Jeffreys prior**. The key observation is that if $p(\phi)$ is non-informative, then any re-parameterization of the prior, such as $\theta = h(\phi)$ for some function h , should also be non-informative. Now, by the change of variables formula,

$$p_\theta(\theta) = p_\phi(\phi) \left| \frac{d\phi}{d\theta} \right| \quad (5.50)$$

so the prior will in general change. However, let us pick

$$p_\phi(\phi) \propto (I(\phi))^{\frac{1}{2}} \quad (5.51)$$

where $I(\phi)$ is the **Fisher information**:

$$I(\phi) \triangleq -\mathbb{E} \left[\left(\frac{d \log p(X|\phi)}{d\phi} \right)^2 \right] \quad (5.52)$$

This is a measure of curvature of the expected negative log likelihood and hence a measure of stability of the MLE (see Section 6.2.2). Now

$$\frac{d \log p(x|\theta)}{d\theta} = \frac{d \log p(x|\phi)}{d\phi} \frac{d\phi}{d\theta} \quad (5.53)$$

Squaring and taking expectations over x , we have

$$I(\theta) = -\mathbb{E} \left[\left(\frac{d \log p(X|\theta)}{d\theta} \right)^2 \right] = I(\phi) \left(\frac{d\phi}{d\theta} \right)^2 \quad (5.54)$$

$$I(\theta)^{\frac{1}{2}} = I(\phi)^{\frac{1}{2}} \left| \frac{d\phi}{d\theta} \right| \quad (5.55)$$

4. Harold Jeffreys, 1891 – 1989, was an English mathematician, statistician, geophysicist, and astronomer.

so we find the transformed prior is

$$p_{\theta}(\theta) = p_{\phi}(\phi) \left| \frac{d\phi}{d\theta} \right| \propto (I(\phi))^{\frac{1}{2}} \left| \frac{d\phi}{d\theta} \right| = I(\theta)^{\frac{1}{2}} \quad (5.56)$$

So $p_{\theta}(\theta)$ and $p_{\phi}(\phi)$ are the same.

Some examples will make this clearer.

5.4.2.1 Example: Jeffreys prior for the Bernoulli and multinoulli

Suppose $X \sim \text{Ber}(\theta)$. The log likelihood for a single sample is

$$\log p(X|\theta) = X \log \theta + (1 - X) \log(1 - \theta) \quad (5.57)$$

The **score function** is just the gradient of the log-likelihood:

$$s(\theta) \triangleq \frac{d}{d\theta} \log p(X|\theta) = \frac{X}{\theta} - \frac{1-X}{1-\theta} \quad (5.58)$$

The **observed information** is the second derivative of the log-likelihood:

$$J(\theta) = -\frac{d^2}{d\theta^2} \log p(X|\theta) = -s'(\theta|X) = \frac{X}{\theta^2} + \frac{1-X}{(1-\theta)^2} \quad (5.59)$$

The Fisher information is the expected information:

$$I(\theta) = E[J(\theta|X)|X \sim \theta] = \frac{\theta}{\theta^2} + \frac{1-\theta}{(1-\theta)^2} = \frac{1}{\theta(1-\theta)} \quad (5.60)$$

Hence Jeffreys' prior is

$$p(\theta) \propto \theta^{-\frac{1}{2}}(1-\theta)^{-\frac{1}{2}} = \frac{1}{\sqrt{\theta(1-\theta)}} \propto \text{Beta}\left(\frac{1}{2}, \frac{1}{2}\right) \quad (5.61)$$

Now consider a multinoulli random variable with K states. One can show that the Jeffreys' prior is given by

$$p(\boldsymbol{\theta}) \propto \text{Dir}\left(\frac{1}{2}, \dots, \frac{1}{2}\right) \quad (5.62)$$

Note that this is different from the more obvious choices of $\text{Dir}(\frac{1}{K}, \dots, \frac{1}{K})$ or $\text{Dir}(1, \dots, 1)$.

5.4.2.2 Example: Jeffreys prior for location and scale parameters

One can show that the Jeffreys prior for a location parameter, such as the Gaussian mean, is $p(\mu) \propto 1$. Thus is an example of a **translation invariant prior**, which satisfies the property that the probability mass assigned to any interval, $[A, B]$ is the same as that assigned to any other shifted interval of the same width, such as $[A - c, B - c]$. That is,

$$\int_{A-c}^{B-c} p(\mu) d\mu = (A - c) - (B - c) = (A - B) = \int_A^B p(\mu) d\mu \quad (5.63)$$

This can be achieved using $p(\mu) \propto 1$, which we can approximate by using a Gaussian with infinite variance, $p(\mu) = \mathcal{N}(\mu|0, \infty)$. Note that this is an **improper prior**, since it does not integrate to 1. Using improper priors is fine as long as the posterior is proper, which will be the case provided we have seen $N \geq 1$ data points, since we can “nail down” the location as soon as we have seen a single data point.

Similarly, one can show that the Jeffreys prior for a scale parameter, such as the Gaussian variance, is $p(\sigma^2) \propto 1/\sigma^2$. This is an example of a **scale invariant prior**, which satisfies the property that the probability mass assigned to any interval $[A, B]$ is the same as that assigned to any other interval $[A/c, B/c]$ which is scaled in size by some constant factor $c > 0$. (For example, if we change units from meters to feet we do not want that to affect our inferences.) This can be achieved by using

$$p(s) \propto 1/s \quad (5.64)$$

To see this, note that

$$\int_{A/c}^{B/c} p(s) ds = [\log s]_{A/c}^{B/c} = \log(B/c) - \log(A/c) \quad (5.65)$$

$$= \log(B) - \log(A) = \int_A^B p(s) ds \quad (5.66)$$

We can approximate this using a degenerate Gamma distribution (Section 2.4.4), $p(s) = \text{Ga}(s|0, 0)$. The prior $p(s) \propto 1/s$ is also improper, but the posterior is proper as soon as we have seen $N \geq 2$ data points (since we need at least two data points to estimate a variance).

5.4.3 Robust priors

In many cases, we are not very confident in our prior, so we want to make sure it does not have an undue influence on the result. This can be done by using **robust priors** (Insua and Ruggeri 2000), which typically have heavy tails, which avoids forcing things to be too close to the prior mean.

Let us consider an example from (Berger 1985, p7). Suppose $x \sim \mathcal{N}(\theta, 1)$. We observe that $x = 5$ and we want to estimate θ . The MLE is of course $\hat{\theta} = 5$, which seems reasonable. The posterior mean under a uniform prior is also $\bar{\theta} = 5$. But now suppose we know that the prior median is 0, and the prior quantiles are at -1 and 1, so $p(\theta \leq -1) = p(-1 < \theta \leq 0) = p(0 < \theta \leq 1) = p(1 < \theta) = 0.25$. Let us also assume the prior is smooth and unimodal.

It is easy to show that a Gaussian prior of the form $\mathcal{N}(\theta|0, 2.19^2)$ satisfies these prior constraints. But in this case the posterior mean is given by 3.43, which doesn't seem very satisfactory.

Now suppose we use as a Cauchy prior $\mathcal{T}(\theta|0, 1, 1)$. This also satisfies the prior constraints of our example. But this time we find (using numerical method integration: see `robustPriorDemo` for the code) that the posterior mean is about 4.6, which seems much more reasonable.

5.4.4 Mixtures of conjugate priors

Robust priors are useful, but can be computationally expensive to use. Conjugate priors simplify the computation, but are often not robust, and not flexible enough to encode our prior knowl-

edge. However, it turns out that a **mixture of conjugate priors** is also conjugate (Exercise 5.1), and can approximate any kind of prior (Dallal and Hall 1983; Diaconis and Ylvisaker 1985). Thus such priors provide a good compromise between computational convenience and flexibility.

For example, suppose we are modeling coin tosses, and we think the coin is either fair, or is biased towards heads. This cannot be represented by a beta distribution. However, we can model it using a mixture of two beta distributions. For example, we might use

$$p(\theta) = 0.5 \text{Beta}(\theta|20, 20) + 0.5 \text{Beta}(\theta|30, 10) \quad (5.67)$$

If θ comes from the first distribution, the coin is fair, but if it comes from the second, it is biased towards heads.

We can represent a mixture by introducing a latent indicator variable z , where $z = k$ means that θ comes from mixture component k . The prior has the form

$$p(\theta) = \sum_k p(z = k)p(\theta|z = k) \quad (5.68)$$

where each $p(\theta|z = k)$ is conjugate, and $p(z = k)$ are called the (prior) mixing weights. One can show (Exercise 5.1) that the posterior can also be written as a mixture of conjugate distributions as follows:

$$p(\theta|\mathcal{D}) = \sum_k p(z = k|\mathcal{D})p(\theta|\mathcal{D}, z = k) \quad (5.69)$$

where $p(Z = k|\mathcal{D})$ are the posterior mixing weights given by

$$p(Z = k|\mathcal{D}) = \frac{p(Z = k)p(\mathcal{D}|Z = k)}{\sum_{k'} p(Z = k')p(\mathcal{D}|Z = k')} \quad (5.70)$$

Here the quantity $p(\mathcal{D}|Z = k)$ is the marginal likelihood for mixture component k (see Section 5.3.2.1).

5.4.4.1 Example

Suppose we use the mixture prior

$$p(\theta) = 0.5\text{Beta}(\theta|a_1, b_1) + 0.5\text{Beta}(\theta|a_2, b_2) \quad (5.71)$$

where $a_1 = b_1 = 20$ and $a_2 = b_2 = 10$. and we observe N_1 heads and N_0 tails. The posterior becomes

$$p(\theta|\mathcal{D}) = p(Z = 1|\mathcal{D})\text{Beta}(\theta|a_1 + N_1, b_1 + N_0) + p(Z = 2|\mathcal{D})\text{Beta}(\theta|a_2 + N_1, b_2 + N_0) \quad (5.72)$$

If $N_1 = 20$ heads and $N_0 = 10$ tails, then, using Equation 5.23, the posterior becomes

$$p(\theta|\mathcal{D}) = 0.346 \text{Beta}(\theta|40, 30) + 0.654 \text{Beta}(\theta|50, 20) \quad (5.73)$$

See Figure 5.10 for an illustration.

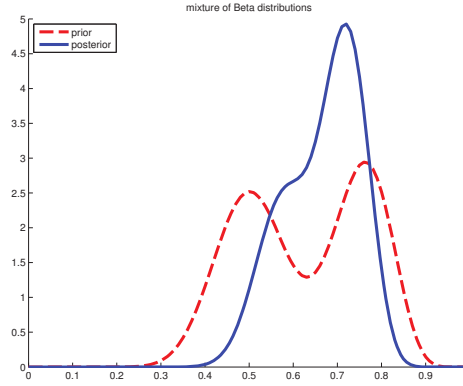


Figure 5.10 A mixture of two Beta distributions. Figure generated by `mixBetaDemo`.

5.4.4.2 Application: Finding conserved regions in DNA and protein sequences

We mentioned that Dirichlet-multinomial models are widely used in biosequence analysis. Let us give a simple example to illustrate some of the machinery that has developed. Specifically, consider the sequence logo discussed in Section 2.3.2.1. Now suppose we want to find locations which represent coding regions of the genome. Such locations often have the same letter across all sequences, because of evolutionary pressure. So we need to find columns which are “pure”, or nearly so, in the sense that they are mostly all As, mostly all Ts, mostly all Cs, or mostly all Gs. One approach is to look for low-entropy columns; these will be ones whose distribution is nearly deterministic (pure).

But suppose we want to associate a confidence measure with our estimates of purity. This can be useful if we believe adjacent locations are conserved together. In this case, we can let $Z_1 = 1$ if location t is conserved, and let $Z_t = 0$ otherwise. We can then add a dependence between adjacent Z_t variables using a Markov chain; see Chapter 17 for details.

In any case, we need to define a likelihood model, $p(\mathbf{N}_t|Z_t)$, where $\mathbf{N}_t$ is the vector of (A,C,G,T) counts for column t . It is natural to make this be a multinomial distribution with parameter θ_t . Since each column has a different distribution, we will want to integrate out θ_t and thus compute the marginal likelihood

$$p(\mathbf{N}_t|Z_t) = \int p(\mathbf{N}_t|\theta_t)p(\theta_t|Z_t)d\theta_t \quad (5.74)$$

But what prior should we use for θ_t ? When $Z_t = 0$ we can use a uniform prior, $p(\theta|Z_t = 0) = \text{Dir}(1, 1, 1, 1)$, but what should we use if $Z_t = 1$? After all, if the column is conserved, it could be a (nearly) pure column of As, Cs, Gs, or Ts. A natural approach is to use a mixture of Dirichlet priors, each one of which is “tilted” towards the appropriate corner of the 4-dimensional simplex, e.g.,

$$p(\theta|Z_t = 1) = \frac{1}{4}\text{Dir}(\theta|(10, 1, 1, 1)) + \cdots + \frac{1}{4}\text{Dir}(\theta|(1, 1, 1, 10)) \quad (5.75)$$

Since this is conjugate, we can easily compute $p(\mathbf{N}_t|Z_t)$. See (Brown et al. 1993) for an

application of these ideas to a real bio-sequence problem.

5.5 Hierarchical Bayes

A key requirement for computing the posterior $p(\theta|\mathcal{D})$ is the specification of a prior $p(\theta|\eta)$, where η are the hyper-parameters. What if we don't know how to set η ? In some cases, we can use uninformative priors, we discussed above. A more Bayesian approach is to put a prior on our priors! In terms of graphical models (Chapter 10), we can represent the situation as follows:

$$\eta \rightarrow \theta \rightarrow \mathcal{D} \quad (5.76)$$

This is an example of a **hierarchical Bayesian model**, also called a **multi-level model**, since there are multiple levels of unknown quantities. We give a simple example below, and we will see many others later in the book.

5.5.1 Example: modeling related cancer rates

Consider the problem of predicting cancer rates in various cities (this example is from (Johnson and Albert 1999, p24)). In particular, suppose we measure the number of people in various cities, N_i , and the number of people who died of cancer in these cities, x_i . We assume $x_i \sim \text{Bin}(N_i, \theta_i)$, and we want to estimate the cancer rates θ_i . One approach is to estimate them all separately, but this will suffer from the sparse data problem (underestimation of the rate of cancer due to small N_i). Another approach is to assume all the θ_i are the same; this is called **parameter tying**. The resulting pooled MLE is just $\hat{\theta} = \frac{\sum_i x_i}{\sum_i N_i}$. But the assumption that all the cities have the same rate is a rather strong one. A compromise approach is to assume that the θ_i are similar, but that there may be city-specific variations. This can be modeled by assuming the θ_i are drawn from some common distribution, say $\theta_i \sim \text{Beta}(a, b)$. The full joint distribution can be written as

$$p(\mathcal{D}, \theta, \eta | \mathbf{N}) = p(\eta) \prod_{i=1}^N \text{Bin}(x_i | N_i, \theta_i) \text{Beta}(\theta_i | \eta) \quad (5.77)$$

where $\eta = (a, b)$.

Note that it is crucial that we infer $\eta = (a, b)$ from the data; if we just clamp it to a constant, the θ_i will be conditionally independent, and there will be no information flow between them. By contrast, by treating η as an unknown (hidden variable), we allow the data-poor cities to **borrow statistical strength** from data-rich ones.

Suppose we compute the joint posterior $p(\eta, \theta | \mathcal{D})$. From this we can get the posterior marginals $p(\theta_i | \mathcal{D})$. In Figure 5.11(a), we plot the posterior means, $\mathbb{E}[\theta_i | \mathcal{D}]$, as blue bars, as well as the population level mean, $\mathbb{E}[a/(a+b) | \mathcal{D}]$, shown as a red line (this represents the average of the θ_i 's). We see that the posterior mean is shrunk towards the pooled estimate more strongly for cities with small sample sizes N_i . For example, city 1 and city 20 both have a 0 observed cancer incidence rate, but city 20 has a smaller population, so its rate is shrunk more towards the population-level estimate (i.e., it is closer to the horizontal red line) than city 1.

Figure 5.11(b) shows the 95% posterior credible intervals for θ_i . We see that city 15, which has a very large population (53,637 people), has small posterior uncertainty. Consequently this city

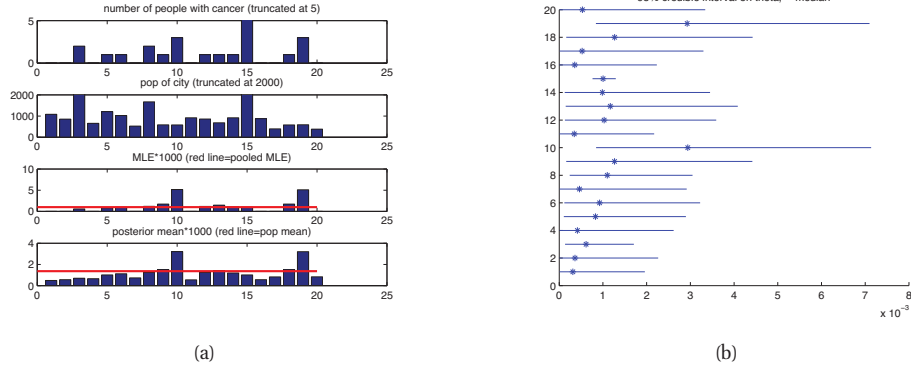


Figure 5.11 (a) Results of fitting the model using the data from (Johnson and Albert 1999, p24). First row: Number of cancer incidents x_i in 20 cities in Missouri. Second row: population size N_i . The largest city (number 15) has a population of $N_{15} = 53637$ and $x_{15} = 54$ incidents, but we truncate the vertical axes of the first two rows so that the differences between the other cities are visible. Third row: MLE $\hat{\theta}_i$. The red line is the pooled MLE. Fourth row: posterior mean $\mathbb{E}[\theta_i|\mathcal{D}]$. The red line is $\mathbb{E}[a/(a+b)|\mathcal{D}]$, the population-level mean. (b) Posterior 95% credible intervals on the cancer rates. Figure generated by `cancerRatesEb`

has the largest impact on the posterior estimate of η , which in turn will impact the estimate of the cancer rates for other cities. Cities 10 and 19, which have the highest MLE, also have the highest posterior uncertainty, reflecting the fact that such a high estimate is in conflict with the prior (which is estimated from all the other cities).

In the above example, we have one parameter per city, modeling the probability the response is on. By making the Bernoulli rate parameter be a function of covariates, $\theta_i = \text{sigm}(\mathbf{w}_i^T \mathbf{x})$, we can model multiple correlated logistic regression tasks. This is called **multi-task learning**, and will be discussed in more detail in Section 9.5.

5.6 Empirical Bayes

In hierarchical Bayesian models, we need to compute the posterior on multiple levels of latent variables. For example, in a two-level model, we need to compute

$$p(\eta, \theta|\mathcal{D}) \propto p(\mathcal{D}|\theta)p(\theta|\eta)p(\eta) \quad (5.78)$$

In some cases, we can analytically marginalize out θ ; this leaves us with the simpler problem of just computing $p(\eta|\mathcal{D})$.

As a computational shortcut, we can approximate the posterior on the hyper-parameters with a point-estimate, $p(\eta|\mathcal{D}) \approx \delta_{\hat{\eta}}(\eta)$, where $\hat{\eta} = \text{argmax}_{\eta} p(\eta|\mathcal{D})$. Since η is typically much smaller than θ in dimensionality, it is less prone to overfitting, so we can safely use a uniform prior on η . Then the estimate becomes

$$\hat{\eta} = \text{argmax}_{\eta} p(\mathcal{D}|\eta) = \text{argmax}_{\eta} \left[\int p(\mathcal{D}|\theta)p(\theta|\eta)d\theta \right] \quad (5.79)$$

where the quantity inside the brackets is the marginal or integrated likelihood, sometimes called the evidence. This overall approach is called **empirical Bayes (EB)** or **type-II maximum likelihood**. In machine learning, it is sometimes called the **evidence procedure**.

Empirical Bayes violates the principle that the prior should be chosen independently of the data. However, we can just view it as a computationally cheap approximation to inference in a hierarchical Bayesian model, just as we viewed MAP estimation as an approximation to inference in the one level model $\theta \rightarrow \mathcal{D}$. In fact, we can construct a hierarchy in which the more integrals one performs, the “more Bayesian” one becomes:

Method	Definition
Maximum likelihood	$\hat{\theta} = \operatorname{argmax}_{\theta} p(\mathcal{D} \theta)$
MAP estimation	$\hat{\theta} = \operatorname{argmax}_{\theta} p(\mathcal{D} \theta)p(\theta \eta)$
ML-II (Empirical Bayes)	$\hat{\eta} = \operatorname{argmax}_{\eta} \int p(\mathcal{D} \theta)p(\theta \eta)d\theta = \operatorname{argmax}_{\eta} p(\mathcal{D} \eta)$
MAP-II	$\hat{\eta} = \operatorname{argmax}_{\eta} \int p(\mathcal{D} \theta)p(\theta \eta)p(\eta)d\theta = \operatorname{argmax}_{\eta} p(\mathcal{D} \eta)p(\eta)$
Full Bayes	$p(\theta, \eta \mathcal{D}) \propto p(\mathcal{D} \theta)p(\theta \eta)p(\eta)$

Note that EB can be shown to have good frequentist properties (see e.g., (Carlin and Louis 1996; Efron 2010)), so it is widely used by non-Bayesians. For example, the popular James-Stein estimator, discussed in Section 6.3.3.2, can be derived using EB.

5.6.1 Example: beta-binomial model

Let us return to the cancer rates model. We can analytically integrate out the θ_i ’s, and write down the marginal likelihood directly, as follows:

$$p(\mathcal{D}|a, b) = \prod_i \int \operatorname{Bin}(x_i|N_i, \theta_i) \operatorname{Beta}(\theta_i|a, b) d\theta_i \quad (5.80)$$

$$= \prod_i \frac{B(a + x_i, b + N_i - x_i)}{B(a, b)} \quad (5.81)$$

Various ways of maximizing this wrt a and b are discussed in (Minka 2000e).

Having estimated a and b , we can plug in the hyper-parameters to compute the posterior $p(\theta_i|\hat{a}, \hat{b}, \mathcal{D})$ in the usual way, using conjugate analysis. The net result is that the posterior mean of each θ_i is a weighted average of its local MLE and the prior means, which depends on $\eta = (a, b)$; but since η is estimated based on all the data, each θ_i is influenced by all the data.

5.6.2 Example: Gaussian-Gaussian model

We now study another example that is analogous to the cancer rates example, except the data is real-valued. We will use a Gaussian likelihood and a Gaussian prior. This will allow us to write down the solution analytically.

In particular, suppose we have data from multiple related groups. For example, x_{ij} could be the test score for student i in school j , for $j = 1 : D$ and $i = 1 : N_j$. We want to estimate the mean score for each school, θ_j . However, since the sample size, N_j , may be small for

some schools, we can regularize the problem by using a hierarchical Bayesian model, where we assume θ_j come from a common prior, $\mathcal{N}(\mu, \tau^2)$.

The joint distribution has the following form:

$$p(\boldsymbol{\theta}, \mathcal{D} | \boldsymbol{\eta}, \sigma^2) = \prod_{j=1}^D \mathcal{N}(\theta_j | \mu, \tau^2) \prod_{i=1}^{N_j} \mathcal{N}(x_{ij} | \theta_j, \sigma^2) \quad (5.82)$$

where we assume σ^2 is known for simplicity. (We relax this assumption in Exercise 24.4.) We explain how to estimate $\boldsymbol{\eta}$ below. Once we have estimated $\boldsymbol{\eta} = (\mu, \tau)$, we can compute the posteriors over the θ_j 's. To do that, it simplifies matters to rewrite the joint distribution in the following form, exploiting the fact that N_j Gaussian measurements with values x_{ij} and variance σ^2 are equivalent to one measurement of value $\bar{x}_j \triangleq \frac{1}{N_j} \sum_{i=1}^{N_j} x_{ij}$ with variance $\sigma_j^2 \triangleq \sigma^2 / N_j$. This yields

$$p(\boldsymbol{\theta}, \mathcal{D} | \hat{\boldsymbol{\eta}}, \sigma^2) = \prod_{j=1}^D \mathcal{N}(\theta_j | \hat{\mu}, \hat{\tau}^2) \mathcal{N}(\bar{x}_j | \theta_j, \sigma_j^2) \quad (5.83)$$

From this, it follows from the results of Section 4.4.1 that the posteriors are given by

$$p(\theta_j | \mathcal{D}, \hat{\mu}, \hat{\tau}^2) = \mathcal{N}(\theta_j | \hat{B}_j \hat{\mu} + (1 - \hat{B}_j) \bar{x}_j, (1 - \hat{B}_j) \sigma_j^2) \quad (5.84)$$

$$\hat{B}_j \triangleq \frac{\sigma_j^2}{\sigma_j^2 + \hat{\tau}^2} \quad (5.85)$$

where $\hat{\mu} = \bar{x}$ and $\hat{\tau}^2$ will be defined below.

The quantity $0 \leq \hat{B}_j \leq 1$ controls the degree of **shrinkage** towards the overall mean, μ . If the data is reliable for group j (e.g., because the sample size N_j is large), then σ_j^2 will be small relative to τ^2 ; hence $\hat{B}_j$ will be small, and we will put more weight on $\bar{x}_j$ when we estimate θ_j . However, groups with small sample sizes will get regularized (shrunk towards the overall mean μ) more heavily. We will see an example of this below.

If $\sigma_j = \sigma$ for all groups j , the posterior mean becomes

$$\hat{\theta}_j = \hat{B} \bar{x} + (1 - \hat{B}) \bar{x}_j = \bar{x} + (1 - \hat{B})(\bar{x}_j - \bar{x}) \quad (5.86)$$

This has exactly the same form as the James Stein estimator discussed in Section 6.3.3.2.

5.6.2.1 Example: predicting baseball scores

We now give an example of shrinkage applied to baseball batting averages, from (Efron and Morris 1975). We observe the number of hits for $D = 18$ players during the first $T = 45$ games. Call the number of hits b_i . We assume $b_j \sim \text{Bin}(T, \theta_j)$, where θ_j is the “true” batting average for player j . The goal is to estimate the θ_j . The MLE is of course $\hat{\theta}_j = x_j$, where $x_j = b_j/T$ is the empirical batting average. However, we can use an EB approach to do better.

To apply the Gaussian shrinkage approach described above, we require that the likelihood be Gaussian, $x_j \sim \mathcal{N}(\theta_j, \sigma^2)$ for known σ^2 . (We drop the i subscript since we assume $N_j = 1$,

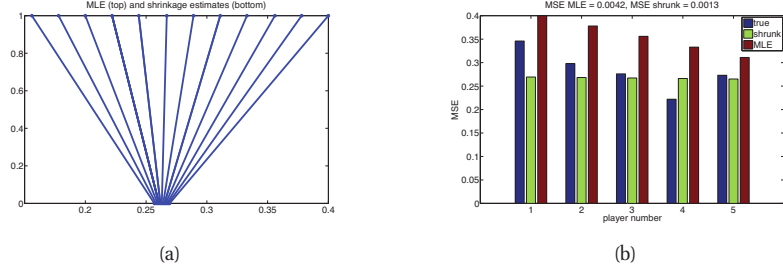


Figure 5.12 (a) MLE parameters (top) and corresponding shrunk estimates (bottom). (b) We plot the true parameters (blue), the posterior mean estimate (green), and the MLEs (red) for 5 of the players. Figure generated by `shrinkageDemoBaseball`.

since x_j already represents the average for player j .) However, in this example we have a binomial likelihood. While this has the right mean, $\mathbb{E}[x_j] = \theta_j$, the variance is not constant:

$$\text{var}[x_j] = \frac{1}{T^2} \text{var}[b_j] = \frac{T\theta_j(1-\theta_j)}{T^2} \quad (5.87)$$

So let us apply a **variance stabilizing transform**⁵ to x_j to better match the Gaussian assumption:

$$y_j = f(y_j) = \sqrt{T} \arcsin(2y_j - 1) \quad (5.88)$$

Now we have approximately $y_j \sim \mathcal{N}(f(\theta_j), 1) = \mathcal{N}(\mu_j, 1)$. We use Gaussian shrinkage to estimate the μ_j using Equation 5.86 with $\sigma^2 = 1$, and we then transform back to get

$$\hat{\theta}_j = 0.5(\sin(\hat{\mu}_j/\sqrt{T}) + 1) \quad (5.89)$$

The results are shown in Figure 5.12(a-b). In (a), we plot the MLE $\hat{\theta}_j$ and the posterior mean $\bar{\theta}_j$. We see that all the estimates have shrunk towards the global mean, 0.265. In (b), we plot the true value θ_j , the MLE $\hat{\theta}_j$ and the posterior mean $\bar{\theta}_j$. (The “true” values of θ_j are estimated from a large number of independent games.) We see that, on average, the shrunk estimate is much closer to the true parameters than the MLE is. Specifically, the mean squared error, defined by $\text{MSE} = \frac{1}{N} \sum_{j=1}^D (\theta_j - \bar{\theta}_j)^2$, is over three times smaller using the shrinkage estimates $\bar{\theta}_j$ than using the MLEs $\hat{\theta}_j$.

5.6.2.2 Estimating the hyper-parameters

In this section, we give an algorithm for estimating η . Suppose initially that $\sigma_j^2 = \sigma^2$ is the same for all groups. In this case, we can derive the EB estimate in closed form, as we now show. From Equation 4.126, we have

$$p(\bar{x}_j | \mu, \tau^2, \sigma^2) = \int \mathcal{N}(\bar{x}_j | \theta_j, \sigma^2) \mathcal{N}(\theta_j | \mu, \tau^2) d\theta_j = \mathcal{N}(\bar{x}_j | \mu, \tau^2 + \sigma^2) \quad (5.90)$$

5. Suppose $\mathbb{E}[X] = \mu$ and $\text{var}[X] = \sigma^2(\mu)$. Let $Y = f(X)$. Then a Taylor series expansions gives $Y \approx f(\mu) + (X - \mu)f'(\mu)$. Hence $\text{var}[Y] \approx f'(\mu)^2 \text{var}[X - \mu] = f'(\mu)^2 \sigma^2(\mu)$. A variance stabilizing transformation is a function f such that $f'(\mu)^2 \sigma^2(\mu)$ is independent of μ .

Hence the marginal likelihood is

$$p(\mathcal{D}|\mu, \tau^2, \sigma^2) = \prod_{j=1}^D \mathcal{N}(\bar{x}_j|\mu, \tau^2 + \sigma^2) \quad (5.91)$$

Thus we can estimate the hyper-parameters using the usual MLEs for a Gaussian. For μ , we have

$$\hat{\mu} = \frac{1}{D} \sum_{j=1}^D \bar{x}_j = \bar{x} \quad (5.92)$$

which is the overall mean.

For the variance, we can use moment matching (which is equivalent to the MLE for a Gaussian): we simply equate the model variance to the empirical variance:

$$\hat{\tau}^2 + \sigma^2 = \frac{1}{D} \sum_{j=1}^D (\bar{x}_j - \bar{x})^2 \triangleq s^2 \quad (5.93)$$

so $\hat{\tau}^2 = s^2 - \sigma^2$. Since we know τ^2 must be positive, it is common to use the following revised estimate:

$$\hat{\tau}^2 = \max\{0, s^2 - \sigma^2\} = (s^2 - \sigma^2)_+ \quad (5.94)$$

Hence the shrinkage factor is

$$\hat{B} = \frac{\sigma^2}{\sigma^2 + \hat{\tau}^2} = \frac{\sigma^2}{\sigma^2 + (s^2 - \sigma^2)_+} \quad (5.95)$$

In the case where the σ_j^2 's are different, we can no longer derive a solution in closed form. Exercise 11.13 discusses how to use the EM algorithm to derive an EB estimate, and Exercise 24.4 discusses how to perform full Bayesian inference in this hierarchical model.

5.7 Bayesian decision theory

We have seen how probability theory can be used to represent and updates our beliefs about the state of the world. However, ultimately our goal is to convert our beliefs into actions. In this section, we discuss the optimal way to do this.

We can formalize any given statistical decision problem as a game against nature (as opposed to a game against other strategic players, which is the topic of game theory, see e.g., (Shoham and Leyton-Brown 2009) for details). In this game, nature picks a state or parameter or label, $y \in \mathcal{Y}$, unknown to us, and then generates an observation, $\mathbf{x} \in \mathcal{X}$, which we get to see. We then have to make a decision, that is, we have to choose an action a from some **action space** $\mathcal{A}$. Finally we incur some **loss**, $L(y, a)$, which measures how compatible our action a is with nature's hidden state y . For example, we might use misclassification loss, $L(y, a) = \mathbb{I}(y \neq a)$, or squared loss, $L(y, a) = (y - a)^2$. We will see some other examples below.

Our goal is to devise a **decision procedure** or **policy**, $\delta : \mathcal{X} \rightarrow \mathcal{A}$, which specifies the optimal action for each possible input. By optimal, we mean the action that minimizes the expected loss:

$$\delta(\mathbf{x}) = \operatorname{argmin}_{a \in \mathcal{A}} \mathbb{E}[L(y, a)] \quad (5.96)$$

In economics, it is more common to talk of a **utility function**; this is just negative loss, $U(y, a) = -L(y, a)$. Thus the above rule becomes

$$\delta(\mathbf{x}) = \operatorname{argmax}_{a \in \mathcal{A}} \mathbb{E}[U(y, a)] \quad (5.97)$$

This is called the **maximum expected utility principle**, and is the essence of what we mean by **rational behavior**.

Note that there are two different interpretations of what we mean by “expected”. In the Bayesian version, which we discuss below, we mean the expected value of y given the data we have seen so far. In the frequentist version, which we discuss in Section 6.3, we mean the expected value of y and $\mathbf{x}$ that we expect to see in the future.

In the Bayesian approach to decision theory, the optimal action, having observed $\mathbf{x}$, is defined as the action a that minimizes the **posterior expected loss**:

$$\rho(a|\mathbf{x}) \triangleq \mathbb{E}_{p(y|\mathbf{x})}[L(y, a)] = \sum_y L(y, a)p(y|\mathbf{x}) \quad (5.98)$$

(If y is continuous (e.g., when we want to estimate a parameter vector), we should replace the sum with an integral.) Hence the **Bayes estimator**, also called the **Bayes decision rule**, is given by

$$\delta(\mathbf{x}) = \operatorname{argmin}_{a \in \mathcal{A}} \rho(a|\mathbf{x}) \quad (5.99)$$

5.7.1 Bayes estimators for common loss functions

In this section we show how to construct Bayes estimators for the loss functions most commonly arising in machine learning.

5.7.1.1 MAP estimate minimizes 0-1 loss

The **0-1 loss** is defined by

$$L(y, a) = \mathbb{I}(y \neq a) = \begin{cases} 0 & \text{if } a = y \\ 1 & \text{if } a \neq y \end{cases} \quad (5.100)$$

This is commonly used in classification problems where y is the true class label and $a = \hat{y}$ is the estimate.

For example, in the two class case, we can write the loss matrix as follows:

	$\hat{y} = 1$	$\hat{y} = 0$
$y = 1$	0	1
$y = 0$	1	0

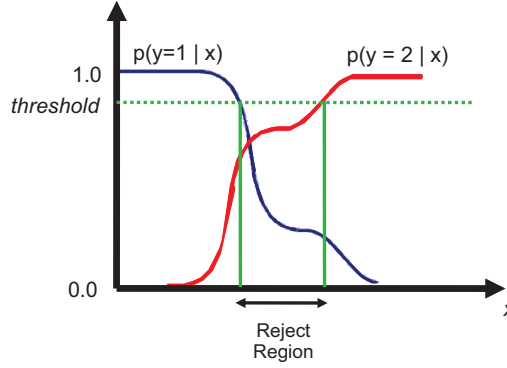


Figure 5.13 For some regions of input space, where the class posteriors are uncertain, we may prefer not to choose class 1 or 2; instead we may prefer the reject option. Based on Figure 1.26 of (Bishop 2006a).

(In Section 5.7.2, we generalize this loss function so it penalizes the two kinds of errors on the off-diagonal differently.)

The posterior expected loss is

$$\rho(a|\mathbf{x}) = p(a \neq y|\mathbf{x}) = 1 - p(y|\mathbf{x}) \quad (5.101)$$

Hence the action that minimizes the expected loss is the posterior mode or MAP estimate

$$y^*(\mathbf{x}) = \arg \max_{y \in \mathcal{Y}} p(y|\mathbf{x}) \quad (5.102)$$

5.7.1.2 Reject option

In classification problems where $p(y|\mathbf{x})$ is very uncertain, we may prefer to choose a **reject action**, in which we refuse to classify the example as any of the specified classes, and instead say “don’t know”. Such ambiguous cases can be handled by e.g., a human expert. See Figure 5.13 for an illustration. This is useful in **risk averse** domains such as medicine and finance.

We can formalize the reject option as follows. Let choosing $a = C + 1$ correspond to picking the reject action, and choosing $a \in \{1, \dots, C\}$ correspond to picking one of the classes. Suppose we define the loss function as

$$L(y = j, a = i) = \begin{cases} 0 & \text{if } i = j \text{ and } i, j \in \{1, \dots, C\} \\ \lambda_r & \text{if } i = C + 1 \\ \lambda_s & \text{otherwise} \end{cases} \quad (5.103)$$

where λ_r is the cost of the reject action, and λ_s is the cost of a substitution error. In Exercise 5.3, you will show that the optimal action is to pick the reject action if the most probable class has a probability below $1 - \frac{\lambda_r}{\lambda_s}$; otherwise you should just pick the most probable class.

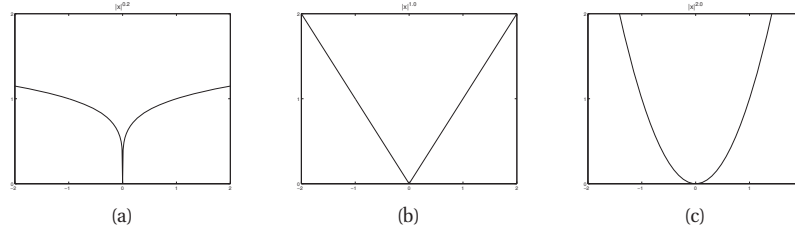


Figure 5.14 (a-c). Plots of the $L(y, a) = |y - a|^q$ vs $|y - a|$ for $q = 0.2$, $q = 1$ and $q = 2$. Figure generated by `lossFunctionFig`.

5.7.1.3 Posterior mean minimizes ℓ_2 (quadratic) loss

For continuous parameters, a more appropriate loss function is **squared error**, ℓ_2 **loss**, or **quadratic loss**, defined as

$$L(y, a) = (y - a)^2 \quad (5.104)$$

The posterior expected loss is given by

$$\rho(a|\mathbf{x}) = \mathbb{E}[(y - a)^2|\mathbf{x}] = \mathbb{E}[y^2|\mathbf{x}] - 2a\mathbb{E}[y|\mathbf{x}] + a^2 \quad (5.105)$$

Hence the optimal estimate is the posterior mean:

$$\frac{\partial}{\partial a} \rho(a|\mathbf{x}) = -2\mathbb{E}[y|\mathbf{x}] + 2a = 0 \Rightarrow \hat{y} = \mathbb{E}[y|\mathbf{x}] = \int yp(y|\mathbf{x})dy \quad (5.106)$$

This is often called the **minimum mean squared error** estimate or **MMSE** estimate.

In a linear regression problem, we have

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y|\mathbf{x}^T \mathbf{w}, \sigma^2) \quad (5.107)$$

In this case, the optimal estimate given some training data $\mathcal{D}$ is given by

$$\mathbb{E}[y|\mathbf{x}, \mathcal{D}] = \mathbf{x}^T \mathbb{E}[\mathbf{w}|\mathcal{D}] \quad (5.108)$$

That is, we just plug-in the posterior mean parameter estimate. Note that this is the optimal thing to do no matter what prior we use for $\mathbf{w}$.

5.7.1.4 Posterior median minimizes ℓ_1 (absolute) loss

The ℓ_2 loss penalizes deviations from the truth quadratically, and thus is sensitive to outliers. A more robust alternative is the absolute or ℓ_1 **loss**, $L(y, a) = |y - a|$ (see Figure 5.14). The optimal estimate is the posterior median, i.e., a value a such that $P(y < a|\mathbf{x}) = P(y \geq a|\mathbf{x}) = 0.5$. See Exercise 5.9 for a proof.

5.7.1.5 Supervised learning

Consider a prediction function $\delta : \mathcal{X} \rightarrow \mathcal{Y}$, and suppose we have some cost function $\ell(y, y')$ which gives the cost of predicting y' when the truth is y . We can define the loss incurred by

taking action δ (i.e., using this predictor) when the unknown state of nature is θ (the parameters of the data generating mechanism) as follows:

$$L(\theta, \delta) \triangleq \mathbb{E}_{(\mathbf{x}, y) \sim p(\mathbf{x}, y | \theta)} [\ell(y, \delta(\mathbf{x}))] = \sum_{\mathbf{x}} \sum_y L(y, \delta(\mathbf{x})) p(\mathbf{x}, y | \theta) \quad (5.109)$$

This is known as the **generalization error**. Our goal is to minimize the posterior expected loss, given by

$$\rho(\delta | \mathcal{D}) = \int p(\theta | \mathcal{D}) L(\theta, \delta) d\theta \quad (5.110)$$

This should be contrasted with the frequentist risk which is defined in Equation 6.47.

5.7.2 The false positive vs false negative tradeoff

In this section, we focus on binary decision problems, such as hypothesis testing, two-class classification, object/ event detection, etc. There are two types of error we can make: a **false positive** (aka **false alarm**), which arises when we estimate $\hat{y} = 1$ but the truth is $y = 0$; or a **false negative** (aka **missed detection**), which arises when we estimate $\hat{y} = 0$ but the truth is $y = 1$. The 0-1 loss treats these two kinds of errors equivalently. However, we can consider the following more general loss matrix:

	$\hat{y} = 1$	$\hat{y} = 0$
$y = 1$	0	L_{FN}
$y = 0$	L_{FP}	0

where L_{FN} is the cost of a false negative, and L_{FP} is the cost of a false positive. The posterior expected loss for the two possible actions is given by

$$\rho(\hat{y} = 0 | \mathbf{x}) = L_{FN} p(y = 1 | \mathbf{x}) \quad (5.111)$$

$$\rho(\hat{y} = 1 | \mathbf{x}) = L_{FP} p(y = 0 | \mathbf{x}) \quad (5.112)$$

Hence we should pick class $\hat{y} = 1$ iff

$$\rho(\hat{y} = 0 | \mathbf{x}) > \rho(\hat{y} = 1 | \mathbf{x}) \quad (5.113)$$

$$\frac{p(y = 1 | \mathbf{x})}{p(y = 0 | \mathbf{x})} > \frac{L_{FP}}{L_{FN}} \quad (5.114)$$

If $L_{FN} = cL_{FP}$, it is easy to show (Exercise 5.10) that we should pick $\hat{y} = 1$ iff $p(y = 1 | \mathbf{x})/p(y = 0 | \mathbf{x}) > \tau$, where $\tau = c/(1 + c)$ (see also (Muller et al. 2004)). For example, if a false negative costs twice as much as false positive, so $c = 2$, then we use a decision threshold of 2/3 before declaring a positive.

Below we discuss ROC curves, which provide a way to study the FP-FN tradeoff without having to choose a specific threshold.

5.7.2.1 ROC curves and all that

Suppose we are solving a binary decision problem, such as classification, hypothesis testing, object detection, etc. Also, assume we have a labeled data set, $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}$. Let $\delta(\mathbf{x}) =$

		Truth		
		1	0	Σ
Estimate	1	TP	FP	$\hat{N}_+ = TP + FP$
	0	FN	TN	$\hat{N}_- = FN + TN$
Σ		$N_+ = TP + FN$	$N_- = FP + TN$	$N = TP + FP + FN + TN$

Table 5.2 Quantities derivable from a confusion matrix. N_+ is the true number of positives, $\hat{N}_+$ is the “called” number of positives, N_- is the true number of negatives, $\hat{N}_-$ is the “called” number of negatives.

	$y = 1$	$y = 0$
$\hat{y} = 1$	$TP/N_+ = \text{TPR} = \text{sensitivity} = \text{recall}$	$FP/N_- = \text{FPR} = \text{type I}$
$\hat{y} = 0$	$FN/N_+ = \text{FNR} = \text{miss rate} = \text{type II}$	$TN/N_- = \text{TNR} = \text{specificity}$

Table 5.3 Estimating $p(\hat{y}|y)$ from a confusion matrix. Abbreviations: FNR = false negative rate, FPR = false positive rate, TNR = true negative rate, TPR = true positive rate.

$\mathbb{I}(f(\mathbf{x}) > \tau)$ be our decision rule, where $f(\mathbf{x})$ is a measure of confidence that $y = 1$ (this should be monotonically related to $p(y = 1|\mathbf{x})$, but does not need to be a probability), and τ is some threshold parameter. For each given value of τ , we can apply our decision rule and count the number of true positives, false positives, true negatives, and false negatives that occur, as shown in Table 5.2. This table of errors is called a **confusion matrix**.

From this table, we can compute the **true positive rate** (TPR), also known as the **sensitivity**, **recall** or **hit rate**, by using $TPR = TP/N_+ \approx p(\hat{y} = 1|y = 1)$. We can also compute the **false positive rate** (FPR), also called the **false alarm rate**, or the **type I error rate**, by using $FPR = FP/N_- \approx p(\hat{y} = 1|y = 0)$. These and other definitions are summarized in Tables 5.3 and 5.4. We can combine these errors in any way we choose to compute a loss function.

However, rather than than computing the TPR and FPR for a fixed threshold τ , we can run our detector for a set of thresholds, and then plot the TPR vs FPR as an implicit function of τ . This is called a **receiver operating characteristic** or **ROC curve**. See Figure 5.15(a) for an example. Any system can achieve the point on the bottom left, $(FPR = 0, TPR = 0)$, by setting $\tau = 1$ and thus classifying everything as negative; similarly any system can achieve the point on the top right, $(FPR = 1, TPR = 1)$, by setting $\tau = 0$ and thus classifying everything as positive. If a system is performing at chance level, then we can achieve any point on the diagonal line $TPR = FPR$ by choosing an appropriate threshold. A system that perfectly separates the positives from negatives has a threshold that can achieve the top left corner, $(FPR = 0, TPR = 1)$; by varying the threshold such a system will “hug” the left axis and then the top axis, as shown in Figure 5.15(a).

The quality of a ROC curve is often summarized as a single number using the **area under the curve** or **AUC**. Higher AUC scores are better; the maximum is obviously 1. Another summary statistic that is used is the **equal error rate** or **EER**, also called the **cross over rate**, defined as the value which satisfies $FPR = FNR$. Since $FNR = 1 - TPR$, we can compute the EER by drawing a line from the top left to the bottom right and seeing where it intersects the ROC curve (see points A and B in Figure 5.15(a)). Lower EER scores are better; the minimum is obviously 0.

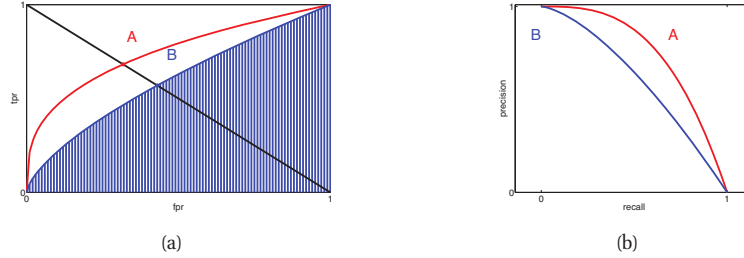


Figure 5.15 (a) ROC curves for two hypothetical classification systems. A is better than B. We plot the true positive rate (TPR) vs the false positive rate (FPR) as we vary the threshold τ . We also indicate the equal error rate (EER) with the red and blue dots, and the area under the curve (AUC) for classifier B. (b) A precision-recall curve for two hypothetical classification systems. A is better than B. Figure generated by PRhand.

	$y = 1$	$y = 0$
$\hat{y} = 1$	$TP/\hat{N}_+ = \text{precision} = \text{PPV}$	$FP/\hat{N}_+ = \text{FDP}$
$\hat{y} = 0$	$FN/\hat{N}_-$	$TN/\hat{N}_- = \text{NPV}$

Table 5.4 Estimating $p(y|\hat{y})$ from a confusion matrix. Abbreviations: FDP = false discovery probability, NPV = negative predictive value, PPV = positive predictive value,

5.7.2.2 Precision recall curves

When trying to detect a rare event (such as retrieving a relevant document or finding a face in an image), the number of negatives is very large. Hence comparing $TPR = TP/N_+$ to $FPR = FP/N_-$ is not very informative, since the FPR will be very small. Hence all the “action” in the ROC curve will occur on the extreme left. In such cases, it is common to plot the TPR versus the number of false positives, rather than vs the false positive rate.

However, in some cases, the very notion of “negative” is not well-defined. For example, when detecting objects in images (see Section 1.2.1.3), if the detector works by classifying patches, then the number of patches examined — and hence the number of true negatives — is a parameter of the algorithm, not part of the problem definition. So we would like to use a measure that only talks about positives.

The **precision** is defined as $TP/\hat{N}_+ = p(y = 1|\hat{y} = 1)$ and the **recall** is defined as $TP/N_+ = p(\hat{y} = 1|y = 1)$. Precision measures what fraction of our detections are actually positive, and recall measures what fraction of the positives we actually detected. If $\hat{y}_i \in \{0, 1\}$ is the predicted label, and $y_i \in \{0, 1\}$ is the true label, we can estimate precision and recall using

$$P = \frac{\sum_i y_i \hat{y}_i}{\sum_i \hat{y}_i}, \quad R = \frac{\sum_i y_i \hat{y}_i}{\sum_i y_i} \quad (5.115)$$

A **precision recall curve** is a plot of precision vs recall as we vary the threshold τ . See Figure 5.15(b). Hugging the top right is the best one can do.

This curve can be summarized as a single number using the mean precision (averaging over

	Class 1			Class 2			Pooled	
	$y = 1$	$y = 0$		$y = 1$	$y = 0$		$y = 1$	$y = 0$
$\hat{y} = 1$	10	10	$\hat{y} = 1$	90	10	$\hat{y} = 1$	100	20
$\hat{y} = 0$	10	970	$\hat{y} = 0$	10	890	$\hat{y} = 0$	20	1860

Table 5.5 Illustration of the difference between macro- and micro-averaging. y is the true label, and $\hat{y}$ is the called label. In this example, the macro-averaged precision is $[10/(10 + 10) + 90/(10 + 90)]/2 = (0.5 + 0.9)/2 = 0.7$. The micro-averaged precision is $100/(100 + 20) \approx 0.83$. Based on Table 13.7 of (Manning et al. 2008).

recall values), which approximates the area under the curve. Alternatively, one can quote the precision for a fixed recall level, such as the precision of the first $K = 10$ entities recalled. This is called the **average precision at K** score. This measure is widely used when evaluating information retrieval systems.

5.7.2.3 F-scores *

For a fixed threshold, one can compute a single precision and recall value. These are often combined into a single statistic called the **F score**, or **F1 score**, which is the harmonic mean of precision and recall:

$$F_1 \triangleq \frac{2}{1/P + 1/R} = \frac{2PR}{R + P} \quad (5.116)$$

Using Equation 5.115, we can write this as

$$F_1 = \frac{2 \sum_{i=1}^N y_i \hat{y}_i}{\sum_{i=1}^N y_i + \sum_{i=1}^N \hat{y}_i} \quad (5.117)$$

This is a widely used measure in information retrieval systems.

To understand why we use the harmonic mean instead of the arithmetic mean, $(P + R)/2$, consider the following scenario. Suppose we recall all entries, so $R = 1$. The precision will be given by the **prevalence**, $p(y = 1)$. Suppose the prevalence is low, say $p(y = 1) = 10^{-4}$. The arithmetic mean of P and R is given by $(P + R)/2 = (10^{-4} + 1)/2 \approx 50\%$. By contrast, the harmonic mean of this strategy is only $\frac{2 \times 10^{-4} \times 1}{1 + 10^{-4}} \approx 0.2\%$.

In the multi-class case (e.g., for document classification problems), there are two ways to generalize F_1 scores. The first is called **macro-averaged F1**, and is defined as $\sum_{c=1}^C F_1(c)/C$, where $F_1(c)$ is the F_1 score obtained on the task of distinguishing class c from all the others. The other is called **micro-averaged F1**, and is defined as the F_1 score where we pool all the counts from each class's contingency table.

Table 5.5 gives a worked example that illustrates the difference. We see that the precision of class 1 is 0.5, and of class 2 is 0.9. The macro-averaged precision is therefore 0.7, whereas the micro-averaged precision is 0.83. The latter is much closer to the precision of class 2 than to the precision of class 1, since class 2 is five times larger than class 1. To give equal weight to each class, use macro-averaging.

5.7.2.4 False discovery rates *

Suppose we are trying to discover a rare phenomenon using some kind of high throughput measurement device, such as a gene expression micro array, or a radio telescope. We will need to make many binary decisions of the form $p(y_i = 1|\mathcal{D}) > \tau$, where $\mathcal{D} = \{\mathbf{x}_i\}_{i=1}^N$ and N may be large. This is called **multiple hypothesis testing**. Note that the difference from standard binary classification is that we are classifying y_i based on all the data, not just based on $\mathbf{x}_i$. So this is a simultaneous classification problem, where we might hope to do better than a series of individual classification problems.

How should we set the threshold τ ? A natural approach is to try to minimize the expected number of false positives. In the Bayesian approach, this can be computed as follows:

$$FD(\tau, \mathcal{D}) \triangleq \sum_i \underbrace{(1 - p_i)}_{\text{pr. error}} \underbrace{\mathbb{I}(p_i > \tau)}_{\text{discovery}} \quad (5.118)$$

where $p_i \triangleq p(y_i = 1|\mathcal{D})$ is your belief that this object exhibits the phenomenon in question. We then define the posterior expected **false discovery rate** as follows:

$$FDR(\tau, \mathcal{D}) \triangleq FD(\tau, \mathcal{D})/N(\tau, \mathcal{D}) \quad (5.119)$$

where $N(\tau, \mathcal{D}) = \sum_i \mathbb{I}(p_i > \tau)$ is the number of discovered items. Given a desired FDR tolerance, say $\alpha = 0.05$, one can then adapt τ to achieve this; this is called the **direct posterior probability approach** to controlling the FDR (Newton et al. 2004; Muller et al. 2004).

In order to control the FDR it is very helpful to estimate the p_i 's jointly (e.g., using a hierarchical Bayesian model, as in Section 5.5), rather than independently. This allows the pooling of statistical strength, and thus lower FDR. See e.g., (Berry and Hochberg 1999) for more information.

5.7.3 Other topics *

In this section, we briefly mention a few other topics related to Bayesian decision theory. We do not have space to go into detail, but we include pointers to the relevant literature.

5.7.3.1 Contextual bandits

A **one-armed bandit** is a colloquial term for a slot machine, found in casinos around the world. The game is this: you insert some money, pull an arm, and wait for the machine to stop; if you're lucky, you win some money. Now imagine there is a bank of K such machines to choose from. Which one should you use? This is called a **multi-armed bandit**, and can be modeled using Bayesian decision theory: there are K possible actions, and each action has an unknown reward (payoff function) r_k . By maintaining a belief state, $p(r_{1:K}|\mathcal{D}) = \prod_k p(r_k|\mathcal{D})$, one can devise an optimal policy; this can be compiled into a series of **Gittins Indices** (Gittins 1989). This optimally solves the **exploration-exploitation** tradeoff, which specifies how many times one should try each action before deciding to go with the winner.

Now consider an extension where each arm, and the player, has an associated feature vector; call all these features $\mathbf{x}$. This is called a **contextual bandit** (see e.g., (Sarkar 1991; Scott 2010; Li et al. 2011)). For example, the “arms” could represent ads or news articles which we want to show to the user, and the features could represent properties of these ads or articles, such

as a bag of words, as well as properties of the user, such as demographics. If we assume a linear model for reward, $r_k = \theta_k^T \mathbf{x}$, we can maintain a distribution over the parameters of each arm, $p(\theta_k | \mathcal{D})$, where $\mathcal{D}$ is a series of tuples of the form $(a, \mathbf{x}, r)$, which specifies which arm was pulled, what its features were, and what the resulting outcome was (e.g., $r = 1$ if the user clicked on the ad, and $r = 0$ otherwise). We discuss ways to compute $p(\theta_k | \mathcal{D})$ from linear and logistic regression models in later chapters.

Given the posterior, we must decide what action to take. One common heuristic, known as **UCB** (which stands for “upper confidence bound”) is to take the action which maximizes

$$k^* = \operatorname{argmax}_{k=1}^K \mu_k + \lambda \sigma_k \quad (5.120)$$

where $\mu_k = \mathbb{E}[r_k | \mathcal{D}]$, $\sigma_k^2 = \operatorname{var}[r_k | \mathcal{D}]$ and λ is a tuning parameter that trades off exploration and exploitation. The intuition is that we should pick actions about which we believe are good (μ_k is large), and/ or actions about which we are uncertain (σ_k is large).

An even simpler method, known as **Thompson sampling**, is as follows. At each step, we pick action k with a probability that is equal to its probability of being the optimal action:

$$p_k = \int \mathbb{I}(\mathbb{E}[r | a, \mathbf{x}, \theta] = \max_{a'} \mathbb{E}[r | a', \mathbf{x}, \theta]) p(\theta | \mathcal{D}) d\theta \quad (5.121)$$

We can approximate this by drawing a single sample from the posterior, $\theta^t \sim p(\theta | \mathcal{D})$, and then choosing $k^* = \operatorname{argmax}_k \mathbb{E}[r | \mathbf{x}, k, \theta^t]$. Despite its simplicity, this has been shown to work quite well (Chapelle and Li 2011).

5.7.3.2 Utility theory

Suppose we are a doctor trying to decide whether to operate on a patient or not. We imagine there are 3 states of nature: the patient has no cancer, the patient has lung cancer, or the patient has breast cancer. Since the action and state space is discrete, we can represent the loss function $L(\theta, a)$ as a **loss matrix**, such as the following:

	Surgery	No surgery
No cancer	20	0
Lung cancer	10	50
Breast cancer	10	60

These numbers reflects the fact that not performing surgery when the patient has cancer is very bad (loss of 50 or 60, depending on the type of cancer), since the patient might die; not performing surgery when the patient does not have cancer incurs no loss (0); performing surgery when the patient does not have cancer is wasteful (loss of 20); and performing surgery when the patient does have cancer is painful but necessary (10).

It is natural to ask where these numbers come from. Ultimately they represent the personal **preferences** or values of a fictitious doctor, and are somewhat arbitrary: just as some people prefer chocolate ice cream and others prefer vanilla, there is no such thing as the “right” loss/ utility function. However, it can be shown (see e.g., (DeGroot 1970)) that any set of consistent preferences can be converted to a scalar loss/ utility function. Note that utility can be measured on an arbitrary scale, such as dollars, since it is only relative values that matter.⁶

6. People are often squeamish about talking about human lives in monetary terms, but all decision making requires

5.7.3.3 Sequential decision theory

So far, we have concentrated on **one-shot decision problems**, where we only have to make one decision and then the game ends. In Section 10.6, we will generalize this to multi-stage or sequential decision problems. Such problems frequently arise in many business and engineering settings. This is closely related to the problem of reinforcement learning. However, further discussion of this point is beyond the scope of this book.

Exercises

Exercise 5.1 Proof that a mixture of conjugate priors is indeed conjugate
Derive Equation 5.69.

Exercise 5.2 Optimal threshold on classification probability

Consider a case where we have learned a conditional probability distribution $P(y|\mathbf{x})$. Suppose there are only two classes, and let $p_0 = P(Y = 0|\mathbf{x})$ and $p_1 = P(Y = 1|\mathbf{x})$. Consider the loss matrix below:

predicted label $\hat{y}$	true label y	
	0	1
0	0	λ_{01}
1	λ_{10}	0

- Show that the decision $\hat{y}$ that minimizes the expected loss is equivalent to setting a probability threshold θ and predicting $\hat{y} = 0$ if $p_1 < \theta$ and $\hat{y} = 1$ if $p_1 \geq \theta$. What is θ as a function of λ_{01} and λ_{10} ? (Show your work.)
- Show a loss matrix where the threshold is 0.1. (Show your work.)

Exercise 5.3 Reject option in classifiers

(Source: (Duda et al. 2001, Q2.13).)

In many classification problems one has the option either of assigning $\mathbf{x}$ to class j or, if you are too uncertain, of choosing the **reject option**. If the cost for rejects is less than the cost of falsely classifying the object, it may be the optimal action. Let α_i mean you choose action i , for $i = 1 : C + 1$, where C is the number of classes and $C + 1$ is the reject action. Let $Y = j$ be the true (but unknown) **state of nature**. Define the loss function as follows

$$\lambda(\alpha_i|Y = j) = \begin{cases} 0 & \text{if } i = j \text{ and } i, j \in \{1, \dots, C\} \\ \lambda_r & \text{if } i = C + 1 \\ \lambda_s & \text{otherwise} \end{cases} \quad (5.122)$$

In otherwords, you incur 0 loss if you correctly classify, you incur λ_r loss (cost) if you choose the reject option, and you incur λ_s loss (cost) if you make a substitution error (misclassification).

tradeoffs, and one needs to use some kind of “currency” to compare different courses of action. Insurance companies do this all the time. Ross Schachter, a decision theorist at Stanford University, likes to tell a story of a school board who rejected a study on asbestos removal from schools because it performed a **cost-benefit analysis**, which was considered “inhumane” because they put a dollar value on children’s health; the result of rejecting the report was that the asbestos was not removed, which is surely more “inhumane”. In medical domains, one often measures utility in terms of **QALY**, or quality-adjusted life-years, instead of dollars, but it’s the same idea. Of course, even if you do not explicitly specify how much you value different people’s lives, your *behavior* will reveal your implicit values/ preferences, and these preferences can then be converted to a real-valued scale, such as dollars or QALY. Inferring a utility function from behavior is called **inverse reinforcement learning**.

Decision $\hat{y}$	true label y	
	0	1
predict 0	0	10
predict 1	10	0
reject	3	3

- Show that the minimum risk is obtained if we decide $Y = j$ if $p(Y = j|\mathbf{x}) \geq p(Y = k|\mathbf{x})$ for all k (i.e., j is the most probable class) and if $p(Y = j|\mathbf{x}) \geq 1 - \frac{\lambda_r}{\lambda_s}$; otherwise we decide to reject.
- Describe qualitatively what happens as λ_r/λ_s is increased from 0 to 1 (i.e., the relative cost of rejection increases).

Exercise 5.4 More reject options

In many applications, the classifier is allowed to “reject” a test example rather than classifying it into one of the classes. Consider, for example, a case in which the cost of a misclassification is \$10 but the cost of having a human manually make the decision is only \$3. We can formulate this as the following loss matrix:

- Suppose $P(y = 1|\mathbf{x})$ is predicted to be 0.2. Which decision minimizes the expected loss?
- Now suppose $P(y = 1|\mathbf{x}) = 0.4$. Now which decision minimizes the expected loss?
- Show that in general, for this loss matrix, but for any posterior distribution, there will be two thresholds θ_0 and θ_1 such that the optimal decision is to predict 0 if $p_1 < \theta_0$, reject if $\theta_0 \leq p_1 \leq \theta_1$, and predict 1 if $p_1 > \theta_1$ (where $p_1 = p(y = 1|\mathbf{x})$). What are these thresholds?

Exercise 5.5 Newsvendor problem

Consider the following classic problem in decision theory/ economics. Suppose you are trying to decide how much quantity Q of some product (e.g., newspapers) to buy to maximize your profits. The optimal amount will depend on how much demand D you think there is for your product, as well as its cost to you C and its selling price P . Suppose D is unknown but has pdf $f(D)$ and cdf $F(D)$. We can evaluate the expected profit by considering two cases: if $D > Q$, then we sell all Q items, and make profit $\pi = (P - C)Q$; but if $D < Q$, we only sell D items, at profit $(P - C)D$, but have wasted $C(Q - D)$ on the unsold items. So the expected profit if we buy quantity Q is

$$E\pi(Q) = \int_Q^\infty (P - C)Q f(D) dD + \int_0^Q (P - C)D f(D) dD - \int_0^Q C(Q - D) f(D) dD \quad (5.123)$$

Simplify this expression, and then take derivatives wrt Q to show that the optimal quantity Q^* (which maximizes the expected profit) satisfies

$$F(Q^*) = \frac{P - C}{P} \quad (5.124)$$

Exercise 5.6 Bayes factors and ROC curves

Let $B = p(D|H_1)/p(D|H_0)$ be the bayes factor in favor of model 1. Suppose we plot two ROC curves, one computed by thresholding B , and the other computed by thresholding $p(H_1|D)$. Will they be the same or different? Explain why.

Exercise 5.7 Bayes model averaging helps predictive accuracy

Let Δ be a quantity that we want to predict, let $\mathcal{D}$ be the observed data and $\mathcal{M}$ be a finite set of models. Suppose our action is to provide a probabilistic prediction $p()$, and the loss function is $L(\Delta, p()) =$

$-\log p(\Delta)$. We can either perform Bayes model averaging and predict using

$$p^{BMA}(\Delta) = \sum_{m \in \mathcal{M}} p(\Delta|m, \mathcal{D})p(m|\mathcal{D}) \quad (5.125)$$

or we could predict using any single model (a plugin approximation)

$$p^m(\Delta) = p(\Delta|m, \mathcal{D}) \quad (5.126)$$

Show that, for all models $m \in \mathcal{M}$, the posterior expected loss using BMA is lower, i.e.,

$$\mathbb{E} [L(\Delta, p^{BMA})] \leq \mathbb{E} [L(\Delta, p^m)] \quad (5.127)$$

where the expectation over Δ is with respect to

$$p(\Delta|\mathcal{D}) = \sum_{m \in \mathcal{M}} p(\Delta|m, \mathcal{D})p(m|\mathcal{D}) \quad (5.128)$$

Hint: use the non-negativity of the KL divergence.

Exercise 5.8 MLE and model selection for a 2d discrete distribution

(Source: Jaakkola.)

Let $x \in \{0, 1\}$ denote the result of a coin toss ($x = 0$ for tails, $x = 1$ for heads). The coin is potentially biased, so that heads occurs with probability θ_1 . Suppose that someone else observes the coin flip and reports to you the outcome, y . But this person is unreliable and only reports the result correctly with probability θ_2 ; i.e., $p(y|x, \theta_2)$ is given by

	$y = 0$	$y = 1$
$x = 0$	θ_2	$1 - \theta_2$
$x = 1$	$1 - \theta_2$	θ_2

Assume that θ_2 is independent of x and θ_1 .

- Write down the joint probability distribution $p(x, y|\theta)$ as a 2×2 table, in terms of $\theta = (\theta_1, \theta_2)$.
- Suppose have the following dataset: $\mathbf{x} = (1, 1, 0, 1, 1, 0, 0)$, $\mathbf{y} = (1, 0, 0, 0, 1, 0, 1)$. What are the MLEs for θ_1 and θ_2 ? Justify your answer. Hint: note that the likelihood function factorizes,

$$p(x, y|\theta) = p(y|x, \theta_2)p(x|\theta_1) \quad (5.129)$$

What is $p(\mathcal{D}|\hat{\theta}, M_2)$ where M_2 denotes this 2-parameter model? (You may leave your answer in fractional form if you wish.)

- Now consider a model with 4 parameters, $\theta = (\theta_{0,0}, \theta_{0,1}, \theta_{1,0}, \theta_{1,1})$, representing $p(x, y|\theta) = \theta_{x,y}$. (Only 3 of these parameters are free to vary, since they must sum to one.) What is the MLE of θ ? What is $p(\mathcal{D}|\hat{\theta}, M_4)$ where M_4 denotes this 4-parameter model?
- Suppose we are not sure which model is correct. We compute the leave-one-out cross validated log likelihood of the 2-parameter model and the 4-parameter model as follows:

$$L(m) = \sum_{i=1}^n \log p(x_i, y_i|m, \hat{\theta}(\mathcal{D}_{-i})) \quad (5.130)$$

and $\hat{\theta}(\mathcal{D}_{-i})$ denotes the MLE computed on $\mathcal{D}$ excluding row i . Which model will CV pick and why? Hint: notice how the table of counts changes when you omit each training case one at a time.

- e. Recall that an alternative to CV is to use the BIC score, defined as

$$\text{BIC}(M, \mathcal{D}) \triangleq \log p(\mathcal{D} | \hat{\boldsymbol{\theta}}_{MLE}) - \frac{\text{dof}(M)}{2} \log N \quad (5.131)$$

where $\text{dof}(M)$ is the number of free parameters in the model, Compute the BIC scores for both models (use log base e). Which model does BIC prefer?

Exercise 5.9 Posterior median is optimal estimate under L1 loss

Prove that the posterior median is optimal estimate under L1 loss.

Exercise 5.10 Decision rule for trading off FPs and FNs

If $L_{FN} = cL_{FP}$, show that we should pick $\hat{y} = 1$ iff $p(y = 1 | \mathbf{x}) / p(y = 0 | \mathbf{x}) > \tau$, where $\tau = c / (1 + c)$

6

Frequentist statistics

6.1 Introduction

The approach to statistical inference that we described in Chapter 5 is known as Bayesian statistics. Perhaps surprisingly, this is considered controversial by some people, whereas the application of Bayes rule to non-statistical problems — such as medical diagnosis (Section 2.2.3.1), spam filtering (Section 3.4.4.1), or airplane tracking (Section 18.2.1) — is not controversial. The reason for the objection has to do with a misguided distinction between parameters of a statistical model and other kinds of unknown quantities.¹

Attempts have been made to devise approaches to statistical inference that avoid treating parameters like random variables, and which thus avoid the use of priors and Bayes rule. Such approaches are known as **frequentist statistics**, **classical statistics** or **orthodox statistics**. Instead of being based on the posterior distribution, they are based on the concept of a sampling distribution. This is the distribution that an estimator has when applied to multiple data sets sampled from the true but unknown distribution; see Section 6.2 for details. It is this notion of variation across repeated trials that forms the basis for modeling uncertainty used by the frequentist approach.

By contrast, in the Bayesian approach, we only ever condition on the actually observed data; there is no notion of repeated trials. This allows the Bayesian to compute the probability of one-off events, as we discussed in Section 2.1. Perhaps more importantly, the Bayesian approach avoids certain paradoxes that plague the frequentist approach (see Section 6.6). Nevertheless, it is important to be familiar with frequentist statistics (especially Section 6.5), since it is widely used in machine learning.

6.2 Sampling distribution of an estimator

In frequentist statistics, a parameter estimate $\hat{\theta}$ is computed by applying an **estimator** δ to some data $\mathcal{D}$, so $\hat{\theta} = \delta(\mathcal{D})$. The parameter is viewed as fixed and the data as random, which is the exact opposite of the Bayesian approach. The uncertainty in the parameter estimate can be measured by computing the **sampling distribution** of the estimator. To understand this

1. Parameters are sometimes considered to represent true (but unknown) physical quantities, which are therefore not random. However, we have seen that it is perfectly reasonable to use a probability distribution to represent one's uncertainty about an unknown constant.

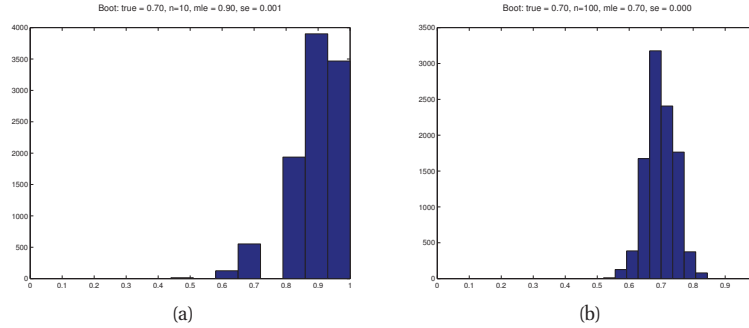


Figure 6.1 A bootstrap approximation to the sampling distribution of $\hat{\theta}$ for a Bernoulli distribution. We use $B = 10,000$ bootstrap samples. The N datascases were generated from $\text{Ber}(\theta = 0.7)$. (a) MLE with $N = 10$. (b) MLE with $N = 100$. Figure generated by `bootstrapDemoBer`.

concept, imagine sampling many different data sets $\mathcal{D}^{(s)}$ from some true model, $p(\cdot|\theta^*)$, i.e., let $\mathcal{D}^{(s)} = \{x_i^{(s)}\}_{i=1}^N$, where $x_i^s \sim p(\cdot|\theta^*)$, and θ^* is the true parameter. Here $s = 1 : S$ indexes the sampled data set, and N is the size of each such dataset. Now apply the estimator $\hat{\theta}(\cdot)$ to each $\mathcal{D}^{(s)}$ to get a set of estimates, $\{\hat{\theta}(\mathcal{D}^{(s)})\}$. As we let $S \rightarrow \infty$, the distribution induced on $\hat{\theta}(\cdot)$ is the sampling distribution of the estimator. We will discuss various ways to use the sampling distribution in later sections. But first we sketch two approaches for computing the sampling distribution itself.

6.2.1 Bootstrap

The **bootstrap** is a simple Monte Carlo technique to approximate the sampling distribution. This is particularly useful in cases where the estimator is a complex function of the true parameters.

The idea is simple. If we knew the true parameters θ^* , we could generate many (say S) fake datasets, each of size N , from the true distribution, $x_i^s \sim p(\cdot|\theta^*)$, for $s = 1 : S$, $i = 1 : N$. We could then compute our estimator from each sample, $\hat{\theta}^s = f(x_{1:N}^s)$ and use the empirical distribution of the resulting samples as our estimate of the sampling distribution. Since θ is unknown, the idea of the **parametric bootstrap** is to generate the samples using $\hat{\theta}(\mathcal{D})$ instead. An alternative, called the **non-parametric bootstrap**, is to sample the x_i^s (with replacement) from the original data $\mathcal{D}$, and then compute the induced distribution as before. Some methods for speeding up the bootstrap when applied to massive data sets are discussed in (Kleiner et al. 2011).

Figure 6.1 shows an example where we compute the sampling distribution of the MLE for a Bernoulli using the parametric bootstrap. (Results using the non-parametric bootstrap are essentially the same.) We see that the sampling distribution is asymmetric, and therefore quite far from Gaussian, when $N = 10$; when $N = 100$, the distribution looks more Gaussian, as theory suggests (see below).

A natural question is: what is the connection between the parameter estimates $\hat{\theta}^s = \hat{\theta}(x_{1:N}^s)$ computed by the bootstrap and parameter values sampled from the posterior, $\theta^s \sim p(\cdot|\mathcal{D})$?

Conceptually they are quite different. But in the common case that the prior is not very strong, they can be quite similar. For example, Figure 6.1(c-d) shows an example where we compute the posterior using a uniform Beta(1,1) prior, and then sample from it. We see that the posterior and the sampling distribution are quite similar. So one can think of the bootstrap distribution as a “poor man’s” posterior; see (Hastie et al. 2001, p235) for details.

However, perhaps surprisingly, bootstrap can be slower than posterior sampling. The reason is that the bootstrap has to fit the model S times, whereas in posterior sampling, we usually only fit the model once (to find a local mode), and then perform local exploration around the mode. Such local exploration is usually much faster than fitting a model from scratch.

6.2.2 Large sample theory for the MLE *

In some cases, the sampling distribution for some estimators can be computed analytically. In particular, it can be shown that, under certain conditions, as the sample size tends to infinity, the sampling distribution of the MLE becomes Gaussian. Informally, the requirement for this result to hold is that each parameter in the model gets to “see” an infinite amount of data, and that the model be identifiable. Unfortunately this excludes many of the models of interest to machine learning. Nevertheless, let us assume we are in a simple setting where the theorem holds.

The center of the Gaussian will be the MLE $\hat{\theta}$. But what about the variance of this Gaussian? Intuitively the variance of the estimator will be (inversely) related to the amount of curvature of the likelihood surface at its peak. If the curvature is large, the peak will be “sharp”, and the variance low; in this case, the estimate is “well determined”. By contrast, if the curvature is small, the peak will be nearly “flat”, so the variance is high.

Let us now formalize this intuition. Define the **score function** as the gradient of the log likelihood evaluated at some point $\hat{\theta}$:

$$\mathbf{s}(\hat{\theta}) \triangleq \nabla \log p(\mathcal{D}|\theta)|_{\hat{\theta}} \quad (6.1)$$

Define the **observed information matrix** as the gradient of the negative score function, or equivalently, the Hessian of the NLL:

$$\mathbf{J}(\hat{\theta}(\mathcal{D})) \triangleq -\nabla \mathbf{s}(\hat{\theta}) = -\nabla_{\theta}^2 \log p(\mathcal{D}|\theta)|_{\hat{\theta}} \quad (6.2)$$

In 1D, this becomes

$$J(\hat{\theta}(\mathcal{D})) = -\frac{d}{d\theta^2} \log p(\mathcal{D}|\theta)|_{\hat{\theta}} \quad (6.3)$$

This is just a measure of curvature of the log-likelihood function at $\hat{\theta}$.

Since we are studying the sampling distribution, $\mathcal{D} = (\mathbf{x}_1, \dots, \mathbf{x}_N)$ is a set of random variables. The **Fisher information matrix** is defined to be the expected value of the observed information matrix:²

$$\mathbf{I}_N(\hat{\theta}|\theta^*) \triangleq \mathbb{E}_{\theta^*} [\mathbf{J}(\hat{\theta}|\mathcal{D})] \quad (6.4)$$

2. This is not the usual definition, but is equivalent to it under standard assumptions. More precisely, the standard definition is as follows (we just give the scalar case to simplify notation): $I(\hat{\theta}|\theta^*) \triangleq \text{var}_{\theta^*} \left[\frac{d}{d\theta} \log p(X|\theta)|_{\hat{\theta}} \right]$, that is, the variance of the score function. If $\hat{\theta}$ is the MLE, it is easy to see that $\mathbb{E}_{\theta^*} \left[\frac{d}{d\theta} \log p(X|\theta)|_{\hat{\theta}} \right] = 0$ (since

where $\mathbb{E}_{\theta^*} [\mathbf{f}(\mathcal{D})] \triangleq \frac{1}{N} \sum_{i=1}^N \mathbf{f}(\mathbf{x}_i) p(\mathbf{x}_i | \theta^*)$ is the expected value of the function $\mathbf{f}$ when applied to data sampled from θ^* . Often θ^* , representing the “true parameter” that generated the data, is assumed known, so we just write $\mathbf{I}_N(\hat{\theta}) \triangleq \mathbf{I}_N(\hat{\theta} | \theta^*)$ for short. Furthermore, it is easy to see that $\mathbf{I}_N(\hat{\theta}) = N \mathbf{I}_1(\hat{\theta})$, because the log-likelihood for a sample of size N is just N times “steeper” than the log-likelihood for a sample of size 1. So we can drop the 1 subscript and just write $\mathbf{I}(\hat{\theta}) \triangleq \mathbf{I}_1(\hat{\theta})$. This is the notation that is usually used.

Now let $\hat{\theta} \triangleq \hat{\theta}_{mle}(\mathcal{D})$ be the MLE, where $\mathcal{D} \sim \theta^*$. It can be shown that

$$\hat{\theta} \rightarrow \mathcal{N}(\theta^*, \mathbf{I}_N(\theta^*)^{-1}) \quad (6.5)$$

as $N \rightarrow \infty$ (see e.g., (Rice 1995, p265) for a proof). We say that the sampling distribution of the MLE is **asymptotically normal**.

What about the variance of the MLE, which can be used as some measure of confidence in the MLE? Unfortunately, θ^* is unknown, so we can’t evaluate the variance of the sampling distribution. However, we can approximate the sampling distribution by replacing θ^* with $\hat{\theta}$. Consequently, the approximate **standard errors** of $\hat{\theta}_k$ are given by

$$\hat{se}_k \triangleq \mathbf{I}_N(\hat{\theta})_{kk}^{-\frac{1}{2}} \quad (6.6)$$

For example, from Equation 5.60 we know that the Fisher information for a binomial sampling model is

$$I(\theta) = \frac{1}{\theta(1-\theta)} \quad (6.7)$$

So the approximate standard error of the MLE is

$$\hat{se} = \frac{1}{\sqrt{I_N(\hat{\theta})}} = \frac{1}{\sqrt{NI(\hat{\theta})}} = \left(\frac{\hat{\theta}(1-\hat{\theta})}{N} \right)^{\frac{1}{2}} \quad (6.8)$$

where $\hat{\theta} = \frac{1}{N} \sum_i X_i$. Compare this to Equation 3.27, which is the posterior standard deviation under a uniform prior.

6.3 Frequentist decision theory

In frequentist or classical decision theory, there is a loss function and a likelihood, but there is no prior and hence no posterior or posterior expected loss. Thus there is no automatic way of deriving an optimal estimator, unlike the Bayesian case. Instead, in the frequentist approach, we are free to choose any estimator or decision procedure $\delta : \mathcal{X} \rightarrow \mathcal{A}$ we want.³

the gradient must be zero at a maximum), so the variance reduces to the expected square of the score function: $I(\hat{\theta} | \theta^*) = \mathbb{E}_{\theta^*} \left[\left(\frac{d}{d\theta} \log p(X | \theta) \right)^2 \right]$. It can be shown (e.g., (Rice 1995, p263)) that $\mathbb{E}_{\theta^*} \left[\left(\frac{d}{d\theta} \log p(X | \theta) \right)^2 \right] = -\mathbb{E}_{\theta^*} \left[\frac{d^2}{d\theta^2} \log p(X | \theta) \right]$, so now the Fisher information reduces to the expected second derivative of the NLL, which is a much more intuitive quantity than the variance of the score.

3. In practice, the frequentist approach is usually only applied to one-shot statistical decision problems — such as classification, regression and parameter estimation — since its non-constructive nature makes it difficult to apply to sequential decision problems, which adapt to data online.

Having chosen an estimator, we define its expected loss or **risk** as follows:

$$R(\theta^*, \delta) \triangleq \mathbb{E}_{p(\tilde{\mathcal{D}}|\theta^*)} [L(\theta^*, \delta(\tilde{\mathcal{D}}))] = \int L(\theta^*, \delta(\tilde{\mathcal{D}})) p(\tilde{\mathcal{D}}|\theta^*) d\tilde{\mathcal{D}} \quad (6.9)$$

where $\tilde{\mathcal{D}}$ is data sampled from “nature’s distribution”, which is represented by parameter θ^* . In other words, the expectation is wrt the sampling distribution of the estimator. Compare this to the Bayesian posterior expected loss:

$$\rho(a|\mathcal{D}, \pi) \triangleq \mathbb{E}_{p(\theta|\mathcal{D}, \pi)} [L(\theta, a)] = \int_{\Theta} L(\theta, a) p(\theta|\mathcal{D}, \pi) d\theta \quad (6.10)$$

We see that the Bayesian approach averages over θ (which is unknown) and conditions on $\mathcal{D}$ (which is known), whereas the frequentist approach averages over $\tilde{\mathcal{D}}$ (thus ignoring the observed data), and conditions on θ^* (which is unknown).

Not only is the frequentist definition unnatural, it cannot even be computed, because θ^* is unknown. Consequently, we cannot compare different estimators in terms of their frequentist risk. We discuss various solutions to this below.

6.3.1 Bayes risk

How do we choose amongst estimators? We need some way to convert $R(\theta^*, \delta)$ into a single measure of quality, $R(\delta)$, which does not depend on knowing θ^* . One approach is to put a prior on θ^* , and then to define **Bayes risk** or **integrated risk** of an estimator as follows:

$$R_B(\delta) \triangleq \mathbb{E}_{p(\theta^*)} [R(\theta^*, \delta)] = \int R(\theta^*, \delta) p(\theta^*) d\theta^* \quad (6.11)$$

A **Bayes estimator** or **Bayes decision rule** is one which minimizes the expected risk:

$$\delta_B \triangleq \underset{\delta}{\operatorname{argmin}} R_B(\delta) \quad (6.12)$$

Note that the integrated risk is also called the **preposterior risk**, since it is before we have seen the data. Minimizing this can be useful for experiment design.

We will now prove a very important theorem, that connects the Bayesian and frequentist approaches to decision theory.

Theorem 6.3.1. *A Bayes estimator can be obtained by minimizing the posterior expected loss for each $\mathbf{x}$.*

Proof. By switching the order of integration, we have

$$R_B(\delta) = \int \left[\sum_{\mathbf{x}} \sum_y L(y, \delta(\mathbf{x})) p(\mathbf{x}, y|\theta^*) \right] p(\theta^*) d\theta^* \quad (6.13)$$

$$= \sum_{\mathbf{x}} \sum_y \int_{\Theta} L(y, \delta(\mathbf{x})) p(\mathbf{x}, y, \theta^*) d\theta^* \quad (6.14)$$

$$= \sum_{\mathbf{x}} \left[\sum_y L(y, \delta(\mathbf{x})) p(y|\mathbf{x}) dy \right] p(\mathbf{x}) \quad (6.15)$$

$$= \sum_{\mathbf{x}} \rho(\delta(\mathbf{x})|\mathbf{x}) p(\mathbf{x}) \quad (6.16)$$

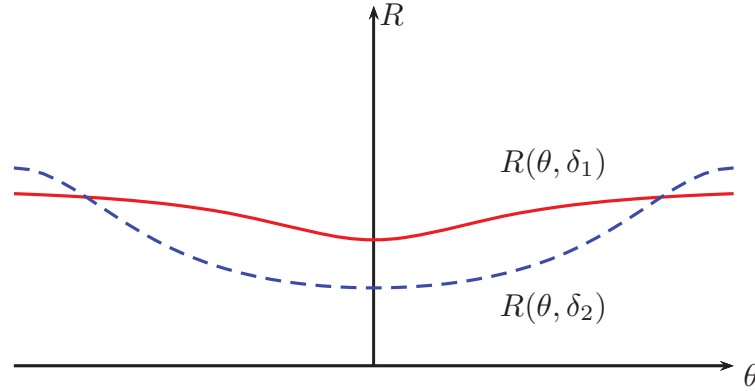


Figure 6.2 Risk functions for two decision procedures, δ_1 and δ_2 . Since δ_1 has lower worst case risk, it is the minimax estimator, even though δ_2 has lower risk for most values of θ . Thus minimax estimators are overly conservative.

To minimize the overall expectation, we just minimize the term inside for each $\mathbf{x}$, so our decision rule is to pick

$$\delta_B(\mathbf{x}) = \operatorname{argmin}_{a \in \mathcal{A}} \rho(a|\mathbf{x}) \quad (6.17)$$

□

Hence we see that the picking the optimal action on a case-by-case basis (as in the Bayesian approach) is optimal on average (as in the frequentist approach). In other words, the Bayesian approach provides a good way of achieving frequentist goals. In fact, one can go further and prove the following.

Theorem 6.3.2 (Wald, 1950). *Every admissible decision rule is a Bayes decision rule with respect to some, possibly improper, prior distribution.*

This theorem shows that *the best way to minimize frequentist risk is to be Bayesian!* See (Bernardo and Smith 1994, p448) for further discussion of this point.

6.3.2 Minimax risk

Obviously some frequentists dislike using Bayes risk since it requires the choice of a prior (although this is only in the evaluation of the estimator, not necessarily as part of its construction). An alternative approach is as follows. Define the **maximum risk** of an estimator as

$$R_{max}(\delta) \triangleq \max_{\theta^*} R(\theta^*, \delta) \quad (6.18)$$

A **minimax rule** is one which minimizes the maximum risk:

$$\delta_{MM} \triangleq \operatorname{argmin}_{\delta} R_{max}(\delta) \quad (6.19)$$

For example, in Figure 6.2, we see that δ_1 has lower worst-case risk than δ_2 , ranging over all possible values of θ^* , so it is the minimax estimator (see Section 6.3.3.1 for an explanation of how to compute a risk function for an actual model).

Minimax estimators have a certain appeal. However, computing them can be hard. And furthermore, they are very pessimistic. In fact, one can show that all minimax estimators are equivalent to Bayes estimators under a **least favorable prior**. In most statistical situations (excluding game theoretic ones), assuming nature is an adversary is not a reasonable assumption.

6.3.3 Admissible estimators

The basic problem with frequentist decision theory is that it relies on knowing the true distribution $p(\cdot|\theta^*)$ in order to evaluate the risk. However, It might be the case that some estimators are worse than others regardless of the value of θ^* . In particular, if $R(\theta, \delta_1) \leq R(\theta, \delta_2)$ for all $\theta \in \Theta$, then we say that δ_1 **dominates** δ_2 . The domination is said to be strict if the inequality is strict for some θ . An estimator is said to be **admissible** if it is not strictly dominated by any other estimator.

6.3.3.1 Example

Let us give an example, based on (Bernardo and Smith 1994). Consider the problem of estimating the mean of a Gaussian. We assume the data is sampled from $x_i \sim \mathcal{N}(\theta^*, \sigma^2 = 1)$ and use quadratic loss, $L(\theta, \hat{\theta}) = (\theta - \hat{\theta})^2$. The corresponding risk function is the MSE. Some possible decision rules or estimators $\hat{\theta}(\mathbf{x}) = \delta(\mathbf{x})$ are as follows:

- $\delta_1(\mathbf{x}) = \bar{x}$, the sample mean
- $\delta_2(\mathbf{x}) = \tilde{x}$, the sample median
- $\delta_3(\mathbf{x}) = \theta_0$, a fixed value
- $\delta_\kappa(\mathbf{x})$, the posterior mean under a $\mathcal{N}(\theta|\theta_0, \sigma^2/\kappa)$ prior:

$$\delta_\kappa(\mathbf{x}) = \frac{N}{N + \kappa} \bar{x} + \frac{\kappa}{N + \kappa} \theta_0 = w\bar{x} + (1 - w)\theta_0 \quad (6.20)$$

For δ_κ , we consider a weak prior, $\kappa = 1$, and a stronger prior, $\kappa = 5$. The prior mean is θ_0 , some fixed value. We assume σ^2 is known. (Thus $\delta_3(\mathbf{x})$ is the same as $\delta_\kappa(\mathbf{x})$ with an infinitely strong prior, $\kappa = \infty$.)

Let us now derive the risk functions analytically. (We can do this since in this toy example, we know the true parameter θ^* .) In Section 6.4.4, we show that the MSE can be decomposed into squared bias plus variance:

$$MSE(\hat{\theta}(\cdot)|\theta^*) = \text{var}[\hat{\theta}] + \text{bias}^2(\hat{\theta}) \quad (6.21)$$

The sample mean is unbiased, so its risk is

$$MSE(\delta_1|\theta^*) = \text{var}[\bar{x}] = \frac{\sigma^2}{N} \quad (6.22)$$

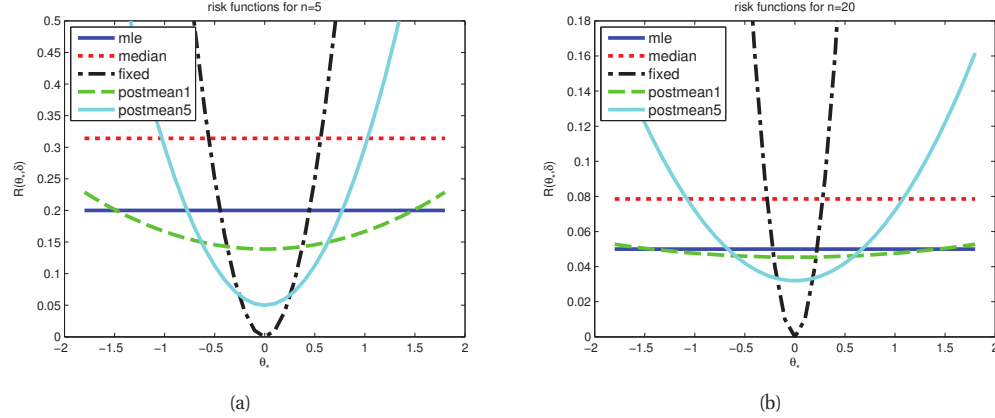


Figure 6.3 Risk functions for estimating the mean of a Gaussian using data sampled $\mathcal{N}(\theta^*, \sigma^2 = 1)$. The solid dark blue horizontal line is the MLE, the solid light blue curved line is the posterior mean when $\kappa = 5$. Left: $N = 5$ samples. Right: $N = 20$ samples. Based on Figure B.1 of (Bernardo and Smith 1994). Figure generated by `riskFnGauss`.

The sample median is also unbiased. One can show that the variance is approximately $\pi/(2N)$, so

$$MSE(\delta_2|\theta^*) = \frac{\pi}{2N} \quad (6.23)$$

For $\delta_3(\mathbf{x}) = \theta_0$, the variance is zero, so

$$MSE(\delta_3|\theta^*) = (\theta^* - \theta_0)^2 \quad (6.24)$$

Finally, for the posterior mean, we have

$$MSE(\delta_\kappa|\theta^*) = \mathbb{E} \left[(w\bar{x} + (1-w)\theta_0 - \theta^*)^2 \right] \quad (6.25)$$

$$= \mathbb{E} \left[(w(\bar{x} - \theta^*) + (1-w)(\theta_0 - \theta^*))^2 \right] \quad (6.26)$$

$$= w^2 \frac{\sigma^2}{N} + (1-w)^2 (\theta_0 - \theta^*)^2 \quad (6.27)$$

$$= \frac{1}{(N + \kappa)^2} (N\sigma^2 + \kappa^2(\theta_0 - \theta^*)^2) \quad (6.28)$$

These functions are plotted in Figure 6.3 for $N \in \{5, 20\}$. We see that in general, the best estimator depends on the value of θ^* , which is unknown. If θ^* is very close to θ_0 , then δ_3 (which just predicts θ_0) is best. If θ^* is within some reasonable range around θ_0 , then the posterior mean, which combines the prior guess of θ_0 with the actual data, is best. If θ^* is far from θ_0 , the MLE is best. None of this should be surprising: a small amount of shrinkage (using the posterior mean with a weak prior) is usually desirable, assuming our prior mean is sensible.

What is more surprising is that the risk of decision rule δ_2 (sample median) is always higher than that of δ_1 (sample mean) for every value of θ^* . Consequently the sample median is an

inadmissible estimator for this particular problem (where the data is assumed to come from a Gaussian).

In practice, the sample median is often better than the sample mean, because it is more robust to outliers. One can show (Minka 2000d) that the median is the Bayes estimator (under squared loss) if we assume the data comes from a Laplace distribution, which has heavier tails than a Gaussian. More generally, we can construct robust estimators by using flexible models of our data, such as mixture models or non-parametric density estimators (Section 14.7.2), and then computing the posterior mean or median.

6.3.3.2 Stein's paradox *

Suppose we have N iid random variables $X_i \sim \mathcal{N}(\theta_i, 1)$, and we want to estimate the θ_i . The obvious estimator is the MLE, which in this case sets $\hat{\theta}_i = x_i$. It turns out that this is an inadmissible estimator under quadratic loss, when $N \geq 4$.

To show this, it suffices to construct an estimator that is better. The James-Stein estimator is one such estimator, and is defined as follows:

$$\hat{\theta}_i = \hat{B}\bar{x} + (1 - \hat{B})x_i = \bar{x} + (1 - \hat{B})(x_i - \bar{x}) \quad (6.29)$$

where $\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$ and $0 < B < 1$ is some tuning constant. This estimate “shrinks” the θ_i towards the overall mean. (We derive this estimator using an empirical Bayes approach in Section 5.6.2.)

It can be shown that this shrinkage estimator has lower frequentist risk (MSE) than the MLE (sample mean) for $N \geq 4$. This is known as **Stein's paradox**. The reason it is called a paradox is illustrated by the following example. Suppose θ_i is the “true” IQ of student i and X_i is his test score. Why should my estimate of θ_i depend on the global mean $\bar{x}$, and hence on some other student's scores? One can create even more paradoxical examples by making the different dimensions be qualitatively different, e.g., θ_1 is my IQ, θ_2 is the average rainfall in Vancouver, etc.

The solution to the paradox is the following. If your goal is to estimate just θ_i , you cannot do better than using x_i , but if the goal is to estimate the whole vector $\boldsymbol{\theta}$, and you use squared error as your loss function, then shrinkage helps. To see this, suppose we want to estimate $\|\boldsymbol{\theta}\|_2^2$ from a single sample $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\theta}, \mathbf{I})$. A simple estimate is $\|\mathbf{x}\|_2^2$, but this will overestimate the result, since

$$\mathbb{E} [\|\mathbf{x}\|_2^2] = \mathbb{E} \left[\sum_i x_i^2 \right] = \sum_{i=1}^N (1 + \theta_i^2) = N + \|\boldsymbol{\theta}\|_2^2 \quad (6.30)$$

Consequently we can reduce our risk by pooling information, even from unrelated sources, and shrinking towards the overall mean. In Section 5.6.2, we give a Bayesian explanation for this. See also (Efron and Morris 1975).

6.3.3.3 Admissibility is not enough

It seems clear that we can restrict our search for good estimators to the class of admissible estimators. But in fact it is easy to construct admissible estimators, as we show in the following example.

Theorem 6.3.3. *Let $X \sim \mathcal{N}(\theta, 1)$, and consider estimating θ under squared loss. Let $\delta_1(x) = \theta_0$, a constant independent of the data. This is an admissible estimator.*

Proof. Suppose not. Then there is some other estimator δ_2 with smaller risk, so $R(\theta^*, \delta_2) \leq R(\theta^*, \delta_1)$, where the inequality must be strict for some θ^* . Suppose the true parameter is $\theta^* = \theta_0$. Then $R(\theta^*, \delta_1) = 0$, and

$$R(\theta^*, \delta_2) = \int (\delta_2(x) - \theta_0)^2 p(x|\theta_0) dx \quad (6.31)$$

Since $0 \leq R(\theta^*, \delta_2) \leq R(\theta^*, \delta_1)$ for all θ^* , and $R(\theta_0, \delta_1) = 0$, we have $R(\theta_0, \delta_2) = 0$ and hence $\delta_2(x) = \theta_0 = \delta_1(x)$. Thus the only way δ_2 can avoid having higher risk than δ_1 at some specific point θ_0 is by being equal to δ_1 . Hence there is no other estimator δ_2 with strictly lower risk, so δ_2 is admissible. $\square$

6.4 Desirable properties of estimators

Since frequentist decision theory does not provide an automatic way to choose the best estimator, we need to come up with other heuristics for choosing amongst them. In this section, we discuss some properties we would like estimators to have. Unfortunately, we will see that we cannot achieve all of these properties at the same time.

6.4.1 Consistent estimators

An estimator is said to be **consistent** if it eventually recovers the true parameters that generated the data as the sample size goes to infinity, i.e., $\hat{\theta}(\mathcal{D}) \rightarrow \theta^*$ as $|\mathcal{D}| \rightarrow \infty$ (where the arrow denotes convergence in probability). Of course, this concept only makes sense if the data actually comes from the specified model with parameters θ^* , which is not usually the case with real data. Nevertheless, it can be a useful theoretical property.

It can be shown that the MLE is a consistent estimator. The intuitive reason is that maximizing likelihood is equivalent to minimizing $\mathbb{KL}(p(\cdot|\theta^*)||p(\cdot|\hat{\theta}))$, where $p(\cdot|\theta^*)$ is the true distribution and $p(\cdot|\hat{\theta})$ is our estimate. We can achieve 0 KL divergence iff $\hat{\theta} = \theta^*$.⁴

6.4.2 Unbiased estimators

The **bias** of an estimator is defined as

$$\text{bias}(\hat{\theta}(\cdot)) = \mathbb{E}_{p(\mathcal{D}|\theta_*)} [\hat{\theta}(\mathcal{D}) - \theta_*] \quad (6.32)$$

where θ_* is the true parameter value. If the bias is zero, the estimator is called **unbiased**. This means the sampling distribution is centered on the true parameter. For example, the MLE for a Gaussian mean is unbiased:

$$\text{bias}(\hat{\mu}) = \mathbb{E}[\bar{x}] - \mu = \mathbb{E}\left[\frac{1}{N} \sum_{i=1}^N x_i\right] - \mu = \frac{N\mu}{N} - \mu = 0 \quad (6.33)$$

4. If the model is unidentifiable, the MLE may select a set of parameters that is different from the true parameters but for which the induced distribution, $p(\cdot|\hat{\theta})$, is the same as the exact distribution. Such parameters are said to be likelihood equivalent.

However, the MLE for a Gaussian variance, $\hat{\sigma}^2$, is not an unbiased estimator of σ^2 . In fact, one can show (Exercise 6.3) that

$$\mathbb{E} [\hat{\sigma}^2] = \frac{N-1}{N} \sigma^2 \quad (6.34)$$

However, the following estimator

$$\hat{\sigma}_{N-1}^2 = \frac{N}{N-1} \hat{\sigma}^2 = \frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2 \quad (6.35)$$

is an unbiased estimator, which we can easily prove as follows:

$$\mathbb{E} [\hat{\sigma}_{N-1}^2] = \mathbb{E} \left[\frac{N}{N-1} \hat{\sigma}^2 \right] = \frac{N}{N-1} \frac{N-1}{N} \sigma^2 = \sigma^2 \quad (6.36)$$

In Matlab, `var(X)` returns $\hat{\sigma}_{N-1}^2$, whereas `var(X,1)` returns $\hat{\sigma}^2$ (the MLE). For large enough N , the difference will be negligible.

Although the MLE may sometimes be a biased estimator, one can show that asymptotically, it is always unbiased. (This is necessary for the MLE to be a consistent estimator.)

Although being unbiased sounds like a desirable property, this is not always true. See Section 6.4.4 and (Lindley 1972) for discussion of this point.

6.4.3 Minimum variance estimators

It seems intuitively reasonable that we want our estimator to be unbiased (although we shall give some arguments against this claim below). However, being unbiased is not enough. For example, suppose we want to estimate the mean of a Gaussian from $\mathcal{D} = \{x_1, \dots, x_N\}$. The estimator that just looks at the first data point, $\hat{\theta}(\mathcal{D}) = x_1$, is an unbiased estimator, but will generally be further from θ_* than the empirical mean $\bar{x}$ (which is also unbiased). So the variance of an estimator is also important.

A natural question is: how long can the variance go? A famous result, called the **Cramer-Rao lower bound**, provides a lower bound on the variance of any unbiased estimator. More precisely,

Theorem 6.4.1 (Cramer-Rao inequality). *Let $X_1, \dots, X_n \sim p(X|\theta_0)$ and $\hat{\theta} = \hat{\theta}(x_1, \dots, x_n)$ be an unbiased estimator of θ_0 . Then, under various smoothness assumptions on $p(X|\theta_0)$, we have*

$$\text{var} [\hat{\theta}] \geq \frac{1}{nI(\theta_0)} \quad (6.37)$$

where $I(\theta_0)$ is the Fisher information matrix (see Section 6.2.2).

A proof can be found e.g., in (Rice 1995, p275).

It can be shown that the MLE achieves the Cramer Rao lower bound, and hence has the smallest asymptotic variance of any unbiased estimator. Thus MLE is said to be **asymptotically optimal**.

6.4.4 The bias-variance tradeoff

Although using an unbiased estimator seems like a good idea, this is not always the case. To see why, suppose we use quadratic loss. As we showed above, the corresponding risk is the MSE. We now derive a very useful decomposition of the MSE. (All expectations and variances are wrt the true distribution $p(\mathcal{D}|\theta^*)$, but we drop the explicit conditioning for notational brevity.) Let $\hat{\theta} = \hat{\theta}(\mathcal{D})$ denote the estimate, and $\bar{\theta} = \mathbb{E}[\hat{\theta}]$ denote the expected value of the estimate (as we vary $\mathcal{D}$). Then we have

$$\mathbb{E}[(\hat{\theta} - \theta^*)^2] = \mathbb{E}\left[\left[(\hat{\theta} - \bar{\theta}) + (\bar{\theta} - \theta^*)\right]^2\right] \quad (6.38)$$

$$= \mathbb{E}\left[(\hat{\theta} - \bar{\theta})^2\right] + 2(\bar{\theta} - \theta^*)\mathbb{E}[\hat{\theta} - \bar{\theta}] + (\bar{\theta} - \theta^*)^2 \quad (6.39)$$

$$= \mathbb{E}\left[(\hat{\theta} - \bar{\theta})^2\right] + (\bar{\theta} - \theta^*)^2 \quad (6.40)$$

$$= \text{var}[\hat{\theta}] + \text{bias}^2(\hat{\theta}) \quad (6.41)$$

In words,

$$\text{MSE} = \text{variance} + \text{bias}^2$$

(6.42)

This is called the **bias-variance tradeoff** (see e.g., (Geman et al. 1992)). What it means is that it might be wise to use a biased estimator, so long as it reduces our variance, assuming our goal is to minimize squared error.

6.4.4.1 Example: estimating a Gaussian mean

Let us give an example, based on (Hoff 2009, p79). Suppose we want to estimate the mean of a Gaussian from $\mathbf{x} = (x_1, \dots, x_N)$. We assume the data is sampled from $x_i \sim \mathcal{N}(\theta^* = 1, \sigma^2)$. An obvious estimate is the MLE. This has a bias of 0 and a variance of

$$\text{var}[\bar{x}|\theta^*] = \frac{\sigma^2}{N} \quad (6.43)$$

But we could also use a MAP estimate. In Section 4.6.1, we show that the MAP estimate under a Gaussian prior of the form $\mathcal{N}(\theta_0, \sigma^2/\kappa_0)$ is given by

$$\tilde{x} \triangleq \frac{N}{N + \kappa_0}\bar{x} + \frac{\kappa_0}{N + \kappa_0}\theta_0 = w\bar{x} + (1 - w)\theta_0 \quad (6.44)$$

where $0 \leq w \leq 1$ controls how much we trust the MLE compared to our prior. (This is also the posterior mean, since the mean and mode of a Gaussian are the same.) The bias and variance are given by

$$\mathbb{E}[\tilde{x}] - \theta^* = w\theta_0 + (1 - w)\theta_0 - \theta^* = (1 - w)(\theta_0 - \theta^*) \quad (6.45)$$

$$\text{var}[\tilde{x}] = w^2 \frac{\sigma^2}{N} \quad (6.46)$$

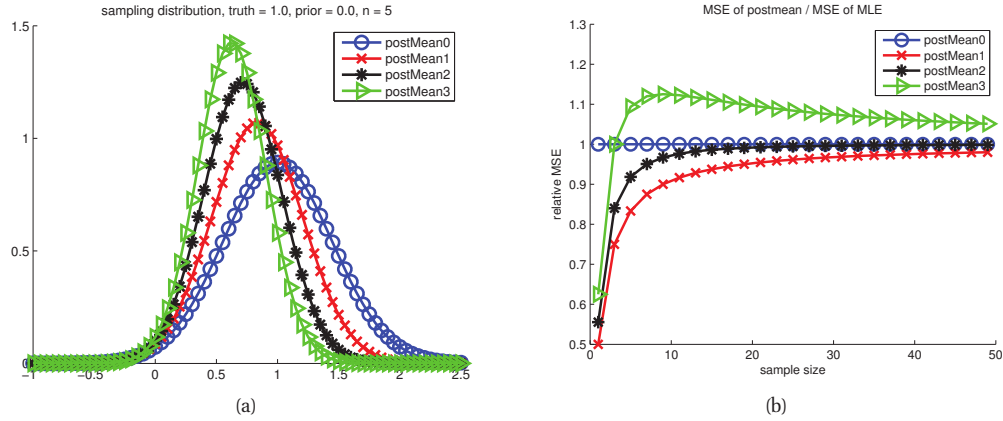


Figure 6.4 Left: Sampling distribution of the MAP estimate with different prior strengths κ_0 . (The MLE corresponds to $\kappa_0 = 0$.) Right: MSE relative to that of the MLE versus sample size. Based on Figure 5.6 of (Hoff 2009). Figure generated by `samplingDistGaussShrinkage`.

So although the MAP estimate is biased (assuming $w < 1$), it has lower variance.

Let us assume that our prior is slightly misspecified, so we use $\theta_0 = 0$, whereas the truth is $\theta^* = 1$. In Figure 6.4(a), we see that the sampling distribution of the MAP estimate for $\kappa_0 > 0$ is biased away from the truth, but has lower variance (is narrower) than that of the MLE.

In Figure 6.4(b), we plot $\text{mse}(\tilde{x})/\text{mse}(\bar{x})$ vs N . We see that the MAP estimate has lower MSE than the MLE, especially for small sample size, for $\kappa_0 \in \{1, 2\}$. The case $\kappa_0 = 0$ corresponds to the MLE, and the case $\kappa_0 = 3$ corresponds to a strong prior, which hurts performance because the prior mean is wrong. It is clearly important to “tune” the strength of the prior, a topic we discuss later.

6.4.4.2 Example: ridge regression

Another important example of the bias variance tradeoff arises in ridge regression, which we discuss in Section 7.5. In brief, this corresponds to MAP estimation for linear regression under a Gaussian prior, $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{0}, \lambda^{-1}\mathbf{I})$. The zero-mean prior encourages the weights to be small, which reduces overfitting; the precision term, λ , controls the strength of this prior. Setting $\lambda = 0$ results in the MLE; using $\lambda > 0$ results in a biased estimate. To illustrate the effect on the variance, consider a simple example. Figure 6.5 on the left plots each individual fitted curve, and on the right plots the average fitted curve. We see that as we increase the strength of the regularizer, the variance decreases, but the bias increases.

6.4.4.3 Bias-variance tradeoff for classification

If we use 0-1 loss instead of squared error, the above analysis breaks down, since the frequentist risk is no longer expressible as squared bias plus variance. In fact, one can show (Exercise 7.2 of (Hastie et al. 2009)) that the bias and variance combine multiplicatively. If the estimate is on

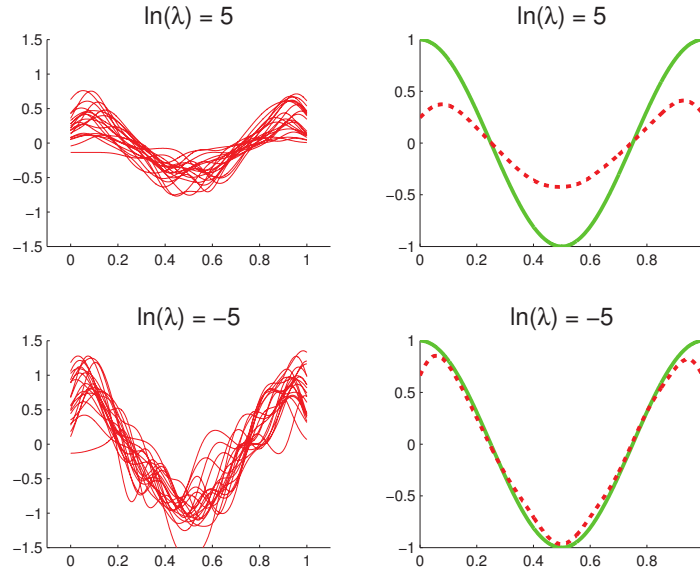


Figure 6.5 Illustration of bias-variance tradeoff for ridge regression. We generate 100 data sets from the true function, shown in solid green. Left: we plot the regularized fit for 20 different data sets. We use linear regression with a Gaussian RBF expansion, with 25 centers evenly spread over the $[0, 1]$ interval. Right: we plot the average of the fits, averaged over all 100 datasets. Top row: strongly regularized: we see that the individual fits are similar to each other (low variance), but the average is far from the truth (high bias). Bottom row: lightly regularized: we see that the individual fits are quite different from each other (high variance), but the average is close to the truth (low bias). Based on (Bishop 2006a) Figure 3.5. Figure generated by `biasVarModelComplexity3`.

the correct side of the decision boundary, then the bias is negative, and decreasing the variance will decrease the misclassification rate. But if the estimate is on the wrong side of the decision boundary, then the bias is positive, so it pays to *increase* the variance (Friedman 1997a). This little known fact illustrates that the bias-variance tradeoff is not very useful for classification. It is better to focus on expected loss (see below), not directly on bias and variance. We can approximate the expected loss using cross validation, as we discuss in Section 6.5.3.

6.5 Empirical risk minimization

Frequentist decision theory suffers from the fundamental problem that one cannot actually compute the risk function, since it relies on knowing the true data distribution. (By contrast, the Bayesian posterior expected loss can always be computed, since it conditions on the data rather than conditioning on θ^* .) However, there is one setting which avoids this problem, and that is where the task is to predict observable quantities, as opposed to estimating hidden variables or parameters. That is, instead of looking at loss functions of the form $L(\theta, \delta(\mathcal{D}))$, where θ is the true but unknown parameter, and $\delta(\mathcal{D})$ is our estimator, let us look at loss

functions of the form $L(y, \delta(\mathbf{x}))$, where y is the true but unknown response, and $\delta(\mathbf{x})$ is our prediction given the input $\mathbf{x}$. In this case, the frequentist risk becomes

$$R(p_*, \delta) \triangleq \mathbb{E}_{(\mathbf{x}, y) \sim p_*} [L(y, \delta(\mathbf{x}))] = \sum_{\mathbf{x}} \sum_y L(y, \delta(\mathbf{x})) p_*(\mathbf{x}, y) \quad (6.47)$$

where p_* represents “nature’s distribution”. Of course, this distribution is unknown, but a simple approach is to use the empirical distribution, derived from some training data, to approximate p_* , i.e.,

$$p_*(\mathbf{x}, y) \approx p_{\text{emp}}(\mathbf{x}, y) \triangleq \frac{1}{N} \sum_{i=1}^N \delta_{\mathbf{x}_i}(\mathbf{x}) \delta_{y_i}(y) \quad (6.48)$$

We then define the **empirical risk** as follows:

$$R_{\text{emp}}(\mathcal{D}, \delta) \triangleq R(p_{\text{emp}}, \delta) = \frac{1}{N} \sum_{i=1}^N L(y_i, \delta(\mathbf{x}_i)) \quad (6.49)$$

In the case of 0-1 loss, $L(y, \delta(\mathbf{x})) = \mathbb{I}(y \neq \delta(\mathbf{x}))$, this becomes the **misclassification rate**. In the case of squared error loss, $L(y, \delta(\mathbf{x})) = (y - \delta(\mathbf{x}))^2$, this becomes the **mean squared error**. We define the task of **empirical risk minimization** or **ERM** as finding a decision procedure (typically a classification rule) to minimize the empirical risk:

$$\delta_{\text{ERM}}(\mathcal{D}) = \underset{\delta}{\operatorname{argmin}} R_{\text{emp}}(\mathcal{D}, \delta) \quad (6.50)$$

In the unsupervised case, we eliminate all references to y , and replace $L(y, \delta(\mathbf{x}))$ with $L(\mathbf{x}, \delta(\mathbf{x}))$, where, for example, $L(\mathbf{x}, \delta(\mathbf{x})) = \|\mathbf{x} - \delta(\mathbf{x})\|_2^2$, which measures the reconstruction error. We can define the decision rule using $\delta(\mathbf{x}) = \text{decode}(\text{encode}(\mathbf{x}))$, as in vector quantization (Section 11.4.2.6) or PCA (section 12.2). Finally, we define the empirical risk as

$$R_{\text{emp}}(\mathcal{D}, \delta) = \frac{1}{N} \sum_{i=1}^N L(\mathbf{x}_i, \delta(\mathbf{x}_i)) \quad (6.51)$$

Of course, we can always trivially minimize this risk by setting $\delta(\mathbf{x}) = \mathbf{x}$, so it is critical that the encoder-decoder go via some kind of bottleneck.

6.5.1 Regularized risk minimization

Note that the empirical risk is equal to the Bayes risk if our prior about “nature’s distribution” is that it is exactly equal to the empirical distribution (Minka 2001b):

$$\mathbb{E} [R(p_*, \delta) | p_* = p_{\text{emp}}] = R_{\text{emp}}(\mathcal{D}, \delta) \quad (6.52)$$

Therefore minimizing the empirical risk will typically result in overfitting. It is therefore often necessary to add a complexity penalty to the objective function:

$$R'(\mathcal{D}, \delta) = R_{\text{emp}}(\mathcal{D}, \delta) + \lambda C(\delta) \quad (6.53)$$

where $C(\delta)$ measures the complexity of the prediction function $\delta(\mathbf{x})$ and λ controls the strength of the complexity penalty. This approach is known as **regularized risk minimization** (RRM). Note that if the loss function is negative log likelihood, and the regularizer is a negative log prior, this is equivalent to MAP estimation.

The two key issues in RRM are: how do we measure complexity, and how do we pick λ . For a linear model, we can define the complexity in terms of its degrees of freedom, discussed in Section 7.5.3. For more general models, we can use the VC dimension, discussed in Section 6.5.4. To pick λ , we can use the methods discussed in Section 6.5.2.

6.5.2 Structural risk minimization

The regularized risk minimization principle says that we should fit the model, for a given complexity penalty, by using

$$\hat{\delta}_\lambda = \operatorname{argmin}_{\delta} [R_{emp}(\mathcal{D}, \delta) + \lambda C(\delta)] \quad (6.54)$$

But how should we pick λ ? We cannot use the training set, since this will underestimate the true risk, a problem known as **optimism of the training error**. As an alternative, we can use the following rule, known as the **structural risk minimization** principle: (Vapnik 1998):

$$\hat{\lambda} = \operatorname{argmin}_{\lambda} \hat{R}(\hat{\delta}_\lambda) \quad (6.55)$$

where $\hat{R}(\delta)$ is an estimate of the risk. There are two widely used estimates: cross validation and theoretical upper bounds on the risk. We discuss both of these below.

6.5.3 Estimating the risk using cross validation

We can estimate the risk of some estimator using a validation set. If we don't have a separate validation set, we can use **cross validation** (CV), as we briefly discussed in Section 1.4.8. More precisely, CV is defined as follows. Let there be $N = |\mathcal{D}|$ data cases in the training set. Denote the data in the k 'th test fold by $\mathcal{D}_k$ and all the other data by $\mathcal{D}_{-k}$. (In **stratified CV**, these folds are chosen so the class proportions (if discrete labels are present) are roughly equal in each fold.) Let $\mathcal{F}$ be a learning algorithm or fitting function that takes a dataset and a model index m (this could a discrete index, such as the degree of a polynomial, or a continuous index, such as the strength of a regularizer) and returns a parameter vector:

$$\hat{\theta}_m = \mathcal{F}(\mathcal{D}, m) \quad (6.56)$$

Finally, let $\mathcal{P}$ be a prediction function that takes an input and a parameter vector and returns a prediction:

$$\hat{y} = \mathcal{P}(\mathbf{x}, \hat{\theta}) = f(\mathbf{x}, \hat{\theta}) \quad (6.57)$$

Thus the combined **fit-predict cycle** is denoted as

$$f_m(\mathbf{x}, \mathcal{D}) = \mathcal{P}(\mathbf{x}, \mathcal{F}(\mathcal{D}, m)) \quad (6.58)$$

The K -fold CV estimate of the risk of f_m is defined by

$$R(m, \mathcal{D}, K) \triangleq \frac{1}{N} \sum_{k=1}^K \sum_{i \in \mathcal{D}_k} L(y_i, \mathcal{P}(\mathbf{x}_i, \mathcal{F}(\mathcal{D}_{-k}, m))) \quad (6.59)$$

Note that we can call the fitting algorithm once per fold. Let $f_m^k(\mathbf{x}) = \mathcal{P}(\mathbf{x}, \mathcal{F}(\mathcal{D}_{-k}, m))$ be the function that was trained on all the data except for the test data in fold k . Then we can rewrite the CV estimate as

$$R(m, \mathcal{D}, K) = \frac{1}{N} \sum_{k=1}^K \sum_{i \in \mathcal{D}_k} L(y_i, f_m^k(\mathbf{x}_i)) = \frac{1}{N} \sum_{i=1}^N L(y_i, f_m^{k(i)}(\mathbf{x}_i)) \quad (6.60)$$

where $k(i)$ is the fold in which i is used as test data. In other words, we predict y_i using a model that was trained on data that does not contain $\mathbf{x}_i$.

Of $K = N$, the method is known as **leave one out cross validation** or LOOCV. In this case, the estimated risk becomes

$$R(m, \mathcal{D}, N) = \frac{1}{N} \sum_{i=1}^N L(y_i, f_m^{-i}(\mathbf{x}_i)) \quad (6.61)$$

where $f_m^i(\mathbf{x}) = \mathcal{P}(\mathbf{x}, \mathcal{F}(\mathcal{D}_{-i}, m))$. This requires fitting the model N times, where for f_m^{-i} we omit the i 'th training case. Fortunately, for some model classes and loss functions (namely linear models and quadratic loss), we can fit the model once, and analytically “remove” the effect of the i 'th training case. This is known as **generalized cross validation** or GCV.

6.5.3.1 Example: using CV to pick λ for ridge regression

As a concrete example, consider picking the strength of the ℓ_2 regularizer in penalized linear regression. We use the following rule:

$$\hat{\lambda} = \arg \min_{\lambda \in [\lambda_{min}, \lambda_{max}]} R(\lambda, \mathcal{D}_{\text{train}}, K) \quad (6.62)$$

where $[\lambda_{min}, \lambda_{max}]$ is a finite range of λ values that we search over, and $R(\lambda, \mathcal{D}_{\text{train}}, K)$ is the K -fold CV estimate of the risk of using λ , given by

$$R(\lambda, \mathcal{D}_{\text{train}}, K) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{k=1}^K \sum_{i \in \mathcal{D}_k} L(y_i, f_{\lambda}^k(\mathbf{x}_i)) \quad (6.63)$$

where $f_{\lambda}^k(\mathbf{x}) = \mathbf{x}^T \hat{\mathbf{w}}_{\lambda}(\mathcal{D}_{-k})$ is the prediction function trained on data excluding fold k , and $\hat{\mathbf{w}}_{\lambda}(\mathcal{D}) = \arg \min_{\mathbf{w}} NLL(\mathbf{w}, \mathcal{D}) + \lambda \|\mathbf{w}\|_2^2$ is the MAP estimate. Figure 6.6(b) gives an example of a CV estimate of the risk vs $\log(\lambda)$, where the loss function is squared error.

When performing classification, we usually use 0-1 loss. In this case, we optimize a convex upper bound on the empirical risk to estimate $\mathbf{w}_{\lambda}$ but we optimize (the CV estimate of) the risk itself to estimate λ . We can handle the non-smooth 0-1 loss function when estimating λ because we are using brute-force search over the entire (one-dimensional) space.

When we have more than one or two tuning parameters, this approach becomes infeasible. In such cases, one can use empirical Bayes, which allows one to optimize large numbers of hyper-parameters using gradient-based optimizers instead of brute-force search. See Section 5.6 for details.

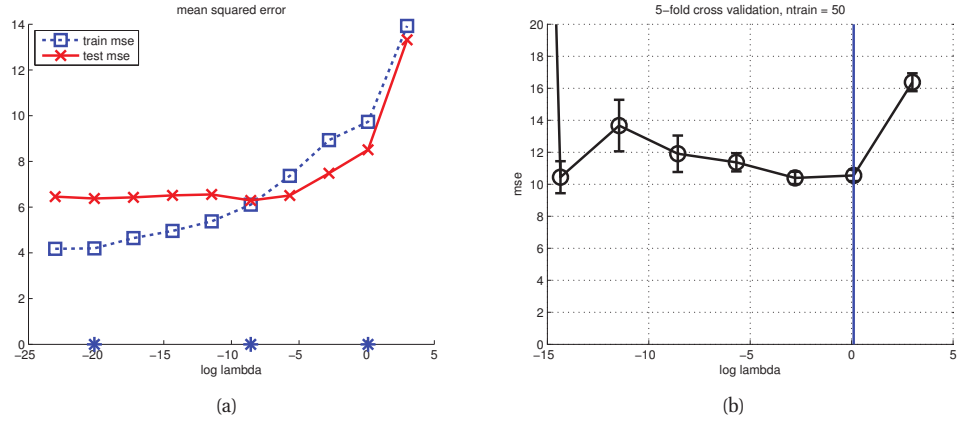


Figure 6.6 (a) Mean squared error for ℓ_2 penalized degree 14 polynomial regression vs log regularizer. Same as in Figures 7.8, except now we have $N = 50$ training points instead of 21. The stars correspond to the values used to plot the functions in Figure 7.7. (b) CV estimate. The vertical scale is truncated for clarity. The blue line corresponds to the value chosen by the one standard error rule. Figure generated by `linregPolyVsRegDemo`.

6.5.3.2 The one standard error rule

The above procedure estimates the risk, but does not give any measure of uncertainty. A standard frequentist measure of uncertainty of an estimate is the standard error of the mean, defined by

$$se = \frac{\hat{\sigma}}{\sqrt{N}} = \sqrt{\frac{\hat{\sigma}^2}{N}} \quad (6.64)$$

where $\hat{\sigma}^2$ is an estimate of the variance of the loss:

$$\hat{\sigma}^2 = \frac{1}{N} \sum_{i=1}^N (L_i - \bar{L})^2, \quad L_i = L(y_i, f_m^{k(i)}(\mathbf{x}_i)) \quad \bar{L} = \frac{1}{N} \sum_{i=1}^N L_i \quad (6.65)$$

Note that σ measures the intrinsic variability of L_i across samples, whereas se measures our uncertainty about the mean $\bar{L}$.

Suppose we apply CV to a set of models and compute the mean and se of their estimated risks. A common heuristic for picking a model from these noisy estimates is to pick the value which corresponds to the simplest model whose risk is no more than one standard error above the risk of the best model; this is called the **one-standard error rule** (Hastie et al. 2001, p216). For example, in Figure 6.6, we see that this heuristic does not choose the lowest point on the curve, but one that is slightly to its right, since that corresponds to a more heavily regularized model with essentially the same empirical performance.

6.5.3.3 CV for model selection in non-probabilistic unsupervised learning

If we are performing unsupervised learning, we must use a loss function such as $L(\mathbf{x}, \delta(\mathbf{x})) = \|\mathbf{x} - \delta(\mathbf{x})\|_2$, which measures reconstruction error. Here $\delta(\mathbf{x})$ is some encode-decode scheme. However, as we discussed in Section 11.5.2, we cannot use CV to determine the complexity of δ , since we will always get lower loss with a more complex model, even if evaluated on the test set. This is because more complex models will compress the data less, and induce less distortion. Consequently, we must either use probabilistic models, or invent other heuristics.

6.5.4 Upper bounding the risk using statistical learning theory *

The principle problem with cross validation is that it is slow, since we have to fit the model multiple times. This motivates the desire to compute analytic approximations or bounds to the generalization error. This is the studied in the field of **statistical learning theory** (SLT). More precisely, SLT tries to bound the risk $R(p_*, h)$ for any data distribution p_* and hypothesis $h \in \mathcal{H}$ in terms of the empirical risk $R_{emp}(\mathcal{D}, h)$, the sample size $N = |\mathcal{D}|$, and the size of the hypothesis space $\mathcal{H}$.

Let us initially consider the case where the hypothesis space is finite, with size $\dim(\mathcal{H}) = |\mathcal{H}|$. In other words, we are selecting a model/ hypothesis from a finite list, rather than optimizing real-valued parameters. Then we can prove the following.

Theorem 6.5.1. *For any data distribution p_* , and any dataset $\mathcal{D}$ of size N drawn from p_* , the probability that our estimate of the error rate will be more than ϵ wrong, in the worst case, is upper bounded as follows:*

$$P\left(\max_{h \in \mathcal{H}} |R_{emp}(\mathcal{D}, h) - R(p_*, h)| > \epsilon\right) \leq 2 \dim(\mathcal{H}) e^{-2N\epsilon^2} \quad (6.66)$$

Proof. To prove this, we need two useful results. First, **Hoeffding's inequality**, which states that if $X_1, \dots, X_N \sim \text{Ber}(\theta)$, then, for any $\epsilon > 0$,

$$P(|\bar{x} - \theta| > \epsilon) \leq 2e^{-2N\epsilon^2} \quad (6.67)$$

where $\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$. Second, the **union bound**, which says that if $A_1, \dots, A_d$ are a set of events, then $P(\cup_{i=1}^d A_i) \leq \sum_{i=1}^d P(A_i)$.

Finally, for notational brevity, let $R(h) = R(h, p_*)$ be the true risk, and $\hat{R}_N(h) = R_{emp}(\mathcal{D}, h)$ be the empirical risk.

Using these results we have

$$P\left(\max_{h \in \mathcal{H}} |\hat{R}_N(h) - R(h)| > \epsilon\right) = P\left(\bigcup_{h \in \mathcal{H}} |\hat{R}_N(h) - R(h)| > \epsilon\right) \quad (6.68)$$

$$\leq \sum_{h \in \mathcal{H}} P(|\hat{R}_N(h) - R(h)| > \epsilon) \quad (6.69)$$

$$\leq \sum_{h \in \mathcal{H}} 2e^{-2N\epsilon^2} = 2 \dim(\mathcal{H}) e^{-2N\epsilon^2} \quad (6.70)$$

□

This bound tells us that the optimism of the training error increases with $\dim(\mathcal{H})$ but decreases with $N = |\mathcal{D}|$, as is to be expected.

If the hypothesis space $\mathcal{H}$ is infinite (e.g., we have real-valued parameters), we cannot use $\dim(\mathcal{H}) = |\mathcal{H}|$. Instead, we can use a quantity called the **Vapnik-Chervonenkis** or **VC** dimension of the hypothesis class. See (Vapnik 1998) for details.

Stepping back from all the theory, the key intuition behind statistical learning theory is quite simple. Suppose we find a model with low empirical risk. If the hypothesis space $\mathcal{H}$ is very big, relative to the data size, then it is quite likely that we just got “lucky” and were given a data set that is well-modeled by our chosen function by chance. However, this does not mean that such a function will have low generalization error. But if the hypothesis class is sufficiently constrained in size, and/or the training set is sufficiently large, then we are unlikely to get lucky in this way, so a low empirical risk is evidence of a low true risk.

Note that optimism of the training error does not necessarily increase with model complexity, but it does increase with the number of different models that are being searched over.

The advantage of statistical learning theory compared to CV is that the bounds on the risk are quicker to compute than using CV. The disadvantage is that it is hard to compute the VC dimension for many interesting models, and the upper bounds are usually very loose (although see (Kaariainen and Langford 2005)).

One can extend statistical learning theory by taking computational complexity of the learner into account. This field is called **computational learning theory** or **COLT**. Most of this work focuses on the case where h is a binary classifier, and the loss function is 0-1 loss. If we observe a low empirical risk, and the hypothesis space is suitably “small”, then we can say that our estimated function is **probably approximately correct** or **PAC**. A hypothesis space is said to be **efficiently PAC-learnable** if there is a polynomial time algorithm that can identify a function that is PAC. See (Kearns and Vazirani 1994) for details.

6.5.5 Surrogate loss functions

Minimizing the loss in the ERM/ RRM framework is not always easy. For example, we might want to optimize the AUC or F1 scores. Or more simply, we might just want to minimize the 0-1 loss, as is common in classification. Unfortunately, the 0-1 risk is a very non-smooth objective and hence is hard to optimize. One alternative is to use maximum likelihood estimation instead, since log-likelihood is a smooth convex upper bound on the 0-1 risk, as we show below.

To see this, consider binary logistic regression, and let $y_i \in \{-1, +1\}$. Suppose our decision function computes the log-odds ratio,

$$f(\mathbf{x}_i) = \log \frac{p(y = 1|\mathbf{x}_i, \mathbf{w})}{p(y = -1|\mathbf{x}_i, \mathbf{w})} = \mathbf{w}^T \mathbf{x}_i = \eta_i \quad (6.71)$$

Then the corresponding probability distribution on the output label is

$$p(y_i|\mathbf{x}_i, \mathbf{w}) = \text{sigm}(y_i \eta_i) \quad (6.72)$$

Let us define the **log-loss** as as

$$L_{\text{nl}}(y, \eta) = -\log p(y|\mathbf{x}, \mathbf{w}) = \log(1 + e^{-y\eta}) \quad (6.73)$$

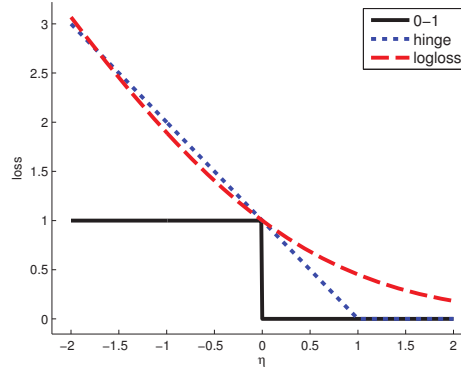


Figure 6.7 Illustration of various loss functions for binary classification. The horizontal axis is the margin $y\eta$, the vertical axis is the loss. The log loss uses log base 2. Figure generated by `hingeLossPlot`.

It is clear that minimizing the average log-loss is equivalent to maximizing the likelihood.

Now consider computing the most probable label, which is equivalent to using $\hat{y} = -1$ if $\eta_i < 0$ and $\hat{y} = +1$ if $\eta_i \geq 0$. The 0-1 loss of our function becomes

$$L_{01}(y, \eta) = \mathbb{I}(y \neq \hat{y}) = \mathbb{I}(y\eta < 0) \quad (6.74)$$

Figure 6.7 plots these two loss functions. We see that the NLL is indeed an upper bound on the 0-1 loss.

Log-loss is an example of a **surrogate loss function**. Another example is the **hinge loss**:

$$L_{\text{hinge}}(y, \eta) = \max(0, 1 - y\eta) \quad (6.75)$$

See Figure 6.7 for a plot. We see that the function looks like a door hinge, hence its name. This loss function forms the basis of a popular classification method known as support vector machines (SVM), which we will discuss in Section 14.5.

The surrogate is usually chosen to be a convex upper bound, since convex functions are easy to minimize. See e.g., (Bartlett et al. 2006) for more information.

6.6 Pathologies of frequentist statistics *

I believe that it would be very difficult to persuade an intelligent person that current [frequentist] statistical practice was sensible, but that there would be much less difficulty with an approach via likelihood and Bayes' theorem. — George Box, 1962.

Frequentist statistics exhibits various forms of weird and undesirable behaviors, known as **pathologies**. We give a few examples below, in order to caution the reader; these and other examples are explained in more detail in (Lindley 1972; Lindley and Phillips 1976; Lindley 1982; Berger 1985; Jaynes 2003; Minka 1999).

6.6.1 Counter-intuitive behavior of confidence intervals

A **confidence interval** is an interval derived from the sampling distribution of an estimator (whereas a Bayesian credible interval is derived from the posterior of a parameter, as we discussed in Section 5.2.2). More precisely, a frequentist confidence interval for some parameter θ is defined by the following (rather un-natural) expression:

$$C'_\alpha(\theta) = (\ell, u) : P(\ell(\tilde{\mathcal{D}}) \leq \theta \leq u(\tilde{\mathcal{D}}) | \tilde{\mathcal{D}} \sim \theta) = 1 - \alpha \quad (6.76)$$

That is, if we sample hypothetical future data $\tilde{\mathcal{D}}$ from θ , then $(\ell(\tilde{\mathcal{D}}), u(\tilde{\mathcal{D}}))$, is a confidence interval if the parameter θ lies inside this interval $1 - \alpha$ percent of the time.

Let us step back for a moment and think about what is going on. In Bayesian statistics, we condition on what is known — namely the observed data, $\mathcal{D}$ — and average over what is not known, namely the parameter θ . In frequentist statistics, we do exactly the opposite: we condition on what is unknown — namely the true parameter value θ — and average over hypothetical future data sets $\tilde{\mathcal{D}}$.

This counter-intuitive definition of confidence intervals can lead to bizarre results. Consider the following example from (Berger 1985, p11). Suppose we draw two integers $\mathcal{D} = (x_1, x_2)$ from

$$p(x|\theta) = \begin{cases} 0.5 & \text{if } x = \theta \\ 0.5 & \text{if } x = \theta + 1 \\ 0 & \text{otherwise} \end{cases} \quad (6.77)$$

If $\theta = 39$, we would expect the following outcomes each with probability 0.25:

$$(39, 39), (39, 40), (40, 39), (40, 40) \quad (6.78)$$

Let $m = \min(x_1, x_2)$ and define the following confidence interval:

$$[\ell(\mathcal{D}), u(\mathcal{D})] = [m, m] \quad (6.79)$$

For the above samples this yields

$$[39, 39], [39, 39], [39, 39], [40, 40] \quad (6.80)$$

Hence Equation 6.79 is clearly a 75% CI, since 39 is contained in 3/4 of these intervals. However, if $\mathcal{D} = (39, 40)$ then $p(\theta = 39|\mathcal{D}) = 1.0$, so we know that θ must be 39, yet we only have 75% “confidence” in this fact.

Another, less contrived example, is as follows. Suppose we want to estimate the parameter θ of a Bernoulli distribution. Let $\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$ be the sample mean. The MLE is $\hat{\theta} = \bar{x}$. An approximate 95% confidence interval for a Bernoulli parameter is $\bar{x} \pm 1.96 \sqrt{\bar{x}(1 - \bar{x})/N}$ (this is called a **Wald interval** and is based on a Gaussian approximation to the Binomial distribution; compare to Equation 3.27). Now consider a single trial, where $N = 1$ and $x_1 = 0$. The MLE is 0, which overfits, as we saw in Section 3.3.4.1. But our 95% confidence interval is also $(0, 0)$, which seems even worse. It can be argued that the above flaw is because we approximated the true sampling distribution with a Gaussian, or because the sample size was too small, or the parameter “too extreme”. However, the Wald interval can behave badly even for large N , and non-extreme parameters (Brown et al. 2001).

6.6.2 p-values considered harmful

Suppose we want to decide whether to accept or reject some baseline model, which we will call the **null hypothesis**. We need to define some decision rule. In frequentist statistics, it is standard to first compute a quantity called the **p-value**, which is defined as the probability (under the null) of observing some **test statistic** $f(\mathcal{D})$ (such as the chi-squared statistic) that is as large *or larger* than that actually observed:⁵

$$\text{pvalue}(\mathcal{D}) \triangleq P(f(\tilde{\mathcal{D}}) \geq f(\mathcal{D}) | \tilde{\mathcal{D}} \sim H_0) \quad (6.81)$$

This quantity relies on computing a **tail area probability** of the sampling distribution; we give an example of how to do this below.

Given the p-value, we define our decision rule as follows: we reject the null hypothesis iff the p-value is less than some threshold, such as $\alpha = 0.05$. If we do reject it, we say the difference between the observed test statistic and the expected test statistic is **statistically significant** at level α . This approach is known as **null hypothesis significance testing**, or **NHST**.

This procedure guarantees that our expected type I (false positive) error rate is at most α . This is sometimes interpreted as saying that frequentist hypothesis testing is very conservative, since it is unlikely to accidentally reject the null hypothesis. But in fact the opposite is the case: because this method only worries about trying to reject the null, it can never gather evidence in favor of the null, no matter how large the sample size. Because of this, p-values tend to overstate the evidence against the null, and are thus very “trigger happy”.

In general there can be huge differences between p-values and the quantity that we really care about, which is the posterior probability of the null hypothesis given the data, $p(H_0|\mathcal{D})$. In particular, Sellke et al. (2001) show that even if the p-value is as small as 0.05, the posterior probability of H_0 is at least 30%, and often much higher. So frequentists often claim to have “significant” evidence of an effect that cannot be explained by the null hypothesis, whereas Bayesians are usually more conservative in their claims. For example, p-values have been used to “prove” that ESP (extra-sensory perception) is real (Wagenmakers et al. 2011), even though ESP is clearly very improbable. For this reason, p-values have been banned from certain medical journals (Matthews 1998).

Another problem with p-values is that their computation depends on decisions you make about when to stop collecting data, even if these decisions don’t change the data you actually observed. For example, suppose I toss a coin $n = 12$ times and observe $s = 9$ successes (heads) and $f = 3$ failures (tails), so $n = s + f$. In this case, n is fixed and s (and hence f) is random. The relevant sampling model is the binomial

$$\text{Bin}(s|n, \theta) = \binom{n}{s} \theta^s (1 - \theta)^{n-s} \quad (6.82)$$

Let the null hypothesis be that the coin is fair, $\theta = 0.5$, where θ is the probability of success (heads). The one-sided p-value, using test statistic $t(s) = s$, is

$$p_1 = P(S \geq 9 | H_0) = \sum_{s=9}^{12} \text{Bin}(s|12, 0.5) = \sum_{s=9}^{12} \binom{12}{s} 0.5^{12} = 0.073 \quad (6.83)$$

5. The reason we cannot just compute the probability of the observed value of the test statistic is that this will have probability zero under a pdf. The p-value is defined in terms of the cdf, so is always a number between 0 and 1.

The two-sided p-value is

$$p_2 = \sum_{s=9}^{12} \text{Bin}(s|12, 0.5) + \sum_{s=0}^3 \text{Bin}(s|12, 0.5) = 0.073 + 0.073 = 0.146 \quad (6.84)$$

In either case, the p-value is larger than the magical 5% threshold, so a frequentist would not reject the null hypothesis.

Now suppose I told you that I actually kept tossing the coin until I observed $f = 3$ tails. In this case, f is fixed and n (and hence $s = n - f$) is random. The probability model becomes the **negative binomial distribution**, given by

$$\text{NegBinom}(s|f, \theta) = \binom{s+f-1}{f-1} \theta^s (1-\theta)^f \quad (6.85)$$

where $f = n - s$.

Note that the term which depends on θ is the same in Equations 6.82 and 6.85, so the posterior over θ would be the same in both cases. However, these two interpretations of the same data give different p-values. In particular, under the negative binomial model we get

$$p_3 = P(S \geq 9|H_0) = \sum_{s=9}^{\infty} \binom{3+s-1}{2} (1/2)^s (1/2)^3 = 0.0327 \quad (6.86)$$

So the p-value is 3%, and suddenly there seems to be significant evidence of bias in the coin! Obviously this is ridiculous: the data is the same, so our inferences about the coin should be the same. After all, I could have chosen the experimental protocol at random. It is the outcome of the experiment that matters, not the details of how I decided which one to run.

Although this might seem like just a mathematical curiosity, this also has significant practical implications. In particular, the fact that the **stopping rule** affects the computation of the p-value means that frequentists often do not terminate experiments early, even when it is obvious what the conclusions are, lest it adversely affect their statistical analysis. If the experiments are costly or harmful to people, this is obviously a bad idea. Perhaps it is not surprising, then, that the US Food and Drug Administration (FDA), which regulates clinical trials of new drugs, has recently become supportive of Bayesian methods⁶, since Bayesian methods are not affected by the stopping rule.

6.6.3 The likelihood principle

The fundamental reason for many of these pathologies is that frequentist inference violates the **likelihood principle**, which says that inference should be based on the likelihood of the observed data, not based on hypothetical future data that you have not observed. Bayes obviously satisfies the likelihood principle, and consequently does not suffer from these pathologies.

A compelling argument in favor of the likelihood principle was presented in (Birnbbaum 1962), who showed that it followed automatically from two simpler principles. The first of these is the **sufficiency principle**, which says that a sufficient statistic contains all the relevant information

6. See <http://ymlb.wordpress.com/2006/06/19/the-us-fda-is-becoming-progressively-more-bayesian/>.

about an unknown parameter (arguably this is true by definition). The second principle is known as **weak conditionality**, which says that inferences should be based on the events that happened, not which might have happened. To motivate this, consider an example from (Berger 1985). Suppose we need to analyse a substance, and can send it either to a laboratory in New York or in California. The two labs seem equally good, so a fair coin is used to decide between them. The coin comes up heads, so the California lab is chosen. When the results come back, should it be taken into account that the coin could have come up tails and thus the New York lab could have been used? Most people would argue that the New York lab is irrelevant, since the tails event didn't happen. This is an example of weak conditionality. Given this principle, one can show that all inferences should only be based on what was observed, which is in contrast to standard frequentist procedures. See (Berger and Wolpert 1988) for further details on the likelihood principle.

6.6.4 Why isn't everyone a Bayesian?

Given these fundamental flaws of frequentist statistics, and the fact that Bayesian methods do not have such flaws, an obvious question to ask is: "Why isn't everyone a Bayesian?" The (frequentist) statistician Bradley Efron wrote a paper with exactly this title (Efron 1986). His short paper is well worth reading for anyone interested in this topic. Below we quote his opening section:

The title is a reasonable question to ask on at least two counts. First of all, everyone used to be a Bayesian. Laplace wholeheartedly endorsed Bayes's formulation of the inference problem, and most 19th-century scientists followed suit. This included Gauss, whose statistical work is usually presented in frequentist terms.

A second and more important point is the cogency of the Bayesian argument. Modern statisticians, following the lead of Savage and de Finetti, have advanced powerful theoretical arguments for preferring Bayesian inference. A byproduct of this work is a disturbing catalogue of inconsistencies in the frequentist point of view.

Nevertheless, everyone is not a Bayesian. The current era (1986) is the first century in which statistics has been widely used for scientific reporting, and in fact, 20th-century statistics is mainly non-Bayesian. However, Lindley (1975) predicts a change for the 21st century.

Time will tell whether Lindley was right...

Exercises

Exercise 6.1 Pessimism of LOOCV

(Source: Witten05, p152.). Suppose we have a completely random labeled dataset (i.e., the features $\mathbf{x}$ tell us nothing about the class labels y) with N_1 examples of class 1, and N_2 examples of class 2, where $N_1 = N_2$. What is the best misclassification rate any method can achieve? What is the estimated misclassification rate of the same method using LOOCV?

Exercise 6.2 James Stein estimator for Gaussian means

Consider the 2 stage model $Y_i|\theta_i \sim \mathcal{N}(\theta_i, \sigma^2)$ and $\theta_i|\mu \sim \mathcal{N}(m_0, \tau_0^2)$. Suppose $\sigma^2 = 500$ is known and we observe the following 6 data points, $i = 1 : 6$:

1505, 1528, 1564, 1498, 1600, 1470

- Find the ML-II estimates of m_0 and τ_0^2 .
- Find the posterior estimates $\mathbb{E}[\theta_i|y_i, m_0, \tau_0]$ and $\text{var}[\theta_i|y_i, m_0, \tau_0]$ for $i = 1$. (The other terms, $i = 2 : 6$, are computed similarly.)
- Give a 95% credible interval for $p(\theta_i|y_i, m_0, \tau_0)$ for $i = 1$. Do you trust this interval (assuming the Gaussian assumption is reasonable)? i.e. is it likely to be too large or too small, or just right?
- What do you expect would happen to your estimates if σ^2 were much smaller (say $\sigma^2 = 1$)? You do not need to compute the numerical answer; just briefly explain what would happen qualitatively, and why.

Exercise 6.3 $\hat{\sigma}_{MLE}^2$ is biased

Show that $\hat{\sigma}_{MLE}^2 = \frac{1}{N} \sum_{n=1}^N (x_n - \hat{\mu})^2$ is a biased estimator of σ^2 , i.e., show

$$\mathbf{E}_{X_1, \dots, X_n \sim \mathcal{N}(\mu, \sigma)}[\hat{\sigma}^2(X_1, \dots, X_n)] \neq \sigma^2$$

Hint: note that $X_1, \dots, X_N$ are independent, and use the fact that the expectation of a product of independent random variables is the product of the expectations.

Exercise 6.4 Estimation of σ^2 when μ is known

Suppose we sample $x_1, \dots, x_N \sim \mathcal{N}(\mu, \sigma^2)$ where μ is a *known* constant. Derive an expression for the MLE for σ^2 in this case. Is it unbiased?

7 *Linear regression*

7.1 Introduction

Linear regression is the “work horse” of statistics and (supervised) machine learning. When augmented with kernels or other forms of basis function expansion, it can model also non-linear relationships. And when the Gaussian output is replaced with a Bernoulli or multinoulli distribution, it can be used for classification, as we will see below. So it pays to study this model in detail.

7.2 Model specification

As we discussed in Section 1.4.5, linear regression is a model of the form

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y|\mathbf{w}^T \mathbf{x}, \sigma^2) \quad (7.1)$$

Linear regression can be made to model non-linear relationships by replacing $\mathbf{x}$ with some non-linear function of the inputs, $\phi(\mathbf{x})$. That is, we use

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y|\mathbf{w}^T \phi(\mathbf{x}), \sigma^2) \quad (7.2)$$

This is known as **basis function expansion**. (Note that the model is still linear in the parameters $\mathbf{w}$, so it is still called linear regression; the importance of this will become clear below.) A simple example are polynomial basis functions, where the model has the form

$$\phi(x) = [1, x, x^2, \dots, x^d] \quad (7.3)$$

Figure 1.18 illustrates the effect of changing d : increasing the degree d allows us to create increasingly complex functions.

We can also apply linear regression to more than 1 input. For example, consider modeling temperature as a function of location. Figure 7.1(a) plots $\mathbb{E}[y|\mathbf{x}] = w_0 + w_1x_1 + w_2x_2$, and Figure 7.1(b) plots $\mathbb{E}[y|\mathbf{x}] = w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2$.

7.3 Maximum likelihood estimation (least squares)

A common way to estimate the parameters of a statistical model is to compute the MLE, which is defined as

$$\hat{\boldsymbol{\theta}} \triangleq \arg \max_{\boldsymbol{\theta}} \log p(\mathcal{D}|\boldsymbol{\theta}) \quad (7.4)$$

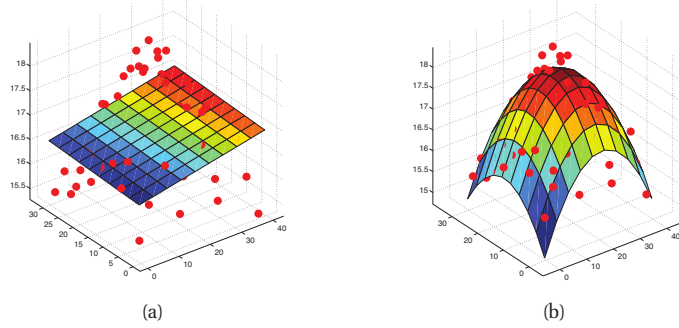


Figure 7.1 Linear regression applied to 2d data. Vertical axis is temperature, horizontal axes are location within a room. Data was collected by some remote sensing motes at Intel's lab in Berkeley, CA (data courtesy of Romain Thibaux). (a) The fitted plane has the form $\hat{f}(\mathbf{x}) = w_0 + w_1x_1 + w_2x_2$. (b) Temperature data is fitted with a quadratic of the form $\hat{f}(\mathbf{x}) = w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2$. Produced by `surfaceFitDemo`.

It is common to assume the training examples are independent and identically distributed, commonly abbreviated to **iid**. This means we can write the log-likelihood as follows:

$$\ell(\boldsymbol{\theta}) \triangleq \log p(\mathcal{D}|\boldsymbol{\theta}) = \sum_{i=1}^N \log p(y_i|\mathbf{x}_i, \boldsymbol{\theta}) \quad (7.5)$$

Instead of maximizing the log-likelihood, we can equivalently minimize the **negative log likelihood** or **NLL**:

$$\text{NLL}(\boldsymbol{\theta}) \triangleq - \sum_{i=1}^N \log p(y_i|\mathbf{x}_i, \boldsymbol{\theta}) \quad (7.6)$$

The NLL formulation is sometimes more convenient, since many optimization software packages are designed to find the minima of functions, rather than maxima.

Now let us apply the method of MLE to the linear regression setting. Inserting the definition of the Gaussian into the above, we find that the log likelihood is given by

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^N \log \left[\left(\frac{1}{2\pi\sigma^2} \right)^{\frac{1}{2}} \exp \left(-\frac{1}{2\sigma^2} (y_i - \mathbf{w}^T \mathbf{x}_i)^2 \right) \right] \quad (7.7)$$

$$= \frac{-1}{2\sigma^2} \text{RSS}(\mathbf{w}) - \frac{N}{2} \log(2\pi\sigma^2) \quad (7.8)$$

RSS stands for **residual sum of squares** and is defined by

$$\text{RSS}(\mathbf{w}) \triangleq \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 \quad (7.9)$$

The RSS is also called the **sum of squared errors**, or SSE, and SSE/N is called the **mean squared error** or **MSE**. It can also be written as the square of the ℓ_2 **norm** of the vector of

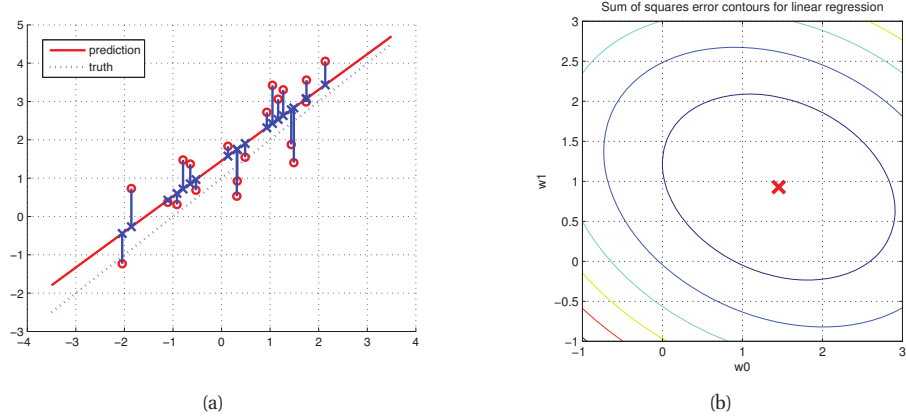


Figure 7.2 (a) In linear least squares, we try to minimize the sum of squared distances from each training point (denoted by a red circle) to its approximation (denoted by a blue cross), that is, we minimize the sum of the lengths of the little vertical blue lines. The red diagonal line represents $\hat{y}(x) = w_0 + w_1x$, which is the least squares regression line. Note that these residual lines are not perpendicular to the least squares line, in contrast to Figure 12.5. Figure generated by `residualsDemo`. (b) Contours of the RSS error surface for the same example. The red cross represents the MLE, $\mathbf{w} = (1.45, 0.93)$. Figure generated by `contoursSSEdemo`.

residual errors:

$$\text{RSS}(\mathbf{w}) = \|\epsilon\|_2^2 = \sum_{i=1}^N \epsilon_i^2 \quad (7.10)$$

where $\epsilon_i = (y_i - \mathbf{w}^T \mathbf{x}_i)$.

We see that the MLE for $\mathbf{w}$ is the one that minimizes the RSS, so this method is known as **least squares**. This method is illustrated in Figure 7.2(a). The training data (x_i, y_i) are shown as red circles, the estimated values $(x_i, \hat{y}_i)$ are shown as blue crosses, and the residuals $\epsilon_i = y_i - \hat{y}_i$ are shown as vertical blue lines. The goal is to find the setting of the parameters (the slope w_1 and intercept w_0) such that the resulting red line minimizes the sum of squared residuals (the lengths of the vertical blue lines).

In Figure 7.2(b), we plot the NLL surface for our linear regression example. We see that it is a quadratic “bowl” with a unique minimum, which we now derive. (Importantly, this is true even if we use basis function expansion, such as polynomials, because the NLL is still *linear in the parameters* $\mathbf{w}$, even if it is not linear in the inputs $\mathbf{x}$.)

7.3.1 Derivation of the MLE

First, we rewrite the objective in a form that is more amenable to differentiation:

$$\text{NLL}(\mathbf{w}) = \frac{1}{2}(\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w}) = \frac{1}{2}\mathbf{w}^T(\mathbf{X}^T\mathbf{X})\mathbf{w} - \mathbf{w}^T(\mathbf{X}^T\mathbf{y}) \quad (7.11)$$

where

$$\mathbf{X}^T \mathbf{X} = \sum_{i=1}^N \mathbf{x}_i \mathbf{x}_i^T = \sum_{i=1}^N \begin{pmatrix} x_{i,1}^2 & \cdots & x_{i,1}x_{i,D} \\ & \ddots & \\ x_{i,D}x_{i,1} & \cdots & x_{i,D}^2 \end{pmatrix} \quad (7.12)$$

is the **sum of squares** matrix and

$$\mathbf{X}^T \mathbf{y} = \sum_{i=1}^N x_i y_i. \quad (7.13)$$

Using results from Equation 4.10, we see that the gradient of this is given by

$$\mathbf{g}(\mathbf{w}) = [\mathbf{X}^T \mathbf{X} \mathbf{w} - \mathbf{X}^T \mathbf{y}] = \sum_{i=1}^N \mathbf{x}_i (\mathbf{w}^T \mathbf{x}_i - y_i) \quad (7.14)$$

Equating to zero we get

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y} \quad (7.15)$$

This is known as the **normal equation**. The corresponding solution $\hat{\mathbf{w}}$ to this linear system of equations is called the **ordinary least squares** or **OLS** solution, which is given by

$$\hat{\mathbf{w}}_{OLS} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

(7.16)

7.3.2 Geometric interpretation

This equation has an elegant geometrical interpretation, as we now explain. We assume $N > D$, so we have more examples than features. The columns of $\mathbf{X}$ define a linear subspace of dimensionality D which is embedded in N dimensions. Let the j 'th column be $\tilde{\mathbf{x}}_j$, which is a vector in $\mathbb{R}^N$. (This should not be confused with $\mathbf{x}_i \in \mathbb{R}^D$, which represents the i 'th data case.) Similarly, $\mathbf{y}$ is a vector in $\mathbb{R}^N$. For example, suppose we have $N = 3$ examples in $D = 2$ dimensions:

$$\mathbf{X} = \begin{pmatrix} 1 & 2 \\ 1 & -2 \\ 1 & 2 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 8.8957 \\ 0.6130 \\ 1.7761 \end{pmatrix} \quad (7.17)$$

These vectors are illustrated in Figure 7.3.

We seek a vector $\hat{\mathbf{y}} \in \mathbb{R}^N$ that lies in this linear subspace and is as close as possible to $\mathbf{y}$, i.e., we want to find

$$\underset{\hat{\mathbf{y}} \in \text{span}(\{\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_D\})}{\text{argmin}} \quad \|\mathbf{y} - \hat{\mathbf{y}}\|_2. \quad (7.18)$$

Since $\hat{\mathbf{y}} \in \text{span}(\mathbf{X})$, there exists some weight vector $\mathbf{w}$ such that

$$\hat{\mathbf{y}} = w_1 \tilde{\mathbf{x}}_1 + \cdots + w_D \tilde{\mathbf{x}}_D = \mathbf{X} \mathbf{w} \quad (7.19)$$

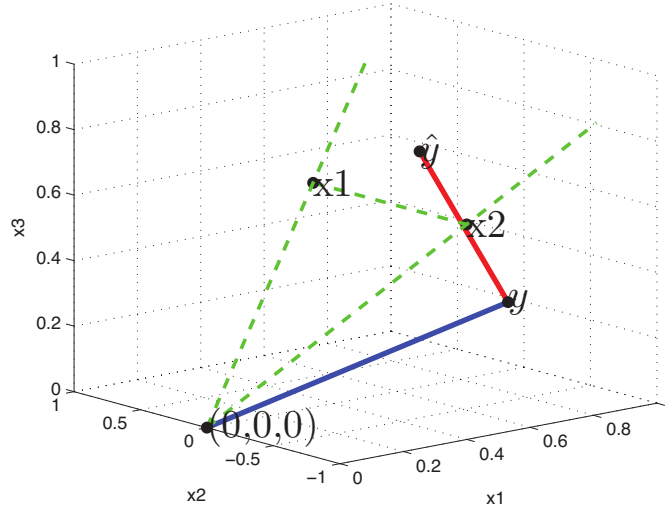


Figure 7.3 Graphical interpretation of least squares for $N = 3$ examples and $D = 2$ features. $\tilde{\mathbf{x}}_1$ and $\tilde{\mathbf{x}}_2$ are vectors in $\mathbb{R}^3$; together they define a 2D plane. $\mathbf{y}$ is also a vector in $\mathbb{R}^3$ but does not lie on this 2D plane. The orthogonal projection of $\mathbf{y}$ onto this plane is denoted $\hat{\mathbf{y}}$. The red line from $\mathbf{y}$ to $\hat{\mathbf{y}}$ is the residual, whose norm we want to minimize. For visual clarity, all vectors have been converted to unit norm. Figure generated by `leastSquaresProjection`.

To minimize the norm of the residual, $\mathbf{y} - \hat{\mathbf{y}}$, we want the residual vector to be orthogonal to every column of $\mathbf{X}$, so $\tilde{\mathbf{x}}_j^T (\mathbf{y} - \hat{\mathbf{y}}) = 0$ for $j = 1 : D$. Hence

$$\tilde{\mathbf{x}}_j^T (\mathbf{y} - \hat{\mathbf{y}}) = 0 \Rightarrow \mathbf{X}^T (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0} \Rightarrow \mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (7.20)$$

Hence our projected value of $\mathbf{y}$ is given by

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}} = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (7.21)$$

This corresponds to an **orthogonal projection** of $\mathbf{y}$ onto the column space of $\mathbf{X}$. The projection matrix $\mathbf{P} \triangleq \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$ is called the **hat matrix**, since it “puts the hat on $\mathbf{y}$ ”.

7.3.3 Convexity

When discussing least squares, we noted that the NLL had a bowl shape with a unique minimum. The technical term for functions like this is **convex**. Convex functions play a very important role in machine learning.

Let us define this concept more precisely. We say a *set* $\mathcal{S}$ is **convex** if for any $\boldsymbol{\theta}, \boldsymbol{\theta}' \in \mathcal{S}$, we have

$$\lambda \boldsymbol{\theta} + (1 - \lambda) \boldsymbol{\theta}' \in \mathcal{S}, \quad \forall \lambda \in [0, 1] \quad (7.22)$$

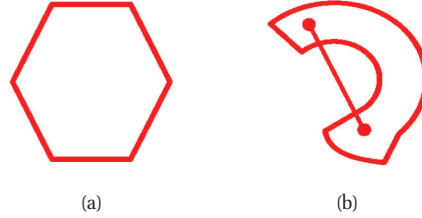


Figure 7.4 (a) Illustration of a convex set. (b) Illustration of a nonconvex set.

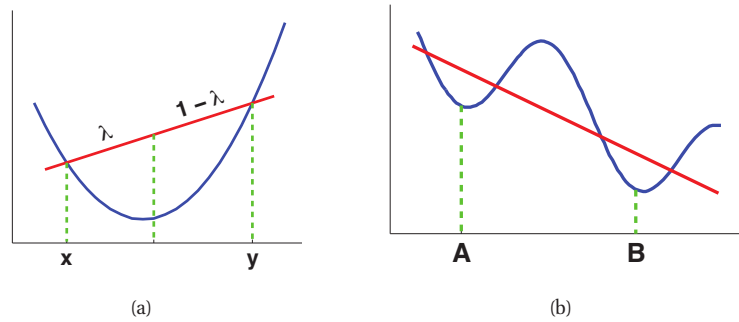


Figure 7.5 (a) Illustration of a convex function. We see that the chord joining $(x, f(x))$ to $(y, f(y))$ lies above the function. (b) A function that is neither convex nor concave. **A** is a local minimum, **B** is a global minimum. Figure generated by `convexFnHand`.

That is, if we draw a line from θ to θ' , all points on the line lie inside the set. See Figure 7.4(a) for an illustration of a convex set, and Figure 7.4(b) for an illustration of a non-convex set.

A function $f(\theta)$ is called **convex** if its **epigraph** (the set of points above the function) defines a convex set. Equivalently, a function $f(\theta)$ is called convex if it is defined on a convex set and if, for any $\theta, \theta' \in S$, and for any $0 \leq \lambda \leq 1$, we have

$$f(\lambda\theta + (1-\lambda)\theta') \leq \lambda f(\theta) + (1-\lambda)f(\theta') \quad (7.23)$$

See Figure 7.5 for a 1d example. A function is called **strictly convex** if the inequality is strict. A function $f(\theta)$ is **concave** if $-f(\theta)$ is convex. Examples of scalar convex functions include θ^2 , e^θ , and $\theta \log \theta$ (for $\theta > 0$). Examples of scalar concave functions include $\log(\theta)$ and $\sqrt{\theta}$.

Intuitively, a (strictly) convex function has a “bowl shape”, and hence has a unique global minimum θ^* corresponding to the bottom of the bowl. Hence its second derivative must be positive everywhere, $\frac{d}{d\theta} f(\theta) > 0$. A twice-continuously differentiable, multivariate function f is convex iff its Hessian is positive definite for all θ .¹ In the machine learning context, the function f often corresponds to the NLL.

1. Recall that the Hessian is the matrix of second partial derivatives, defined by $H_{jk} = \frac{\partial^2 f(\theta)}{\partial \theta_j \partial \theta_k}$. Also, recall that a matrix $\mathbf{H}$ is **positive definite** iff $\mathbf{v}^T \mathbf{H} \mathbf{v} > 0$ for any non-zero vector $\mathbf{v}$.

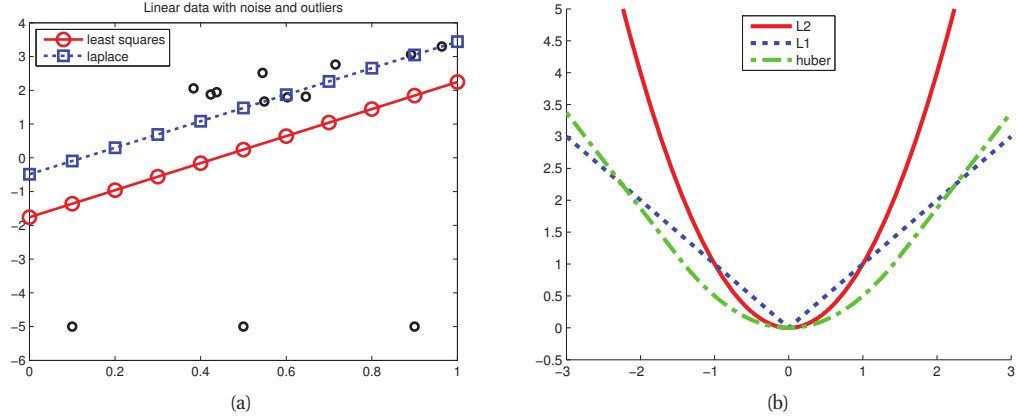


Figure 7.6 (a) Illustration of robust linear regression. Figure generated by `linregRobustDemoCombined`. (b) Illustration of ℓ_2 , ℓ_1 , and Huber loss functions. Figure generated by `huberLossDemo`.

Models where the NLL is convex are desirable, since this means we can always find the globally optimal MLE. We will see many examples of this later in the book. However, many models of interest will not have concave likelihoods. In such cases, we will discuss ways to derive locally optimal parameter estimates.

7.4 Robust linear regression *

It is very common to model the noise in regression models using a Gaussian distribution with zero mean and constant variance, $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$, where $\epsilon_i = y_i - \mathbf{w}^T \mathbf{x}_i$. In this case, maximizing likelihood is equivalent to minimizing the sum of squared residuals, as we have seen. However, if we have **outliers** in our data, this can result in a poor fit, as illustrated in Figure 7.6(a). (The outliers are the points on the bottom of the figure.) This is because squared error penalizes deviations quadratically, so points far from the line have more affect on the fit than points near to the line.

One way to achieve **robustness** to outliers is to replace the Gaussian distribution for the response variable with a distribution that has **heavy tails**. Such a distribution will assign higher likelihood to outliers, without having to perturb the straight line to “explain” them.

One possibility is to use the Laplace distribution, introduced in Section 2.4.3. If we use this as our observation model for regression, we get the following likelihood:

$$p(y|\mathbf{x}, \mathbf{w}, b) = \text{Lap}(y|\mathbf{w}^T \mathbf{x}, b) \propto \exp\left(-\frac{1}{b}|y - \mathbf{w}^T \mathbf{x}|\right) \quad (7.24)$$

The robustness arises from the use of $|y - \mathbf{w}^T \mathbf{x}|$ instead of $(y - \mathbf{w}^T \mathbf{x})^2$. For simplicity, we will assume b is fixed. Let $r_i \triangleq y_i - \mathbf{w}^T \mathbf{x}_i$ be the i 'th residual. The NLL has the form

$$\ell(\mathbf{w}) = \sum_i |r_i(\mathbf{w})| \quad (7.25)$$

Likelihood	Prior	Name	Section
Gaussian	Uniform	Least squares	7.3
Gaussian	Gaussian	Ridge	7.5
Gaussian	Laplace	Lasso	13.3
Laplace	Uniform	Robust regression	7.4
Student	Uniform	Robust regression	Exercise 11.12

Table 7.1 Summary of various likelihoods and priors used for linear regression. The likelihood refers to the distributional form of $p(y|\mathbf{x}, \mathbf{w}, \sigma^2)$, and the prior refers to the distributional form of $p(\mathbf{w})$. MAP estimation with a uniform distribution corresponds to MLE.

Unfortunately, this is a non-linear objective function, which is hard to optimize. Fortunately, we can convert the NLL to a linear objective, subject to linear constraints, using the following **split variable** trick. First we define

$$r_i \triangleq r_i^+ - r_i^- \quad (7.26)$$

and then we impose the linear inequality constraints that $r_i^+ \geq 0$ and $r_i^- \geq 0$. Now the constrained objective becomes

$$\min_{\mathbf{w}, \mathbf{r}^+, \mathbf{r}^-} \sum_i (r_i^+ - r_i^-) \quad \text{s.t.} \quad r_i^+ \geq 0, r_i^- \geq 0, \mathbf{w}^T \mathbf{x}_i + r_i^+ - r_i^- = y_i \quad (7.27)$$

This is an example of a **linear program** with $D + 2N$ unknowns and $3N$ constraints.

Since this is a convex optimization problem, it has a unique solution. To solve an LP, we must first write it in standard form, which as follows:

$$\min_{\boldsymbol{\theta}} \mathbf{f}^T \boldsymbol{\theta} \quad \text{s.t.} \quad \mathbf{A} \boldsymbol{\theta} \leq \mathbf{b}, \mathbf{A}_{eq} \boldsymbol{\theta} = \mathbf{b}_{eq}, \mathbf{l} \leq \boldsymbol{\theta} \leq \mathbf{u} \quad (7.28)$$

In our current example, $\boldsymbol{\theta} = (\mathbf{w}, \mathbf{r}^+, \mathbf{r}^-)$, $\mathbf{f} = [0, 1, 1]$, $\mathbf{A} = \mathbf{I}$, $\mathbf{b} = \mathbf{0}$, $\mathbf{A}_{eq} = [\mathbf{X}, \mathbf{I}, -\mathbf{I}]$, $\mathbf{b}_{eq} = \mathbf{y}$, $\mathbf{l} = [-\infty \mathbf{1}, \mathbf{0}, \mathbf{0}]$, $\mathbf{u} = \mathbf{0}$. This can be solved by any LP solver (see e.g., (Boyd and Vandenberghe 2004)). See Figure 7.6(a) for an example of the method in action.

An alternative to using NLL under a Laplace likelihood is to minimize the **Huber loss** function (Huber 1964), defined as follows:

$$L_H(r, \delta) = \begin{cases} r^2/2 & \text{if } |r| \leq \delta \\ \delta|r| - \delta^2/2 & \text{if } |r| > \delta \end{cases} \quad (7.29)$$

This is equivalent to ℓ_2 for errors that are smaller than δ , and is equivalent to ℓ_1 for larger errors. See Figure 7.6(b). The advantage of this loss function is that it is everywhere differentiable, using the fact that $\frac{d}{dr}|r| = \text{sign}(r)$ if $r \neq 0$. We can also check that the function is C_1 continuous, since the gradients of the two parts of the function match at $r = \pm\delta$, namely $\frac{d}{dr}L_H(r, \delta)|_{r=\delta} = \delta$. Consequently optimizing the Huber loss is much faster than using the Laplace likelihood, since we can use standard smooth optimization methods (such as quasi-Newton) instead of linear programming.

Figure 7.6(a) gives an illustration of the Huber loss function. The results are qualitatively similar to the probabilistic methods. (In fact, it turns out that the Huber method also has a probabilistic interpretation, although it is rather unnatural (Pontil et al. 1998).)

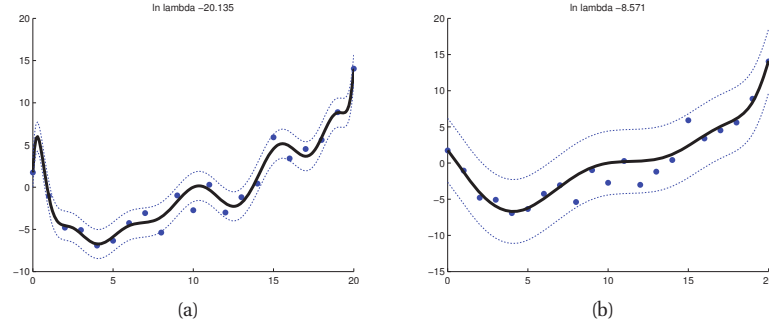


Figure 7.7 Degree 14 Polynomial fit to $N = 21$ data points with increasing amounts of ℓ_2 regularization. Data was generated from noise with variance $\sigma^2 = 4$. The error bars, representing the noise variance σ^2 , get wider as the fit gets smoother, since we are ascribing more of the data variation to the noise. Figure generated by `linregPolyVsRegDemo`.

7.5 Ridge regression

One problem with ML estimation is that it can result in overfitting. In this section, we discuss a way to ameliorate this problem by using MAP estimation with a Gaussian prior. For simplicity, we assume a Gaussian likelihood, rather than a robust likelihood.

7.5.1 Basic idea

The reason that the MLE can overfit is that it is picking the parameter values that are the best for modeling the training data; but if the data is noisy, such parameters often result in complex functions. As a simple example, suppose we fit a degree 14 polynomial to $N = 21$ data points using least squares. The resulting curve is very “wiggly”, as shown in Figure 7.7(a). The corresponding least squares coefficients (excluding w_0) are as follows:

6.560, -36.934, -109.255, 543.452, 1022.561, -3046.224, -3768.013,
8524.540, 6607.897, -12640.058, -5530.188, 9479.730, 1774.639, -2821.526

We see that there are many large positive and negative numbers. These balance out exactly to make the curve “wiggle” in just the right way so that it almost perfectly interpolates the data. But this situation is unstable: if we changed the data a little, the coefficients would change a lot.

We can encourage the parameters to be small, thus resulting in a smoother curve, by using a zero-mean Gaussian prior:

$$p(\mathbf{w}) = \prod_j \mathcal{N}(w_j | 0, \tau^2) \quad (7.30)$$

where $1/\tau^2$ controls the strength of the prior. The corresponding MAP estimation problem becomes

$$\arg\max_{\mathbf{w}} \sum_{i=1}^N \log \mathcal{N}(y_i | w_0 + \mathbf{w}^T \mathbf{x}_i, \sigma^2) + \sum_{j=1}^D \log \mathcal{N}(w_j | 0, \tau^2) \quad (7.31)$$

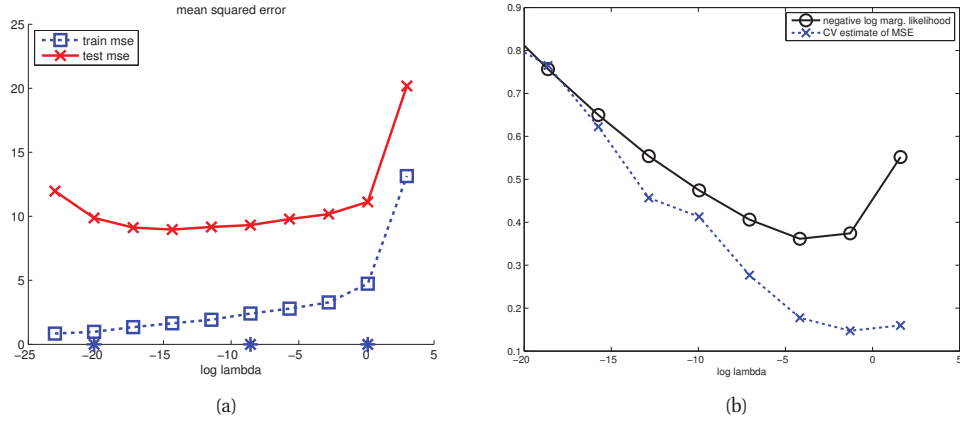


Figure 7.8 (a) Training error (dotted blue) and test error (solid red) for a degree 14 polynomial fit by ridge regression, plotted vs $\log(\lambda)$. Data was generated from noise with variance $\sigma^2 = 4$ (training set has size $N = 21$). Note: Models are ordered from complex (small regularizer) on the left to simple (large regularizer) on the right. The stars correspond to the values used to plot the functions in Figure 7.7. (b) Estimate of performance using training set. Dotted blue: 5-fold cross-validation estimate of future MSE. Solid black: negative log marginal likelihood, $-\log p(\mathcal{D}|\lambda)$. Both curves have been vertically rescaled to $[0,1]$ to make them comparable. Figure generated by `linregPolyVsRegDemo`.

It is a simple exercise to show that this is equivalent to minimizing the following:

$$J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + \mathbf{w}^T \mathbf{x}_i))^2 + \lambda \|\mathbf{w}\|_2^2 \quad (7.32)$$

where $\lambda \triangleq \sigma^2/\tau^2$ and $\|\mathbf{w}\|_2^2 = \sum_j w_j^2 = \mathbf{w}^T \mathbf{w}$ is the squared two-norm. Here the first term is the MSE/ NLL as usual, and the second term, $\lambda \geq 0$, is a complexity penalty. The corresponding solution is given by

$$\hat{\mathbf{w}}_{\text{ridge}} = (\lambda \mathbf{I}_D + \mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (7.33)$$

This technique is known as **ridge regression**, or **penalized least squares**. In general, adding a Gaussian prior to the parameters of a model to encourage them to be small is called ℓ_2 **regularization** or **weight decay**. Note that the offset term w_0 is not regularized, since this just affects the height of the function, not its complexity. By penalizing the sum of the magnitudes of the weights, we ensure the function is simple (since $\mathbf{w} = \mathbf{0}$ corresponds to a straight line, which is the simplest possible function, corresponding to a constant.)

We illustrate this idea in Figure 7.7, where we see that increasing λ results in smoother functions. The resulting coefficients also become smaller. For example, using $\lambda = 10^{-3}$, we have

2.128, 0.807, 16.457, 3.704, -24.948, -10.472, -2.625, 4.360, 13.711, 10.063, 8.716, 3.966, -9.349, -9.232

In Figure 7.8(a), we plot the MSE on the training and test sets vs $\log(\lambda)$. We see that, as we increase λ (so the model becomes more constrained), the error on the training set increases. For the test set, we see the characteristic U-shaped curve, where the model overfits and then underfits. It is common to use cross validation to pick λ , as shown in Figure 7.8(b). In Section 1.4.8, we will discuss a more probabilistic approach.

We will consider a variety of different priors in this book. Each of these corresponds to a different form of **regularization**. This technique is very widely used to prevent overfitting.

7.5.2 Numerically stable computation *

Interestingly, ridge regression, which works better statistically, is also easier to fit numerically, since $(\lambda \mathbf{I}_D + \mathbf{X}^T \mathbf{X})$ is much better conditioned (and hence more likely to be invertible) than $\mathbf{X}^T \mathbf{X}$, at least for suitable large λ .

Nevertheless, inverting matrices is still best avoided, for reasons of numerical stability. (Indeed, if you write `w=inv(X' * X)*X'*y` in Matlab, it will give you a warning.) We now describe a useful trick for fitting ridge regression models (and hence by extension, computing vanilla OLS estimates) that is more numerically robust. We assume the prior has the form $p(\mathbf{w}) = \mathcal{N}(\mathbf{0}, \Lambda^{-1})$, where Λ is the precision matrix. In the case of ridge regression, $\Lambda = (1/\tau^2)\mathbf{I}$. To avoid penalizing the w_0 term, we should center the data first, as explained in Exercise 7.5.

First let us augment the original data with some “virtual data” coming from the prior:

$$\tilde{\mathbf{X}} = \begin{pmatrix} \mathbf{X}/\sigma \\ \sqrt{\Lambda} \end{pmatrix}, \quad \tilde{\mathbf{y}} = \begin{pmatrix} \mathbf{y}/\sigma \\ \mathbf{0}_{D \times 1} \end{pmatrix} \quad (7.34)$$

where $\Lambda = \sqrt{\Lambda}\sqrt{\Lambda}^T$ is a **Cholesky decomposition** of Λ . We see that $\tilde{\mathbf{X}}$ is $(N + D) \times D$, where the extra rows represent pseudo-data from the prior.

We now show that the NLL on this expanded data is equivalent to *penalized* NLL on the original data:

$$f(\mathbf{w}) = (\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\mathbf{w})^T (\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\mathbf{w}) \quad (7.35)$$

$$= \left(\begin{pmatrix} \mathbf{y}/\sigma \\ \mathbf{0} \end{pmatrix} - \begin{pmatrix} \mathbf{X}/\sigma \\ \sqrt{\Lambda} \end{pmatrix} \mathbf{w} \right)^T \left(\begin{pmatrix} \mathbf{y}/\sigma \\ \mathbf{0} \end{pmatrix} - \begin{pmatrix} \mathbf{X}/\sigma \\ \sqrt{\Lambda} \end{pmatrix} \mathbf{w} \right) \quad (7.36)$$

$$= \begin{pmatrix} \frac{1}{\sigma}(\mathbf{y} - \mathbf{X}\mathbf{w}) \\ -\sqrt{\Lambda}\mathbf{w} \end{pmatrix}^T \begin{pmatrix} \frac{1}{\sigma}(\mathbf{y} - \mathbf{X}\mathbf{w}) \\ -\sqrt{\Lambda}\mathbf{w} \end{pmatrix} \quad (7.37)$$

$$= \frac{1}{\sigma^2}(\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}) + (\sqrt{\Lambda}\mathbf{w})^T (\sqrt{\Lambda}\mathbf{w}) \quad (7.38)$$

$$= \frac{1}{\sigma^2}(\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}) + \mathbf{w}^T \Lambda \mathbf{w} \quad (7.39)$$

Hence the MAP estimate is given by

$$\hat{\mathbf{w}}_{ridge} = (\tilde{\mathbf{X}}^T \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^T \tilde{\mathbf{y}} \quad (7.40)$$

as we claimed.

Now let

$$\tilde{\mathbf{X}} = \mathbf{Q}\mathbf{R} \quad (7.41)$$

be the **QR decomposition** of $\mathbf{X}$, where $\mathbf{Q}$ is orthonormal (meaning $\mathbf{Q}^T\mathbf{Q} = \mathbf{Q}\mathbf{Q}^T = \mathbf{I}$), and $\mathbf{R}$ is upper triangular. Then

$$(\tilde{\mathbf{X}}^T\tilde{\mathbf{X}})^{-1} = (\mathbf{R}^T\mathbf{Q}^T\mathbf{Q}\mathbf{R})^{-1} = (\mathbf{R}^T\mathbf{R})^{-1} = \mathbf{R}^{-1}\mathbf{R}^{-T} \quad (7.42)$$

Hence

$$\hat{\mathbf{w}}_{ridge} = \mathbf{R}^{-1}\mathbf{R}^{-T}\mathbf{R}^T\mathbf{Q}^T\tilde{\mathbf{y}} = \mathbf{R}^{-1}\mathbf{Q}\tilde{\mathbf{y}} \quad (7.43)$$

Note that $\mathbf{R}$ is easy to invert since it is upper triangular. This gives us a way to compute the ridge estimate while avoiding having to invert $(\mathbf{A} + \mathbf{X}^T\mathbf{X})$.

We can use this technique to find the MLE, by simply computing the QR decomposition of the unaugmented matrix $\mathbf{X}$, and using the original $\mathbf{y}$. This is the method of choice for solving least squares problems. (In fact, it is so common that it can be implemented in one line of Matlab, using the **backslash operator**: `w=X\y`.) Note that computing the QR decomposition of an $N \times D$ matrix takes $O(ND^2)$ time, and is numerically very stable.

If $D \gg N$, we should first perform an SVD decomposition. In particular, let $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T$ be the SVD of $\mathbf{X}$, where $\mathbf{V}^T\mathbf{V} = \mathbf{I}_N$, $\mathbf{U}\mathbf{U}^T = \mathbf{U}^T\mathbf{U} = \mathbf{I}_N$, and $\mathbf{S}$ is a diagonal $N \times N$ matrix. Now let $\mathbf{Z} = \mathbf{U}\mathbf{D}$ be an $N \times N$ matrix. Then we can rewrite the ridge estimate thus:

$$\hat{\mathbf{w}}_{ridge} = \mathbf{V}(\mathbf{Z}^T\mathbf{Z} + \lambda\mathbf{I}_N)^{-1}\mathbf{Z}^T\mathbf{y} \quad (7.44)$$

In other words, we can replace the D -dimensional vectors $\mathbf{x}_i$ with the N -dimensional vectors $\mathbf{z}_i$ and perform our penalized fit as before. We then transform the N -dimensional solution to the D -dimensional solution by multiplying by $\mathbf{V}$. Geometrically, we are rotating to a new coordinate system in which all but the first N coordinates are zero. This does not affect the solution since the spherical Gaussian prior is rotationally invariant. The overall time is now $O(DN^2)$ operations.

7.5.3 Connection with PCA *

In this section, we discuss an interesting connection between ridge regression and PCA (Section 12.2), which gives further insight into why ridge regression works well. Our discussion is based on (Hastie et al. 2009, p66).

Let $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T$ be the SVD of $\mathbf{X}$. From Equation 7.44, we have

$$\hat{\mathbf{w}}_{ridge} = \mathbf{V}(\mathbf{S}^2 + \lambda\mathbf{I})^{-1}\mathbf{S}\mathbf{U}^T\mathbf{y} \quad (7.45)$$

Hence the ridge predictions on the training set are given by

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}}_{ridge} = \mathbf{U}\mathbf{S}\mathbf{V}^T\mathbf{V}(\mathbf{S}^2 + \lambda\mathbf{I})^{-1}\mathbf{S}\mathbf{U}^T\mathbf{y} \quad (7.46)$$

$$= \mathbf{U}\tilde{\mathbf{S}}\mathbf{U}^T\mathbf{y} = \sum_{j=1}^D \mathbf{u}_j\tilde{S}_{jj}\mathbf{u}_j^T\mathbf{y} \quad (7.47)$$

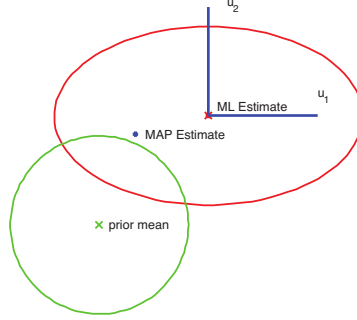


Figure 7.9 Geometry of ridge regression. The likelihood is shown as an ellipse, and the prior is shown as a circle centered on the origin. Based on Figure 3.15 of (Bishop 2006b). Figure generated by `geomRidge`

where

$$\tilde{S}_{jj} \triangleq [\mathbf{S}(\mathbf{S}^2 + \lambda \mathbf{I})^{-1} \mathbf{S}]_{jj} = \frac{\sigma_j^2}{\sigma_j^2 + \lambda} \quad (7.48)$$

and σ_j are the singular values of $\mathbf{X}$. Hence

$$\hat{\mathbf{y}} = \mathbf{X} \hat{\mathbf{w}}_{ridge} = \sum_{j=1}^D \mathbf{u}_j \frac{\sigma_j^2}{\sigma_j^2 + \lambda} \mathbf{u}_j^T \mathbf{y} \quad (7.49)$$

In contrast, the least squares prediction is

$$\hat{\mathbf{y}} = \mathbf{X} \hat{\mathbf{w}}_{ls} = (\mathbf{U} \mathbf{S} \mathbf{V}^T)(\mathbf{V} \mathbf{S}^{-1} \mathbf{U}^T \mathbf{y}) = \mathbf{U} \mathbf{U}^T \mathbf{y} = \sum_{j=1}^D \mathbf{u}_j \mathbf{u}_j^T \mathbf{y} \quad (7.50)$$

If σ_j^2 is small compared to λ , then direction $\mathbf{u}_j$ will not have much effect on the prediction. In view of this, we *define* the effective number of **degrees of freedom** of the model as follows:

$$\text{dof}(\lambda) = \sum_{j=1}^D \frac{\sigma_j^2}{\sigma_j^2 + \lambda} \quad (7.51)$$

When $\lambda = 0$, $\text{dof}(\lambda) = D$, and as $\lambda \rightarrow \infty$, $\text{dof}(\lambda) \rightarrow 0$.

Let us try to understand why this behavior is desirable. In Section 7.6, we show that $\text{cov}[\mathbf{w}|\mathcal{D}] = \sigma^2(\mathbf{X}^T \mathbf{X})^{-1}$, if we use a uniform prior for $\mathbf{w}$. Thus the directions in which we are most uncertain about $\mathbf{w}$ are determined by the eigenvectors of this matrix with the smallest eigenvalues, as shown in Figure 4.1. Furthermore, in Section 12.2.3, we show that the squared singular values σ_j^2 are equal to the eigenvalues of $\mathbf{X}^T \mathbf{X}$. Hence small singular values σ_j correspond to directions with high posterior variance. It is these directions which ridge shrinks the most.

This process is illustrated in Figure 7.9. The horizontal w_1 parameter is not well determined by the data (has high posterior variance), but the vertical w_2 parameter is well-determined. Hence w_2^{map} is close to $\hat{w}_2^{mle}$, but w_1^{map} is shifted strongly towards the prior mean, which is 0. (Compare to Figure 4.14(c), which illustrated sensor fusion with sensors of different reliabilities.) In this way, ill-determined parameters are reduced in size towards 0. This is called **shrinkage**.

There is a related, but different, technique called **principal components regression**. The idea is this: first use PCA to reduce the dimensionality to K dimensions, and then use these low dimensional features as input to regression. However, this technique does not work as well as ridge in terms of predictive accuracy (Hastie et al. 2001, p70). The reason is that in PC regression, only the first K (derived) dimensions are retained, and the remaining $D - K$ dimensions are entirely ignored. By contrast, ridge regression uses a “soft” weighting of all the dimensions.

7.5.4 Regularization effects of big data

Regularization is the most common way to avoid overfitting. However, another effective approach — which is not always available — is to use lots of data. It should be intuitively obvious that the more training data we have, the better we will be able to learn.² So we expect the test set error to decrease to some plateau as N increases.

This is illustrated in Figure 7.10, where we plot the mean squared error incurred on the test set achieved by polynomial regression models of different degrees vs N (a plot of error vs training set size is known as a **learning curve**). The level of the plateau for the test error consists of two terms: an irreducible component that all models incur, due to the intrinsic variability of the generating process (this is called the **noise floor**); and a component that depends on the discrepancy between the generating process (the “truth”) and the model: this is called **structural error**.

In Figure 7.10, the truth is a degree 2 polynomial, and we try fitting polynomials of degrees 1, 2 and 25 to this data. Call the 3 models $\mathcal{M}_1$, $\mathcal{M}_2$ and $\mathcal{M}_{25}$. We see that the structural error for models $\mathcal{M}_2$ and $\mathcal{M}_{25}$ is zero, since both are able to capture the true generating process. However, the structural error for $\mathcal{M}_1$ is substantial, which is evident from the fact that the plateau occurs high above the noise floor.

For any model that is expressive enough to capture the truth (i.e., one with small structural error), the test error will go to the noise floor as $N \rightarrow \infty$. However, it will typically go to zero faster for simpler models, since there are fewer parameters to estimate. In particular, for finite training sets, there will be some discrepancy between the parameters that we estimate and the best parameters that we could estimate given the particular model class. This is called **approximation error**, and goes to zero as $N \rightarrow \infty$, but it goes to zero faster for simpler models. This is illustrated in Figure 7.10. See also Exercise 7.1.

In domains with lots of data, simple methods can work surprisingly well (Halevy et al. 2009). However, there are still reasons to study more sophisticated learning methods, because there will always be problems for which we have little data. For example, even in such a data-rich domain as web search, as soon as we want to start personalizing the results, the amount of data available for any given user starts to look small again (relative to the complexity of the problem).

2. This assumes the training data is randomly sampled, and we don't just get repetitions of the same examples. Having informatively sampled data can help even more; this is the motivation for an approach known as active learning, where you get to choose your training data.

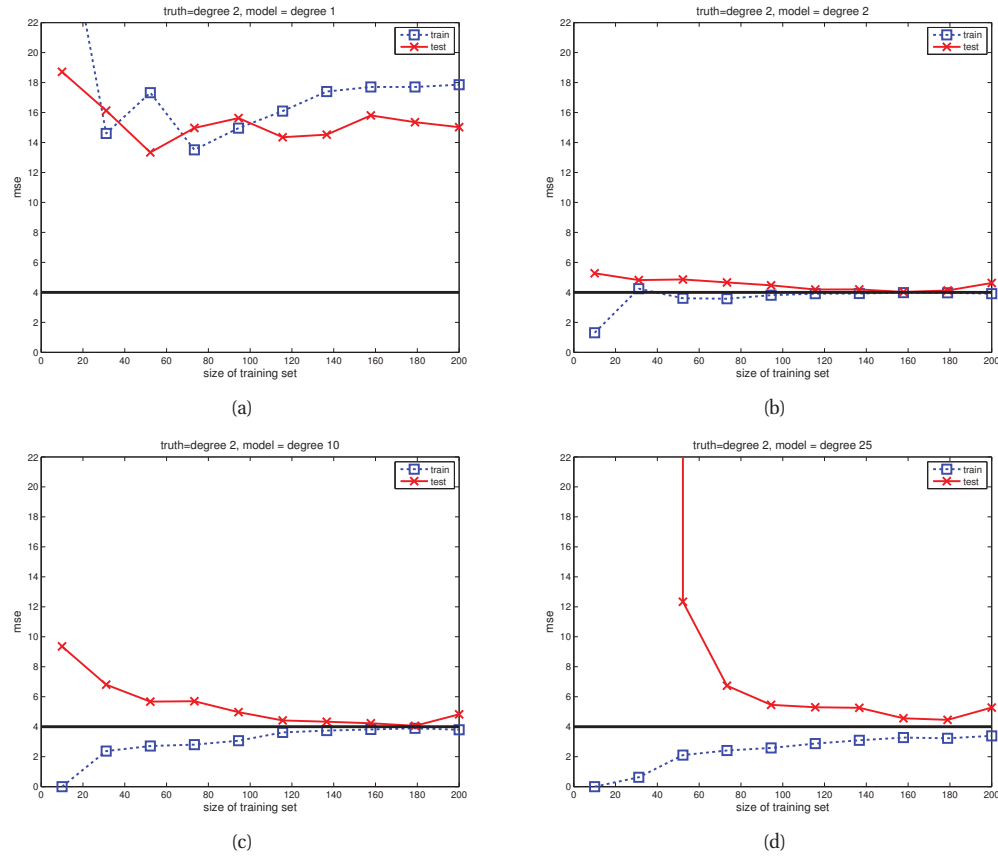


Figure 7.10 MSE on training and test sets vs size of training set, for data generated from a degree 2 polynomial with Gaussian noise of variance $\sigma^2 = 4$. We fit polynomial models of varying degree to this data. (a) Degree 1. (b) Degree 2. (c) Degree 10. (d) Degree 25. Note that for small training set sizes, the test error of the degree 25 polynomial is higher than that of the degree 2 polynomial, due to overfitting, but this difference vanishes once we have enough data. Note also that the degree 1 polynomial is too simple and has high test error even given large amounts of training data. Figure generated by `linregPolyVsN`.

In such cases, we may want to learn multiple related models at the same time, which is known as multi-task learning. This will allow us to “borrow statistical strength” from tasks with lots of data and to share it with tasks with little data. We will discuss ways to do later in the book.

7.6 Bayesian linear regression

Although ridge regression is a useful way to compute a point estimate, sometimes we want to compute the full posterior over $\mathbf{w}$ and σ^2 . For simplicity, we will initially assume the noise variance σ^2 is known, so we focus on computing $p(\mathbf{w}|\mathcal{D}, \sigma^2)$. Then in Section 7.6.3 we consider

the general case, where we compute $p(\mathbf{w}, \sigma^2 | \mathcal{D})$. We assume throughout a Gaussian likelihood model. Performing Bayesian inference with a robust likelihood is also possible, but requires more advanced techniques (see Exercise 24.5).

7.6.1 Computing the posterior

In linear regression, the likelihood is given by

$$p(\mathbf{y} | \mathbf{X}, \mathbf{w}, \mu, \sigma^2) = \mathcal{N}(\mathbf{y} | \mu + \mathbf{X}\mathbf{w}, \sigma^2 \mathbf{I}_N) \quad (7.52)$$

$$\propto \exp \left(-\frac{1}{2\sigma^2} (\mathbf{y} - \mu \mathbf{1}_N - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mu \mathbf{1}_N - \mathbf{X}\mathbf{w}) \right) \quad (7.53)$$

where μ is an offset term. If the inputs are centered, so $\sum_i x_{ij} = 0$ for each j , the mean of the output is equally likely to be positive or negative. So let us put an improper prior on μ of the form $p(\mu) \propto 1$, and then integrate it out to get

$$p(\mathbf{y} | \mathbf{X}, \mathbf{w}, \sigma^2) \propto \exp \left(-\frac{1}{2\sigma^2} \|\mathbf{y} - \bar{y} \mathbf{1}_N - \mathbf{X}\mathbf{w}\|_2^2 \right) \quad (7.54)$$

where $\bar{y} = \frac{1}{N} \sum_{i=1}^N y_i$ is the empirical mean of the output. For notational simplicity, we shall assume the output has been centered, and write $\mathbf{y}$ for $\mathbf{y} - \bar{y} \mathbf{1}_N$.

The conjugate prior to the above Gaussian likelihood is also a Gaussian, which we will denote by $p(\mathbf{w}) = \mathcal{N}(\mathbf{w} | \mathbf{w}_0, \mathbf{V}_0)$. Using Bayes rule for Gaussians, Equation 4.125, the posterior is given by

$$p(\mathbf{w} | \mathbf{X}, \mathbf{y}, \sigma^2) \propto \mathcal{N}(\mathbf{w} | \mathbf{w}_0, \mathbf{V}_0) \mathcal{N}(\mathbf{y} | \mathbf{X}\mathbf{w}, \sigma^2 \mathbf{I}_N) = \mathcal{N}(\mathbf{w} | \mathbf{w}_N, \mathbf{V}_N) \quad (7.55)$$

$$\mathbf{w}_N = \mathbf{V}_N \mathbf{V}_0^{-1} \mathbf{w}_0 + \frac{1}{\sigma^2} \mathbf{V}_N \mathbf{X}^T \mathbf{y} \quad (7.56)$$

$$\mathbf{V}_N^{-1} = \mathbf{V}_0^{-1} + \frac{1}{\sigma^2} \mathbf{X}^T \mathbf{X} \quad (7.57)$$

$$\mathbf{V}_N = \sigma^2 (\sigma^2 \mathbf{V}_0^{-1} + \mathbf{X}^T \mathbf{X})^{-1} \quad (7.58)$$

If $\mathbf{w}_0 = \mathbf{0}$ and $\mathbf{V}_0 = \tau^2 \mathbf{I}$, then the posterior mean reduces to the ridge estimate, if we define $\lambda = \frac{\sigma^2}{\tau^2}$. This is because the mean and mode of a Gaussian are the same.

To gain insight into the posterior distribution (and not just its mode), let us consider a 1D example:

$$y(x, \mathbf{w}) = w_0 + w_1 x + \epsilon \quad (7.59)$$

where the “true” parameters are $w_0 = -0.3$ and $w_1 = 0.5$. In Figure 7.11 we plot the prior, the likelihood, the posterior, and some samples from the posterior predictive. In particular, the right hand column plots the function $y(x, \mathbf{w}^{(s)})$ where x ranges over $[-1, 1]$, and $\mathbf{w}^{(s)} \sim \mathcal{N}(\mathbf{w} | \mathbf{w}_N, \mathbf{V}_N)$ is a sample from the parameter posterior. Initially, when we sample from the prior (first row), our predictions are “all over the place”, since our prior is uniform. After we see one data point (second row), our posterior becomes constrained by the corresponding likelihood, and our predictions pass close to the observed data. However, we see that the posterior has a ridge-like shape, reflecting the fact that there are many possible solutions, with different

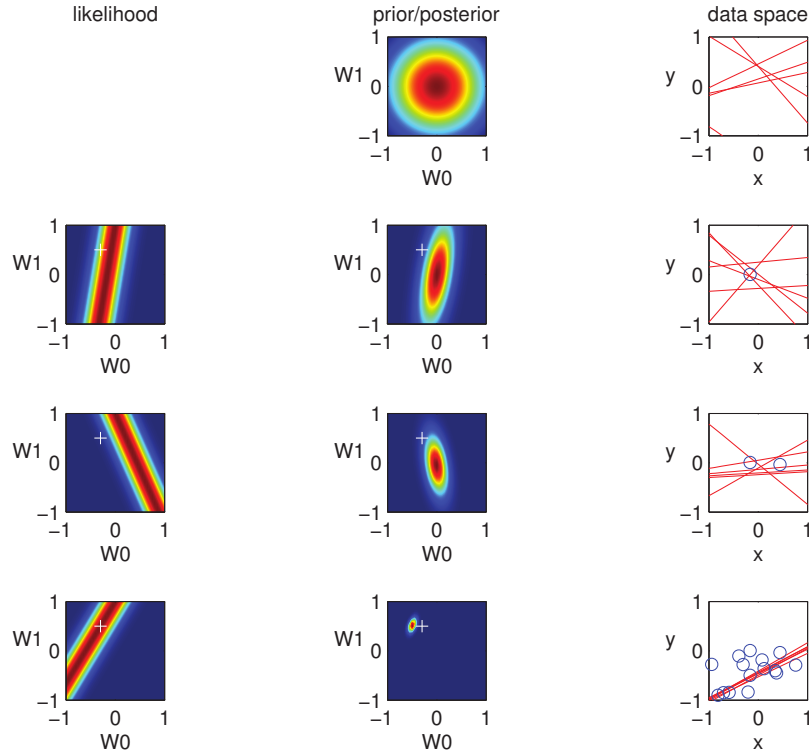


Figure 7.11 Sequential Bayesian updating of a linear regression model $p(y|\mathbf{x}) = \mathcal{N}(y|w_0x_0 + w_1x_1, \sigma^2)$. Row 0 represents the prior, row 1 represents the first data point (x_1, y_1) , row 2 represents the second data point (x_2, y_2) , row 3 represents the 20th data point (x_{20}, y_{20}) . Left column: likelihood function for current data point. Middle column: posterior given data so far, $p(\mathbf{w}|\mathbf{x}_{1:n}, y_{1:n})$ (so the first line is the prior). Right column: samples from the current prior/posterior predictive distribution. The white cross in columns 1 and 2 represents the true parameter value; we see that the mode of the posterior rapidly (after 20 samples) converges to this point. The blue circles in column 3 are the observed data points. Based on Figure 3.7 of (Bishop 2006a). Figure generated by `bayesLinRegDemo2d`.

slopes/intercepts. This makes sense since we cannot uniquely infer two parameters from one observation. After we see two data points (third row), the posterior becomes much narrower, and our predictions all have similar slopes and intercepts. After we observe 20 data points (last row), the posterior is essentially a delta function centered on the true value, indicated by a white cross. (The estimate converges to the truth since the data was generated from this model, and because Bayes is a consistent estimator; see Section 6.4.1 for discussion of this point.)

7.6.2 Computing the posterior predictive

It's tough to make predictions, especially about the future. — Yogi Berra

In machine learning, we often care more about predictions than about interpreting the parameters. Using Equation 4.126, we can easily show that the posterior predictive distribution at a test point $\mathbf{x}$ is also Gaussian:

$$p(y|\mathbf{x}, \mathcal{D}, \sigma^2) = \int \mathcal{N}(y|\mathbf{x}^T \mathbf{w}, \sigma^2) \mathcal{N}(\mathbf{w}|\mathbf{w}_N, \mathbf{V}_N) d\mathbf{w} \quad (7.60)$$

$$= \mathcal{N}(y|\mathbf{w}_N^T \mathbf{x}, \sigma_N^2(\mathbf{x})) \quad (7.61)$$

$$\sigma_N^2(\mathbf{x}) = \sigma^2 + \mathbf{x}^T \mathbf{V}_N \mathbf{x} \quad (7.62)$$

The variance in this prediction, $\sigma_N^2(\mathbf{x})$, depends on two terms: the variance of the observation noise, σ^2 , and the variance in the parameters, $\mathbf{V}_N$. The latter translates into variance about observations in a way which depends on how close $\mathbf{x}$ is to the training data $\mathcal{D}$. This is illustrated in Figure 7.12, where we see that the error bars get larger as we move away from the training points, representing increased uncertainty. This is important for applications such as active learning, where we want to model what we don't know as well as what we do. By contrast, the plugin approximation has constant sized error bars, since

$$p(y|\mathbf{x}, \mathcal{D}, \sigma^2) \approx \int \mathcal{N}(y|\mathbf{x}^T \mathbf{w}, \sigma^2) \delta_{\hat{\mathbf{w}}}(\mathbf{w}) d\mathbf{w} = p(y|\mathbf{x}, \hat{\mathbf{w}}, \sigma^2) \quad (7.63)$$

See Figure 7.12(a).

7.6.3 Bayesian inference when σ^2 is unknown *

In this section, we apply the results in Section 4.6.3 to the problem of computing $p(\mathbf{w}, \sigma^2|\mathcal{D})$ for a linear regression model. This generalizes the results from Section 7.6.1 where we assumed σ^2 was known. In the case where we use an uninformative prior, we will see some interesting connections to frequentist statistics.

7.6.3.1 Conjugate prior

As usual, the likelihood has the form

$$p(\mathbf{y}|\mathbf{X}, \mathbf{w}, \sigma^2) = \mathcal{N}(\mathbf{y}|\mathbf{X}\mathbf{w}, \sigma^2 \mathbf{I}_N) \quad (7.64)$$

By analogy to Section 4.6.3, one can show that the natural conjugate prior has the following form:

$$p(\mathbf{w}, \sigma^2) = \text{NIG}(\mathbf{w}, \sigma^2|\mathbf{w}_0, \mathbf{V}_0, a_0, b_0) \quad (7.65)$$

$$\triangleq \mathcal{N}(\mathbf{w}|\mathbf{w}_0, \sigma^2 \mathbf{V}_0) \text{IG}(\sigma^2|a_0, b_0) \quad (7.66)$$

$$= \frac{b_0^{a_0}}{(2\pi)^{D/2} |\mathbf{V}_0|^{\frac{1}{2}} \Gamma(a_0)} (\sigma^2)^{-(a_0 + (D/2) + 1)} \quad (7.67)$$

$$\times \exp \left[-\frac{(\mathbf{w} - \mathbf{w}_0)^T \mathbf{V}_0^{-1} (\mathbf{w} - \mathbf{w}_0) + 2b_0}{2\sigma^2} \right] \quad (7.68)$$

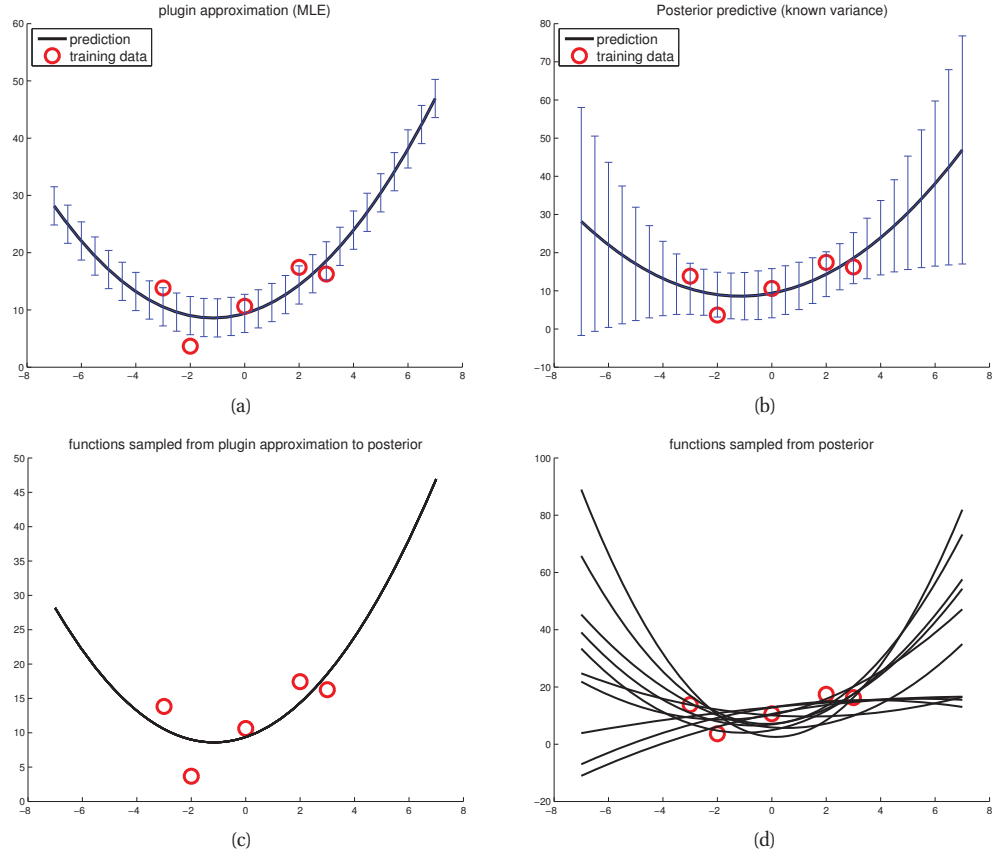


Figure 7.12 (a) Plug-in approximation to predictive density (we plug in the MLE of the parameters). (b) Posterior predictive density, obtained by integrating out the parameters. Black curve is posterior mean, error bars are 2 standard deviations of the posterior predictive density. (c) 10 samples from the plugin approximation to posterior predictive. (d) 10 samples from the posterior predictive. Figure generated by `linregPostPredDemo`.

With this prior and likelihood, one can show that the posterior has the following form:

$$p(\mathbf{w}, \sigma^2 | \mathcal{D}) = \text{NIG}(\mathbf{w}, \sigma^2 | \mathbf{w}_N, \mathbf{V}_N, a_N, b_N) \quad (7.69)$$

$$\mathbf{w}_N = \mathbf{V}_N (\mathbf{V}_0^{-1} \mathbf{w}_0 + \mathbf{X}^T \mathbf{y}) \quad (7.70)$$

$$\mathbf{V}_N = (\mathbf{V}_0^{-1} + \mathbf{X}^T \mathbf{X})^{-1} \quad (7.71)$$

$$a_N = a_0 + n/2 \quad (7.72)$$

$$b_N = b_0 + \frac{1}{2} (\mathbf{w}_0^T \mathbf{V}_0^{-1} \mathbf{w}_0 + \mathbf{y}^T \mathbf{y} - \mathbf{w}_N^T \mathbf{V}_N^{-1} \mathbf{w}_N) \quad (7.73)$$

The expressions for $\mathbf{w}_N$ and $\mathbf{V}_N$ are similar to the case where σ^2 is known. The expression for a_N is also intuitive, since it just updates the counts. The expression for b_N can be interpreted

as follows: it is the prior sum of squares, b_0 , plus the empirical sum of squares, $\mathbf{y}^T \mathbf{y}$, plus a term due to the error in the prior on $\mathbf{w}$.

The posterior marginals are as follows:

$$p(\sigma^2 | \mathcal{D}) = \text{IG}(a_N, b_N) \quad (7.74)$$

$$p(\mathbf{w} | \mathcal{D}) = \mathcal{T}(\mathbf{w}_N, \frac{b_N}{a_N} \mathbf{V}_N, 2a_N) \quad (7.75)$$

We give a worked example of using these equations in Section 7.6.3.3.

By analogy to Section 4.6.3.6, the posterior predictive distribution is a Student T distribution. In particular, given m new test inputs $\tilde{\mathbf{X}}$, we have

$$p(\tilde{\mathbf{y}} | \tilde{\mathbf{X}}, \mathcal{D}) = \mathcal{T}(\tilde{\mathbf{y}} | \tilde{\mathbf{X}} \mathbf{w}_N, \frac{b_N}{a_N} (\mathbf{I}_m + \tilde{\mathbf{X}} \mathbf{V}_N \tilde{\mathbf{X}}^T), 2a_N) \quad (7.76)$$

The predictive variance has two components: $(b_N/a_N) \mathbf{I}_m$ due to the measurement noise, and $(b_N/a_N) \tilde{\mathbf{X}} \mathbf{V}_N \tilde{\mathbf{X}}^T$ due to the uncertainty in $\mathbf{w}$. This latter term varies depending on how close the test inputs are to the training data.

It is common to set $a_0 = b_0 = 0$, corresponding to an uninformative prior for σ^2 , and to set $\mathbf{w}_0 = \mathbf{0}$ and $\mathbf{V}_0 = g(\mathbf{X}^T \mathbf{X})^{-1}$ for any positive value g . This is called Zellner's **g-prior** (Zellner 1986). Here g plays a role analogous to $1/\lambda$ in ridge regression. However, the prior covariance is proportional to $(\mathbf{X}^T \mathbf{X})^{-1}$ rather than $\mathbf{I}$. This ensures that the posterior is invariant to scaling of the inputs (Minka 2000b). See also Exercise 7.10.

We will see below that if we use an uninformative prior, the posterior precision given N measurements is $\mathbf{V}_N^{-1} = \mathbf{X}^T \mathbf{X}$. The **unit information prior** is defined to contain as much information as one sample (Kass and Wasserman 1995). To create a unit information prior for linear regression, we need to use $\mathbf{V}_0^{-1} = \frac{1}{N} \mathbf{X}^T \mathbf{X}$, which is equivalent to the g-prior with $g = N$.

7.6.3.2 Uninformative prior

An uninformative prior can be obtained by considering the uninformative limit of the conjugate g-prior, which corresponds to setting $g = \infty$. This is equivalent to an improper NIG prior with $\mathbf{w}_0 = \mathbf{0}$, $\mathbf{V}_0 = \infty \mathbf{I}$, $a_0 = 0$ and $b_0 = 0$, which gives $p(\mathbf{w}, \sigma^2) \propto \sigma^{-(D+2)}$.

Alternatively, we can start with the semi-conjugate prior $p(\mathbf{w}, \sigma^2) = p(\mathbf{w})p(\sigma^2)$, and take each term to its uninformative limit individually, which gives $p(\mathbf{w}, \sigma^2) \propto \sigma^{-2}$. This is equivalent to an improper NIG prior with $\mathbf{w}_0 = \mathbf{0}$, $\mathbf{V} = \infty \mathbf{I}$, $a_0 = -D/2$ and $b_0 = 0$. The corresponding posterior is given by

$$p(\mathbf{w}, \sigma^2 | \mathcal{D}) = \text{NIG}(\mathbf{w}, \sigma^2 | \mathbf{w}_N, \mathbf{V}_N, a_N, b_N) \quad (7.77)$$

$$\mathbf{w}_N = \hat{\mathbf{w}}_{mle} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (7.78)$$

$$\mathbf{V}_N = (\mathbf{X}^T \mathbf{X})^{-1} \quad (7.79)$$

$$a_N = \frac{N - D}{2} \quad (7.80)$$

$$b_N = \frac{s^2}{2} \quad (7.81)$$

$$s^2 \triangleq (\mathbf{y} - \mathbf{X} \hat{\mathbf{w}}_{mle})^T (\mathbf{y} - \mathbf{X} \hat{\mathbf{w}}_{mle}) \quad (7.82)$$

w_j	$\mathbb{E}[w_j \mathcal{D}]$	$\sqrt{\text{var}[w_j \mathcal{D}]}$	95% CI	sig
w0	10.998	3.06027	[4.652, 17.345]	*
w1	-0.004	0.00156	[-0.008, -0.001]	*
w2	-0.054	0.02190	[-0.099, -0.008]	*
w3	0.068	0.09947	[-0.138, 0.274]	
w4	-1.294	0.56381	[-2.463, -0.124]	*
w5	0.232	0.10438	[0.015, 0.448]	*
w6	-0.357	1.56646	[-3.605, 2.892]	
w7	-0.237	1.00601	[-2.324, 1.849]	
w8	0.181	0.23672	[-0.310, 0.672]	
w9	-1.285	0.86485	[-3.079, 0.508]	
w10	-0.433	0.73487	[-1.957, 1.091]	

Table 7.2 Posterior mean, standard deviation and credible intervals for a linear regression model with an uninformative prior fit to the caterpillar data. Produced by `linregBayesCaterpillar`.

The marginal distribution of the weights is given by

$$p(\mathbf{w}|\mathcal{D}) = \mathcal{T}(\mathbf{w}|\hat{\mathbf{w}}, \frac{s^2}{N-D}\mathbf{C}, N-D) \quad (7.83)$$

where $\mathbf{C} = (\mathbf{X}^T \mathbf{X})^{-1}$ and $\hat{\mathbf{w}}$ is the MLE. We discuss the implications of these equations below.

7.6.3.3 An example where Bayesian and frequentist inference coincide *

The use of a (semi-conjugate) uninformative prior is interesting because the resulting posterior turns out to be equivalent to the results from frequentist statistics (see also Section 4.6.3.9). In particular, from Equation 7.83 we have

$$p(w_j|\mathcal{D}) = T(w_j|\hat{w}_j, \frac{C_{jj}s^2}{N-D}, N-D) \quad (7.84)$$

This is equivalent to the sampling distribution of the MLE which is given by the following (see e.g., (Rice 1995, p542), (Casella and Berger 2002, p554)):

$$\frac{w_j - \hat{w}_j}{s_j} \sim t_{N-D} \quad (7.85)$$

where

$$s_j = \sqrt{\frac{s^2 C_{jj}}{N-D}} \quad (7.86)$$

is the standard error of the estimated parameter. (See Section 6.2 for a discussion of sampling distributions.) Consequently, the frequentist confidence interval and the Bayesian marginal credible interval for the parameters are the same in this case.

As a worked example of this, consider the caterpillar dataset from (Marin and Robert 2007). (The details of what the data mean don't matter for our present purposes.) We can compute

the posterior mean and standard deviation, and the 95% credible intervals (CI) for the regression coefficients using Equation 7.84. The results are shown in Table 7.2. It is easy to check that these 95% credible intervals are identical to the 95% confidence intervals computed using standard frequentist methods (see `linregBayesCaterpillar` for the code).

We can also use these marginal posteriors to compute if the coefficients are “significantly” different from 0. An informal way to do this (without using decision theory) is to check if its 95% CI excludes 0. From Table 7.2, we see that the CIs for coefficients 0, 1, 2, 4, 5 are all significant by this measure, so we put a little star by them. It is easy to check that these results are the same as those produced by standard frequentist software packages which compute p-values at the 5% level.

Although the correspondence between the Bayesian and frequentist results might seem appealing to some readers, recall from Section 6.6 that frequentist inference is riddled with pathologies. Also, note that the MLE does not even exist when $N < D$, so standard frequentist inference theory breaks down in this setting. Bayesian inference theory still works, although it requires the use of proper priors. (See (Maruyama and George 2008) for one extension of the g-prior to the case where $D > N$.)

7.6.4 EB for linear regression (evidence procedure)

So far, we have assumed the prior is known. In this section, we describe an empirical Bayes procedure for picking the hyper-parameters. More precisely, we choose $\boldsymbol{\eta} = (\alpha, \lambda)$ to maximize the marginal likelihood, where $\lambda = 1/\sigma^2$ be the precision of the observation noise and α is the precision of the prior, $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{0}, \alpha^{-1}\mathbf{I})$. This is known as the **evidence procedure** (MacKay 1995b).³ See Section 13.7.4 for the algorithmic details.

The evidence procedure provides an alternative to using cross validation. For example, in Figure 7.13(b), we plot the log marginal likelihood for different values of α , as well as the maximum value found by the optimizer. We see that, in this example, we get the same result as 5-CV, shown in Figure 7.13(a). (We kept $\lambda = 1/\sigma^2$ fixed in both methods, to make them comparable.)

The principle practical advantage of the evidence procedure over CV will become apparent in Section 13.7, where we generalize the prior by allowing a different α_j for every feature. This can be used to perform feature selection, using a technique known as automatic relevancy determination or ARD. By contrast, it would not be possible to use CV to tune D different hyper-parameters.

The evidence procedure is also useful when comparing different kinds of models, since it provides a good approximation to the evidence:

$$p(\mathcal{D}|m) = \int \int p(\mathcal{D}|\mathbf{w}, m) p(\mathbf{w}|m, \boldsymbol{\eta}) p(\boldsymbol{\eta}|m) d\mathbf{w} d\boldsymbol{\eta} \quad (7.87)$$

$$\approx \max_{\boldsymbol{\eta}} \int p(\mathcal{D}|\mathbf{w}, m) p(\mathbf{w}|m, \boldsymbol{\eta}) p(\boldsymbol{\eta}|m) d\mathbf{w} \quad (7.88)$$

It is important to (at least approximately) integrate over $\boldsymbol{\eta}$ rather than setting it arbitrarily, for reasons discussed in Section 5.3.2.5. Indeed, this is the method we used to evaluate the marginal

3. Alternatively, we could integrate out λ analytically, as shown in Section 7.6.3, and just optimize α (Buntine and Weigend 1991). However, it turns out that this is less accurate than optimizing both α and λ (MacKay 1999).

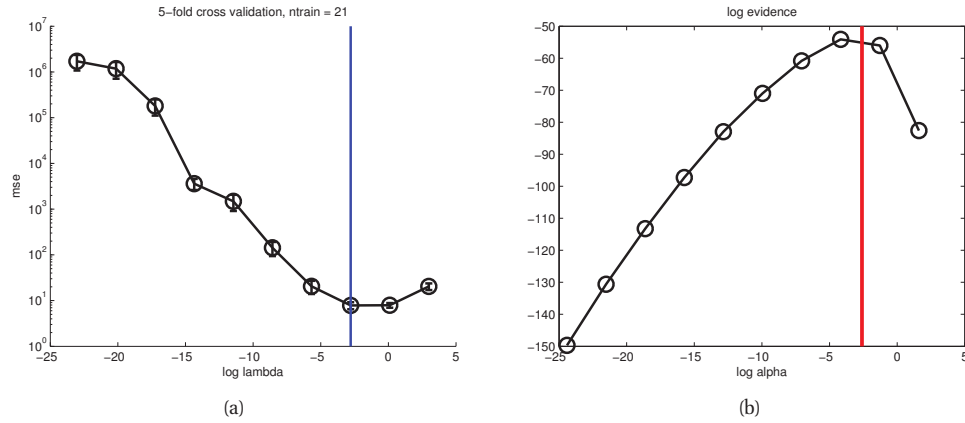


Figure 7.13 (a) Estimate of test MSE produced by 5-fold cross-validation vs $\log(\lambda)$. The smallest value is indicated by the vertical line. Note the vertical scale is in log units. (c) Log marginal likelihood vs $\log(\alpha)$. The largest value is indicated by the vertical line. Figure generated by `linregPolyVsRegDemo`.

likelihood for the polynomial regression models in Figures 5.7 and 5.8. For a “more Bayesian” approach, in which we model our uncertainty about $\boldsymbol{\eta}$ rather than computing point estimates, see Section 21.5.2.

Exercises

Exercise 7.1 Behavior of training set error with increasing sample size

The error on the test will always decrease as we get more training data, since the model will be better estimated. However, as shown in Figure 7.10, for sufficiently complex models, the error on the training set can increase as we get more training data, until we reach some plateau. Explain why.

Exercise 7.2 Multi-output linear regression

(Source: Jaakkola.)

When we have multiple independent outputs in linear regression, the model becomes

$$p(\mathbf{y}|\mathbf{x}, \mathbf{W}) = \prod_{j=1}^M \mathcal{N}(y_j | \mathbf{w}_j^T \mathbf{x}_i, \sigma_j^2) \quad (7.89)$$

Since the likelihood factorizes across dimensions, so does the MLE. Thus

$$\hat{\mathbf{W}} = [\hat{\mathbf{w}}_1, \dots, \hat{\mathbf{w}}_M] \quad (7.90)$$

where $\hat{\mathbf{w}}_j = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{Y}_{:,j}$.

In this exercise we apply this result to a model with 2 dimensional response vector $\mathbf{y}_i \in \mathbb{R}^2$. Suppose we have some binary input data, $x_i \in \{0, 1\}$. The training data is as follows:

x	y
0	$(-1, -1)^T$
0	$(-1, -2)^T$
0	$(-2, -1)^T$
1	$(1, 1)^T$
1	$(1, 2)^T$
1	$(2, 1)^T$

Let us embed each x_i into 2d using the following basis function:

$$\phi(0) = (1, 0)^T, \quad \phi(1) = (0, 1)^T \quad (7.91)$$

The model becomes

$$\hat{\mathbf{y}} = \mathbf{W}^T \phi(x) \quad (7.92)$$

where $\mathbf{W}$ is a 2×2 matrix. Compute the MLE for $\mathbf{W}$ from the above data.

Exercise 7.3 Centering and ridge regression

Assume that $\bar{\mathbf{x}} = 0$, so the input data has been centered. Show that the optimizer of

$$J(\mathbf{w}, w_0) = (\mathbf{y} - \mathbf{X}\mathbf{w} - w_0\mathbf{1})^T (\mathbf{y} - \mathbf{X}\mathbf{w} - w_0\mathbf{1}) + \lambda \mathbf{w}^T \mathbf{w} \quad (7.93)$$

is

$$\hat{w}_0 = \bar{y} \quad (7.94)$$

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y} \quad (7.95)$$

Exercise 7.4 MLE for σ^2 for linear regression

Show that the MLE for the error variance in linear regression is given by

$$\hat{\sigma}^2 = \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{x}_i^T \hat{\mathbf{w}})^2 \quad (7.96)$$

This is just the empirical variance of the residual errors when we plug in our estimate of $\hat{\mathbf{w}}$.

Exercise 7.5 MLE for the offset term in linear regression

Linear regression has the form $\mathbb{E}[y|\mathbf{x}] = w_0 + \mathbf{w}^T \mathbf{x}$. It is common to include a column of 1's in the design matrix, so we can solve for the offset term w_0 term and the other parameters $\mathbf{w}$ at the same time using the normal equations. However, it is also possible to solve for $\mathbf{w}$ and w_0 separately. Show that

$$\hat{w}_0 = \frac{1}{N} \sum_i y_i - \frac{1}{N} \sum_i \mathbf{x}_i^T \mathbf{w} = \bar{y} - \bar{\mathbf{x}}^T \mathbf{w} \quad (7.97)$$

So $\hat{w}_0$ models the difference in the average output from the average predicted output. Also, show that

$$\hat{\mathbf{w}} = (\mathbf{X}_c^T \mathbf{X}_c)^{-1} \mathbf{X}_c^T \mathbf{y}_c = \left[\sum_{i=1}^N (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T \right]^{-1} \left[\sum_{i=1}^N (y_i - \bar{y})(\mathbf{x}_i - \bar{\mathbf{x}}) \right] \quad (7.98)$$

where $\mathbf{X}_c$ is the centered input matrix containing $\mathbf{x}_i^c = \mathbf{x}_i - \bar{\mathbf{x}}$ along its rows, and $\mathbf{y}_c = \mathbf{y} - \bar{y}$ is the centered output vector. Thus we can first compute $\hat{\mathbf{w}}$ on centered data, and then estimate w_0 using $\bar{y} - \bar{\mathbf{x}}^T \hat{\mathbf{w}}$.

Exercise 7.6 MLE for simple linear regression

Simple linear regression refers to the case where the input is scalar, so $D = 1$. Show that the MLE in this case is given by the following equations, which may be familiar from basic statistics classes:

$$w_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} = \frac{\sum_i x_i y_i - N \bar{x} \bar{y}}{\sum_i x_i^2 - N \bar{x}^2} \approx \frac{\text{cov}[X, Y]}{\text{var}[X]} \quad (7.99)$$

$$w_0 = \bar{y} - w_1 \bar{x} \approx \mathbb{E}[Y] - w_1 \mathbb{E}[X] \quad (7.100)$$

See `linregDemo1` for a demo.

Exercise 7.7 Sufficient statistics for online linear regression

(Source: Jaakkola.) Consider fitting the model $\hat{y} = w_0 + w_1 x$ using least squares. Unfortunately we did not keep the original data, x_i, y_i , but we do have the following functions (statistics) of the data:

$$\bar{x}^{(n)} = \frac{1}{n} \sum_{i=1}^n x_i, \quad \bar{y}^{(n)} = \frac{1}{n} \sum_{i=1}^n y_i \quad (7.101)$$

$$C_{xx}^{(n)} = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2, \quad C_{xy}^{(n)} = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}), \quad C_{yy}^{(n)} = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2 \quad (7.102)$$

- What are the minimal set of statistics that we need to estimate w_1 ? (Hint: see Equation 7.99.)
- What are the minimal set of statistics that we need to estimate w_0 ? (Hint: see Equation 7.97.)
- Suppose a new data point, x_{n+1}, y_{n+1} arrives, and we want to update our sufficient statistics without looking at the old data, which we have not stored. (This is useful for online learning.) Show that we can this for $\bar{x}$ as follows.

$$\bar{x}^{(n+1)} \triangleq \frac{1}{n+1} \sum_{i=1}^{n+1} x_i = \frac{1}{n+1} (n\bar{x}^{(n)} + x_{n+1}) \quad (7.103)$$

$$= \bar{x}^{(n)} + \frac{1}{n+1} (x_{n+1} - \bar{x}^{(n)}) \quad (7.104)$$

This has the form: new estimate is old estimate plus correction. We see that the size of the correction diminishes over time (i.e., as we get more samples). Derive a similar expression to update $\bar{y}$

- Show that one can update $C_{xy}^{(n+1)}$ recursively using

$$C_{xy}^{(n+1)} = \frac{1}{n+1} \left[x_{n+1} y_{n+1} + n C_{xy}^{(n)} + n \bar{x}^{(n)} \bar{y}^{(n)} - (n+1) \bar{x}^{(n+1)} \bar{y}^{(n+1)} \right] \quad (7.105)$$

Derive a similar expression to update C_{xx} .

- Implement the online learning algorithm, i.e., write a function of the form `[w,ss] = linregUpdateSS(ss, x, y)`, where `x` and `y` are scalars and `ss` is a structure containing the sufficient statistics.
- Plot the coefficients over “time”, using the dataset in `linregDemo1`. (Specifically, use `[x,y] = polyDataMake('sampling','thibaux')`.) Check that they converge to the solution given by the batch (offline) learner (i.e., ordinary least squares). Your result should look like Figure 7.14.

Turn in your derivation, code and plot.

Exercise 7.8 Bayesian linear regression in 1d with known σ^2

(Source: Bolstad.) Consider fitting a model of the form

$$p(y|x, \theta) = \mathcal{N}(y|w_0 + w_1 x, \sigma^2) \quad (7.106)$$

to the data shown below:

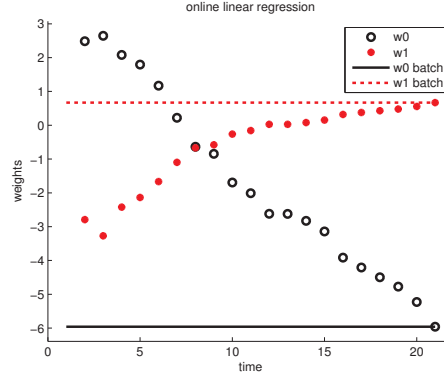


Figure 7.14 Regression coefficients over time. Produced by Exercise 7.7.

$\mathbf{x} = [94, 96, 94, 95, 104, 106, 108, 113, 115, 121, 131];$
 $\mathbf{y} = [0.47, 0.75, 0.83, 0.98, 1.18, 1.29, 1.40, 1.60, 1.75, 1.90, 2.23];$

- a. Compute an unbiased estimate of σ^2 using

$$\hat{\sigma}^2 = \frac{1}{N-2} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (7.107)$$

(The denominator is $N-2$ since we have 2 inputs, namely the offset term and x .) Here $\hat{y}_i = \hat{w}_0 + \hat{w}_1 x_i$, and $\hat{\mathbf{w}} = (\hat{w}_0, \hat{w}_1)$ is the MLE.

- b. Now assume the following prior on $\mathbf{w}$:

$$p(\mathbf{w}) = p(w_0)p(w_1) \quad (7.108)$$

Use an (improper) uniform prior on w_0 and a $\mathcal{N}(0, 1)$ prior on w_1 . Show that this can be written as a Gaussian prior of the form $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{w}_0, \mathbf{V}_0)$. What are $\mathbf{w}_0$ and $\mathbf{V}_0$?

- c. Compute the marginal posterior of the slope, $p(w_1|\mathcal{D}, \sigma^2)$, where $\mathcal{D}$ is the data above, and σ^2 is the unbiased estimate computed above. What is $\mathbb{E}[w_1|\mathcal{D}, \sigma^2]$ and $\text{var}[w_1|\mathcal{D}, \sigma^2]$? Show your work. (You can use Matlab if you like.) Hint: the posterior variance is a very small number!
- d. What is a 95% credible interval for w_1 ?

Exercise 7.9 Generative model for linear regression

Linear regression is the problem of estimating $E[Y|\mathbf{x}]$ using a linear function of the form $w_0 + \mathbf{w}^T \mathbf{x}$. Typically we assume that the conditional distribution of Y given $\mathbf{X}$ is Gaussian. We can either estimate this conditional Gaussian directly (a discriminative approach), or we can fit a Gaussian to the joint distribution of $\mathbf{X}, Y$ and then derive $E[Y|\mathbf{X} = \mathbf{x}]$.

In Exercise 7.5 we showed that the discriminative approach leads to these equations

$$E[Y|\mathbf{x}] = w_0 + \mathbf{w}^T \mathbf{x} \quad (7.109)$$

$$w_0 = \bar{y} - \bar{\mathbf{x}}^T \mathbf{w} \quad (7.110)$$

$$\mathbf{w} = (\mathbf{X}_c^T \mathbf{X}_c)^{-1} \mathbf{X}_c^T \mathbf{y}_c \quad (7.111)$$

where $\mathbf{X}_c = \mathbf{X} - \bar{\mathbf{X}}$ is the centered input matrix, and $\bar{\mathbf{X}} = \mathbf{1}_n \bar{\mathbf{x}}^T$ replicates $\bar{\mathbf{x}}$ across the rows. Similarly, $\mathbf{y}_c = \mathbf{y} - \bar{\mathbf{y}}$ is the centered output vector, and $\bar{\mathbf{y}} = \mathbf{1}_n \bar{y}$ replicates $\bar{y}$ across the rows.

- By finding the maximum likelihood estimates of Σ_{XX} , Σ_{XY} , μ_X and μ_Y , derive the above equations by fitting a joint Gaussian to $\mathbf{X}$, Y and using the formula for conditioning a Gaussian (see Section 4.3.1). Show your work.
- What are the advantages and disadvantages of this approach compared to the standard discriminative approach?

Exercise 7.10 Bayesian linear regression using the g-prior

Show that when we use the g-prior, $p(\mathbf{w}, \sigma^2) = \text{NIG}(\mathbf{w}, \sigma^2 | \mathbf{0}, g(\mathbf{X}^T \mathbf{X})^{-1}, 0, 0)$, the posterior has the following form:

$$p(\mathbf{w}, \sigma^2 | \mathcal{D}) = \text{NIG}(\mathbf{w}, \sigma^2 | \mathbf{w}_N, \mathbf{V}_N, a_N, b_N) \quad (7.112)$$

$$\mathbf{V}_N = \frac{g}{g+1} (\mathbf{X}^T \mathbf{X})^{-1} \quad (7.113)$$

$$\mathbf{w}_N = \frac{g}{g+1} \hat{\mathbf{w}}_{mle} \quad (7.114)$$

$$a_N = N/2 \quad (7.115)$$

$$b_N = \frac{s^2}{2} + \frac{1}{2(g+1)} \hat{\mathbf{w}}_{mle}^T \mathbf{X}^T \mathbf{X} \hat{\mathbf{w}}_{mle} \quad (7.116)$$

$$(7.117)$$

8

Logistic regression

8.1 Introduction

One way to build a probabilistic classifier is to create a joint model of the form $p(y, \mathbf{x})$ and then to condition on $\mathbf{x}$, thereby deriving $p(y|\mathbf{x})$. This is called the generative approach. An alternative approach is to fit a model of the form $p(y|\mathbf{x})$ directly. This is called the **discriminative** approach, and is the approach we adopt in this chapter. In particular, we will assume discriminative models which are linear in the parameters. This will turn out to significantly simplify model fitting, as we will see. In Section 8.6, we compare the generative and discriminative approaches, and in later chapters, we will consider non-linear and non-parametric discriminative models.

8.2 Model specification

As we discussed in Section 1.4.6, logistic regression corresponds to the following binary classification model:

$$p(y|\mathbf{x}, \mathbf{w}) = \text{Ber}(y|\text{sigm}(\mathbf{w}^T \mathbf{x})) \quad (8.1)$$

A 1d example is shown in Figure 1.19(b). Logistic regression can easily be extended to higher-dimensional inputs. For example, Figure 8.1 shows plots of $p(y = 1|\mathbf{x}, \mathbf{w}) = \text{sigm}(\mathbf{w}^T \mathbf{x})$ for 2d input and different weight vectors $\mathbf{w}$. If we threshold these probabilities at 0.5, we induce a linear decision boundary, whose normal (perpendicular) is given by $\mathbf{w}$.

8.3 Model fitting

In this section, we discuss algorithms for estimating the parameters of a logistic regression model.

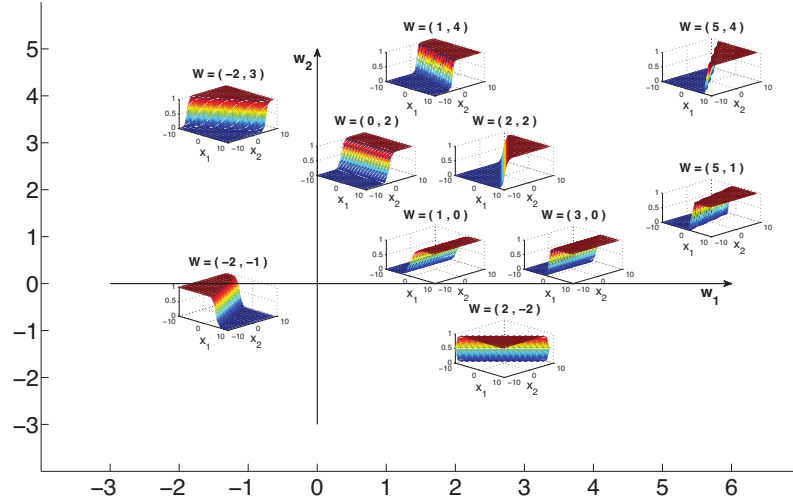


Figure 8.1 Plots of $\text{sigm}(w_1x_1 + w_2x_2)$. Here $\mathbf{w} = (w_1, w_2)$ defines the normal to the decision boundary. Points to the right of this have $\text{sigm}(\mathbf{w}^T \mathbf{x}) > 0.5$, and points to the left have $\text{sigm}(\mathbf{w}^T \mathbf{x}) < 0.5$. Based on Figure 39.3 of (MacKay 2003). Figure generated by `sigmoidplot2D`.

8.3.1 MLE

The negative log-likelihood for logistic regression is given by

$$\text{NLL}(\mathbf{w}) = - \sum_{i=1}^N \log[\mu_i^{\mathbb{I}(y_i=1)} \times (1 - \mu_i)^{\mathbb{I}(y_i=0)}] \quad (8.2)$$

$$= - \sum_{i=1}^N [y_i \log \mu_i + (1 - y_i) \log(1 - \mu_i)] \quad (8.3)$$

This is also called the **cross-entropy** error function (see Section 2.8.2).

Another way of writing this is as follows. Suppose $\tilde{y}_i \in \{-1, +1\}$ instead of $y_i \in \{0, 1\}$. We have $p(y = 1) = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x})}$ and $p(y = 1) = \frac{1}{1 + \exp(+\mathbf{w}^T \mathbf{x})}$. Hence

$$\text{NLL}(\mathbf{w}) = \sum_{i=1}^N \log(1 + \exp(-\tilde{y}_i \mathbf{w}^T \mathbf{x}_i)) \quad (8.4)$$

Unlike linear regression, we can no longer write down the MLE in closed form. Instead, we need to use an optimization algorithm to compute it. For this, we need to derive the gradient and Hessian.

In the case of logistic regression, one can show (Exercise 8.3) that the gradient and Hessian

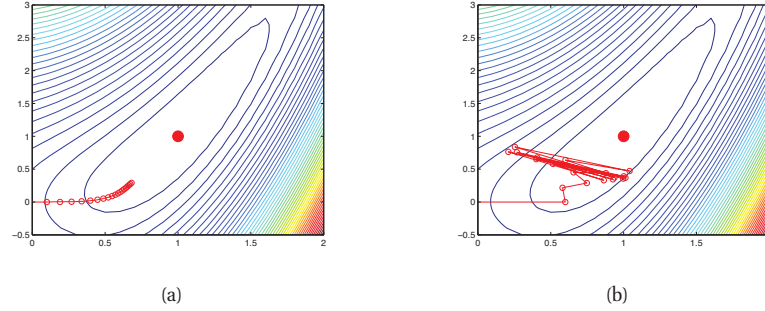


Figure 8.2 Gradient descent on a simple function, starting from $(0, 0)$, for 20 steps, using a fixed learning rate (step size) η . The global minimum is at $(1, 1)$. (a) $\eta = 0.1$. (b) $\eta = 0.6$. Figure generated by `steepestDescentDemo`.

of this are given by the following

$$\mathbf{g} = \frac{d}{d\mathbf{w}} f(\mathbf{w}) = \sum_i (\mu_i - y_i) \mathbf{x}_i = \mathbf{X}^T (\boldsymbol{\mu} - \mathbf{y}) \quad (8.5)$$

$$\mathbf{H} = \frac{d}{d\mathbf{w}} \mathbf{g}(\mathbf{w})^T = \sum_i (\nabla_{\mathbf{w}} \mu_i) \mathbf{x}_i^T = \sum_i \mu_i (1 - \mu_i) \mathbf{x}_i \mathbf{x}_i^T \quad (8.6)$$

$$= \mathbf{X}^T \mathbf{S} \mathbf{X} \quad (8.7)$$

where $\mathbf{S} \triangleq \text{diag}(\mu_i(1 - \mu_i))$. One can also show (Exercise 8.3) that $\mathbf{H}$ is positive definite. Hence the NLL is convex and has a unique global minimum. Below we discuss some methods for finding this minimum.

8.3.2 Steepest descent

Perhaps the simplest algorithm for unconstrained optimization is **gradient descent**, also known as **steepest descent**. This can be written as follows:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta_k \mathbf{g}_k \quad (8.8)$$

where η_k is the **step size** or **learning rate**. The main issue in gradient descent is: how should we set the step size? This turns out to be quite tricky. If we use a constant learning rate, but make it too small, convergence will be very slow, but if we make it too large, the method can fail to converge at all. This is illustrated in Figure 8.2. where we plot the following (convex) function

$$f(\boldsymbol{\theta}) = 0.5(\theta_1^2 - \theta_2)^2 + 0.5(\theta_1 - 1)^2, \quad (8.9)$$

We arbitrarily decide to start from $(0, 0)$. In Figure 8.2(a), we use a fixed step size of $\eta = 0.1$; we see that it moves slowly along the valley. In Figure 8.2(b), we use a fixed step size of $\eta = 0.6$; we see that the algorithm starts oscillating up and down the sides of the valley and never converges to the optimum.

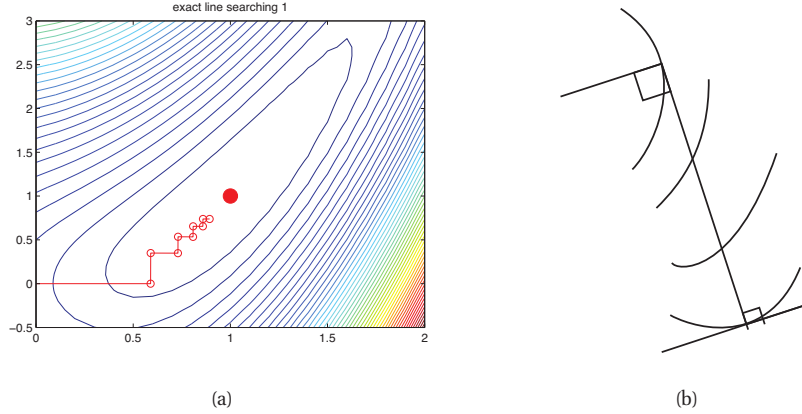


Figure 8.3 (a) Steepest descent on the same function as Figure 8.2, starting from $(0, 0)$, using line search. Figure generated by `steepestDescentDemo`. (b) Illustration of the fact that at the end of a line search (top of picture), the local gradient of the function will be perpendicular to the search direction. Based on Figure 10.6.1 of (Press et al. 1988).

Let us develop a more stable method for picking the step size, so that the method is guaranteed to converge to a local optimum no matter where we start. (This property is called **global convergence**, which should not be confused with convergence to the global optimum!) By Taylor's theorem, we have

$$f(\boldsymbol{\theta} + \eta \mathbf{d}) \approx f(\boldsymbol{\theta}) + \eta \mathbf{g}^T \mathbf{d} \quad (8.10)$$

where $\mathbf{d}$ is our descent direction. So if η is chosen small enough, then $f(\boldsymbol{\theta} + \eta \mathbf{d}) < f(\boldsymbol{\theta})$, since the gradient will be negative. But we don't want to choose the step size η too small, or we will move very slowly and may not reach the minimum. So let us pick η to minimize

$$\phi(\eta) = f(\boldsymbol{\theta}_k + \eta \mathbf{d}_k) \quad (8.11)$$

This is called **line minimization** or **line search**. There are various methods for solving this 1d optimization problem; see (Nocedal and Wright 2006) for details.

Figure 8.3(a) demonstrates that line search does indeed work for our simple problem. However, we see that the steepest descent path with exact line searches exhibits a characteristic **zig-zag** behavior. To see why, note that an exact line search satisfies $\eta_k = \arg \min_{\eta > 0} \phi(\eta)$. A necessary condition for the optimum is $\phi'(\eta) = 0$. By the chain rule, $\phi'(\eta) = \mathbf{d}^T \mathbf{g}$, where $\mathbf{g} = f'(\boldsymbol{\theta} + \eta \mathbf{d})$ is the gradient at the end of the step. So we either have $\mathbf{g} = \mathbf{0}$, which means we have found a stationary point, or $\mathbf{g} \perp \mathbf{d}$, which means that exact search stops at a point where the local gradient is perpendicular to the search direction. Hence consecutive directions will be orthogonal (see Figure 8.3(b)). This explains the zig-zag behavior.

One simple heuristic to reduce the effect of zig-zagging is to add a **momentum** term, $(\boldsymbol{\theta}_k - \boldsymbol{\theta}_{k-1})$, as follows:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta_k \mathbf{g}_k + \mu_k (\boldsymbol{\theta}_k - \boldsymbol{\theta}_{k-1}) \quad (8.12)$$

where $0 \leq \mu_k \leq 1$ controls the importance of the momentum term. In the optimization community, this is known as the **heavy ball method** (see e.g., (Bertsekas 1999)).

An alternative way to minimize “zig-zagging” is to use the method of **conjugate gradients** (see e.g., (Nocedal and Wright 2006, ch 5) or (Golub and van Loan 1996, Sec 10.2)). This is the method of choice for quadratic objectives of the form $f(\boldsymbol{\theta}) = \boldsymbol{\theta}^T \mathbf{A} \boldsymbol{\theta}$, which arise when solving linear systems. However, non-linear CG is less popular.

8.3.3 Newton’s method

Algorithm 8.1: Newton’s method for minimizing a strictly convex function

```

1 Initialize  $\boldsymbol{\theta}_0$ ;
2 for  $k = 1, 2, \dots$  until convergence do
3   Evaluate  $\mathbf{g}_k = \nabla f(\boldsymbol{\theta}_k)$ ;
4   Evaluate  $\mathbf{H}_k = \nabla^2 f(\boldsymbol{\theta}_k)$ ;
5   Solve  $\mathbf{H}_k \mathbf{d}_k = -\mathbf{g}_k$  for  $\mathbf{d}_k$ ;
6   Use line search to find stepsize  $\eta_k$  along  $\mathbf{d}_k$ ;
7    $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \eta_k \mathbf{d}_k$ ;
```

One can derive faster optimization methods by taking the curvature of the space (i.e., the Hessian) into account. These are called **second order** optimization methods. The primary example is **Newton’s algorithm**. This is an iterative algorithm which consists of updates of the form

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta_k \mathbf{H}_k^{-1} \mathbf{g}_k \quad (8.13)$$

The full pseudo-code is given in Algorithm 2.

This algorithm can be derived as follows. Consider making a second-order Taylor series approximation of $f(\boldsymbol{\theta})$ around $\boldsymbol{\theta}_k$:

$$f_{quad}(\boldsymbol{\theta}) = f_k + \mathbf{g}_k^T (\boldsymbol{\theta} - \boldsymbol{\theta}_k) + \frac{1}{2} (\boldsymbol{\theta} - \boldsymbol{\theta}_k)^T \mathbf{H}_k (\boldsymbol{\theta} - \boldsymbol{\theta}_k) \quad (8.14)$$

Let us rewrite this as

$$f_{quad}(\boldsymbol{\theta}) = \boldsymbol{\theta}^T \mathbf{A} \boldsymbol{\theta} + \mathbf{b}^T \boldsymbol{\theta} + c \quad (8.15)$$

where

$$\mathbf{A} = \frac{1}{2} \mathbf{H}_k, \quad \mathbf{b} = \mathbf{g}_k - \mathbf{H}_k \boldsymbol{\theta}_k, \quad c = f_k - \mathbf{g}_k^T \boldsymbol{\theta}_k + \frac{1}{2} \boldsymbol{\theta}_k^T \mathbf{H}_k \boldsymbol{\theta}_k \quad (8.16)$$

The minimum of f_{quad} is at

$$\boldsymbol{\theta} = -\frac{1}{2} \mathbf{A}^{-1} \mathbf{b} = \boldsymbol{\theta}_k - \mathbf{H}_k^{-1} \mathbf{g}_k \quad (8.17)$$

Thus the Newton step $\mathbf{d}_k = -\mathbf{H}_k^{-1} \mathbf{g}_k$ is what should be added to $\boldsymbol{\theta}_k$ to minimize the second order approximation of f around $\boldsymbol{\theta}_k$. See Figure 8.4(a) for an illustration.

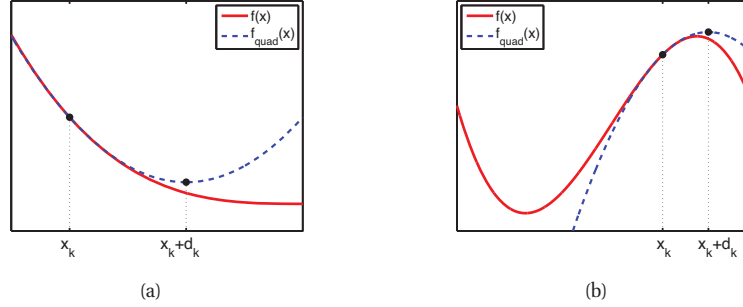


Figure 8.4 Illustration of Newton's method for minimizing a 1d function. (a) The solid curve is the function $f(x)$. The dotted line $f_{quad}(x)$ is its second order approximation at x_k . The Newton step d_k is what must be added to x_k to get to the minimum of $f_{quad}(x)$. Based on Figure 13.4 of (Vandenberghe 2006). Figure generated by `newtonsMethodMinQuad`. (b) Illustration of Newton's method applied to a nonconvex function. We fit a quadratic around the current point x_k and move to its stationary point, $x_{k+1} = x_k + d_k$. Unfortunately, this is a local maximum, not minimum. This means we need to be careful about the extent of our quadratic approximation. Based on Figure 13.11 of (Vandenberghe 2006). Figure generated by `newtonsMethodNonConvex`.

In its simplest form (as listed), Newton's method requires that $\mathbf{H}_k$ be positive definite, which will hold if the function is strictly convex. If not, the objective function is not convex, then $\mathbf{H}_k$ may not be positive definite, so $\mathbf{d}_k = -\mathbf{H}_k^{-1}\mathbf{g}_k$ may not be a descent direction (see Figure 8.4(b) for an example). In this case, one simple strategy is to revert to steepest descent, $\mathbf{d}_k = -\mathbf{g}_k$. The **Levenberg Marquardt** algorithm is an adaptive way to blend between Newton steps and steepest descent steps. This method is widely used when solving nonlinear least squares problems. An alternative approach is this: Rather than computing $\mathbf{d}_k = -\mathbf{H}_k^{-1}\mathbf{g}_k$ directly, we can solve the linear system of equations $\mathbf{H}_k\mathbf{d}_k = -\mathbf{g}_k$ for $\mathbf{d}_k$ using conjugate gradient (CG). If $\mathbf{H}_k$ is not positive definite, we can simply truncate the CG iterations as soon as negative curvature is detected; this is called **truncated Newton**.

8.3.4 Iteratively reweighted least squares (IRLS)

Let us now apply Newton's algorithm to find the MLE for binary logistic regression. The Newton update at iteration $k + 1$ for this model is as follows (using $\eta_k = 1$, since the Hessian is exact):

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \mathbf{H}^{-1}\mathbf{g}_k \quad (8.18)$$

$$= \mathbf{w}_k + (\mathbf{X}^T \mathbf{S}_k \mathbf{X})^{-1} \mathbf{X}^T (\mathbf{y} - \boldsymbol{\mu}_k) \quad (8.19)$$

$$= (\mathbf{X}^T \mathbf{S}_k \mathbf{X})^{-1} [(\mathbf{X}^T \mathbf{S}_k \mathbf{X}) \mathbf{w}_k + \mathbf{X}^T (\mathbf{y} - \boldsymbol{\mu}_k)] \quad (8.20)$$

$$= (\mathbf{X}^T \mathbf{S}_k \mathbf{X})^{-1} \mathbf{X}^T [\mathbf{S}_k \mathbf{X} \mathbf{w}_k + \mathbf{y} - \boldsymbol{\mu}_k] \quad (8.21)$$

$$= (\mathbf{X}^T \mathbf{S}_k \mathbf{X})^{-1} \mathbf{X}^T \mathbf{S}_k \mathbf{z}_k \quad (8.22)$$

where we have defined the **working response** as

$$\mathbf{z}_k \triangleq \mathbf{X} \mathbf{w}_k + \mathbf{S}_k^{-1} (\mathbf{y} - \boldsymbol{\mu}_k) \quad (8.23)$$

Equation 8.22 is an example of a **weighted least squares problem**, which is a minimizer of

$$\sum_{i=1}^N S_{ki} (z_{ki} - \mathbf{w}^T \mathbf{x}_i)^2 \quad (8.24)$$

Since $\mathbf{S}_k$ is a diagonal matrix, we can rewrite the targets in component form (for each case $i = 1 : N$) as

$$z_{ki} = \mathbf{w}_k^T \mathbf{x}_i + \frac{y_i - \mu_{ki}}{\mu_{ki}(1 - \mu_{ki})} \quad (8.25)$$

This algorithm is known as **iteratively reweighted least squares** or **IRLS** for short, since at each iteration, we solve a weighted least squares problem, where the weight matrix $\mathbf{S}_k$ changes at each iteration. See Algorithm 10 for some pseudocode.

Algorithm 8.2: Iteratively reweighted least squares (IRLS)

```

1  $\mathbf{w} = \mathbf{0}_D$ ;
2  $w_0 = \log(\bar{y}/(1 - \bar{y}))$ ;
3 repeat
4    $\eta_i = w_0 + \mathbf{w}^T \mathbf{x}_i$ ;
5    $\mu_i = \text{sigm}(\eta_i)$ ;
6    $s_i = \mu_i(1 - \mu_i)$ ;
7    $z_i = \eta_i + \frac{y_i - \mu_i}{s_i}$ ;
8    $\mathbf{S} = \text{diag}(s_{1:N})$ ;
9    $\mathbf{w} = (\mathbf{X}^T \mathbf{S} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{S} \mathbf{z}$ ;
10 until converged;
```

8.3.5 Quasi-Newton (variable metric) methods

The mother of all second-order optimization algorithm is Newton's algorithm, which we discussed in Section 8.3.3. Unfortunately, it may be too expensive to compute $\mathbf{H}$ explicitly. **Quasi-Newton** methods iteratively build up an approximation to the Hessian using information gleaned from the gradient vector at each step. The most common method is called **BFGS** (named after its inventors, Broyden, Fletcher, Goldfarb and Shanno), which updates the approximation to the Hessian $\mathbf{B}_k \approx \mathbf{H}_k$ as follows:

$$\mathbf{B}_{k+1} = \mathbf{B}_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{(\mathbf{B}_k \mathbf{s}_k)(\mathbf{B}_k \mathbf{s}_k)^T}{\mathbf{s}_k^T \mathbf{B}_k \mathbf{s}_k} \quad (8.26)$$

$$\mathbf{s}_k = \boldsymbol{\theta}_k - \boldsymbol{\theta}_{k-1} \quad (8.27)$$

$$\mathbf{y}_k = \mathbf{g}_k - \mathbf{g}_{k-1} \quad (8.28)$$

This is a rank-two update to the matrix, and ensures that the matrix remains positive definite (under certain restrictions on the step size). We typically start with a diagonal approximation, $\mathbf{B}_0 = \mathbf{I}$. Thus BFGS can be thought of as a “diagonal plus low-rank” approximation to the Hessian.

Alternatively, BFGS can iteratively update an approximation to the inverse Hessian, $\mathbf{C}_k \approx \mathbf{H}_k^{-1}$, as follows:

$$\mathbf{C}_{k+1} = \left(\mathbf{I} - \frac{\mathbf{s}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} \right) \mathbf{C}_k \left(\mathbf{I} - \frac{\mathbf{y}_k \mathbf{s}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} \right) + \frac{\mathbf{s}_k \mathbf{s}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} \quad (8.29)$$

Since storing the Hessian takes $O(D^2)$ space, for very large problems, one can use **limited memory BFGS**, or **L-BFGS**, where $\mathbf{H}_k$ or $\mathbf{H}_k^{-1}$ is approximated by a diagonal plus low rank matrix. In particular, the product $\mathbf{H}_k^{-1} \mathbf{g}_k$ can be obtained by performing a sequence of inner products with $\mathbf{s}_k$ and $\mathbf{y}_k$, using only the m most recent $(\mathbf{s}_k, \mathbf{y}_k)$ pairs, and ignoring older information. The storage requirements are therefore $O(mD)$. Typically $m \sim 20$ suffices for good performance. See (Nocedal and Wright 2006, p177) for more information. L-BFGS is often the method of choice for most unconstrained smooth optimization problems that arise in machine learning (although see Section 8.5).

8.3.6 ℓ_2 regularization

Just as we prefer ridge regression to linear regression, so we should prefer MAP estimation for logistic regression to computing the MLE. In fact, regularization is important in the classification setting even if we have lots of data. To see why, suppose the data is linearly separable. In this case, the MLE is obtained when $\|\mathbf{w}\| \rightarrow \infty$, corresponding to an infinitely steep sigmoid function, $\mathbb{I}(\mathbf{w}^T \mathbf{x} > w_0)$, also known as a **linear threshold unit**. This assigns the maximal amount of probability mass to the training data. However, such a solution is very brittle and will not generalize well.

To prevent this, we can use ℓ_2 regularization, just as we did with ridge regression. We note that the new objective, gradient and Hessian have the following forms:

$$f'(\mathbf{w}) = \text{NLL}(\mathbf{w}) + \lambda \mathbf{w}^T \mathbf{w} \quad (8.30)$$

$$\mathbf{g}'(\mathbf{w}) = \mathbf{g}(\mathbf{w}) + \lambda \mathbf{w} \quad (8.31)$$

$$\mathbf{H}'(\mathbf{w}) = \mathbf{H}(\mathbf{w}) + \lambda \mathbf{I} \quad (8.32)$$

It is a simple matter to pass these modified equations into any gradient-based optimizer.

8.3.7 Multi-class logistic regression

Now we consider **multinomial logistic regression**, sometimes called a **maximum entropy classifier**. This is a model of the form

$$p(y = c | \mathbf{x}, \mathbf{W}) = \frac{\exp(\mathbf{w}_c^T \mathbf{x})}{\sum_{c'=1}^C \exp(\mathbf{w}_{c'}^T \mathbf{x})} \quad (8.33)$$

A slight variant, known as a **conditional logit model**, normalizes over a different set of classes for each data case; this can be useful for modeling choices that users make between different sets of items that are offered to them.

Let us now introduce some notation. Let $\mu_{ic} = p(y_i = c | \mathbf{x}_i, \mathbf{W}) = \mathcal{S}(\boldsymbol{\eta}_i)_c$, where $\boldsymbol{\eta}_i = \mathbf{W}^T \mathbf{x}_i$ is a $C \times 1$ vector. Also, let $y_{ic} = \mathbb{I}(y_i = c)$ be the one-of- C encoding of y_i ; thus $\mathbf{y}_i$ is a bit vector, in which the c 'th bit turns on iff $y_i = c$. Following (Krishnapuram et al. 2005), let us

set $\mathbf{w}_C = \mathbf{0}$, to ensure identifiability, and define $\mathbf{w} = \text{vec}(\mathbf{W}(:, 1 : C-1))$ to be a $D \times (C-1)$ column vector.

With this, the log-likelihood can be written as

$$\ell(\mathbf{W}) = \log \prod_{i=1}^N \prod_{c=1}^C \mu_{ic}^{y_{ic}} = \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log \mu_{ic} \quad (8.34)$$

$$= \sum_{i=1}^N \left[\left(\sum_{c=1}^C y_{ic} \mathbf{w}_c^T \mathbf{x}_i \right) - \log \left(\sum_{c'=1}^C \exp(\mathbf{w}_{c'}^T \mathbf{x}_i) \right) \right] \quad (8.35)$$

Define the NLL as

$$f(\mathbf{w}) = -\ell(\mathbf{w}) \quad (8.36)$$

We now proceed to compute the gradient and Hessian of this expression. Since $\mathbf{w}$ is block-structured, the notation gets a bit heavy, but the ideas are simple. It helps to define $\mathbf{A} \otimes \mathbf{B}$ be the **kroncker product** of matrices $\mathbf{A}$ and $\mathbf{B}$. If $\mathbf{A}$ is an $m \times n$ matrix and $\mathbf{B}$ is a $p \times q$ matrix, then $\mathbf{A} \otimes \mathbf{B}$ is the $mp \times nq$ block matrix

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{11}\mathbf{B} & \cdots & a_{1n}\mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{m1}\mathbf{B} & \cdots & a_{mn}\mathbf{B} \end{bmatrix} \quad (8.37)$$

Returning to the task at hand, one can show (Exercise 8.4) that the gradient is given by

$$\mathbf{g}(\mathbf{W}) = \nabla f(\mathbf{w}) = \sum_{i=1}^N (\boldsymbol{\mu}_i - \mathbf{y}_i) \otimes \mathbf{x}_i \quad (8.38)$$

where $\mathbf{y}_i = (\mathbb{I}(y_i = 1), \dots, \mathbb{I}(y_i = C-1))$ and $\boldsymbol{\mu}_i(\mathbf{W}) = [p(y_i = 1|\mathbf{x}_i, \mathbf{W}), \dots, p(y_i = C-1|\mathbf{x}_i, \mathbf{W})]$ are column vectors of length $C-1$. For example, if we have $D = 3$ feature dimensions and $C = 3$ classes, this becomes

$$\mathbf{g}(\mathbf{W}) = \sum_i \begin{pmatrix} (\mu_{i1} - y_{i1})x_{i1} \\ (\mu_{i1} - y_{i1})x_{i2} \\ (\mu_{i1} - y_{i1})x_{i3} \\ (\mu_{i2} - y_{i2})x_{i1} \\ (\mu_{i2} - y_{i2})x_{i2} \\ (\mu_{i2} - y_{i2})x_{i3} \end{pmatrix} \quad (8.39)$$

In other words, for each class c , the derivative for the weights in the c 'th column is

$$\nabla_{\mathbf{w}_c} f(\mathbf{W}) = \sum_i (\mu_{ic} - y_{ic}) \mathbf{x}_i \quad (8.40)$$

This has the same form as in the binary logistic regression case, namely an error term times $\mathbf{x}_i$. (This turns out to be a general property of distributions in the exponential family, as we will see in Section 9.3.2.)

One can also show (Exercise 8.4) that the Hessian is the following block structured $D(C - 1) \times D(C - 1)$ matrix:

$$\mathbf{H}(\mathbf{W}) = \nabla^2 f(\mathbf{w}) = \sum_{i=1}^N (\text{diag}(\boldsymbol{\mu}_i) - \boldsymbol{\mu}_i \boldsymbol{\mu}_i^T) \otimes (\mathbf{x}_i \mathbf{x}_i^T) \quad (8.41)$$

For example, if we have 3 features and 3 classes, this becomes

$$\mathbf{H}(\mathbf{W}) = \sum_i \begin{pmatrix} \mu_{i1} - \mu_{i1}^2 & -\mu_{i1}\mu_{i2} \\ -\mu_{i1}\mu_{i2} & \mu_{i2} - \mu_{i2}^2 \end{pmatrix} \otimes \begin{pmatrix} x_{i1}x_{i1} & x_{i1}x_{i2} & x_{i1}x_{i3} \\ x_{i2}x_{i1} & x_{i2}x_{i2} & x_{i2}x_{i3} \\ x_{i3}x_{i1} & x_{i3}x_{i2} & x_{i3}x_{i3} \end{pmatrix} \quad (8.42)$$

$$= \sum_i \begin{pmatrix} (\mu_{i1} - \mu_{i1}^2)\mathbf{X}_i & -\mu_{i1}\mu_{i2}\mathbf{X}_i \\ -\mu_{i1}\mu_{i2}\mathbf{X}_i & (\mu_{i2} - \mu_{i2}^2)\mathbf{X}_i \end{pmatrix} \quad (8.43)$$

where $\mathbf{X}_i = \mathbf{x}_i \mathbf{x}_i^T$. In other words, the block c, c' submatrix is given by

$$\mathbf{H}_{c,c'}(\mathbf{W}) = \sum_i \mu_{ic} (\delta_{c,c'} - \mu_{i,c'}) \mathbf{x}_i \mathbf{x}_i^T \quad (8.44)$$

This is also a positive definite matrix, so there is a unique MLE.

Now consider minimizing

$$f'(\mathbf{W}) \triangleq -\log p(\mathcal{D}|\mathbf{w}) - \log p(\mathbf{W}) \quad (8.45)$$

where $p(\mathbf{W}) = \prod_c \mathcal{N}(\mathbf{w}_c | \mathbf{0}, \mathbf{V}_0)$. The new objective, its gradient and Hessian are given by

$$f'(\mathbf{W}) = f(\mathbf{W}) + \frac{1}{2} \sum_c \mathbf{w}_c \mathbf{V}_0^{-1} \mathbf{w}_c \quad (8.46)$$

$$\mathbf{g}'(\mathbf{W}) = \mathbf{g}(\mathbf{W}) + \mathbf{V}_0^{-1} \left(\sum_c \mathbf{w}_c \right) \quad (8.47)$$

$$\mathbf{H}'(\mathbf{W}) = \mathbf{H}(\mathbf{W}) + \mathbf{I}_C \otimes \mathbf{V}_0^{-1} \quad (8.48)$$

This can be passed to any gradient-based optimizer to find the MAP estimate. Note, however, that the Hessian has size $O((CD) \times (CD))$, which is C times more row and columns than in the binary case, so limited memory BFGS is more appropriate than Newton's method. See `logregFit` for some Matlab code.

8.4 Bayesian logistic regression

It is natural to want to compute the full posterior over the parameters, $p(\mathbf{w}|\mathcal{D})$, for logistic regression models. This can be useful for any situation where we want to associate confidence intervals with our predictions (e.g., this is necessary when solving contextual bandit problems, discussed in Section 5.7.3.1).

Unfortunately, unlike the linear regression case, this cannot be done exactly, since there is no convenient conjugate prior for logistic regression. We discuss one simple approximation below; some other approaches include MCMC (Section 24.3.3.1), variational inference (Section 21.8.1.1), expectation propagation (Kuss and Rasmussen 2005), etc. For notational simplicity, we stick to binary logistic regression.

8.4.1 Laplace approximation

In this section, we discuss how to make a Gaussian approximation to a posterior distribution. The approximation works as follows. Suppose $\boldsymbol{\theta} \in \mathbb{R}^D$. Let

$$p(\boldsymbol{\theta}|\mathcal{D}) = \frac{1}{Z} e^{-E(\boldsymbol{\theta})} \quad (8.49)$$

where $E(\boldsymbol{\theta})$ is called an **energy function**, and is equal to the negative log of the unnormalized log posterior, $E(\boldsymbol{\theta}) = -\log p(\boldsymbol{\theta}, \mathcal{D})$, with $Z = p(\mathcal{D})$ being the normalization constant. Performing a Taylor series expansion around the mode $\boldsymbol{\theta}^*$ (i.e., the lowest energy state) we get

$$E(\boldsymbol{\theta}) \approx E(\boldsymbol{\theta}^*) + (\boldsymbol{\theta} - \boldsymbol{\theta}^*)^T \mathbf{g} + \frac{1}{2} (\boldsymbol{\theta} - \boldsymbol{\theta}^*)^T \mathbf{H} (\boldsymbol{\theta} - \boldsymbol{\theta}^*) \quad (8.50)$$

where $\mathbf{g}$ is the gradient and $\mathbf{H}$ is the Hessian of the energy function evaluated at the mode:

$$\mathbf{g} \triangleq \nabla E(\boldsymbol{\theta})|_{\boldsymbol{\theta}^*}, \quad \mathbf{H} \triangleq \frac{\partial^2 E(\boldsymbol{\theta})}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}^T} |_{\boldsymbol{\theta}^*} \quad (8.51)$$

Since $\boldsymbol{\theta}^*$ is the mode, the gradient term is zero. Hence

$$\hat{p}(\boldsymbol{\theta}|\mathcal{D}) \approx \frac{1}{Z} e^{-E(\boldsymbol{\theta}^*)} \exp \left[-\frac{1}{2} (\boldsymbol{\theta} - \boldsymbol{\theta}^*)^T \mathbf{H} (\boldsymbol{\theta} - \boldsymbol{\theta}^*) \right] \quad (8.52)$$

$$= \mathcal{N}(\boldsymbol{\theta}|\boldsymbol{\theta}^*, \mathbf{H}^{-1}) \quad (8.53)$$

$$Z = p(\mathcal{D}) \approx \int \hat{p}(\boldsymbol{\theta}|\mathcal{D}) d\boldsymbol{\theta} = e^{-E(\boldsymbol{\theta}^*)} (2\pi)^{D/2} |\mathbf{H}|^{-\frac{1}{2}} \quad (8.54)$$

The last line follows from normalization constant of the multivariate Gaussian.

Equation 8.54 is known as the **Laplace approximation** to the marginal likelihood. Therefore Equation 8.52 is sometimes called the **Laplace approximation** to the posterior. However, in the statistics community, the term “Laplace approximation” refers to a more sophisticated method (see e.g. (Rue et al. 2009) for details). It may therefore be better to use the term “**Gaussian approximation**” to refer to Equation 8.52. A Gaussian approximation is often a reasonable approximation, since posteriors often become more “Gaussian-like” as the sample size increases, for reasons analogous to the central limit theorem. (In physics, there is an analogous technique known as a **saddle point approximation**.)

8.4.2 Derivation of the BIC

We can use the Gaussian approximation to write the log marginal likelihood as follows, dropping irrelevant constants:

$$\log p(\mathcal{D}) \approx \log p(\mathcal{D}|\boldsymbol{\theta}^*) + \log p(\boldsymbol{\theta}^*) - \frac{1}{2} \log |\mathbf{H}| \quad (8.55)$$

The penalization terms which are added to the $\log p(\mathcal{D}|\boldsymbol{\theta}^*)$ are sometimes called the **Occam factor**, and are a measure of model complexity. If we have a uniform prior, $p(\boldsymbol{\theta}) \propto 1$, we can drop the second term, and replace $\boldsymbol{\theta}^*$ with the MLE, $\hat{\boldsymbol{\theta}}$.

We now focus on approximating the third term. We have $\mathbf{H} = \sum_{i=1}^N \mathbf{H}_i$, where $\mathbf{H}_i = \nabla \nabla \log p(\mathcal{D}_i | \boldsymbol{\theta})$. Let us approximate each $\mathbf{H}_i$ by a fixed matrix $\hat{\mathbf{H}}$. Then we have

$$\log |\mathbf{H}| = \log |N\hat{\mathbf{H}}| = \log(N^d |\hat{\mathbf{H}}|) = D \log N + \log |\hat{\mathbf{H}}| \quad (8.56)$$

where $D = \dim(\boldsymbol{\theta})$ and we have assumed $\mathbf{H}$ is full rank. We can drop the $\log |\hat{\mathbf{H}}|$ term, since it is independent of N , and thus will get overwhelmed by the likelihood. Putting all the pieces together, we recover the BIC score (Section 5.3.2.4):

$$\log p(\mathcal{D}) \approx \log p(\mathcal{D} | \hat{\boldsymbol{\theta}}) - \frac{D}{2} \log N \quad (8.57)$$

8.4.3 Gaussian approximation for logistic regression

Now let us apply the Gaussian approximation to logistic regression. We will use a Gaussian prior of the form $p(\mathbf{w}) = \mathcal{N}(\mathbf{w} | \mathbf{0}, \mathbf{V}_0)$, just as we did in MAP estimation. The approximate posterior is given by

$$p(\mathbf{w} | \mathcal{D}) \approx \mathcal{N}(\mathbf{w} | \hat{\mathbf{w}}, \mathbf{H}^{-1}) \quad (8.58)$$

where $\hat{\mathbf{w}} = \arg \min_{\mathbf{w}} E(\mathbf{w})$, $E(\mathbf{w}) = -(\log p(\mathcal{D} | \mathbf{w}) + \log p(\mathbf{w}))$, and $\mathbf{H} = \nabla^2 E(\mathbf{w})|_{\hat{\mathbf{w}}}$.

As an example, consider the linearly separable 2D data in Figure 8.5(a). There are many parameter settings that correspond to lines that perfectly separate the training data; we show 4 examples. The likelihood surface is shown in Figure 8.5(b), where we see that the likelihood is unbounded as we move up and to the right in parameter space, along a ridge where $w_2/w_1 = 2.35$ (this is indicated by the diagonal line). The reasons for this is that we can maximize the likelihood by driving $\|\mathbf{w}\|$ to infinity (subject to being on this line), since large regression weights make the sigmoid function very steep, turning it into a step function. Consequently the MLE is not well defined when the data is linearly separable.

To regularize the problem, let us use a vague spherical prior centered at the origin, $\mathcal{N}(\mathbf{w} | \mathbf{0}, 100\mathbf{I})$. Multiplying this spherical prior by the likelihood surface results in a highly skewed posterior, shown in Figure 8.5(c). (The posterior is skewed because the likelihood function “chops off” regions of parameter space (in a “soft” fashion) which disagree with the data.) The MAP estimate is shown by the blue dot. Unlike the MLE, this is not at infinity.

The Gaussian approximation to this posterior is shown in Figure 8.5(d). We see that this is a symmetric distribution, and therefore not a great approximation. Of course, it gets the mode correct (by construction), and it at least represents the fact that there is more uncertainty along the southwest-northeast direction (which corresponds to uncertainty about the orientation of separating lines) than perpendicular to this. Although a crude approximation, this is surely better than approximating the posterior by a delta function, which is what MAP estimation does.

8.4.4 Approximating the posterior predictive

Given the posterior, we can compute credible intervals, perform hypothesis tests, etc., just as we did in Section 7.6.3.3 in the case of linear regression. But in machine learning, interest usually focusses on prediction. The posterior predictive distribution has the form

$$p(y | \mathbf{x}, \mathcal{D}) = \int p(y | \mathbf{x}, \mathbf{w}) p(\mathbf{w} | \mathcal{D}) d\mathbf{w} \quad (8.59)$$

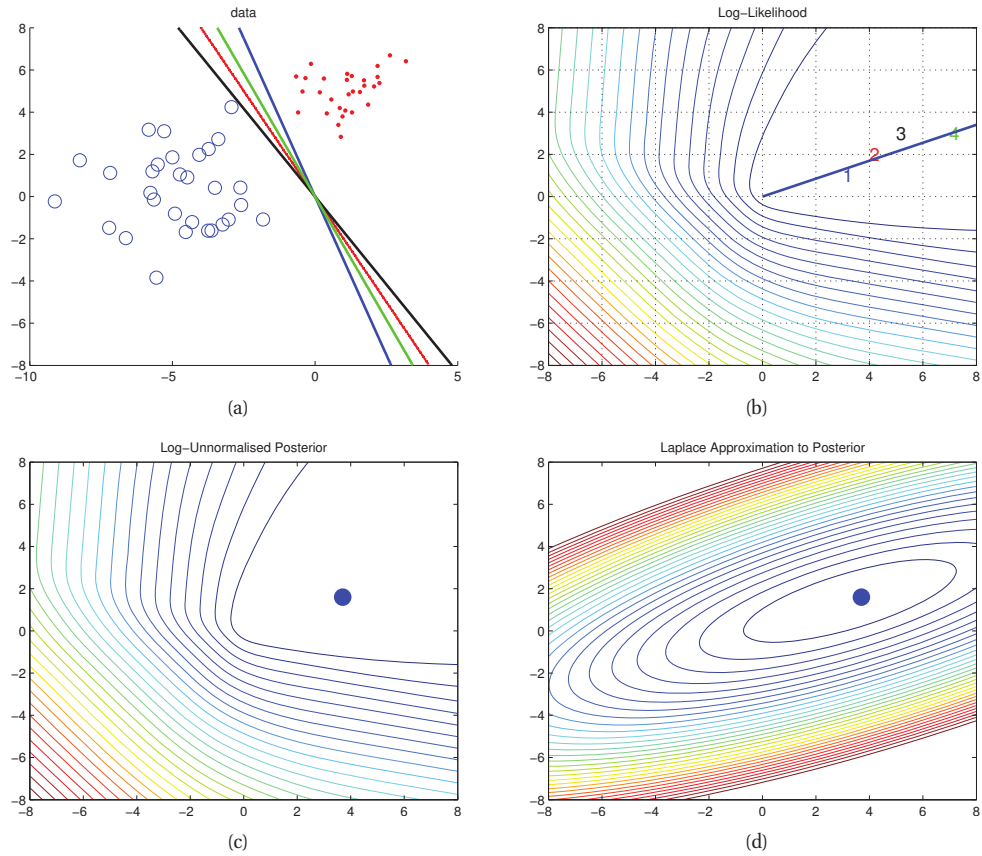


Figure 8.5 (a) Two-class data in 2d. (b) Log-likelihood for a logistic regression model. The line is drawn from the origin in the direction of the MLE (which is at infinity). The numbers correspond to 4 points in parameter space, corresponding to the lines in (a). (c) Unnormalized log posterior (assuming vague spherical prior). (d) Laplace approximation to posterior. Based on a figure by Mark Girolami. Figure generated by `logregLaplaceGirolamiDemo`.

Unfortunately this integral is intractable.

The simplest approximation is the plug-in approximation, which, in the binary case, takes the form

$$p(y = 1|\mathbf{x}, \mathcal{D}) \approx p(y = 1|\mathbf{x}, \mathbb{E}[\mathbf{w}]) \quad (8.60)$$

where $\mathbb{E}[\mathbf{w}]$ is the posterior mean. In this context, $\mathbb{E}[\mathbf{w}]$ is called the **Bayes point**. Of course, such a plug-in estimate underestimates the uncertainty. We discuss some better approximations below.

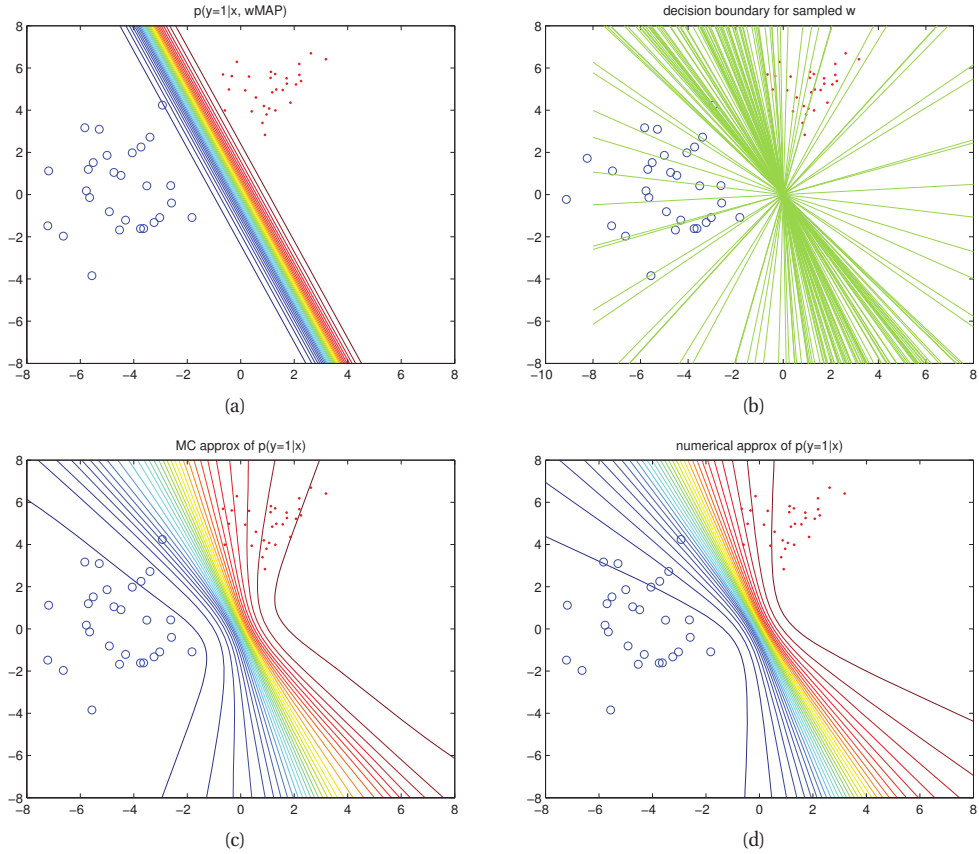


Figure 8.6 Posterior predictive distribution for a logistic regression model in 2d. Top left: contours of $p(y = 1|\mathbf{x}, \hat{\mathbf{w}}_{MAP})$. Top right: samples from the posterior predictive distribution. Bottom left: Averaging over these samples. Bottom right: moderated output (probit approximation). Based on a figure by Mark Girolami. Figure generated by `logregLaplaceGirolamiDemo`.

8.4.4.1 Monte Carlo approximation

A better approach is to use a Monte Carlo approximation, as follows:

$$p(y = 1|\mathbf{x}, \mathcal{D}) \approx \frac{1}{S} \sum_{s=1}^S \text{sigm}((\mathbf{w}^s)^T \mathbf{x}) \quad (8.61)$$

where $\mathbf{w}^s \sim p(\mathbf{w}|\mathcal{D})$ are samples from the posterior. (This technique can be trivially extended to the multi-class case.) If we have approximated the posterior using Monte Carlo, we can reuse these samples for prediction. If we made a Gaussian approximation to the posterior, we can draw *independent* samples from the Gaussian using standard methods.

Figure 8.6(b) shows samples from the posterior predictive for our 2d example. Figure 8.6(c)

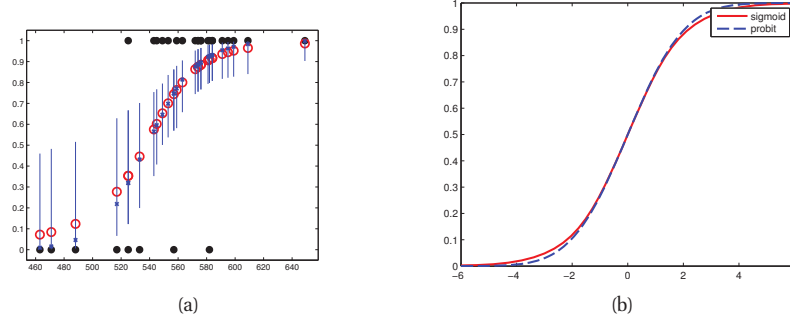


Figure 8.7 (a) Posterior predictive density for SAT data. The red circle denotes the posterior mean, the blue cross the posterior median, and the blue lines denote the 5th and 95th percentiles of the predictive distribution. Figure generated by `logregSATdemoBayes`. (b) The logistic (sigmoid) function $\text{sigm}(x)$ in solid red, with the rescaled probit function $\Phi(\lambda x)$ in dotted blue superimposed. Here $\lambda = \sqrt{\pi/8}$, which was chosen so that the derivatives of the two curves match at $x = 0$. Based on Figure 4.9 of (Bishop 2006b). Figure generated by `probitPlot`. Figure generated by `probitRegDemo`.

shows the average of these samples. By averaging over multiple predictions, we see that the uncertainty in the decision boundary “splays out” as we move further from the training data. So although the decision boundary is linear, the posterior predictive density is not linear. Note also that the posterior mean decision boundary is roughly equally far from both classes; this is the Bayesian analog of the large margin principle discussed in Section 14.5.2.2.

Figure 8.7(a) shows an example in 1d. The red dots denote the mean of the posterior predictive evaluated at the training data. The vertical blue lines denote 95% credible intervals for the posterior predictive; the small blue star is the median. We see that, with the Bayesian approach, we are able to model our uncertainty about the probability a student will pass the exam based on his SAT score, rather than just getting a point estimate.

8.4.4.2 Probit approximation (moderated output) *

If we have a Gaussian approximation to the posterior $p(\mathbf{w}|\mathcal{D}) \approx \mathcal{N}(\mathbf{w}|\mathbf{m}_N, \mathbf{V}_N)$, we can also compute a deterministic approximation to the posterior predictive distribution, at least in the binary case. We proceed as follows:

$$p(y = 1|\mathbf{x}, \mathcal{D}) \approx \int \text{sigm}(\mathbf{w}^T \mathbf{x}) p(\mathbf{w}|\mathcal{D}) d\mathbf{w} = \int \text{sigm}(a) \mathcal{N}(a|\mu_a, \sigma_a^2) da \quad (8.62)$$

$$a \triangleq \mathbf{w}^T \mathbf{x} \quad (8.63)$$

$$\mu_a \triangleq \mathbb{E}[a] = \mathbf{m}_N^T \mathbf{x} \quad (8.64)$$

$$\sigma_a^2 \triangleq \text{var}[a] = \int p(a|\mathcal{D})[a^2 - \mathbb{E}[a^2]] da \quad (8.65)$$

$$= \int p(\mathbf{w}|\mathcal{D})[(\mathbf{w}^T \mathbf{x})^2 - (\mathbf{m}_N^T \mathbf{x})^2] d\mathbf{w} = \mathbf{x}^T \mathbf{V}_N \mathbf{x} \quad (8.66)$$

Thus we see that we need to evaluate the expectation of a sigmoid with respect to a Gaussian. This can be approximated by exploiting the fact that the sigmoid function is similar to the **probit** function, which is given by the cdf of the standard normal:

$$\Phi(a) \triangleq \int_{-\infty}^a \mathcal{N}(x|0, 1)dx \quad (8.67)$$

Figure 8.7(b) plots the sigmoid and probit functions. We have rescaled the axes so that $\text{sigm}(a)$ has the same slope as $\Phi(\lambda a)$ at the origin, where $\lambda^2 = \pi/8$.

The advantage of using the probit is that one can convolve it with a Gaussian analytically:

$$\int \Phi(\lambda a) \mathcal{N}(a|\mu, \sigma^2) da = \Phi\left(\frac{a}{(\lambda^{-2} + \sigma^2)^{\frac{1}{2}}}\right) \quad (8.68)$$

We now plug in the approximation $\text{sigm}(a) \approx \Phi(\lambda a)$ to both sides of this equation to get

$$\int \text{sigm}(a) \mathcal{N}(a|\mu, \sigma^2) da \approx \text{sigm}(\kappa(\sigma^2)\mu) \quad (8.69)$$

$$\kappa(\sigma^2) \triangleq (1 + \pi\sigma^2/8)^{-\frac{1}{2}} \quad (8.70)$$

Applying this to the logistic regression model we get the following expression (first suggested in (Spiegelhalter and Lauritzen 1990)):

$$p(y = 1|\mathbf{x}, \mathcal{D}) \approx \text{sigm}(\kappa(\sigma_a^2)\mu_a) \quad (8.71)$$

Figure 8.6(d) indicates that this gives very similar results to the Monte Carlo approximation.

Using Equation 8.71 is sometimes called a **moderated output**, since it is less extreme than the plug-in estimate. To see this, note that $0 \leq \kappa(\sigma^2) \leq 1$ and hence

$$\text{sigm}(\kappa(\sigma^2)\mu) \leq \text{sigm}(\mu) = p(y = 1|\mathbf{x}, \hat{\mathbf{w}}) \quad (8.72)$$

where the inequality is strict if $\mu \neq 0$. If $\mu > 0$, we have $p(y = 1|\mathbf{x}, \hat{\mathbf{w}}) > 0.5$, but the moderated prediction is always closer to 0.5, so it is less confident. However, the decision boundary occurs whenever $p(y = 1|\mathbf{x}, \mathcal{D}) = \text{sigm}(\kappa(\sigma^2)\mu) = 0.5$, which implies $\mu = \hat{\mathbf{w}}^T \mathbf{x} = 0$. Hence the decision boundary for the moderated approximation is the same as for the plug-in approximation. So the number of misclassifications will be the same for the two methods, but the log-likelihood will not. (Note that in the multiclass case, taking into account posterior covariance gives different answers than the plug-in approach: see Exercise 3.10.3 of (Rasmussen and Williams 2006).)

8.4.5 Residual analysis (outlier detection) *

It is sometimes useful to detect data cases which are “outliers”. This is called **residual analysis** or **case analysis**. In a regression setting, this can be performed by computing $r_i = y_i - \hat{y}_i$, where $\hat{y}_i = \hat{\mathbf{w}}^T \mathbf{x}_i$. These values should follow a $\mathcal{N}(0, \sigma^2)$ distribution, if the modelling assumptions are correct. This can be assessed by creating a **qq-plot**, where we plot the N theoretical quantiles of a Gaussian distribution against the N empirical quantiles of the r_i . Points that deviate from the straightline are potential outliers.

Classical methods, based on residuals, do not work well for binary data, because they rely on asymptotic normality of the test statistics. However, adopting a Bayesian approach, we can just define outliers to be points which which $p(y_i|\hat{y}_i)$ is small, where we typically use $\hat{y}_i = \text{sigm}(\hat{\mathbf{w}}^T \mathbf{x}_i)$. Note that $\hat{\mathbf{w}}$ was estimated from all the data. A better method is to exclude $(\mathbf{x}_i, y_i)$ from the estimate of $\mathbf{w}$ when predicting y_i . That is, we define outliers to be points which have low probability under the cross-validated posterior predictive distribution, defined by

$$p(y_i|\mathbf{x}_i, \mathbf{x}_{-i}, \mathbf{y}_{-i}) = \int p(y_i|\mathbf{x}_i, \mathbf{w}) \prod_{i' \neq i} p(y_{i'}|\mathbf{x}_{i'}, \mathbf{w}) p(\mathbf{w}) d\mathbf{w} \quad (8.73)$$

This can be efficiently approximated by sampling methods (Gelfand 1996). For further discussion of residual analysis in logistic regression models, see e.g., (Johnson and Albert 1999, Sec 3.4).

8.5 Online learning and stochastic optimization

Traditionally machine learning is performed **offline**, which means we have a **batch** of data, and we optimize an equation of the following form

$$f(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N f(\boldsymbol{\theta}, \mathbf{z}_i) \quad (8.74)$$

where $\mathbf{z}_i = (\mathbf{x}_i, y_i)$ in the supervised case, or just $\mathbf{x}_i$ in the unsupervised case, and $f(\boldsymbol{\theta}, \mathbf{z}_i)$ is some kind of loss function. For example, we might use

$$f(\boldsymbol{\theta}, \mathbf{z}_i) = -\log p(y_i|\mathbf{x}_i, \boldsymbol{\theta}) \quad (8.75)$$

in which case we are trying to maximize the likelihood. Alternatively, we might use

$$f(\boldsymbol{\theta}, \mathbf{z}_i) = L(y_i, h(\mathbf{x}_i, \boldsymbol{\theta})) \quad (8.76)$$

where $h(\mathbf{x}_i, \boldsymbol{\theta})$ is a prediction function, and $L(y, \hat{y})$ is some other loss function such as squared error or the Huber loss. In frequentist decision theory, the average loss is called the risk (see Section 6.3), so this overall approach is called empirical risk minimization or ERM (see Section 6.5 for details).

However, if we have **streaming data**, we need to perform **online learning**, so we can update our estimates as each new data point arrives rather than waiting until “the end” (which may never occur). And even if we have a batch of data, we might want to treat it like a stream if it is too large to hold in main memory. Below we discuss learning methods for this kind of scenario.¹

1. A simple implementation trick can be used to speed up batch learning algorithms when applied to data sets that are too large to hold in memory. First note that the naive implementation makes a pass over the data file, from the beginning to end, accumulating the sufficient statistics and gradients as it goes; then an update is performed and the process repeats. Unfortunately, at the end of each pass, the data from the beginning of the file will have been evicted from the cache (since we are assuming it cannot all fit into memory). Rather than going back to the beginning of the file and reloading it, we can simply work backwards from the end of the file, which is already in memory. We then repeat this forwards-backwards pattern over the data. This simple trick is known as **rocking**.

8.5.1 Online learning and regret minimization

Suppose that at each step, “nature” presents a sample $\mathbf{z}_k$ and the “learner” must respond with a parameter estimate $\boldsymbol{\theta}_k$. In the theoretical machine learning community, the objective used in online learning is the **regret**, which is the averaged loss incurred relative to the best we could have gotten in hindsight using a single fixed parameter value:

$$\text{regret}_k \triangleq \frac{1}{k} \sum_{t=1}^k f(\boldsymbol{\theta}_t, \mathbf{z}_t) - \min_{\boldsymbol{\theta}^* \in \Theta} \frac{1}{k} \sum_{t=1}^k f(\boldsymbol{\theta}^*, \mathbf{z}_t) \quad (8.77)$$

For example, imagine we are investing in the stock-market. Let θ_j be the amount we invest in stock j , and let z_j be the return on this stock. Our loss function is $f(\boldsymbol{\theta}, \mathbf{z}) = -\boldsymbol{\theta}^T \mathbf{z}$. The regret is how much better (or worse) we did by trading at each step, rather than adopting a “buy and hold” strategy using an oracle to choose which stocks to buy.

One simple algorithm for online learning is **online gradient descent** (Zinkevich 2003), which is as follows: at each step k , update the parameters using

$$\boldsymbol{\theta}_{k+1} = \text{proj}_{\Theta}(\boldsymbol{\theta}_k - \eta_k \mathbf{g}_k) \quad (8.78)$$

where $\text{proj}_{\mathcal{V}}(\mathbf{v}) = \text{argmin}_{\mathbf{w} \in \mathcal{V}} \|\mathbf{w} - \mathbf{v}\|_2$ is the projection of vector $\mathbf{v}$ onto space $\mathcal{V}$, $\mathbf{g}_k = \nabla f(\boldsymbol{\theta}_k, \mathbf{z}_k)$ is the gradient, and η_k is the step size. (The projection step is only needed if the parameter must be constrained to live in a certain subset of $\mathbb{R}^D$. See Section 13.4.3 for details.) Below we will see how this approach to regret minimization relates to more traditional objectives, such as MLE.

There are a variety of other approaches to regret minimization which are beyond the scope of this book (see e.g., Cesa-Bianchi and Lugosi (2006) for details).

8.5.2 Stochastic optimization and risk minimization

Now suppose that instead of minimizing regret with respect to the past, we want to minimize expected loss in the future, as is more common in (frequentist) statistical learning theory. That is, we want to minimize

$$f(\boldsymbol{\theta}) = \mathbb{E}[f(\boldsymbol{\theta}, \mathbf{z})] \quad (8.79)$$

where the expectation is taken over future data. Optimizing functions where some of the variables in the objective are random is called **stochastic optimization**.²

Suppose we receive an infinite stream of samples from the distribution. One way to optimize stochastic objectives such as Equation 8.79 is to perform the update in Equation 8.78 at each step. This is called **stochastic gradient descent** or **SGD** (Nemirovski and Yudin 1978). Since we typically want a single parameter estimate, we can use a running average:

$$\bar{\boldsymbol{\theta}}_k = \frac{1}{k} \sum_{t=1}^k \boldsymbol{\theta}_t \quad (8.80)$$

2. Note that in stochastic optimization, the objective is stochastic, and therefore the algorithms will be, too. However, it is also possible to apply stochastic optimization algorithms to deterministic objectives. Examples include simulated annealing (Section 24.6.1) and stochastic gradient descent applied to the empirical risk minimization problem. There are some interesting theoretical connections between online learning and stochastic optimization (Cesa-Bianchi and Lugosi 2006), but this is beyond the scope of this book.

This is called **Polyak-Ruppert averaging**, and can be implemented recursively as follows:

$$\bar{\theta}_k = \bar{\theta}_{k-1} - \frac{1}{k}(\bar{\theta}_{k-1} - \theta_k) \quad (8.81)$$

See e.g., (Spall 2003; Kushner and Yin 2003) for details.

8.5.2.1 Setting the step size

We now discuss some sufficient conditions on the learning rate to guarantee convergence of SGD. These are known as the **Robbins-Monro** conditions:

$$\sum_{k=1}^{\infty} \eta_k = \infty, \quad \sum_{k=1}^{\infty} \eta_k^2 < \infty. \quad (8.82)$$

The set of values of η_k over time is called the learning rate **schedule**. Various formulas are used, such as $\eta_k = 1/k$, or the following (Bottou 1998; Bach and Moulines 2011):

$$\eta_k = (\tau_0 + k)^{-\kappa} \quad (8.83)$$

where $\tau_0 \geq 0$ slows down early iterations of the algorithm, and $\kappa \in (0.5, 1]$ controls the rate at which old values of are forgotten.

The need to adjust these tuning parameters is one of the main drawback of stochastic optimization. One simple heuristic (Bottou 2007) is as follows: store an initial subset of the data, and try a range of η values on this subset; then choose the one that results in the fastest decrease in the objective and apply it to all the rest of the data. Note that this may not result in convergence, but the algorithm can be terminated when the performance improvement on a hold-out set plateaus (this is called **early stopping**).

8.5.2.2 Per-parameter step sizes

One drawback of SGD is that it uses the same step size for all parameters. We now briefly present a method known as **adagrad** (short for adaptive gradient) (Duchi et al. 2010), which is similar in spirit to a diagonal Hessian approximation. (See also (Schaul et al. 2012) for a similar approach.) In particular, if $\theta_i(k)$ is parameter i at time k , and $g_i(k)$ is its gradient, then we make an update as follows:

$$\theta_i(k+1) = \theta_i(k) - \eta \frac{g_i(k)}{\tau_0 + \sqrt{s_i(k)}} \quad (8.84)$$

where the diagonal step size vector is the gradient vector squared, summed over all time steps. This can be recursively updated as follows:

$$s_i(k) = s_i(k-1) + g_i(k)^2 \quad (8.85)$$

The result is a per-parameter step size that adapts to the curvature of the loss function. This method was original derived for the regret minimization case, but it can be applied more generally.

8.5.2.3 SGD compared to batch learning

If we don't have an infinite data stream, we can “simulate” one by sampling data points at random from our training set. Essentially we are optimizing Equation 8.74 by treating it as an expectation with respect to the empirical distribution.

Algorithm 8.3: Stochastic gradient descent

```

1 Initialize  $\theta, \eta$ ;
2 repeat
3   Randomly permute data;
4   for  $i = 1 : N$  do
5      $\mathbf{g} = \nabla f(\theta, \mathbf{z}_i)$ ;
6      $\theta \leftarrow \text{proj}_{\Theta}(\theta - \eta \mathbf{g})$ ;
7     Update  $\eta$ ;
8 until converged;
```

In theory, we should sample with replacement, although in practice it is usually better to randomly permute the data and sample without replacement, and then to repeat. A single such pass over the entire data set is called an **epoch**. See Algorithm 8 for some pseudocode.

In this offline case, it is often better to compute the gradient of a **mini-batch** of B data cases. If $B = 1$, this is standard SGD, and if $B = N$, this is standard **steepest descent**. Typically $B \sim 100$ is used.

Although a simple first-order method, SGD performs surprisingly well on some problems, especially ones with large data sets (Bottou 2007). The intuitive reason for this is that one can get a fairly good estimate of the gradient by looking at just a few examples. Carefully evaluating precise gradients using large datasets is often a waste of time, since the algorithm will have to recompute the gradient again anyway at the next step. It is often a better use of computer time to have a noisy estimate and to move rapidly through parameter space. As an extreme example, suppose we double the training set by duplicating every example. Batch methods will take twice as long, but online methods will be unaffected, since the direction of the gradient has not changed (doubling the size of the data changes the magnitude of the gradient, but that is irrelevant, since the gradient is being scaled by the step size anyway).

In addition to enhanced speed, SGD is often less prone to getting stuck in shallow local minima, because it adds a certain amount of “noise”. Consequently it is quite popular in the machine learning community for fitting models with non-convex objectives, such as neural networks (Section 16.5) and deep belief networks (Section 28.1).

8.5.3 The LMS algorithm

As an example of SGD, let us consider how to compute the MLE for linear regression in an online fashion. We derived the batch gradient in Equation 7.14. The online gradient at iteration k is given by

$$\mathbf{g}_k = \mathbf{x}_i(\theta_k^T \mathbf{x}_i - y_i) \quad (8.86)$$

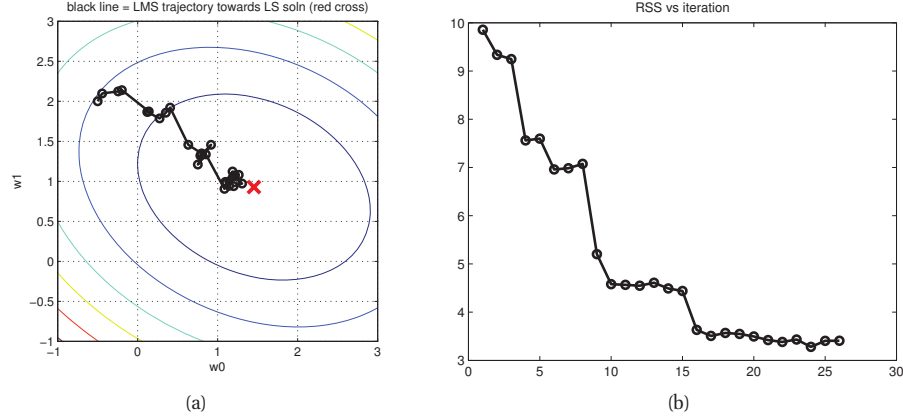


Figure 8.8 Illustration of the LMS algorithm. Left: we start from $\theta = (-0.5, 2)$ and slowly converging to the least squares solution of $\hat{\theta} = (1.45, 0.92)$ (red cross). Right: plot of objective function over time. Note that it does not decrease monotonically. Figure generated by LMSdemo.

where $i = i(k)$ is the training example to use at iteration k . If the data set is streaming, we use $i(k) = k$; we shall assume this from now on, for notational simplicity. Equation 8.86 is easy to interpret: it is the feature vector $\mathbf{x}_k$ weighted by the difference between what we predicted, $\hat{y}_k = \theta_k^T \mathbf{x}_k$, and the true response, y_k ; hence the gradient acts like an error signal.

After computing the gradient, we take a step along it as follows:

$$\theta_{k+1} = \theta_k - \eta_k (\hat{y}_k - y_k) \mathbf{x}_k \quad (8.87)$$

(There is no need for a projection step, since this is an unconstrained optimization problem.) This algorithm is called the **least mean squares** or **LMS** algorithm, and is also known as the **delta rule**, or the **Widrow-Hoff rule**.

Figure 8.8 shows the results of applying this algorithm to the data shown in Figure 7.2. We start at $\theta = (-0.5, 2)$ and converge (in the sense that $\|\theta_k - \theta_{k-1}\|_2^2$ drops below a threshold of 10^{-2}) in about 26 iterations.

Note that LMS may require multiple passes through the data to find the optimum. By contrast, the recursive least squares algorithm, which is based on the Kalman filter and which uses second-order information, finds the optimum in a single pass (see Section 18.2.3). See also Exercise 7.7.

8.5.4 The perceptron algorithm

Now let us consider how to fit a binary logistic regression model in an online manner. The batch gradient was given in Equation 8.5. In the online case, the weight update has the simple form

$$\theta_k = \theta_{k-1} - \eta_k \mathbf{g}_i = \theta_{k-1} - \eta_k (\mu_i - y_i) \mathbf{x}_i \quad (8.88)$$

where $\mu_i = p(y_i = 1 | \mathbf{x}_i, \theta_k) = \mathbb{E}[y_i | \mathbf{x}_i, \theta_k]$. We see that this has exactly the same form as the LMS algorithm. Indeed, this property holds for all generalized linear models (Section 9.3).

We now consider an approximation to this algorithm. Specifically, let

$$\hat{y}_i = \arg \max_{y \in \{0,1\}} p(y|\mathbf{x}_i, \boldsymbol{\theta}) \quad (8.89)$$

represent the most probable class label. We replace $\mu_i = p(y = 1|\mathbf{x}_i, \boldsymbol{\theta}) = \text{sigm}(\boldsymbol{\theta}^T \mathbf{x}_i)$ in the gradient expression with $\hat{y}_i$. Thus the approximate gradient becomes

$$\mathbf{g}_i \approx (\hat{y}_i - y_i)\mathbf{x}_i \quad (8.90)$$

It will make the algebra simpler if we assume $y \in \{-1, +1\}$ rather than $y \in \{0, 1\}$. In this case, our prediction becomes

$$\hat{y}_i = \text{sign}(\boldsymbol{\theta}^T \mathbf{x}_i) \quad (8.91)$$

Then if $\hat{y}_i y_i = -1$, we have made an error, but if $\hat{y}_i y_i = +1$, we guessed the right label.

At each step, we update the weight vector by adding on the gradient. The key observation is that, if we predicted correctly, then $\hat{y}_i = y_i$, so the (approximate) gradient is zero and we do not change the weight vector. But if $\mathbf{x}_i$ is misclassified, we update the weights as follows: If $\hat{y}_i = 1$ but $y_i = -1$, then the negative gradient is $-(\hat{y}_i - y_i)\mathbf{x}_i = -2\mathbf{x}_i$; and if $\hat{y}_i = -1$ but $y_i = 1$, then the negative gradient is $-(\hat{y}_i - y_i)\mathbf{x}_i = 2\mathbf{x}_i$. We can absorb the factor of 2 into the learning rate η and just write the update, in the case of a misclassification, as

$$\boldsymbol{\theta}_k = \boldsymbol{\theta}_{k-1} + \eta_k y_i \mathbf{x}_i \quad (8.92)$$

Since it is only the sign of the weights that matter, not the magnitude, we will set $\eta_k = 1$. See Algorithm 11 for the pseudocode.

One can show that this method, known as the **perceptron algorithm** (Rosenblatt 1958), will converge, provided the data is linearly separable, i.e., that there exist parameters $\boldsymbol{\theta}$ such that predicting with $\text{sign}(\boldsymbol{\theta}^T \mathbf{x})$ achieves 0 error on the training set. However, if the data is not linearly separable, the algorithm will not converge, and even if it does converge, it may take a long time. There are much better ways to train logistic regression models (such as using proper SGD, without the gradient approximation, or IRLS, discussed in Section 8.3.4). However, the perceptron algorithm is historically important: it was one of the first machine learning algorithms ever derived (by Frank Rosenblatt in 1957), and was even implemented in analog hardware. In addition, the algorithm can be used to fit models where computing marginals $p(y_i|\mathbf{x}, \boldsymbol{\theta})$ is more expensive than computing the MAP output, $\arg \max_{\mathbf{y}} p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta})$; this arises in some structured-output classification problems. See Section 19.7 for details.

8.5.5 A Bayesian view

Another approach to online learning is to adopt a Bayesian view. This is conceptually quite simple: we just apply Bayes rule recursively:

$$p(\boldsymbol{\theta}|\mathcal{D}_{1:k}) \propto p(\mathcal{D}_k|\boldsymbol{\theta})p(\boldsymbol{\theta}|\mathcal{D}_{1:k-1}) \quad (8.93)$$

This has the obvious advantage of returning a posterior instead of just a point estimate. It also allows for the online adaptation of hyper-parameters, which is important since cross-validation cannot be used in an online setting. Finally, it has the (less obvious) advantage that it can be

Algorithm 8.4: Perceptron algorithm

```

1 Input: linearly separable data set  $\mathbf{x}_i \in \mathbb{R}^D$ ,  $y_i \in \{-1, +1\}$  for  $i = 1 : N$ ;
2 Initialize  $\boldsymbol{\theta}_0$ ;
3  $k \leftarrow 0$ ;
4 repeat
5    $k \leftarrow k + 1$ ;
6    $i \leftarrow k \bmod N$ ;
7   if  $\hat{y}_i \neq y_i$  then
8      $\boldsymbol{\theta}_{k+1} \leftarrow \boldsymbol{\theta}_k + y_i \mathbf{x}_i$ 
9   else
10    no-op
11 until converged;

```

quicker than SGD. To see why, note that by modeling the posterior variance of each parameter in addition to its mean, we effectively associate a different learning rate for each parameter (de Freitas et al. 2000), which is a simple way to model the curvature of the space. These variances can then be adapted using the usual rules of probability theory. By contrast, getting second-order optimization methods to work online is more tricky (see e.g., (Schraudolph et al. 2007; Suneahg et al. 2009; Bordes et al. 2009, 2010)).

As a simple example, in Section 18.2.3 we show how to use the Kalman filter to fit a linear regression model online. Unlike the LMS algorithm, this converges to the optimal (offline) answer in a single pass over the data. An extension which can learn a robust non-linear regression model in an online fashion is described in (Ting et al. 2010). For the GLM case, we can use an assumed density filter (Section 18.5.3), where we approximate the posterior by a Gaussian with a diagonal covariance; the variance terms serve as a per-parameter step-size. See Section 18.5.3.2 for details. Another approach is to use particle filtering (Section 23.5); this was used in (Andrieu et al. 2000) for sequentially learning a kernelized linear/logistic regression model.

8.6 Generative vs discriminative classifiers

In Section 4.2.2, we showed that the posterior over class labels induced by Gaussian discriminant analysis (GDA) has exactly the same form as logistic regression, namely $p(y = 1|\mathbf{x}) = \text{sigm}(\mathbf{w}^T \mathbf{x})$. The decision boundary is therefore a linear function of $\mathbf{x}$ in both cases. Note, however, that many generative models can give rise to a logistic regression posterior, e.g., if each class-conditional density is Poisson, $p(x|y = c) = \text{Poi}(x|\lambda_c)$. So the assumptions made by GDA are much stronger than the assumptions made by logistic regression.

A further difference between these models is the way they are trained. When fitting a discriminative model, we usually maximize the conditional log likelihood $\sum_{i=1}^N \log p(y_i|\mathbf{x}_i, \boldsymbol{\theta})$, whereas when fitting a generative model, we usually maximize the joint log likelihood, $\sum_{i=1}^N \log p(y_i, \mathbf{x}_i|\boldsymbol{\theta})$. It is clear that these can, in general, give different results (see Exercise 4.20).

When the Gaussian assumptions made by GDA are correct, the model will need less training data than logistic regression to achieve a certain level of performance, but if the Gaussian

assumptions are incorrect, logistic regression will do better (Ng and Jordan 2002). This is because discriminative models do not need to model the distribution of the features. This is illustrated in Figure 8.10. We see that the class conditional densities are rather complex; in particular, $p(x|y = 1)$ is a multimodal distribution, which might be hard to estimate. However, the class posterior, $p(y = c|x)$, is a simple sigmoidal function, centered on the threshold value of 0.55. This suggests that, in general, discriminative methods will be more accurate, since their “job” is in some sense easier. However, accuracy is not the only important factor when choosing a method. Below we discuss some other advantages and disadvantages of each approach.

8.6.1 Pros and cons of each approach

- **Easy to fit?** As we have seen, it is usually very easy to fit generative classifiers. For example, in Sections 3.5.1.1 and 4.2.4, we show that we can fit a naive Bayes model and an LDA model by simple counting and averaging. By contrast, logistic regression requires solving a convex optimization problem (see Section 8.3.4 for the details), which is much slower.
- **Fit classes separately?** In a generative classifier, we estimate the parameters of each class conditional density independently, so we do not have to retrain the model when we add more classes. In contrast, in discriminative models, all the parameters interact, so the whole model must be retrained if we add a new class. (This is also the case if we train a generative model to maximize a discriminative objective Salojärvi et al. (2005).)
- **Handle missing features easily?** Sometimes some of the inputs (components of $\mathbf{x}$) are not observed. In a generative classifier, there is a simple method for dealing with this, as we discuss in Section 8.6.2. However, in a discriminative classifier, there is no principled solution to this problem, since the model assumes that $\mathbf{x}$ is always available to be conditioned on (although see (Marlin 2008) for some heuristic approaches).
- **Can handle unlabeled training data?** There is much interest in **semi-supervised learning**, which uses unlabeled data to help solve a supervised task. This is fairly easy to do using generative models (see e.g., (Lasserre et al. 2006; Liang et al. 2007)), but is much harder to do with discriminative models.
- **Symmetric in inputs and outputs?** We can run a generative model “backwards”, and infer probable inputs given the output by computing $p(\mathbf{x}|y)$. This is not possible with a discriminative model. The reason is that a generative model defines a joint distribution on $\mathbf{x}$ and y , and hence treats both inputs and outputs symmetrically.
- **Can handle feature preprocessing?** A big advantage of discriminative methods is that they allow us to preprocess the input in arbitrary ways, e.g., we can replace $\mathbf{x}$ with $\phi(\mathbf{x})$, which could be some basis function expansion, as illustrated in Figure 8.9. It is often hard to define a generative model on such pre-processed data, since the new features are correlated in complex ways.
- **Well-calibrated probabilities?** Some generative models, such as naive Bayes, make strong independence assumptions which are often not valid. This can result in very extreme posterior class probabilities (very near 0 or 1). Discriminative models, such as logistic regression, are usually better calibrated in terms of their probability estimates.

We see that there are arguments for and against both kinds of models. It is therefore useful to have both kinds in your “toolbox”. See Table 8.1 for a summary of the classification and

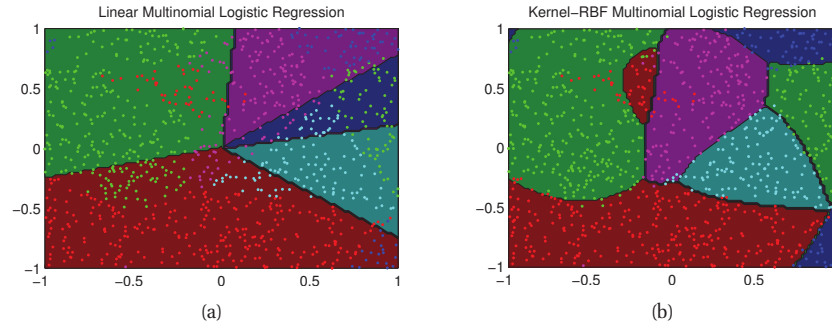


Figure 8.9 (a) Multinomial logistic regression for 5 classes in the original feature space. (b) After basis function expansion, using RBF kernels with a bandwidth of 1, and using all the data points as centers. Figure generated by `logregMultinomKernelDemo`.

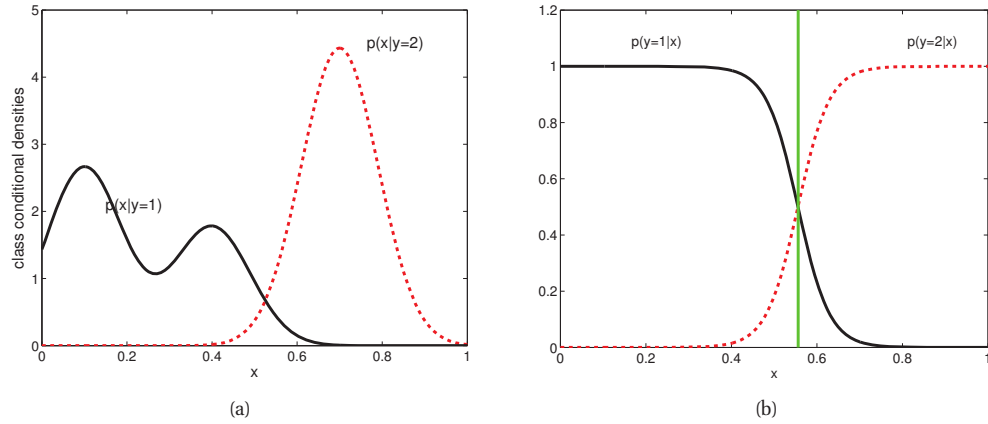


Figure 8.10 The class-conditional densities $p(x|y = c)$ (left) may be more complex than the class posteriors $p(y = c|x)$ (right). Based on Figure 1.27 of (Bishop 2006a). Figure generated by `generativeVsDiscrim`.

regression techniques we cover in this book.

8.6.2 Dealing with missing data

Sometimes some of the inputs (components of $\mathbf{x}$) are not observed; this could be due to a sensor failure, or a failure to complete an entry in a survey, etc. This is called the **missing data problem** (Little and Rubin 1987). The ability to handle missing data in a principled way is one of the biggest advantages of generative models.

To formalize our assumptions, we can associate a binary response variable $r_i \in \{0, 1\}$, that specifies whether each value $\mathbf{x}_i$ is observed or not. The joint model has the form $p(\mathbf{x}_i, r_i | \theta, \phi) = p(r_i | \mathbf{x}_i, \phi) p(\mathbf{x}_i | \theta)$, where ϕ are the parameters controlling whether the item

Model	Classif/regr	Gen/Discr	Param/Non	Section
Discriminant analysis	Classif	Gen	Param	Sec. 4.2.2, 4.2.4
Naive Bayes classifier	Classif	Gen	Param	Sec. 3.5, 3.5.1.2
Tree-augmented Naive Bayes classifier	Classif	Gen	Param	Sec. 10.2.1
Linear regression	Regr	Discrim	Param	Sec. 1.4.5, 7.3, 7.6,
Logistic regression	Classif	Discrim	Param	Sec. 1.4.6, 8.3.4, 8.4.3, 21.8.1.1
Sparse linear/ logistic regression	Both	Discrim	Param	Ch. 13
Mixture of experts	Both	Discrim	Param	Sec. 11.2.4
Multilayer perceptron (MLP)/ Neural network	Both	Discrim	Param	Ch. 16
Conditional random field (CRF)	Classif	Discrim	Param	Sec. 19.6
K nearest neighbor classifier	Classif	Gen	Non	Sec. 1.4.2, 14.7.3
(Infinite) Mixture Discriminant analysis	Classif	Gen	Non	Sec. 14.7.3
Classification and regression trees (CART)	Both	Discrim	Non	Sec. 16.2
Boosted model	Both	Discrim	Non	Sec. 16.4
Sparse kernelized lin/logreg (SKLR)	Both	Discrim	Non	Sec. 14.3.2
Relevance vector machine (RVM)	Both	Discrim	Non	Sec. 14.3.2
Support vector machine (SVM)	Both	Discrim	Non	Sec. 14.5
Gaussian processes (GP)	Both	Discrim	Non	Ch. 15
Smoothing splines	Regr	Discrim	Non	Section 15.4.6

Table 8.1 List of various models for classification and regression which we discuss in this book. Columns are as follows: Model name; is the model suitable for classification, regression, or both; is the model generative or discriminative; is the model parametric or non-parametric; list of sections in book which discuss the model. See also <http://pmtk3.googlecode.com/svn/trunk/docs/tutorial/html/tutorialSupervised.html> for the PMTK equivalents of these models. Any generative probabilistic model (e.g., HMMs, Boltzmann machines, Bayesian networks, etc.) can be turned into a classifier by using it as a class conditional density.

is observed or not. If we assume $p(r_i|\mathbf{x}_i, \phi) = p(r_i|\phi)$, we say the data is **missing completely at random** or **MCAR**. If we assume $p(r_i|\mathbf{x}_i, \phi) = p(r_i|\mathbf{x}_i^o, \phi)$, where $\mathbf{x}_i^o$ is the observed part of $\mathbf{x}_i$, we say the data is **missing at random** or **MAR**. If neither of these assumptions hold, we say the data is **not missing at random** or **NMAR**. In this case, we have to model the missing data mechanism, since the pattern of missingness is informative about the values of the missing data and the corresponding parameters. This is the case in most collaborative filtering problems, for example. See e.g., (Marlin 2008) for further discussion. We will henceforth assume the data is MAR.

When dealing with missing data, it is helpful to distinguish the cases when there is missingness only at test time (so the training data is **complete data**), from the harder case when there is missingness also at training time. We will discuss these two cases below. Note that the class label is always missing at test time, by definition; if the class label is also sometimes missing at training time, the problem is called semi-supervised learning.

8.6.2.1 Missing data at test time

In a generative classifier, we can handle features that are MAR by marginalizing them out. For example, if we are missing the value of x_1 , we can compute

$$p(y = c | \mathbf{x}_{2:D}, \boldsymbol{\theta}) \propto p(y = c | \boldsymbol{\theta}) p(\mathbf{x}_{2:D} | y = c, \boldsymbol{\theta}) \quad (8.94)$$

$$= p(y = c | \boldsymbol{\theta}) \sum_{x_1} p(x_1, \mathbf{x}_{2:D} | y = c, \boldsymbol{\theta}) \quad (8.95)$$

If we make the naive Bayes assumption, the marginalization can be performed as follows:

$$\sum_{x_1} p(x_1, x_{2:D} | y = c, \boldsymbol{\theta}) = \left[\sum_{x_1} p(x_1 | \boldsymbol{\theta}_{1c}) \right] \prod_{j=2}^D p(x_j | \boldsymbol{\theta}_{jc}) = \prod_{j=2}^D p(x_j | \boldsymbol{\theta}_{jc}) \quad (8.96)$$

where we exploited the fact that $\sum_{x_1} p(x_1 | y = c, \boldsymbol{\theta}) = 1$. Hence in a naive Bayes classifier, we can simply ignore missing features at test time. Similarly, in discriminant analysis, no matter what regularization method was used to estimate the parameters, we can always analytically marginalize out the missing variables (see Section 4.3):

$$p(\mathbf{x}_{2:D} | y = c, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}_{2:D} | \boldsymbol{\mu}_{c,2:D}, \boldsymbol{\Sigma}_{c,2:D,2:D}) \quad (8.97)$$

8.6.2.2 Missing data at training time

Missing data at training time is harder to deal with. In particular, computing the MLE or MAP estimate is no longer a simple optimization problem, for reasons discussed in Section 11.3.2. However, soon we will study a variety of more sophisticated algorithms (such as EM algorithm, in Section 11.4) for finding approximate ML or MAP estimates in such cases.

8.6.3 Fisher's linear discriminant analysis (FLDA) *

Discriminant analysis is a generative approach to classification, which requires fitting an MVN to the features. As we have discussed, this can be problematic in high dimensions. An alternative approach is to reduce the dimensionality of the features $\mathbf{x} \in \mathbb{R}^D$ and then fit an MVN to the resulting low-dimensional features $\mathbf{z} \in \mathbb{R}^L$. The simplest approach is to use a linear projection matrix, $\mathbf{z} = \mathbf{W}\mathbf{x}$, where $\mathbf{W}$ is a $L \times D$ matrix. One approach to finding $\mathbf{W}$ would be to use PCA (Section 12.2); the result would be very similar to RDA (Section 4.2.6), since SVD and PCA are essentially equivalent. However, PCA is an unsupervised technique that does not take class labels into account. Thus the resulting low dimensional features are not necessarily optimal for classification, as illustrated in Figure 8.11. An alternative approach is to find the matrix $\mathbf{W}$ such that the low-dimensional data can be classified as well as possible using a Gaussian class-conditional density model. The assumption of Gaussianity is reasonable since we are computing linear combinations of (potentially non-Gaussian) features. This approach is called **Fisher's linear discriminant analysis**, or **FLDA**.

FLDA is an interesting hybrid of discriminative and generative techniques. The drawback of this technique is that it is restricted to using $L \leq C - 1$ dimensions, regardless of D , for reasons that we will explain below. In the two-class case, this means we are seeking a single vector $\mathbf{w}$ onto which we can project the data. Below we derive the optimal $\mathbf{w}$ in the two-class case. We

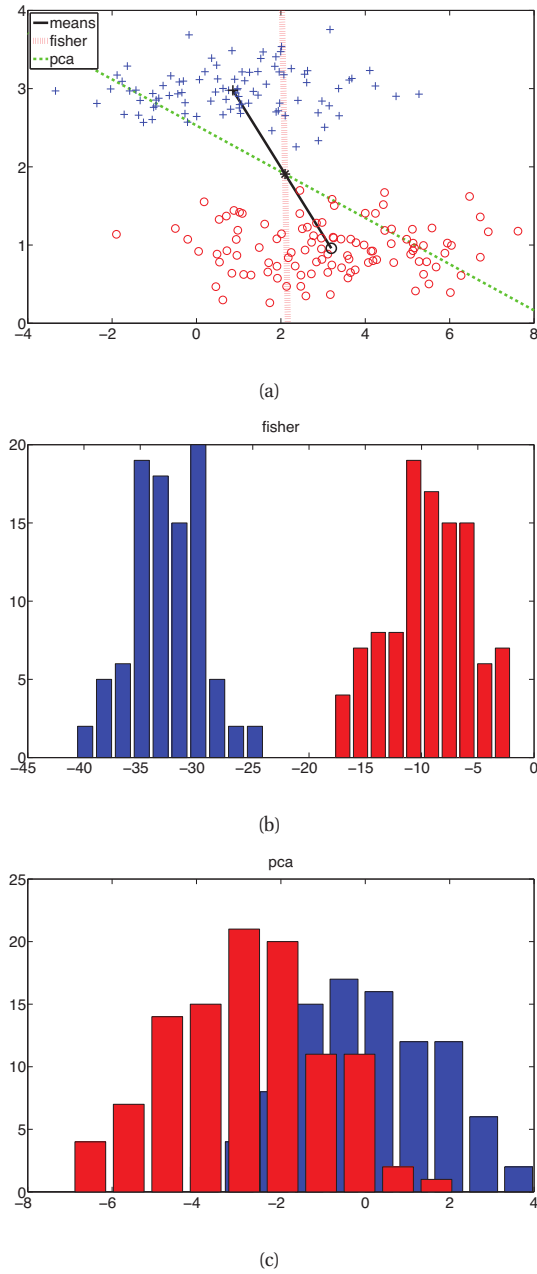


Figure 8.11 Example of Fisher's linear discriminant. (a) Two class data in 2D. Dashed green line = first principal basis vector. Dotted red line = Fisher's linear discriminant vector. Solid black line joins the class-conditional means. (b) Projection of points onto Fisher's vector shows good class separation. (c) Projection of points onto PCA vector shows poor class separation. Figure generated by `fisherLDA_demo`.

then generalize to the multi-class case, and finally we give a probabilistic interpretation of this technique.

8.6.3.1 Derivation of the optimal 1d projection

We now derive this optimal direction $\mathbf{w}$, for the two-class case, following the presentation of (Bishop 2006b, Sec 4.1.4). Define the class-conditional means as

$$\boldsymbol{\mu}_1 = \frac{1}{N_1} \sum_{i:y_i=1} \mathbf{x}_i, \quad \boldsymbol{\mu}_2 = \frac{1}{N_2} \sum_{i:y_i=2} \mathbf{x}_i \quad (8.98)$$

Let $m_k = \mathbf{w}^T \boldsymbol{\mu}_k$ be the projection of each mean onto the line $\mathbf{w}$. Also, let $z_i = \mathbf{w}^T \mathbf{x}_i$ be the projection of the data onto the line. The variance of the projected points is proportional to

$$s_k^2 = \sum_{i:y_i=k} (z_i - m_k)^2 \quad (8.99)$$

The goal is to find $\mathbf{w}$ such that we maximize the distance between the means, $m_2 - m_1$, while also ensuring the projected clusters are “tight”:

$$J(\mathbf{w}) = \frac{(m_2 - m_1)^2}{s_1^2 + s_2^2} \quad (8.100)$$

We can rewrite the right hand side of the above in terms of $\mathbf{w}$ as follows

$$J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}} \quad (8.101)$$

where $\mathbf{S}_B$ is the between-class scatter matrix given by

$$\mathbf{S}_B = (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^T \quad (8.102)$$

and $\mathbf{S}_W$ is the within-class scatter matrix, given by

$$\mathbf{S}_W = \sum_{i:y_i=1} (\mathbf{x}_i - \boldsymbol{\mu}_1)(\mathbf{x}_i - \boldsymbol{\mu}_1)^T + \sum_{i:y_i=2} (\mathbf{x}_i - \boldsymbol{\mu}_2)(\mathbf{x}_i - \boldsymbol{\mu}_2)^T \quad (8.103)$$

To see this, note that

$$\mathbf{w}^T \mathbf{S}_B \mathbf{w} = \mathbf{w}^T (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^T \mathbf{w} = (m_2 - m_1)(m_2 - m_1) \quad (8.104)$$

and

$$\mathbf{w}^T \mathbf{S}_W \mathbf{w} = \sum_{i:y_i=1} \mathbf{w}^T (\mathbf{x}_i - \boldsymbol{\mu}_1)(\mathbf{x}_i - \boldsymbol{\mu}_1)^T \mathbf{w} + \sum_{i:y_i=2} \mathbf{w}^T (\mathbf{x}_i - \boldsymbol{\mu}_2)(\mathbf{x}_i - \boldsymbol{\mu}_2)^T \mathbf{w} \quad (8.105)$$

$$= \sum_{i:y_i=1} (z_i - m_1)^2 + \sum_{i:y_i=2} (z_i - m_2)^2 \quad (8.106)$$

Equation 8.101 is a ratio of two scalars; we can take its derivative with respect to $\mathbf{w}$ and equate to zero. One can show (Exercise 12.6) that that $J(\mathbf{w})$ is maximized when

$$\mathbf{S}_B \mathbf{w} = \lambda \mathbf{S}_W \mathbf{w} \quad (8.107)$$

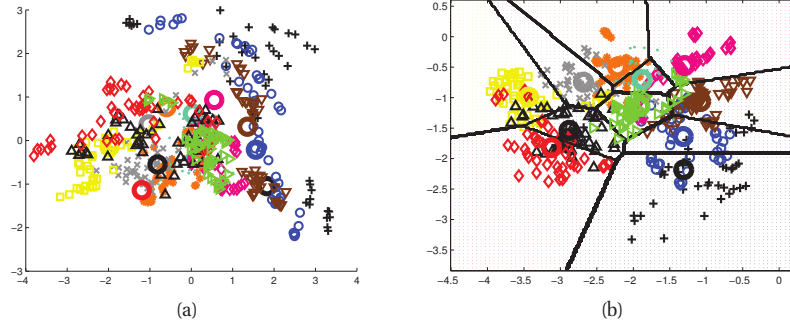


Figure 8.12 (a) PCA projection of vowel data to 2d. (b) FLDA projection of vowel data to 2d. We see there is better class separation in the FLDA case. Based on Figure 4.11 of (Hastie et al. 2009). Figure generated by `fisherDiscrimVowelDemo`, by Hannes Bretschneider.

where

$$\lambda = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}} \quad (8.108)$$

Equation 8.107 is called a **generalized eigenvalue** problem. If $\mathbf{S}_W$ is invertible, we can convert it to a regular eigenvalue problem:

$$\mathbf{S}_W^{-1} \mathbf{S}_B \mathbf{w} = \lambda \mathbf{w} \quad (8.109)$$

However, in the two class case, there is a simpler solution. In particular, since

$$\mathbf{S}_B \mathbf{w} = (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^T \mathbf{w} = (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(m_2 - m_1) \quad (8.110)$$

then, from Equation 8.109 we have

$$\lambda \mathbf{w} = \mathbf{S}_W^{-1} (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(m_2 - m_1) \quad (8.111)$$

$$\mathbf{w} \propto \mathbf{S}_W^{-1} (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) \quad (8.112)$$

Since we only care about the directionality, and not the scale factor, we can just set

$$\mathbf{w} = \mathbf{S}_W^{-1} (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) \quad (8.113)$$

This is the optimal solution in the two-class case. If $\mathbf{S}_W \propto \mathbf{I}$, meaning the pooled covariance matrix is isotropic, then $\mathbf{w}$ is proportional to the vector that joins the class means. This is an intuitively reasonable direction to project onto, as shown in Figure 8.11.

8.6.3.2 Extension to higher dimensions and multiple classes

We can extend the above idea to multiple classes, and to higher dimensional subspaces, by finding a projection *matrix* $\mathbf{W}$ which maps from D to L so as to maximize

$$J(\mathbf{W}) = \frac{|\mathbf{W} \boldsymbol{\Sigma}_B \mathbf{W}^T|}{|\mathbf{W} \boldsymbol{\Sigma}_W \mathbf{W}^T|} \quad (8.114)$$

where

$$\Sigma_B \triangleq \sum_c \frac{N_c}{N} (\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^T \quad (8.115)$$

$$\Sigma_W \triangleq \sum_c \frac{N_c}{N} \Sigma_c \quad (8.116)$$

$$\Sigma_c \triangleq \frac{1}{N_c} \sum_{i: y_i=c} (\mathbf{x}_i - \boldsymbol{\mu}_c)(\mathbf{x}_i - \boldsymbol{\mu}_c)^T \quad (8.117)$$

The solution can be shown to be

$$\mathbf{W} = \Sigma_W^{-\frac{1}{2}} \mathbf{U} \quad (8.118)$$

where $\mathbf{U}$ are the L leading eigenvectors of $\Sigma_W^{-\frac{1}{2}} \Sigma_B \Sigma_W^{-\frac{1}{2}}$, assuming Σ_W is non-singular. (If it is singular, we can first perform PCA on all the data.)

Figure 8.12 gives an example of this method applied to some $D = 10$ dimensional speech data, representing $C = 11$ different vowel sounds. We see that FLDA gives better class separation than PCA.

Note that FLDA is restricted to finding at most a $L \leq C - 1$ dimensional linear subspace, no matter how large D , because the rank of the between class covariance matrix Σ_B is $C - 1$. (The -1 term arises because of the $\boldsymbol{\mu}$ term, which is a linear function of the $\boldsymbol{\mu}_c$.) This is a rather severe restriction which limits the usefulness of FLDA.

8.6.3.3 Probabilistic interpretation of FLDA *

To find a valid probabilistic interpretation of FLDA, we follow the approach of (Kumar and Andreo 1998; Zhou et al. 2009). They proposed a model known as **heteroscedastic LDA** (HLDA), which works as follows. Let $\mathbf{W}$ be a $D \times D$ invertible matrix, and let $\mathbf{z}_i = \mathbf{W}\mathbf{x}_i$ be a transformed version of the data. We now fit full covariance Gaussians to the transformed data, one per class, but with the constraint that only the first L components will be class-specific; the remaining $H = D - L$ components will be shared across classes, and will thus not be discriminative. That is, we use

$$p(\mathbf{z}_i | \boldsymbol{\theta}, y_i = c) = \mathcal{N}(\mathbf{z}_i | \boldsymbol{\mu}_c, \Sigma_c) \quad (8.119)$$

$$\boldsymbol{\mu}_c \triangleq (\mathbf{m}_c; \mathbf{m}_0) \quad (8.120)$$

$$\Sigma_c \triangleq \begin{pmatrix} \mathbf{S}_c & \mathbf{0} \\ \mathbf{0} & \mathbf{S}_0 \end{pmatrix} \quad (8.121)$$

where $\mathbf{m}_0$ is the shared H dimensional mean and $\mathbf{S}_0$ is the shared $H \times H$ covariace. The pdf of the original (untransformed) data is given by

$$p(\mathbf{x}_i | y_i = c, \mathbf{W}, \boldsymbol{\theta}) = |\mathbf{W}| \mathcal{N}(\mathbf{W}\mathbf{x}_i | \boldsymbol{\mu}_c, \Sigma_c) \quad (8.122)$$

$$= |\mathbf{W}| \mathcal{N}(\mathbf{W}_L \mathbf{x}_i | \mathbf{m}_c, \mathbf{S}_c) \mathcal{N}(\mathbf{W}_H \mathbf{x}_i | \mathbf{m}_0, \mathbf{S}_0) \quad (8.123)$$

where $\mathbf{W} = \begin{pmatrix} \mathbf{W}_L \\ \mathbf{W}_H \end{pmatrix}$. For fixed $\mathbf{W}$, it is easy to derive the MLE for $\boldsymbol{\theta}$. One can then optimize $\mathbf{W}$ using gradient methods.

In the special case that the Σ_c are diagonal, there is a closed-form solution for $\mathbf{W}$ (Gales 1999). And in the special case the Σ_c are all equal, we recover classical LDA (Zhou et al. 2009).

In view of this this result, it should be clear that HLDA will outperform LDA if the class covariances are not equal within the discriminative subspace (i.e., if the assumption that Σ_c is independent of c is a poor assumption). This is easy to demonstrate on synthetic data, and is also the case on more challenging tasks such as speech recognition (Kumar and Andreo 1998). Furthermore, we can extend the model by allowing each class to use its own projection matrix; this is known as **multiple LDA** (Gales 2002).

Exercises

Exercise 8.1 Spam classification using logistic regression

Consider the email spam data set discussed on p300 of (Hastie et al. 2009). This consists of 4601 email messages, from which 57 features have been extracted. These are as follows:

- 48 features, in $[0, 100]$, giving the percentage of words in a given message which match a given word on the list. The list contains words such as “business”, “free”, “george”, etc. (The data was collected by George Forman, so his name occurs quite a lot.)
- 6 features, in $[0, 100]$, giving the percentage of characters in the email that match a given character on the list. The characters are ; ([! \$ #
- Feature 55: The average length of an uninterrupted sequence of capital letters (max is 40.3, mean is 4.9)
- Feature 56: The length of the longest uninterrupted sequence of capital letters (max is 45.0, mean is 52.6)
- Feature 57: The sum of the lengths of uninterrupted sequence of capital letters (max is 25.6, mean is 282.2)

Load the data from `spamData.mat`, which contains a training set (of size 3065) and a test set (of size 1536).

One can imagine performing several kinds of preprocessing to this data. Try each of the following separately:

- Standardize the columns so they all have mean 0 and unit variance.
- Transform the features using $\log(x_{ij} + 0.1)$.
- Binarize the features using $\mathbb{I}(x_{ij} > 0)$.

For each version of the data, fit a logistic regression model. Use cross validation to choose the strength of the ℓ_2 regularizer. Report the mean error rate on the training and test sets. You should get numbers similar to this:

method	train	test
std	0.082	0.079
log	0.052	0.059
binary	0.065	0.072

(The precise values will depend on what regularization value you choose.) Turn in your code and numerical results.

(See also Exercise 8.2.

Exercise 8.2 Spam classification using naive Bayes

We will re-examine the dataset from Exercise 8.1.

- Use `naiveBayesFit` and `naiveBayesPredict` on the binarized spam data. What is the training and test error? (You can try different settings of the pseudocount α if you like (this corresponds to the $\text{Beta}(\alpha, \alpha)$ prior each θ_{jc}), although the default of $\alpha = 1$ is probably fine.) Turn in your error rates.
- Modify the code so it can handle real-valued features. Use a Gaussian density for each feature; fit it with maximum likelihood. What are the training and test error rates on the standardized data and the log transformed data? Turn in your 4 error rates and code.

Exercise 8.3 Gradient and Hessian of log-likelihood for logistic regression

- Let $\sigma(a) = \frac{1}{1+e^{-a}}$ be the sigmoid function. Show that

$$\frac{d\sigma(a)}{da} = \sigma(a)(1 - \sigma(a)) \quad (8.124)$$

- Using the previous result and the chain rule of calculus, derive an expression for the gradient of the log likelihood (Equation 8.5).
- The Hessian can be written as $\mathbf{H} = \mathbf{X}^T \mathbf{S} \mathbf{X}$, where $\mathbf{S} \triangleq \text{diag}(\mu_1(1 - \mu_1), \dots, \mu_n(1 - \mu_n))$. Show that $\mathbf{H}$ is positive definite. (You may assume that $0 < \mu_i < 1$, so the elements of $\mathbf{S}$ will be strictly positive, and that $\mathbf{X}$ is full rank.)

Exercise 8.4 Gradient and Hessian of log-likelihood for multinomial logistic regression

- Let $\mu_{ik} = \mathcal{S}(\boldsymbol{\eta}_i)_k$. Prove that the Jacobian of the softmax is

$$\frac{\partial \mu_{ik}}{\partial \eta_{ij}} = \mu_{ik}(\delta_{kj} - \mu_{ij}) \quad (8.125)$$

where $\delta_{kj} = I(k = j)$.

- Hence show that

$$\nabla_{\mathbf{w}_c} \ell = \sum_i (y_{ic} - \mu_{ic}) \mathbf{x}_i \quad (8.126)$$

Hint: use the chain rule and the fact that $\sum_c y_{ic} = 1$.

- Show that the block submatrix of the Hessian for classes c and c' is given by

$$\mathbf{H}_{c,c'} = - \sum_i \mu_{ic}(\delta_{c,c'} - \mu_{i,c'}) \mathbf{x}_i \mathbf{x}_i^T \quad (8.127)$$

Exercise 8.5 Symmetric version of ℓ_2 regularized multinomial logistic regression

(Source: Ex 18.3 of (Hastie et al. 2009).)

Multiclass logistic regression has the form

$$p(y = c | \mathbf{x}, \mathbf{W}) = \frac{\exp(w_{c0} + \mathbf{w}_c^T \mathbf{x})}{\sum_{k=1}^C \exp(w_{k0} + \mathbf{w}_k^T \mathbf{x})} \quad (8.128)$$

where $\mathbf{W}$ is a $(D + 1) \times C$ weight matrix. We can arbitrarily define $\mathbf{w}_c = \mathbf{0}$ for one of the classes, say $c = C$, since $p(y = C | \mathbf{x}, \mathbf{W}) = 1 - \sum_{c=1}^{C-1} p(y = c | \mathbf{x}, \mathbf{W})$. In this case, the model has the form

$$p(y = c | \mathbf{x}, \mathbf{W}) = \frac{\exp(w_{c0} + \mathbf{w}_c^T \mathbf{x})}{1 + \sum_{k=1}^{C-1} \exp(w_{k0} + \mathbf{w}_k^T \mathbf{x})} \quad (8.129)$$

If we don't "clamp" one of the vectors to some constant value, the parameters will be unidentifiable. However, suppose we don't clamp $\mathbf{w}_c = \mathbf{0}$, so we are using Equation 8.128, but we add ℓ_2 regularization by optimizing

$$\sum_{i=1}^N \log p(y_i | \mathbf{x}_i, \mathbf{W}) - \lambda \sum_{c=1}^C \|\mathbf{w}_c\|_2^2 \quad (8.130)$$

Show that at the optimum we have $\sum_{c=1}^C \hat{w}_{cj} = 0$ for $j = 1 : D$. (For the unregularized $\hat{w}_{c0}$ terms, we still need to enforce that $w_{0C} = 0$ to ensure identifiability of the offset.)

Exercise 8.6 Elementary properties of ℓ_2 regularized logistic regression

(Source: Jaaakkola.). Consider minimizing

$$J(\mathbf{w}) = -\ell(\mathbf{w}, \mathcal{D}_{\text{train}}) + \lambda \|\mathbf{w}\|_2^2 \quad (8.131)$$

where

$$\ell(\mathbf{w}, \mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{i \in \mathcal{D}} \log \sigma(y_i \mathbf{x}_i^T \mathbf{w}) \quad (8.132)$$

is the average log-likelihood on data set $\mathcal{D}$, for $y_i \in \{-1, +1\}$. Answer the following true/ false questions.

- $J(\mathbf{w})$ has multiple locally optimal solutions: T/F?
- Let $\hat{\mathbf{w}} = \arg \min_{\mathbf{w}} J(\mathbf{w})$ be a global optimum. $\hat{\mathbf{w}}$ is sparse (has many zero entries): T/F?
- If the training data is linearly separable, then some weights w_j might become infinite if $\lambda = 0$: T/F?
- $\ell(\hat{\mathbf{w}}, \mathcal{D}_{\text{train}})$ always increases as we increase λ : T/F?
- $\ell(\hat{\mathbf{w}}, \mathcal{D}_{\text{test}})$ always increases as we increase λ : T/F?

Exercise 8.7 Regularizing separate terms in 2d logistic regression

(Source: Jaaakkola.)

- Consider the data in Figure 8.13, where we fit the model $p(y = 1 | \mathbf{x}, \mathbf{w}) = \sigma(w_0 + w_1 x_1 + w_2 x_2)$. Suppose we fit the model by maximum likelihood, i.e., we minimize

$$J(\mathbf{w}) = -\ell(\mathbf{w}, \mathcal{D}_{\text{train}}) \quad (8.133)$$

where $\ell(\mathbf{w}, \mathcal{D}_{\text{train}})$ is the log likelihood on the training set. Sketch a possible decision boundary corresponding to $\hat{\mathbf{w}}$. (Copy the figure first (a rough sketch is enough), and then superimpose your answer on your copy, since you will need multiple versions of this figure). Is your answer (decision boundary) unique? How many classification errors does your method make on the training set?

- Now suppose we regularize only the w_0 parameter, i.e., we minimize

$$J_0(\mathbf{w}) = -\ell(\mathbf{w}, \mathcal{D}_{\text{train}}) + \lambda w_0^2 \quad (8.134)$$

Suppose λ is a very large number, so we regularize w_0 all the way to 0, but all other parameters are unregularized. Sketch a possible decision boundary. How many classification errors does your method make on the training set? Hint: consider the behavior of simple linear regression, $w_0 + w_1 x_1 + w_2 x_2$ when $x_1 = x_2 = 0$.

- Now suppose we heavily regularize only the w_1 parameter, i.e., we minimize

$$J_1(\mathbf{w}) = -\ell(\mathbf{w}, \mathcal{D}_{\text{train}}) + \lambda w_1^2 \quad (8.135)$$

Sketch a possible decision boundary. How many classification errors does your method make on the training set?

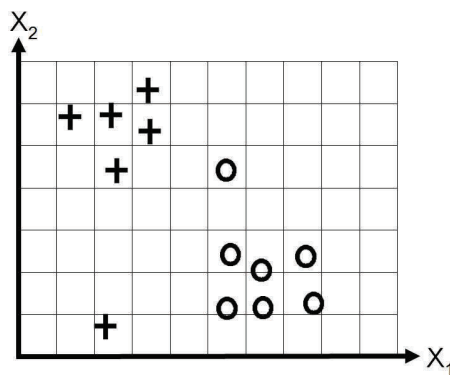


Figure 8.13 Data for logistic regression question.

- d. Now suppose we heavily regularize only the w_2 parameter. Sketch a possible decision boundary. How many classification errors does your method make on the training set?

9 *Generalized linear models and the exponential family*

9.1 Introduction

We have now encountered a wide variety of probability distributions: the Gaussian, the Bernoulli, the Student t , the uniform, the gamma, etc. It turns out that most of these are members of a broader class of distributions known as the **exponential family**.¹ In this chapter, we discuss various properties of this family. This allows us to derive theorems and algorithms with very broad applicability.

We will see how we can easily use any member of the exponential family as a class-conditional density in order to make a generative classifier. In addition, we will discuss how to build discriminative models, where the response variable has an exponential family distribution, whose mean is a linear function of the inputs; this is known as a generalized linear model, and generalizes the idea of logistic regression to other kinds of response variables.

9.2 The exponential family

Before defining the exponential family, we mention several reasons why it is important:

- It can be shown that, under certain regularity conditions, the exponential family is the only family of distributions with finite-sized sufficient statistics, meaning that we can compress the data into a fixed-sized summary without loss of information. This is particularly useful for online learning, as we will see later.
- The exponential family is the only family of distributions for which conjugate priors exist, which simplifies the computation of the posterior (see Section 9.2.5).
- The exponential family can be shown to be the family of distributions that makes the least set of assumptions subject to some user-chosen constraints (see Section 9.2.6).
- The exponential family is at the core of generalized linear models, as discussed in Section 9.3.
- The exponential family is at the core of variational inference, as discussed in Section 21.2.

1. The exceptions are the Student t , which does not have the right form, and the uniform distribution, which does not have fixed support independent of the parameter values.

9.2.1 Definition

A pdf or pmf $p(\mathbf{x}|\boldsymbol{\theta})$, for $\mathbf{x} = (x_1, \dots, x_m) \in \mathcal{X}^m$ and $\boldsymbol{\theta} \in \Theta \subseteq \mathbb{R}^d$, is said to be in the **exponential family** if it is of the form

$$p(\mathbf{x}|\boldsymbol{\theta}) = \frac{1}{Z(\boldsymbol{\theta})} h(\mathbf{x}) \exp[\boldsymbol{\theta}^T \boldsymbol{\phi}(\mathbf{x})] \quad (9.1)$$

$$= h(\mathbf{x}) \exp[\boldsymbol{\theta}^T \boldsymbol{\phi}(\mathbf{x}) - A(\boldsymbol{\theta})] \quad (9.2)$$

where

$$Z(\boldsymbol{\theta}) = \int_{\mathcal{X}^m} h(\mathbf{x}) \exp[\boldsymbol{\theta}^T \boldsymbol{\phi}(\mathbf{x})] d\mathbf{x} \quad (9.3)$$

$$A(\boldsymbol{\theta}) = \log Z(\boldsymbol{\theta}) \quad (9.4)$$

Here $\boldsymbol{\theta}$ are called the **natural parameters** or **canonical parameters**, $\boldsymbol{\phi}(\mathbf{x}) \in \mathbb{R}^d$ is called a vector of **sufficient statistics**, $Z(\boldsymbol{\theta})$ is called the **partition function**, $A(\boldsymbol{\theta})$ is called the **log partition function** or **cumulant function**, and $h(\mathbf{x})$ is the a scaling constant, often 1. If $\boldsymbol{\phi}(\mathbf{x}) = \mathbf{x}$, we say it is a **natural exponential family**.

Equation 9.2 can be generalized by writing

$$p(\mathbf{x}|\boldsymbol{\theta}) = h(\mathbf{x}) \exp[\boldsymbol{\eta}(\boldsymbol{\theta})^T \boldsymbol{\phi}(\mathbf{x}) - A(\boldsymbol{\eta}(\boldsymbol{\theta}))] \quad (9.5)$$

where $\boldsymbol{\eta}$ is a function that maps the parameters $\boldsymbol{\theta}$ to the canonical parameters $\boldsymbol{\eta} = \boldsymbol{\eta}(\boldsymbol{\theta})$. If $\dim(\boldsymbol{\theta}) < \dim(\boldsymbol{\eta}(\boldsymbol{\theta}))$, it is called a **curved exponential family**, which means we have more sufficient statistics than parameters. If $\boldsymbol{\eta}(\boldsymbol{\theta}) = \boldsymbol{\theta}$, the model is said to be in **canonical form**. We will assume models are in canonical form unless we state otherwise.

9.2.2 Examples

Let us consider some examples to make things clearer.

9.2.2.1 Bernoulli

The Bernoulli for $x \in \{0, 1\}$ can be written in exponential family form as follows:

$$\text{Ber}(x|\mu) = \mu^x (1 - \mu)^{1-x} = \exp[x \log(\mu) + (1 - x) \log(1 - \mu)] = \exp[\boldsymbol{\phi}(x)^T \boldsymbol{\theta}] \quad (9.6)$$

where $\boldsymbol{\phi}(x) = [\mathbb{I}(x = 0), \mathbb{I}(x = 1)]$ and $\boldsymbol{\theta} = [\log(\mu), \log(1 - \mu)]$. However, this representation is **over-complete** since there is a linear dependence between the features:

$$\mathbf{1}^T \boldsymbol{\phi}(x) = \mathbb{I}(x = 0) + \mathbb{I}(x = 1) = 1 \quad (9.7)$$

Consequently $\boldsymbol{\theta}$ is not uniquely identifiable. It is common to require that the representation be **minimal**, which means there is a unique $\boldsymbol{\theta}$ associated with the distribution. In this case, we can just define

$$\text{Ber}(x|\mu) = (1 - \mu) \exp \left[x \log \left(\frac{\mu}{1 - \mu} \right) \right] \quad (9.8)$$

Now we have $\phi(x) = x$, $\theta = \log\left(\frac{\mu}{1-\mu}\right)$, which is the **log-odds ratio**, and $Z = 1/(1-\mu)$. We can recover the mean parameter μ from the canonical parameter using

$$\mu = \text{sigm}(\theta) = \frac{1}{1 + e^{-\theta}} \quad (9.9)$$

9.2.2.2 Multinoulli

We can represent the multinoulli as a minimal exponential family as follows (where $x_k = \mathbb{I}(x = k)$):

$$\text{Cat}(x|\boldsymbol{\mu}) = \prod_{k=1}^K \mu_k^{x_k} = \exp \left[\sum_{k=1}^K x_k \log \mu_k \right] \quad (9.10)$$

$$= \exp \left[\sum_{k=1}^{K-1} x_k \log \mu_k + \left(1 - \sum_{k=1}^{K-1} x_k \right) \log \left(1 - \sum_{k=1}^{K-1} \mu_k \right) \right] \quad (9.11)$$

$$= \exp \left[\sum_{k=1}^{K-1} x_k \log \left(\frac{\mu_k}{1 - \sum_{j=1}^{K-1} \mu_j} \right) + \log \left(1 - \sum_{k=1}^{K-1} \mu_k \right) \right] \quad (9.12)$$

$$= \exp \left[\sum_{k=1}^{K-1} x_k \log \left(\frac{\mu_k}{\mu_K} \right) + \log \mu_K \right] \quad (9.13)$$

where $\mu_K = 1 - \sum_{k=1}^{K-1} \mu_k$. We can write this in exponential family form as follows:

$$\text{Cat}(x|\boldsymbol{\theta}) = \exp(\boldsymbol{\theta}^T \boldsymbol{\phi}(\mathbf{x}) - A(\boldsymbol{\theta})) \quad (9.14)$$

$$\boldsymbol{\theta} = [\log \frac{\mu_1}{\mu_K}, \dots, \log \frac{\mu_{K-1}}{\mu_K}] \quad (9.15)$$

$$\boldsymbol{\phi}(x) = [\mathbb{I}(x=1), \dots, \mathbb{I}(x=K-1)] \quad (9.16)$$

We can recover the mean parameters from the canonical parameters using

$$\mu_k = \frac{e^{\theta_k}}{1 + \sum_{j=1}^{K-1} e^{\theta_j}} \quad (9.17)$$

From this, we find

$$\mu_K = 1 - \frac{\sum_{j=1}^{K-1} e^{\theta_j}}{1 + \sum_{j=1}^{K-1} e^{\theta_j}} = \frac{1}{\sum_{j=1}^{K-1} e^{\theta_j} + 1} \quad (9.18)$$

and hence

$$A(\boldsymbol{\theta}) = \log \left(1 + \sum_{k=1}^{K-1} e^{\theta_k} \right) \quad (9.19)$$

If we define $\theta_K = 0$, we can write $\boldsymbol{\mu} = \mathcal{S}(\boldsymbol{\theta})$ and $A(\boldsymbol{\theta}) = \log \sum_{k=1}^K e^{\theta_k}$, where $\mathcal{S}$ is the softmax function in Equation 4.39.

9.2.2.3 Univariate Gaussian

The univariate Gaussian can be written in exponential family form as follows:

$$\mathcal{N}(x|\mu, \sigma^2) = \frac{1}{(2\pi\sigma^2)^{\frac{1}{2}}} \exp\left[-\frac{1}{2\sigma^2}(x - \mu)^2\right] \quad (9.20)$$

$$= \frac{1}{(2\pi\sigma^2)^{\frac{1}{2}}} \exp\left[-\frac{1}{2\sigma^2}x^2 + \frac{\mu}{\sigma^2}x - \frac{1}{2\sigma^2}\mu^2\right] \quad (9.21)$$

$$= \frac{1}{Z(\boldsymbol{\theta})} \exp(\boldsymbol{\theta}^T \boldsymbol{\phi}(x)) \quad (9.22)$$

where

$$\boldsymbol{\theta} = \begin{pmatrix} \mu/\sigma^2 \\ -\frac{1}{2\sigma^2} \end{pmatrix} \quad (9.23)$$

$$\boldsymbol{\phi}(x) = \begin{pmatrix} x \\ x^2 \end{pmatrix} \quad (9.24)$$

$$Z(\mu, \sigma^2) = \sqrt{2\pi}\sigma \exp\left[\frac{\mu^2}{2\sigma^2}\right] \quad (9.25)$$

$$A(\boldsymbol{\theta}) = \frac{-\theta_1^2}{4\theta_2} - \frac{1}{2} \log(-2\theta_2) - \frac{1}{2} \log(2\pi) \quad (9.26)$$

9.2.2.4 Non-examples

Not all distributions of interest belong to the exponential family. For example, the uniform distribution, $X \sim \text{Unif}(a, b)$, does not, since the support of the distribution depends on the parameters. Also, the Student T distribution (Section 11.4.5) does not belong, since it does not have the required form.

9.2.3 Log partition function

An important property of the exponential family is that derivatives of the log partition function can be used to generate **cumulants** of the sufficient statistics.² For this reason, $A(\boldsymbol{\theta})$ is sometimes called a **cumulant function**. We will prove this for a 1-parameter distribution; this can be generalized to a K -parameter distribution in a straightforward way. For the first

2. The first and second cumulants of a distribution are its mean $\mathbb{E}[X]$ and variance $\text{var}[X]$, whereas the first and second moments are its mean $\mathbb{E}[X]$ and $\mathbb{E}[X^2]$.

derivative we have

$$\frac{dA}{d\theta} = \frac{d}{d\theta} \left(\log \int \exp(\theta\phi(x)) h(x) dx \right) \quad (9.27)$$

$$= \frac{\frac{d}{d\theta} \int \exp(\theta\phi(x)) h(x) dx}{\int \exp(\theta\phi(x)) h(x) dx} \quad (9.28)$$

$$= \frac{\int \phi(x) \exp(\theta\phi(x)) h(x) dx}{\exp(A(\theta))} \quad (9.29)$$

$$= \int \phi(x) \exp(\theta\phi(x) - A(\theta)) h(x) dx \quad (9.30)$$

$$= \int \phi(x) p(x) dx = \mathbb{E}[\phi(x)] \quad (9.31)$$

For the second derivative we have

$$\frac{d^2 A}{d\theta^2} = \int \phi(x) \exp(\theta\phi(x) - A(\theta)) h(x) (\phi(x) - A'(\theta)) dx \quad (9.32)$$

$$= \int \phi(x) p(x) (\phi(x) - A'(\theta)) dx \quad (9.33)$$

$$= \int \phi^2(x) p(x) dx - A'(\theta) \int \phi(x) p(x) dx \quad (9.34)$$

$$= \mathbb{E}[\phi^2(X)] - \mathbb{E}[\phi(x)]^2 = \text{var}[\phi(x)] \quad (9.35)$$

where we used the fact that $A'(\theta) = \frac{dA}{d\theta} = \mathbb{E}[\phi(x)]$.

In the multivariate case, we have that

$$\frac{\partial^2 A}{\partial \theta_i \partial \theta_j} = \mathbb{E}[\phi_i(x) \phi_j(x)] - \mathbb{E}[\phi_i(x)] \mathbb{E}[\phi_j(x)] \quad (9.36)$$

and hence

$$\nabla^2 A(\boldsymbol{\theta}) = \text{cov}[\boldsymbol{\phi}(\mathbf{x})] \quad (9.37)$$

Since the covariance is positive definite, we see that $A(\boldsymbol{\theta})$ is a convex function (see Section 7.3.3).

9.2.3.1 Example: the Bernoulli distribution

For example, consider the Bernoulli distribution. We have $A(\theta) = \log(1 + e^\theta)$, so the mean is given by

$$\frac{dA}{d\theta} = \frac{e^\theta}{1 + e^\theta} = \frac{1}{1 + e^{-\theta}} = \text{sigm}(\theta) = \mu \quad (9.38)$$

The variance is given by

$$\frac{d^2 A}{d\theta^2} = \frac{d}{d\theta} (1 + e^{-\theta})^{-1} = (1 + e^{-\theta})^{-2} \cdot e^{-\theta} \quad (9.39)$$

$$= \frac{e^{-\theta}}{1 + e^{-\theta}} \frac{1}{1 + e^{-\theta}} = \frac{1}{e^\theta + 1} \frac{1}{1 + e^{-\theta}} = (1 - \mu)\mu \quad (9.40)$$

9.2.4 MLE for the exponential family

The likelihood of an exponential family model has the form

$$p(\mathcal{D}|\boldsymbol{\theta}) = \left[\prod_{i=1}^N h(\mathbf{x}_i) \right] g(\boldsymbol{\theta})^N \exp \left(\boldsymbol{\eta}(\boldsymbol{\theta})^T \left[\sum_{i=1}^N \boldsymbol{\phi}(\mathbf{x}_i) \right] \right) \quad (9.41)$$

We see that the sufficient statistics are N and

$$\boldsymbol{\phi}(\mathcal{D}) = \left[\sum_{i=1}^N \phi_1(\mathbf{x}_i), \dots, \sum_{i=1}^N \phi_K(\mathbf{x}_i) \right] \quad (9.42)$$

For example, for the Bernoulli model we have $\boldsymbol{\phi} = [\sum_i \mathbb{I}(x_i = 1)]$, and for the univariate Gaussian, we have $\boldsymbol{\phi} = [\sum_i x_i, \sum_i x_i^2]$. (We also need to know the sample size, N .)

The **Pitman-Koopman-Darmois theorem** states that, under certain regularity conditions, the exponential family is the only family of distributions with finite sufficient statistics. (Here, finite means of a size independent of the size of the data set.)

One of the conditions required in this theorem is that the support of the distribution not be dependent on the parameter. For a simple example of such a distribution, consider the uniform distribution

$$p(x|\theta) = U(x|\theta) = \frac{1}{\theta} \mathbb{I}(0 \leq x \leq \theta) \quad (9.43)$$

The likelihood is given by

$$p(\mathcal{D}|\theta) = \theta^{-N} \mathbb{I}(0 \leq \max\{x_i\} \leq \theta) \quad (9.44)$$

So the sufficient statistics are N and $s(\mathcal{D}) = \max_i x_i$. This is finite in size, but the uniform distribution is not in the exponential family because its support set, $\mathcal{X}$, depends on the parameters.

We now describe how to compute the MLE for a canonical exponential family model. Given N iid data points $\mathcal{D} = (x_1, \dots, x_N)$, the log-likelihood is

$$\log p(\mathcal{D}|\boldsymbol{\theta}) = \boldsymbol{\theta}^T \boldsymbol{\phi}(\mathcal{D}) - N A(\boldsymbol{\theta}) \quad (9.45)$$

Since $-A(\boldsymbol{\theta})$ is concave in $\boldsymbol{\theta}$, and $\boldsymbol{\theta}^T \boldsymbol{\phi}(\mathcal{D})$ is linear in $\boldsymbol{\theta}$, we see that the log likelihood is concave, and hence has a unique global maximum. To derive this maximum, we use the fact that the derivative of the log partition function yields the expected value of the sufficient statistic vector (Section 9.2.3):

$$\nabla_{\boldsymbol{\theta}} \log p(\mathcal{D}|\boldsymbol{\theta}) = \boldsymbol{\phi}(\mathcal{D}) - N \mathbb{E}[\boldsymbol{\phi}(\mathbf{X})] \quad (9.46)$$

Setting this gradient to zero, we see that at the MLE, the empirical average of the sufficient statistics must equal the model's theoretical expected sufficient statistics, i.e., $\hat{\boldsymbol{\theta}}$ must satisfy

$$\mathbb{E}[\boldsymbol{\phi}(\mathbf{X})] = \frac{1}{N} \sum_{i=1}^N \boldsymbol{\phi}(\mathbf{x}_i) \quad (9.47)$$

This is called **moment matching**. For example, in the Bernoulli distribution, we have $\phi(X) = \mathbb{I}(X = 1)$, so the MLE satisfies

$$\mathbb{E}[\phi(X)] = p(X = 1) = \hat{\mu} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(x_i = 1) \quad (9.48)$$

9.2.5 Bayes for the exponential family *

We have seen that exact Bayesian analysis is considerably simplified if the prior is conjugate to the likelihood. Informally this means that the prior $p(\boldsymbol{\theta}|\boldsymbol{\tau})$ has the same form as the likelihood $p(\mathcal{D}|\boldsymbol{\theta})$. For this to make sense, we require that the likelihood have finite sufficient statistics, so that we can write $p(\mathcal{D}|\boldsymbol{\theta}) = p(\mathbf{s}(\mathcal{D})|\boldsymbol{\theta})$. This suggests that the only family of distributions for which conjugate priors exist is the exponential family. We will derive the form of the prior and posterior below.

9.2.5.1 Likelihood

The likelihood of the exponential family is given by

$$p(\mathcal{D}|\boldsymbol{\theta}) \propto g(\boldsymbol{\theta})^N \exp(\boldsymbol{\eta}(\boldsymbol{\theta})^T \mathbf{s}_N) \quad (9.49)$$

where $\mathbf{s}_N = \sum_{i=1}^N \mathbf{s}(\mathbf{x}_i)$. In terms of the canonical parameters this becomes

$$p(\mathcal{D}|\boldsymbol{\eta}) \propto \exp(N\boldsymbol{\eta}^T \bar{\mathbf{s}} - NA(\boldsymbol{\eta})) \quad (9.50)$$

where $\bar{\mathbf{s}} = \frac{1}{N} \mathbf{s}_N$.

9.2.5.2 Prior

The natural conjugate prior has the form

$$p(\boldsymbol{\theta}|\nu_0, \boldsymbol{\tau}_0) \propto g(\boldsymbol{\theta})^{\nu_0} \exp(\boldsymbol{\eta}(\boldsymbol{\theta})^T \boldsymbol{\tau}_0) \quad (9.51)$$

Let us write $\boldsymbol{\tau}_0 = \nu_0 \bar{\boldsymbol{\tau}}_0$, to separate out the size of the prior pseudo-data, ν_0 , from the mean of the sufficient statistics on this pseudo-data, $\bar{\boldsymbol{\tau}}_0$. In canonical form, the prior becomes

$$p(\boldsymbol{\eta}|\nu_0, \bar{\boldsymbol{\tau}}_0) \propto \exp(\nu_0 \boldsymbol{\eta}^T \bar{\boldsymbol{\tau}}_0 - \nu_0 A(\boldsymbol{\eta})) \quad (9.52)$$

9.2.5.3 Posterior

The posterior is given by

$$p(\boldsymbol{\theta}|\mathcal{D}) = p(\boldsymbol{\theta}|\nu_N, \boldsymbol{\tau}_N) = p(\boldsymbol{\theta}|\nu_0 + N, \boldsymbol{\tau}_0 + \mathbf{s}_N) \quad (9.53)$$

So we see that we just update the hyper-parameters by adding. In canonical form, this becomes

$$p(\boldsymbol{\eta}|\mathcal{D}) \propto \exp(\boldsymbol{\eta}^T (\nu_0 \bar{\boldsymbol{\tau}}_0 + N\bar{\mathbf{s}}) - (\nu_0 + N)A(\boldsymbol{\eta})) \quad (9.54)$$

$$= p(\boldsymbol{\eta}|\nu_0 + N, \frac{\nu_0 \bar{\boldsymbol{\tau}}_0 + N\bar{\mathbf{s}}}{\nu_0 + N}) \quad (9.55)$$

So we see that the posterior hyper-parameters are a convex combination of the prior mean hyper-parameters and the average of the sufficient statistics.

9.2.5.4 Posterior predictive density

Let us derive a generic expression for the predictive density for future observables $\mathcal{D}' = (\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_{N'})$ given past data $\mathcal{D} = (\mathbf{x}_1, \dots, \mathbf{x}_N)$ as follows. For notational brevity, we will combine the sufficient statistics with the size of the data, as follows: $\tilde{\boldsymbol{\tau}}_0 = (\nu_0, \boldsymbol{\tau}_0)$, $\tilde{\mathbf{s}}(\mathcal{D}) = (N, \mathbf{s}(\mathcal{D}))$, and $\tilde{\mathbf{s}}(\mathcal{D}') = (N', \mathbf{s}(\mathcal{D}'))$. So the prior becomes

$$p(\boldsymbol{\theta}|\tilde{\boldsymbol{\tau}}_0) = \frac{1}{Z(\tilde{\boldsymbol{\tau}}_0)} g(\boldsymbol{\theta})^{\nu_0} \exp(\boldsymbol{\eta}(\boldsymbol{\theta})^T \boldsymbol{\tau}_0) \quad (9.56)$$

The likelihood and posterior have a similar form. Hence

$$p(\mathcal{D}'|\mathcal{D}) = \int p(\mathcal{D}'|\boldsymbol{\theta}) p(\boldsymbol{\theta}|\mathcal{D}) d\boldsymbol{\theta} \quad (9.57)$$

$$= \left[\prod_{i=1}^{N'} h(\tilde{\mathbf{x}}_i) \right] Z(\tilde{\boldsymbol{\tau}}_0 + \tilde{\mathbf{s}}(\mathcal{D}))^{-1} \int g(\boldsymbol{\theta})^{\nu_0 + N + N'} d\boldsymbol{\theta} \quad (9.58)$$

$$\times \exp \left(\sum_k \eta_k(\boldsymbol{\theta}) \left(\tau_k + \sum_{i=1}^N s_k(\mathbf{x}_i) + \sum_{i=1}^{N'} s_k(\tilde{\mathbf{x}}_i) \right) \right) d\boldsymbol{\theta} \quad (9.59)$$

$$= \left[\prod_{i=1}^{N'} h(\tilde{\mathbf{x}}_i) \right] \frac{Z(\tilde{\boldsymbol{\tau}}_0 + \tilde{\mathbf{s}}(\mathcal{D}) + \tilde{\mathbf{s}}(\mathcal{D}'))}{Z(\tilde{\boldsymbol{\tau}}_0 + \tilde{\mathbf{s}}(\mathcal{D}))} \quad (9.60)$$

If $N = 0$, this becomes the marginal likelihood of $\mathcal{D}'$, which reduces to the familiar form of normalizer of the posterior divided by the normalizer of the prior, multiplied by a constant.

9.2.5.5 Example: Bernoulli distribution

As a simple example, let us revisit the Beta-Bernoulli model in our new notation.

The likelihood is given by

$$p(\mathcal{D}|\theta) = (1 - \theta)^N \exp \left(\log\left(\frac{\theta}{1 - \theta}\right) \sum_i x_i \right) \quad (9.61)$$

Hence the conjugate prior is given by

$$p(\theta|\nu_0, \tau_0) \propto (1 - \theta)^{\nu_0} \exp \left(\log\left(\frac{\theta}{1 - \theta}\right) \tau_0 \right) \quad (9.62)$$

$$= \theta^{\tau_0} (1 - \theta)^{\nu_0 - \tau_0} \quad (9.63)$$

If we define $\alpha = \tau_0 + 1$ and $\beta = \nu_0 - \tau_0 + 1$, we see that this is a beta distribution.

We can derive the posterior as follows, where $s = \sum_i \mathbb{I}(x_i = 1)$ is the sufficient statistic:

$$p(\theta|\mathcal{D}) \propto \theta^{\tau_0 + s} (1 - \theta)^{\nu_0 - \tau_0 + n - s} \quad (9.64)$$

$$= \theta^{\tau_n} (1 - \theta)^{\nu_n - \tau_n} \quad (9.65)$$

We can derive the posterior predictive distribution as follows. Assume $p(\theta) = \text{Beta}(\theta|\alpha, \beta)$, and let $s = s(\mathcal{D})$ be the number of heads in the past data. We can predict the probability of a

given sequence of future heads, $\mathcal{D}' = (\tilde{x}_1, \dots, \tilde{x}_m)$, with sufficient statistic $s' = \sum_{i=1}^m \mathbb{I}(\tilde{x}_i = 1)$, as follows:

$$p(\mathcal{D}'|\mathcal{D}) = \int_0^1 p(\mathcal{D}'|\theta) \text{Beta}(\theta|\alpha_n, \beta_n) d\theta \quad (9.66)$$

$$= \frac{\Gamma(\alpha_n + \beta_n)}{\Gamma(\alpha_n)\Gamma(\beta_n)} \int_0^1 \theta^{\alpha_n + t' - 1} (1 - \theta)^{\beta_n + m - t' - 1} d\theta \quad (9.67)$$

$$= \frac{\Gamma(\alpha_n + \beta_n)}{\Gamma(\alpha_n)\Gamma(\beta_n)} \frac{\Gamma(\alpha_{n+m})\Gamma(\beta_{n+m})}{\Gamma(\alpha_{n+m} + \beta_{n+m})} \quad (9.68)$$

where

$$\alpha_{n+m} = \alpha_n + s' = \alpha + s + s' \quad (9.69)$$

$$\beta_{n+m} = \beta_n + (m - s') = \beta + (n - s) + (m - s') \quad (9.70)$$

9.2.6 Maximum entropy derivation of the exponential family *

Although the exponential family is convenient, is there any deeper justification for its use? It turns out that there is: it is the distribution that makes the least number of assumptions about the data, subject to a specific set of user-specified constraints, as we explain below. In particular, suppose all we know is the expected values of certain features or functions:

$$\sum_{\mathbf{x}} f_k(\mathbf{x}) p(\mathbf{x}) = F_k \quad (9.71)$$

where F_k are known constants, and $f_k(\mathbf{x})$ is an arbitrary function. The principle of **maximum entropy** or **maxent** says we should pick the distribution with maximum entropy (closest to uniform), subject to the constraints that the moments of the distribution match the empirical moments of the specified functions.

To maximize entropy subject to the constraints in Equation 9.71, and the constraints that $p(\mathbf{x}) \geq 0$ and $\sum_{\mathbf{x}} p(\mathbf{x}) = 1$, we need to use Lagrange multipliers. The Lagrangian is given by

$$J(p, \lambda) = - \sum_{\mathbf{x}} p(\mathbf{x}) \log p(\mathbf{x}) + \lambda_0 (1 - \sum_{\mathbf{x}} p(\mathbf{x})) + \sum_k \lambda_k (F_k - \sum_{\mathbf{x}} p(\mathbf{x}) f_k(\mathbf{x})) \quad (9.72)$$

We can use the calculus of variations to take derivatives wrt the function p , but we will adopt a simpler approach and treat p as a fixed length vector (since we are assuming $\mathbf{x}$ is discrete). Then we have

$$\frac{\partial J}{\partial p(\mathbf{x})} = -1 - \log p(\mathbf{x}) - \lambda_0 - \sum_k \lambda_k f_k(\mathbf{x}) \quad (9.73)$$

Setting $\frac{\partial J}{\partial p(\mathbf{x})} = 0$ yields

$$p(\mathbf{x}) = \frac{1}{Z} \exp(- \sum_k \lambda_k f_k(\mathbf{x})) \quad (9.74)$$

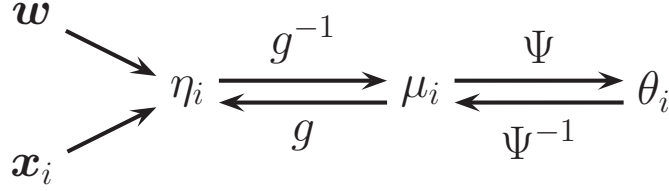


Figure 9.1 A visualization of the various features of a GLM. Based on Figure 8.3 of (Jordan 2007).

where $Z = e^{1+\lambda_0}$. Using the sum to one constraint, we have

$$1 = \sum_{\mathbf{x}} p(\mathbf{x}) = \frac{1}{Z} \sum_{\mathbf{x}} \exp\left(-\sum_k \lambda_k f_k(\mathbf{x})\right) \quad (9.75)$$

Hence the normalization constant is given by

$$Z = \sum_{\mathbf{x}} \exp\left(-\sum_k \lambda_k f_k(\mathbf{x})\right) \quad (9.76)$$

Thus the maxent distribution $p(\mathbf{x})$ has the form of the exponential family (Section 9.2), also known as the **Gibbs distribution**.

9.3 Generalized linear models (GLMs)

Linear and logistic regression are examples of **generalized linear models**, or **GLMs** (McCullagh and Nelder 1989). These are models in which the output density is in the exponential family (Section 9.2), and in which the mean parameters are a linear combination of the inputs, passed through a possibly nonlinear function, such as the logistic function. We describe GLMs in more detail below. We focus on scalar outputs for notational simplicity. (This excludes multinomial logistic regression, but this is just to simplify the presentation.)

9.3.1 Basics

To understand GLMs, let us first consider the case of an unconditional distribution for a scalar response variable:

$$p(y_i|\theta, \sigma^2) = \exp\left[\frac{y_i\theta - A(\theta)}{\sigma^2} + c(y_i, \sigma^2)\right] \quad (9.77)$$

where σ^2 is the **dispersion parameter** (often set to 1), θ is the natural parameter, A is the partition function, and c is a normalization constant. For example, in the case of logistic regression, θ is the log-odds ratio, $\theta = \log\left(\frac{\mu}{1-\mu}\right)$, where $\mu = \mathbb{E}[y] = p(y=1)$ is the mean parameter (see Section 9.2.2.1). To convert from the mean parameter to the natural parameter

Distrib.	Link $g(\mu)$	$\theta = \psi(\mu)$	$\mu = \psi^{-1}(\theta) = \mathbb{E}[y]$
$\mathcal{N}(\mu, \sigma^2)$	identity	$\theta = \mu$	$\mu = \theta$
$\text{Bin}(N, \mu)$	logit	$\theta = \log(\frac{\mu}{1-\mu})$	$\mu = \text{sigm}(\theta)$
$\text{Poi}(\mu)$	log	$\theta = \log(\mu)$	$\mu = e^\theta$

Table 9.1 Canonical link functions ψ and their inverses for some common GLMs.

we can use a function ψ , so $\theta = \Psi(\mu)$. This function is uniquely determined by the form of the exponential family distribution. In fact, this is an invertible mapping, so we have $\mu = \Psi^{-1}(\theta)$. Furthermore, we know from Section 9.2.3 that the mean is given by the derivative of the partition function, so we have $\mu = \Psi^{-1}(\theta) = A'(\theta)$.

Now let us add inputs/ covariates. We first define a linear function of the inputs:

$$\eta_i = \mathbf{w}^T \mathbf{x}_i \quad (9.78)$$

We now make the mean of the distribution be some invertible monotonic function of this linear combination. By convention, this function, known as the **mean function**, is denoted by g^{-1} , so

$$\mu_i = g^{-1}(\eta_i) = g^{-1}(\mathbf{w}^T \mathbf{x}_i) \quad (9.79)$$

See Figure 9.1 for a summary of the basic model.

The inverse of the mean function, namely $g()$, is called the **link function**. We are free to choose almost any function we like for g , so long as it is invertible, and so long as g^{-1} has the appropriate range. For example, in logistic regression, we set $\mu_i = g^{-1}(\eta_i) = \text{sigm}(\eta_i)$.

One particularly simple form of link function is to use $g = \psi$; this is called the **canonical link function**. In this case, $\theta_i = \eta_i = \mathbf{w}^T \mathbf{x}_i$, so the model becomes

$$p(y_i | \mathbf{x}_i, \mathbf{w}, \sigma^2) = \exp \left[\frac{y_i \mathbf{w}^T \mathbf{x}_i - A(\mathbf{w}^T \mathbf{x}_i)}{\sigma^2} + c(y_i, \sigma^2) \right] \quad (9.80)$$

In Table 9.1, we list some distributions and their canonical link functions. We see that for the Bernoulli/ binomial distribution, the canonical link is the logit function, $g(\mu) = \log(\mu/(1-\mu))$, whose inverse is the logistic function, $\mu = \text{sigm}(\eta)$.

Based on the results in Section 9.2.3, we can show that the mean and variance of the response variable are as follows:

$$\mathbb{E}[y | \mathbf{x}_i, \mathbf{w}, \sigma^2] = \mu_i = A'(\theta_i) \quad (9.81)$$

$$\text{var}[y | \mathbf{x}_i, \mathbf{w}, \sigma^2] = \sigma_i^2 = A''(\theta_i) \sigma^2 \quad (9.82)$$

To make the notation clearer, let us consider some simple examples.

- For linear regression, we have

$$\log p(y_i | \mathbf{x}_i, \mathbf{w}, \sigma^2) = \frac{y_i \mu_i - \frac{\mu_i^2}{2}}{\sigma^2} - \frac{1}{2} \left(\frac{y_i^2}{\sigma^2} + \log(2\pi\sigma^2) \right) \quad (9.83)$$

where $y_i \in \mathbb{R}$, and $\theta_i = \mu_i = \mathbf{w}^T \mathbf{x}_i$. Here $A(\theta) = \theta^2/2$, so $\mathbb{E}[y_i] = \mu_i$ and $\text{var}[y_i] = \sigma^2$.

- For binomial regression, we have

$$\log p(y_i | \mathbf{x}_i, \mathbf{w}) = y_i \log\left(\frac{\pi_i}{1 - \pi_i}\right) + N_i \log(1 - \pi_i) + \log\left(\binom{N_i}{y_i}\right) \quad (9.84)$$

where $y_i \in \{0, 1, \dots, N_i\}$, $\pi_i = \text{sigm}(\mathbf{w}^T \mathbf{x}_i)$, $\theta_i = \log(\pi_i / (1 - \pi_i)) = \mathbf{w}^T \mathbf{x}_i$, and $\sigma^2 = 1$. Here $A(\theta) = N_i \log(1 + e^\theta)$, so $\mathbb{E}[y_i] = N_i \pi_i = \mu_i$, $\text{var}[y_i] = N_i \pi_i (1 - \pi_i)$.

- For **poisson regression**, we have

$$\log p(y_i | \mathbf{x}_i, \mathbf{w}) = y_i \log \mu_i - \mu_i - \log(y_i!) \quad (9.85)$$

where $y_i \in \{0, 1, 2, \dots\}$, $\mu_i = \exp(\mathbf{w}^T \mathbf{x}_i)$, $\theta_i = \log(\mu_i) = \mathbf{w}^T \mathbf{x}_i$, and $\sigma^2 = 1$. Here $A(\theta) = e^\theta$, so $\mathbb{E}[y_i] = \text{var}[y_i] = \mu_i$. Poisson regression is widely used in bio-statistical applications, where y_i might represent the number of diseases of a given person or place, or the number of reads at a genomic location in a high-throughput sequencing context (see e.g., (Kuan et al. 2009)).

9.3.2 ML and MAP estimation

One of the appealing properties of GLMs is that they can be fit using exactly the same methods that we used to fit logistic regression. In particular, the log-likelihood has the following form:

$$\ell(\mathbf{w}) = \log p(\mathcal{D} | \mathbf{w}) = \frac{1}{\sigma^2} \sum_{i=1}^N \ell_i \quad (9.86)$$

$$\ell_i \triangleq \theta_i y_i - A(\theta_i) \quad (9.87)$$

We can compute the gradient vector using the chain rule as follows:

$$\frac{d\ell_i}{dw_j} = \frac{d\ell_i}{d\theta_i} \frac{d\theta_i}{d\mu_i} \frac{d\mu_i}{d\eta_i} \frac{d\eta_i}{dw_j} \quad (9.88)$$

$$= (y_i - A'(\theta_i)) \frac{d\theta_i}{d\mu_i} \frac{d\mu_i}{d\eta_i} x_{ij} \quad (9.89)$$

$$= (y_i - \mu_i) \frac{d\theta_i}{d\mu_i} \frac{d\mu_i}{d\eta_i} x_{ij} \quad (9.90)$$

If we use a canonical link, $\theta_i = \eta_i$, this simplifies to

$$\nabla_{\mathbf{w}} \ell(\mathbf{w}) = \frac{1}{\sigma^2} \left[\sum_{i=1}^N (y_i - \mu_i) \mathbf{x}_i \right] \quad (9.91)$$

which is a sum of the input vectors, weighted by the errors. This can be used inside a (stochastic) gradient descent procedure, discussed in Section 8.5.2. However, for improved efficiency, we should use a second-order method. If we use a canonical link, the Hessian is given by

$$\mathbf{H} = -\frac{1}{\sigma^2} \sum_{i=1}^N \frac{d\mu_i}{d\theta_i} \mathbf{x}_i \mathbf{x}_i^T = -\frac{1}{\sigma^2} \mathbf{X}^T \mathbf{S} \mathbf{X} \quad (9.92)$$

Name	Formula
Logistic	$g^{-1}(\eta) = \text{sigm}(\eta) = \frac{e^\eta}{1+e^\eta}$
Probit	$g^{-1}(\eta) = \Phi(\eta)$
Log-log	$g^{-1}(\eta) = \exp(-\exp(-\eta))$
Complementary log-log	$g^{-1}(\eta) = 1 - \exp(-\exp(\eta))$

Table 9.2 Summary of some possible mean functions for binary regression.

where $\mathbf{S} = \text{diag}(\frac{d\mu_1}{d\theta_1}, \dots, \frac{d\mu_N}{d\theta_N})$ is a diagonal weighting matrix. This can be used inside the IRLS algorithm (Section 8.3.4). Specifically, we have the following Newton update:

$$\mathbf{w}_{t+1} = (\mathbf{X}^T \mathbf{S}_t \mathbf{X})^{-1} \mathbf{X}^T \mathbf{S}_t \mathbf{z}_t \quad (9.93)$$

$$\mathbf{z}_t = \boldsymbol{\theta}_t + \mathbf{S}_t^{-1}(\mathbf{y} - \boldsymbol{\mu}_t) \quad (9.94)$$

where $\boldsymbol{\theta}_t = \mathbf{X}\mathbf{w}_t$ and $\boldsymbol{\mu}_t = g^{-1}(\boldsymbol{\eta}_t)$.

If we extend the derivation to handle non-canonical links, we find that the Hessian has another term. However, it turns out that the expected Hessian is the same as in Equation 9.92; using the expected Hessian (known as the Fisher information matrix) instead of the actual Hessian is known as the **Fisher scoring method**.

It is straightforward to modify the above procedure to perform MAP estimation with a Gaussian prior: we just modify the objective, gradient and Hessian, just as we added ℓ_2 regularization to logistic regression in Section 8.3.6.

9.3.3 Bayesian inference

Bayesian inference for GLMs is usually conducted using MCMC (Chapter 24). Possible methods include Metropolis Hastings with an IRLS-based proposal (Gamerman 1997), Gibbs sampling using adaptive rejection sampling (ARS) for each full-conditional (Dellaportas and Smith 1993), etc. See e.g., (Dey et al. 2000) for further information. It is also possible to use the Gaussian approximation (Section 8.4.1) or variational inference (Section 21.8.1.1).

9.4 Probit regression

In (binary) logistic regression, we use a model of the form $p(y = 1|\mathbf{x}_i, \mathbf{w}) = \text{sigm}(\mathbf{w}^T \mathbf{x}_i)$. In general, we can write $p(y = 1|\mathbf{x}_i, \mathbf{w}) = g^{-1}(\mathbf{w}^T \mathbf{x}_i)$, for any function g^{-1} that maps $[-\infty, \infty]$ to $[0, 1]$. Several possible mean functions are listed in Table 9.2.

In this section, we focus on the case where $g^{-1}(\eta) = \Phi(\eta)$, where $\Phi(\eta)$ is the cdf of the standard normal. This is known as **probit regression**. The probit function is very similar to the logistic function, as shown in Figure 8.7(b). However, this model has some advantages over logistic regression, as we will see.

9.4.1 ML/MAP estimation using gradient-based optimization

We can find the MLE for probit regression using standard gradient methods. Let $\mu_i = \mathbf{w}^T \mathbf{x}_i$, and let $\tilde{y}_i \in \{-1, +1\}$. Then the gradient of the log-likelihood for a specific case is given by

$$\mathbf{g}_i \triangleq \frac{d}{d\mathbf{w}} \log p(\tilde{y}_i | \mathbf{w}^T \mathbf{x}_i) = \frac{d\mu_i}{d\mathbf{w}} \frac{d}{d\mu_i} \log p(\tilde{y}_i | \mathbf{w}^T \mathbf{x}_i) = \mathbf{x}_i \frac{\tilde{y}_i \phi(\mu_i)}{\Phi(\tilde{y}_i \mu_i)} \quad (9.95)$$

where ϕ is the standard normal pdf, and Φ is its cdf. Similarly, the Hessian for a single case is given by

$$\mathbf{H}_i = \frac{d}{d\mathbf{w}^2} \log p(\tilde{y}_i | \mathbf{w}^T \mathbf{x}_i) = -\mathbf{x}_i \left(\frac{\phi(\mu_i)^2}{\Phi(\tilde{y}_i \mu_i)^2} + \frac{\tilde{y}_i \mu_i \phi(\mu_i)}{\Phi(\tilde{y}_i \mu_i)} \right) \mathbf{x}_i^T \quad (9.96)$$

We can modify these expressions to compute the MAP estimate in a straightforward manner. In particular, if we use the prior $p(\mathbf{w}) = \mathcal{N}(\mathbf{0}, \mathbf{V}_0)$, the gradient and Hessian of the penalized log likelihood have the form $\sum_i \mathbf{g}_i + 2\mathbf{V}_0^{-1}\mathbf{w}$ and $\sum_i \mathbf{H}_i + 2\mathbf{V}_0^{-1}$. These expressions can be passed to any gradient-based optimizer. See `probitRegDemo` for a demo.

9.4.2 Latent variable interpretation

We can interpret the probit (and logistic) model as follows. First, let us associate each item $\mathbf{x}_i$ with two latent utilities, u_{0i} and u_{1i} , corresponding to the possible choices of $y_i = 0$ and $y_i = 1$. We then assume that the observed choice is whichever action has larger utility. More precisely, the model is as follows:

$$u_{0i} \triangleq \mathbf{w}_0^T \mathbf{x}_i + \delta_{0i} \quad (9.97)$$

$$u_{1i} \triangleq \mathbf{w}_1^T \mathbf{x}_i + \delta_{1i} \quad (9.98)$$

$$y_i = \mathbb{I}(u_{1i} > u_{0i}) \quad (9.99)$$

where δ 's are error terms, representing all the other factors that might be relevant in decision making that we have chosen not to (or are unable to) model. This is called a **random utility model** or **RUM** (McFadden 1974; Train 2009).

Since it is only the difference in utilities that matters, let us define $z_i = u_{1i} - u_{0i} + \epsilon_i$, where $\epsilon_i = \delta_{1i} - \delta_{0i}$. If the δ 's have a Gaussian distribution, then so does ϵ_i . Thus we can write

$$z_i \triangleq \mathbf{w}^T \mathbf{x}_i + \epsilon_i \quad (9.100)$$

$$\epsilon_i \sim \mathcal{N}(0, 1) \quad (9.101)$$

$$y_i = 1 = \mathbb{I}(z_i \geq 0) \quad (9.102)$$

Following (Fruhwirth-Schnatter and Fruhwirth 2010), we call this the difference RUM or **dRUM** model.

When we marginalize out z_i , we recover the probit model:

$$p(y_i = 1 | \mathbf{x}_i, \mathbf{w}) = \int \mathbb{I}(z_i \geq 0) \mathcal{N}(z_i | \mathbf{w}^T \mathbf{x}_i, 1) dz_i \quad (9.103)$$

$$= p(\mathbf{w}^T \mathbf{x}_i + \epsilon \geq 0) = p(\epsilon \geq -\mathbf{w}^T \mathbf{x}_i) \quad (9.104)$$

$$= 1 - \Phi(-\mathbf{w}^T \mathbf{x}_i) = \Phi(\mathbf{w}^T \mathbf{x}_i) \quad (9.105)$$

where we used the symmetry of the Gaussian.³ This latent variable interpretation provides an alternative way to fit the model, as discussed in Section 11.4.6.

Interestingly, if we use a Gumbel distribution for the δ 's, we induce a logistic distribution for ϵ_i , and the model reduces to logistic regression. See Section 24.5.1 for further details.

9.4.3 Ordinal probit regression *

One advantage of the latent variable interpretation of probit regression is that it is easy to extend to the case where the response variable is ordinal, that is, it can take on C discrete values which can be ordered in some way, such as low, medium and high. This is called **ordinal regression**. The basic idea is as follows. We introduce $C + 1$ thresholds γ_j and set

$$y_i = j \quad \text{if} \quad \gamma_{j-1} < z_i \leq \gamma_j \quad (9.106)$$

where $\gamma_0 \leq \dots \leq \gamma_C$. For identifiability reasons, we set $\gamma_0 = -\infty$, $\gamma_1 = 0$ and $\gamma_C = \infty$. For example, if $C = 2$, this reduces to the standard binary probit model, whereby $z_i < 0$ produces $y_i = 0$ and $z_i \geq 0$ produces $y_i = 1$. If $C = 3$, we partition the real line into 3 intervals: $(-\infty, 0]$, $(0, \gamma_2]$, (γ_2, ∞) . We can vary the parameter γ_2 to ensure the right relative amount of probability mass falls in each interval, so as to match the empirical frequencies of each class label.

Finding the MLEs for this model is a bit trickier than for binary probit regression, since we need to optimize for $\mathbf{w}$ and γ , and the latter must obey an ordering constraint. See e.g., (Kawakatsu and Largey 2009) for an approach based on EM. It is also possible to derive a simple Gibbs sampling algorithm for this model (see e.g., (Hoff 2009, p216)).

9.4.4 Multinomial probit models *

Now consider the case where the response variable can take on C unordered categorical values, $y_i \in \{1, \dots, C\}$. The **multinomial probit** model is defined as follows:

$$z_{ic} = \mathbf{w}^T \mathbf{x}_{ic} + \epsilon_{ic} \quad (9.107)$$

$$\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{R}) \quad (9.108)$$

$$y_i = \arg \max_c z_{ic} \quad (9.109)$$

See e.g., (Dow and Endersby 2004; Scott 2009; Fruhwirth-Schnatter and Fruhwirth 2010) for more details on the model and its connection to multinomial logistic regression. (By defining $\mathbf{w} = [\mathbf{w}_1, \dots, \mathbf{w}_C]$, and $\mathbf{x}_{ic} = [\mathbf{0}, \dots, \mathbf{0}, \mathbf{x}_i, \mathbf{0}, \dots, \mathbf{0}]$, we can recover the more familiar formulation $z_{ic} = \mathbf{x}_i^T \mathbf{w}_c$.) Since only relative utilities matter, we constrain $\mathbf{R}$ to be a correlation matrix. If instead of setting $y_i = \arg \max_c z_{ic}$ we use $y_{ic} = \mathbb{I}(z_{ic} > 0)$, we get a model known as **multivariate probit**, which is one way to model C correlated binary outcomes (see e.g., (Talhok et al. 2011)).

3. Note that the assumption that the Gaussian noise term is zero mean and unit variance is made without loss of generality. To see why, suppose we used some other mean μ and variance σ^2 . Then we could easily rescale $\mathbf{w}$ and add an offset term without changing the likelihood. since $P(\mathcal{N}(0, 1) \geq -\mathbf{w}^T \mathbf{x}) = P(\mathcal{N}(\mu, \sigma^2) \geq -(\mathbf{w}^T \mathbf{x} + \mu)/\sigma)$.

9.5 Multi-task learning

Sometimes we want to fit many related classification or regression models. It is often reasonable to assume the input-output mapping is similar across these different models, so we can get better performance by fitting all the parameters at the same time. In machine learning, this setup is often called **multi-task learning** (Caruana 1998), **transfer learning** (e.g., (Raina et al. 2005)), or **learning to learn** (Thrun and Pratt 1997). In statistics, this is usually tackled using hierarchical Bayesian models (Bakker and Heskes 2003), as we discuss below, although there are other possible methods (see e.g., (Chai 2010)).

9.5.1 Hierarchical Bayes for multi-task learning

Let y_{ij} be the response of the i 'th item in group j , for $i = 1 : N_j$ and $j = 1 : J$. For example, j might index schools, i might index students within a school, and y_{ij} might be the test score, as in Section 5.6.2. Or j might index people, and i might index purchases, and y_{ij} might be the identity of the item that was purchased (this is known as **discrete choice modeling** (Train 2009)). Let $\mathbf{x}_{ij}$ be a feature vector associated with y_{ij} . The goal is to fit the models $p(y_j|\mathbf{x}_j)$ for all j .

Although some groups may have lots of data, there is often a long tail, where the majority of groups have little data. Thus we can't reliably fit each model separately, but we don't want to use the same model for all groups. As a compromise, we can fit a separate model for each group, but encourage the model parameters to be similar across groups. More precisely, suppose $\mathbb{E}[y_{ij}|\mathbf{x}_{ij}] = g(\mathbf{x}_{ij}^T \beta_j)$, where g is the link function for the GLM. Furthermore, suppose $\beta_j \sim \mathcal{N}(\beta_*, \sigma_j^2 \mathbf{I})$, and that $\beta_* \sim \mathcal{N}(\mu, \sigma_*^2 \mathbf{I})$. In this model, groups with small sample size borrow statistical strength from the groups with larger sample size, because the β_j 's are correlated via the latent common parents β_* (see Section 5.5 for further discussion of this point). The term σ_j^2 controls how much group j depends on the common parents and the σ_*^2 term controls the strength of the overall prior.

Suppose, for simplicity, that $\mu = \mathbf{0}$, and that σ_j^2 and σ_*^2 are all known (e.g., they could be set by cross validation). The overall log probability has the form

$$\log p(\mathcal{D}|\beta) + \log p(\beta) = \sum_j \left[\log p(\mathcal{D}_j|\beta_j) - \frac{\|\beta_j - \beta_*\|^2}{2\sigma_j^2} \right] - \frac{\|\beta_*\|^2}{2\sigma_*^2} \quad (9.110)$$

We can perform MAP estimation of $\beta = (\beta_{1:J}, \beta_*)$ using standard gradient methods. Alternatively, we can perform an iterative optimization scheme, alternating between optimizing the β_j and the β_* ; since the likelihood and prior are convex, this is guaranteed to converge to the global optimum. Note that once the models are trained, we can discard β_* , and use each model separately.

9.5.2 Application to personalized email spam filtering

An interesting application of multi-task learning is **personalized spam filtering**. Suppose we want to fit one classifier per user, β_j . Since most users do not label their email as spam or not, it will be hard to estimate these models independently. So we will let the β_j have a common prior β_* , representing the parameters of a generic user.

In this case, we can emulate the behavior of the above model with a simple trick (Daume 2007b; Attenberg et al. 2009; Weinberger et al. 2009): we make two copies of each feature $\mathbf{x}_i$, one concatenated with the user id, and one not. The effect will be to learn a predictor of the form

$$\mathbb{E}[y_i|\mathbf{x}_i, u] = (\beta_*, \mathbf{w}_1, \dots, \mathbf{w}_J)^T [\mathbf{x}_i, \mathbb{I}(u=1)\mathbf{x}_i, \dots, \mathbb{I}(u=J)\mathbf{x}_i] \quad (9.111)$$

where u is the user id. In other words,

$$\mathbb{E}[y_i|\mathbf{x}_i, u=j] = (\beta_*^T + \mathbf{w}_j)^T \mathbf{x}_i \quad (9.112)$$

Thus β_* will be estimated from everyone's email, whereas $\mathbf{w}_j$ will just be estimated from user j 's email.

To see the correspondence with the above hierarchical Bayesian model, define $\mathbf{w}_j = \beta_j - \beta_*$. Then the log probability of the original model can be rewritten as

$$\sum_j \left[\log p(\mathcal{D}_j | \beta_* + \mathbf{w}_j) - \frac{\|\mathbf{w}_j\|^2}{2\sigma_j^2} \right] - \frac{\|\beta_*\|^2}{2\sigma_*^2} \quad (9.113)$$

If we assume $\sigma_j^2 = \sigma_*^2$, the effect is the same as using the augmented feature trick, with the same regularizer strength for both $\mathbf{w}_j$ and β_* . However, one typically gets better performance by not requiring that σ_j^2 be equal to σ_*^2 (Finkel and Manning 2009).

9.5.3 Application to domain adaptation

Domain adaptation is the problem of training a set of classifiers on data drawn from different distributions, such as email and newswire text. This problem is obviously a special case of multi-task learning, where the tasks are the same.

(Finkel and Manning 2009) used the above hierarchical Bayesian model to perform domain adaptation for two NLP tasks, namely named entity recognition and parsing. They report reasonably large improvements over fitting separate models to each dataset, and small improvements over the approach of pooling all the data and fitting a single model.

9.5.4 Other kinds of prior

In multi-task learning, it is common to assume that the prior is Gaussian. However, sometimes other priors are more suitable. For example, consider the task of **conjoint analysis**, which requires figuring out which features of a product customers like best. This can be modelled using the same hierarchical Bayesian setup as above, but where we use a sparsity-promoting prior on β_j , rather than a Gaussian prior. This is called **multi-task feature selection**. See e.g., (Lenk et al. 1996; Argyriou et al. 2008) for some possible approaches.

It is not always reasonable to assume that all tasks are all equally similar. If we pool the parameters across tasks that are qualitatively different, the performance will be worse than not using pooling, because the inductive bias of our prior is wrong. Indeed, it has been found experimentally that sometimes multi-task learning does worse than solving each task separately (this is called **negative transfer**).

One way around this problem is to use a more flexible prior, such as a mixture of Gaussians. Such flexible priors can provide robustness against prior mis-specification. See e.g., (Xue et al. 2007; Jacob et al. 2008) for details. One can of course combine mixtures with sparsity-promoting priors (Ji et al. 2009). Many other variants are possible.

9.6 Generalized linear mixed models *

Suppose we generalize the multi-task learning scenario to allow the response to include information at the group level, $\mathbf{x}_j$, as well as at the item level, $\mathbf{x}_{ij}$. Similarly, we can allow the parameters to vary across groups, β_j , or to be tied across groups, α . This gives rise to the following model:

$$\mathbb{E}[y_{ij}|\mathbf{x}_{ij}, \mathbf{x}_j] = g(\phi_1(\mathbf{x}_{ij})^T \beta_j + \phi_2(\mathbf{x}_j)^T \beta'_j + \phi_3(\mathbf{x}_{ij})^T \alpha + \phi_4(\mathbf{x}_j)^T \alpha') \quad (9.114)$$

where the ϕ_k are basis functions. This model can be represented pictorially as shown in Figure 9.2(a). (Such figures will be explained in Chapter 10.) Note that the number of β_j parameters grows with the number of groups, whereas the size of α is fixed.

Frequentists call the terms β_j **random effects**, since they vary randomly across groups, but they call α a **fixed effect**, since it is viewed as a fixed but unknown constant. A model with both fixed and random effects is called a **mixed model**. If $p(y|\mathbf{x})$ is a GLM, the overall model is called a **generalized linear mixed effects model** or **GLMM**. Such models are widely used in statistics.

9.6.1 Example: semi-parametric GLMMs for medical data

Consider the following example from (Wand 2009). Suppose y_{ij} is the amount of spinal bone mineral density (SBMD) for person j at measurement i . Let x_{ij} be the age of person, and let x_j be their ethnicity, which can be one of: White, Asian, Black, or Hispanic. The primary goal is to determine if there are significant differences in the mean SBMD among the four ethnic groups, after accounting for age. The data is shown in the light gray lines in Figure 9.2(b). We see that there is a nonlinear effect of SBMD vs age, so we will use a **semi-parametric model** which combines linear regression with non-parametric regression (Ruppert et al. 2003). We also see that there is variation across individuals within each group, so we will use a mixed effects model. Specifically, we will use $\phi_1(\mathbf{x}_{ij}) = 1$ to account for the random effect of each person; $\phi_2(x_{ij}) = 0$ since no other coefficients are person-specific; $\phi_3(x_{ij}) = [b_k(x_{ij})]$, where b_k is the k 'th spline basis functions (see Section 15.4.6.2), to account for the nonlinear effect of age; and $\phi_4(x_j) = [\mathbb{I}(x_j = w), \mathbb{I}(x_j = a), \mathbb{I}(x_j = b), \mathbb{I}(x_j = h)]$ to account for the effect of the different ethnicities. Furthermore, we use a linear link function. The overall model is therefore

$$\mathbb{E}[y_{ij}|x_{ij}, x_j] = \beta_j + \alpha^T \mathbf{b}(x_{ij}) + \epsilon_{ij} \quad (9.115)$$

$$+ \alpha'_w \mathbb{I}(x_j = w) + \alpha'_a \mathbb{I}(x_j = a) + \alpha'_b \mathbb{I}(x_j = b) + \alpha'_h \mathbb{I}(x_j = h) \quad (9.116)$$

where $\epsilon_{ij} \sim \mathcal{N}(0, \sigma_y^2)$. α contains the non-parametric part of the model related to age, α' contains the parametric part of the model related to ethnicity, and β_j is a random offset for person j . We endow all of these regression coefficients with separate Gaussian priors. We can then perform posterior inference to compute $p(\alpha, \alpha', \beta, \sigma^2 | \mathcal{D})$ (see Section 9.6.2 for

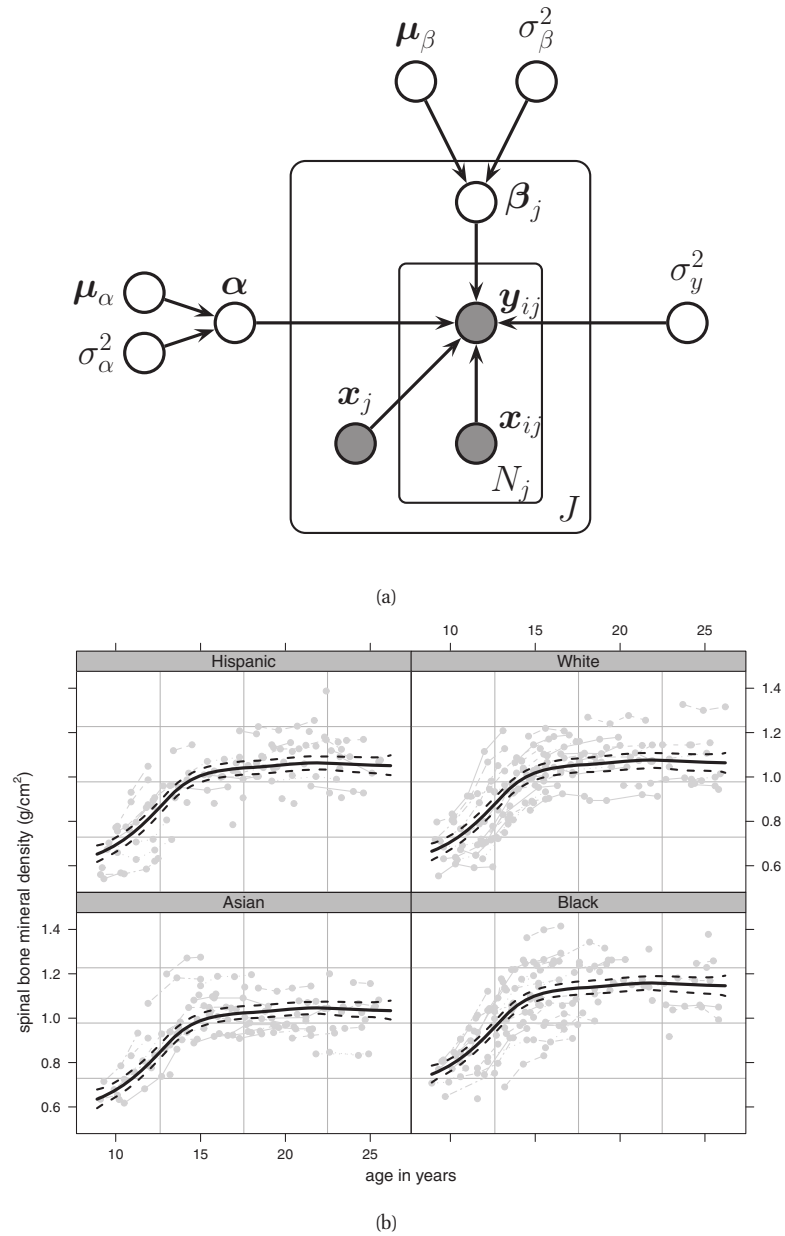


Figure 9.2 (a) Directed graphical model for generalized linear mixed effects model with J groups. (b) Spinal bone mineral density vs age for four different ethnic groups. Raw data is shown in the light gray lines. Fitted model shown in black (solid is the posterior predicted mean, dotted is the posterior predictive variance). From Figure 9 of (Wand 2009). Used with kind permission of Matt Wand

computational details). After fitting the model, we can compute the prediction for each group. See Figure 9.2(b) for the results. We can also perform significance testing, by computing $p(\alpha_g - \alpha_w | \mathcal{D})$ for each ethnic group g relative to some baseline (say, White), as we did in Section 5.2.3.

9.6.2 Computational issues

The principle problem with GLMMs is that they can be difficult to fit, for two reasons. First, $p(y_{ij} | \theta)$ may not be conjugate to the prior $p(\theta)$ where $\theta = (\alpha, \beta)$. Second, there are two levels of unknowns in the model, namely the regression coefficients θ and the means and variances of the priors $\eta = (\mu, \sigma)$.

One approach is to adopt fully Bayesian inference methods, such as variational Bayes (Hall et al. 2011) or MCMC (Gelman and Hill 2007). We discuss VB in Section 21.5, and MCMC in Section 24.1.

An alternative approach is to use empirical Bayes, which we discuss in general terms in Section 5.6. In the context of a GLMM, we can use the EM algorithm (Section 11.4), where in the E step we compute $p(\theta | \eta, \mathcal{D})$, and in the M step we optimize η . If the linear regression setting, the E step can be performed exactly, but in general we need to use approximations. Traditional methods use numerical quadrature or Monte Carlo (see e.g., (Breslow and Clayton 1993)). A faster approach is to use variational EM; see (Braun and McAuliffe 2010) for an application of variational EM to a multi-level discrete choice modeling problem.

In frequentist statistics, there is a popular method for fitting GLMMs called **generalized estimating equations** or **GEE** (Hardin and Hilbe 2003). However, we do not recommend this approach, since it is not as statistically efficient as likelihood-based methods (see Section 6.4.3). In addition, it can only provide estimates of the population parameters α , but not the random effects β_j , which are sometimes of interest in themselves.

9.7 Learning to rank *

In this section, we discuss the **learning to rank** or **LETOR** problem. That is, we want to learn a function that can rank order a set of items (we will be more precise below). The most common application is to information retrieval. Specifically, suppose we have a query q and a set of documents $d^1, \dots, d^m$ that might be relevant to q (e.g., all documents that contain the string q). We would like to sort these documents in decreasing order of relevance and show the top k to the user. Similar problems arise in other areas, such as collaborative filtering. (Ranking players in a game or tournament setting is a slightly different kind of problem; see Section 22.5.5.)

Below we summarize some methods for solving this problem, following the presentation of (Liu 2009). This material is not based on GLMs, but we include it in this chapter anyway for lack of a better place.

A standard way to measure the relevance of a document d to a query q is to use a probabilistic language model based on a bag of words model. That is, we define $\text{sim}(q, d) \triangleq p(q|d) = \prod_{i=1}^n p(q_i|d)$, where q_i is the i 'th word or term, and $p(q_i|d)$ is a multinoulli distribution estimated from document d . In practice, we need to smooth the estimated distribution, for example by using a Dirichlet prior, representing the overall frequency of each word. This can be

estimated from all documents in the system. More precisely, we can use

$$p(t|d) = (1 - \lambda) \frac{\text{TF}(t, d)}{\text{LEN}(d)} + \lambda p(t|\text{background}) \quad (9.117)$$

where $\text{TF}(t, d)$ is the frequency of term t in document d , $\text{LEN}(d)$ is the number of words in d , and $0 < \lambda < 1$ is a smoothing parameter (see e.g., Zhai and Lafferty (2004) for details).

However, there might be many other signals that we can use to measure relevance. For example, the PageRank of a web document is a measure of its authoritativeness, derived from the web's link structure (see Section 17.2.4 for details). We can also compute how often and where the query occurs in the document. Below we discuss how to learn how to combine all these signals.⁴

9.7.1 The pointwise approach

Suppose we collect some training data representing the relevance of a set of documents for each query. Specifically, for each query q , suppose that we retrieve m possibly relevant documents d_j , for $j = 1 : m$. For each query document pair, we define a feature vector, $\mathbf{x}(q, d)$. For example, this might contain the query-document similarity score and the page rank score of the document. Furthermore, suppose we have a set of labels y_j representing the degree of relevance of document d_j to query q . Such labels might be binary (e.g., relevant or irrelevant), or they may represent a degree of relevance (e.g., very relevant, somewhat relevant, irrelevant). Such labels can be obtained from query logs, by thresholding the number of times a document was clicked on for a given query.

If we have binary relevance labels, we can solve the problem using a standard binary classification scheme to estimate, $p(y = 1|\mathbf{x}(q, d))$. If we have ordered relevancy labels, we can use ordinal regression to predict the rating, $p(y = r|\mathbf{x}(q, d))$. In either case, we can then sort the documents by this scoring metric. This is called the **pointwise approach** to LETOR, and is widely used because of its simplicity. However, this method does not take into account the location of each document in the list. Thus it penalizes errors at the end of the list just as much as errors at the beginning, which is often not the desired behavior. In addition, each decision about relevance is made very myopically.

9.7.2 The pairwise approach

There is evidence (e.g., (Carterette et al. 2008)) that people are better at judging the relative relevance of two items rather than absolute relevance. Consequently, the data might tell us that d_j is more relevant than d_k for a given query, or vice versa. We can model this kind of data using a binary classifier of the form $p(y_{jk}|\mathbf{x}(q, d_j), \mathbf{x}(q, d_k))$, where we set $y_{jk} = 1$ if $\text{rel}(d_j, q) > \text{rel}(d_k, q)$ and $y_{jk} = 0$ otherwise.

One way to model such a function is as follows:

$$p(y_{jk} = 1|\mathbf{x}_j, \mathbf{x}_k) = \text{sigm}(f(\mathbf{x}_j) - f(\mathbf{x}_k)) \quad (9.118)$$

4. Rather surprisingly, Google does not (or at least, did not as of 2008) using such learning methods in its search engine. Source: Peter Norvig, quoted in <http://anand.typepad.com/datawocky/2008/05/are-human-experts-less-prone-to-catastrophic-errors-than-machine-learned-models.html>.

where $f(\mathbf{x})$ is a scoring function, often taken to be linear, $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$. This is a special kind of neural network known as **RankNet** (Burges et al. 2005) (see Section 16.5 for a general discussion of neural networks). We can find the MLE of $\mathbf{w}$ by maximizing the log likelihood, or equivalently, by minimizing the cross entropy loss, given by

$$L = \sum_{i=1}^N \sum_{j=1}^{m_i} \sum_{k=j+1}^{m_i} L_{ijk} \quad (9.119)$$

$$\begin{aligned} -L_{ijk} &= \mathbb{I}(y_{ijk} = 1) \log p(y_{ijk} = 1 | \mathbf{x}_{ij}, \mathbf{x}_{ik}, \mathbf{w}) \\ &\quad + \mathbb{I}(y_{ijk} = 0) \log p(y_{ijk} = 0 | \mathbf{x}_{ij}, \mathbf{x}_{ik}, \mathbf{w}) \end{aligned} \quad (9.120)$$

This can be optimized using gradient descent. A variant of RankNet is used by Microsoft's Bing search engine.⁵

9.7.3 The listwise approach

The pairwise approach suffers from the problem that decisions about relevance are made just based on a pair of items (documents), rather than considering the full context. We now consider methods that look at the entire list of items at the same time.

We can define a total order on a list by specifying a permutation of its indices, π . To model our uncertainty about π , we can use the **Plackett-Luce** distribution, which derives its name from independent work by (Plackett 1975) and (Luce 1959). This has the following form:

$$p(\pi | \mathbf{s}) = \prod_{j=1}^m \frac{s_j}{\sum_{u=j}^m s_u} \quad (9.121)$$

where $s_j = s(\pi^{-1}(j))$ is the score of the document ranked at the j 'th position.

To understand Equation 9.121, let us consider a simple example. Suppose $\pi = (A, B, C)$. Then we have that $p(\pi)$ is the probability of A being ranked first, times the probability of B being ranked second given that A is ranked first, times the probability of C being ranked third given that A and B are ranked first and second. In other words,

$$p(\pi | \mathbf{s}) = \frac{s_A}{s_A + s_B + s_C} \times \frac{s_B}{s_B + s_C} \times \frac{s_C}{s_C} \quad (9.122)$$

To incorporate features, we can define $s(d) = f(\mathbf{x}(q, d))$, where we often take f to be a linear function, $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$. This is known as the **ListNet** model (Cao et al. 2007). To train this model, let $\mathbf{y}_i$ be the relevance scores of the documents for query i . We then minimize the cross entropy term

$$- \sum_i \sum_{\pi} p(\pi | \mathbf{y}_i) \log p(\pi | \mathbf{s}_i) \quad (9.123)$$

Of course, as stated, this is intractable, since the i 'th term needs to sum over $m_i!$ permutations. To make this tractable, we can consider permutations over the top k positions only:

$$p(\pi_{1:k} | \mathbf{s}_{1:m}) = \prod_{j=1}^k \frac{s_j}{\sum_{u=1}^m s_u} \quad (9.124)$$

5. Source: http://www.bing.com/community/site_blogs/b/search/archive/2009/06/01/user-needs-features-and-the-science-behind-bing.aspx.

There are only $m!/(m-k)!$ such permutations. If we set $k = 1$, we can evaluate each cross entropy term (and its derivative) in $O(m)$ time.

In the special case where only one document from the presented list is deemed relevant, say $y_i = c$, we can instead use multinomial logistic regression:

$$p(y_i = c | \mathbf{x}) = \frac{\exp(s_c)}{\sum_{c'=1}^m \exp(s_{c'})} \quad (9.125)$$

This often performs at least as well as ranking methods, at least in the context of collaborative filtering (Yang et al. 2011).

9.7.4 Loss functions for ranking

There are a variety of ways to measure the performance of a ranking system, which we summarize below.

- **Mean reciprocal rank (MRR).** For a query q , let the rank position of its first relevant document be denoted by $r(q)$. Then we define the **mean reciprocal rank** to be $1/r(q)$. This is a very simple performance measure.
- **Mean average precision (MAP).** In the case of binary relevance labels, we can define the **precision at k** of some ordering as follows:

$$\text{P@}k(\pi) \triangleq \frac{\text{num. relevant documents in the top } k \text{ positions of } \pi}{k} \quad (9.126)$$

We then define the average precision as follows:

$$\text{AP}(\pi) \triangleq \frac{\sum_k \text{P@}k(\pi) \cdot I_k}{\text{num. relevant documents}} \quad (9.127)$$

where I_k is 1 iff document k is relevant. For example, if we have the relevancy labels $\mathbf{y} = (1, 0, 1, 0, 1)$, then the AP is $\frac{1}{3}(\frac{1}{1} + \frac{2}{3} + \frac{3}{5}) \approx 0.76$. Finally, we define the **mean average precision** as the AP averaged over all queries.

- **Normalized discounted cumulative gain (NDCG).** Suppose the relevance labels have multiple levels. We can define the **discounted cumulative gain** of the first k items in an ordering as follows:

$$\text{DCG@}k(\mathbf{r}) = r_1 + \sum_{i=2}^k \frac{r_i}{\log_2 i} \quad (9.128)$$

where r_i is the relevance of item i and the $\log_2$ term is used to discount items later in the list. Table 9.3 gives a simple numerical example. An alternative definition, that places stronger emphasis on retrieving relevant documents, uses

$$\text{DCG@}k(\mathbf{r}) = \sum_{i=1}^k \frac{2^{r_i} - 1}{\log_2(1 + i)} \quad (9.129)$$

The trouble with DCG is that it varies in magnitude just because the length of a returned list may vary. It is therefore common to normalize this measure by the ideal DCG, which is

i	1	2	3	4	5	6
r_i	3	2	3	0	1	2
$\log_2 i$	0	1	1.59	2.0	2.32	2.59
$\frac{r_i}{\log_2 i}$	N/A	2	1.887	0	0.431	0.772

Table 9.3 Illustration of how to compute NDCG, from http://en.wikipedia.org/wiki/Discounted_cumulative_gain. The value r_i is the relevance score of the item in position i . From this, we see that $\text{DCG}@6 = 3 + (2 + 1.887 + 0 + 0.431 + 0.772) = 8.09$. The maximum DCG is obtained using the ordering with scores 3, 3, 2, 2, 1, 0. Hence the ideal DCG is 8.693, and so the normalized DCG is $8.09 / 8.693 = 0.9306$.

the DCG obtained by using the optimal ordering: $\text{IDCG}@k(\mathbf{r}) = \arg\max_{\pi} \text{DCG}@k(\mathbf{r})$. This can be easily computed by sorting $r_{1:m}$ and then computing $\text{DCG}@k$. Finally, we define the **normalized discounted cumulative gain** or **NDCG** as DCG/IDCG . Table 9.3 gives a simple numerical example. The NDCG can be averaged over queries to give a measure of performance.

- **Rank correlation.** We can measure the correlation between the ranked list, π , and the relevance judgment, π^* , using a variety of methods. One approach, known as the (weighted) **Kendall's τ** statistics, is defined in terms of the weighted pairwise inconsistency between the two lists:

$$\tau(\pi, \pi^*) = \frac{\sum_{u < v} w_{uv} [1 + \text{sgn}(\pi_u - \pi_v) \text{sgn}(\pi_u^* - \pi_v^*)]}{2 \sum_{u < v} w_{uv}} \quad (9.130)$$

A variety of other measures are commonly used.

These loss functions can be used in different ways. In the Bayesian approach, we first fit the model using posterior inference; this depends on the likelihood and prior, but not the loss. We then choose our actions at test time to minimize the expected future loss. One way to do this is to sample parameters from the posterior, $\theta^s \sim p(\theta|\mathcal{D})$, and then evaluate, say, the precision@k for different thresholds, averaging over θ^s . See (Zhang et al. 2010) for an example of such an approach.

In the frequentist approach, we try to minimize the empirical loss on the training set. The problem is that these loss functions are not differentiable functions of the model parameters. We can either use gradient-free optimization methods, or we can minimize a surrogate loss function instead. Cross entropy loss (i.e., negative log likelihood) is an example of a widely used surrogate loss function.

Another loss, known as **weighted approximate-rank pairwise** or **WARP** loss, proposed in (Usunier et al. 2009) and extended in (Weston et al. 2010), provides a better approximation to the precision@k loss. WARP is defined as follows:

$$\text{WARP}(\mathbf{f}(\mathbf{x}, :), y) \triangleq L(\text{rank}(\mathbf{f}(\mathbf{x}, :), y)) \quad (9.131)$$

$$\text{rank}(\mathbf{f}(\mathbf{x}, :), y) = \sum_{y' \neq y} \mathbb{I}(f(\mathbf{x}, y') \geq f(\mathbf{x}, y)) \quad (9.132)$$

$$L(k) \triangleq \sum_{j=1}^k \alpha_j, \quad \text{with } \alpha_1 \geq \alpha_2 \geq \dots \geq 0 \quad (9.133)$$

Here $\mathbf{f}(x, :) = [f(\mathbf{x}, 1), \dots, f(\mathbf{x}, |y|)]$ is the vector of scores for each possible output label, or, in IR terms, for each possible document corresponding to input query $\mathbf{x}$. The expression $\text{rank}(\mathbf{f}(\mathbf{x}, :), y)$ measures the rank of the true label y assigned by this scoring function. Finally, L transforms the integer rank into a real-valued penalty. Using $\alpha_1 = 1$ and $\alpha_{j>1} = 0$ would optimize the proportion of top-ranked correct labels. Setting $\alpha_{1:k}$ to be non-zero values would optimize the top k in the ranked list, which will induce good performance as measured by MAP or precision@k. As it stands, WARP loss is still hard to optimize, but it can be further approximated by Monte Carlo sampling, and then optimized by gradient descent, as described in (Weston et al. 2010).

Exercises

Exercise 9.1 Conjugate prior for univariate Gaussian in exponential family form

Derive the conjugate prior for μ and $\lambda = 1/\sigma^2$ for a univariate Gaussian using the exponential family, by analogy to Section 9.2.5.5. By suitable reparameterization, show that the prior has the form $p(\mu, \lambda) = \mathcal{N}(\mu|\gamma, \lambda(2\alpha - 1))\text{Ga}(\lambda|\alpha, \beta)$, and thus only has 3 free parameters.

Exercise 9.2 The MVN is in the exponential family

Show that we can write the MVN in exponential family form. Hint: use the information form defined in Section 4.3.3.

10

Directed graphical models (Bayes nets)

10.1 Introduction

I basically know of two principles for treating complicated systems in simple ways: the first is the principle of modularity and the second is the principle of abstraction. I am an apologist for computational probability in machine learning because I believe that probability theory implements these two principles in deep and intriguing ways — namely through factorization and through averaging. Exploiting these two mechanisms as fully as possible seems to me to be the way forward in machine learning. — Michael Jordan, 1997 (quoted in (Frey 1998)).

Suppose we observe multiple correlated variables, such as words in a document, pixels in an image, or genes in a microarray. How can we compactly *represent* the **joint distribution** $p(\mathbf{x}|\boldsymbol{\theta})$? How can we use this distribution to *infer* one set of variables given another in a reasonable amount of computation time? And how can we *learn* the parameters of this distribution with a reasonable amount of data? These questions are at the core of probabilistic modeling, inference and learning, and form the topic of this chapter.

10.1.1 Chain rule

By the **chain rule** of probability, we can always represent a joint distribution as follows, using any ordering of the variables:

$$p(x_{1:V}) = p(x_1)p(x_2|x_1)p(x_3|x_2, x_1)p(x_4|x_1, x_2, x_3) \dots p(x_V|x_{1:V-1}) \quad (10.1)$$

where V is the number of variables, the Matlab-like notation $1 : V$ denotes the set $\{1, 2, \dots, V\}$, and where we have dropped the conditioning on the fixed parameters $\boldsymbol{\theta}$ for brevity. The problem with this expression is that it becomes more and more complicated to represent the conditional distributions $p(x_t|\mathbf{x}_{1:t-1})$ as t gets large.

For example, suppose all the variables have K states. We can represent $p(x_1)$ as a table of $O(K)$ numbers, representing a discrete distribution (there are actually only $K - 1$ free parameters, due to the sum-to-one constraint, but we write $O(K)$ for simplicity). Similarly, we can represent $p(x_2|x_1)$ as a table of $O(K^2)$ numbers by writing $p(x_2 = j|x_1 = i) = T_{ij}$; we say that $\mathbf{T}$ is a **stochastic matrix**, since it satisfies the constraint $\sum_j T_{ij} = 1$ for all rows i , and $0 \leq T_{ij} \leq 1$ for all entries. Similarly, we can represent $p(x_3|x_1, x_2)$ as a 3d table with

$O(K^3)$ numbers. These are called **conditional probability tables** or **CPTs**. We see that there are $O(K^V)$ parameters in the model. We would need an awful lot of data to learn so many parameters.

One solution is to replace each CPT with a more parsimonious **conditional probability distribution** or **CPD**, such as multinomial logistic regression, i.e., $p(x_t = k | \mathbf{x}_{1:t-1}) = \mathcal{S}(\mathbf{W}_t \mathbf{x}_{1:t-1})_k$. The total number of parameters is now only $O(K^2 V^2)$, making this a compact density model (Neal 1992; Frey 1998). This is adequate if all we want to do is evaluate the probability of a fully observed vector $\mathbf{x}_{1:T}$. For example, we can use this model to define a class-conditional density, $p(\mathbf{x} | y = c)$, thus making a generative classifier (Bengio and Bengio 2000). However, this model is not useful for other kinds of prediction tasks, since each variable depends on all the previous variables. So we need another approach.

10.1.2 Conditional independence

The key to efficiently representing large joint distributions is to make some assumptions about conditional independence (**CI**). Recall from Section 2.2.4 that X and Y are conditionally independent given Z , denoted $X \perp Y | Z$, if and only if (iff) the conditional joint can be written as a product of conditional marginals, i.e.,

$$X \perp Y | Z \iff p(X, Y | Z) = p(X | Z)p(Y | Z) \quad (10.2)$$

Let us see why this might help. Suppose we assume that $x_{t+1} \perp \mathbf{x}_{1:t-1} | x_t$, or in words, “the future is independent of the past given the present”. This is called the (first order) **Markov assumption**. Using this assumption, plus the chain rule, we can write the joint distribution as follows:

$$p(\mathbf{x}_{1:V}) = p(x_1) \prod_{t=1}^V p(x_t | x_{t-1}) \quad (10.3)$$

This is called a (first-order) **Markov chain**. They can be characterized by an initial distribution over states, $p(x_1 = i)$, plus a **state transition matrix** $p(x_t = j | x_{t-1} = i)$. See Section 17.2 for more information.

10.1.3 Graphical models

Although the first-order Markov assumption is useful for defining distributions on 1d sequences, how can we define distributions on 2d images, or 3d videos, or, in general, arbitrary collections of variables (such as genes belonging to some biological pathway)? This is where graphical models come in.

A **graphical model (GM)** is a way to represent a joint distribution by making CI assumptions. In particular, the nodes in the graph represent random variables, and the (lack of) edges represent CI assumptions. (A better name for these models would in fact be “independence diagrams”, but the term “graphical models” is now entrenched.) There are several kinds of graphical model, depending on whether the graph is directed, undirected, or some combination of directed and undirected. In this chapter, we just study directed graphs. We consider undirected graphs in Chapter 19.

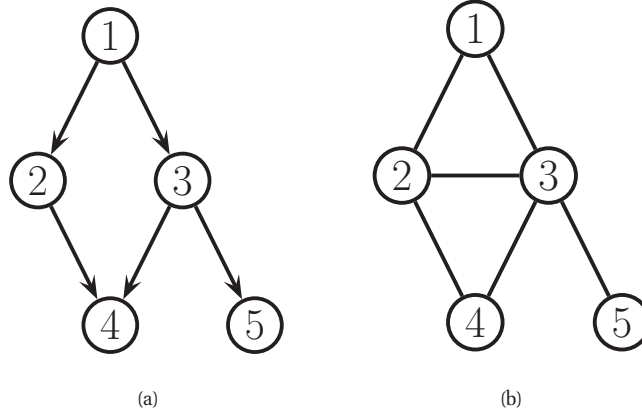


Figure 10.1 (a) A simple DAG on 5 nodes, numbered in topological order. Node 1 is the root, nodes 4 and 5 are the leaves. (b) A simple undirected graph, with the following maximal cliques: $\{1, 2, 3\}$, $\{2, 3, 4\}$, $\{3, 5\}$.

10.1.4 Graph terminology

Before we continue, we must define a few basic terms, most of which are very intuitive.

A **graph** $G = (\mathcal{V}, \mathcal{E})$ consists of a set of **nodes** or **vertices**, $\mathcal{V} = \{1, \dots, V\}$, and a set of **edges**, $\mathcal{E} = \{(s, t) : s, t \in \mathcal{V}\}$. We can represent the graph by its **adjacency matrix**, in which we write $G(s, t) = 1$ to denote $(s, t) \in \mathcal{E}$, that is, if $s \rightarrow t$ is an edge in the graph. If $G(s, t) = 1$ iff $G(t, s) = 1$, we say the graph is **undirected**, otherwise it is **directed**. We usually assume $G(s, s) = 0$, which means there are no **self loops**.

Here are some other terms we will commonly use:

- **Parent** For a directed graph, the **parents** of a node is the set of all nodes that feed into it: $\text{pa}(s) \triangleq \{t : G(t, s) = 1\}$.
- **Child** For a directed graph, the **children** of a node is the set of all nodes that feed out of it: $\text{ch}(s) \triangleq \{t : G(s, t) = 1\}$.
- **Family** For a directed graph, the **family** of a node is the node and its parents, $\text{fam}(s) = \{s\} \cup \text{pa}(s)$.
- **Root** For a directed graph, a **root** is a node with no parents.
- **Leaf** For a directed graph, a **leaf** is a node with no children.
- **Ancestors** For a directed graph, the **ancestors** are the parents, grand-parents, etc of a node. That is, the ancestors of t is the set of nodes that connect to t via a trail: $\text{anc}(t) \triangleq \{s : s \rightsquigarrow t\}$.
- **Descendants** For a directed graph, the **descendants** are the children, grand-children, etc of a node. That is, the descendants of s is the set of nodes that can be reached via trails from s : $\text{desc}(s) \triangleq \{t : s \rightsquigarrow t\}$.
- **Neighbors** For any graph, we define the **neighbors** of a node as the set of all immediately connected nodes, $\text{nbr}(s) \triangleq \{t : G(s, t) = 1 \vee G(t, s) = 1\}$. For an undirected graph, we

write $s \sim t$ to indicate that s and t are neighbors (so $(s, t) \in \mathcal{E}$ is an edge in the graph).

- **Degree** The **degree** of a node is the number of neighbors. For directed graphs, we speak of the **in-degree** and **out-degree**, which count the number of parents and children.
- **Cycle or loop** For any graph, we define a **cycle** or **loop** to be a series of nodes such that we can get back to where we started by following edges, $s_1 - s_2 \cdots - s_n - s_1$, $n \geq 2$. If the graph is directed, we may speak of a directed cycle. For example, in Figure 10.1(a), there are no directed cycles, but $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$ is an undirected cycle.
- **DAG** A **directed acyclic graph** or **DAG** is a directed graph with no directed cycles. See Figure 10.1(a) for an example.
- **Topological ordering** For a DAG, a **topological ordering** or **total ordering** is a numbering of the nodes such that parents have lower numbers than their children. For example, in Figure 10.1(a), we can use $(1, 2, 3, 4, 5)$, or $(1, 3, 2, 5, 4)$, etc.
- **Path or trail** A **path** or **trail** $s \rightsquigarrow t$ is a series of directed edges leading from s to t .
- **Tree** An undirected **tree** is an undirected graph with no cycles. A directed tree is a DAG in which there are no directed cycles. If we allow a node to have multiple parents, we call it a **polytree**, otherwise we call it a moral directed tree.
- **Forest** A **forest** is a set of trees.
- **Subgraph** A (node-induced) **subgraph** G_A is the graph created by using the nodes in A and their corresponding edges, $G_A = (\mathcal{V}_A, \mathcal{E}_A)$.
- **Clique** For an undirected graph, a **clique** is a set of nodes that are all neighbors of each other. A **maximal clique** is a clique which cannot be made any larger without losing the clique property. For example, in Figure 10.1(b), $\{1, 2\}$ is a clique but it is not maximal, since we can add 3 and still maintain the clique property. In fact, the maximal cliques are as follows: $\{1, 2, 3\}$, $\{2, 3, 4\}$, $\{3, 5\}$.

10.1.5 Directed graphical models

A **directed graphical model** or **DGM** is a GM whose graph is a DAG. These are more commonly known as **Bayesian networks**. However, there is nothing inherently “Bayesian” about Bayesian networks: they are just a way of defining probability distributions. These models are also called **belief networks**. The term “belief” here refers to subjective probability. Once again, there is nothing inherently subjective about the kinds of probability distributions represented by DGMs. Finally, these models are sometimes called **causal networks**, because the directed arrows are sometimes interpreted as representing causal relations. However, there is nothing inherently causal about DGMs. (See Section 26.6.1 for a discussion of causal DGMs.) For these reasons, we use the more neutral (but less glamorous) term DGM.

The key property of DAGs is that the nodes can be ordered such that parents come before children. This is called a topological ordering, and it can be constructed from any DAG. Given such an order, we define the **ordered Markov property** to be the assumption that a node only depends on its immediate parents, not on all predecessors in the ordering, i.e.,

$$x_s \perp \mathbf{x}_{\text{pred}(s) \setminus \text{pa}(s)} \mid \mathbf{x}_{\text{pa}(s)} \quad (10.4)$$

where $\text{pa}(s)$ are the parents of node s , and $\text{pred}(s)$ are the predecessors of node s in the ordering. This is a natural generalization of the first-order Markov property to from chains to general DAGs.

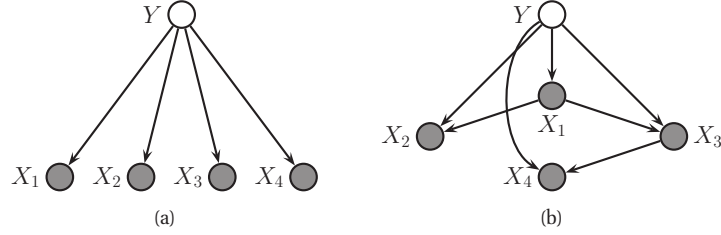


Figure 10.2 (a) A naive Bayes classifier represented as a DGM. We assume there are $D = 4$ features, for simplicity. Shaded nodes are observed, unshaded nodes are hidden. (b) Tree-augmented naive Bayes classifier for $D = 4$ features. In general, the tree topology can change depending on the value of y .

For example, the DAG in Figure 10.1(a) encodes the following joint distribution:

$$p(\mathbf{x}_{1:5}) = p(x_1)p(x_2|x_1)p(x_3|x_1, x_2)p(x_4|x_1, x_2, x_3)p(x_5|x_1, x_2, x_3, x_4) \quad (10.5)$$

$$= p(x_1)p(x_2|x_1)p(x_3|x_1)p(x_4|x_2, x_3)p(x_5|x_3) \quad (10.6)$$

In general, we have

$$p(\mathbf{x}_{1:V}|G) = \prod_{t=1}^V p(x_t|\mathbf{x}_{\text{pa}(t)}) \quad (10.7)$$

where each term $p(x_t|\mathbf{x}_{\text{pa}(t)})$ is a CPD. We have written the distribution as $p(\mathbf{x}|G)$ to emphasize that this equation only holds if the CI assumptions encoded in DAG G are correct. However, we will usually drop this explicit conditioning for brevity. If each node has $O(F)$ parents and K states, the number of parameters in the model is $O(VK^F)$, which is much less than the $O(K^V)$ needed by a model which makes no CI assumptions.

10.2 Examples

In this section, we show a wide variety of commonly used probabilistic models can be conveniently represented as DGMs.

10.2.1 Naive Bayes classifiers

In Section 3.5, we introduced the naive Bayes classifier. This assumes the features are conditionally independent given the class label. This assumption is illustrated in Figure 10.2(a). This allows us to write the joint distribution as follows:

$$p(y, \mathbf{x}) = p(y) \prod_{j=1}^D p(x_j|y) \quad (10.8)$$

The naive Bayes assumption is rather naive, since it assumes the features are conditionally independent. One way to capture correlation between the features is to use a graphical model. In particular, if the model is a tree, the method is known as a **tree-augmented naive Bayes**

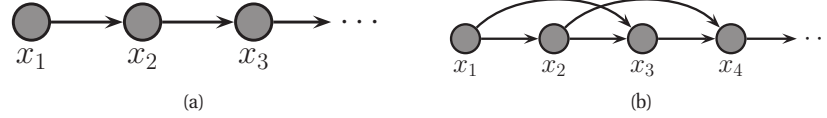


Figure 10.3 A first and second order Markov chain.

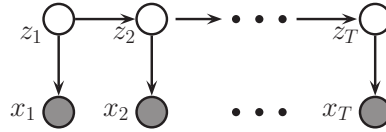


Figure 10.4 A first-order HMM.

classifier or **TAN** model (Friedman et al. 1997). This is illustrated in Figure 10.2(b). The reason to use a tree, as opposed to a generic graph, is two-fold. First, it is easy to find the optimal tree structure using the Chow-Liu algorithm, as explained in Section 26.3. Second, it is easy to handle missing features in a tree-structured model, as we explain in Section 20.2.

10.2.2 Markov and hidden Markov models

Figure 10.3(a) illustrates a first-order Markov chain as a DAG. Of course, the assumption that the immediate past, x_{t-1} , captures everything we need to know about the entire history, $\mathbf{x}_{1:t-2}$, is a bit strong. We can relax it a little by adding a dependence from x_{t-2} to x_t as well; this is called a **second order Markov chain**, and is illustrated in Figure 10.3(b). The corresponding joint has the following form:

$$p(\mathbf{x}_{1:T}) = p(x_1, x_2)p(x_3|x_1, x_2)p(x_4|x_2, x_3) \dots = p(x_1, x_2) \prod_{t=3}^T p(x_t|x_{t-1}, x_{t-2}) \quad (10.9)$$

We can create higher-order Markov models in a similar way. See Section 17.2 for a more detailed discussion of Markov models.

Unfortunately, even the second-order Markov assumption may be inadequate if there are long-range correlations amongst the observations. We can't keep building ever higher order models, since the number of parameters will blow up. An alternative approach is to assume that there is an underlying hidden process, that can be modeled by a first-order Markov chain, but that the data is a noisy observation of this process. The result is known as a **hidden Markov model** or **HMM**, and is illustrated in Figure 10.4. Here z_t is known as a **hidden variable** at “time” t , and x_t is the observed variable. (We put “time” in quotation marks, since these models can be applied to any kind of sequence data, such as genomics or language, where t represents location rather than time.) The CPD $p(z_t|z_{t-1})$ is the **transition model**, and the CPD $p(\mathbf{x}_t|z_t)$ is the **observation model**.

h_0	h_1	h_2	$P(v = 0 h_1, h_2)$	$P(v = 1 h_1, h_2)$
1	0	0	θ_0	$1 - \theta_0$
1	1	0	$\theta_0\theta_1$	$1 - \theta_0\theta_1$
1	0	1	$\theta_0\theta_2$	$1 - \theta_0\theta_2$
1	1	1	$\theta_0\theta_1\theta_2$	$1 - \theta_0\theta_1\theta_2$

Table 10.1 Noisy-OR CPD for 2 parents augmented with leak node. We have omitted the t subscript for brevity.

The hidden variables often represent quantities of interest, such as the identity of the word that someone is currently speaking. The observed variables are what we measure, such as the acoustic waveform. What we would like to do is estimate the hidden state given the data, i.e., to compute $p(z_t|\mathbf{x}_{1:t}, \boldsymbol{\theta})$. This is called **state estimation**, and is just another form of probabilistic inference. See Chapter 17 for further details on HMMs.

10.2.3 Medical diagnosis

Consider modeling the relationship between various variables that are measured in an intensive care unit (ICU), such as the breathing rate of a patient, their blood pressure, etc. The **alarm network** in Figure 10.5(a) is one way to represent these (in)dependencies (Beinlich et al. 1989). This model has 37 variables and 504 parameters.

Since this model was created by hand, by a process called **knowledge engineering**, it is known as a **probabilistic expert system**. In Section 10.4, we discuss how to learn the parameters of DGMs from data, assuming the graph structure is known, and in Chapter 26, we discuss how to learn the graph structure itself.

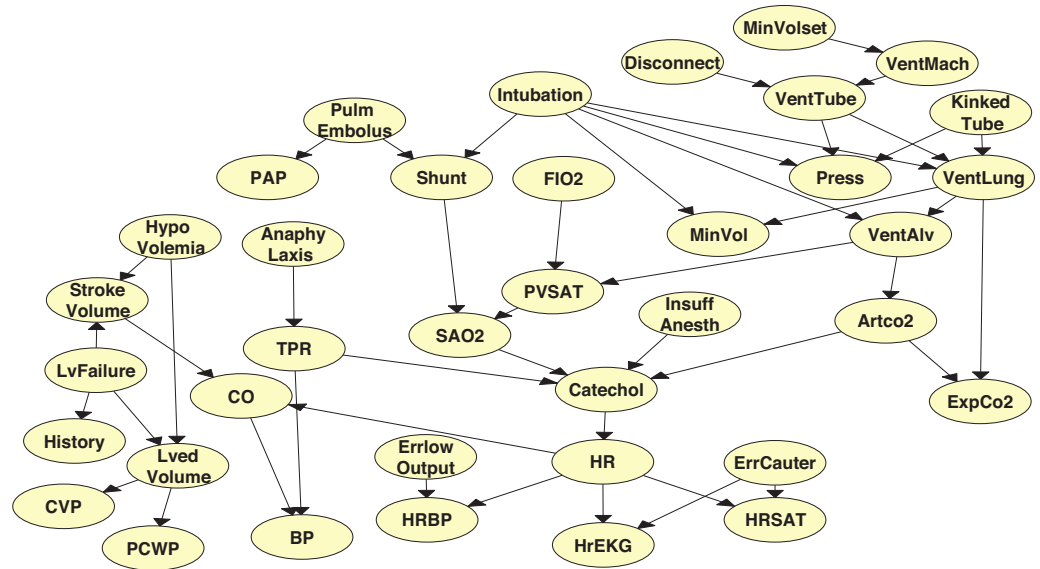
A different kind of medical diagnosis network, known as the **quick medical reference** or **QMR** network (Shwe et al. 1991), is shown in Figure 10.5(b). This was designed to model infectious diseases. The QMR model is a **bipartite graph** structure, with diseases (causes) at the top and symptoms or findings at the bottom. All nodes are binary. We can write the distribution as follows:

$$p(\mathbf{v}, \mathbf{h}) = \prod_s p(h_s) \prod_t p(v_t | \mathbf{h}_{\text{pa}(t)}) \quad (10.10)$$

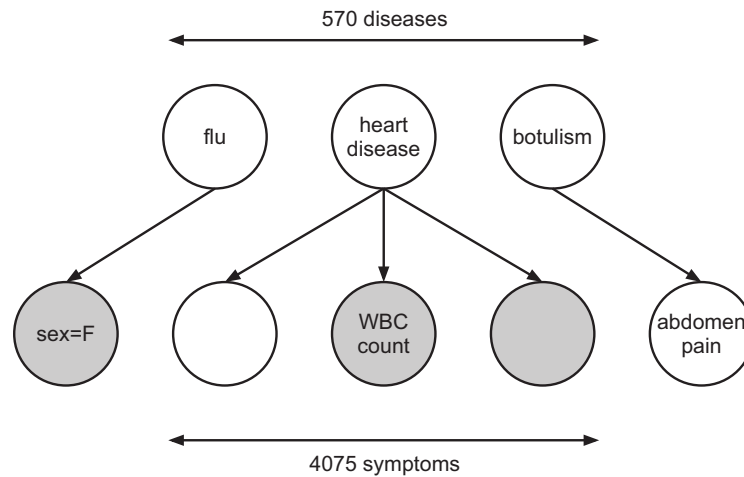
where h_s represent the **hidden nodes** (diseases), and v_t represent the **visible nodes** (symptoms).

The CPD for the root nodes are just Bernoulli distributions, representing the prior probability of that disease. Representing the CPDs for the leaves (symptoms) using CPTs would require too many parameters, because the **fan-in** (number of parents) of many leaf nodes is very high. A natural alternative is to use logistic regression to model the CPD, $p(v_t = 1 | \mathbf{h}_{\text{pa}(t)}) = \text{sigm}(\mathbf{w}_t^T \mathbf{h}_{\text{pa}(t)})$. (A DGM in which the CPDs are logistic regression distributions is known as a **sigmoid belief net** (Neal 1992).) However, since the parameters of this model were created by hand, an alternative CPD, known as the **noisy-OR** model, was used.

The noisy-OR model assumes that if a parent is on, then the child will usually also be on (since it is an or-gate), but occasionally the “links” from parents to child may fail, independently at random. In this case, even if the parent is on, the child may be off. To model this more precisely, let $\theta_{st} = 1 - q_{st}$ be the probability that the $s \rightarrow t$ link fails, so $q_{st} = 1 - \theta_{st} = p(v_t =$



(a)



(b)

Figure 10.5 (a) The alarm network. Figure generated by `visualizeAlarmNetwork`. (b) The QMR network.

G^p	G^m	$p(X = a)$	$p(X = b)$	$p(X = o)$	$p(X = ab)$
a	a	1	0	0	0
a	b	0	0	0	1
a	o	1	0	0	0
b	a	0	0	0	1
b	b	0	1	0	0
b	o	0	1	0	0
o	a	1	0	0	0
o	b	0	1	0	0
o	o	0	0	1	0

Table 10.2 CPT which encodes a mapping from genotype to phenotype (bloodtype). This is a deterministic, but many-to-one, mapping.

$1|h_s = 1, \mathbf{h}_{-s} = 0$) is the probability that s can activate t on its own (its “causal power”). The only way for the child to be off is if all the links from all parents that are on fail independently at random. Thus

$$p(v_t = 0|\mathbf{h}) = \prod_{s \in \text{pa}(t)} \theta_{st}^{1(h_s=1)} \quad (10.11)$$

Obviously, $p(v_t = 1|\mathbf{h}) = 1 - p(v_t = 0|\mathbf{h})$.

If we observe that $v_t = 1$ but all its parents are off, then this contradicts the model. Such a data case would get probability zero under the model, which is problematic, because it is possible that someone exhibits a symptom but does not have any of the specified diseases. To handle this, we add a dummy **leak node** h_0 , which is always on; this represents “all other causes”. The parameter q_{0t} represents the probability that the background leak can cause the effect on its own. The modified CPD becomes $p(v_t = 0|\mathbf{h}) = \theta_{0t} \prod_{s \in \text{pa}(t)} \theta_{st}^{h_s}$. See Table 10.1 for a numerical example.

If we define $w_{st} \triangleq \log(\theta_{st})$, we can rewrite the CPD as

$$p(v_t = 1|\mathbf{h}) = 1 - \exp \left(w_{0t} + \sum_s h_s w_{st} \right) \quad (10.12)$$

We see that this is similar to a logistic regression model.

Bipartite models with noisy-OR CPDs are called **BN2O** models. It is relatively easy to set the θ_{st} parameters by hand, based on domain expertise. However, it is also possible to learn them from data (see e.g. (Neal 1992; Meek and Heckerman 1997)). Noisy-OR CPDs have also proved useful in modeling human causal learning (Griffiths and Tenenbaum 2005), as well as general binary classification settings (Yuille and Zheng 2009).

10.2.4 Genetic linkage analysis *

Another important (and historically very early) application of DGMs is to the problem of **genetic linkage analysis**. We start with a **pedigree graph**, which is a DAG that representing the relationship between parents and children, as shown in Figure 10.6(a). We then convert this to a DGM, as we explain below. Finally we perform probabilistic inference in the resulting model.

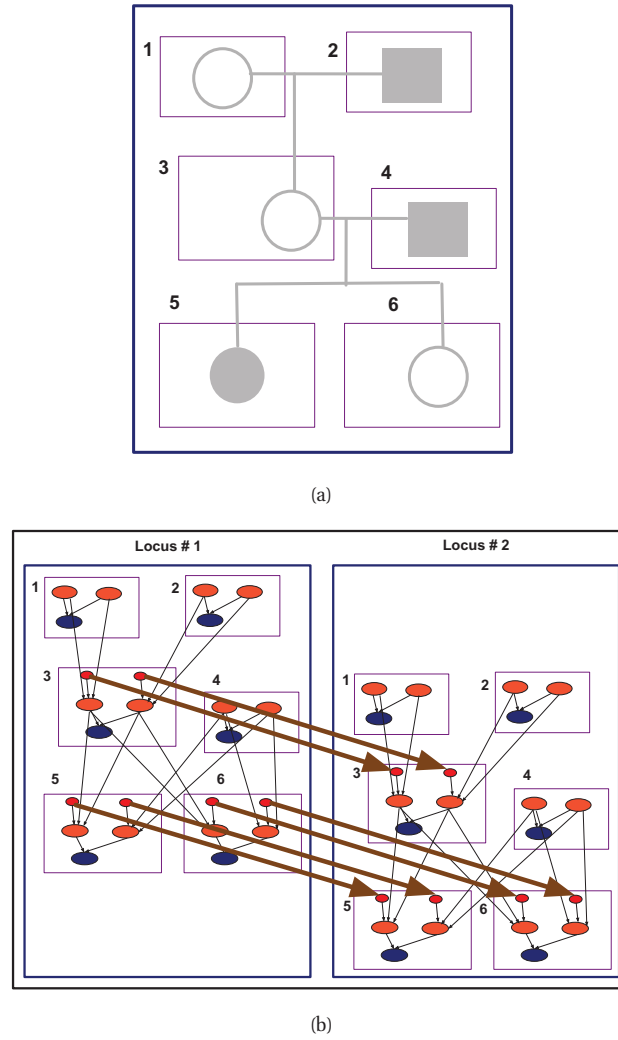


Figure 10.6 Left: family tree, circles are females, squares are males. Individuals with the disease of interest are highlighted. Right: DGM for two loci. Blue nodes X_{ij} is the observed phenotype for individual i at locus j . All other nodes are hidden. Orange nodes $G_{ij}^{p/m}$ is the paternal/ maternal allele. Small red nodes $z_{ijl}^{p/m}$ are the paternal/ maternal selection switching variables. These are linked across loci, $z_{ij}^m \rightarrow z_{i,j+1}^m$ and $z_{ij}^p \rightarrow z_{i,j+1}^p$. The founder (root) nodes do not have any parents, and hence do not need switching variables. Based on Figure 3 from (Friedman et al. 2000).

In more detail, for each person (or animal) i and location or locus j along the genome, we create three nodes: the observed **marker** X_{ij} (which can be a property such as blood type, or just a fragment of DNA that can be measured), and two hidden **alleles**, G_{ij}^m and G_{ij}^p , one inherited from i 's mother (maternal allele) and the other from i 's father (paternal allele). Together, the ordered pair $\mathbf{G}_{ij} = (G_{ij}^m, G_{ij}^p)$ constitutes i 's hidden **genotype** at locus j .

Obviously we must add $G_{ij}^m \rightarrow X_{ij}$ and $G_{ij}^p \rightarrow X_{ij}$ arcs representing the fact that genotypes cause phenotypes (observed manifestations of genotypes). The CPD $p(X_{ij}|G_{ij}^m, G_{ij}^p)$ is called the **penetrance model**. As a very simple example, suppose $X_{ij} \in \{A, B, O, AB\}$ represents person i 's observed bloodtype, and $G_{ij}^m, G_{ij}^p \in \{A, B, O\}$ is their genotype. We can represent the penetrance model using the deterministic CPD shown in Table 10.2. For example, A dominates O, so if a person has genotype AO or OA, their phenotype will be A.

In addition, we add arcs from i 's mother and father into $\mathbf{G}_{ij}$, reflecting the **Mendelian inheritance** of genetic material from one's parents. More precisely, let $m_i = k$ be i 's mother. Then G_{ij}^m could either be equal to G_{kj}^m or G_{kj}^p , that is, i 's maternal allele is a copy of one of its mother's two alleles. Let Z_{ij}^m be a hidden variable that specifies the choice. We can model this using the following CPD, known as the **inheritance model**:

$$p(G_{ij}^m | G_{kj}^m, G_{kj}^p, Z_{ij}^m) = \begin{cases} \mathbb{I}(G_{ij}^m = G_{kj}^m) & \text{if } Z_{ij}^m = m \\ \mathbb{I}(G_{ij}^m = G_{kj}^p) & \text{if } Z_{ij}^m = p \end{cases} \quad (10.13)$$

We can define $p(G_{ij}^p | G_{kj}^m, G_{kj}^p, Z_{ij}^p)$ similarly, where $k = p_i$ is i 's father. The values of the Z_{ij} are said to specify the **phase** of the genotype. The values of $G_{i,j}^p$, $G_{i,j}^m$, $Z_{i,j}^p$ and $Z_{i,j}^m$ constitute the **haplotype** of person i at locus j .¹

Next, we need to specify the prior for the root nodes, $p(G_{ij}^m)$ and $p(G_{ij}^p)$. This is called the **founder model**, and represents the overall prevalence of different kinds of alleles in the population. We usually assume independence between the loci for these founder alleles.

Finally, we need to specify priors for the switch variables that control the inheritance process. These variables are spatially correlated, since adjacent sites on the genome are typically inherited together (recombination events are rare). We can model this by imposing a two-state Markov chain on the Z 's, where the probability of switching state at locus j is given by $\theta_j = \frac{1}{2}(1 - e^{-2d_j})$, where d_j is the distance between loci j and $j + 1$. This is called the **recombination model**.

The resulting DGM is shown in Figure 10.6(b): it is a series of replicated pedigree DAGs, augmented with switching Z variables, which are linked using Markov chains. (There is a related model known as **phylogenetic HMM** (Siepel and Haussler 2003), which is used to model evolution amongst phylogenies.)

As a simplified example of how this model can be used, suppose we only have one locus, corresponding to blood type. For brevity, we will drop the j index. Suppose we observe $x_i = A$. Then there are 3 possible genotypes: $\mathbf{G}_i$ is (A, A) , (A, O) or (O, A) . There is ambiguity because the genotype to phenotype mapping is many-to-one. We want to reverse this mapping. This is known as an **inverse problem**. Fortunately, we can use the blood types of relatives to help disambiguate the evidence. Information will “flow” from the other $x_{i'}$'s up to their $\mathbf{G}_{i'}$'s, then across to i 's $\mathbf{G}_i$ via the pedigree DAG. Thus we can combine our **local evidence** $p(x_i | \mathbf{G}_i)$

1. Sometimes the observed marker is equal to the unphased genotype, which is the unordered set $\{G_{ij}^p, G_{ij}^m\}$; however, the phased or hidden genotype is not directly measurable.

with an informative prior, $p(\mathbf{G}_i|\mathbf{x}_{-i})$, conditioned on the other data, to get a less entropic local posterior, $p(G_i|\mathbf{x}) \propto p(x_i|G_i)p(G_i|\mathbf{x}_{-i})$.

In practice, the model is used to try to determine where along the genome a given disease-causing gene is assumed to lie — this is the genetic linkage analysis task. The method works as follows. First, suppose all the parameters of the model, including the distance between all the marker loci, are known. The only unknown is the location of the disease-causing gene. If there are L marker loci, we construct $L + 1$ models: in model ℓ , we postulate that the disease gene comes after marker ℓ , for $0 < \ell < L + 1$. We can estimate the Markov switching parameter $\hat{\theta}_\ell$, and hence the distance d_ℓ between the disease gene and its nearest known locus. We measure the quality of that model using its likelihood, $p(\mathcal{D}|\hat{\theta}_\ell)$. We then can then pick the model with highest likelihood (which is equivalent to the MAP model under a uniform prior).

Note, however, that computing the likelihood requires marginalizing out all the hidden Z and G variables. See (Fishelson and Geiger 2002) and the references therein for some exact methods for this task; these are based on the variable elimination algorithm, which we discuss in Section 20.3. Unfortunately, for reasons we explain in Section 20.5, exact methods can be computationally intractable if the number of individuals and/or loci is large. See (Albers et al. 2006) for an approximate method for computing the likelihood; this is based on a form of variational inference, which we will discuss in Section 22.4.1.

10.2.5 Directed Gaussian graphical models *

Consider a DGM where all the variables are real-valued, and all the CPDs have the following form:

$$p(x_t|\mathbf{x}_{\text{pa}(t)}) = \mathcal{N}(x_t|\mu_t + \mathbf{w}_t^T \mathbf{x}_{\text{pa}(t)}, \sigma_t^2) \quad (10.14)$$

This is called a **linear Gaussian** CPD. As we show below, multiplying all these CPDs together results in a large joint Gaussian distribution of the form $p(\mathbf{x}) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma})$. This is called a directed GGM, or a **Gaussian Bayes net**.

We now explain how to derive $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ from the CPD parameters, following (Shachter and Kenley 1989, App. B). For convenience, we will rewrite the CPDs in the following form:

$$x_t = \mu_t + \sum_{s \in \text{pa}(t)} w_{ts}(x_s - \mu_s) + \sigma_t z_t \quad (10.15)$$

where $z_t \sim \mathcal{N}(0, 1)$, σ_t is the conditional standard deviation of x_t given its parents, w_{ts} is the strength of the $s \rightarrow t$ edge, and μ_t is the local mean.²

It is easy to see that the global mean is just the concatenation of the local means, $\boldsymbol{\mu} = (\mu_1, \dots, \mu_D)$. We now derive the global covariance, $\boldsymbol{\Sigma}$. Let $\mathbf{S} \triangleq \text{diag}(\boldsymbol{\sigma})$ be a diagonal matrix containing the standard deviations. We can rewrite Equation 10.15 in matrix-vector form as follows:

$$(\mathbf{x} - \boldsymbol{\mu}) = \mathbf{W}(\mathbf{x} - \boldsymbol{\mu}) + \mathbf{S}\mathbf{z} \quad (10.16)$$

2. If we do not subtract off the parent's mean (i.e., if we use $x_t = \mu_t + \sum_{s \in \text{pa}(t)} w_{ts}x_s + \sigma_t z_t$), the derivation of $\boldsymbol{\Sigma}$ is much messier, as can be seen by looking at (Bishop 2006b, p370).

Now let $\mathbf{e}$ be a vector of noise terms:

$$\mathbf{e} \triangleq \mathbf{S}\mathbf{z} \quad (10.17)$$

We can rearrange this to get

$$\mathbf{e} = (\mathbf{I} - \mathbf{W})(\mathbf{x} - \boldsymbol{\mu}) \quad (10.18)$$

Since $\mathbf{W}$ is lower triangular (because $w_{ts} = 0$ if $t > s$ in the topological ordering), we have that $\mathbf{I} - \mathbf{W}$ is lower triangular with 1s on the diagonal. Hence

$$\begin{pmatrix} e_1 \\ e_2 \\ \vdots \\ e_d \end{pmatrix} = \begin{pmatrix} 1 & & & & \\ -w_{21} & 1 & & & \\ -w_{32} & -w_{31} & 1 & & \\ \vdots & & & \ddots & \\ -w_{d1} & -w_{d2} & \dots & -w_{d,d-1} & 1 \end{pmatrix} \begin{pmatrix} x_1 - \mu_1 \\ x_2 - \mu_2 \\ \vdots \\ x_d - \mu_d \end{pmatrix} \quad (10.19)$$

Since $\mathbf{I} - \mathbf{W}$ is always invertible, we can write

$$\mathbf{x} - \boldsymbol{\mu} = (\mathbf{I} - \mathbf{W})^{-1} \mathbf{e} \triangleq \mathbf{U}\mathbf{e} = \mathbf{U}\mathbf{S}\mathbf{z} \quad (10.20)$$

where we defined $\mathbf{U} = (\mathbf{I} - \mathbf{W})^{-1}$. Thus the regression weights correspond to a Cholesky decomposition of $\boldsymbol{\Sigma}$, as we now show:

$$\boldsymbol{\Sigma} = \text{cov}[\mathbf{x}] = \text{cov}[\mathbf{x} - \boldsymbol{\mu}] \quad (10.21)$$

$$= \text{cov}[\mathbf{U}\mathbf{S}\mathbf{z}] = \mathbf{U}\mathbf{S} \text{cov}[\mathbf{z}] \mathbf{S}\mathbf{U}^T = \mathbf{U}\mathbf{S}^2\mathbf{U}^T \quad (10.22)$$

10.3 Inference

We have seen that graphical models provide a compact way to define joint probability distributions. Given such a joint distribution, what can we do with it? The main use for such a joint distribution is to perform **probabilistic inference**. This refers to the task of estimating unknown quantities from known quantities. For example, in Section 10.2.2, we introduced HMMs, and said that one of the goals is to estimate the hidden states (e.g., words) from the observations (e.g., speech signal). And in Section 10.2.4, we discussed genetic linkage analysis, and said that one of the goals is to estimate the likelihood of the data under various DAGs, corresponding to different hypotheses about the location of the disease-causing gene.

In general, we can pose the inference problem as follows. Suppose we have a set of correlated random variables with joint distribution $p(\mathbf{x}_{1:V}|\boldsymbol{\theta})$. (In this section, we are assuming the parameters $\boldsymbol{\theta}$ of the model are known. We discuss how to learn the parameters in Section 10.4.) Let us partition this vector into the **visible variables** $\mathbf{x}_v$, which are observed, and the **hidden variables**, $\mathbf{x}_h$, which are unobserved. Inference refers to computing the posterior distribution of the unknowns given the knowns:

$$p(\mathbf{x}_h|\mathbf{x}_v, \boldsymbol{\theta}) = \frac{p(\mathbf{x}_h, \mathbf{x}_v|\boldsymbol{\theta})}{p(\mathbf{x}_v|\boldsymbol{\theta})} = \frac{p(\mathbf{x}_h, \mathbf{x}_v|\boldsymbol{\theta})}{\sum_{\mathbf{x}'_h} p(\mathbf{x}'_h, \mathbf{x}_v|\boldsymbol{\theta})} \quad (10.23)$$

Essentially we are **conditioning** on the data by **clamping** the visible variables to their observed values, $\mathbf{x}_v$, and then normalizing, to go from $p(\mathbf{x}_h, \mathbf{x}_v)$ to $p(\mathbf{x}_h|\mathbf{x}_v)$. The normalization constant $p(\mathbf{x}_v|\boldsymbol{\theta})$ is the likelihood of the data, also called the **probability of the evidence**.

Sometimes only some of the hidden variables are of interest to us. So let us partition the hidden variables into **query variables**, $\mathbf{x}_q$, whose value we wish to know, and the remaining **nuisance variables**, $\mathbf{x}_n$, which we are not interested in. We can compute what we are interested in by **marginalizing out** the nuisance variables:

$$p(\mathbf{x}_q|\mathbf{x}_v, \boldsymbol{\theta}) = \sum_{\mathbf{x}_n} p(\mathbf{x}_q, \mathbf{x}_n|\mathbf{x}_v, \boldsymbol{\theta}) \quad (10.24)$$

In Section 4.3.1, we saw how to perform all these operations for a multivariate Gaussian in $O(V^3)$ time, where V is the number of variables. What if we have discrete random variables, with say K states each? If the joint distribution is represented as a multi-dimensional table, we can always perform these operations exactly, but this will take $O(K^V)$ time. In Chapter 20, we explain how to exploit the factorization encoded by the GM to perform these operations in $O(VK^{w+1})$ time, where w is a quantity known as the treewidth of the graph. This measures how “tree-like” the graph is. If the graph is a tree (or a chain), we have $w = 1$, so for these models, inference takes time linear in the number of nodes. Unfortunately, for more general graphs, exact inference can take time exponential in the number of nodes, as we explain in Section 20.5. We will therefore examine various approximate inference schemes later in the book.

10.4 Learning

In the graphical models literature, it is common to distinguish between inference and learning. Inference means computing (functions of) $p(\mathbf{x}_h|\mathbf{x}_v, \boldsymbol{\theta})$, where v are the visible nodes, h are the hidden nodes, and $\boldsymbol{\theta}$ are the parameters of the model, assumed to be known. Learning usually means computing a MAP estimate of the parameters given data:

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \sum_{i=1}^N \log p(\mathbf{x}_{i,v}|\boldsymbol{\theta}) + \log p(\boldsymbol{\theta}) \quad (10.25)$$

where $\mathbf{x}_{i,v}$ are the visible variables in case i . If we have a uniform prior, $p(\boldsymbol{\theta}) \propto 1$, this reduces to the MLE, as usual.

If we adopt a Bayesian view, the parameters are unknown variables and should also be inferred. Thus to a Bayesian, there is no distinction between inference and learning. In fact, we can just add the parameters as nodes to the graph, condition on $\mathcal{D}$, and then infer the values of all the nodes. (We discuss this in more detail below.)

In this view, the main difference between hidden variables and parameters is that the number of hidden variables grows with the amount of training data (since there is usually a set of hidden variables for each observed data case), whereas the number of parameters is usually fixed (at least in a parametric model). This means that we must integrate out the hidden variables to avoid overfitting, but we may be able to get away with point estimation techniques for parameters, which are fewer in number.

10.4.1 Plate notation

When inferring parameters from data, we often assume the data is iid. We can represent this assumption explicitly using a graphical model, as shown in Figure 10.7(a). This illustrates the

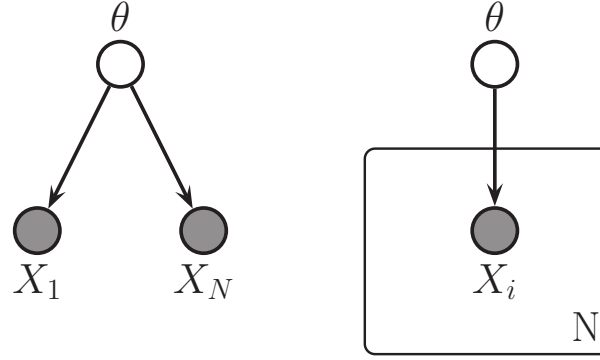


Figure 10.7 Left: data points x_i are conditionally independent given θ . Right: Plate notation. This represents the same model as the one on the left, except the repeated x_i nodes are inside a box, known as a plate; the number in the lower right hand corner, N , specifies the number of repetitions of the X_i node.

assumption that each data case was generated independently but from the same distribution. Notice that the data cases are only independent conditional on the parameters θ ; marginally, the data cases are dependent. Nevertheless, we can see that, in this example, the order in which the data cases arrive makes no difference to our beliefs about θ , since all orderings will have the same sufficient statistics. Hence we say the data is **exchangeable**.

To avoid visual clutter, it is common to use a form of **syntactic sugar** called **plates**: we simply draw a little box around the repeated variables, with the convention that nodes within the box will get repeated when the model is **unrolled**. We often write the number of copies or repetitions in the bottom right corner of the box. See Figure 10.7(b) for a simple example. The corresponding joint distribution has the form

$$p(\theta, \mathcal{D}) = p(\theta) \left[\prod_{i=1}^N p(\mathbf{x}_i | \theta) \right] \quad (10.26)$$

This DGM represents the CI assumptions behind the models we considered in Chapter 5.

A slightly more complex example is shown in Figure 10.8. On the left we show a naive Bayes classifier that has been “unrolled” for D features, but uses a plate to represent repetition over cases $i = 1 : N$. The version on the right shows the same model using **nested plate** notation. When a variable is inside two plates, it will have two sub-indices. For example, we write θ_{jc} to represent the parameter for feature j in class-conditional density c . Note that plates can be nested or crossing. Notational devices for modeling more complex parameter tying patterns can be devised (e.g., (Heckerman et al. 2004)), but these are not widely used. What is not clear from the figure is that θ_{jc} is used to generate x_{ij} iff $y_i = c$, otherwise it is ignored. This is an example of **context specific independence**, since the CI relationship $x_{ij} \perp \theta_{jc}$ only holds if $y_i \neq c$.

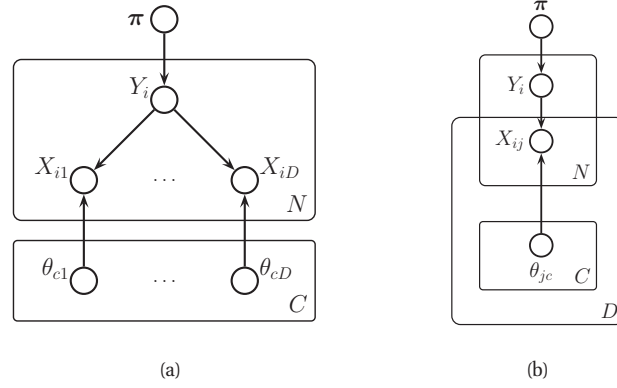


Figure 10.8 Naive Bayes classifier as a DGM. (a) With single plates. (b) With nested plates.

10.4.2 Learning from complete data

If all the variables are fully observed in each case, so there is no missing data and there are no hidden variables, we say the data is **complete**. For a DGM with complete data, the likelihood is given by

$$p(\mathcal{D}|\boldsymbol{\theta}) = \prod_{i=1}^N p(\mathbf{x}_i|\boldsymbol{\theta}) = \prod_{i=1}^N \prod_{t=1}^V p(x_{it}|\mathbf{x}_{i,\text{pa}(t)}, \boldsymbol{\theta}_t) = \prod_{t=1}^V p(\mathcal{D}_t|\boldsymbol{\theta}_t) \quad (10.27)$$

where $\mathcal{D}_t$ is the data associated with node t and its parents, i.e., the t 'th family. This is a product of terms, one per CPD. We say that the likelihood **decomposes** according to the graph structure.

Now suppose that the prior factorizes as well:

$$p(\boldsymbol{\theta}) = \prod_{t=1}^V p(\boldsymbol{\theta}_t) \quad (10.28)$$

Then clearly the posterior also factorizes:

$$p(\boldsymbol{\theta}|\mathcal{D}) \propto p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}) = \prod_{t=1}^V p(\mathcal{D}_t|\boldsymbol{\theta}_t)p(\boldsymbol{\theta}_t) \quad (10.29)$$

This means we can compute the posterior of each CPD independently. In other words,

$$\text{factored prior plus factored likelihood implies factored posterior} \quad (10.30)$$

Let us consider an example, where all CPDs are tabular, thus extending the earlier results of Section 3.5.1.2, where discussed Bayesian naive Bayes. We have a separate row (i.e., a separate multinoulli distribution) for each **conditioning case**, i.e., for each combination of parent values, as in Table 10.2. Formally, we can write the t 'th CPT as $x_t|\mathbf{x}_{\text{pa}(t)} = c \sim \text{Cat}(\boldsymbol{\theta}_{tc})$, where $\theta_{tck} \triangleq p(x_t = k|\mathbf{x}_{\text{pa}(t)} = c)$, for $k = 1 : K_t$, $c = 1 : C_t$ and $t = 1 : T$. Here K_t is the number

of states for node t , $C_t \triangleq \prod_{s \in \text{pa}(t)} K_s$ is the number of parent combinations, and T is the number of nodes. Obviously $\sum_k \theta_{tck} = 1$ for each row of each CPT.

Let us put a separate Dirichlet prior on each row of each CPT, i.e., $\theta_{tc} \sim \text{Dir}(\alpha_{tc})$. Then we can compute the posterior by simply adding the pseudo counts to the empirical counts to get $\theta_{tc} | \mathcal{D} \sim \text{Dir}(\mathbf{N}_{tc} + \alpha_{tc})$, where N_{tck} is the number of times that node t is in state k while its parents are in state c :

$$N_{tck} \triangleq \sum_{i=1}^N \mathbb{I}(x_{i,t} = k, x_{i,\text{pa}(t)} = c) \quad (10.31)$$

From Equation 2.77, the mean of this distribution is given by the following:

$$\bar{\theta}_{tck} = \frac{N_{tck} + \alpha_{tck}}{\sum_{k'} (N_{tck'} + \alpha_{tck'})} \quad (10.32)$$

For example, consider the DGM in Figure 10.1(a). Suppose the training data consists of the following 5 cases:

x_1	x_2	x_3	x_4	x_5
0	0	1	0	0
0	1	1	1	1
1	1	0	1	0
0	1	1	0	0
0	1	1	1	0

Below we list all the sufficient statistics N_{tck} , and the posterior mean parameters $\bar{\theta}_{tck}$ under a Dirichlet prior with $\alpha_{tck} = 1$ (corresponding to add-one smoothing) for the $t = 4$ node:

x_2	x_3	$N_{tck=1}$	$N_{tck=0}$	$\bar{\theta}_{tck=1}$	$\bar{\theta}_{tck=0}$
0	0	0	0	1/2	1/2
1	0	1	0	2/3	1/3
0	1	0	1	1/3	2/3
1	1	2	1	3/5	2/5

It is easy to show that the MLE has the same form as Equation 10.32, except without the α_{tck} terms, i.e.,

$$\hat{\theta}_{tck} = \frac{N_{tck}}{\sum_{k'} N_{tck'}} \quad (10.33)$$

Of course, the MLE suffers from the zero-count problem discussed in Section 3.3.4.1, so it is important to use a prior to regularize the estimation problem.

10.4.3 Learning with missing and/or latent variables

If we have missing data and/or hidden variables, the likelihood no longer factorizes, and indeed it is no longer convex, as we explain in detail in Section 11.3. This means we will usually can only compute a locally optimal ML or MAP estimate. Bayesian inference of the parameters is even harder. We discuss suitable approximate inference techniques in later chapters.

10.5 Conditional independence properties of DGMs

At the heart of any graphical model is a set of conditional independence (CI) assumptions. We write $\mathbf{x}_A \perp_G \mathbf{x}_B | \mathbf{x}_C$ if A is independent of B given C in the graph G , using the semantics to be defined below. Let $I(G)$ be the set of all such CI statements encoded by the graph.

We say that G is an **I-map** (independence map) for p , or that p is **Markov** wrt G , iff $I(G) \subseteq I(p)$, where $I(p)$ is the set of all CI statements that hold for distribution p . In other words, the graph is an I-map if it does not make any assertions of CI that are not true of the distribution. This allows us to use the graph as a safe proxy for p when reasoning about p 's CI properties. This is helpful for designing algorithms that work for large classes of distributions, regardless of their specific numerical parameters θ .

Note that the fully connected graph is an I-map of all distributions, since it makes no CI assertions at all (since it is not missing any edges). We therefore say G is a **minimal I-map** of p if G is an I-map of p , and if there is no $G' \subseteq G$ which is an I-map of p .

It remains to specify how to determine if $\mathbf{x}_A \perp_G \mathbf{x}_B | \mathbf{x}_C$. Deriving these independencies for undirected graphs is easy (see Section 19.2), but the DAG situation is somewhat complicated, because of the need to respect the orientation of the directed edges. We give the details below.

10.5.1 d-separation and the Bayes Ball algorithm (global Markov properties)

First, we introduce some definitions. We say an *undirected path* P is **d-separated** by a set of nodes E (containing the evidence) iff at least one of the following conditions hold:

1. P contains a chain, $s \rightarrow m \rightarrow t$ or $s \leftarrow m \leftarrow t$, where $m \in E$
2. P contains a tent or fork, $s \swarrow^m \searrow t$, where $m \in E$
3. P contains a **collider** or **v-structure**, $s \searrow_m \swarrow t$, where m is not in E and nor is any descendant of m .

Next, we say that a *set of nodes* A is d-separated from a different set of nodes B given a third observed set E iff each undirected path from every node $a \in A$ to every node $b \in B$ is d-separated by E . Finally, we define the CI properties of a DAG as follows:

$$\mathbf{x}_A \perp_G \mathbf{x}_B | \mathbf{x}_E \iff A \text{ is d-separated from } B \text{ given } E \quad (10.34)$$

The **Bayes ball algorithm** (Shachter 1998) is a simple way to see if A is d-separated from B given E , based on the above definition. The idea is this. We “shade” all nodes in E , indicating that they are observed. We then place “balls” at each node in A , let them “bounce around” according to some rules, and then ask if any of the balls reach any of the nodes in B . The three main rules are shown in Figure 10.9. Notice that balls can travel opposite to edge directions. We see that a ball can pass through a chain, but not if it is shaded in the middle. Similarly, a ball can pass through a fork, but not if it is shaded in the middle. However, a ball cannot pass through a v-structure, unless it is shaded in the middle.

We can justify the 3 rules of Bayes ball as follows. First consider a chain structure $X \rightarrow Y \rightarrow Z$, which encodes

$$p(x, y, z) = p(x)p(y|x)p(z|y) \quad (10.35)$$

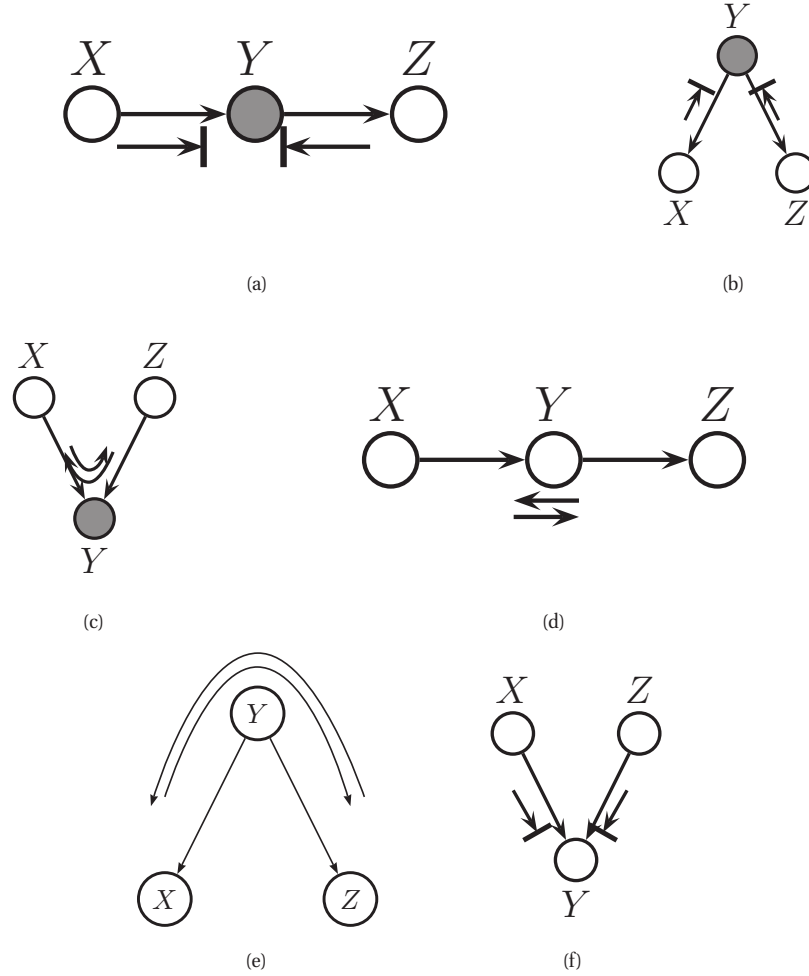


Figure 10.9 Bayes ball rules. A shaded node is one we condition on. If there is an arrow hitting a bar, it means the ball cannot pass through; otherwise the ball can pass through. Based on (Jordan 2007).

When we condition on y , are x and z independent? We have

$$p(x, z|y) = \frac{p(x)p(y|x)p(z|y)}{p(y)} = \frac{p(x, y)p(z|y)}{p(y)} = p(x|y)p(z|y) \quad (10.36)$$

and therefore $x \perp z|y$. So observing the middle node of chain breaks it in two (as in a Markov chain).

Now consider the tent structure $X \leftarrow Y \rightarrow Z$. The joint is

$$p(x, y, z) = p(y)p(x|y)p(z|y) \quad (10.37)$$

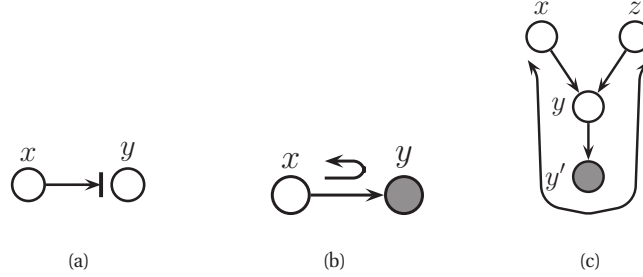


Figure 10.10 (a-b) Bayes ball boundary conditions. (c) Example of why we need boundary conditions. y' is an observed child of y , rendering y “effectively observed”, so the ball bounces back up on its way from x to z .

When we condition on y , are x and z independent? We have

$$p(x, z|y) = \frac{p(x, y, z)}{p(y)} = \frac{p(y)p(x|y)p(z|y)}{p(y)} = p(x|y)p(z|y) \quad (10.38)$$

and therefore $x \perp z|y$. So observing a root node separates its children (as in a naive Bayes classifier: see Section 3.5).

Finally consider a v-structure $X \rightarrow Y \leftarrow Z$. The joint is

$$p(x, y, z) = p(x)p(z)p(y|x, z) \quad (10.39)$$

When we condition on y , are x and z independent? We have

$$p(x, z|y) = \frac{p(x)p(z)p(y|x, z)}{p(y)} \quad (10.40)$$

so $x \not\perp z|y$. However, in the unconditional distribution, we have

$$p(x, z) = p(x)p(z) \quad (10.41)$$

so we see that x and z are marginally independent. So we see that conditioning on a common child at the bottom of a v-structure makes its parents become dependent. This important effect is called **explaining away**, **inter-causal reasoning**, or **Berkson's paradox**. As an example of explaining away, suppose we toss two coins, representing the binary numbers 0 and 1, and we observe the “sum” of their values. A priori, the coins are independent, but once we observe their sum, they become coupled (e.g., if the sum is 1, and the first coin is 0, then we know the second coin is 1).

Finally, Bayes Ball also needs the “boundary conditions” shown in Figure 10.10(a-b). To understand where these rules come from, consider Figure 10.10(c). Suppose Y' is a noise-free copy of Y . Then if we observe Y' , we effectively observe Y as well, so the parents X and Z have to compete to explain this. So if we send a ball down $X \rightarrow Y \rightarrow Y'$, it should “bounce back” up along $Y' \rightarrow Y \rightarrow Z$. However, if Y and all its children are hidden, the ball does not bounce back.

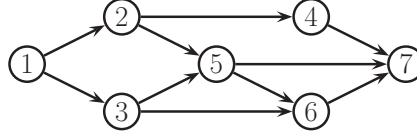


Figure 10.11 A DGM.

For example, in Figure 10.11, we see that $x_2 \perp x_6 | x_5$, since the $2 \rightarrow 5 \rightarrow 6$ path is blocked by x_5 (which is observed), the $2 \rightarrow 4 \rightarrow 7 \rightarrow 6$ path is blocked by x_7 (which is hidden), and the $2 \rightarrow 1 \rightarrow 3 \rightarrow 6$ path is blocked by x_1 (which is hidden). However, we also see that $x_2 \not\perp x_6 | x_5, x_7$, since now the $2 \rightarrow 4 \rightarrow 7 \rightarrow 6$ path is no longer blocked by x_7 (which is observed). Exercise 10.2 gives you some more practice in determining CI relationships for DGMs.

10.5.2 Other Markov properties of DGMs

From the d-separation criterion, one can conclude that

$$t \perp \text{nd}(t) \setminus \text{pa}(t) | \text{pa}(t) \quad (10.42)$$

where the **non-descendants** of a node $\text{nd}(t)$ are all the nodes except for its descendants, $\text{nd}(t) = \mathcal{V} \setminus \{t \cup \text{desc}(t)\}$. Equation 10.42 is called the **directed local Markov property**. For example, in Figure 10.11, we have $\text{nd}(3) = \{2, 4\}$, and $\text{pa}(3) = 1$, so $3 \perp 2, 4 | 1$.

A special case of this property is when we only look at predecessors of a node according to some topological ordering. We have

$$t \perp \text{pred}(t) \setminus \text{pa}(t) | \text{pa}(t) \quad (10.43)$$

which follows since $\text{pred}(t) \subseteq \text{nd}(t)$. This is called the **ordered Markov property**, which justifies Equation 10.7. For example, in Figure 10.11, if we use the ordering $1, 2, \dots, 7$, we find $\text{pred}(3) = \{1, 2\}$ and $\text{pa}(3) = 1$, so $3 \perp 2 | 1$.

We have now described three Markov properties for DAGs: the directed global Markov property G in Equation 10.34, the ordered Markov property O in Equation 10.43, and the directed local Markov property L in Equation 10.42. It is obvious that $G \implies L \implies O$. What is less obvious, but nevertheless true, is that $O \implies L \implies G$ (see e.g., (Koller and Friedman 2009) for the proof). Hence all these properties are equivalent.

Furthermore, any distribution p that is Markov wrt G can be factorized as in Equation 10.7; this is called the factorization property F . It is obvious that $O \implies F$, but one can show that the converse also holds (see e.g., (Koller and Friedman 2009) for the proof).

10.5.3 Markov blanket and full conditionals

The set of nodes that renders a node t conditionally independent of all the other nodes in the graph is called t 's **Markov blanket**; we will denote this by $\text{mb}(t)$. One can show that the Markov blanket of a node in a DGM is equal to the parents, the children, and the **co-parents**,

i.e., other nodes who are also parents of its children:

$$\text{mb}(t) \triangleq \text{ch}(t) \cup \text{pa}(t) \cup \text{copa}(t) \quad (10.44)$$

For example, in Figure 10.11, we have

$$\text{mb}(5) = \{6, 7\} \cup \{2, 3\} \cup \{4\} = \{2, 3, 4, 6, 7\} \quad (10.45)$$

where 4 is a co-parent of 5 because they share a common child, namely 7.

To see why the co-parents are in the Markov blanket, note that when we derive $p(x_t | \mathbf{x}_{-t}) = p(x_t, \mathbf{x}_{-t}) / p(\mathbf{x}_{-t})$, all the terms that do not involve x_t will cancel out between numerator and denominator, so we are left with a product of CPDs which contain x_t in their **scope**. Hence

$$p(x_t | \mathbf{x}_{-t}) \propto p(x_t | \mathbf{x}_{\text{pa}(t)}) \prod_{s \in \text{ch}(t)} p(x_s | \mathbf{x}_{\text{pa}(s)}) \quad (10.46)$$

For example, in Figure 10.11 we have

$$p(x_5 | \mathbf{x}_{-5}) \propto p(x_5 | x_2, x_3) p(x_6 | x_3, x_5) p(x_7 | x_4, x_5, x_6) \quad (10.47)$$

The resulting expression is called t 's **full conditional**, and will prove to be important when we study Gibbs sampling (Section 24.2).

10.6 Influence (decision) diagrams *

We can represent multi-stage (Bayesian) decision problems by using a graphical notation known as a **decision diagram** or an **influence diagram** (Howard and Matheson 1981; Kjaerulff and Madsen 2008). This extends directed graphical models by adding **decision nodes** (also called **action nodes**), represented by rectangles, and **utility nodes** (also called **value nodes**), represented by diamonds. The original random variables are called **chance nodes**, and are represented by ovals, as usual.

Figure 10.12(a) gives a simple example, illustrating the famous **oil wild-catter** problem.³ In this problem, you have to decide whether to drill an oil well or not. You assume there are 3 states of nature: $d = 1$ means drill, $d = 0$ means don't drill. You assume there are 3 states of nature: $o = 0$ means the well is dry, $o = 1$ means it is wet (has some oil), and $o = 2$ means it is soaking (has a lot of oil). Suppose your prior beliefs are $p(o) = [0.5, 0.3, 0.2]$. Finally, you must specify the utility function $U(d, o)$. Since the states and actions are discrete, we can represent it as a table (analogous to a CPT in a DGM). Suppose we use the following numbers, in dollars:

	$o = 0$	$o = 1$	$o = 2$
$d = 0$	0	0	0
$d = 1$	-70	50	200

We see that if you don't drill, you incur no costs, but also make no money. If you drill a dry well, you lose \$70; if you drill a wet well, you gain \$50; and if you drill a soaking well, you gain \$200. Your prior expected utility if you drill is given by

$$EU(d = 1) = \sum_{o=0}^2 p(o) U(d, o) = 0.5 \cdot (-70) + 0.3 \cdot 50 + 0.2 \cdot 200 = 20 \quad (10.48)$$

3. This example is originally from (Raiffa 1968). Our presentation is based on some notes by Daphne Koller.

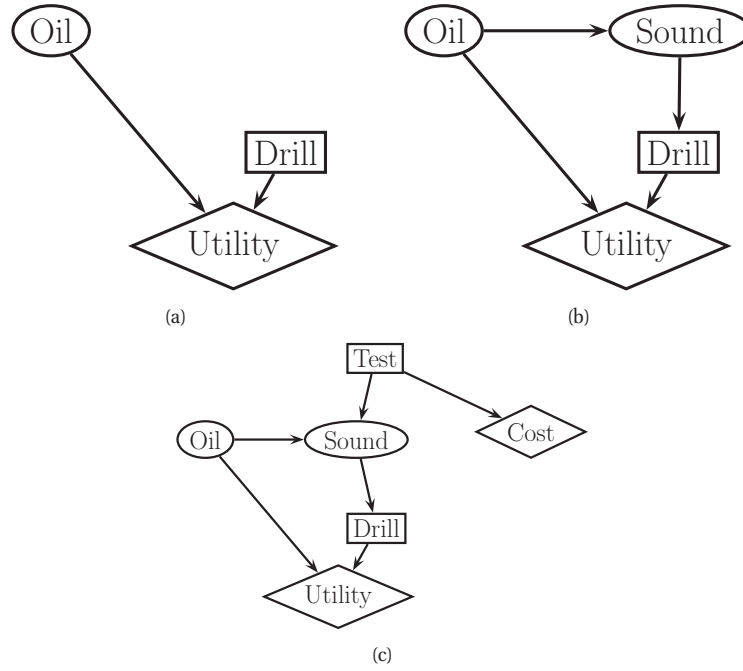


Figure 10.12 (a) Influence diagram for basic oil wild catter problem. (b) An extension in which we have an information arc from the Sound chance node to the Drill decision node. (c) An extension in which we get to decide whether to perform the test or not.

Your expected utility if you don't drill is 0. So your maximum expected utility is

$$MEU = \max\{EU(d = 0), EU(d = 1)\} = \max\{0, 20\} = 20 \quad (10.49)$$

and therefore the optimal action is to drill:

$$d^* = \arg \max\{EU(d = 0), EU(d = 1)\} = 1 \quad (10.50)$$

Now let us consider a slight extension to the model. Suppose you perform a sounding to estimate the state of the well. The sounding observation can be in one of 3 states: $s = 0$ is a diffuse reflection pattern, suggesting no oil; $s = 1$ is an open reflection pattern, suggesting some oil; and $s = 2$ is a closed reflection pattern, indicating lots of oil. Since S is caused by O , we add an $O \rightarrow S$ arc to our model. In addition, we assume that the outcome of the sounding test will be available before we decide whether to drill or not; hence we add an **information arc** from S to D . This is illustrated in Figure 10.12(b).

Let us model the reliability of our sensor using the following conditional distribution for $p(s|o)$:

	$s = 0$	$s = 1$	$s = 2$
$o = 0$	0.6	0.3	0.1
$o = 1$	0.3	0.4	0.3
$o = 2$	0.1	0.4	0.5

Suppose we do the sounding test and we observe $s = 0$. The posterior over the oil state is

$$p(o|s = 0) = [0.732, 0.219, 0.049] \quad (10.51)$$

Now your posterior expected utility of performing action d is

$$EU(d|s = 0) = \sum_{o=0}^2 p(o|s = 0)U(o, d) \quad (10.52)$$

If $d = 1$, this gives

$$EU(d = 1|s = 0) = 0.732 \times (-70) + 0.219 \times 50 + 0.049 \times 200 = -30.5 \quad (10.53)$$

However, if $d = 0$, then $EU(d = 0|s = 0) = 0$, since not drilling incurs no cost. So if we observe $s = 0$, we are better off not drilling, which makes sense.

Now suppose we do the sounding test and we observe $s = 1$. By similar reasoning, one can show that $EU(d = 1|s = 1) = 32.9$, which is higher than $EU(d = 0|s = 1) = 0$. Similarly, if we observe $s = 2$, we have $EU(d = 1|s = 2) = 87.5$ which is much higher than $EU(d = 0|s = 2) = 0$. Hence the optimal policy $d^*(s)$ is as follows: if $s = 0$, choose $d^*(0) = 0$ and get \$0; if $s = 1$, choose $d^*(1) = 1$ and get \$32.9; and if $s = 2$, choose $d^*(2) = 1$ and get \$87.5.

You can compute your **expected profit** or maximum expected utility as follows:

$$MEU = \sum_s p(s)EU(d^*(s)|s) \quad (10.54)$$

This is the expected utility given possible outcomes of the sounding test, assuming you act optimally given the outcome. The prior marginal on the outcome of the test is

$$p(s) = \sum_o p(o)p(s|o) = [0.41, 0.35, 0.24] \quad (10.55)$$

Hence your maximum expected utility is

$$MEU = 0.41 \times 0 + 0.35 \times 32.9 + 0.24 \times 87.5 = 32.2 \quad (10.56)$$

Now suppose you can choose whether to do the test or not. This can be modelled as shown in Figure 10.12(c), where we add a new test node T . If $T = 1$, we do the test, and S can enter 1 of 3 states, determined by O , exactly as above. If $T = 0$, we don't do the test, and S enters a special unknown state. There is also some cost associated with performing the test.

Is it worth doing the test? This depends on how much our MEU changes if we know the outcome of the test (namely the state of S). If you don't do the test, we have $MEU = 20$ from Equation 10.49. If you do the test, you have $MEU = 32.2$ from Equation 10.56. So the improvement in utility if you do the test (and act optimally on its outcome) is \$12.2. This is

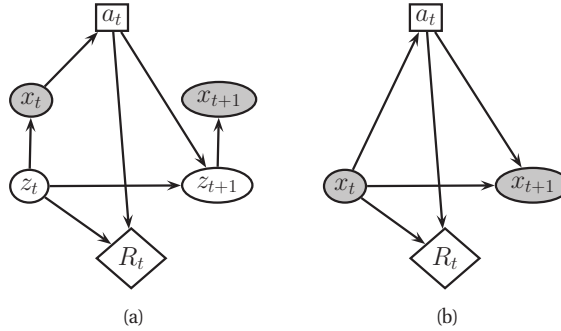


Figure 10.13 (a) A POMDP, shown as an influence diagram. z_t are hidden world states. We implicitly make the **no forgetting** assumption, which effectively means that a_t has arrows coming into it from all previous observations, $x_{1:t}$. (b) An MDP, shown as an influence diagram.

called the **value of perfect information** (VPI). So we should do the test as long as it costs less than \$12.2.

In terms of graphical models, the VPI of a variable T can be determined by computing the MEU for the base influence diagram, I , and then computing the MEU for the same influence diagram where we add information arcs from T to the action nodes, and then computing the difference. In other words,

$$\text{VPI} = \text{MEU}(I + T \rightarrow D) - \text{MEU}(I) \quad (10.57)$$

where D is the decision node and T is the variable we are measuring.

It is possible to modify the variable elimination algorithm (Section 20.3) so that it computes the optimal policy given an influence diagram. These methods essentially work backwards from the final time-step, computing the optimal decision at each step assuming all following actions are chosen optimally. See e.g., (Lauritzen and Nilsson 2001; Kjaerulff and Madsen 2008) for details.

We could continue to extend the model in various ways. For example, we could imagine a dynamical system in which we test, observe outcomes, perform actions, move on to the next oil well, and continue drilling (and polluting) in this way. In fact, many problems in robotics, business, medicine, public policy, etc. can be usefully formulated as influence diagrams unrolled over time (Raiffa 1968; Lauritzen and Nilsson 2001; Kjaerulff and Madsen 2008).

A generic model of this form is shown in Figure 10.13(a). This is known as a **partially observed Markov decision process** or **POMDP** (pronounced “pom-d-p”). This is basically a hidden Markov model (Section 17.3) augmented with action and reward nodes. This can be used to model the **perception-action** cycle that all intelligent agents use (see e.g., (Kaelbling et al. 1998) for details).

A special case of a POMDP, in which the states are fully observed, is called a **Markov decision process** or **MDP**, shown in Figure 10.13(b). This is much easier to solve, since we only have to compute a mapping from observed states to actions. This can be solved using dynamic programming (see e.g., (Sutton and Barto 1998) for details).

In the POMDP case, the information arc from x_t to a_t is not sufficient to uniquely determine

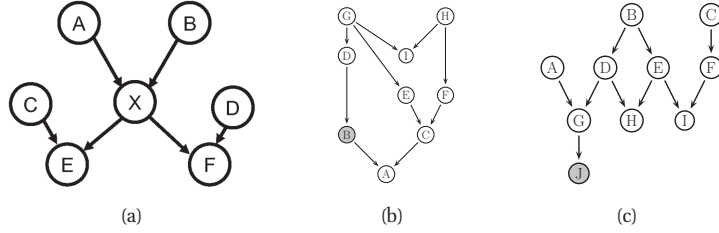


Figure 10.14 Some DGMs.

the best action, since the state is not fully observed. Instead, we need to choose actions based on our **belief state**, $p(z_t | \mathbf{x}_{1:t}, a_{1:t})$. Since the belief updating process is deterministic (see Section 17.4.2), we can compute a **belief state MDP**. For details on to compute the policies for such models, see e.g., (Kaelbling et al. 1998; Spaan and Vlassis 2005).

Exercises

Exercise 10.1 Marginalizing a node in a DGM

(Source: Koller.)

Consider the DAG G in Figure 10.14(a). Assume it is a minimal I-map for $p(A, B, C, D, E, F, X)$. Now consider marginalizing out X . Construct a new DAG G' which is a minimal I-map for $p(A, B, C, D, E, F)$. Specify (and justify) which extra edges need to be added.

Exercise 10.2 Bayes Ball

(Source: Jordan.)

Here we compute some global independence statements from some directed graphical models. You can use the “Bayes ball” algorithm, the d-separation criterion, or the method of converting to an undirected graph (all should give the same results).

- Consider the DAG in Figure 10.14(b). List all variables that are independent of A given evidence on B .
- Consider the DAG in Figure 10.14(c). List all variables that are independent of A given evidence on J .

Exercise 10.3 Markov blanket for a DGM

Prove that the full conditional for node i in a DGM is given by

$$p(X_i | X_{-i}) \propto p(X_i | Pa(X_i)) \prod_{Y_j \in ch(X_i)} p(Y_j | Pa(Y_j)) \quad (10.58)$$

where $ch(X_i)$ are the children of X_i and $Pa(Y_j)$ are the parents of Y_j .

Exercise 10.4 Hidden variables in DGMs

Consider the DGMs in Figure 11.1 which both define $p(X_{1:6})$, where we number empty nodes left to right, top to bottom. The graph on the left defines the joint as

$$p(X_{1:6}) = \sum_h p(X_1)p(X_2)p(X_3)p(H = h | X_{1:3})p(X_4 | H = h)p(X_5 | H = h)p(X_6 | H = h) \quad (10.59)$$

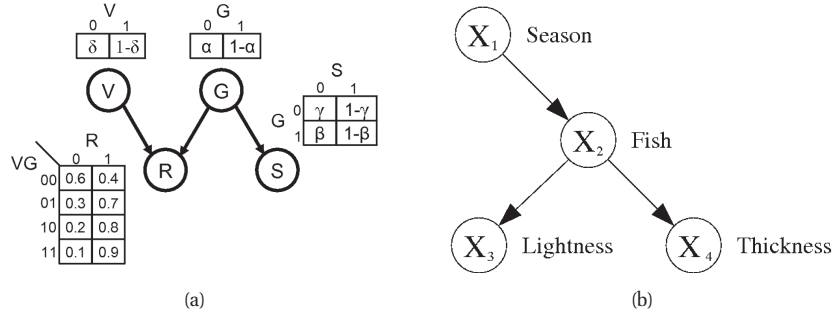


Figure 10.15 (a) Weather BN. (b) Fishing BN.

where we have marginalized over the hidden variable H . The graph on the right defines the joint as

$$p(X_{1:6}) = p(X_1)p(X_2)p(X_3)p(X_4|X_{1:3})p(X_5|X_{1:4})p(X_6|X_{1:5}) \quad (10.60)$$

- (5 points) Assuming all nodes (including H) are binary and all CPDs are tabular, prove that the model on the left has 17 free parameters.
- (5 points) Assuming all nodes are binary and all CPDs are tabular, prove that the model on the right has 59 free parameters.
- (5 points) Suppose we have a data set $\mathcal{D} = X_{1:6}^n$ for $n = 1 : N$, where we observe the X s but not H , and we want to estimate the parameters of the CPDs using maximum likelihood. For which model is this easier? Explain your answer.

Exercise 10.5 Bayes nets for a rainy day

(Source: Nando de Freitas.). In this question you must model a problem with 4 binary variables: G = "gray", V = "Vancouver", R = "rain" and S = "sad". Consider the directed graphical model describing the relationship between these variables shown in Figure 10.15(a).

- Write down an expression for $P(S = 1|V = 1)$ in terms of $\alpha, \beta, \gamma, \delta$.
- Write down an expression for $P(S = 1|V = 0)$. Is this the same or different to $P(S = 1|V = 1)$? Explain why.
- Find maximum likelihood estimates of α, β, γ using the following data set, where each row is a training case. (You may state your answers without proof.)

V	G	R	S
1	1	1	1
1	1	0	1
1	0	0	0

(10.61)

Exercise 10.6 Fishing nets

(Source: (Duda et al. 2001).) Consider the Bayes net shown in Figure 10.15(b). Here, the nodes represent the following variables

$$X_1 \in \{\text{winter, spring, summer, autumn}\}, X_2 \in \{\text{salmon, sea bass}\} \quad (10.62)$$

$$X_3 \in \{\text{light, medium, dark}\}, X_4 \in \{\text{wide, thin}\} \quad (10.63)$$

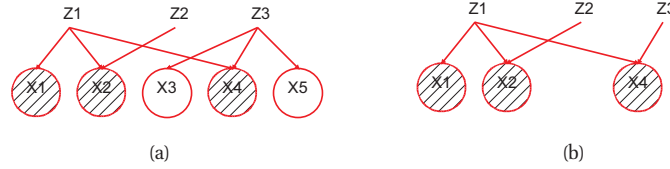


Figure 10.16 (a) A QMR-style network with some hidden leaves. (b) Removing the barren nodes.

The corresponding conditional probability tables are

$$p(x_1) = \begin{pmatrix} .25 & .25 & .25 & .25 \end{pmatrix}, \quad p(x_2|x_1) = \begin{pmatrix} .9 & .1 \\ .3 & .7 \\ .4 & .6 \\ .8 & .2 \end{pmatrix} \quad (10.64)$$

$$p(x_3|x_2) = \begin{pmatrix} .33 & .33 & .34 \\ .8 & .1 & .1 \end{pmatrix}, \quad p(x_4|x_2) = \begin{pmatrix} .4 & .6 \\ .95 & .05 \end{pmatrix} \quad (10.65)$$

Note that in $p(x_4|x_2)$, the rows represent x_2 and the columns x_4 (so each row sums to one and represents the child of the CPD). Thus $p(x_4 = \text{thin}|x_2 = \text{sea bass}) = 0.05$, $p(x_4 = \text{thin}|x_2 = \text{salmon}) = 0.6$, etc.

Answer the following queries. You may use matlab or do it by hand. In either case, show your work.

- Suppose the fish was caught on December 20 — the end of autumn and the beginning of winter — and thus let $p(x_1) = (.5, 0, 0, .5)$ instead of the above prior. (This is called **soft evidence**, since we do not know the exact value of X_1 , but we have a distribution over it.) Suppose the lightness has not been measured but it is known that the fish is thin. Classify the fish as salmon or sea bass.
- Suppose all we know is that the fish is thin and medium lightness. What season is it now, most likely? Use $p(x_1) = (.25, .25, .25, .25)$

Exercise 10.7 Removing leaves in BN20 networks

- Consider the QMR network, where only some of the symptoms are observed. For example, in Figure 10.16(a), X_4 and X_5 are hidden. Show that we can safely remove all the hidden leaf nodes without affecting the posterior over the disease nodes, i.e., prove that we can compute $p(z_{1:3}|x_1, x_2, x_4)$ using the network in Figure 10.16(b). This is called barren node removal, and can be applied to any DGM.
- Now suppose we partition the leaves into three groups: on, off and unknown. Clearly we can remove the unknown leaves, since they are hidden and do not affect their parents. Show that we can analytically remove the leaves that are in the “off state”, by absorbing their effect into the prior of the parents. (This trick only works for noisy-OR CPDs.)

Exercise 10.8 Handling negative findings in the QMR network

Consider the QMR network. Let $\mathbf{d}$ be the hidden diseases, $\mathbf{f}^-$ be the negative findings (leaf nodes that are off), and $\mathbf{f}^+$ be the positive findings (leaf nodes that are on). We can compute the posterior $p(\mathbf{d}|\mathbf{f}^+\mathbf{f}^-)$ in two steps: first absorb the negative findings, $p(\mathbf{d}|\mathbf{f}^-) \propto p(\mathbf{d})p(\mathbf{f}^-|\mathbf{d})$, then absorb the positive findings, $p(\mathbf{d}|\mathbf{f}^+, \mathbf{f}^-) \propto p(\mathbf{d}|\mathbf{f}^-)p(\mathbf{f}^+|\mathbf{d})$. Show that the first step can be done in $O(|\mathbf{d}||\mathbf{f}^-|)$ time, where $|\mathbf{d}|$ is the number of diseases and $|\mathbf{f}^-|$ is the number of negative findings. For simplicity, you can ignore leak nodes. (Intuitively, the reason for this is that there is no correlation induced amongst the parents when the finding is off, since there is no explaining away.)

Exercise 10.9 Moralization does not introduce new independence statements

Recall that the process of moralizing a DAG means connecting together all “unmarried” parents that share a common child, and then dropping all the arrows. Let M be the moralization of DAG G . Show that $CI(M) \subseteq CI(G)$, where CI are the set of conditional independence statements implied by the model.

11

Mixture models and the EM algorithm

11.1 Latent variable models

In Chapter 10 we showed how graphical models can be used to define high-dimensional joint probability distributions. The basic idea is to model dependence between two variables by adding an edge between them in the graph. (Technically the graph represents conditional independence, but you get the point.)

An alternative approach is to assume that the observed variables are correlated because they arise from a hidden common “cause”. Model with hidden variables are also known as **latent variable models** or **LVMs**. As we will see in this chapter, such models are harder to fit than models with no latent variables. However, they can have significant advantages, for two main reasons. First, LVMs often have fewer parameters than models that directly represent correlation in the visible space. This is illustrated in Figure 11.1. If all nodes (including H) are binary and all CPDs are tabular, the model on the left has 17 free parameters, whereas the model on the right has 59 free parameters.

Second, the hidden variables in an LVM can serve as a **bottleneck**, which computes a compressed representation of the data. This forms the basis of unsupervised learning, as we will see. Figure 11.2 illustrates some generic LVM structures that can be used for this purpose. In general there are L latent variables, $z_{i1}, \dots, z_{iL}$, and D visible variables, $x_{i1}, \dots, x_{iD}$, where usually $D \gg L$. If we have $L > 1$, there are many latent factors contributing to each observation, so we have a many-to-many mapping. If $L = 1$, we only have a single latent variable; in this case, z_i is usually discrete, and we have a one-to-many mapping. We can also have a many-to-one mapping, representing different competing factors or causes for each observed variable; such models form the basis of probabilistic matrix factorization, discussed in Section 27.6.2. Finally, we can have a one-to-one mapping, which can be represented as $\mathbf{z}_i \rightarrow \mathbf{x}_i$. By allowing $\mathbf{z}_i$ and/or $\mathbf{x}_i$ to be vector-valued, this representation can subsume all the others. Depending on the form of the likelihood $p(\mathbf{x}_i|\mathbf{z}_i)$ and the prior $p(\mathbf{z}_i)$, we can generate a variety of different models, as summarized in Table 11.1.

11.2 Mixture models

The simplest form of LVM is when $z_i \in \{1, \dots, K\}$, representing a discrete latent state. We will use a discrete prior for this, $p(z_i) = \text{Cat}(\boldsymbol{\pi})$. For the likelihood, we use $p(\mathbf{x}_i|z_i = k) = p_k(\mathbf{x}_i)$,

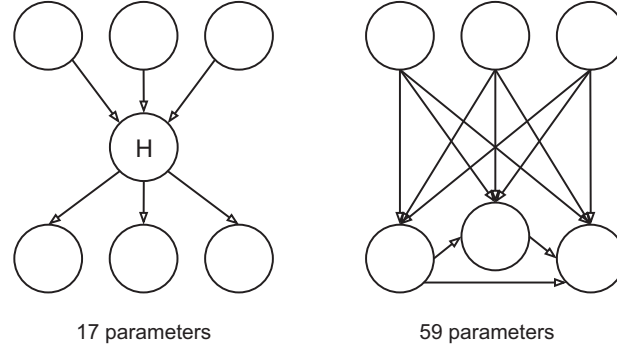


Figure 11.1 A DGM with and without hidden variables. The leaves represent medical symptoms. The roots represent primary causes, such as smoking, diet and exercise. The hidden variable can represent mediating factors, such as heart disease, which might not be directly visible.

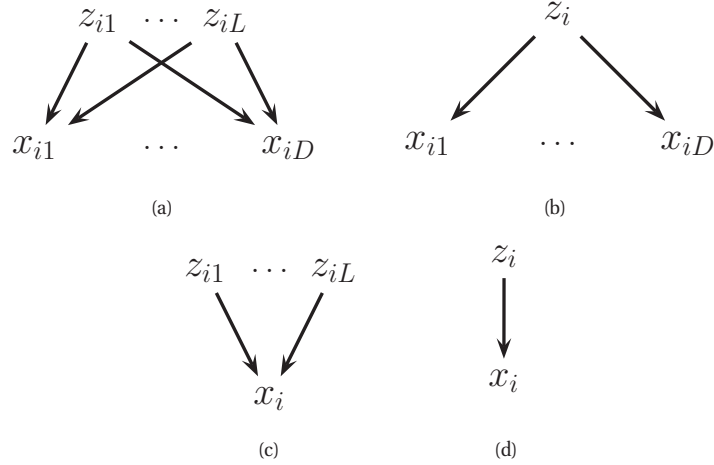


Figure 11.2 A latent variable model represented as a DGM. (a) Many-to-many. (b) One-to-many. (c) Many-to-one. (d) One-to-one.

where p_k is the k 'th **base distribution** for the observations; this can be of any type. The overall model is known as a **mixture model**, since we are mixing together the K base distributions as follows:

$$p(\mathbf{x}_i|\boldsymbol{\theta}) = \sum_{k=1}^K \pi_k p_k(\mathbf{x}_i|\boldsymbol{\theta}) \quad (11.1)$$

This is a **convex combination** of the p_k 's, since we are taking a weighted sum, where the **mixing weights** π_k satisfy $0 \leq \pi_k \leq 1$ and $\sum_{k=1}^K \pi_k = 1$. We give some examples below.

$p(\mathbf{x}_i \mathbf{z}_i)$	$p(\mathbf{z}_i)$	Name	Section
MVN	Discrete	Mixture of Gaussians	11.2.1
Prod. Discrete	Discrete	Mixture of multinomials	11.2.2
Prod. Gaussian	Prod. Gaussian	Factor analysis/ probabilistic PCA	12.1.5
Prod. Gaussian	Prod. Laplace	Probabilistic ICA/ sparse coding	12.6
Prod. Discrete	Prod. Gaussian	Multinomial PCA	27.2.3
Prod. Discrete	Dirichlet	Latent Dirichlet allocation	27.3
Prod. Noisy-OR	Prod. Bernoulli	BN20/ QMR	10.2.3
Prod. Bernoulli	Prod. Bernoulli	Sigmoid belief net	27.7

Table 11.1 Summary of some popular directed latent variable models. Here “Prod” means product, so “Prod. Discrete” in the likelihood means a factored distribution of the form $\prod_j \text{Cat}(x_{ij}|\mathbf{z}_i)$, and “Prod. Gaussian” means a factored distribution of the form $\prod_j \mathcal{N}(x_{ij}|\mathbf{z}_i)$. “PCA” stands for “principal components analysis”. “ICA” stands for “independent components analysis”.

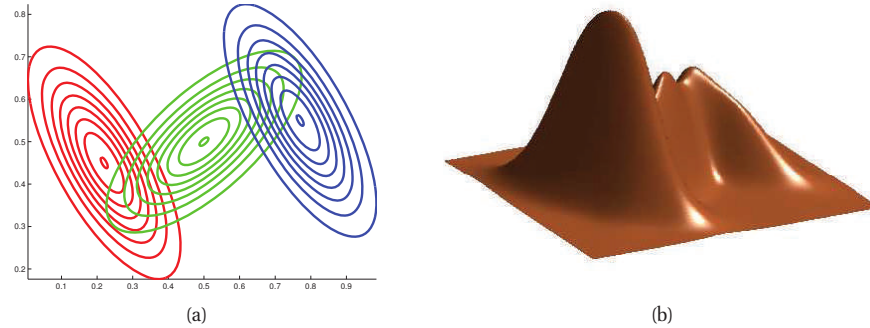


Figure 11.3 A mixture of 3 Gaussians in 2d. (a) We show the contours of constant probability for each component in the mixture. (b) A surface plot of the overall density. Based on Figure 2.23 of (Bishop 2006a). Figure generated by `mixGaussPlotDemo`.

11.2.1 Mixtures of Gaussians

The most widely used mixture model is the **mixture of Gaussians** (MOG), also called a **Gaussian mixture model** or **GMM**. In this model, each base distribution in the mixture is a multivariate Gaussian with mean $\boldsymbol{\mu}_k$ and covariance matrix $\boldsymbol{\Sigma}_k$. Thus the model has the form

$$p(\mathbf{x}_i|\boldsymbol{\theta}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_i|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \quad (11.2)$$

Figure 11.3 shows a mixture of 3 Gaussians in 2D. Each mixture component is represented by a different set of elliptical contours. Given a sufficiently large number of mixture components, a GMM can be used to approximate any density defined on $\mathbb{R}^D$.

11.2.2 Mixture of multinoullis

We can use mixture models to define density models on many kinds of data. For example, suppose our data consist of D -dimensional bit vectors. In this case, an appropriate class-conditional density is a product of Bernoullis:

$$p(\mathbf{x}_i | z_i = k, \boldsymbol{\theta}) = \prod_{j=1}^D \text{Ber}(x_{ij} | \mu_{jk}) = \prod_{j=1}^D \mu_{jk}^{x_{ij}} (1 - \mu_{jk})^{1-x_{ij}} \quad (11.3)$$

where μ_{jk} is the probability that bit j turns on in cluster k .

The latent variables do not have to any meaning, we might simply introduce latent variables in order to make the model more powerful. For example, one can show (Exercise 11.8) that the mean and covariance of the mixture distribution are given by

$$\mathbb{E}[\mathbf{x}] = \sum_k \pi_k \boldsymbol{\mu}_k \quad (11.4)$$

$$\text{cov}[\mathbf{x}] = \sum_k \pi_k [\boldsymbol{\Sigma}_k + \boldsymbol{\mu}_k \boldsymbol{\mu}_k^T] - \mathbb{E}[\mathbf{x}] \mathbb{E}[\mathbf{x}]^T \quad (11.5)$$

where $\boldsymbol{\Sigma}_k = \text{diag}(\mu_{jk}(1 - \mu_{jk}))$. So although the component distributions are factorized, the joint distribution is not. Thus the mixture distribution can capture correlations between variables, unlike a single product-of-Bernoullis model.

11.2.3 Using mixture models for clustering

There are two main applications of mixture models. The first is to use them as a **black-box** density model, $p(\mathbf{x}_i)$. This can be useful for a variety of tasks, such as data compression, outlier detection, and creating generative classifiers, where we model each class-conditional density $p(\mathbf{x} | y = c)$ by a mixture distribution (see Section 14.7.3).

The second, and more common, application of mixture models is to use them for clustering. We discuss this topic in detail in Chapter 25, but the basic idea is simple. We first fit the mixture model, and then compute $p(z_i = k | \mathbf{x}_i, \boldsymbol{\theta})$, which represents the posterior probability that point i belongs to cluster k . This is known as the **responsibility** of cluster k for point i , and can be computed using Bayes rule as follows:

$$r_{ik} \triangleq p(z_i = k | \mathbf{x}_i, \boldsymbol{\theta}) = \frac{p(z_i = k | \boldsymbol{\theta}) p(\mathbf{x}_i | z_i = k, \boldsymbol{\theta})}{\sum_{k'=1}^K p(z_i = k' | \boldsymbol{\theta}) p(\mathbf{x}_i | z_i = k', \boldsymbol{\theta})} \quad (11.6)$$

This procedure is called **soft clustering**, and is identical to the computations performed when using a generative classifier. The difference between the two models only arises at training time: in the mixture case, we never observe z_i , whereas with a generative classifier, we do observe y_i (which plays the role of z_i).

We can represent the amount of uncertainty in the cluster assignment by using $1 - \max_k r_{ik}$. Assuming this is small, it may be reasonable to compute a **hard clustering** using the MAP estimate, given by

$$z_i^* = \arg \max_k r_{ik} = \arg \max_k \log p(\mathbf{x}_i | z_i = k, \boldsymbol{\theta}) + \log p(z_i = k | \boldsymbol{\theta}) \quad (11.7)$$

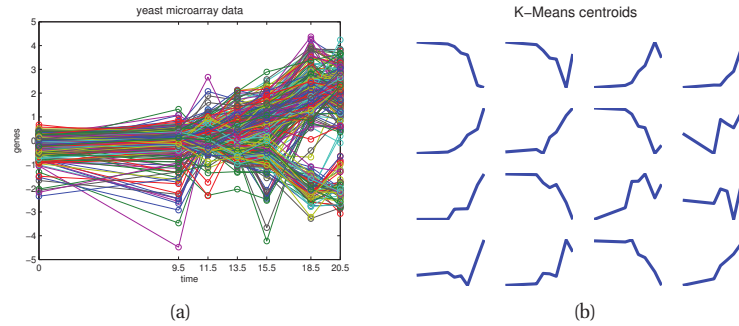


Figure 11.4 (a) Some yeast gene expression data plotted as a time series. (c) Visualizing the 16 cluster centers produced by K-means. Figure generated by `kmeansYeastDemo`.

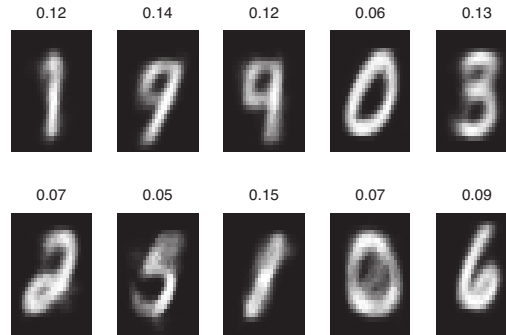


Figure 11.5 We fit a mixture of 10 Bernoullis to the binarized MNIST digit data. We show the MLE for the corresponding cluster means, μ_k . The numbers on top of each image represent the mixing weights $\hat{\pi}_k$. No labels were used when training the model. Figure generated by `mixBerMnistEM`.

Hard clustering using a GMM is illustrated in Figure 1.8, where we cluster some data representing the height and weight of people. The colors represent the hard assignments. Note that the identity of the labels (colors) used is immaterial; we are free to rename all the clusters, without affecting the partitioning of the data; this is called **label switching**.

Another example is shown in Figure 11.4. Here the data vectors $\mathbf{x}_i \in \mathbb{R}^7$ represent the expression levels of different genes at 7 different time points. We clustered them using a GMM. We see that there are several kinds of genes, such as those whose expression level goes up monotonically over time (in response to a given stimulus), those whose expression level goes down monotonically, and those with more complex response patterns. We have clustered the series into $K = 16$ groups. (See Section 11.5 for details on how to choose K .) For example, we can represent each cluster by a **prototype** or **centroid**. This is shown in Figure 11.4(b).

As an example of clustering binary data, consider a binarized version of the MNIST handwritten digit dataset (see Figure 1.5(a)), where we ignore the class labels. We can fit a mixture of

Bernoullis to this, using $K = 10$, and then visualize the resulting centroids, $\hat{\mu}_k$, as shown in Figure 11.5. We see that the method correctly discovered some of the digit classes, but overall the results aren't great: it has created multiple clusters for some digits, and no clusters for others. There are several possible reasons for these "errors":

- The model is very simple and does not capture the relevant visual characteristics of a digit. For example, each pixel is treated independently, and there is no notion of shape or a stroke.
- Although we think there should be 10 clusters, some of the digits actually exhibit a fair degree of visual variety. For example, there are two ways of writing 7's (with and without the cross bar). Figure 1.5(a) illustrates some of the range in writing styles. Thus we need $K \gg 10$ clusters to adequately model this data. However, if we set K to be large, there is nothing in the model or algorithm preventing the extra clusters from being used to create multiple versions of the same digit, and indeed this is what happens. We can use model selection to prevent too many clusters from being chosen but what looks visually appealing and what makes a good density estimator may be quite different.
- The likelihood function is not convex, so we may be stuck in a local optimum, as we explain in Section 11.3.2.

This example is typical of mixture modeling, and goes to show one must be very cautious trying to "interpret" any clusters that are discovered by the method. (Adding a little bit of supervision, or using informative priors, can help a lot.)

11.2.4 Mixtures of experts

Section 14.7.3 described how to use mixture models in the context of generative classifiers. We can also use them to create discriminative models for classification and regression. For example, consider the data in Figure 11.6(a). It seems like a good model would be three different linear regression functions, each applying to a different part of the input space. We can model this by allowing the mixing weights and the mixture densities to be input-dependent:

$$p(y_i | \mathbf{x}_i, z_i = k, \boldsymbol{\theta}) = \mathcal{N}(y_i | \mathbf{w}_k^T \mathbf{x}_i, \sigma_k^2) \quad (11.8)$$

$$p(z_i | \mathbf{x}_i, \boldsymbol{\theta}) = \text{Cat}(z_i | \mathcal{S}(\mathbf{V}^T \mathbf{x}_i)) \quad (11.9)$$

See Figure 11.7(a) for the DGM.

This model is called a **mixture of experts** or MoE (Jordan and Jacobs 1994). The idea is that each submodel is considered to be an "expert" in a certain region of input space. The function $p(z_i = k | \mathbf{x}_i, \boldsymbol{\theta})$ is called a **gating function**, and decides which expert to use, depending on the input values. For example, Figure 11.6(b) shows how the three experts have "carved up" the 1d input space, Figure 11.6(a) shows the predictions of each expert individually (in this case, the experts are just linear regression models), and Figure 11.6(c) shows the overall prediction of the model, obtained using

$$p(y_i | \mathbf{x}_i, \boldsymbol{\theta}) = \sum_k p(z_i = k | \mathbf{x}_i, \boldsymbol{\theta}) p(y_i | \mathbf{x}_i, z_i = k, \boldsymbol{\theta}) \quad (11.10)$$

We discuss how to fit this model in Section 11.4.3.

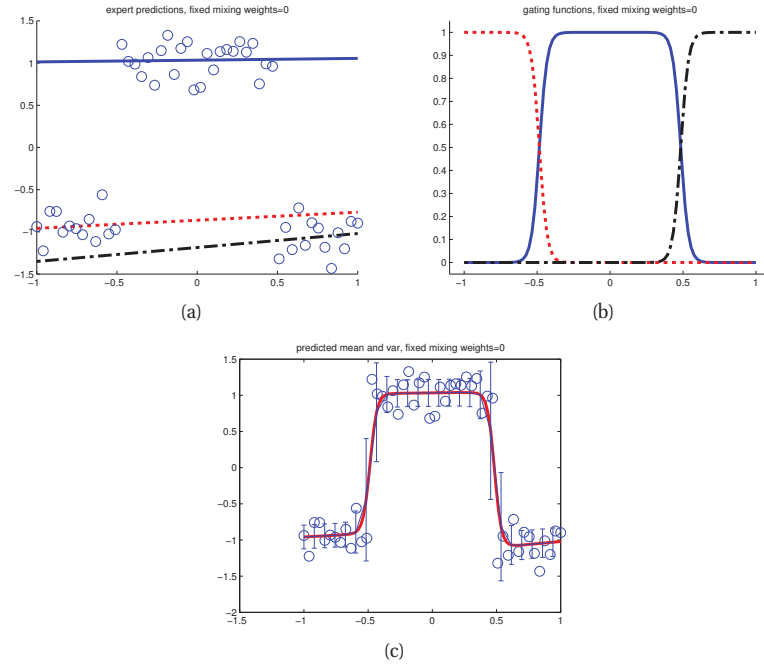


Figure 11.6 (a) Some data fit with three separate regression lines. (b) Gating functions for three different “experts”. (c) The conditionally weighted average of the three expert predictions. Figure generated by `mixexpDemo`.

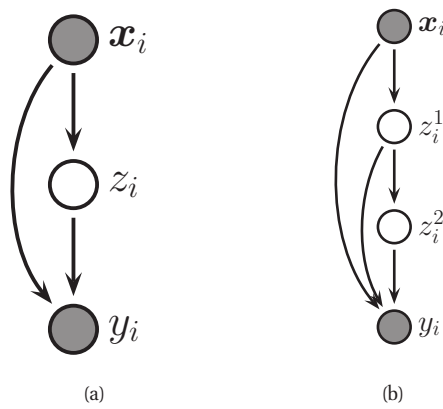


Figure 11.7 (a) A mixture of experts. (b) A hierarchical mixture of experts.

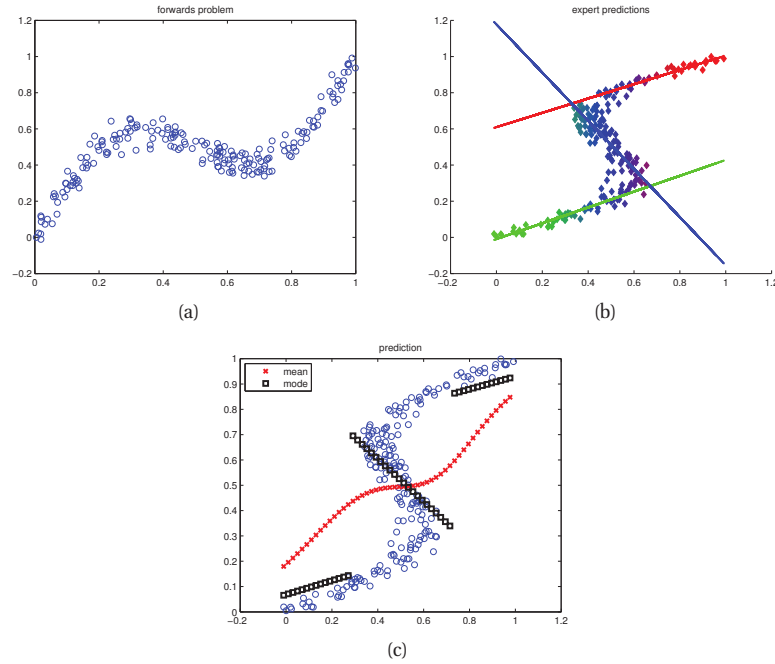


Figure 11.8 (a) Some data from a simple forwards model. (b) Some data from the inverse model, fit with a mixture of 3 linear regressions. Training points are color coded by their responsibilities. (c) The predictive mean (red cross) and mode (black square). Based on Figures 5.20 and 5.21 of (Bishop 2006b). Figure generated by `mixexpDemoOneToMany`.

It should be clear that we can “plug in” any model for the expert. For example, we can use neural networks (Chapter 16) to represent both the gating functions and the experts. The result is known as a **mixture density network**. Such models are slower to train, but can be more flexible than mixtures of experts. See (Bishop 1994) for details.

It is also possible to make each expert be itself a mixture of experts. This gives rise to a model known as the **hierarchical mixture of experts**. See Figure 11.7(b) for the DGM, and Section 16.2.6 for further details.

11.2.4.1 Application to inverse problems

Mixtures of experts are useful in solving **inverse problems**. These are problems where we have to invert a many-to-one mapping. A typical example is in robotics, where the location of the end effector (hand) $\mathbf{y}$ is uniquely determined by the joint angles of the motors, $\mathbf{x}$. However, for any given location $\mathbf{y}$, there are many settings of the joints $\mathbf{x}$ that can produce it. Thus the inverse mapping $\mathbf{x} = f^{-1}(\mathbf{y})$ is not unique. Another example is **kinematic tracking** of people from video (Bo et al. 2008), where the mapping from image appearance to pose is not unique, due to self occlusion, etc.

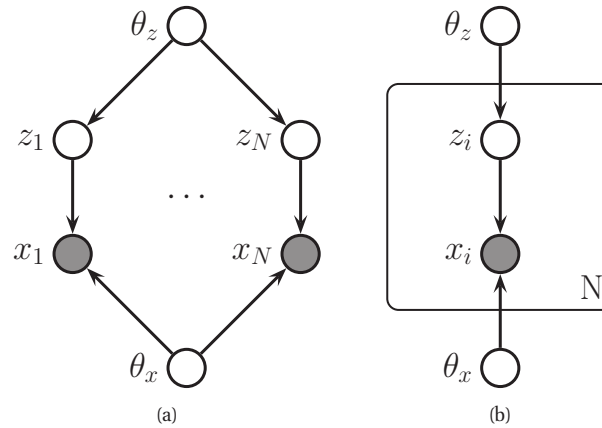


Figure 11.9 A LVM represented as a DGM. Left: Model is unrolled for N examples. Right: same model using plate notation.

A simpler example, for illustration purposes, is shown in Figure 11.8(a). We see that this defines a function, $y = f(x)$, since for every value x along the horizontal axis, there is a unique response y . This is sometimes called the **forwards model**. Now consider the problem of computing $x = f^{-1}(y)$. The corresponding inverse model is shown in Figure 11.8(b); this is obtained by simply interchanging the x and y axes. Now we see that for some values along the horizontal axis, there are multiple possible outputs, so the inverse is not uniquely defined. For example, if $y = 0.8$, then x could be 0.2 or 0.8. Consequently, the predictive distribution, $p(x|y, \theta)$ is multimodal.

We can fit a mixture of linear experts to this data. Figure 11.8(b) shows the prediction of each expert, and Figure 11.8(c) shows (a plugin approximation to) the posterior predictive mode and mean. Note that the posterior mean does not yield good predictions. In fact, any model which is trained to minimize mean squared error — even if the model is a flexible nonlinear model, such as neural network — will work poorly on inverse problems such as this. However, the posterior mode, where the mode is input dependent, provides a reasonable approximation.

11.3 Parameter estimation for mixture models

We have seen how to compute the posterior over the hidden variables given the observed variables, assuming the parameters are known. In this section, we discuss how to learn the parameters.

In Section 10.4.2, we showed that when we have complete data and a factored prior, the posterior over the parameters also factorizes, making computation very simple. Unfortunately this is no longer true if we have hidden variables and/or missing data. The reason is apparent from looking at Figure 11.9. If the z_i were observed, then by d-separation, we see that $\theta_z \perp \theta_x | \mathcal{D}$, and hence the posterior will factorize. But since, in an LVM, the z_i are hidden, the parameters are no longer independent, and the posterior does not factorize, making it much harder to

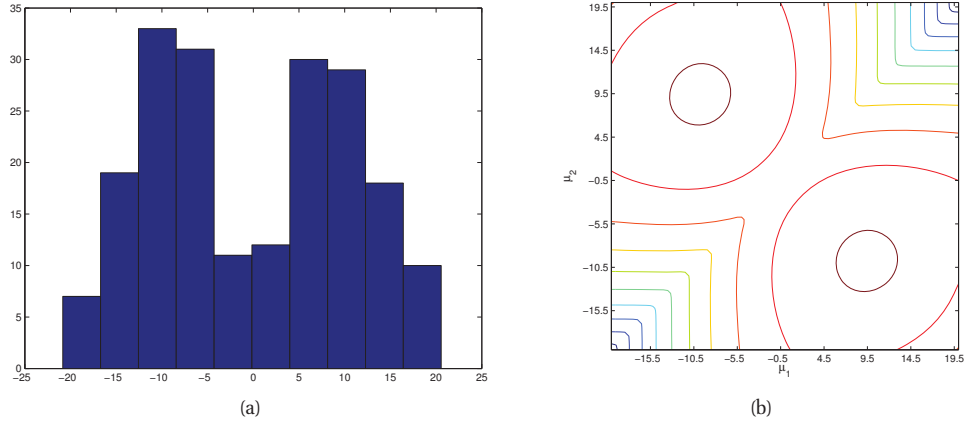


Figure 11.10 Left: $N = 200$ data points sampled from a mixture of 2 Gaussians in 1d, with $\pi_k = 0.5$, $\sigma_k = 5$, $\mu_1 = -10$ and $\mu_2 = 10$. Right: Likelihood surface $p(\mathcal{D}|\mu_1, \mu_2)$, with all other parameters set to their true values. We see the two symmetric modes, reflecting the unidentifiability of the parameters. Figure generated by `mixGaussLikSurfaceDemo`.

compute. This also complicates the computation of MAP and ML estimates, as we discuss below.

11.3.1 Unidentifiability

The main problem with computing $p(\theta|\mathcal{D})$ for an LVM is that the posterior may have multiple modes. To see why, consider a GMM. If the z_i were all observed, we would have a unimodal posterior for the parameters:

$$p(\theta|\mathcal{D}) = \text{Dir}(\pi|\mathcal{D}) \prod_{k=1}^K \text{NIW}(\mu_k, \Sigma_k|\mathcal{D}) \quad (11.11)$$

Consequently we can easily find the globally optimal MAP estimate (and hence globally optimal MLE).

But now suppose the z_i 's are hidden. In this case, for each of the possible ways of “filling in” the z_i 's, we get a different unimodal likelihood. Thus when we marginalize out over the z_i 's, we get a multi-modal posterior for $p(\theta|\mathcal{D})$.¹ These modes correspond to different labelings of the clusters. This is illustrated in Figure 11.10(b), where we plot the likelihood function, $p(\mathcal{D}|\mu_1, \mu_2)$, for a 2D GMM with $K = 2$ for the data is shown in Figure 11.10(a). We see two peaks, one corresponding to the case where $\mu_1 = -10$, $\mu_2 = 10$, and the other to the case where $\mu_1 = 10$, $\mu_2 = -10$. We say the parameters are not **identifiable**, since there is not a unique MLE. Therefore there cannot be a unique MAP estimate (assuming the prior does not rule out certain labelings), and hence the posterior must be multimodal. The question of how many modes there

1. Do not confuse multimodality of the parameter posterior, $p(\theta|\mathcal{D})$, with the multimodality defined by the model, $p(\mathbf{x}|\theta)$. In the latter case, if we have K clusters, we would expect to only get K peaks, although it is theoretically possible to get more than K , at least if $D > 1$ (Carreira-Perpinan and Williams 2003).

are in the parameter posterior is hard to answer. There are $K!$ possible labelings, but some of the peaks might get merged. Nevertheless, there can be an exponential number, since finding the optimal MLE for a GMM is NP-hard (Aloise et al. 2009; Drineas et al. 2004).

Unidentifiability can cause a problem for Bayesian inference. For example, suppose we draw some samples from the posterior, $\theta^{(s)} \sim p(\theta|\mathcal{D})$, and then average them, to try to approximate the posterior mean, $\bar{\theta} = \frac{1}{S} \sum_{s=1}^S \theta^{(s)}$. (This kind of Monte Carlo approach is explained in more detail in Chapter 24.) If the samples come from different modes, the average will be meaningless. Note, however, that it is reasonable to average the posterior predictive distributions, $p(\mathbf{x}) \approx \frac{1}{S} \sum_{s=1}^S p(\mathbf{x}|\theta^{(s)})$, since the likelihood function is invariant to which mode the parameters came from.

A variety of solutions have been proposed to the unidentifiability problem. These solutions depend on the details of the model and the inference algorithm that is used. For example, see (Stephens 2000) for an approach to handling unidentifiability in mixture models using MCMC.

The approach we will adopt in this chapter is much simpler: we just compute a single local mode, i.e., we perform approximate MAP estimation. (We say “approximate” since finding the globally optimal MLE, and hence MAP estimate, is NP-hard, at least for mixture models (Aloise et al. 2009).) This is by far the most common approach, because of its simplicity. It is also a reasonable approximation, at least if the sample size is sufficiently large. To see why, consider Figure 11.9(a). We see that there are N latent variables, each of which gets to “see” one data point each. However, there are only two latent parameters, each of which gets to see N data points. So the posterior uncertainty about the parameters is typically much less than the posterior uncertainty about the latent variables. This justifies the common strategy of computing $p(z_i|\mathbf{x}_i, \hat{\theta})$, but not bothering to compute $p(\theta|\mathcal{D})$. In Section 5.6, we will study hierarchical Bayesian models, which essentially put structure on top of the parameters. In such models, it is important to model $p(\theta|\mathcal{D})$, so that the parameters can send information between themselves. If we used a point estimate, this would not be possible.

11.3.2 Computing a MAP estimate is non-convex

In the previous sections, we have argued, rather heuristically, that the likelihood function has multiple modes, and hence that finding an MAP or ML estimate will be hard. In this section, we show this result by more algebraic means, which sheds some additional insight into the problem. Our presentation is based in part on (Rennie 2004).

Consider the log-likelihood for an LVM:

$$\log p(\mathcal{D}|\theta) = \sum_i \log \left[\sum_{\mathbf{z}_i} p(\mathbf{x}_i, \mathbf{z}_i|\theta) \right] \quad (11.12)$$

Unfortunately, this objective is hard to maximize. since we cannot push the log inside the sum. This precludes certain algebraic simplifications, but does not prove the problem is hard.

Now suppose the joint probability distribution $p(\mathbf{z}_i, \mathbf{x}_i|\theta)$ is in the exponential family, which means it can be written as follows:

$$p(\mathbf{x}, \mathbf{z}|\theta) = \frac{1}{Z(\theta)} \exp[\theta^T \phi(\mathbf{x}, \mathbf{z})] \quad (11.13)$$

where $\phi(\mathbf{x}, \mathbf{z})$ are the sufficient statistics, and $Z(\boldsymbol{\theta})$ is the normalization constant (see Section 9.2 for more details). It can be shown (Exercise 9.2) that the MVN is in the exponential family, as are nearly all of the distributions we have encountered so far, including Dirichlet, multinomial, Gamma, Wishart, etc. (The Student distribution is a notable exception.) Furthermore, mixtures of exponential families are also in the exponential family, providing the mixing indicator variables are observed (Exercise 11.11).

With this assumption, the **complete data log likelihood** can be written as follows:

$$\ell_c(\boldsymbol{\theta}) = \sum_i \log p(\mathbf{x}_i, \mathbf{z}_i | \boldsymbol{\theta}) = \boldsymbol{\theta}^T \left(\sum_i \phi(\mathbf{x}_i, \mathbf{z}_i) \right) - NZ(\boldsymbol{\theta}) \quad (11.14)$$

The first term is clearly linear in $\boldsymbol{\theta}$. One can show that $Z(\boldsymbol{\theta})$ is a convex function (Boyd and Vandenberghe 2004), so the overall objective is concave (due to the minus sign), and hence has a unique maximum.

Now consider what happens when we have missing data. The **observed data log likelihood** is given by

$$\ell(\boldsymbol{\theta}) = \sum_i \log \sum_{\mathbf{z}_i} p(\mathbf{x}_i, \mathbf{z}_i | \boldsymbol{\theta}) = \sum_i \log \left[\sum_{\mathbf{z}_i} e^{\boldsymbol{\theta}^T \phi(\mathbf{z}_i, \mathbf{x}_i)} \right] - N \log Z(\boldsymbol{\theta}) \quad (11.15)$$

One can show that the log-sum-exp function is convex (Boyd and Vandenberghe 2004), and we know that $Z(\boldsymbol{\theta})$ is convex. However, the difference of two convex functions is not, in general, convex. So the objective is neither convex nor concave, and has local optima.

The disadvantage of non-convex functions is that it is usually hard to find their global optimum. Most optimization algorithms will only find a local optimum; which one they find depends on where they start. There are some algorithms, such as simulated annealing (Section 24.6.1) or genetic algorithms, that claim to always find the global optimum, but this is only under unrealistic assumptions (e.g., if they are allowed to be cooled “infinitely slowly”, or allowed to run “infinitely long”). In practice, we will run a local optimizer, perhaps using **multiple random restarts** to increase our chance of finding a “good” local optimum. Of course, careful initialization can help a lot, too. We give examples of how to do this on a case-by-case basis.

Note that a convex method for fitting mixtures of Gaussians has been proposed. The idea is to assign one cluster per data point, and select from amongst them, using a convex ℓ_1 -type penalty, rather than trying to optimize the locations of the cluster centers. See (Lashkari and Golland 2007) for details. This is essentially an unsupervised version of the approach used in sparse kernel logistic regression, which we will discuss in Section 14.3.2. Note, however, that the ℓ_1 penalty, although convex, is not necessarily a good way to promote sparsity, as discussed in Chapter 13. In fact, as we will see in that Chapter, some of the best sparsity-promoting methods use non-convex penalties, and use EM to optimize them! The moral of the story is: do not be afraid of non-convexity.

11.4 The EM algorithm

For many models in machine learning and statistics, computing the ML or MAP parameter estimate is easy provided we observe all the values of all the relevant random variables, i.e., if

Model	Section
Mix. Gaussians	11.4.2
Mix. experts	11.4.3
Factor analysis	12.1.5
Student T	11.4.5
Probit regression	11.4.6
DGM with hidden variables	11.4.4
MVN with missing data	11.6.1
HMMs	17.5.2
Shrinkage estimates of Gaussian means	Exercise 11.13

Table 11.2 Some models discussed in this book for which EM can be easily applied to find the ML/ MAP parameter estimate.

we have complete data. However, if we have missing data and/or latent variables, then computing the ML/MAP estimate becomes hard.

One approach is to use a generic gradient-based optimizer to find a local minimum of the **negative log likelihood** or **NLL**, given by

$$\text{NLL}(\boldsymbol{\theta}) = - \triangleq \frac{1}{N} \log p(\mathcal{D}|\boldsymbol{\theta}) \quad (11.16)$$

However, we often have to enforce constraints, such as the fact that covariance matrices must be positive definite, mixing weights must sum to one, etc., which can be tricky (see Exercise 11.5). In such cases, it is often much simpler (but not always faster) to use an algorithm called **expectation maximization**, or **EM** for short (Dempster et al. 1977; Meng and van Dyk 1997; McLachlan and Krishnan 1997). This is a simple iterative algorithm, often with closed-form updates at each step. Furthermore, the algorithm automatically enforces the required constraints.

EM exploits the fact that if the data were fully observed, then the ML/ MAP estimate would be easy to compute. In particular, EM is an iterative algorithm which alternates between inferring the missing values given the parameters (E step), and then optimizing the parameters given the “filled in” data (M step). We give the details below, followed by several examples. We end with a more theoretical discussion, where we put the algorithm in a larger context. See Table 11.2 for a summary of the applications of EM in this book.

11.4.1 Basic idea

Let $\mathbf{x}_i$ be the visible or observed variables in case i , and let $\mathbf{z}_i$ be the hidden or missing variables. The goal is to maximize the log likelihood of the observed data:

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^N \log p(\mathbf{x}_i|\boldsymbol{\theta}) = \sum_{i=1}^N \log \left[\sum_{\mathbf{z}_i} p(\mathbf{x}_i, \mathbf{z}_i|\boldsymbol{\theta}) \right] \quad (11.17)$$

Unfortunately this is hard to optimize, since the log cannot be pushed inside the sum.

EM gets around this problem as follows. Define the **complete data log likelihood** to be

$$\ell_c(\boldsymbol{\theta}) \triangleq \sum_{i=1}^N \log p(\mathbf{x}_i, \mathbf{z}_i | \boldsymbol{\theta}) \quad (11.18)$$

This cannot be computed, since $\mathbf{z}_i$ is unknown. So let us define the **expected complete data log likelihood** as follows:

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{t-1}) = \mathbb{E} [\ell_c(\boldsymbol{\theta}) | \mathcal{D}, \boldsymbol{\theta}^{t-1}] \quad (11.19)$$

where t is the current iteration number. Q is called the **auxiliary function**. The expectation is taken wrt the old parameters, $\boldsymbol{\theta}^{t-1}$, and the observed data $\mathcal{D}$. The goal of the **E step** is to compute $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{t-1})$, or rather, the terms inside of it which the MLE depends on; these are known as the **expected sufficient statistics** or ESS. In the **M step**, we optimize the Q function wrt $\boldsymbol{\theta}$:

$$\boldsymbol{\theta}^t = \arg \max_{\boldsymbol{\theta}} Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{t-1}) \quad (11.20)$$

To perform MAP estimation, we modify the M step as follows:

$$\boldsymbol{\theta}^t = \arg \max_{\boldsymbol{\theta}} Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{t-1}) + \log p(\boldsymbol{\theta}) \quad (11.21)$$

The E step remains unchanged.

In Section 11.4.7 we show that the EM algorithm monotonically increases the log likelihood of the observed data (plus the log prior, if doing MAP estimation), or it stays the same. So if the objective ever goes down, there must be a bug in our math or our code. (This is a surprisingly useful debugging tool!)

Below we explain how to perform the E and M steps for several simple models, that should make things clearer.

11.4.2 EM for GMMs

In this section, we discuss how to fit a mixture of Gaussians using EM. Fitting other kinds of mixture models requires a straightforward modification — see Exercise 11.3. We assume the number of mixture components, K , is known (see Section 11.5 for discussion of this point).

11.4.2.1 Auxiliary function

The expected complete data log likelihood is given by

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t-1)}) \triangleq \mathbb{E} \left[\sum_i \log p(\mathbf{x}_i, z_i | \boldsymbol{\theta}) \right] \quad (11.22)$$

$$= \sum_i \mathbb{E} \left[\log \left[\prod_{k=1}^K (\pi_k p(\mathbf{x}_i | \boldsymbol{\theta}_k))^{\mathbb{I}(z_i=k)} \right] \right] \quad (11.23)$$

$$= \sum_i \sum_k \mathbb{E} [\mathbb{I}(z_i = k)] \log [\pi_k p(\mathbf{x}_i | \boldsymbol{\theta}_k)] \quad (11.24)$$

$$= \sum_i \sum_k p(z_i = k | \mathbf{x}_i, \boldsymbol{\theta}^{(t-1)}) \log [\pi_k p(\mathbf{x}_i | \boldsymbol{\theta}_k)] \quad (11.25)$$

$$= \sum_i \sum_k r_{ik} \log \pi_k + \sum_i \sum_k r_{ik} \log p(\mathbf{x}_i | \boldsymbol{\theta}_k) \quad (11.26)$$

where $r_{ik} \triangleq p(z_i = k | \mathbf{x}_i, \boldsymbol{\theta}^{(t-1)})$ is the **responsibility** that cluster k takes for data point i . This is computed in the E step, described below.

11.4.2.2 E step

The E step has the following simple form, which is the same for any mixture model:

$$r_{ik} = \frac{\pi_k p(\mathbf{x}_i | \boldsymbol{\theta}_k^{(t-1)})}{\sum_{k'} \pi_{k'} p(\mathbf{x}_i | \boldsymbol{\theta}_{k'}^{(t-1)})} \quad (11.27)$$

11.4.2.3 M step

In the M step, we optimize Q wrt $\boldsymbol{\pi}$ and the $\boldsymbol{\theta}_k$. For $\boldsymbol{\pi}$, we obviously have

$$\pi_k = \frac{1}{N} \sum_i r_{ik} = \frac{r_k}{N} \quad (11.28)$$

where $r_k \triangleq \sum_i r_{ik}$ is the weighted number of points assigned to cluster k .

To derive the M step for the $\boldsymbol{\mu}_k$ and $\boldsymbol{\Sigma}_k$ terms, we look at the parts of Q that depend on $\boldsymbol{\mu}_k$ and $\boldsymbol{\Sigma}_k$. We see that the result is

$$\ell(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = \sum_k \sum_i r_{ik} \log p(\mathbf{x}_i | \boldsymbol{\theta}_k) \quad (11.29)$$

$$= -\frac{1}{2} \sum_i r_{ik} [\log |\boldsymbol{\Sigma}_k| + (\mathbf{x}_i - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}_k^{-1} (\mathbf{x}_i - \boldsymbol{\mu}_k)] \quad (11.30)$$

This is just a weighted version of the standard problem of computing the MLEs of an MVN (see Section 4.1.3). One can show (Exercise 11.2) that the new parameter estimates are given by

$$\boldsymbol{\mu}_k = \frac{\sum_i r_{ik} \mathbf{x}_i}{r_k} \quad (11.31)$$

$$\boldsymbol{\Sigma}_k = \frac{\sum_i r_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^T}{r_k} = \frac{\sum_i r_{ik} \mathbf{x}_i \mathbf{x}_i^T}{r_k} - \boldsymbol{\mu}_k \boldsymbol{\mu}_k^T \quad (11.32)$$

These equations make intuitive sense: the mean of cluster k is just the weighted average of all points assigned to cluster k , and the covariance is proportional to the weighted empirical scatter matrix.

After computing the new estimates, we set $\theta^t = (\pi_k, \mu_k, \Sigma_k)$ for $k = 1 : K$, and go to the next E step.

11.4.2.4 Example

An example of the algorithm in action is shown in Figure 11.11. We start with $\mu_1 = (-1, 1)$, $\Sigma_1 = \mathbf{I}$, $\mu_2 = (1, -1)$, $\Sigma_2 = \mathbf{I}$. We color code points such that blue points come from cluster 1 and red points from cluster 2. More precisely, we set the color to

$$\text{color}(i) = r_{i1}\text{blue} + r_{i2}\text{red} \quad (11.33)$$

so ambiguous points appear purple. After 20 iterations, the algorithm has converged on a good clustering. (The data was standardized, by removing the mean and dividing by the standard deviation, before processing. This often helps convergence.)

11.4.2.5 K-means algorithm

There is a popular variant of the EM algorithm for GMMs known as the **K-means algorithm**, which we now discuss. Consider a GMM in which we make the following assumptions: $\Sigma_k = \sigma^2 \mathbf{I}_D$ is fixed, and $\pi_k = 1/K$ is fixed, so only the cluster centers, $\mu_k \in \mathbb{R}^D$, have to be estimated. Now consider the following delta-function approximation to the posterior computed during the E step:

$$p(z_i = k | \mathbf{x}_i, \theta) \approx \mathbb{I}(k = z_i^*) \quad (11.34)$$

where $z_i^* = \arg\max_k p(z_i = k | \mathbf{x}_i, \theta)$. This is sometimes called **hard EM**, since we are making a hard assignment of points to clusters. Since we assumed an equal spherical covariance matrix for each cluster, the most probable cluster for $\mathbf{x}_i$ can be computed by finding the nearest prototype:

$$z_i^* = \arg \min_k \|\mathbf{x}_i - \mu_k\|_2^2 \quad (11.35)$$

Hence in each E step, we must find the Euclidean distance between N data points and K cluster centers, which takes $O(NKD)$ time. However, this can be sped up using various techniques, such as applying the triangle inequality to avoid some redundant computations (Elkan 2003). Given the hard cluster assignments, the M step updates each cluster center by computing the mean of all points assigned to it:

$$\mu_k = \frac{1}{N_k} \sum_{i: z_i = k} \mathbf{x}_i \quad (11.36)$$

See Algorithm 5 for the pseudo-code.

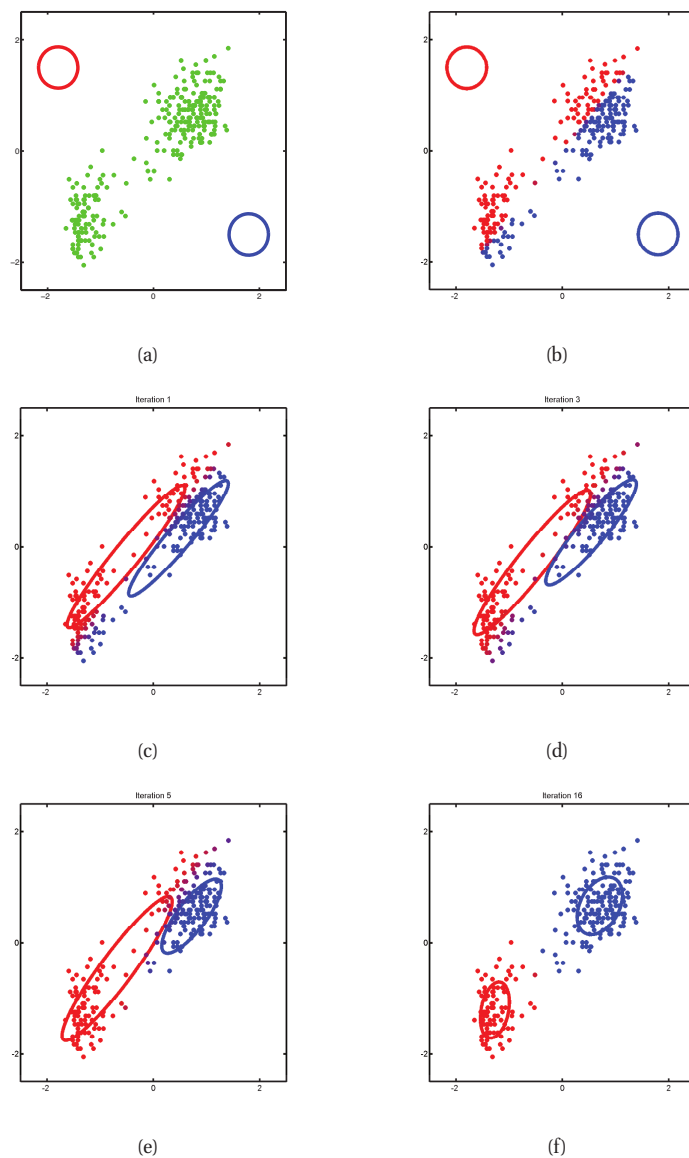


Figure 11.11 Illustration of the EM for a GMM applied to the Old Faithful data. (a) Initial (random) values of the parameters. (b) Posterior responsibility of each point computed in the first E step. The degree of redness indicates the degree to which the point belongs to the red cluster, and similarly for blue; this purple points have a roughly uniform posterior over clusters. (c) We show the updated parameters after the first M step. (d) After 3 iterations. (e) After 5 iterations. (f) After 16 iterations. Based on (Bishop 2006a) Figure 9.8. Figure generated by `mixGaussDemoFaithful`.

Algorithm 11.1: K-means algorithm

```

1 initialize  $\mathbf{m}_k$ ;
2 repeat
3   Assign each data point to its closest cluster center:  $z_i = \arg \min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2$ ;
4   Update each cluster center by computing the mean of all points assigned to it:
       $\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{i: z_i=k} \mathbf{x}_i$ ;
5 until converged;
```

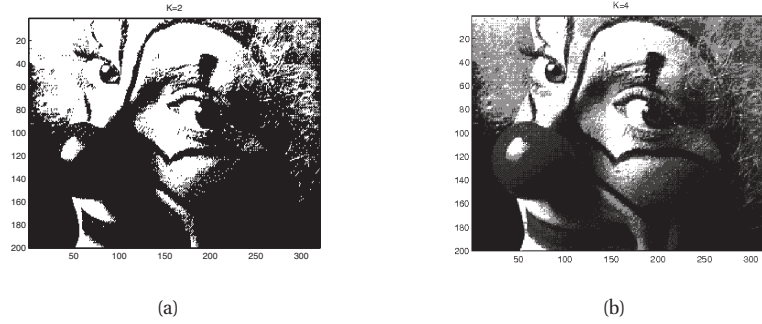


Figure 11.12 An image compressed using vector quantization with a codebook of size K . (a) $K = 2$. (b) $K = 4$. Figure generated by `vqDemo`.

11.4.2.6 Vector quantization

Since K-means is not a proper EM algorithm, it is not maximizing likelihood. Instead, it can be interpreted as a greedy algorithm for approximately minimizing a loss function related to data compression, as we now explain.

Suppose we want to perform lossy compression of some real-valued vectors, $\mathbf{x}_i \in \mathbb{R}^D$. A very simple approach to this is to use **vector quantization** or **VQ**. The basic idea is to replace each real-valued vector $\mathbf{x}_i \in \mathbb{R}^D$ with a discrete symbol $z_i \in \{1, \dots, K\}$, which is an index into a **codebook** of K prototypes, $\boldsymbol{\mu}_k \in \mathbb{R}^D$. Each data vector is encoded by using the index of the most similar prototype, where similarity is measured in terms of Euclidean distance:

$$\text{encode}(\mathbf{x}_i) = \arg \min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2 \quad (11.37)$$

We can define a cost function that measures the quality of a codebook by computing the **reconstruction error** or **distortion** it induces:

$$J(\boldsymbol{\mu}, \mathbf{z} | K, \mathbf{X}) \triangleq \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - \text{decode}(\text{encode}(\mathbf{x}_i))\|^2 = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - \boldsymbol{\mu}_{z_i}\|^2 \quad (11.38)$$

where $\text{decode}(k) = \boldsymbol{\mu}_k$. The K-means algorithm can be thought of as a simple iterative scheme for minimizing this objective.

Of course, we can achieve zero distortion if we assign one prototype to every data vector, but that takes $O(NDC)$ space, where N is the number of real-valued data vectors, each of

length D , and C is the number of bits needed to represent a real-valued scalar (the quantization accuracy). However, in many data sets, we see similar vectors repeatedly, so rather than storing them many times, we can store them once and then create pointers to them. Hence we can reduce the space requirement to $O(N \log_2 K + KDC)$: the $O(N \log_2 K)$ term arises because each of the N data vectors needs to specify which of the K codewords it is using (the pointers); and the $O(KDC)$ term arises because we have to store each codebook entry, each of which is a D -dimensional vector. Typically the first term dominates the second, so we can approximate the **rate** of the encoding scheme (number of bits needed per object) as $O(\log_2 K)$, which is typically much less than $O(DC)$.

One application of VQ is to image compression. Consider the $N = 200 \times 320 = 64,000$ pixel image in Figure 11.12; this is gray-scale, so $D = 1$. If we use one byte to represent each pixel (a gray-scale intensity of 0 to 255), then $C = 8$, so we need $NC = 512,000$ bits to represent the image. For the compressed image, we need $N \log_2 K + KC$ bits. For $K = 4$, this is about 128kb, a factor of 4 compression. For $K = 8$, this is about 192kb, a factor of 2.6 compression, at negligible perceptual loss (see Figure 11.12(b)). Greater compression could be achieved if we modelled spatial correlation between the pixels, e.g., if we encoded 5x5 blocks (as used by JPEG). This is because the residual errors (differences from the model's predictions) would be smaller, and would take fewer bits to encode.

11.4.2.7 Initialization and avoiding local minima

Both K-means and EM need to be initialized. It is common to pick K data points at random, and to make these be the initial cluster centers. Or we can pick the centers sequentially so as to try to “cover” the data. That is, we pick the initial point uniformly at random. Then each subsequent point is picked from the remaining points with probability proportional to its squared distance to the points's closest cluster center. This is known as **farthest point clustering** (Gonzales 1985), or **k-means++** (Arthur and Vassilvitskii 2007; Bahmani et al. 2012). Surprisingly, this simple trick can be shown to guarantee that the distortion is never more than $O(\log K)$ worse than optimal (Arthur and Vassilvitskii 2007).

An heuristic that is commonly used in the speech recognition community is to incrementally “grow” GMMs: we initially give each cluster a score based on its mixture weight; after each round of training, we consider splitting the cluster with the highest score into two, with the new centroids being random perturbations of the original centroid, and the new scores being half of the old scores. If a new cluster has too small a score, or too narrow a variance, it is removed. We continue in this way until the desired number of clusters is reached. See (Figueiredo and Jain 2002) for a similar incremental approach.

11.4.2.8 MAP estimation

As usual, the MLE may overfit. The overfitting problem is particularly severe in the case of GMMs. To understand the problem, suppose for simplicity that $\Sigma_k = \sigma_k^2 I$, and that $K = 2$. It is possible to get an infinite likelihood by assigning one of the centers, say μ_2 , to a single data point, say $\mathbf{x}_1$, since then the 1st term makes the following contribution to the likelihood:

$$\mathcal{N}(\mathbf{x}_1 | \mu_2, \sigma_2^2 I) = \frac{1}{\sqrt{2\pi\sigma_2^2}} e^0 \quad (11.39)$$

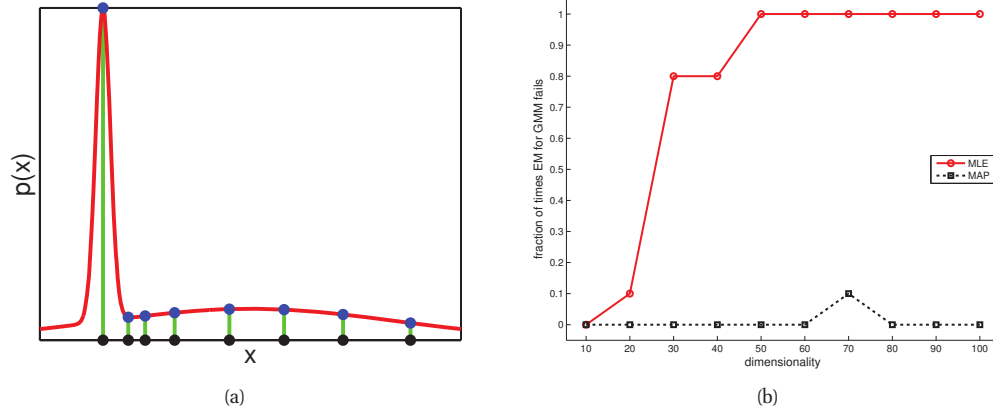


Figure 11.13 (a) Illustration of how singularities can arise in the likelihood function of GMMs. Based on (Bishop 2006a) Figure 9.7. Figure generated by `mixGaussSingularity`. (b) Illustration of the benefit of MAP estimation vs ML estimation when fitting a Gaussian mixture model. We plot the fraction of times (out of 5 random trials) each method encounters numerical problems vs the dimensionality of the problem, for $N = 100$ samples. Solid red (upper curve): MLE. Dotted black (lower curve): MAP. Figure generated by `mixGaussMLvsMAP`.

Hence we can drive this term to infinity by letting $\sigma_2 \rightarrow 0$, as shown in Figure 11.13(a). We will call this the “collapsing variance problem”.

An easy solution to this is to perform MAP estimation. The new auxiliary function is the expected complete data log-likelihood plus the log prior:

$$Q'(\theta, \theta^{old}) = \left[\sum_i \sum_k r_{ik} \log \pi_{ik} + \sum_i \sum_k r_{ik} \log p(\mathbf{x}_i | \theta_k) \right] + \log p(\pi) + \sum_k \log p(\theta_k) \quad (11.40)$$

Note that the E step remains unchanged, but the M step needs to be modified, as we now explain.

For the prior on the mixture weights, it is natural to use a Dirichlet prior, $\pi \sim \text{Dir}(\alpha)$, since this is conjugate to the categorical distribution. The MAP estimate is given by

$$\pi_k = \frac{r_k + \alpha_k - 1}{N + \sum_k \alpha_k - K} \quad (11.41)$$

If we use a uniform prior, $\alpha_k = 1$, this reduces to Equation 11.28.

The prior on the parameters of the class conditional densities, $p(\theta_k)$, depends on the form of the class conditional densities. We discuss the case of GMMs below, and leave MAP estimation for mixtures of Bernoullis to Exercise 11.3.

For simplicity, let us consider a conjugate prior of the form

$$p(\mu_k, \Sigma_k) = \text{NIW}(\mu_k, \Sigma_k | \mathbf{m}_0, \kappa_0, \nu_0, \mathbf{S}_0) \quad (11.42)$$

From Section 4.6.3, the MAP estimate is given by

$$\hat{\boldsymbol{\mu}}_k = \frac{r_k \bar{\mathbf{x}}_k + \kappa_0 \mathbf{m}_0}{r_k + \kappa_0} \quad (11.43)$$

$$(11.44)$$

$$\bar{\mathbf{x}}_k \triangleq \frac{\sum_i r_{ik} \mathbf{x}_i}{r_k} \quad (11.45)$$

$$\hat{\boldsymbol{\Sigma}}_k = \frac{\mathbf{S}_0 + \mathbf{S}_k + \frac{\kappa_0 r_k}{\kappa_0 + r_k} (\bar{\mathbf{x}}_k - \mathbf{m}_0)(\bar{\mathbf{x}}_k - \mathbf{m}_0)^T}{\nu_0 + r_k + D + 2} \quad (11.46)$$

$$\mathbf{S}_k \triangleq \sum_i r_{ik} (\mathbf{x}_i - \bar{\mathbf{x}}_k)(\mathbf{x}_i - \bar{\mathbf{x}}_k)^T \quad (11.47)$$

We now illustrate the benefits of using MAP estimation instead of ML estimation in the context of GMMs. We apply EM to some synthetic data in D dimensions, using either ML or MAP estimation. We count the trial as a “failure” if there are numerical issues involving singular matrices. For each dimensionality, we conduct 5 random trials. The results are illustrated in Figure 11.13(b) using $N = 100$. We see that as soon as D becomes even moderately large, ML estimation crashes and burns, whereas MAP estimation never encounters numerical problems.

When using MAP estimation, we need to specify the hyper-parameters. Here we mention some simple heuristics for setting them (Fraley and Raftery 2007, p163). We can set $\kappa_0 = 0$, so that the $\boldsymbol{\mu}_k$ are unregularized, since the numerical problems only arise from $\boldsymbol{\Sigma}_k$. In this case, the MAP estimates simplify to $\hat{\boldsymbol{\mu}}_k = \bar{\mathbf{x}}_k$ and $\hat{\boldsymbol{\Sigma}}_k = \frac{\mathbf{S}_0 + \mathbf{S}_k}{\nu_0 + r_k + D + 2}$, which is not quite so scary-looking.

Now we discuss how to set $\mathbf{S}_0$. One possibility is to use

$$\mathbf{S}_0 = \frac{1}{K^{1/D}} \text{diag}(s_1^2, \dots, s_D^2) \quad (11.48)$$

where $s_j = (1/N) \sum_{i=1}^N (x_{ij} - \bar{x}_j)^2$ is the pooled variance for dimension j . (The reason for the $\frac{1}{K^{1/D}}$ term is that the resulting volume of each ellipsoid is then given by $|\mathbf{S}_0| = \frac{1}{K} |\text{diag}(s_1^2, \dots, s_D^2)|$.) The parameter ν_0 controls how strongly we believe this prior. The weakest prior we can use, while still being proper, is to set $\nu_0 = D + 2$, so this is a common choice.

11.4.3 EM for mixture of experts

We can fit a mixture of experts model using EM in a straightforward manner. The expected complete data log likelihood is given by

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{old}) = \sum_{i=1}^N \sum_{k=1}^K r_{ik} \log[\pi_{ik} \mathcal{N}(y_i | \mathbf{w}_k^T \mathbf{x}_i, \sigma_k^2)] \quad (11.49)$$

$$\pi_{i,k} \triangleq \mathcal{S}(\mathbf{V}^T \mathbf{x}_i)_k \quad (11.50)$$

$$r_{ik} \propto \pi_{ik}^{old} \mathcal{N}(y_i | \mathbf{x}_i^T \mathbf{w}_k^{old}, (\sigma_k^{old})^2) \quad (11.51)$$

So the E step is the same as in a standard mixture model, except we have to replace π_k with $\pi_{i,k}$ when computing r_{ik} .

In the M step, we need to maximize $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{old})$ wrt $\mathbf{w}_k$, σ_k^2 and $\mathbf{V}$. For the regression parameters for model k , the objective has the form

$$Q(\boldsymbol{\theta}_k, \boldsymbol{\theta}^{old}) = \sum_{i=1}^N r_{ik} \left\{ -\frac{1}{\sigma_k^2} (y_i - \mathbf{w}_k^T \mathbf{x}_i) \right\} \quad (11.52)$$

We recognize this as a weighted least squares problem, which makes intuitive sense: if r_{ik} is small, then data point i will be downweighted when estimating model k 's parameters. From Section 8.3.4 we can immediately write down the MLE as

$$\mathbf{w}_k = (\mathbf{X}^T \mathbf{R}_k \mathbf{X})^{-1} \mathbf{X}^T \mathbf{R}_k \mathbf{y} \quad (11.53)$$

where $\mathbf{R}_k = \text{diag}(r_{:,k})$. The MLE for the variance is given by

$$\sigma_k^2 = \frac{\sum_{i=1}^N r_{ik} (y_i - \mathbf{w}_k^T \mathbf{x}_i)^2}{\sum_{i=1}^N r_{ik}} \quad (11.54)$$

We replace the estimate of the unconditional mixing weights $\boldsymbol{\pi}$ with the estimate of the gating parameters, $\mathbf{V}$. The objective has the form

$$\ell(\mathbf{V}) = \sum_i \sum_k r_{ik} \log \pi_{i,k} \quad (11.55)$$

We recognize this as equivalent to the log-likelihood for multinomial logistic regression in Equation 8.34, except we replace the “hard” 1-of- C encoding $\mathbf{y}_i$ with the “soft” 1-of- K encoding $\mathbf{r}_i$. Thus we can estimate $\mathbf{V}$ by fitting a logistic regression model to soft target labels.

11.4.4 EM for DGMs with hidden variables

We can generalize the ideas behind EM for mixtures of experts to compute the MLE or MAP estimate for an arbitrary DGM. We could use gradient-based methods (Binder et al. 1997), but it is much simpler to use EM (Lauritzen 1995): in the E step, we just estimate the hidden variables, and in the M step, we will compute the MLE using these filled-in values. We give the details below.

For simplicity of presentation, we will assume all CPDs are tabular. Based on Section 10.4.2, let us write each CPT as follows:

$$p(x_{it} | \mathbf{x}_{i, \text{pa}(t)}, \boldsymbol{\theta}_t) = \prod_{c=1}^{K_{\text{pa}(t)}} \prod_{k=1}^{K_t} \theta_{tck}^{\mathbb{I}(x_{it}=i, \mathbf{x}_{i, \text{pa}(t)}=c)} \quad (11.56)$$

The log-likelihood of the complete data is given by

$$\log p(\mathcal{D} | \boldsymbol{\theta}) = \sum_{t=1}^V \sum_{c=1}^{K_{\text{pa}(t)}} \sum_{k=1}^{K_t} N_{tck} \log \theta_{tck} \quad (11.57)$$

where $N_{tck} = \sum_{i=1}^N \mathbb{I}(x_{it} = i, \mathbf{x}_{i, \text{pa}(t)} = c)$ are the empirical counts. Hence the expected complete data log-likelihood has the form

$$\mathbb{E} [\log p(\mathcal{D} | \boldsymbol{\theta})] = \sum_t \sum_c \sum_k \bar{N}_{tck} \log \theta_{tck} \quad (11.58)$$

where

$$\bar{N}_{tck} = \sum_{i=1}^N \mathbb{E} [\mathbb{I}(x_{it} = i, \mathbf{x}_{i,\text{pa}(t)} = c)] = \sum_i p(x_{it} = k, \mathbf{x}_{i,\text{pa}(t)} = c | \mathcal{D}_i) \quad (11.59)$$

where $\mathcal{D}_i$ are all the visible variables in case i .

The quantity $p(x_{it}, \mathbf{x}_{i,\text{pa}(t)} | \mathcal{D}_i, \boldsymbol{\theta})$ is known as a **family marginal**, and can be computed using any GM inference algorithm. The $\bar{N}_{tjk}$ are the expected sufficient statistics, and constitute the output of the E step.

Given these ESS, the M step has the simple form

$$\hat{\theta}_{tck} = \frac{\bar{N}_{tck}}{\sum_{k'} \bar{N}_{tjk'}} \quad (11.60)$$

This can be proved by adding Lagrange multipliers (to enforce the constraint $\sum_k \theta_{tjk} = 1$) to the expected complete data log likelihood, and then optimizing each parameter vector $\boldsymbol{\theta}_{tc}$ separately. We can modify this to perform MAP estimation with a Dirichlet prior by simply adding pseudo counts to the expected counts.

11.4.5 EM for the Student distribution *

One problem with the Gaussian distribution is that it is sensitive to outliers, since the log-probability only decays quadratically with distance from the center. A more robust alternative is the Student t distribution, as discussed in Section ??.

Unlike the case of a Gaussian, there is no closed form formula for the MLE of a Student, even if we have no missing data, so we must resort to iterative optimization methods. The easiest one to use is EM, since it automatically enforces the constraints that ν is positive and that $\boldsymbol{\Sigma}$ is symmetric positive definite. In addition, the resulting algorithm turns out to have a simple intuitive form, as we see below.

At first blush, it might not be apparent why EM can be used, since there is no missing data. The key idea is to introduce an “artificial” hidden or auxiliary variable in order to simplify the algorithm. In particular, we will exploit the fact that a Student distribution can be written as a **Gaussian scale mixture**:

$$\mathcal{T}(\mathbf{x}_i | \boldsymbol{\mu}, \boldsymbol{\Sigma}, \nu) = \int \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}, \boldsymbol{\Sigma}/z_i) \text{Ga}(z_i | \frac{\nu}{2}, \frac{\nu}{2}) dz_i \quad (11.61)$$

(See Exercise 11.1 for a proof of this in the 1d case.) This can be thought of as an “infinite” mixture of Gaussians, each one with a slightly different covariance matrix.

Treating the z_i as missing data, we can write the complete data log likelihood as

$$\ell_c(\boldsymbol{\theta}) = \sum_{i=1}^N [\log \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}, \boldsymbol{\Sigma}/z_i) + \log \text{Ga}(z_i | \nu/2, \nu/2)] \quad (11.62)$$

$$= \sum_{i=1}^N \left[-\frac{D}{2} \log(2\pi) - \frac{1}{2} \log |\boldsymbol{\Sigma}| - \frac{z_i}{2} \delta_i + \frac{\nu}{2} \log \frac{\nu}{2} - \log \Gamma\left(\frac{\nu}{2}\right) \right] \quad (11.63)$$

$$+ \frac{\nu}{2} (\log z_i - z_i) + \left(\frac{D}{2} - 1 \right) \log z_i \quad (11.64)$$

where we have defined the Mahalanobis distance to be

$$\delta_i = (\mathbf{x}_i - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x}_i - \boldsymbol{\mu}) \quad (11.65)$$

We can partition this into two terms, one involving $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$, and the other involving ν . We have, dropping irrelevant constants,

$$\ell_c(\boldsymbol{\theta}) = L_N(\boldsymbol{\mu}, \boldsymbol{\Sigma}) + L_G(\nu) \quad (11.66)$$

$$L_N(\boldsymbol{\mu}, \boldsymbol{\Sigma}) \triangleq -\frac{1}{2}N \log |\boldsymbol{\Sigma}| - \frac{1}{2} \sum_{i=1}^N z_i \delta_i \quad (11.67)$$

$$L_G(\nu) \triangleq -N \log \Gamma(\nu/2) + \frac{1}{2}N\nu \log(\nu/2) + \frac{1}{2}\nu \sum_{i=1}^N (\log z_i - z_i) \quad (11.68)$$

11.4.5.1 EM with ν known

Let us first derive the algorithm with ν assumed known, for simplicity. In this case, we can ignore the L_G term, so we only need to figure out how to compute $\mathbb{E}[z_i]$ wrt the old parameters.

From Section 4.6.2.2 we have

$$p(z_i | \mathbf{x}_i, \boldsymbol{\theta}) = \text{Ga}(z_i | \frac{\nu + D}{2}, \frac{\nu + \delta_i}{2}) \quad (11.69)$$

Now if $z_i \sim \text{Ga}(a, b)$, then $\mathbb{E}[z_i] = a/b$. Hence the E step at iteration t is

$$\bar{z}_i^{(t)} \triangleq \mathbb{E}[z_i | \mathbf{x}_i, \boldsymbol{\theta}^{(t)}] = \frac{\nu^{(t)} + D}{\nu^{(t)} + \delta_i^{(t)}} \quad (11.70)$$

The M step is obtained by maximizing $\mathbb{E}[L_N(\boldsymbol{\mu}, \boldsymbol{\Sigma})]$ to yield

$$\hat{\boldsymbol{\mu}}^{(t+1)} = \frac{\sum_i \bar{z}_i^{(t)} \mathbf{x}_i}{\sum_i \bar{z}_i^{(t)}} \quad (11.71)$$

$$\hat{\boldsymbol{\Sigma}}^{(t+1)} = \frac{1}{N} \sum_i \bar{z}_i^{(t)} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}^{(t+1)})(\mathbf{x}_i - \hat{\boldsymbol{\mu}}^{(t+1)})^T \quad (11.72)$$

$$= \frac{1}{N} \left[\sum_i \bar{z}_i^{(t)} \mathbf{x}_i \mathbf{x}_i^T - \left(\sum_{i=1}^N \bar{z}_i^{(t)} \right) \hat{\boldsymbol{\mu}}^{(t+1)} (\hat{\boldsymbol{\mu}}^{(t+1)})^T \right] \quad (11.73)$$

These results are quite intuitive: the quantity $\bar{z}_i$ is the precision of measurement i , so if it is small, the corresponding data point is down-weighted when estimating the mean and covariance. This is how the Student achieves robustness to outliers.

11.4.5.2 EM with ν unknown

To compute the MLE for the degrees of freedom, we first need to compute the expectation of $L_G(\nu)$, which involves z_i and $\log z_i$. Now if $z_i \sim \text{Ga}(a, b)$, then one can show that

$$\bar{\ell}_i^{(t)} \triangleq \mathbb{E}[\log z_i | \boldsymbol{\theta}^{(t)}] = \Psi(a) - \log b \quad (11.74)$$

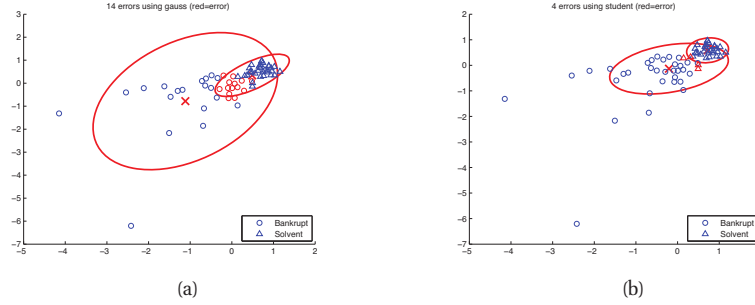


Figure 11.14 Mixture modeling on the bankruptcy data set. Left: Gaussian class conditional densities. Right: Student class conditional densities. Points that belong to class 1 are shown as triangles, points that belong to class 2 are shown as circles. The estimated labels, based on the posterior probability of belonging to each mixture component, are computed. If these are incorrect, the point is colored red, otherwise it is colored blue. (Training data is in black.) Figure generated by `mixStudentBankruptcyDemo`.

where $\Psi(x) = \frac{d}{dx} \log \Gamma(x)$ is the digamma function. Hence, from Equation 11.69, we have

$$\bar{\ell}_i^{(t)} = \Psi\left(\frac{\nu^{(t)} + D}{2}\right) - \log\left(\frac{\nu^{(t)} + \delta_i^{(t)}}{2}\right) \quad (11.75)$$

$$= \log(\bar{z}_i^{(t)}) + \Psi\left(\frac{\nu^{(t)} + D}{2}\right) - \log\left(\frac{\nu^{(t)} + D}{2}\right) \quad (11.76)$$

Substituting into Equation 11.68, we have

$$\mathbb{E}[L_G(\nu)] = -N \log \Gamma(\nu/2) + \frac{N\nu}{2} \log(\nu/2) + \frac{\nu}{2} \sum_i (\bar{\ell}_i^{(t)} - \bar{z}_i^{(t)}) \quad (11.77)$$

The gradient of this expression is equal to

$$\frac{d}{d\nu} \mathbb{E}[L_G(\nu)] = -\frac{N}{2} \Psi(\nu/2) + \frac{N}{2} \log(\nu/2) + \frac{N}{2} + \frac{1}{2} \sum_i (\bar{\ell}_i^{(t)} - \bar{z}_i^{(t)}) \quad (11.78)$$

This has a unique solution in the interval $(0, +\infty]$ which can be found using a 1d constrained optimizer.

Performing a gradient-based optimization in the M step, rather than a closed-form update, is an example of what is known as the **generalized EM** algorithm. One can show that EM will still converge to a local optimum even if we only perform a “partial” improvement to the parameters in the M step.

11.4.5.3 Mixtures of Student distributions

It is easy to extend the above methods to fit a mixture of Student distributions. See Exercise 11.4 for the details.

Let us consider a small example from (Lo 2009, ch3). We have a $N = 66$, $D = 2$ data set regarding the bankruptcy patterns of certain companies. The first feature specifies the ratio

of retained earnings (RE) to total assets, and the second feature specifies the ratio of earnings before interests and taxes (EBIT) to total assets. We fit two models to this data, ignoring the class labels: a mixture of 2 Gaussians, and a mixture of 2 Students. We then use each fitted model to classify the data. We compute the most probable cluster membership and treat this as $\hat{y}_i$. We then compare $\hat{y}_i$ to the true labels y_i and compute an error rate. If this is more than 50%, we permute the latent labels (i.e., we consider cluster 1 to represent class 2 and vice versa), and then recompute the error rate. Points which are misclassified are then shown in red. The result is shown in Figure 11.14. We see that the Student model made 4 errors, the Gaussian model made 21. This is because the class-conditional densities contain some extreme values, causing the Gaussian to be a poor choice.

11.4.6 EM for probit regression *

In Section 9.4.2, we described the latent variable interpretation of probit regression. Recall that this has the form $p(y_i = 1|z_i) = \mathbb{I}(z_i > 0)$, where $z_i \sim \mathcal{N}(\mathbf{w}^T \mathbf{x}_i, 1)$ is latent. We now show how to fit this model using EM. (Although it is possible to fit probit regression models using gradient based methods, as shown in Section 9.4.1, this EM-based approach has the advantage that it generalized to many other kinds of models, as we will see later on.)

The complete data log likelihood has the following form, assuming a $\mathcal{N}(\mathbf{0}, \mathbf{V}_0)$ prior on $\mathbf{w}$:

$$\ell(\mathbf{z}, \mathbf{w} | \mathbf{V}_0) = \log p(\mathbf{y} | \mathbf{z}) + \log \mathcal{N}(\mathbf{z} | \mathbf{X}\mathbf{w}, \mathbf{I}) + \log \mathcal{N}(\mathbf{w} | \mathbf{0}, \mathbf{V}_0) \quad (11.79)$$

$$= \sum_i \log p(y_i | z_i) - \frac{1}{2}(\mathbf{z} - \mathbf{X}\mathbf{w})^T (\mathbf{z} - \mathbf{X}\mathbf{w}) - \frac{1}{2}\mathbf{w}^T \mathbf{V}_0^{-1} \mathbf{w} + \text{const} \quad (11.80)$$

The posterior in the E step is a **truncated Gaussian**:

$$p(z_i | y_i, \mathbf{x}_i, \mathbf{w}) = \begin{cases} \mathcal{N}(z_i | \mathbf{w}^T \mathbf{x}_i, 1) \mathbb{I}(z_i > 0) & \text{if } y_i = 1 \\ \mathcal{N}(z_i | \mathbf{w}^T \mathbf{x}_i, 1) \mathbb{I}(z_i < 0) & \text{if } y_i = 0 \end{cases} \quad (11.81)$$

In Equation 11.80, we see that $\mathbf{w}$ only depends linearly on $\mathbf{z}$, so we just need to compute $\mathbb{E}[z_i | y_i, \mathbf{x}_i, \mathbf{w}]$. Exercise 11.15 asks you to show that the posterior mean is given by

$$\mathbb{E}[z_i | \mathbf{w}, \mathbf{x}_i] = \begin{cases} \mu_i + \frac{\phi(\mu_i)}{1 - \Phi(-\mu_i)} = \mu_i + \frac{\phi(\mu_i)}{\Phi(\mu_i)} & \text{if } y_i = 1 \\ \mu_i - \frac{\phi(\mu_i)}{\Phi(-\mu_i)} = \mu_i - \frac{\phi(\mu_i)}{1 - \Phi(\mu_i)} & \text{if } y_i = 0 \end{cases} \quad (11.82)$$

where $\mu_i = \mathbf{w}^T \mathbf{x}_i$.

In the M step, we estimate $\mathbf{w}$ using ridge regression, where $\boldsymbol{\mu} = \mathbb{E}[\mathbf{z}]$ is the output we are trying to predict. Specifically, we have

$$\hat{\mathbf{w}} = (\mathbf{V}_0^{-1} + \mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \boldsymbol{\mu} \quad (11.83)$$

The EM algorithm is simple, but can be much slower than direct gradient methods, as illustrated in Figure 11.15. This is because the posterior entropy in the E step is quite high, since we only observe that z is positive or negative, but are given no information from the likelihood about its magnitude. Using a stronger regularizer can help speed convergence, because it constrains the range of plausible z values. In addition, one can use various speedup tricks, such as data augmentation (van Dyk and Meng 2001), but we do not discuss that here.

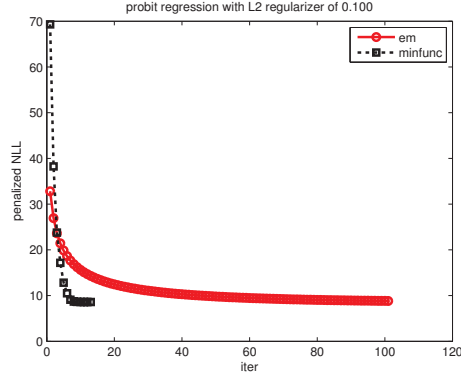


Figure 11.15 Fitting a probit regression model in 2d using a quasi-Newton method or EM. Figure generated by probitRegDemo.

11.4.7 Theoretical basis for EM *

In this section, we show that EM monotonically increases the observed data log likelihood until it reaches a local maximum (or saddle point, although such points are usually unstable). Our derivation will also serve as the basis for various generalizations of EM that we will discuss later.

11.4.7.1 Expected complete data log likelihood is a lower bound

Consider an arbitrary distribution $q(\mathbf{z}_i)$ over the hidden variables. The observed data log likelihood can be written as follows:

$$\ell(\boldsymbol{\theta}) \triangleq \sum_{i=1}^N \log \left[\sum_{\mathbf{z}_i} p(\mathbf{x}_i, \mathbf{z}_i | \boldsymbol{\theta}) \right] = \sum_{i=1}^N \log \left[\sum_{\mathbf{z}_i} q(\mathbf{z}_i) \frac{p(\mathbf{x}_i, \mathbf{z}_i | \boldsymbol{\theta})}{q(\mathbf{z}_i)} \right] \quad (11.84)$$

Now $\log(u)$ is a *concave* function, so from Jensen's inequality (Equation 2.113) we have the following *lower bound*:

$$\ell(\boldsymbol{\theta}) \geq \sum_i \sum_{\mathbf{z}_i} q_i(\mathbf{z}_i) \log \frac{p(\mathbf{x}_i, \mathbf{z}_i | \boldsymbol{\theta})}{q_i(\mathbf{z}_i)} \quad (11.85)$$

Let us denote this lower bound as follows:

$$Q(\boldsymbol{\theta}, q) \triangleq \sum_i \mathbb{E}_{q_i} [\log p(\mathbf{x}_i, \mathbf{z}_i | \boldsymbol{\theta})] + \mathbb{H}(q_i) \quad (11.86)$$

where $\mathbb{H}(q_i)$ is the entropy of q_i .

The above argument holds for any positive distribution q . Which one should we choose? Intuitively we should pick the q that yields the tightest lower bound. The lower bound is a sum

over i of terms of the following form:

$$L(\boldsymbol{\theta}, q_i) = \sum_{\mathbf{z}_i} q_i(\mathbf{z}_i) \log \frac{p(\mathbf{x}_i, \mathbf{z}_i | \boldsymbol{\theta})}{q_i(\mathbf{z}_i)} \quad (11.87)$$

$$= \sum_{\mathbf{z}_i} q_i(\mathbf{z}_i) \log \frac{p(\mathbf{z}_i | \mathbf{x}_i, \boldsymbol{\theta}) p(\mathbf{x}_i | \boldsymbol{\theta})}{q_i(\mathbf{z}_i)} \quad (11.88)$$

$$= \sum_{\mathbf{z}_i} q_i(\mathbf{z}_i) \log \frac{p(\mathbf{z}_i | \mathbf{x}_i, \boldsymbol{\theta})}{q_i(\mathbf{z}_i)} + \sum_{\mathbf{z}_i} q_i(\mathbf{z}_i) \log p(\mathbf{x}_i | \boldsymbol{\theta}) \quad (11.89)$$

$$= -\mathbb{KL}(q_i(\mathbf{z}_i) || p(\mathbf{z}_i | \mathbf{x}_i, \boldsymbol{\theta})) + \log p(\mathbf{x}_i | \boldsymbol{\theta}) \quad (11.90)$$

The $p(\mathbf{x}_i | \boldsymbol{\theta})$ term is independent of q_i , so we can maximize the lower bound by setting $q_i(\mathbf{z}_i) = p(\mathbf{z}_i | \mathbf{x}_i, \boldsymbol{\theta})$. Of course, $\boldsymbol{\theta}$ is unknown, so instead we use $q_i^t(\mathbf{z}_i) = p(\mathbf{z}_i | \mathbf{x}_i, \boldsymbol{\theta}^t)$, where $\boldsymbol{\theta}^t$ is our estimate of the parameters at iteration t . This is the output of the E step.

Plugging this in to the lower bound we get

$$Q(\boldsymbol{\theta}, q^t) = \sum_i \mathbb{E}_{q_i^t} [\log p(\mathbf{x}_i, \mathbf{z}_i | \boldsymbol{\theta})] + \mathbb{H}(q_i^t) \quad (11.91)$$

We recognize the first term as the expected complete data log likelihood. The second term is a constant wrt $\boldsymbol{\theta}$. So the M step becomes

$$\boldsymbol{\theta}^{t+1} = \arg \max_{\boldsymbol{\theta}} Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t) = \arg \max_{\boldsymbol{\theta}} \sum_i \mathbb{E}_{q_i^t} [\log p(\mathbf{x}_i, \mathbf{z}_i | \boldsymbol{\theta})] \quad (11.92)$$

as usual.

Now comes the punchline. Since we used $q_i^t(\mathbf{z}_i) = p(\mathbf{z}_i | \mathbf{x}_i, \boldsymbol{\theta}^t)$, the KL divergence becomes zero, so $L(\boldsymbol{\theta}^t, q_i) = \log p(\mathbf{x}_i | \boldsymbol{\theta}^t)$, and hence

$$Q(\boldsymbol{\theta}^t, \boldsymbol{\theta}^t) = \sum_i \log p(\mathbf{x}_i | \boldsymbol{\theta}^t) = \ell(\boldsymbol{\theta}^t) \quad (11.93)$$

We see that the lower bound is tight after the E step. Since the lower bound “touches” the function, maximizing the lower bound will also “push up” on the function itself. That is, the M step is guaranteed to modify the parameters so as to increase the likelihood of the observed data (unless it is already at a local maximum).

This process is sketched in Figure 11.16. The dashed red curve is the original function (the observed data log-likelihood). The solid blue curve is the lower bound, evaluated at $\boldsymbol{\theta}^t$; this touches the objective function at $\boldsymbol{\theta}^t$. We then set $\boldsymbol{\theta}^{t+1}$ to the maximum of the lower bound (blue curve), and fit a new bound at that point (dotted green curve). The maximum of this new bound becomes $\boldsymbol{\theta}^{t+2}$, etc. (Compare this to Newton’s method in Figure 8.4(a), which repeatedly fits and then optimizes a quadratic approximation.)

11.4.7.2 EM monotonically increases the observed data log likelihood

We now prove that EM monotonically increases the observed data log likelihood until it reaches a local optimum. We have

$$\ell(\boldsymbol{\theta}^{t+1}) \geq Q(\boldsymbol{\theta}^{t+1}, \boldsymbol{\theta}^t) \geq Q(\boldsymbol{\theta}^t, \boldsymbol{\theta}^t) = \ell(\boldsymbol{\theta}^t) \quad (11.94)$$

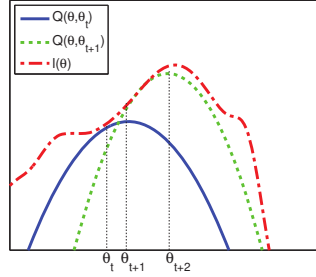


Figure 11.16 Illustration of EM as a bound optimization algorithm. Based on Figure 9.14 of (Bishop 2006a). Figure generated by `emLogLikelihoodMax`.

where the first inequality follows since $Q(\theta, \cdot)$ is a lower bound on $\ell(\theta)$; the second inequality follows since, by definition, $Q(\theta^{t+1}, \theta^t) = \max_{\theta} Q(\theta, \theta^t) \geq Q(\theta^t, \theta^t)$; and the final equality follows Equation 11.93.

As a consequence of this result, if you do not observe monotonic increase of the observed data log likelihood, you must have an error in your math and/or code. (If you are performing MAP estimation, you must add on the log prior term to the objective.) This is a surprisingly powerful debugging tool.

11.4.8 Online EM

When dealing with large or streaming datasets, it is important to be able to learn online, as we discussed in Section 8.5. There are two main approaches to **online EM** in the literature. The first approach, known as **incremental EM** (Neal and Hinton 1998), optimizes the lower bound $Q(\theta, q_1, \dots, q_N)$ one q_i at a time; however, this requires storing the expected sufficient statistics for each data case. The second approach, known as **stepwise EM** (Sato and Ishii 2000; Cappe and Mouline 2009; Cappe 2010), is based on stochastic approximation theory, and only requires constant memory use. We explain both approaches in more detail below, following the presentation of (Liang and Klein Liang and Klein).

11.4.8.1 Batch EM review

Before explaining online EM, we review batch EM in a more abstract setting. Let $\phi(\mathbf{x}, \mathbf{z})$ be a vector of sufficient statistics for a single data case. (For example, for a mixture of multinoullis, this would be the count vector $a(j)$, which is the number of cluster j was used in $\mathbf{z}$, plus the matrix $B(j, v)$, which is of the number of times the hidden state was j and the observed letter was v .) Let $\mathbf{s}_i = \sum_{\mathbf{z}} p(\mathbf{z}|\mathbf{x}_i, \theta) \phi(\mathbf{x}_i, \mathbf{z})$ be the expected sufficient statistics for case i , and $\boldsymbol{\mu} = \sum_{i=1}^N \mathbf{s}_i$ be the sum of the ESS. Given $\boldsymbol{\mu}$, we can derive an ML or MAP estimate of the parameters in the M step; we will denote this operation by $\theta(\boldsymbol{\mu})$. (For example, in the case of mixtures of multinoullis, we just need to normalize $\mathbf{a}$ and each row of $\mathbf{B}$.) With this notation under our belt, the pseudo code for batch EM is as shown in Algorithm 8.

Algorithm 11.2: Batch EM algorithm

```

1 initialize  $\mu$ ;
2 repeat
3    $\mu^{new} = \mathbf{0}$  ;
4   for each example  $i = 1 : N$  do
5      $\mathbf{s}_i := \sum_{\mathbf{z}} p(\mathbf{z}|\mathbf{x}_i, \theta(\mu)) \phi(\mathbf{x}_i, \mathbf{z})$  ;
6      $\mu^{new} := \mu^{new} + \mathbf{s}_i$  ;
7    $\mu := \mu^{new}$ ;
8 until converged;

```

11.4.8.2 Incremental EM

In incremental EM (Neal and Hinton 1998), we keep track of μ as well as the $\mathbf{s}_i$. When we come to a data case, we swap out the old $\mathbf{s}_i$ and replace it with the new $\mathbf{s}_i^{new}$, as shown in the code in Algorithm 8. Note that we can exploit the sparsity of $\mathbf{s}_i^{new}$ to speedup the computation of θ , since most components of μ will not have changed.

Algorithm 11.3: Incremental EM algorithm

```

1 initialize  $\mathbf{s}_i$  for  $i = 1 : N$ ;
2  $\mu = \sum_i \mathbf{s}_i$ ;
3 repeat
4   for each example  $i = 1 : N$  in a random order do
5      $\mathbf{s}_i^{new} := \sum_{\mathbf{z}} p(\mathbf{z}|\mathbf{x}_i, \theta(\mu)) \phi(\mathbf{x}_i, \mathbf{z})$  ;
6      $\mu := \mu + \mathbf{s}_i^{new} - \mathbf{s}_i$ ;
7      $\mathbf{s}_i := \mathbf{s}_i^{new}$ ;
8 until converged;

```

This can be viewed as maximizing the lower bound $Q(\theta, q_1, \dots, q_N)$ by optimizing q_1 , then θ , then q_2 , then θ , etc. As such, this method is guaranteed to monotonically converge to a local maximum of the lower bound and to the log likelihood itself.

11.4.8.3 Stepwise EM

In stepwise EM, whenever we compute a new $\mathbf{s}_i$, we move μ towards it, as shown in Algorithm 7.² At iteration k , the stepsize has value η_k , which must satisfy the Robbins-Monro conditions in Equation 8.82. For example, (Liang and Klein Liang and Klein) use $\eta_k = (2 + k)^{-\kappa}$ for $0.5 < \kappa \leq 1$. We can get somewhat better behavior by using a minibatch of size m before each update. It is possible to optimize m and κ to maximize the training set likelihood, by

2. A detail: As written the update for μ does not exploit the sparsity of $\mathbf{s}_i$. We can fix this by storing $\mathbf{m} = \frac{\mu}{\prod_{j < k} (1 - \eta_j)}$ instead of μ , and then using the sparse update $\mathbf{m} := \mathbf{m} + \frac{\eta_k}{\prod_{j < k} (1 - \eta_j)} \mathbf{s}_i$. This will not affect the results (i.e., $\theta(\mu) = \theta(\mathbf{m})$), since scaling the counts by a global constant has no effect.

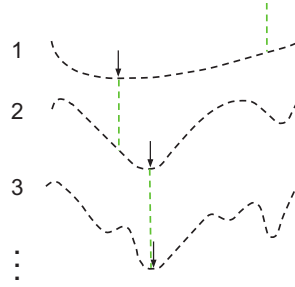


Figure 11.17 Illustration of deterministic annealing. Based on http://en.wikipedia.org/wiki/Graduated_optimization.

trying different values in parallel for an initial trial period; this can significantly speed up the algorithm.

Algorithm 11.4: Stepwise EM algorithm

```

1 initialize  $\mu$ ;  $k = 0$  ;
2 repeat
3   for each example  $i = 1 : N$  in a random order do
4      $\mathbf{s}_i := \sum_{\mathbf{z}} p(\mathbf{z}|\mathbf{x}_i, \boldsymbol{\theta}(\mu)) \phi(\mathbf{x}_i, \mathbf{z})$  ;
5      $\mu := (1 - \eta_k)\mu + \eta_k \mathbf{s}_i$ ;
6    $k := k + 1$ 
7 until converged;
```

(Liang and Klein Liang and Klein) compare batch EM, incremental EM, and stepwise EM on four different unsupervised language modeling tasks. They found that stepwise EM (using $\kappa \approx 0.7$ and $m \approx 1000$) was faster than incremental EM, and both were much faster than batch EM. In terms of accuracy, stepwise EM was usually as good or sometimes even better than batch EM; incremental EM was often worse than either of the other methods.

11.4.9 Other EM variants *

EM is one of the most widely used algorithms in statistics and machine learning. Not surprisingly, many variations have been proposed. We briefly mention a few below, some of which we will use in later chapters. See (McLachlan and Krishnan 1997) for more information.

- **Annealed EM** In general, EM will only converge to a local maximum. To increase the chance of finding the global maximum, we can use a variety of methods. One approach is to use a method known as **deterministic annealing** (Rose 1998). The basic idea is to “smooth” the posterior “landscape” by raising it to a temperature, and then gradually cooling it, all the while slowly tracking the global maximum. See Figure 11.17. for a sketch. (A stochastic version

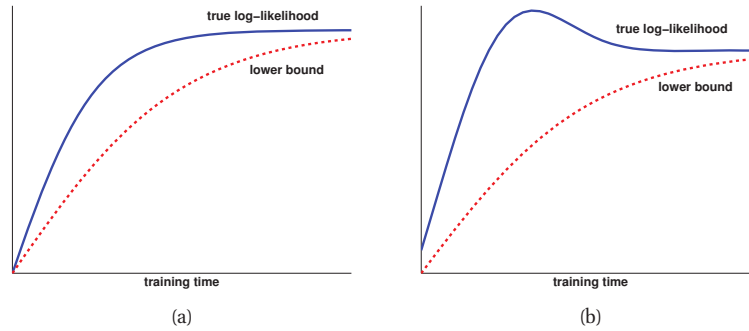


Figure 11.18 Illustration of possible behaviors of variational EM. (a) The lower bound increases at each iteration, and so does the likelihood. (b) The lower bound increases but the likelihood decreases. In this case, the algorithm is closing the gap between the approximate and true posterior. This can have a regularizing effect. Based on Figure 6 of (Saul et al. 1996). Figure generated by `varEMbound`.

of this algorithm is described in Section 24.6.1.) An annealed version of EM is described in (Ueda and Nakano 1998).

- **Variational EM** In Section 11.4.7, we showed that the optimal thing to do in the E step is to make q_i be the exact posterior over the latent variables, $q_i^t(\mathbf{z}_i) = p(\mathbf{z}_i|\mathbf{x}_i, \boldsymbol{\theta}^t)$. In this case, the lower bound on the log likelihood will be tight, so the M step will “push up” on the log-likelihood itself. However, sometimes it is computationally intractable to perform exact inference in the E step, but we may be able to perform approximate inference. If we can ensure that the E step is performing inference based on a lower bound to the likelihood, then the M step can be seen as monotonically increasing this lower bound (see Figure 11.18). This is called **variational EM** (Neal and Hinton 1998). See Chapter 21 for some variational inference methods that can be used in the E step.
- **Monte Carlo EM** Another approach to handling an intractable E step is to use a Monte Carlo approximation to the expected sufficient statistics. That is, we draw samples from the posterior, $\mathbf{z}_i^s \sim p(\mathbf{z}_i|\mathbf{x}_i, \boldsymbol{\theta}^t)$, and then compute the sufficient statistics for each completed vector, $(\mathbf{x}_i, \mathbf{z}_i^s)$, and then average the results. This is called **Monte Carlo EM** or **MCEM** (Wei and Tanner 1990). (If we only draw a single sample, it is called **stochastic EM** (Celeux and Diebolt 1985).) One way to draw samples is to use MCMC (see Chapter 24). However, if we have to wait for MCMC to converge inside each E step, the method becomes very slow. An alternative is to use stochastic approximation, and only perform “brief” sampling in the E step, followed by a partial parameter update. This is called **stochastic approximation EM** (Delyon et al. 1999) and tends to work better than MCEM. Another alternative is to apply MCMC to infer the parameters as well as the latent variables (a fully Bayesian approach), thus eliminating the distinction between E and M steps. See Chapter 24 for details.
- **Generalized EM** Sometimes we can perform the E step exactly, but we cannot perform the M step exactly. However, we can still monotonically increase the log likelihood by performing a “partial” M step, in which we merely increase the expected complete data log likelihood, rather than maximizing it. For example, we might follow a few gradient steps. This is called

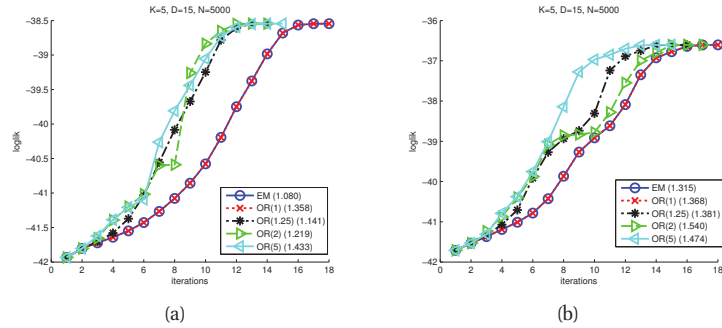


Figure 11.19 Illustration of adaptive over-relaxed EM applied to a mixture of 5 Gaussians in 15 dimensions. We show the algorithm applied to two different datasets, randomly sampled from a mixture of 10 Gaussians. We plot the convergence for different update rates η . Using $\eta = 1$ gives the same results as regular EM. The actual running time is printed in the legend. Figure generated by `mixGaussOverRelaxedEmDemo`.

the **generalized EM** or **GEM** algorithm. (This is an unfortunate term, since there are many ways to generalize EM....)

- **ECM(E) algorithm** The **ECM** algorithm stands for “expectation conditional maximization”, and refers to optimizing the parameters in the M step sequentially, if they turn out to be dependent. The **ECME** algorithm, which stands for “ECM either” (Liu and Rubin 1995), is a variant of ECM in which we maximize the expected complete data log likelihood (the Q function) as usual, or the observed data log likelihood, during one or more of the conditional maximization steps. The latter can be much faster, since it ignores the results of the E step, and directly optimizes the objective of interest. A standard example of this is when fitting the Student T distribution. For fixed ν , we can update Σ as usual, but then to update ν , we replace the standard update of the form $\nu^{t+1} = \arg \max_{\nu} Q((\mu^{t+1}, \Sigma^{t+1}, \nu), \theta^t)$ with $\nu^{t+1} = \arg \max_{\nu} \log p(\mathcal{D} | \mu^{t+1}, \Sigma^{t+1}, \nu)$. See (McLachlan and Krishnan 1997) for more information.
- **Over-relaxed EM** Vanilla EM can be quite slow, especially if there is lots of missing data. The adaptive **overrelaxed EM algorithm** (Salakhutdinov and Roweis 2003) performs an update of the form $\theta^{t+1} = \theta^t + \eta(M(\theta^t) - \theta^t)$, where η is a step-size parameter, and $M(\theta^t)$ is the usual update computed during the M step. Obviously this reduces to standard EM if $\eta = 1$, but using larger values of η can result in faster convergence. See Figure 11.19 for an illustration. Unfortunately, using too large a value of η can cause the algorithm to fail to converge.

Finally, note that EM is in fact just a special case of a larger class of algorithms known as **bound optimization** or **MM** algorithms (MM stands for **minorize-maximize**). See (Hunter and Lange 2004) for further discussion.

11.5 Model selection for latent variable models

When using LVMs, we must specify the number of latent variables, which controls the model complexity. In particular, in the case of mixture models, we must specify K , the number of clusters. Choosing these parameters is an example of model selection. We discuss some approaches below.

11.5.1 Model selection for probabilistic models

The optimal Bayesian approach, discussed in Section 5.3, is to pick the model with the largest marginal likelihood, $K^* = \operatorname{argmax}_k p(\mathcal{D}|K)$.

There are two problems with this. First, evaluating the marginal likelihood for LVMs is quite difficult. In practice, simple approximations, such as BIC, can be used (see e.g., (Fraley and Raftery 2002)). Alternatively, we can use the cross-validated likelihood as a performance measure, although this can be slow, since it requires fitting each model F times, where F is the number of CV folds.

The second issue is the need to search over a potentially large number of models. The usual approach is to perform exhaustive search over all candidate values of K . However, sometimes we can set the model to its maximal size, and then rely on the power of the Bayesian Occam's razor to "kill off" unwanted components. An example of this will be shown in Section 21.6.1.6, when we discuss variational Bayes.

An alternative approach is to perform stochastic sampling in the space of models. Traditional approaches, such as (Green 1998, 2003; Lunn et al. 2009), are based on reversible jump MCMC, and use birth moves to propose new centers, and death moves to kill off old centers. However, this can be slow and difficult to implement. A simpler approach is to use a Dirichlet process mixture model, which can be fit using Gibbs sampling, but still allows for an unbounded number of mixture components; see Section 25.2 for details.

Perhaps surprisingly, these sampling-based methods can be faster than the simple approach of evaluating the quality of each K separately. The reason is that fitting the model for each K is often slow. By contrast, the sampling methods can often quickly determine that a certain value of K is poor, and thus they need not waste time in that part of the posterior.

11.5.2 Model selection for non-probabilistic methods

What if we are not using a probabilistic model? For example, how do we choose K for the K -means algorithm? Since this does not correspond to a probability model, there is no likelihood, so none of the methods described above can be used.

An obvious proxy for the likelihood is the reconstruction error. Define the squared reconstruction error of a data set $\mathcal{D}$, using model complexity K , as follows:

$$E(\mathcal{D}, K) = \frac{1}{|\mathcal{D}|} \sum_{i \in \mathcal{D}} \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 \quad (11.95)$$

In the case of K -means, the reconstruction is given by $\hat{\mathbf{x}}_i = \boldsymbol{\mu}_{z_i}$, where $z_i = \operatorname{argmin}_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2$, as explained in Section 11.4.2.6.

Figure 11.20(a) plots the reconstruction error on the *test set* for K -means. We notice that the error decreases with increasing model complexity! The reason for this behavior is as follows:

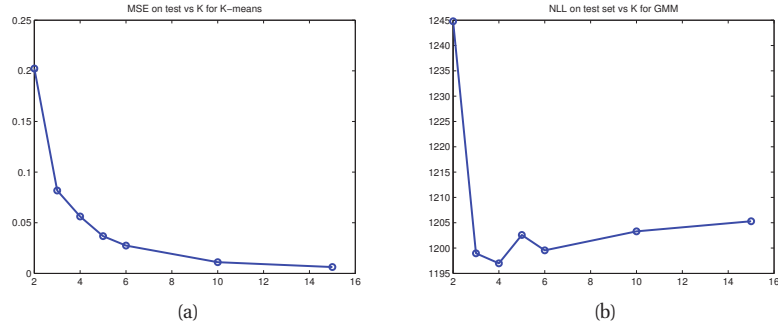


Figure 11.20 Test set performance vs K for data generated from a mixture of 3 Gaussians in 1d (data is shown in Figure 11.21(a)). (a) MSE on test set for K-means. (b) Negative log likelihood on test set for GMM. Figure generated by `kmeansModelSel1d`.

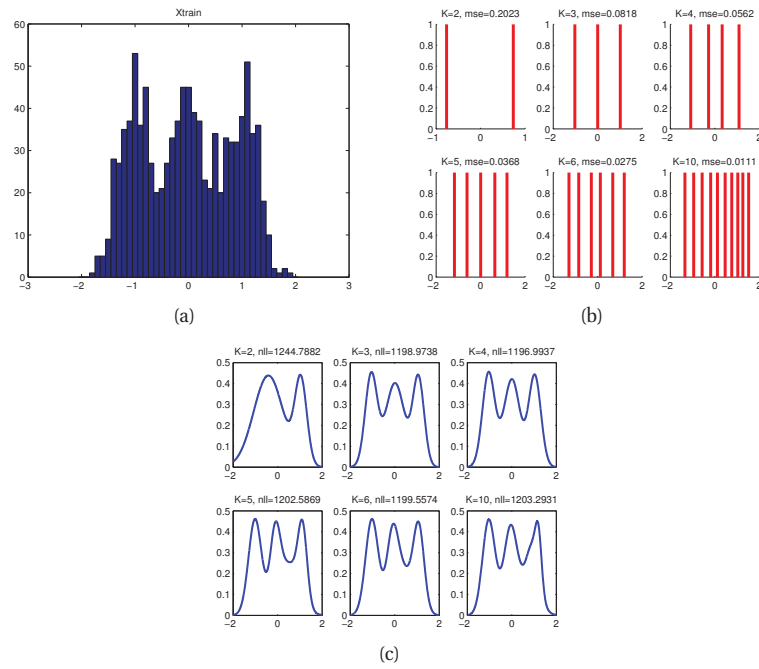


Figure 11.21 Synthetic data generated from a mixture of 3 Gaussians in 1d. (a) Histogram of training data. (Test data looks essentially the same.) (b) Centroids estimated by K-means for $K \in \{2, 3, 4, 5, 6, 10\}$. (c) GMM density model estimated by EM for the same values of K . Figure generated by `kmeansModelSel1d`.

when we add more and more centroids to K -means, we can “tile” the space more densely, as shown in Figure 11.21(b). Hence any given test vector is more likely to find a close prototype to accurately represent it as K increases, thus decreasing reconstruction error. However, if we use a probabilistic model, such as the GMM, and plot the negative log-likelihood, we get the usual U-shaped curve on the test set, as shown in Figure 11.20(b).

In supervised learning, we can always use cross validation to select between non-probabilistic models of different complexity, but this is not the case with unsupervised learning. Although this is not a novel observation (e.g., it is mentioned in passing in (Hastie et al. 2009, p519), one of the standard references in this field), it is perhaps not as widely appreciated as it should be. In fact, it is one of the more compelling arguments in favor of probabilistic models.

Given that cross validation doesn’t work, and supposing one is unwilling to use probabilistic models (for some bizarre reason...), how can one choose K ? The most common approach is to plot the reconstruction error on the training set versus K , and to try to identify a **knee** or **kink** in the curve. The idea is that for $K < K^*$, where K^* is the “true” number of clusters, the rate of decrease in the error function will be high, since we are splitting apart things that should not be grouped together. However, for $K > K^*$, we are splitting apart “natural” clusters, which does not reduce the error by as much.

This kink-finding process can be automated by use of the **gap statistic** (Tibshirani et al. 2001). Nevertheless, identifying such kinks can be hard, as shown in Figure 11.20(a), since the loss function usually drops off gradually. A different approach to “kink finding” is described in Section 12.3.2.1.

11.6 Fitting models with missing data

Suppose we want to fit a joint density model by maximum likelihood, but we have “holes” in our data matrix, due to missing data (usually represented by NaNs). More formally, let $O_{ij} = 1$ if component j of data case i is observed, and let $O_{ij} = 0$ otherwise. Let $\mathbf{X}_v = \{x_{ij} : O_{ij} = 1\}$ be the visible data, and $\mathbf{X}_h = \{x_{ij} : O_{ij} = 0\}$ be the missing or hidden data. Our goal is to compute

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} p(\mathbf{X}_v | \boldsymbol{\theta}, \mathbf{O}) \quad (11.96)$$

Under the missing at random assumption (see Section 8.6.2), we have

$$p(\mathbf{X}_v | \boldsymbol{\theta}, \mathbf{O}) = \prod_{i=1}^N p(\mathbf{x}_{iv} | \boldsymbol{\theta}) \quad (11.97)$$

where $\mathbf{x}_{iv}$ is a vector created from row i and the columns indexed by the set $\{j : O_{ij} = 1\}$. Hence the log-likelihood has the form

$$\log p(\mathbf{X}_v | \boldsymbol{\theta}) = \sum_i \log p(\mathbf{x}_{iv} | \boldsymbol{\theta}) \quad (11.98)$$

where

$$p(\mathbf{x}_{iv} | \boldsymbol{\theta}) = \sum_{\mathbf{x}_{ih}} p(\mathbf{x}_{iv}, \mathbf{x}_{ih} | \boldsymbol{\theta}) \quad (11.99)$$

and $\mathbf{x}_{ih}$ is the vector of hidden variables for case i (assumed discrete for notational simplicity). Substituting in, we get

$$\log p(\mathbf{X}_v | \boldsymbol{\theta}) = \sum_i \log \left[\sum_{\mathbf{x}_{ih}} p(\mathbf{x}_{iv}, \mathbf{x}_{ih} | \boldsymbol{\theta}) \right] \quad (11.100)$$

Unfortunately, this objective is hard to maximize. since we cannot push the log inside the sum. However, we can use the EM algorithm to compute a local optimum. We give an example of this below.

11.6.1 EM for the MLE of an MVN with missing data

Suppose we want to fit an MVN by maximum likelihood, but we have missing data. We can use EM to find a local maximum of the objective, as we explain below.

11.6.1.1 Getting started

To get the algorithm started, we can compute the MLE based on those rows of the data matrix that are fully observed. If there are no such rows, we can use some ad-hoc imputation procedures, and then compute an initial MLE.

11.6.1.2 E step

Once we have $\boldsymbol{\theta}^{t-1}$, we can compute the expected complete data log likelihood at iteration t as follows:

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{t-1}) = \mathbb{E} \left[\sum_{i=1}^N \log \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}, \boldsymbol{\Sigma}) | \mathcal{D}, \boldsymbol{\theta}^{t-1} \right] \quad (11.101)$$

$$= -\frac{N}{2} \log |2\pi \boldsymbol{\Sigma}| - \frac{1}{2} \sum_i \mathbb{E} [(\mathbf{x}_i - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x}_i - \boldsymbol{\mu})] \quad (11.102)$$

$$= -\frac{N}{2} \log |2\pi \boldsymbol{\Sigma}| - \frac{1}{2} \text{tr}(\boldsymbol{\Sigma}^{-1} \sum_i \mathbb{E} [(\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^T]) \quad (11.103)$$

$$= -\frac{N}{2} \log |\boldsymbol{\Sigma}| - \frac{ND}{2} \log(2\pi) - \frac{1}{2} \text{tr}(\boldsymbol{\Sigma}^{-1} \mathbb{E} [\mathbf{S}(\boldsymbol{\mu})]) \quad (11.104)$$

where

$$\mathbb{E} [\mathbf{S}(\boldsymbol{\mu})] \triangleq \sum_i \left(\mathbb{E} [\mathbf{x}_i \mathbf{x}_i^T] + \boldsymbol{\mu} \boldsymbol{\mu}^T - 2\boldsymbol{\mu} \mathbb{E} [\mathbf{x}_i]^T \right) \quad (11.105)$$

(We drop the conditioning of the expectation on $\mathcal{D}$ and $\boldsymbol{\theta}^{t-1}$ for brevity.) We see that we need to compute $\sum_i \mathbb{E} [\mathbf{x}_i]$ and $\sum_i \mathbb{E} [\mathbf{x}_i \mathbf{x}_i^T]$; these are the expected sufficient statistics.

To compute these quantities, we use the results from Section 4.3.1. Specifically, consider case i , where components v are observed and components h are unobserved. We have

$$\mathbf{x}_{ih} | \mathbf{x}_{iv}, \boldsymbol{\theta} \sim \mathcal{N}(\mathbf{m}_i, \mathbf{V}_i) \quad (11.106)$$

$$\mathbf{m}_i \triangleq \boldsymbol{\mu}_h + \boldsymbol{\Sigma}_{hv} \boldsymbol{\Sigma}_{vv}^{-1} (\mathbf{x}_{iv} - \boldsymbol{\mu}_v) \quad (11.107)$$

$$\mathbf{V}_i \triangleq \boldsymbol{\Sigma}_{hh} - \boldsymbol{\Sigma}_{hv} \boldsymbol{\Sigma}_{vv}^{-1} \boldsymbol{\Sigma}_{vh} \quad (11.108)$$

Hence the expected sufficient statistics are

$$\mathbb{E}[\mathbf{x}_i] = (\mathbb{E}[\mathbf{x}_{ih}]; \mathbf{x}_{iv}) = (\mathbf{m}_i; \mathbf{x}_{iv}) \quad (11.109)$$

where we have assumed (without loss of generality) that the unobserved variables come before the observed variables in the node ordering.

To compute $\mathbb{E}[\mathbf{x}_i \mathbf{x}_i^T]$, we use the result that $\text{cov}[\mathbf{x}] = \mathbb{E}[\mathbf{x} \mathbf{x}^T] - \mathbb{E}[\mathbf{x}] \mathbb{E}[\mathbf{x}^T]$. Hence

$$\mathbb{E}[\mathbf{x}_i \mathbf{x}_i^T] = \mathbb{E}\left[\begin{pmatrix} \mathbf{x}_{ih} \\ \mathbf{x}_{iv} \end{pmatrix} \begin{pmatrix} \mathbf{x}_{ih}^T & \mathbf{x}_{iv}^T \end{pmatrix}\right] = \begin{pmatrix} \mathbb{E}[\mathbf{x}_{ih} \mathbf{x}_{ih}^T] & \mathbb{E}[\mathbf{x}_{ih} \mathbf{x}_{iv}^T] \\ \mathbf{x}_{iv} \mathbb{E}[\mathbf{x}_{ih}]^T & \mathbf{x}_{iv} \mathbf{x}_{iv}^T \end{pmatrix} \quad (11.110)$$

$$\mathbb{E}[\mathbf{x}_{ih} \mathbf{x}_{ih}^T] = \mathbb{E}[\mathbf{x}_{ih}] \mathbb{E}[\mathbf{x}_{ih}]^T + \mathbf{V}_i \quad (11.111)$$

11.6.1.3 M step

By solving $\nabla Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t-1)}) = \mathbf{0}$, we can show that the M step is equivalent to plugging these ESS into the usual MLE equations to get

$$\boldsymbol{\mu}^t = \frac{1}{N} \sum_i \mathbb{E}[\mathbf{x}_i] \quad (11.112)$$

$$\boldsymbol{\Sigma}^t = \frac{1}{N} \sum_i \mathbb{E}[\mathbf{x}_i \mathbf{x}_i^T] - \boldsymbol{\mu}^t (\boldsymbol{\mu}^t)^T \quad (11.113)$$

Thus we see that EM is *not* equivalent to simply replacing variables by their expectations and applying the standard MLE formula; that would ignore the posterior variance and would result in an incorrect estimate. Instead we must compute the expectation of the sufficient statistics, and plug that into the usual equation for the MLE. We can easily modify the algorithm to perform MAP estimation, by plugging in the ESS into the equation for the MAP estimate. For an implementation, see `gaussMissingFitEm`.

11.6.1.4 Example

As an example of this procedure in action, let us reconsider the imputation problem from Section 4.3.2.3, which had $N = 100$ 10-dimensional data cases, with 50% missing data. Let us fit the parameters using EM. Call the resulting parameters $\hat{\boldsymbol{\theta}}$. We can use our model for predictions by computing $\mathbb{E}[\mathbf{x}_{ih} | \mathbf{x}_{iv}, \hat{\boldsymbol{\theta}}]$. Figure 11.22(a-b) indicates that the results obtained using the learned parameters are almost as good as with the true parameters. Not surprisingly, performance improves with more data, or as the fraction of missing data is reduced.

11.6.1.5 Extension to the GMM case

It is straightforward to fit a mixture of Gaussians in the presence of partially observed data vectors $\mathbf{x}_i$. We leave the details as an exercise.

Exercises

Exercise 11.1 Student T as infinite mixture of Gaussians

Derive Equation 11.61. For simplicity, assume a one-dimensional distribution.

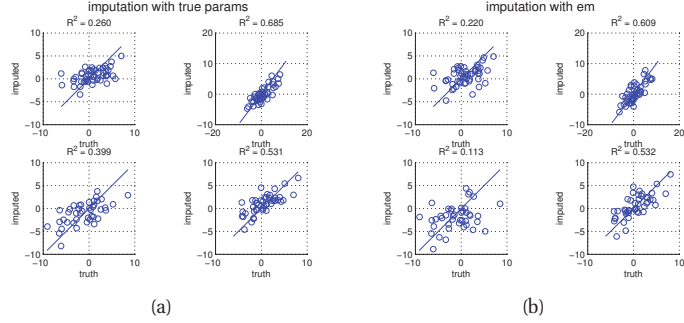


Figure 11.22 Illustration of data imputation. (a) Scatter plot of true values vs imputed values using true parameters. (b) Same as (a), but using parameters estimated with EM. Figure generated by `gaussImputationDemo`.

Exercise 11.2 EM for mixtures of Gaussians

Show that the M step for ML estimation of a mixture of Gaussians is given by

$$\boldsymbol{\mu}_k = \frac{\sum_i r_{ik} \mathbf{x}_i}{r_k} \quad (11.114)$$

$$\boldsymbol{\Sigma}_k = \frac{\sum_i r_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^T}{r_k} = \frac{\sum_i r_{ik} \mathbf{x}_i \mathbf{x}_i^T - r_k \boldsymbol{\mu}_k \boldsymbol{\mu}_k^T}{r_k} \quad (11.115)$$

Exercise 11.3 EM for mixtures of Bernoullis

- Show that the M step for ML estimation of a mixture of Bernoullis is given by

$$\mu_{kj} = \frac{\sum_i r_{ik} x_{ij}}{\sum_i r_{ik}} \quad (11.116)$$

- Show that the M step for MAP estimation of a mixture of Bernoullis with a $\beta(\alpha, \beta)$ prior is given by

$$\mu_{kj} = \frac{(\sum_i r_{ik} x_{ij}) + \alpha - 1}{(\sum_i r_{ik}) + \alpha + \beta - 2} \quad (11.117)$$

Exercise 11.4 EM for mixture of Student distributions

Derive the EM algorithm for ML estimation of a mixture of multivariate Student T distributions.

Exercise 11.5 Gradient descent for fitting GMM

Consider the Gaussian mixture model

$$p(\mathbf{x}|\boldsymbol{\theta}) = \sum_k \pi_k \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \quad (11.118)$$

Define the log likelihood as

$$\ell(\boldsymbol{\theta}) = \sum_{n=1}^N \log p(\mathbf{x}_n|\boldsymbol{\theta}) \quad (11.119)$$

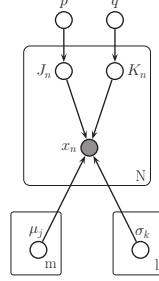


Figure 11.23 A mixture of Gaussians with two discrete latent indicators. J_n specifies which mean to use, and K_n specifies which variance to use.

Define the posterior responsibility that cluster k has for datapoint n as follows:

$$r_{nk} \triangleq p(z_n = k | \mathbf{x}_n, \boldsymbol{\theta}) = \frac{\pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{k'=1}^K \pi_{k'} \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_{k'}, \boldsymbol{\Sigma}_{k'})} \quad (11.120)$$

- a. Show that the gradient of the log-likelihood wrt $\boldsymbol{\mu}_k$ is

$$\frac{d}{d\boldsymbol{\mu}_k} \ell(\boldsymbol{\theta}) = \sum_n r_{nk} \boldsymbol{\Sigma}_k^{-1} (\mathbf{x}_n - \boldsymbol{\mu}_k) \quad (11.121)$$

- b. Derive the gradient of the log-likelihood wrt π_k . (For now, ignore any constraints on π_k .)
c. One way to handle the constraint that $\sum_{k=1}^K \pi_k = 1$ is to reparameterize using the softmax function:

$$\pi_k \triangleq \frac{e^{w_k}}{\sum_{k'=1}^K e^{w_{k'}}} \quad (11.122)$$

Here $w_k \in \mathbb{R}$ are unconstrained parameters. Show that

$$\frac{d}{dw_k} \ell(\boldsymbol{\theta}) = \sum_n r_{nk} - \pi_k \quad (11.123)$$

(There may be a constant factor missing in the above expression...) Hint: use the chain rule and the fact that

$$\frac{d\pi_j}{dw_k} = \begin{cases} \pi_j(1 - \pi_j) & \text{if } j = k \\ -\pi_j \pi_k & \text{if } j \neq k \end{cases} \quad (11.124)$$

which follows from Exercise 8.4(1).

- d. Derive the gradient of the log-likelihood wrt $\boldsymbol{\Sigma}_k$. (For now, ignore any constraints on $\boldsymbol{\Sigma}_k$.)
e. One way to handle the constraint that $\boldsymbol{\Sigma}_k$ be a symmetric positive definite matrix is to reparameterize using a Cholesky decomposition, $\boldsymbol{\Sigma}_k = \mathbf{R}_k^T \mathbf{R}_k$, where $\mathbf{R}_k$ is an upper-triangular, but otherwise unconstrained matrix. Derive the gradient of the log-likelihood wrt $\mathbf{R}_k$.

Exercise 11.6 EM for a finite scale mixture of Gaussians

(Source: Jaakkola.) Consider the graphical model in Figure 11.23 which defines the following:

$$p(x_n | \boldsymbol{\theta}) = \sum_{j=1}^m p_j \left[\sum_{k=1}^l q_k N(x_n | \mu_j, \sigma_k^2) \right] \quad (11.125)$$

where $\theta = \{p_1, \dots, p_m, \mu_1, \dots, \mu_m, q_1, \dots, q_l, \sigma_1^2, \dots, \sigma_l^2\}$ are all the parameters. Here $p_j \triangleq P(J_n = j)$ and $q_k \triangleq P(K_n = k)$ are the equivalent of mixture weights. We can think of this as a mixture of m non-Gaussian components, where each component distribution is a scale mixture, $p(x|j; \theta) = \sum_{k=1}^l q_k N(x; \mu_j, \sigma_k^2)$, combining Gaussians with different variances (scales).

We will now derive a generalized EM algorithm for this model. (Recall that in generalized EM, we do a partial update in the M step, rather than finding the exact maximum.)

- Derive an expression for the responsibilities, $P(J_n = j, K_n = k|x_n, \theta)$, needed for the E step.
- Write out a full expression for the expected complete log-likelihood

$$Q(\theta^{new}, \theta^{old}) = E_{\theta^{old}} \sum_{n=1}^N \log P(J_n, K_n, x_n | \theta^{new}) \quad (11.126)$$

- Solving the M-step would require us to jointly optimize the means $\mu_1, \dots, \mu_m$ and the variances $\sigma_1^2, \dots, \sigma_l^2$. It will turn out to be simpler to first solve for the μ_j 's given fixed σ_j^2 's, and subsequently solve for σ_j^2 's given the new values of μ_j 's. For brevity, we will just do the first part. Derive an expression for the maximizing μ_j 's given fixed $\sigma_{1:l}^2$, i.e., solve $\frac{\partial Q}{\partial \mu^{new}} = 0$.

Exercise 11.7 Manual calculation of the M step for a GMM

(Source: de Freitas.) In this question we consider clustering 1D data with a mixture of 2 Gaussians using the EM algorithm. You are given the 1-D data points $x = [1 \quad 10 \quad 20]$. Suppose the output of the E step is the following matrix:

$$\mathbf{R} = \begin{bmatrix} 1 & 0 \\ 0.4 & 0.6 \\ 0 & 1 \end{bmatrix} \quad (11.127)$$

where entry $r_{i,c}$ is the probability of observation x_i belonging to cluster c (the responsibility of cluster c for data point i). You just have to compute the M step. You may state the equations for maximum likelihood estimates of these quantities (which you should know) without proof; you just have to apply the equations to this data set. You may leave your answer in fractional form. Show your work.

- Write down the likelihood function you are trying to optimize.
- After performing the M step for the mixing weights π_1, π_2 , what are the new values?
- After performing the M step for the means μ_1 and μ_2 , what are the new values?

Exercise 11.8 Moments of a mixture of Gaussians

Consider a mixture of K Gaussians

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \quad (11.128)$$

- Show that

$$\mathbb{E}[\mathbf{x}] = \sum_k \pi_k \boldsymbol{\mu}_k \quad (11.129)$$

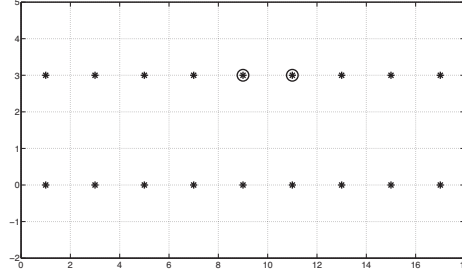


Figure 11.24 Some data points in 2d. Circles represent the initial guesses for $\mathbf{m}_1$ and $\mathbf{m}_2$.

b. Show that

$$\text{cov}[\mathbf{x}] = \sum_k \pi_k [\Sigma_k + \boldsymbol{\mu}_k \boldsymbol{\mu}_k^T] - \mathbb{E}[\mathbf{x}] \mathbb{E}[\mathbf{x}]^T \quad (11.130)$$

Hint: use the fact that $\text{cov}[\mathbf{x}] = \mathbb{E}[\mathbf{x}\mathbf{x}^T] - \mathbb{E}[\mathbf{x}] \mathbb{E}[\mathbf{x}]^T$.

Exercise 11.9 K-means clustering by hand

(Source: Jaakkola.)

In Figure 11.24, we show some data points which lie on the integer grid. (Note that the x-axis has been compressed; distances should be measured using the actual grid coordinates.) Suppose we apply the K-means algorithm to this data, using $K = 2$ and with the centers initialized at the two circled data points. Draw the final clusters obtained after K-means converges (show the approximate location of the new centers and group together all the points assigned to each center). Hint: think about shortest Euclidean distance.

Exercise 11.10 Deriving the K-means cost function

Show that

$$J_W(\mathbf{z}) = \frac{1}{2} \sum_{k=1}^K \sum_{i: z_i=k} \sum_{i': z_{i'}=k} (x_i - x_{i'})^2 = \sum_{k=1}^K n_k \sum_{i: z_i=k} (x_i - \bar{x}_k)^2 \quad (11.131)$$

Hint: note that, for any μ ,

$$\sum_i (x_i - \mu)^2 = \sum_i [(x_i - \bar{x}) - (\mu - \bar{x})]^2 \quad (11.132)$$

$$= \sum_i (x_i - \bar{x})^2 + \sum_i (\bar{x} - \mu)^2 - 2 \sum_i (x_i - \bar{x})(\mu - \bar{x}) \quad (11.133)$$

$$= ns^2 + n(\bar{x} - \mu)^2 \quad (11.134)$$

where $s^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$, since

$$\sum_i (x_i - \bar{x})(\mu - \bar{x}) = (\mu - \bar{x}) \left(\sum_i x_i - n\bar{x} \right) = (\mu - \bar{x})(n\bar{x} - n\bar{x}) = 0 \quad (11.135)$$

Exercise 11.11 Visible mixtures of Gaussians are in the exponential family

Show that the joint distribution $p(x, z|\boldsymbol{\theta})$ for a 1d GMM can be represented in exponential family form.

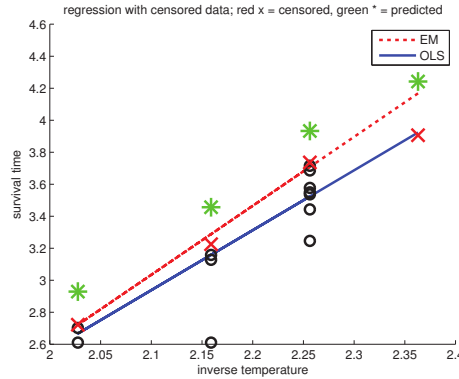


Figure 11.25 Example of censored linear regression. Black circles are observed training points, red crosses are observed but censored training points. Green stars are predicted values of the censored training points. We also show the lines fit by least squares (ignoring censoring) and by EM. Based on Figure 5.6 of (Tanner 1996). Figure generated by `linregCensoredSchmeeHahnDemo`, written by Hannes Bretschneider.

Exercise 11.12 EM for robust linear regression with a Student t likelihood

Consider a model of the form

$$p(y_i | \mathbf{x}_i, \mathbf{w}, \sigma^2, \nu) = \mathcal{T}(y_i | \mathbf{w}^T \mathbf{x}_i, \sigma^2, \nu) \quad (11.136)$$

Derive an EM algorithm to compute the MLE for $\mathbf{w}$. You may assume ν and σ^2 are fixed, for simplicity. Hint: see Section 11.4.5.

Exercise 11.13 EM for EB estimation of Gaussian shrinkage model

Extend the results of Section 5.6.2.2 to the case where the σ_j^2 are not equal (but are known). Hint: treat the θ_j as hidden variables, and then to integrate them out in the E step, and maximize $\boldsymbol{\eta} = (\mu, \tau^2)$ in the M step.

Exercise 11.14 EM for censored linear regression

Censored regression refers to the case where one knows the outcome is at least (or at most) a certain value, but the precise value is unknown. This arises in many different settings. For example, suppose one is trying to learn a model that can predict how long a program will take to run, for different settings of its parameters. One may abort certain runs if they seem to be taking too long; the resulting run times are said to be **right censored**. For such runs, all we know is that $y_i \geq c_i$, where c_i is the censoring time, that is, $y_i = \min(z_i, c_i)$, where z_i is the true running time and y_i is the observed running time. We can also define **left censored** and **interval censored** models.³ Derive an EM algorithm for fitting a linear regression model to right-censored data. Hint: use the results from Exercise 11.15. See Figure 11.25 for an example, based on the data from (Schmee and Hahn 1979). We notice that the EM line is tilted upwards more, since the model takes into account the fact that the truncated values are actually higher than the observed values.

3. There is a closely related model in econometrics called the **Tobit model**, in which $y_i = \max(z_i, 0)$, so we only get to observe positive outcomes. An example of this is when z_i represents “desired investment”, and y_i is actual investment. Probit regression (Section 9.4) is another example.

Exercise 11.15 Posterior mean and variance of a truncated Gaussian

Let $z_i = \mu_i + \sigma\epsilon_i$, where $\epsilon_i \sim \mathcal{N}(0, 1)$. Sometimes, such as in probit regression or censored regression, we do not observe z_i , but we observe the fact that it is above some threshold, namely we observe the event $E = \mathbb{I}(z_i \geq c_i) = \mathbb{I}(\epsilon_i \geq \frac{c_i - \mu_i}{\sigma})$. (See Exercise 11.14 for details on censored regression, and Section 11.4.6 for probit regression.) Show that

$$\mathbb{E}[z_i | z_i \geq c_i] = \mu_i + \sigma H\left(\frac{c_i - \mu_i}{\sigma}\right) \quad (11.137)$$

and

$$\mathbb{E}[z_i^2 | z_i \geq c_i] = \mu_i^2 + \sigma^2 + \sigma(c_i + \mu_i)H\left(\frac{c_i - \mu_i}{\sigma}\right) \quad (11.138)$$

where we have defined

$$H(u) \triangleq \frac{\phi(u)}{1 - \Phi(u)} \quad (11.139)$$

and where $\phi(u)$ is the pdf of a standard Gaussian, and $\Phi(u)$ is its cdf.

Hint 1: we have $p(\epsilon_i | E) = \frac{p(\epsilon_i, E)}{p(E)}$, where E is some event of interest.

Hint 2: It can be shown that

$$\frac{d}{dw} \mathcal{N}(w|0, 1) = -w\mathcal{N}(w|0, 1) \quad (11.140)$$

and hence

$$\int_b^c w\mathcal{N}(w|0, 1) = \mathcal{N}(b|0, 1) - \mathcal{N}(c|0, 1) \quad (11.141)$$

12 *Latent linear models*

12.1 Factor analysis

One problem with mixture models is that they only use a single latent variable to generate the observations. In particular, each observation can only come from one of K prototypes. One can think of a mixture model as using K hidden binary variables, representing a one-hot encoding of the cluster identity. But because these variables are mutually exclusive, the model is still limited in its representational power.

An alternative is to use a vector of real-valued latent variables, $\mathbf{z}_i \in \mathbb{R}^L$. The simplest prior to use is a Gaussian (we will consider other choices later):

$$p(\mathbf{z}_i) = \mathcal{N}(\mathbf{z}_i | \boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0) \quad (12.1)$$

If the observations are also continuous, so $\mathbf{x}_i \in \mathbb{R}^D$, we may use a Gaussian for the likelihood. Just as in linear regression, we will assume the mean is a linear function of the (hidden) inputs, thus yielding

$$p(\mathbf{x}_i | \mathbf{z}_i, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{W}\mathbf{z}_i + \boldsymbol{\mu}, \boldsymbol{\Psi}) \quad (12.2)$$

where $\mathbf{W}$ is a $D \times L$ matrix, known as the **factor loading matrix**, and $\boldsymbol{\Psi}$ is a $D \times D$ covariance matrix. We take $\boldsymbol{\Psi}$ to be diagonal, since the whole point of the model is to “force” $\mathbf{z}_i$ to explain the correlation, rather than “baking it in” to the observation’s covariance. This overall model is called **factor analysis** or **FA**. The special case in which $\boldsymbol{\Psi} = \sigma^2 \mathbf{I}$ is called **probabilistic principal components analysis** or **PPCA**. The reason for this name will become apparent later.

The generative process, where $L = 1$, $D = 2$ and $\boldsymbol{\Psi}$ is diagonal, is illustrated in Figure 12.1. We take an isotropic Gaussian “spray can” and slide it along the 1d line defined by $\mathbf{w}z_i + \mu$. This induces an elongated (and hence correlated) Gaussian in 2d.

12.1.1 FA is a low rank parameterization of an MVN

FA can be thought of as a way of specifying a joint density model on $\mathbf{x}$ using a small number of parameters. To see this, note that from Equation 4.126, the induced marginal distribution $p(\mathbf{x}_i | \boldsymbol{\theta})$ is a Gaussian:

$$p(\mathbf{x}_i | \boldsymbol{\theta}) = \int \mathcal{N}(\mathbf{x}_i | \mathbf{W}\mathbf{z}_i + \boldsymbol{\mu}, \boldsymbol{\Psi}) \mathcal{N}(\mathbf{z}_i | \boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0) d\mathbf{z}_i \quad (12.3)$$

$$= \mathcal{N}(\mathbf{x}_i | \mathbf{W}\boldsymbol{\mu}_0 + \boldsymbol{\mu}, \boldsymbol{\Psi} + \mathbf{W}\boldsymbol{\Sigma}_0\mathbf{W}^T) \quad (12.4)$$

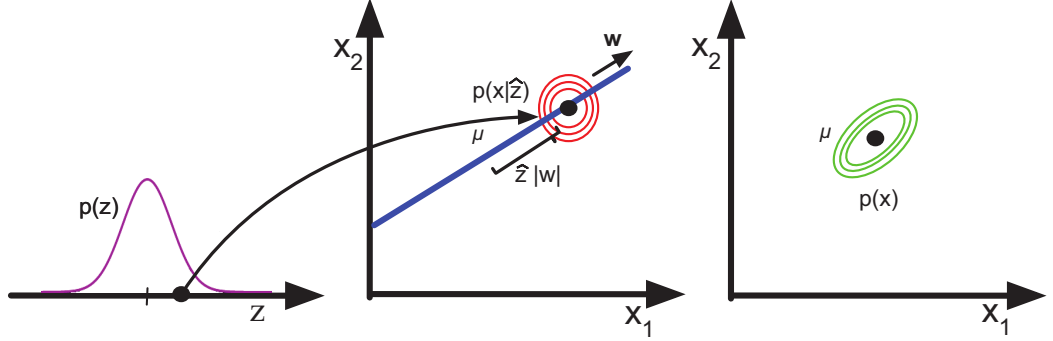


Figure 12.1 Illustration of the PPCA generative process, where we have $L = 1$ latent dimension generating $D = 2$ observed dimensions. Based on Figure 12.9 of (Bishop 2006b).

From this, we see that we can set $\mu_0 = \mathbf{0}$ without loss of generality, since we can always absorb $\mathbf{W}\mu_0$ into μ . Similarly, we can set $\Sigma_0 = \mathbf{I}$ without loss of generality, because we can always “emulate” a correlated prior by using defining a new weight matrix, $\tilde{\mathbf{W}} = \mathbf{W}\Sigma_0^{-\frac{1}{2}}$. Then we find

$$\text{cov}[\mathbf{x}|\boldsymbol{\theta}] = \tilde{\mathbf{W}}^T + \mathbb{E}[\epsilon\epsilon^T] = (\mathbf{W}\Sigma_0^{-\frac{1}{2}})\Sigma_0(\mathbf{W}\Sigma_0^{-\frac{1}{2}})^T + \Psi = \mathbf{W}\mathbf{W}^T + \Psi \quad (12.5)$$

We thus see that FA approximates the covariance matrix of the visible vector using a low-rank decomposition:

$$\mathbf{C} \triangleq \text{cov}[\mathbf{x}] = \mathbf{W}\mathbf{W}^T + \Psi \quad (12.6)$$

This only uses $O(LD)$ parameters, which allows a flexible compromise between a full covariance Gaussian, with $O(D^2)$ parameters, and a diagonal covariance, with $O(D)$ parameters. Note that if we did not restrict Ψ to be diagonal, we could trivially set Ψ to a full covariance matrix; then we could set $\mathbf{W} = \mathbf{0}$, in which case the latent factors would not be required.

12.1.2 Inference of the latent factors

Although FA can be thought of as just a way to define a density on $\mathbf{x}$, it is often used because we hope that the latent factors $\mathbf{z}$ will reveal something interesting about the data. To do this, we need to compute the posterior over the latent factors. We can use Bayes rule for Gaussians to give

$$p(\mathbf{z}_i|\mathbf{x}_i, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{z}_i|\mathbf{m}_i, \Sigma_i) \quad (12.7)$$

$$\Sigma_i \triangleq (\Sigma_0^{-1} + \mathbf{W}^T\Psi^{-1}\mathbf{W})^{-1} \quad (12.8)$$

$$\mathbf{m}_i \triangleq \Sigma_i(\mathbf{W}^T\Psi^{-1}(\mathbf{x}_i - \mu) + \Sigma_0^{-1}\mu_0) \quad (12.9)$$

Note that in the FA model, Σ_i is actually independent of i , so we can denote it by Σ . Computing this matrix takes $O(L^3 + L^2D)$ time, and computing each $\mathbf{m}_i = \mathbb{E}[\mathbf{z}_i|\mathbf{x}_i, \boldsymbol{\theta}]$ takes $O(L^2 + LD)$ time. The $\mathbf{m}_i$ are sometimes called the latent **scores**, or latent **factors**.

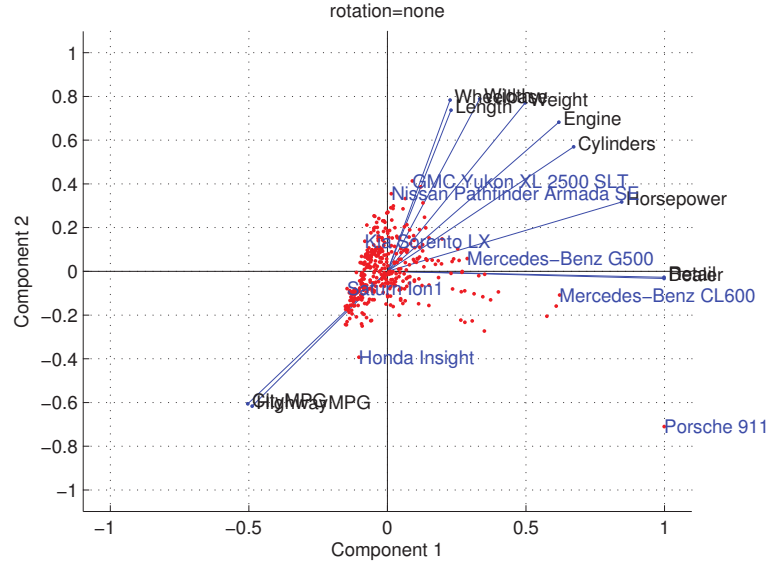


Figure 12.2 2D projection of 2004 cars data based on factor analysis. The blue text are the names of cars corresponding to certain chosen points. Figure generated by `faBiplotDemo`.

Let us give a simple example, based (Shalizi 2009). We consider a dataset of $D = 11$ variables and $N = 387$ cases describing various aspects of cars, such as the engine size, the number of cylinders, the miles per gallon (MPG), the price, etc. We first fit a $L = 2$ dimensional model. We can plot the $\mathbf{m}_i$ scores as points in $\mathbb{R}^2$, to visualize the data, as shown in Figure 12.2.

To get a better understanding of the “meaning” of the latent factors, we can project unit vectors corresponding to each of the feature dimensions, $\mathbf{e}_1 = (1, 0, \dots, 0)$, $\mathbf{e}_2 = (0, 1, 0, \dots, 0)$, etc. into the low dimensional space. These are shown as blue lines in Figure 12.2; this is known as a **biplot**. We see that the horizontal axis represents price, corresponding to the features labeled “dealer” and “retail”, with expensive cars on the right. The vertical axis represents fuel efficiency (measured in terms of MPG) versus size: heavy vehicles are less efficient and are higher up, whereas light vehicles are more efficient and are lower down. We can “verify” this interpretation by clicking on some points, and finding the closest exemplars in the training set, and printing their names, as in Figure 12.2. However, in general, interpreting latent variable models is fraught with difficulties, as we discuss in Section 12.1.3.

12.1.3 Unidentifiability

Just like with mixture models, FA is also unidentifiable. To see this, suppose $\mathbf{R}$ is an arbitrary orthogonal rotation matrix, satisfying $\mathbf{R}\mathbf{R}^T = \mathbf{I}$. Let us define $\tilde{\mathbf{W}} = \mathbf{W}\mathbf{R}$; then the likelihood

function of this modified matrix is the same as for the unmodified matrix, since

$$\text{cov}[\mathbf{x}] = \tilde{\mathbf{W}}\mathbb{E}[\mathbf{z}\mathbf{z}^T]\tilde{\mathbf{W}}^T + \mathbb{E}[\boldsymbol{\epsilon}\boldsymbol{\epsilon}^T] \quad (12.10)$$

$$= \mathbf{W}\mathbf{R}\mathbf{R}^T\mathbf{W}^T + \boldsymbol{\Psi} = \mathbf{W}\mathbf{W}^T + \boldsymbol{\Psi} \quad (12.11)$$

Geometrically, multiplying $\mathbf{W}$ by an orthogonal matrix is like rotating $\mathbf{z}$ before generating $\mathbf{x}$; but since $\mathbf{z}$ is drawn from an isotropic Gaussian, this makes no difference to the likelihood. Consequently, we cannot uniquely identify $\mathbf{W}$, and therefore cannot uniquely identify the latent factors, either.

To ensure a unique solution, we need to remove $L(L-1)/2$ degrees of freedom, since that is the number of orthonormal matrices of size $L \times L$.¹ In total, the FA model has $D + LD - L(L-1)/2$ free parameters (excluding the mean), where the first term arises from $\boldsymbol{\Psi}$. Obviously we require this to be less than or equal to $D(D+1)/2$, which is the number of parameters in an unconstrained (but symmetric) covariance matrix. This gives us an upper bound on L , as follows:

$$L_{max} = \lfloor D + 0.5(1 - \sqrt{1 + 8D}) \rfloor \quad (12.12)$$

For example, $D = 6$ implies $L \leq 3$. But we usually never choose this upper bound, since it would result in overfitting (see discussion in Section 12.3 on how to choose L).

Unfortunately, even if we set $L < L_{max}$, we still cannot uniquely identify the parameters, since the rotational ambiguity still exists. Non-identifiability does not affect the predictive performance of the model. However, it does affect the loading matrix, and hence the interpretation of the latent factors. Since factor analysis is often used to uncover structure in the data, this problem needs to be addressed. Here are some commonly used solutions:

- **Forcing $\mathbf{W}$ to be orthonormal** Perhaps the cleanest solution to the identifiability problem is to force $\mathbf{W}$ to be orthonormal, and to order the columns by decreasing variance of the corresponding latent factors. This is the approach adopted by PCA, which we will discuss in Section 12.2. The result is not necessarily more interpretable, but at least it is unique.
- **Forcing $\mathbf{W}$ to be lower triangular** One way to achieve identifiability, which is popular in the Bayesian community (e.g., (Lopes and West 2004)), is to ensure that the first visible feature is only generated by the first latent factor, the second visible feature is only generated by the first two latent factors, and so on. For example, if $L = 3$ and $D = 4$, the corresponding factor loading matrix is given by

$$\mathbf{W} = \begin{pmatrix} w_{11} & 0 & 0 \\ w_{21} & w_{22} & 0 \\ w_{31} & w_{32} & w_{33} \\ w_{41} & w_{42} & w_{43} \end{pmatrix} \quad (12.13)$$

We also require that $w_{jj} > 0$ for $j = 1 : L$. The total number of parameters in this constrained matrix is $D + DL - L(L-1)/2$, which is equal to the number of uniquely identifiable parameters. The disadvantage of this method is that the first L visible variables,

1. To see this, note that there are $L-1$ free parameters in $\mathbf{R}$ in the first column (since the column vector must be normalized to unit length), there are $L-2$ free parameters in the second column (which must be orthogonal to the first), and so on.

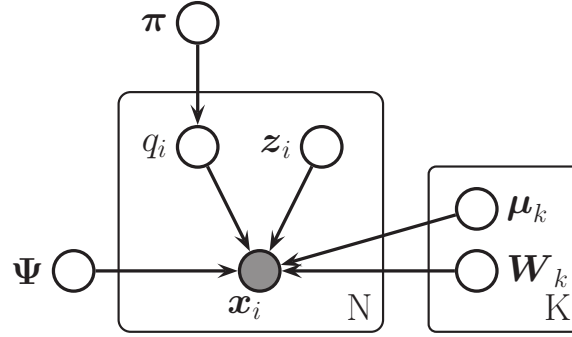


Figure 12.3 Mixture of factor analysers as a DGM.

known as the **founder variables**, affect the interpretation of the latent factors, and so must be chosen carefully.

- **Sparsity promoting priors on the weights** Instead of pre-specifying which entries in $\mathbf{W}$ are zero, we can encourage the entries to be zero, using ℓ_1 regularization (Zou et al. 2006), ARD (Bishop 1999; Archambeau and Bach 2008), or spike-and-slab priors (Rattray et al. 2009). This is called sparse factor analysis. This does not necessarily ensure a unique MAP estimate, but it does encourage interpretable solutions. See Section 13.8.
- **Choosing an informative rotation matrix** There are a variety of heuristic methods that try to find rotation matrices $\mathbf{R}$ which can be used to modify $\mathbf{W}$ (and hence the latent factors) so as to try to increase the interpretability, typically by encouraging them to be (approximately) sparse. One popular method is known as **varimax** (Kaiser 1958).
- **Use of non-Gaussian priors for the latent factors** In Section 12.6, we will discuss how replacing $p(\mathbf{z}_i)$ with a non-Gaussian distribution can enable us to sometimes uniquely identify $\mathbf{W}$ as well as the latent factors. This technique is known as ICA.

12.1.4 Mixtures of factor analysers

The FA model assumes that the data lives on a low dimensional linear manifold. In reality, most data is better modeled by some form of low dimensional *curved* manifold. We can approximate a curved manifold by a piecewise linear manifold. This suggests the following model: let the k 'th linear subspace of dimensionality L_k be represented by $\mathbf{W}_k$, for $k = 1 : K$. Suppose we have a latent indicator $q_i \in \{1, \dots, K\}$ specifying which subspace we should use to generate the data. We then sample $\mathbf{z}_i$ from a Gaussian prior and pass it through the $\mathbf{W}_k$ matrix (where $k = q_i$), and add noise. More precisely, the model is as follows:

$$p(\mathbf{x}_i | \mathbf{z}_i, q_i = k, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k + \mathbf{W}_k \mathbf{z}_i, \boldsymbol{\Psi}) \quad (12.14)$$

$$p(\mathbf{z}_i | \boldsymbol{\theta}) = \mathcal{N}(\mathbf{z}_i | \mathbf{0}, \mathbf{I}) \quad (12.15)$$

$$p(q_i | \boldsymbol{\theta}) = \text{Cat}(q_i | \boldsymbol{\pi}) \quad (12.16)$$

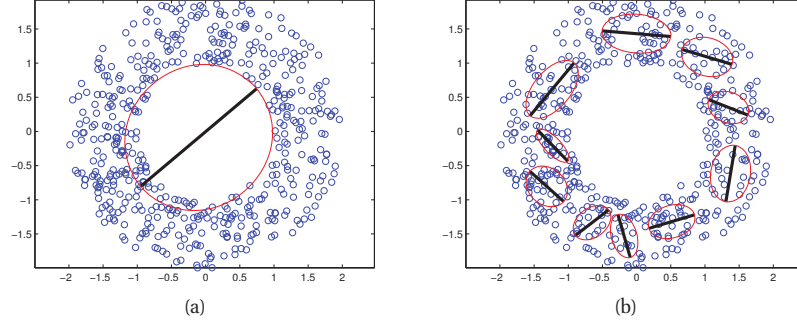


Figure 12.4 Mixture of 1d PPCAs fit to a dataset, for $K = 1, 10$. Figure generated by mixPpcaDemoNetlab.

This is called a **mixture of factor analysers** (MFA) (Hinton et al. 1997). The CI assumptions are represented in Figure 12.3.

Another way to think about this model is as a low-rank version of a mixture of Gaussians. In particular, this model needs $O(KLD)$ parameters instead of the $O(KD^2)$ parameters needed for a mixture of full covariance Gaussians. This can reduce overfitting. In fact, MFA is a good generic density model for high-dimensional real-valued data.

12.1.5 EM for factor analysis models

Using the results from Chapter 4, it is straightforward to derive an EM algorithm to fit an FA model. With just a little more work, we can fit a mixture of FAs. Below we state the results without proof. The derivation can be found in (Ghahramani and Hinton 1996a); however, deriving these equations yourself is a useful exercise if you want to become proficient at the math.

To obtain the results for a single factor analyser, just set $r_{ic} = 1$ and $c = 1$ in the equations below. In Section 12.2.5 we will see a further simplification of these equations that arises when fitting a PPCA model, where the results will turn out to have a particularly simple and elegant interpretation.

In the E step, we compute the posterior responsibility of cluster c for data point i using

$$r_{ic} \triangleq p(q_i = c | \mathbf{x}_i, \boldsymbol{\theta}) \propto \pi_c \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_c, \mathbf{W}_c \mathbf{W}_c^T + \boldsymbol{\Psi}) \quad (12.17)$$

The conditional posterior for $\mathbf{z}_i$ is given by

$$p(\mathbf{z}_i | \mathbf{x}_i, q_i = c, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{z}_i | \mathbf{m}_{ic}, \boldsymbol{\Sigma}_{ic}) \quad (12.18)$$

$$\boldsymbol{\Sigma}_{ic} \triangleq (\mathbf{I}_L + \mathbf{W}_c^T \boldsymbol{\Psi}_c^{-1} \mathbf{W}_c)^{-1} \quad (12.19)$$

$$\mathbf{m}_{ic} \triangleq \boldsymbol{\Sigma}_{ic} (\mathbf{W}_c^T \boldsymbol{\Psi}_c^{-1} (\mathbf{x}_i - \boldsymbol{\mu}_c)) \quad (12.20)$$

In the M step, it is easiest to estimate $\boldsymbol{\mu}_c$ and $\mathbf{W}_c$ at the same time, by defining $\tilde{\mathbf{W}}_c =$

$(\mathbf{W}_c, \boldsymbol{\mu}_c)$, $\tilde{\mathbf{z}} = (\mathbf{z}, 1)$, Also, define

$$\mathbf{b}_{ic} \triangleq \mathbb{E}[\tilde{\mathbf{z}}|\mathbf{x}_i, q_i = c] = [\mathbf{m}_{ic}; 1] \quad (12.21)$$

$$\mathbf{C}_{ic} \triangleq \mathbb{E}[\tilde{\mathbf{z}}\tilde{\mathbf{z}}^T|\mathbf{x}_i, q_i = c] = \begin{pmatrix} \mathbb{E}[\mathbf{z}\mathbf{z}^T|\mathbf{x}_i, q_i = c] & \mathbb{E}[\mathbf{z}|\mathbf{x}_i, q_i = c] \\ \mathbb{E}[\mathbf{z}|\mathbf{x}_i, q_i = c]^T & 1 \end{pmatrix} \quad (12.22)$$

Then the M step is as follows:

$$\hat{\mathbf{W}}_c = \left[\sum_i r_{ic} \mathbf{x}_i \mathbf{b}_{ic}^T \right] \left[\sum_i r_{ic} \mathbf{C}_{ic} \right]^{-1} \quad (12.23)$$

$$\hat{\Psi} = \frac{1}{N} \text{diag} \left\{ \sum_{ic} r_{ic} \left(\mathbf{x}_i - \hat{\mathbf{W}}_c \mathbf{b}_{ic} \right) \mathbf{x}_i^T \right\} \quad (12.24)$$

$$\hat{\pi}_c = \frac{1}{N} \sum_{i=1}^N r_{ic} \quad (12.25)$$

Note that these updates are for “vanilla” EM. A much faster version of this algorithm, based on ECM, is described in (Zhao and Yu 2008).

12.1.6 Fitting FA models with missing data

In many applications, such as collaborative filtering, we have missing data. One virtue of the EM approach to fitting an FA/PPCA model is that it is easy to extend to this case. However, overfitting can be a problem if there is a lot of missing data. Consequently it is important to perform MAP estimation or to use Bayesian inference. See e.g., (Ilin and Raiko 2010) for details.

12.2 Principal components analysis (PCA)

Consider the FA model where we constrain $\Psi = \sigma^2 \mathbf{I}$, and $\mathbf{W}$ to be orthonormal. It can be shown (Tipping and Bishop 1999) that, as $\sigma^2 \rightarrow 0$, this model reduces to classical (non-probabilistic) **principal components analysis (PCA)**, also known as the **Karhunen Loeve** transform. The version where $\sigma^2 > 0$ is known as **probabilistic PCA (PPCA)** (Tipping and Bishop 1999), or **sensible PCA** (Roweis 1997). (An equivalent result was derived independently, from a different perspective, in (Moghaddam and Pentland 1995).)

To make sense of this result, we first have to learn about classical PCA. We then connect PCA to the SVD. And finally we return to discuss PPCA.

12.2.1 Classical PCA: statement of the theorem

The **synthesis view** of classical PCA is summarized in the following theorem.

Theorem 12.2.1. *Suppose we want to find an orthogonal set of L linear basis vectors $\mathbf{w}_j \in \mathbb{R}^D$, and the corresponding scores $\mathbf{z}_i \in \mathbb{R}^L$, such that we minimize the average **reconstruction error***

$$J(\mathbf{W}, \mathbf{Z}) = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 \quad (12.26)$$

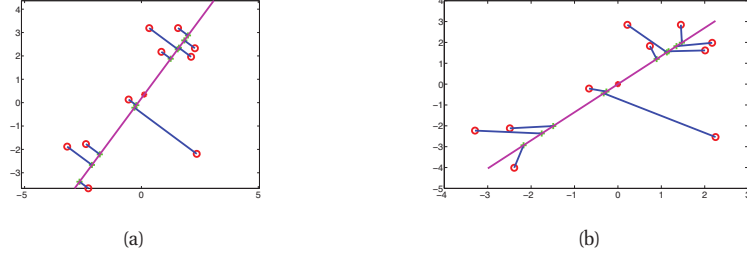


Figure 12.5 An illustration of PCA and PPCA where $D = 2$ and $L = 1$. Circles are the original data points, crosses are the reconstructions. The red star is the data mean. (a) PCA. The points are orthogonally projected onto the line. Figure generated by `pcaDemo2d`. (b) PPCA. The projection is no longer orthogonal: the reconstructions are shrunk towards the data mean (red star). Based on Figure 7.6 of (Nabney 2001). Figure generated by `ppcaDemo2d`.

where $\hat{\mathbf{x}}_i = \mathbf{W}\mathbf{z}_i$, subject to the constraint that $\mathbf{W}$ is orthonormal. Equivalently, we can write this objective as follows:

$$J(\mathbf{W}, \mathbf{Z}) = \|\mathbf{X} - \mathbf{W}\mathbf{Z}^T\|_F^2 \quad (12.27)$$

where $\mathbf{Z}$ is an $N \times L$ matrix with the $\mathbf{z}_i$ in its rows, and $\|\mathbf{A}\|_F$ is the **Frobenius norm** of matrix $\mathbf{A}$, defined by

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2} = \sqrt{\text{tr}(\mathbf{A}^T \mathbf{A})} = \|\mathbf{A}(\cdot)\|_2 \quad (12.28)$$

The optimal solution is obtained by setting $\hat{\mathbf{W}} = \mathbf{V}_L$, where $\mathbf{V}_L$ contains the L eigenvectors with largest eigenvalues of the empirical covariance matrix, $\hat{\Sigma} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i \mathbf{x}_i^T$. (We assume the $\mathbf{x}_i$ have zero mean, for notational simplicity.) Furthermore, the optimal low-dimensional encoding of the data is given by $\hat{\mathbf{z}}_i = \mathbf{W}^T \mathbf{x}_i$, which is an orthogonal projection of the data onto the column space spanned by the eigenvectors.

An example of this is shown in Figure 12.5(a) for $D = 2$ and $L = 1$. The diagonal line is the vector $\mathbf{w}_1$; this is called the first principal component or principal direction. The data points $\mathbf{x}_i \in \mathbb{R}^2$ are orthogonally projected onto this line to get $z_i \in \mathbb{R}$. This is the best 1-dimensional approximation to the data. (We will discuss Figure 12.5(b) later.)

In general, it is hard to visualize higher dimensional data, but if the data happens to be a set of images, it is easy to do so. Figure 12.6 shows the first three principal vectors, reshaped as images, as well as the reconstruction of a specific image using a varying number of basis vectors. (We discuss how to choose L in Section 11.5.)

Below we will show that the principal directions are the ones along which the data shows maximal variance. This means that PCA can be “misled” by directions in which the variance is high merely because of the measurement scale. Figure 12.7(a) shows an example, where the vertical axis (weight) uses a large range than the horizontal axis (height), resulting in a line that looks somewhat “unnatural”. It is therefore standard practice to standardize the data first, or

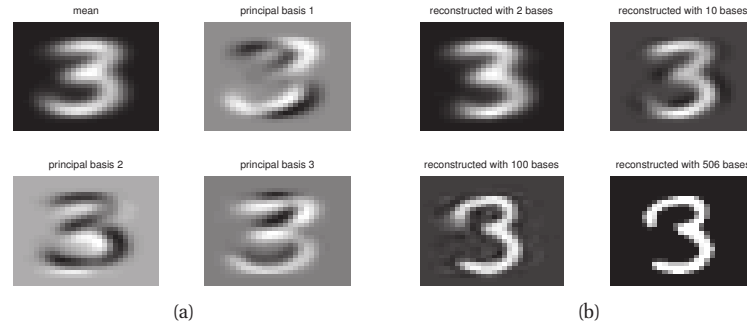


Figure 12.6 (a) The mean and the first three PC basis vectors (eigendigits) based on 25 images of the digit 3 (from the MNIST dataset). (b) Reconstruction of an image based on 2, 10, 100 and all the basis vectors. Figure generated by `pcaImageDemo`.

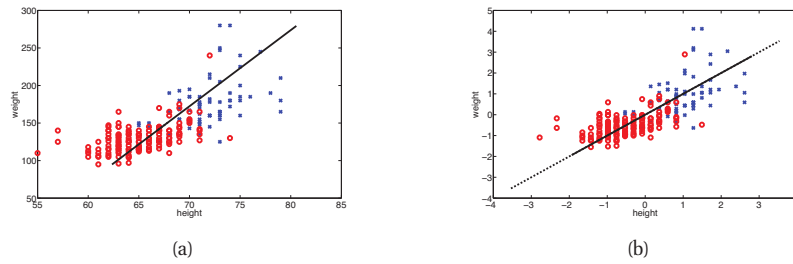


Figure 12.7 Effect of standardization on PCA applied to the height/ weight dataset. Left: PCA of raw data. Right: PCA of standardized data. Figure generated by `pcaDemoHeightWeight`.

equivalently, to work with correlation matrices instead of covariance matrices. The benefits of this are apparent from Figure 12.7(b).

12.2.2 Proof *

Proof. We use $\mathbf{w}_j \in \mathbb{R}^D$ to denote the j 'th principal direction, $\mathbf{x}_i \in \mathbb{R}^D$ to denote the i 'th high-dimensional observation, $\mathbf{z}_i \in \mathbb{R}^L$ to denote the i 'th low-dimensional representation, and $\tilde{\mathbf{z}}_j \in \mathbb{R}^N$ to denote the $[z_{1j}, \dots, z_{Nj}]$, which is the j 'th component of all the low-dimensional vectors.

Let us start by estimating the best 1d solution, $\mathbf{w}_1 \in \mathbb{R}^D$, and the corresponding projected points $\tilde{\mathbf{z}}_1 \in \mathbb{R}^N$. We will find the remaining bases $\mathbf{w}_2, \mathbf{w}_3$, etc. later. The reconstruction error

is given by

$$J(\mathbf{w}_1, \mathbf{z}_1) = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - z_{i1} \mathbf{w}_1\|^2 = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i - z_{i1} \mathbf{w}_1)^T (\mathbf{x}_i - z_{i1} \mathbf{w}_1) \quad (12.29)$$

$$= \frac{1}{N} \sum_{i=1}^N [\mathbf{x}_i^T \mathbf{x}_i - 2z_{i1} \mathbf{w}_1^T \mathbf{x}_i + z_{i1}^2 \mathbf{w}_1^T \mathbf{w}_1] \quad (12.30)$$

$$= \frac{1}{N} \sum_{i=1}^N [\mathbf{x}_i^T \mathbf{x}_i - 2z_{i1} \mathbf{w}_1^T \mathbf{x}_i + z_{i1}^2] \quad (12.31)$$

since $\mathbf{w}_1^T \mathbf{w}_1 = 1$ (by the orthonormality assumption). Taking derivatives wrt z_{i1} and equating to zero gives

$$\frac{\partial}{\partial z_{i1}} J(\mathbf{w}_1, \mathbf{z}_1) = \frac{1}{N} [-2\mathbf{w}_1^T \mathbf{x}_i + 2z_{i1}] = 0 \Rightarrow z_{i1} = \mathbf{w}_1^T \mathbf{x}_i \quad (12.32)$$

So the optimal reconstruction weights are obtained by orthogonally projecting the data onto the first principal direction, $\mathbf{w}_1$ (see Figure 12.5(a)). Plugging back in gives

$$J(\mathbf{w}_1) = \frac{1}{N} \sum_{i=1}^N [\mathbf{x}_i^T \mathbf{x}_i - z_{i1}^2] = \text{const} - \frac{1}{N} \sum_{i=1}^N z_{i1}^2 \quad (12.33)$$

Now the variance of the projected coordinates is given by

$$\text{var} [\tilde{\mathbf{z}}_1] = \mathbb{E} [\tilde{\mathbf{z}}_1^2] - (\mathbb{E} [\tilde{\mathbf{z}}_1])^2 = \frac{1}{N} \sum_{i=1}^N z_{i1}^2 - 0 \quad (12.34)$$

since

$$\mathbb{E} [z_{i1}] = \mathbb{E} [\mathbf{x}_i^T \mathbf{w}_1] = \mathbb{E} [\mathbf{x}_i]^T \mathbf{w}_1 = 0 \quad (12.35)$$

because the data has been centered. From this, we see that *minimizing* the reconstruction error is equivalent to *maximizing* the variance of the projected data, i.e.,

$$\arg \min_{\mathbf{w}_1} J(\mathbf{w}_1) = \arg \max_{\mathbf{w}_1} \text{var} [\tilde{\mathbf{z}}_1] \quad (12.36)$$

This is why it is often said that PCA finds the directions of maximal variance. This is called the **analysis view** of PCA.

The variance of the projected data can be written as

$$\frac{1}{N} \sum_{i=1}^N z_{i1}^2 = \frac{1}{N} \sum_{i=1}^N \mathbf{w}_1^T \mathbf{x}_i \mathbf{x}_i^T \mathbf{w}_1 = \mathbf{w}_1^T \hat{\Sigma} \mathbf{w}_1 \quad (12.37)$$

where $\hat{\Sigma} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i \mathbf{x}_i^T$ is the empirical covariance matrix (or correlation matrix if the data is standardized).

We can trivially maximize the variance of the projection (and hence minimize the reconstruction error) by letting $\|\mathbf{w}_1\| \rightarrow \infty$, so we impose the constraint $\|\mathbf{w}_1\| = 1$ and instead maximize

$$\tilde{J}(\mathbf{w}_1) = \mathbf{w}_1^T \hat{\Sigma} \mathbf{w}_1 + \lambda_1 (\mathbf{w}_1^T \mathbf{w}_1 - 1) \quad (12.38)$$

where λ_1 is the Lagrange multiplier. Taking derivatives and equating to zero we have

$$\frac{\partial}{\partial \mathbf{w}_1} \tilde{J}(\mathbf{w}_1) = 2\hat{\Sigma} \mathbf{w}_1 - 2\lambda_1 \mathbf{w}_1 = 0 \quad (12.39)$$

$$\hat{\Sigma} \mathbf{w}_1 = \lambda_1 \mathbf{w}_1 \quad (12.40)$$

Hence the direction that maximizes the variance is an eigenvector of the covariance matrix. Left multiplying by $\mathbf{w}_1$ (and using $\mathbf{w}_1^T \mathbf{w}_1 = 1$) we find that the variance of the projected data is

$$\mathbf{w}_1^T \hat{\Sigma} \mathbf{w}_1 = \lambda_1 \quad (12.41)$$

Since we want to maximize the variance, we pick the eigenvector which corresponds to the largest eigenvalue.

Now let us find another direction $\mathbf{w}_2$ to further minimize the reconstruction error, subject to $\mathbf{w}_1^T \mathbf{w}_2 = 0$ and $\mathbf{w}_2^T \mathbf{w}_2 = 1$. The error is

$$J(\mathbf{w}_1, \mathbf{z}_1, \mathbf{w}_2, \mathbf{z}_2) = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - z_{i1} \mathbf{w}_1 - z_{i2} \mathbf{w}_2\|^2 \quad (12.42)$$

Optimizing wrt $\mathbf{w}_1$ and $\mathbf{z}_1$ gives the same solution as before. Exercise 12.4 asks you to show that $\frac{\partial J}{\partial \mathbf{z}_2} = 0$ yields $z_{i2} = \mathbf{w}_2^T \mathbf{x}_i$. In other words, the second principal encoding is gotten by projecting onto the second principal direction. Substituting in yields

$$J(\mathbf{w}_2) = \frac{1}{n} \sum_{i=1}^N [\mathbf{x}_i^T \mathbf{x}_i - \mathbf{w}_1^T \mathbf{x}_i \mathbf{x}_i^T \mathbf{w}_1 - \mathbf{w}_2^T \mathbf{x}_i \mathbf{x}_i^T \mathbf{w}_2] = \text{const} - \mathbf{w}_2^T \hat{\Sigma} \mathbf{w}_2 \quad (12.43)$$

Dropping the constant term and adding the constraints yields

$$\tilde{J}(\mathbf{w}_2) = -\mathbf{w}_2^T \hat{\Sigma} \mathbf{w}_2 + \lambda_2 (\mathbf{w}_2^T \mathbf{w}_2 - 1) + \lambda_{12} (\mathbf{w}_2^T \mathbf{w}_1 - 0) \quad (12.44)$$

Exercise 12.4 asks you to show that the solution is given by the eigenvector with the second largest eigenvalue:

$$\hat{\Sigma} \mathbf{w}_2 = \lambda_2 \mathbf{w}_2 \quad (12.45)$$

The proof continues in this way. (Formally one can use induction.) $\square$

12.2.3 Singular value decomposition (SVD)

We have defined the solution to PCA in terms of eigenvectors of the covariance matrix. However, there is another way to obtain the solution, based on the **singular value decomposition**, or **SVD**. This basically generalizes the notion of eigenvectors from square matrices to any kind of matrix.

In particular, any (real) $N \times D$ matrix $\mathbf{X}$ can be decomposed as follows

$$\underbrace{\mathbf{X}}_{N \times D} = \underbrace{\mathbf{U}}_{N \times N} \underbrace{\mathbf{S}}_{N \times D} \underbrace{\mathbf{V}^T}_{D \times D} \quad (12.46)$$

where $\mathbf{U}$ is an $N \times N$ matrix whose columns are orthonormal (so $\mathbf{U}^T \mathbf{U} = \mathbf{I}_N$), $\mathbf{V}$ is $D \times D$ matrix whose rows and columns are orthonormal (so $\mathbf{V}^T \mathbf{V} = \mathbf{V} \mathbf{V}^T = \mathbf{I}_D$), and $\mathbf{S}$ is a $N \times D$ matrix containing the $r = \min(N, D)$ **singular values** $\sigma_i \geq 0$ on the main diagonal, with 0s filling the rest of the matrix. The columns of $\mathbf{U}$ are the left singular vectors, and the columns of $\mathbf{V}$ are the right singular vectors. See Figure 12.8(a) for an example.

Since there are at most D singular values (assuming $N > D$), the last $N - D$ columns of $\mathbf{U}$ are irrelevant, since they will be multiplied by 0. The **economy sized SVD**, or **thin SVD**, avoids computing these unnecessary elements. Let us denote this decomposition by $\hat{\mathbf{U}} \hat{\mathbf{S}} \hat{\mathbf{V}}^T$. If $N > D$, we have

$$\underbrace{\mathbf{X}}_{N \times D} = \underbrace{\hat{\mathbf{U}}}_{N \times D} \underbrace{\hat{\mathbf{S}}}_{D \times D} \underbrace{\hat{\mathbf{V}}^T}_{D \times D} \quad (12.47)$$

as in Figure 12.8(a). If $N < D$, we have

$$\underbrace{\mathbf{X}}_{N \times D} = \underbrace{\hat{\mathbf{U}}}_{N \times N} \underbrace{\hat{\mathbf{S}}}_{N \times N} \underbrace{\hat{\mathbf{V}}^T}_{N \times D} \quad (12.48)$$

Computing the economy-sized SVD takes $O(ND \min(N, D))$ time (Golub and van Loan 1996, p254).

The connection between eigenvectors and singular vectors is the following. For an arbitrary real matrix $\mathbf{X}$, if $\mathbf{X} = \mathbf{U} \mathbf{S} \mathbf{V}^T$, we have

$$\mathbf{X}^T \mathbf{X} = \mathbf{V} \mathbf{S}^T \mathbf{U}^T \mathbf{U} \mathbf{S} \mathbf{V}^T = \mathbf{V} (\mathbf{S}^T \mathbf{S}) \mathbf{V}^T = \mathbf{V} \mathbf{D} \mathbf{V}^T \quad (12.49)$$

where $\mathbf{D} = \mathbf{S}^2$ is a diagonal matrix containing the squares singular values. Hence

$$(\mathbf{X}^T \mathbf{X}) \mathbf{V} = \mathbf{V} \mathbf{D} \quad (12.50)$$

so the eigenvectors of $\mathbf{X}^T \mathbf{X}$ are equal to $\mathbf{V}$, the right singular vectors of $\mathbf{X}$, and the eigenvalues of $\mathbf{X}^T \mathbf{X}$ are equal to $\mathbf{D}$, the squared singular values. Similarly

$$\mathbf{X} \mathbf{X}^T = \mathbf{U} \mathbf{S} \mathbf{V}^T \mathbf{V} \mathbf{S}^T \mathbf{U}^T = \mathbf{U} (\mathbf{S} \mathbf{S}^T) \mathbf{U}^T \quad (12.51)$$

$$(\mathbf{X} \mathbf{X}^T) \mathbf{U} = \mathbf{U} (\mathbf{S} \mathbf{S}^T) = \mathbf{U} \mathbf{D} \quad (12.52)$$

so the eigenvectors of $\mathbf{X} \mathbf{X}^T$ are equal to $\mathbf{U}$, the left singular vectors of $\mathbf{X}$. Also, the eigenvalues of $\mathbf{X} \mathbf{X}^T$ are equal to the squared singular values. We can summarize all this as follows:

$$\mathbf{U} = \text{evec}(\mathbf{X} \mathbf{X}^T), \quad \mathbf{V} = \text{evec}(\mathbf{X}^T \mathbf{X}), \quad \mathbf{S}^2 = \text{eval}(\mathbf{X} \mathbf{X}^T) = \text{eval}(\mathbf{X}^T \mathbf{X}) \quad (12.53)$$

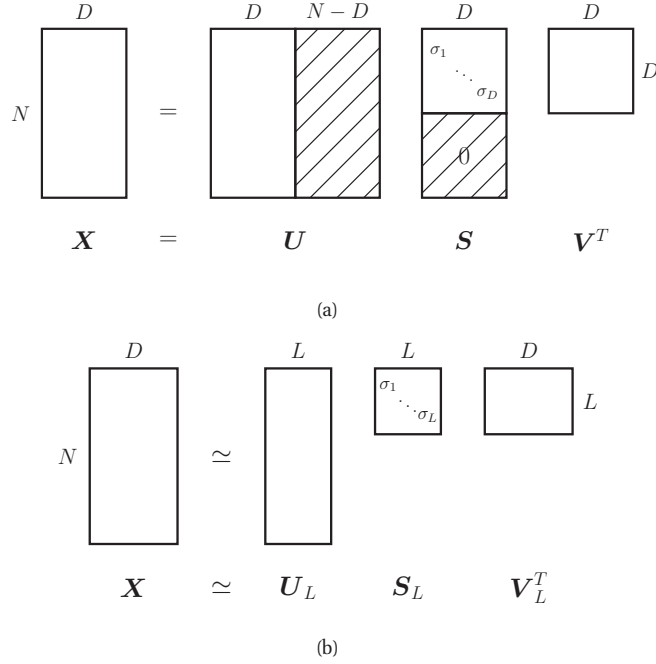


Figure 12.8 (a) SVD decomposition of non-square matrices $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T$. The shaded parts of $\mathbf{S}$, and all the off-diagonal terms, are zero. The shaded entries in $\mathbf{U}$ and $\mathbf{S}$ are not computed in the economy-sized version, since they are not needed. (b) Truncated SVD approximation of rank L .

Since the eigenvectors are unaffected by linear scaling of a matrix, we see that the right singular vectors of $\mathbf{X}$ are equal to the eigenvectors of the empirical covariance $\hat{\Sigma}$. Furthermore, the eigenvalues of $\hat{\Sigma}$ are a scaled version of the squared singular values. This means we can perform PCA using just a few lines of code (see `pcaPmtk`).

However, the connection between PCA and SVD goes deeper. From Equation 12.46, we can represent a rank r matrix as follows:

$$\mathbf{X} = \sigma_1 \begin{pmatrix} | \\ \mathbf{u}_1 \\ | \end{pmatrix} \begin{pmatrix} - & \mathbf{v}_1^T & - \end{pmatrix} + \cdots + \sigma_r \begin{pmatrix} | \\ \mathbf{u}_r \\ | \end{pmatrix} \begin{pmatrix} - & \mathbf{v}_r^T & - \end{pmatrix} \quad (12.54)$$

If the singular values die off quickly as in Figure 12.10, we can produce a rank L approximation to the matrix as follows:

$$\mathbf{X} \approx \mathbf{U}_{:,1:L} \mathbf{S}_{1:L,1:L} \mathbf{V}_{:,1:L}^T \quad (12.55)$$

This is called a **truncated SVD** (see Figure 12.8(b)). The total number of parameters needed to represent an $N \times D$ matrix using a rank L approximation is

$$NL + LD + L = L(N + D + 1) \quad (12.56)$$

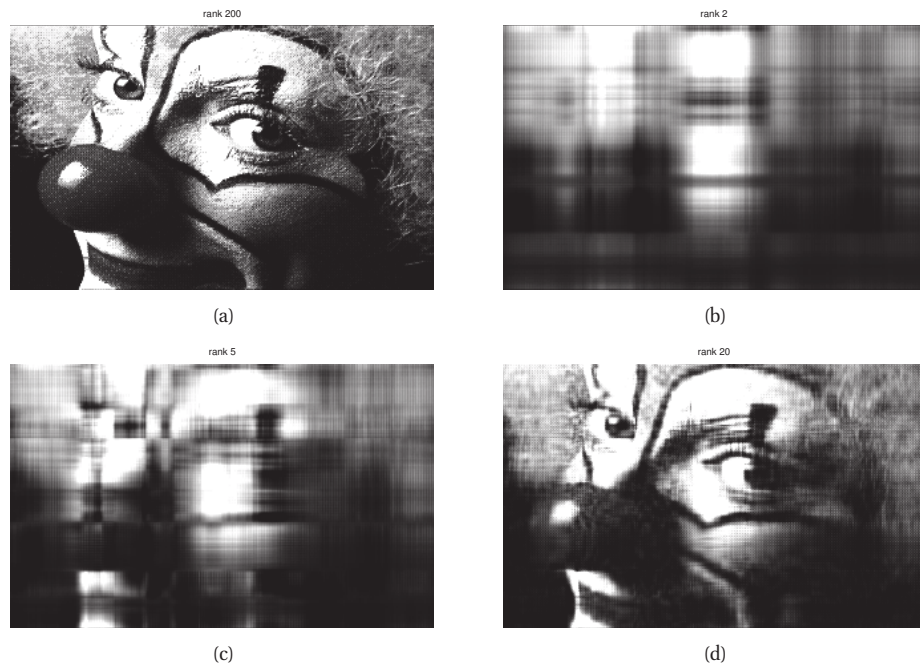


Figure 12.9 Low rank approximations to an image. Top left: The original image is of size 200×320 , so has rank 200. Subsequent images have ranks 2, 5, and 20. Figure generated by `svdImageDemo`.

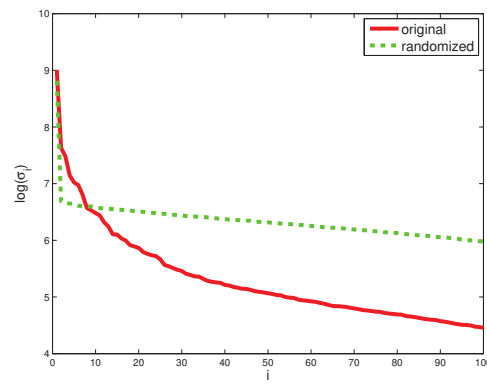


Figure 12.10 First 50 log singular values for the clown image (solid red line), and for a data matrix obtained by randomly shuffling the pixels (dotted green line). Figure generated by `svdImageDemo`.

As an example, consider the 200×320 pixel image in Figure 12.9(top left). This has 64,000 numbers in it. We see that a rank 20 approximation, with only $(200 + 320 + 1) \times 20 = 10,420$ numbers is a very good approximation.

One can show that the error in this approximation is given by

$$\|\mathbf{X} - \mathbf{X}_L\|_F \approx \sigma_{L+1} \quad (12.57)$$

Furthermore, one can show that the SVD offers the best rank L approximation to a matrix (best in the sense of minimizing the above Frobenius norm).

Let us connect this back to PCA. Let $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T$ be a truncated SVD of $\mathbf{X}$. We know that $\hat{\mathbf{W}} = \mathbf{V}$, and that $\hat{\mathbf{Z}} = \mathbf{X}\hat{\mathbf{W}}$, so

$$\hat{\mathbf{Z}} = \mathbf{U}\mathbf{S}\mathbf{V}^T\mathbf{V} = \mathbf{U}\mathbf{S} \quad (12.58)$$

Furthermore, the optimal reconstruction is given by $\hat{\mathbf{X}} = \hat{\mathbf{Z}}\hat{\mathbf{W}}^T$, so we find

$$\hat{\mathbf{X}} = \mathbf{U}\mathbf{S}\mathbf{V}^T \quad (12.59)$$

This is precisely the same as a truncated SVD approximation! This is another illustration of the fact that PCA is the best low rank approximation to the data.

12.2.4 Probabilistic PCA

We are now ready to revisit PPCA. One can show the following remarkable result.

Theorem 12.2.2 (Tipping and Bishop 1999). *Consider a factor analysis model in which $\Psi = \sigma^2\mathbf{I}$ and $\mathbf{W}$ is orthogonal. The observed data log likelihood is given by*

$$\log p(\mathbf{X}|\mathbf{W}, \sigma^2) = -\frac{N}{2} \ln |\mathbf{C}| - \frac{1}{2} \sum_{i=1}^N \mathbf{x}_i^T \mathbf{C}^{-1} \mathbf{x}_i = -\frac{N}{2} \ln |\mathbf{C}| + \text{tr}(\mathbf{C}^{-1} \hat{\mathbf{S}}) \quad (12.60)$$

where $\mathbf{C} = \mathbf{W}\mathbf{W}^T + \sigma^2\mathbf{I}$ and $\mathbf{S} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i \mathbf{x}_i^T = (1/N) \mathbf{X}^T \mathbf{X}$. (We are assuming centered data, for notational simplicity.) The maxima of the log-likelihood are given by

$$\hat{\mathbf{W}} = \mathbf{V}(\mathbf{\Lambda} - \sigma^2\mathbf{I})^{\frac{1}{2}} \mathbf{R} \quad (12.61)$$

where $\mathbf{R}$ is an arbitrary $L \times L$ orthogonal matrix, $\mathbf{V}$ is the $D \times L$ matrix whose columns are the first L eigenvectors of $\mathbf{S}$, and $\mathbf{\Lambda}$ is the corresponding diagonal matrix of eigenvalues. Without loss of generality, we can set $\mathbf{R} = \mathbf{I}$. Furthermore, the MLE of the noise variance is given by

$$\hat{\sigma}^2 = \frac{1}{D-L} \sum_{j=L+1}^D \lambda_j \quad (12.62)$$

which is the average variance associated with the discarded dimensions.

Thus, as $\sigma^2 \rightarrow 0$, we have $\hat{\mathbf{W}} \rightarrow \mathbf{V}$, as in classical PCA. What about $\hat{\mathbf{Z}}$? It is easy to see that the posterior over the latent factors is given by

$$p(\mathbf{z}_i | \mathbf{x}_i, \hat{\boldsymbol{\theta}}) = \mathcal{N}(\mathbf{z}_i | \hat{\mathbf{F}}^{-1} \hat{\mathbf{W}}^T \mathbf{x}_i, \sigma^2 \hat{\mathbf{F}}^{-1}) \quad (12.63)$$

$$\hat{\mathbf{F}} \triangleq \hat{\mathbf{W}}^T \hat{\mathbf{W}} + \hat{\sigma}^2 \mathbf{I} \quad (12.64)$$

(Do not confuse $\mathbf{F} = \mathbf{W}^T \mathbf{W} + \sigma^2 \mathbf{I}$ with $\mathbf{C} = \mathbf{W} \mathbf{W}^T + \sigma^2 \mathbf{I}$.) Hence, as $\sigma^2 \rightarrow 0$, we find $\hat{\mathbf{W}} \rightarrow \mathbf{V}$, $\hat{\mathbf{F}} \rightarrow \mathbf{I}$ and $\hat{\mathbf{z}}_i \rightarrow \mathbf{V}^T \mathbf{x}_i$. Thus the posterior mean is obtained by an orthogonal projection of the data onto the column space of $\mathbf{V}$, as in classical PCA.

Note, however, that if $\sigma^2 > 0$, the posterior mean is not an orthogonal projection, since it is shrunk somewhat towards the prior mean, as illustrated in Figure 12.5(b). This sounds like an undesirable property, but it means that the reconstructions will be closer to the overall data mean, $\hat{\boldsymbol{\mu}} = \bar{\mathbf{x}}$.

12.2.5 EM algorithm for PCA

Although the usual way to fit a PCA model uses eigenvector methods, or the SVD, we can also use EM, which will turn out to have some advantages that we discuss below. EM for PCA relies on the probabilistic formulation of PCA. However the algorithm continues to work in the zero noise limit, $\sigma^2 = 0$, as shown by (Roweis 1997).

Let $\tilde{\mathbf{Z}}$ be a $L \times N$ matrix storing the posterior means (low-dimensional representations) along its columns. Similarly, let $\tilde{\mathbf{X}} = \mathbf{X}^T$ store the original data along its columns. From Equation 12.63, when $\sigma^2 = 0$, we have

$$\tilde{\mathbf{Z}} = (\mathbf{W}^T \mathbf{W})^{-1} \mathbf{W}^T \tilde{\mathbf{X}} \quad (12.65)$$

This constitutes the E step. Notice that this is just an orthogonal projection of the data.

From Equation 12.23, the M step is given by

$$\hat{\mathbf{W}} = \left[\sum_i \mathbf{x}_i \mathbb{E}[\mathbf{z}_i]^T \right] \left[\sum_i \mathbb{E}[\mathbf{z}_i] \mathbb{E}[\mathbf{z}_i]^T \right]^{-1} \quad (12.66)$$

where we exploited the fact that $\boldsymbol{\Sigma} = \text{cov}[\mathbf{z}_i | \mathbf{x}_i, \boldsymbol{\theta}] = 0 \mathbf{I}$ when $\sigma^2 = 0$. It is worth comparing this expression to the MLE for multi-output linear regression (Equation 7.89), which has the form $\mathbf{W} = (\sum_i \mathbf{y}_i \mathbf{x}_i^T) (\sum_i \mathbf{x}_i \mathbf{x}_i^T)^{-1}$. Thus we see that the M step is like linear regression where we replace the observed inputs by the expected values of the latent variables.

In summary, here is the entire algorithm:

- **E step** $\tilde{\mathbf{Z}} = (\mathbf{W}^T \mathbf{W})^{-1} \mathbf{W}^T \tilde{\mathbf{X}}$
- **M step** $\mathbf{W} = \tilde{\mathbf{X}} \tilde{\mathbf{Z}}^T (\tilde{\mathbf{Z}} \tilde{\mathbf{Z}}^T)^{-1}$

(Tipping and Bishop 1999) showed that the only stable fixed point of the EM algorithm is the globally optimal solution. That is, the EM algorithm converges to a solution where $\mathbf{W}$ spans the same linear subspace as that defined by the first L eigenvectors. However, if we want $\mathbf{W}$ to be orthogonal, and to contain the eigenvectors in descending order of eigenvalue, we have to orthogonalize the resulting matrix (which can be done quite cheaply). Alternatively, we can modify EM to give the principal basis directly (Ahn and Oh 2003).

This algorithm has a simple physical analogy in the case $D = 2$ and $L = 1$ (Roweis 1997). Consider some points in $\mathbb{R}^2$ attached by springs to a rigid rod, whose orientation is defined by a vector $\mathbf{w}$. Let z_i be the location where the i 'th spring attaches to the rod. In the E step, we hold the rod fixed, and let the attachment points slide around so as to minimize the spring energy (which is proportional to the sum of squared residuals). In the M step, we hold the attachment

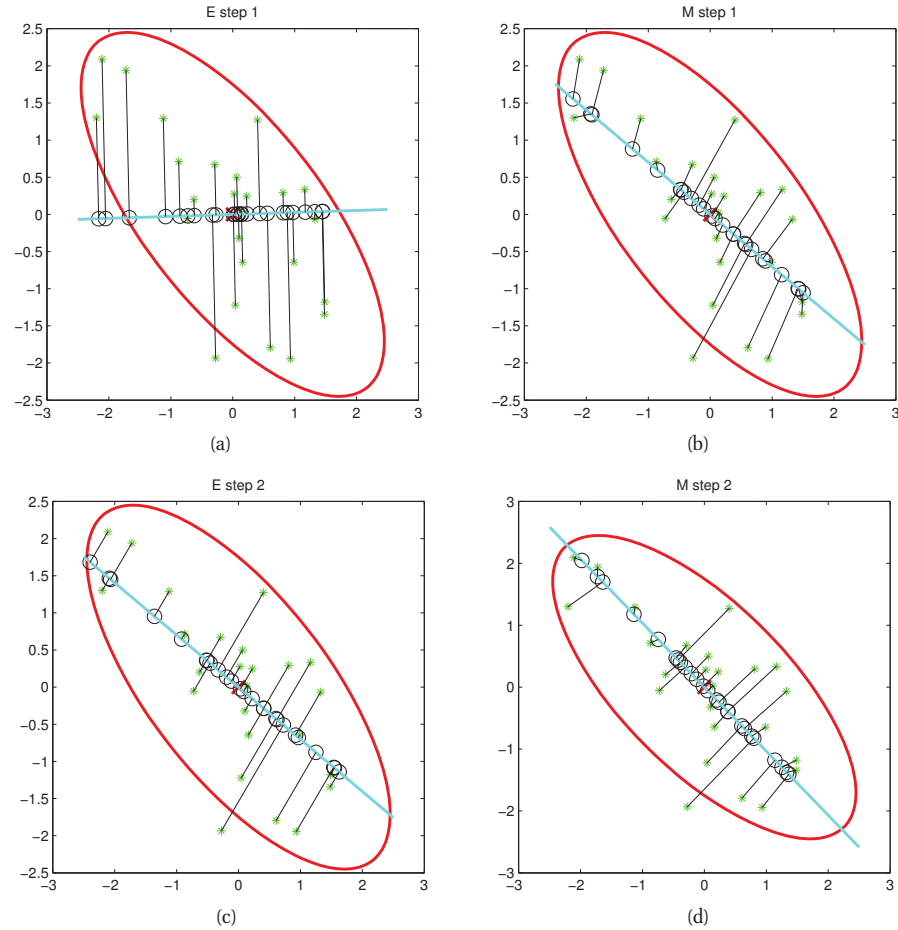


Figure 12.11 Illustration of EM for PCA when $D = 2$ and $L = 1$. Green stars are the original data points, black circles are their reconstructions. The weight vector $\mathbf{w}$ is represented by blue line. (a) We start with a random initial guess of $\mathbf{w}$. The E step is represented by the orthogonal projections. (b) We update the rod $\mathbf{w}$ in the M step, keeping the projections onto the rod (black circles) fixed. (c) Another E step. The black circles can 'slide' along the rod, but the rod stays fixed. (d) Another M step. Based on Figure 12.12 of (Bishop 2006b). Figure generated by `pcaEmStepByStep`.

points fixed and let the rod rotate so as to minimize the spring energy. See Figure 12.11 for an illustration.

Apart from this pleasing intuitive interpretation, EM for PCA has the following advantages over eigenvector methods:

- EM can be faster. In particular, assuming $N, D \gg L$, the dominant cost of EM is the projection operation in the E step, so the overall time is $O(TLND)$, where T is the number of

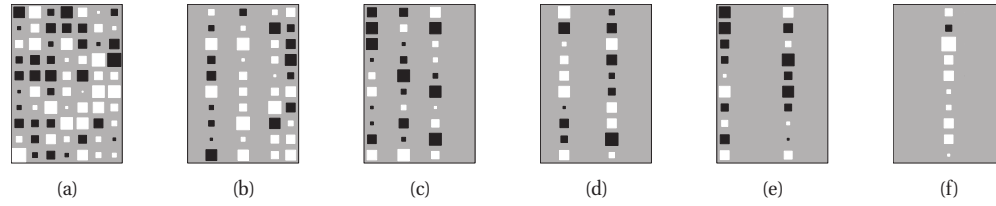


Figure 12.12 Illustration of estimating the effective dimensionalities in a mixture of factor analysers using VBEM. The blank columns have been forced to 0 via the ARD mechanism. The data was generated from 6 clusters with intrinsic dimensionalities of 7, 4, 3, 2, 2, 1, which the method has successfully estimated. Source: Figure 4.4 of (Beal 2003). Used with kind permission of Matt Beal.

iterations. (Roweis 1997) showed experimentally that the number of iterations is usually very small (the mean was 3.6), regardless of N or D . (This results depends on the ratio of eigenvalues of the empirical covariance matrix.) This is much faster than the $O(\min(ND^2, DN^2))$ time required by straightforward eigenvector methods, although more sophisticated eigenvector methods, such as the Lanczos algorithm, have running times comparable to EM.

- EM can be implemented in an online fashion, i.e., we can update our estimate of $\mathbf{W}$ as the data streams in.
- EM can handle missing data in a simple way (see Section 12.1.6).
- EM can be extended to handle mixtures of PPCA/ FA models.
- EM can be modified to variational EM or to variational Bayes EM to fit more complex models.

12.3 Choosing the number of latent dimensions

In Section 11.5, we discussed how to choose the number of components K in a mixture model. In this section, we discuss how to choose the number of latent dimensions L in a FA/PCA model.

12.3.1 Model selection for FA/PPCA

If we use a probabilistic model, we can in principle compute $L^* = \operatorname{argmax}_L p(L|\mathcal{D})$. However, there are two problems with this. First, evaluating the marginal likelihood for IVMs is quite difficult. In practice, simple approximations, such as BIC or variational lower bounds (see Section 21.5), can be used (see also (Minka 2000a)). Alternatively, we can use the cross-validated likelihood as a performance measure, although this can be slow, since it requires fitting each model F times, where F is the number of CV folds.

The second issue is the need to search over a potentially large number of models. The usual approach is to perform exhaustive search over all candidate values of L . However, sometimes we can set the model to its maximal size, and then use a technique called automatic relevancy determination (Section 13.7), combined with EM, to automatically prune out irrelevant weights.

number of points per cluster	intrinsic dimensionalities					
	1	7	4	3	2	2
8	2				1	
8	1	2				
16	1	4				2
32	1	6	3	3	2	2
64	1	7	4	3	2	2
128	1	7	4	3	2	2

Figure 12.13 We show the estimated number of clusters, and their estimated dimensionalities, as a function of sample size. The VBEM algorithm found two different solutions when $N = 8$. Note that more clusters, with larger effective dimensionalities, are discovered as the sample sizes increases. Source: Table 4.1 of (Beal 2003). Used with kind permission of Matt Beal.

This technique will be described in a supervised context in Chapter 13, but can be adapted to the (M)FA context as shown in (Bishop 1999; Ghahramani and Beal 2000).

Figure 12.12 illustrates this approach applied to a mixture of FAs fit to a small synthetic dataset. The figures visualize the weight matrices for each cluster, using **Hinton diagrams**, where the size of the square is proportional to the value of the entry in the matrix.² We see that many of them are sparse. Figure 12.13 shows that the degree of sparsity depends on the amount of training data, in accord with the Bayesian Occam’s razor. In particular, when the sample size is small, the method automatically prefers simpler models, but as the sample size gets sufficiently large, the method converges on the “correct” solution, which is one with 6 subspaces of dimensionality 1, 2, 2, 3, 4 and 7.

Although the ARD/ EM method is elegant, it still needs to perform search over K . This is done using “birth” and “death” moves (Ghahramani and Beal 2000). An alternative approach is to perform stochastic sampling in the space of models. Traditional approaches, such as (Lopes and West 2004), are based on reversible jump MCMC, and also use birth and death moves. However, this can be slow and difficult to implement. More recent approaches use non-parametric priors, combined with Gibbs sampling, see e.g., (Paisley and Carin 2009).

12.3.2 Model selection for PCA

Since PCA is not a probabilistic model, we cannot use any of the methods described above. An obvious proxy for the likelihood is the reconstruction error:

$$E(\mathcal{D}, L) = \frac{1}{|\mathcal{D}|} \sum_{i \in \mathcal{D}} \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 \quad (12.67)$$

In the case of PCA, the reconstruction is given by $\hat{\mathbf{x}}_i = \mathbf{W}\mathbf{z}_i + \boldsymbol{\mu}$, where $\mathbf{z}_i = \mathbf{W}^T(\mathbf{x}_i - \boldsymbol{\mu})$ and $\mathbf{W}$ and $\boldsymbol{\mu}$ are estimated from $\mathcal{D}_{\text{train}}$.

2. Geoff Hinton is an English professor of computer science at the University of Toronto.

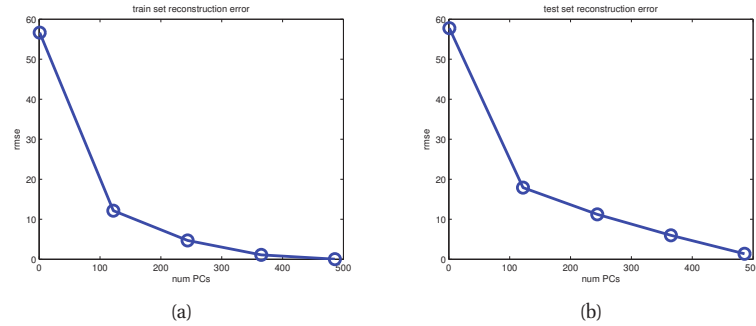


Figure 12.14 Reconstruction error on MNIST vs number of latent dimensions used by PCA. (a) Training set. (b) Test set. Figure generated by `pcaOverfitDemo`.

Figure 12.14(a) plots $E(\mathcal{D}_{\text{train}}, L)$ vs L on the MNIST training data in Figure 12.6. We see that it drops off quite quickly, indicating that we can capture most of the empirical correlation of the pixels with a small number of factors, as illustrated qualitatively in Figure 12.6.

Exercise 12.5 asks you to prove that the residual error from only using L terms is given by the sum of the discarded eigenvalues:

$$E(\mathcal{D}_{\text{train}}, L) = \sum_{j=L+1}^D \lambda_j \quad (12.68)$$

Therefore an alternative to plotting the error is to plot the retained eigenvalues, in decreasing order. This is called a **scree plot**, because “the plot looks like the side of a mountain, and ‘scree’ refers to the debris fallen from a mountain and lying at its base”.³ This will have the same shape as the residual error plot.

A related quantity is the **fraction of variance explained**, defined as

$$F(\mathcal{D}_{\text{train}}, L) = \frac{\sum_{j=1}^L \lambda_j}{\sum_{j'=1}^{L_{\text{max}}} \lambda_{j'}} \quad (12.69)$$

This captures the same information as the scree plot.

Of course, if we use $L = \text{rank}(\mathbf{X})$, we get zero reconstruction error on the training set. To avoid overfitting, it is natural to plot reconstruction error on the test set. This is shown in Figure 12.14(b). Here we see that the error continues to go down even as the model becomes more complex! Thus we do not get the usual U-shaped curve that we typically expect to see.

What is going on? The problem is that PCA is not a proper generative model of the data. It is merely a compression technique. If you give it more latent dimensions, it will be able to approximate the test data more accurately. By contrast, a probabilistic model enjoys a Bayesian Occam’s razor effect (Section 5.3.1), in that it gets “punished” if it wastes probability mass on parts of the space where there is little data. This is illustrated in Figure 12.15, which plots the

3. Quotation from <http://janda.org/workshop/factoranalysis/SPSSrun/SPSS08.htm>.

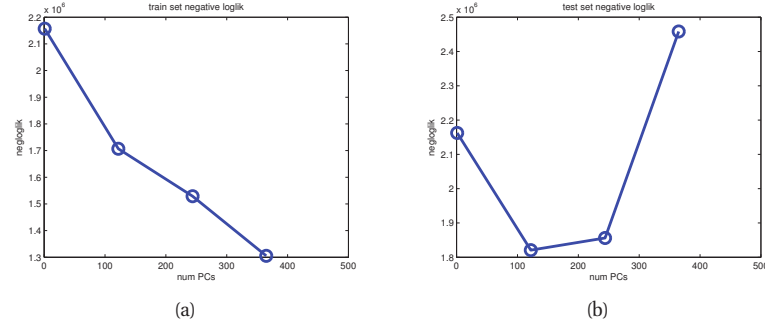


Figure 12.15 Negative log likelihood on MNIST vs number of latent dimensions used by PPCA. (a) Training set. (b) Test set. Figure generated by `pcaOverfitDemo`.

negative log likelihood, computed using PPCA, vs L . Here, on the test set, we see the usual U-shaped curve.

These results are analogous to those in Section 11.5.2, where we discussed the issue of choosing K in the K-means algorithm vs using a GMM.

12.3.2.1 Profile likelihood

Although there is no U-shape, there is sometimes a “regime change” in the plots, from relatively large errors to relatively small. One way to automate the detection of this is described in (Zhu and Ghodsi 2006). The idea is this. Let λ_k be some measure of the error incurred by a model of size k , such that $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_{L_{max}}$. In PCA, these are the eigenvalues, but the method can also be applied to K-means. Now consider partitioning these values into two groups, depending on whether $k < L$ or $k > L$, where L is some threshold which we will determine. To measure the quality of L , we will use a simple change-point model, where $\lambda_k \sim \mathcal{N}(\mu_1, \sigma^2)$ if $k \leq L$, and $\lambda_k \sim \mathcal{N}(\mu_2, \sigma^2)$ if $k > L$. (It is important that σ^2 be the same in both models, to prevent overfitting in the case where one regime has less data than the other.) Within each of the two regimes, we assume the λ_k are iid, which is obviously incorrect, but is adequate for our present purposes. We can fit this model for each $L = 1 : L_{max}$ by partitioning the data and computing the MLEs, using a pooled estimate of the variance:

$$\mu_1(L) = \frac{\sum_{k \leq L} \lambda_k}{L}, \quad \mu_2(L) = \frac{\sum_{k > L} \lambda_k}{N - L} \quad (12.70)$$

$$\sigma^2(L) = \frac{\sum_{k \leq L} (\lambda_k - \mu_1(L))^2 + \sum_{k > L} (\lambda_k - \mu_2(L))^2}{N} \quad (12.71)$$

We can then evaluate the **profile log likelihood**

$$\ell(L) = \sum_{k=1}^L \log \mathcal{N}(\lambda_k | \mu_1(L), \sigma^2(L)) + \sum_{k=L+1}^K \log \mathcal{N}(\lambda_k | \mu_2(L), \sigma^2(L)) \quad (12.72)$$

Finally, we choose $L^* = \arg \max \ell(L)$. This is illustrated in Figure 12.16. On the left, we plot the scree plot, which has the same shape as in Figure 12.14(a). On the right, we plot the profile

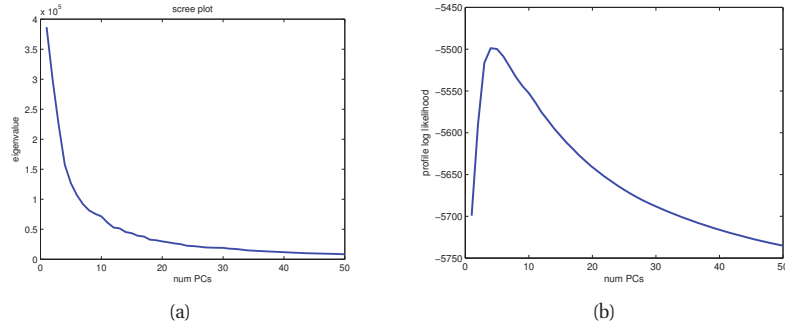


Figure 12.16 (a) Scree plot for training set, corresponding to Figure 12.14(a). (b) Profile likelihood. Figure generated by `pcaOverfitDemo`.

likelihood. Rather miraculously, we see a fairly well-determined peak.

12.4 PCA for categorical data

In this section, we consider extending the factor analysis model to the case where the observed data is categorical rather than real-valued. That is, the data has the form $y_{ij} \in \{1, \dots, C\}$, where $j = 1 : R$ is the number of observed response variables. We assume each y_{ij} is generated from a latent variable $\mathbf{z}_i \in \mathbb{R}^L$, with a Gaussian prior, which is passed through the softmax function as follows:

$$p(\mathbf{z}_i) = \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (12.73)$$

$$p(y_i | \mathbf{z}_i, \boldsymbol{\theta}) = \prod_{r=1}^R \text{Cat}(y_{ir} | \mathcal{S}(\mathbf{W}_r^T \mathbf{z}_i + \mathbf{w}_{0r})) \quad (12.74)$$

where $\mathbf{W}_r \in \mathbb{R}^{L \times M}$ is the factor loading matrix for response j , and $\mathbf{w}_{0r} \in \mathbb{R}^M$ is the offset term for response r , and $\boldsymbol{\theta} = (\mathbf{W}_r, \mathbf{w}_{0r})_{r=1}^R$. (We need an explicit offset term, since clamping one element of $\mathbf{z}_i$ to 1 can cause problems when computing the posterior covariance.) As in factor analysis, we have defined the prior mean to be $\mathbf{m}_0 = \mathbf{0}$ and the prior covariance $\mathbf{V}_0 = \mathbf{I}$, since we can capture non-zero mean by changing $\mathbf{w}_{0j}$ and non-identity covariance by changing $\mathbf{W}_r$. We will call this **categorical PCA**. See Chapter 27 for a discussion of related models.

It is interesting to study what kinds of distributions we can induce on the observed variables by varying the parameters. For simplicity, we assume there is a single ternary response variable, so y_i lives in the 3d probability simplex. Figure 12.17 shows what happens when we vary the parameters of the prior, $\mathbf{m}_0$ and $\mathbf{V}_0$, which is equivalent to varying the parameters of the likelihood, $\mathbf{W}_1$ and $\mathbf{w}_{01}$. We see that this can define fairly complex distributions over the simplex. This induced distribution is known as the **logistic normal** distribution (Aitchison 1982).

We can fit this model to data using a modified version of EM. The basic idea is to infer a Gaussian approximation to the posterior $p(\mathbf{z}_i | y_i, \boldsymbol{\theta})$ in the E step, and then to maximize $\boldsymbol{\theta}$ in the M step. The details for the multiclass case, can be found in (Khan et al. 2010) (see

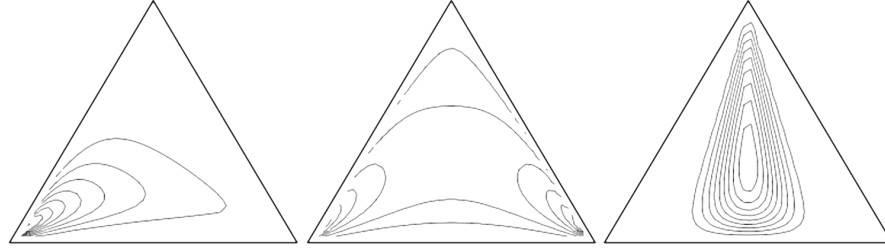


Figure 12.17 Some examples of the logistic normal distribution defined on the 3d simplex. (a) Diagonal covariance and non-zero mean. (b) Negative correlation between states 1 and 2. (c) Positive correlation between states 1 and 2. Source: Figure 1 of (Blei and Lafferty 2007). Used with kind permission of David Blei.

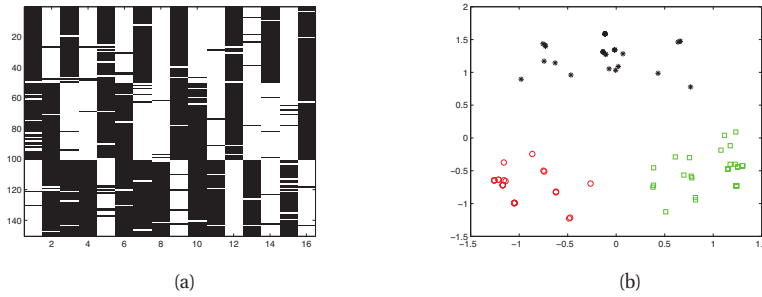


Figure 12.18 Left: 150 synthetic 16 dimensional bit vectors. Right: the 2d embedding learned by binary PCA, using variational EM. We have color coded points by the identity of the true “prototype” that generated them. Figure generated by `binaryFaDemoTipping`.

also Section 21.8.1.1). The details for the binary case for the the sigmoid link can be found in Exercise 21.9, and for the probit link in Exercise 21.10.

One application of such a model is to visualize high dimensional categorical data. Figure 12.18(a) shows a simple example where we have 150 6-dimensional bit vectors. It is clear that each sample is just a noisy copy of one of three binary prototypes. We fit a 2d catFA to this model, yielding approximate MLEs $\hat{\theta}$. In Figure 12.18(b), we plot $\mathbb{E}[\mathbf{z}_i | \mathbf{x}_i, \hat{\theta}]$. We see that there are three distinct clusters, as is to be expected.

In (Khan et al. 2010), we show that this model outperforms finite mixture models on the task of imputing missing entries in design matrices consisting of real and categorical data. This is useful for analysing social science survey data, which often has missing data and variables of mixed type.

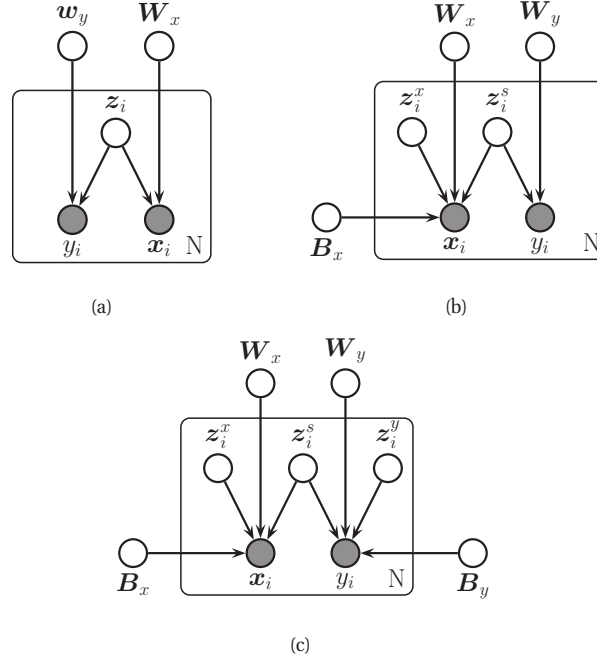


Figure 12.19 Gaussian latent factor models for paired data. (a) Supervised PCA. (b) Partial least squares. (c) Canonical correlation analysis.

12.5 PCA for paired and multi-view data

It is common to have a pair of related datasets, e.g., gene expression and gene copy number, or movie ratings by users and movie reviews. It is natural to want to combine these together into a low-dimensional embedding. This is an example of **data fusion**. In some cases, we might want to predict one element of the pair, say x_{i1} , from the other one, x_{i2} , via the low-dimensional “bottleneck”.

Below we discuss various latent Gaussian models for these tasks, following the presentation of (Virtanen 2010). The models easily generalize from pairs to sets of data, $\mathbf{x}_{im}$, for $m = 1 : M$. We focus on the case where $\mathbf{x}_{im} \in \mathbb{R}^{D_m}$. In this case, the joint distribution is multivariate Gaussian, so we can easily fit the models using EM, or Gibbs sampling.

We can generalize the models to handle discrete and count data by using the exponential family as a response distribution instead of the Gaussian, as we explain in Section 27.2.2. However, this will require the use of approximate inference in the E step (or an analogous modification to MCMC).

12.5.1 Supervised PCA (latent factor regression)

Consider the following model, illustrated in Figure 12.19(a):

$$p(\mathbf{z}_i) = \mathcal{N}(\mathbf{0}, \mathbf{I}_L) \quad (12.75)$$

$$p(y_i|\mathbf{z}_i) = \mathcal{N}(\mathbf{w}_y^T \mathbf{z}_i + \mu_y, \sigma_y^2) \quad (12.76)$$

$$p(\mathbf{x}_i|\mathbf{z}_i) = \mathcal{N}(\mathbf{W}_x \mathbf{z}_i + \boldsymbol{\mu}_x, \sigma_x^2 \mathbf{I}_D) \quad (12.77)$$

In (Yu et al. 2006), this is called **supervised PCA**. In (West 2003), this is called **Bayesian factor regression**. This model is like PCA, except that the target variable y_i is taken into account when learning the low dimensional embedding. Since the model is jointly Gaussian, we have

$$y_i|\mathbf{x}_i \sim \mathcal{N}(\mathbf{x}_i^T \mathbf{w}, \sigma_y^2 + \mathbf{w}_y^T \mathbf{C} \mathbf{w}_y) \quad (12.78)$$

where $\mathbf{w} = \boldsymbol{\Psi}^{-1} \mathbf{W}_x \mathbf{C} \mathbf{w}_y$, $\boldsymbol{\Psi} = \sigma_x^2 \mathbf{I}_D$, and $\mathbf{C}^{-1} = \mathbf{I} + \mathbf{W}_x^T \boldsymbol{\Psi}^{-1} \mathbf{W}_x$. So although this is a joint density model of $(y_i, \mathbf{x}_i)$, we can infer the implied conditional distribution.

We now show an interesting connection to Zellner's g-prior. Suppose $p(\mathbf{w}_y) = \mathcal{N}(\mathbf{0}, \frac{1}{g} \boldsymbol{\Sigma}^2)$, and let $\mathbf{X} = \mathbf{R} \mathbf{V}^T$ be the SVD of $\mathbf{X}$, where $\mathbf{V}^T \mathbf{V} = \mathbf{I}$ and $\mathbf{R}^T \mathbf{R} = \boldsymbol{\Sigma}^2 = \text{diag}(\sigma_j^2)$ contains the squared singular values. Then one can show (West 2003) that

$$p(\mathbf{w}) = \mathcal{N}(\mathbf{0}, g \mathbf{V}^{-T} \boldsymbol{\Sigma}^{-2} \mathbf{V}^{-1}) = \mathcal{N}(\mathbf{0}, g(\mathbf{X}^T \mathbf{X})^{-1}) \quad (12.79)$$

So the dependence of the prior for $\mathbf{w}$ on $\mathbf{X}$ arises from the fact that $\mathbf{w}$ is derived indirectly by a joint model of $\mathbf{X}$ and $\mathbf{y}$.

The above discussion focussed on regression. (Guo 2009) generalizes CCA to the exponential family, which is more appropriate if $\mathbf{x}_i$ and/or y_i are discrete. Although we can no longer compute the conditional $p(y_i|\mathbf{x}_i, \boldsymbol{\theta})$ in closed form, the model has a similar interpretation to the regression case, namely that we are predicting the response via a latent “bottleneck”.

The basic idea of compressing $\mathbf{x}_i$ to predict y_i can be formulated using information theory. In particular, we might want to find an encoding distribution $p(\mathbf{z}|\mathbf{x})$ such that we minimize

$$\mathbb{I}(X; Z) - \beta \mathbb{I}(X; Y) \quad (12.80)$$

where $\beta \geq 0$ is some parameter controlling the tradeoff between compression and predictive accuracy. This is known as the **information bottleneck** (Tishby et al. 1999). Often Z is taken to be discrete, as in clustering. However, in the Gaussian case, IB is closely related to CCA (Chechik et al. 2005).

We can easily generalize CCA to the case where y_i is a vector of responses to be predicted, as in multi-label classification. (Ma et al. 2008; Williamson and Ghahramani 2008) used this model to perform collaborative filtering, where the goal is to predict $y_{ij} \in \{1, \dots, 5\}$, the rating person i gives to movie j , where the “side information” $\mathbf{x}_i$ takes the form of a list of i 's friends. The intuition behind this approach is that knowledge of who your friends are, as well as the ratings of all other users, should help predict which movies you will like. In general, any setting where the tasks are correlated could benefit from CCA. Once we adopt a probabilistic view, various extensions are straightforward. For example, we can easily generalize to the semi-supervised case, where we do not observe y_i for all i (Yu et al. 2006).

12.5.1.1 Discriminative supervised PCA

One problem with this model is that it puts as much weight on predicting the inputs $\mathbf{x}_i$ as the outputs $\mathbf{y}_i$. This can be partially alleviated by using a weighted objective of the following form (Rish et al. 2008):

$$\ell(\boldsymbol{\theta}) = \prod_i p(\mathbf{y}_i | \boldsymbol{\eta}_{iy})^{\alpha_y} p(\mathbf{x}_i | \boldsymbol{\eta}_{ix})^{\alpha_x} \quad (12.81)$$

where the α_m control the relative importance of the data sources, and $\boldsymbol{\eta}_{im} = \mathbf{W}_m \mathbf{z}_i$. For Gaussian data, we can see that α_m just controls the noise variance:

$$\ell(\boldsymbol{\theta}) \propto \prod_i \exp(-\frac{1}{2}\alpha_x \|\mathbf{x}_i^T - \boldsymbol{\eta}_{ix}\|^2) \exp(-\frac{1}{2}\alpha_y \|\mathbf{y}_i^T - \boldsymbol{\eta}_{iy}\|^2) \quad (12.82)$$

This interpretation holds more generally for the exponential family. Note, however, that it is hard to estimate the α_m parameters, because changing them changes the normalization constant of the likelihood. We give an alternative approach to weighting $\mathbf{y}$ more heavily below.

12.5.2 Partial least squares

The technique of **partial least squares (PLS)** (Gustafsson 2001; Sun et al. 2009) is an asymmetric or more “discriminative” form of supervised PCA. The key idea is to allow some of the (co)variance in the input features to be explained by its own subspace, $\mathbf{z}_i^x$, and to let the rest of the subspace, $\mathbf{z}_i^s$, be shared between input and output. The model has the form

$$p(\mathbf{z}_i) = \mathcal{N}(\mathbf{z}_i^s | \mathbf{0}, \mathbf{I}_{L_s}) \mathcal{N}(\mathbf{z}_i^x | \mathbf{0}, \mathbf{I}_{L_x}) \quad (12.83)$$

$$p(\mathbf{y}_i | \mathbf{z}_i) = \mathcal{N}(\mathbf{W}_y \mathbf{z}_i^s + \boldsymbol{\mu}_y, \sigma^2 \mathbf{I}_{D_y}) \quad (12.84)$$

$$p(\mathbf{x}_i | \mathbf{z}_i) = \mathcal{N}(\mathbf{W}_x \mathbf{z}_i^s + \mathbf{B}_x \mathbf{z}_i^x + \boldsymbol{\mu}_x, \sigma^2 \mathbf{I}_{D_x}) \quad (12.85)$$

See Figure 12.19(b). The corresponding induced distribution on the visible variables has the form

$$p(\mathbf{v}_i | \boldsymbol{\theta}) = \int \mathcal{N}(\mathbf{v}_i | \mathbf{W} \mathbf{z}_i + \boldsymbol{\mu}, \sigma^2 \mathbf{I}) \mathcal{N}(\mathbf{z}_i | \mathbf{0}, \mathbf{I}) d\mathbf{z}_i = \mathcal{N}(\mathbf{v}_i | \boldsymbol{\mu}, \mathbf{W} \mathbf{W}^T + \sigma^2 \mathbf{I}) \quad (12.86)$$

where $\mathbf{v}_i = (\mathbf{x}_i; \mathbf{y}_i)$, $\boldsymbol{\mu} = (\boldsymbol{\mu}_y; \boldsymbol{\mu}_x)$ and

$$\mathbf{W} = \begin{pmatrix} \mathbf{W}_y & \mathbf{0} \\ \mathbf{W}_x & \mathbf{B}_x \end{pmatrix} \quad (12.87)$$

$$\mathbf{W} \mathbf{W}^T = \begin{pmatrix} \mathbf{W}_y \mathbf{W}_y^T & \mathbf{W}_x \mathbf{W}_x^T \\ \mathbf{W}_x \mathbf{W}_y^T & \mathbf{W}_x \mathbf{W}_x^T + \mathbf{B}_x \mathbf{B}_x^T \end{pmatrix} \quad (12.88)$$

We should choose L large enough so that the shared subspace does not capture covariate-specific variation.

This model can be easily generalized to discrete data using the exponential family (Virtanen 2010).

12.5.3 Canonical correlation analysis

Canonical correlation analysis or **CCA** is like a symmetric unsupervised version of PLS: it allows each view to have its own “private” subspace, but there is also a shared subspace. If we have two observed variables, $\mathbf{x}_i$ and $\mathbf{y}_i$, then we have three latent variables, $\mathbf{z}_i^s \in \mathbb{R}^{L_0}$ which is shared, $\mathbf{z}_i^x \in \mathbb{R}^{L_x}$ and $\mathbf{z}_i^y \in \mathbb{R}^{L_y}$ which are private. We can write the model as follows (Bach and Jordan 2005):

$$p(\mathbf{z}_i) = \mathcal{N}(\mathbf{z}_i^s | \mathbf{0}, \mathbf{I}_{L_0}) \mathcal{N}(\mathbf{z}_i^x | \mathbf{0}, \mathbf{I}_{L_x}) \mathcal{N}(\mathbf{z}_i^y | \mathbf{0}, \mathbf{I}_{L_y}) \quad (12.89)$$

$$p(\mathbf{x}_i | \mathbf{z}_i) = \mathcal{N}(\mathbf{x}_i | \mathbf{B}_x \mathbf{z}_i^x + \mathbf{W}_x \mathbf{z}_i^s + \boldsymbol{\mu}_x, \sigma^2 \mathbf{I}_{D_x}) \quad (12.90)$$

$$p(\mathbf{y}_i | \mathbf{z}_i) = \mathcal{N}(\mathbf{y}_i | \mathbf{B}_y \mathbf{z}_i^y + \mathbf{W}_y \mathbf{z}_i^s + \boldsymbol{\mu}_y, \sigma^2 \mathbf{I}_{D_y}) \quad (12.91)$$

See Figure 12.19(c). The corresponding observed joint distribution has the form

$$p(\mathbf{v}_i | \boldsymbol{\theta}) = \int \mathcal{N}(\mathbf{v}_i | \mathbf{W} \mathbf{z}_i + \boldsymbol{\mu}, \sigma^2 \mathbf{I}) \mathcal{N}(\mathbf{z}_i | \mathbf{0}, \mathbf{I}) d\mathbf{z}_i = \mathcal{N}(\mathbf{v}_i | \boldsymbol{\mu}, \mathbf{W} \mathbf{W}^T + \sigma^2 \mathbf{I}_D) \quad (12.92)$$

where

$$\mathbf{W} = \begin{pmatrix} \mathbf{W}_x & \mathbf{B}_x & \mathbf{0} \\ \mathbf{W}_y & \mathbf{0} & \mathbf{B}_y \end{pmatrix} \quad (12.93)$$

$$\mathbf{W} \mathbf{W}^T = \begin{pmatrix} \mathbf{W}_x \mathbf{W}_x^T + \mathbf{B}_x \mathbf{B}_x^T & \mathbf{W}_x \mathbf{W}_y^T \\ \mathbf{W}_y \mathbf{W}_x^T & \mathbf{W}_y \mathbf{W}_y^T + \mathbf{B}_y \mathbf{B}_y^T \end{pmatrix} \quad (12.94)$$

One can compute the MLE for this model using EM. (Bach and Jordan 2005) show that the resulting MLE is equivalent (up to rotation and scaling) to the classical, non-probabilistic view. However, the advantages of the probabilistic view are many: we can trivially generalize to $M > 2$ observed variables; we can create mixtures of CCA (Viinikanoja et al. 2010); we can create sparse versions of CCA using ARD (Archambeau and Bach 2008); we can generalize to the exponential family (Klami et al. 2010); we can perform Bayesian inference of the parameters (Wang 2007; Klami and Kaski 2008); we can handle non-parametric sparsity-promoting priors for $\mathbf{W}$ and $\mathbf{B}$ (Rai and Daume 2009); and so on.

12.6 Independent Component Analysis (ICA)

Consider the following situation. You are in a crowded room and many people are speaking. Your ears essentially act as two microphones, which are listening to a linear combination of the different speech signals in the room. Your goal is to deconvolve the mixed signals into their constituent parts. This is known as the **cocktail party problem**, and is an example of **blind signal separation** (BSS), or **blind source separation**, where “blind” means we know “nothing” about the source of the signals. Besides the obvious applications to acoustic signal processing, this problem also arises when analysing EEG and MEG signals, financial data, and any other dataset (not necessarily temporal) where latent sources or factors get mixed together in a linear way.

We can formalize the problem as follows. Let $\mathbf{x}_t \in \mathbb{R}^D$ be the observed signal at the sensors at “time” t , and $\mathbf{z}_t \in \mathbb{R}^L$ be the vector of source signals. We assume that

$$\mathbf{x}_t = \mathbf{W} \mathbf{z}_t + \boldsymbol{\epsilon}_t \quad (12.95)$$

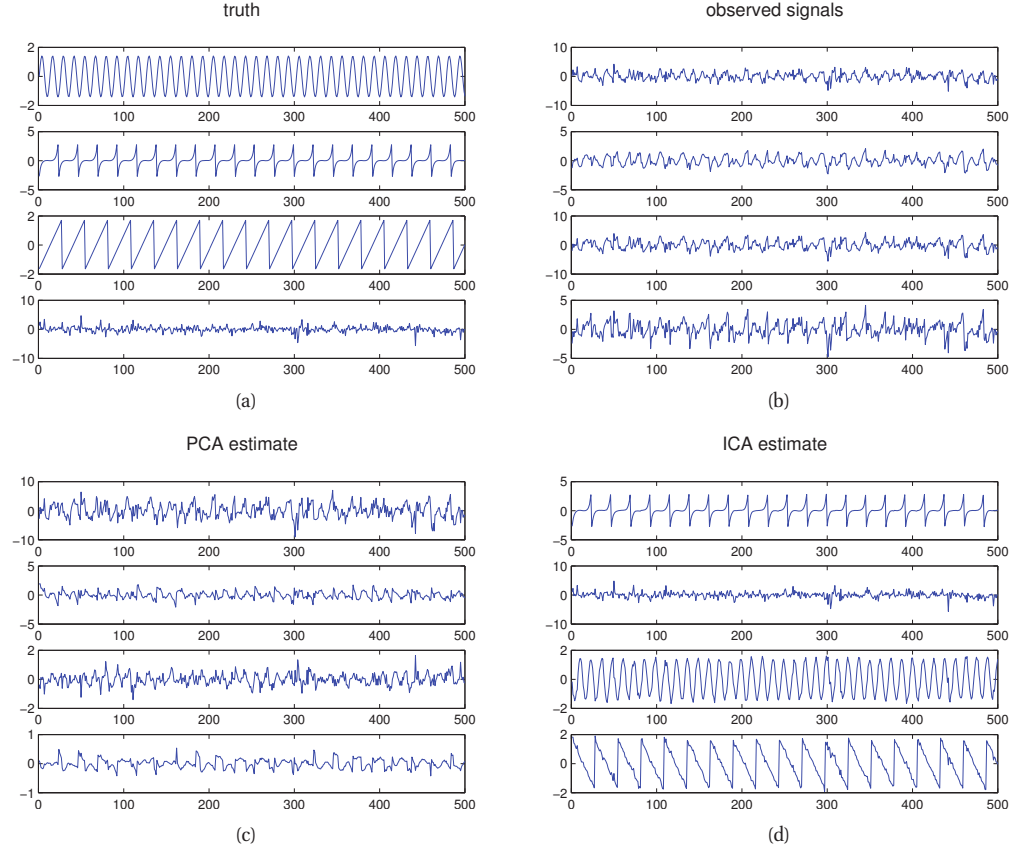


Figure 12.20 Illustration of ICA applied to 500 iid samples of a 4d source signal. (a) Latent signals. (b) Observations. (c) PCA estimate. (d) ICA estimate. Figure generated by `icaDemo`, written by Aapo Hyvärinen.

where $\mathbf{W}$ is an $D \times L$ matrix, and $\epsilon_t \sim \mathcal{N}(\mathbf{0}, \Psi)$. In this section, we treat each time point as an independent observation, i.e., we do not model temporal correlation (so we could replace the t index with i , but we stick with t to be consistent with much of the ICA literature). The goal is to infer the source signals, $p(\mathbf{z}_t | \mathbf{x}_t, \boldsymbol{\theta})$, as illustrated in Figure 12.20. In this context, $\mathbf{W}$ is called the **mixing matrix**. If $L = D$ (number of sources = number of sensors), it will be a square matrix. Often we will assume the noise level, $|\Psi|$, is zero, for simplicity.

So far, the model is identical to factor analysis (or PCA if there is no noise, except we don't in general require orthogonality of $\mathbf{W}$). However, we will use a different prior for $p(\mathbf{z}_t)$. In PCA, we assume each source is independent, and has a Gaussian distribution

$$p(\mathbf{z}_t) = \prod_{j=1}^L \mathcal{N}(z_{tj} | 0, 1) \quad (12.96)$$

We will now relax this Gaussian assumption and let the source distributions be any *non-Gaussian*

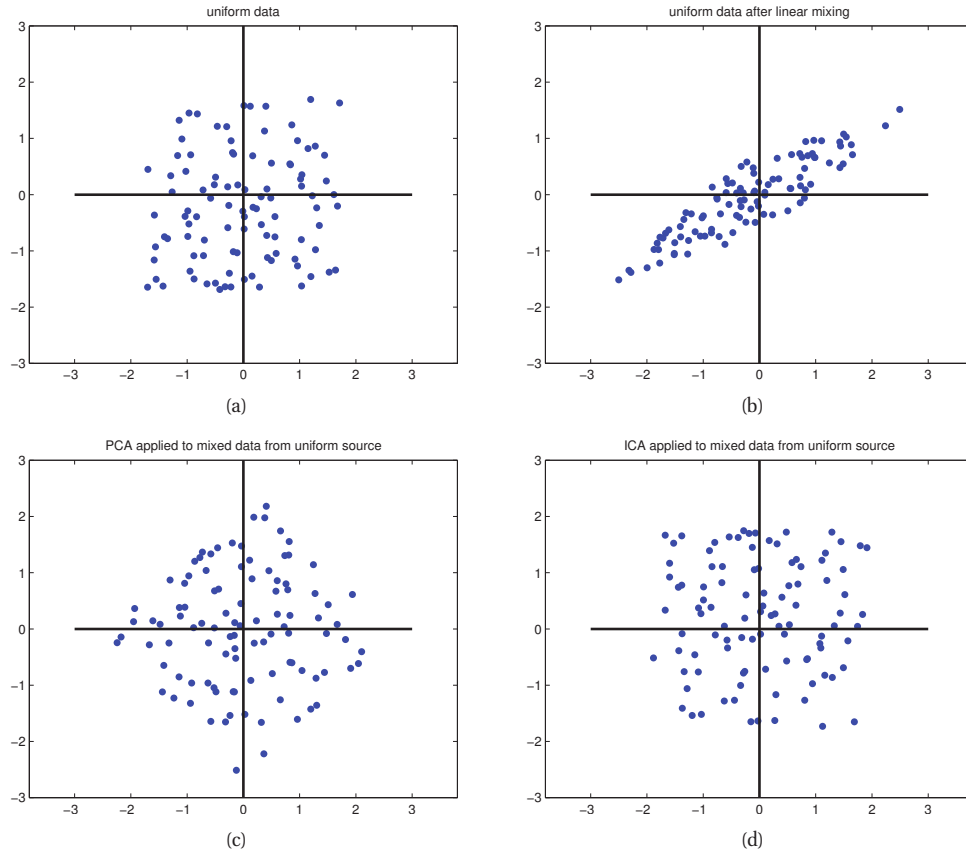


Figure 12.21 Illustration of ICA and PCA applied to 100 iid samples of a 2d source signal with a uniform distribution. (a) Latent signals. (b) Observations. (c) PCA estimate. (d) ICA estimate. Figure generated by `icaDemoUniform`, written by Aapo Hyvarinen.

distribution

$$p(\mathbf{z}_t) = \prod_{j=1}^L p_j(z_{tj}) \quad (12.97)$$

Without loss of generality, we can constrain the variance of the source distributions to be 1, because any other variance can be modelled by scaling the rows of $\mathbf{W}$ appropriately. The resulting model is known as **independent component analysis** or **ICA**.

The reason the Gaussian distribution is disallowed as a source prior in ICA is that it does not permit unique recovery of the sources, as illustrated in Figure 12.20(c). This is because the PCA likelihood is invariant to any orthogonal transformation of the sources $\mathbf{z}_t$ and mixing matrix $\mathbf{W}$. PCA can recover the best linear subspace in which the signals lie, but cannot uniquely recover the signals themselves.

To illustrate this, suppose we have two independent sources with uniform distributions, as shown in Figure 12.21(a). Now suppose we have the following mixing matrix

$$\mathbf{W} = \begin{pmatrix} 2 & 3 \\ 2 & 1 \end{pmatrix} \quad (12.98)$$

Then we observe the data shown in Figure 12.21(b) (assuming no noise). If we apply PCA followed by scaling to this, we get the result in Figure 12.21(c). This corresponds to a whitening of the data. To uniquely recover the sources, we need to perform an additional rotation. The trouble is, there is no information in the symmetric Gaussian posterior to tell us which angle to rotate by. In a sense, PCA solves “half” of the problem, since it identifies the linear subspace; all that ICA has to do is then to identify the appropriate rotation. (Hence we see that ICA is not that different from methods such as varimax, which seek good rotations of the latent factors to enhance interpretability.)

Figure 12.21(d) shows that ICA can recover the source, up to a permutation of the indices and possible sign change. ICA requires that $\mathbf{W}$ is square and hence invertible. In the non-square case (e.g., where we have more sources than sensors), we cannot uniquely recover the true signal, but we can compute the posterior $p(\mathbf{z}_t | \mathbf{x}_t, \hat{\mathbf{W}})$, which represents our beliefs about the source. In both cases, we need to estimate $\mathbf{W}$ as well as the source distributions p_j . We discuss how to do this below.

12.6.1 Maximum likelihood estimation

In this section, we discuss ways to estimate square mixing matrices $\mathbf{W}$ for the noise-free ICA model. As usual, we will assume that the observations have been centered; hence we can also assume $\mathbf{z}$ is zero-mean. In addition, we assume the observations have been whitened, which can be done with PCA.

If the data is centered and whitened, we have $\mathbb{E}[\mathbf{x}\mathbf{x}^T] = \mathbf{I}$. But in the noise free case, we also have

$$\text{cov}[\mathbf{x}] = \mathbb{E}[\mathbf{x}\mathbf{x}^T] = \mathbf{W}\mathbb{E}[\mathbf{z}\mathbf{z}^T]\mathbf{W}^T = \mathbf{W}\mathbf{W}^T \quad (12.99)$$

Hence we see that $\mathbf{W}$ must be orthogonal. This reduces the number of parameters we have to estimate from D^2 to $D(D-1)/2$. It will also simplify the math and the algorithms.

Let $\mathbf{V} = \mathbf{W}^{-1}$; these are often called the **recognition weights**, as opposed to $\mathbf{W}$, which are the **generative weights**.⁴

Since $\mathbf{x} = \mathbf{W}\mathbf{z}$, we have, from Equation 2.89,

$$p_x(\mathbf{W}\mathbf{z}_t) = p_z(\mathbf{z}_t) |\det(\mathbf{W}^{-1})| = p_z(\mathbf{V}\mathbf{x}_t) |\det(\mathbf{V})| \quad (12.100)$$

Hence we can write the log-likelihood, assuming T iid samples, as follows:

$$\frac{1}{T} \log p(\mathcal{D} | \mathbf{V}) = \log |\det(\mathbf{V})| + \frac{1}{T} \sum_{j=1}^L \sum_{t=1}^T \log p_j(\mathbf{v}_j^T \mathbf{x}_t) \quad (12.101)$$

4. In the literature, it is common to denote the generative weights by $\mathbf{A}$ and the recognition weights by $\mathbf{W}$, but we are trying to be consistent with the notation used earlier in this chapter.

where $\mathbf{v}_j$ is the j 'th row of $\mathbf{V}$. Since we are constraining $\mathbf{V}$ to be orthogonal, the first term is a constant, so we can drop it. We can also replace the average over the data with an expectation operator to get the following objective

$$\text{NLL}(\mathbf{V}) = \sum_{j=1}^L \mathbb{E}[G_j(z_j)] \quad (12.102)$$

where $z_j = \mathbf{v}_j^T \mathbf{x}$ and $G_j(z) \triangleq -\log p_j(z)$. We want to minimize this subject to the constraint that the rows of $\mathbf{V}$ are orthogonal. We also want them to be unit norm, since this ensures that the variance of the factors is unity (since, with whitened data, $\mathbb{E}[\mathbf{v}_j^T \mathbf{x}] = \|\mathbf{v}_j\|^2$), which is necessary to fix the scale of the weights. In otherwords, $\mathbf{V}$ should be an orthonormal matrix.

It is straightforward to derive a gradient descent algorithm to fit this model; however, it is rather slow. One can also derive a faster algorithm that follows the natural gradient; see e.g., (MacKay 2003, ch 34) for details. A popular alternative is to use an approximate Newton method, which we discuss in Section 12.6.2. Another approach is to use EM, which we discuss in Section 12.6.3.

12.6.2 The FastICA algorithm

We now describe the **fast ICA** algorithm, based on (Hyvarinen and Oja 2000), which we will show is an approximate Newton method for fitting ICA models.

For simplicity of presentation, we initially assume there is only one latent factor. In addition, we initially assume all source distributions are known and are the same, so we can just write $G(z) = -\log p(z)$. Let $g(z) = \frac{d}{dz}G(z)$. The constrained objective, and its gradient and Hessian, are given by

$$f(\mathbf{v}) = \mathbb{E}[G(\mathbf{v}^T \mathbf{x})] + \lambda(1 - \mathbf{v}^T \mathbf{v}) \quad (12.103)$$

$$\nabla f(\mathbf{v}) = \mathbb{E}[\mathbf{x}g(\mathbf{v}^T \mathbf{x})] - \beta \mathbf{v} \quad (12.104)$$

$$\mathbf{H}(\mathbf{v}) = \mathbb{E}[\mathbf{x}\mathbf{x}^T g'(\mathbf{v}^T \mathbf{x})] - \beta \mathbf{I} \quad (12.105)$$

where $\beta = 2\lambda$ is a Lagrange multiplier. Let us make the approximation

$$\mathbb{E}[\mathbf{x}\mathbf{x}^T g'(\mathbf{v}^T \mathbf{x})] \approx \mathbb{E}[\mathbf{x}\mathbf{x}^T] \mathbb{E}[g'(\mathbf{v}^T \mathbf{x})] = \mathbb{E}[g'(\mathbf{v}^T \mathbf{x})] \quad (12.106)$$

This makes the Hessian very easy to invert, giving rise to the following Newton update:

$$\mathbf{v}^* \triangleq \mathbf{v} - \frac{\mathbb{E}[\mathbf{x}g(\mathbf{v}^T \mathbf{x})] - \beta \mathbf{v}}{\mathbb{E}[g'(\mathbf{v}^T \mathbf{x})] - \beta} \quad (12.107)$$

One can rewrite this in the following way

$$\mathbf{v}^* \triangleq \mathbb{E}[\mathbf{x}g(\mathbf{v}^T \mathbf{x})] - \mathbb{E}[g'(\mathbf{v}^T \mathbf{x})] \mathbf{v} \quad (12.108)$$

(In practice, the expectations can be replaced by Monte Carlo estimates from the training set, which gives an efficient online learning algorithm.) After performing this update, one should project back onto the constraint surface using

$$\mathbf{v}^{new} \triangleq \frac{\mathbf{v}^*}{\|\mathbf{v}^*\|} \quad (12.109)$$

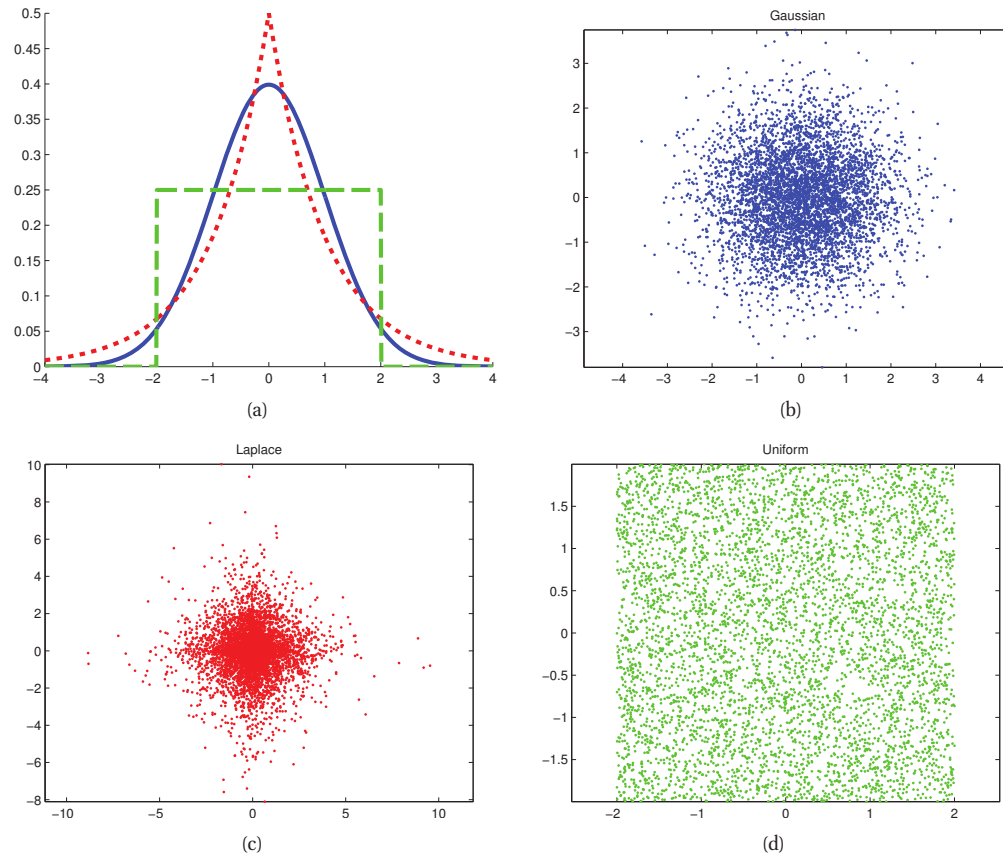


Figure 12.22 Illustration of Gaussian, sub-Gaussian (uniform) and super-Gaussian (Laplace) distributions in 1d and 2d. Figure generated by `subSuperGaussPlot`, written by Kevin Swersky.

One iterates this algorithm until convergence. (Due to the sign ambiguity of $\mathbf{v}$, the values of $\mathbf{v}$ may not converge, but the direction defined by this vector should converge, so one can assess convergence by monitoring $|\mathbf{v}^T \mathbf{v}^{new}|$, which should approach 1.)

Since the objective is not convex, there are multiple local optima. We can use this fact to learn multiple different weight vectors or **features**. We can either learn the features sequentially and then project out the part of $\mathbf{v}_j$ that lies in the subspace defined by earlier features, or we can learn them in parallel, and orthogonalize $\mathbf{V}$ in parallel. This latter approach is usually preferred, since, unlike PCA, the features are not ordered in any way. So the first feature is not “more important” than the second, and hence it is better to treat them symmetrically.

12.6.2.1 Modeling the source densities

So far, we have assumed that $G(z) = -\log p(z)$ is known. What kinds of models might be reasonable as signal priors? We know that using Gaussians (which correspond to quadratic functions for G) won't work. So we want some kind of non-Gaussian distribution. In general, there are several kinds of non-Gaussian distributions, such as the following:

- **Super-Gaussian distributions** These are distributions which have a big spike at the mean, and hence (in order to ensure unit variance) have heavy tails. The Laplace distribution is a classic example. See Figure 12.22. Formally, we say a distribution is **super-Gaussian** or **leptokurtic** (“lepto” coming from the Greek for “thin”) if $\text{kurt}(z) > 0$, where $\text{kurt}(z)$ is the **kurtosis** of the distribution, defined by

$$\text{kurt}(z) \triangleq \frac{\mu_4}{\sigma^4} - 3 \quad (12.110)$$

where σ is the standard deviation, and μ_k is the k 'th **central moment**, or moment about the mean:

$$\mu_k \triangleq \mathbb{E}[(X - \mathbb{E}[X])^k] \quad (12.111)$$

(So $\mu_1 = \mu$ is the mean, and $\mu_2 = \sigma^2$ is the variance.) It is conventional to subtract 3 in the definition of kurtosis to make the kurtosis of a Gaussian variable equal to zero.

- **Sub-Gaussian distributions** A **sub-Gaussian** or **platykurtic** (“platy” coming from the Greek for “broad”) distribution has negative kurtosis. These are distributions which are much flatter than a Gaussian. The uniform distribution is a classic example. See Figure 12.22.
- **Skewed distributions** Another way to “be non-Gaussian” is to be asymmetric. One measure of this is **skewness**, defined by

$$\text{skew}(z) \triangleq \frac{\mu_3}{\sigma^3} \quad (12.112)$$

An example of a (right) skewed distribution is the gamma distribution (see Figure 2.9).

When one looks at the empirical distribution of many natural signals, such as images and speech, when passed through certain linear filters, they tend to be very super-Gaussian. This result holds both for the kind of linear filters found in certain parts of the brain, such as the simple cells found in the primary visual cortex, as well as for the kinds of linear filters used in signal processing, such as wavelet transforms. One obvious choice for modeling natural signals with ICA is therefore the Laplace distribution. For mean zero and variance 1, this has a log pdf given by

$$\log p(z) = -\sqrt{2}|z| - \log(\sqrt{2}) \quad (12.113)$$

Since the Laplace prior is not differentiable at the origin, it is more common to use other, smoother super-Gaussian distributions. One example is the logistic distribution. The corresponding log pdf, for the case where the mean is zero and the variance is 1 (so $\mu = 0$ and $s = \frac{\sqrt{3}}{\pi}$), is given by the following:

$$\log p(z) = -2 \log \cosh\left(\frac{\pi}{2\sqrt{3}}z\right) - \log \frac{4\sqrt{3}}{\pi} \quad (12.114)$$

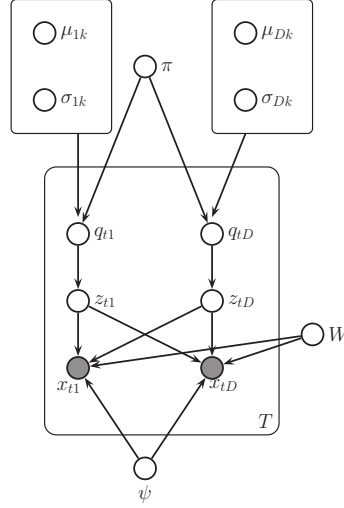


Figure 12.23 Modeling the source distributions using a mixture of univariate Gaussians (the independent factor analysis model of (Moulines et al. 1997; Attias 1999)).

Various ways of estimating $G(Z) = -\log p(z)$ are discussed in the seminal paper (Pham and Garrat 1997). However, when fitting ICA by maximum likelihood, it is not critical that the exact shape of the source distribution be known (although it is important to know whether it is sub or super Gaussian). Consequently, it is common to just use $G(z) = \sqrt{z}$ or $G(z) = \log \cosh(z)$ instead of the more complex expressions above.

12.6.3 Using EM

An alternative to assuming a particular form for $G(z)$, or equivalently for $p(z)$, is to use a flexible non-parametric density estimator, such as a mixture of (uni-variate) Gaussians:

$$p(q_j = k) = \pi_k \quad (12.115)$$

$$p(z_j | q_j = k) = \mathcal{N}(\mu_{j,k}, \sigma_{j,k}^2) \quad (12.116)$$

$$p(\mathbf{x} | \mathbf{z}) = \mathcal{N}(\mathbf{W}\mathbf{z}, \Psi) \quad (12.117)$$

This approach was proposed in (Moulines et al. 1997; Attias 1999), and the corresponding graphical model is shown in Figure 12.23.

It is possible to derive an exact EM algorithm for this model. The key observation is that it is possible to compute $\mathbb{E}[\mathbf{z}_t | \mathbf{x}_t, \boldsymbol{\theta}]$ exactly by summing over all K^L combinations of the $\mathbf{q}_t$ variables, where K is the number of mixture components per source. (If this is too expensive, one can use a variational mean field approximation (Attias 1999).) We can then estimate all the source distributions in parallel by fitting a standard GMM to $\mathbb{E}[\mathbf{z}_t]$. When the source GMMs are

known, we can compute the marginals $p_j(z_j)$ very easily, using

$$p_j(z_j) = \sum_{k=1}^K \pi_{j,k} \mathcal{N}(z_j | \mu_{j,k}, \sigma_{j,k}^2) \quad (12.118)$$

Given the p_j 's, we can then use an ICA algorithm to estimate $\mathbf{W}$. Of course, these steps should be interleaved. The details can be found in (Attias 1999).

12.6.4 Other estimation principles *

It is quite common to estimate the parameters of ICA models using methods that seem different to maximum likelihood. We will review some of these methods below, because they give additional insight into ICA. However, we will also see that these methods in fact are equivalent to maximum likelihood after all. Our presentation is based on (Hyvarinen and Oja 2000).

12.6.4.1 Maximizing non-Gaussianity

An early approach to ICA was to find a matrix $\mathbf{V}$ such that the distribution $\mathbf{z} = \mathbf{V}\mathbf{x}$ is as far from Gaussian as possible. (There is a related approach in statistics called **projection pursuit**.) One measure of non-Gaussianity is kurtosis, but this can be sensitive to outliers. Another measure is the **negentropy**, defined as

$$\text{negentropy}(z) \triangleq \mathbb{H}(\mathcal{N}(\mu, \sigma^2)) - \mathbb{H}(z) \quad (12.119)$$

where $\mu = \mathbb{E}[z]$ and $\sigma^2 = \text{var}[z]$. Since the Gaussian is the maximum entropy distribution, this measure is always non-negative and becomes large for distributions that are highly non-Gaussian.

We can define our objective as maximizing

$$J(\mathbf{V}) = \sum_j \text{negentropy}(z_j) = \sum_j \mathbb{H}(\mathcal{N}(\mu_j, \sigma_j^2)) - \mathbb{H}(z_j) \quad (12.120)$$

where $\mathbf{z} = \mathbf{V}\mathbf{x}$. If we fix $\mathbf{V}$ to be orthogonal, and if we whiten the data, the covariance of $\mathbf{z}$ will be $\mathbf{I}$ independently of $\mathbf{V}$, so the first term is a constant. Hence

$$J(\mathbf{V}) = \sum_j -\mathbb{H}(z_j) + \text{const} = \sum_j \mathbb{E}[\log p(z_j)] + \text{const} \quad (12.121)$$

which we see is equal (up to a sign change, and irrelevant constants) to the log-likelihood in Equation 12.102.

12.6.4.2 Minimizing mutual information

One measure of dependence of a set of random variables is the **multi-information**:

$$I(\mathbf{z}) \triangleq \mathbb{KL}\left(p(\mathbf{z}) \parallel \prod_j p(z_j)\right) = \sum_j \mathbb{H}(z_j) - \mathbb{H}(\mathbf{z}) \quad (12.122)$$

We would like to minimize this, since we are trying to find independent components. Put another way, we want the best possible factored approximation to the joint distribution.

Now since $\mathbf{z} = \mathbf{V}\mathbf{x}$, we have

$$I(\mathbf{z}) = \sum_j \mathbb{H}(z_j) - \mathbb{H}(\mathbf{V}\mathbf{x}) \quad (12.123)$$

If we constrain $\mathbf{V}$ to be orthogonal, we can drop the last term, since then $\mathbb{H}(\mathbf{V}\mathbf{x}) = \mathbb{H}(\mathbf{x})$ (since multiplying by $\mathbf{V}$ does not change the shape of the distribution), and $\mathbb{H}(\mathbf{x})$ is a constant which is solely determined by the empirical distribution. Hence we have $I(\mathbf{z}) = \sum_j \mathbb{H}(z_j)$. Minimizing this is equivalent to maximizing the negentropy, which is equivalent to maximum likelihood.

12.6.4.3 Maximizing mutual information (infomax)

Instead of trying to minimize the mutual information between the components of $\mathbf{z}$, let us imagine a neural network where $\mathbf{x}$ is the input and $y_j = \phi(\mathbf{v}_j^T \mathbf{x}) + \epsilon$ is the noisy output, where ϕ is some nonlinear scalar function, and $\epsilon \sim \mathcal{N}(0, 1)$. It seems reasonable to try to maximize the information flow through this system, a principle known as **infomax**. (Bell and Sejnowski 1995). That is, we want to maximize the mutual information between $\mathbf{y}$ (the internal neural representation) and $\mathbf{x}$ (the observed input signal). We have $\mathbb{I}(\mathbf{x}; \mathbf{y}) = \mathbb{H}(\mathbf{y}) - \mathbb{H}(\mathbf{y}|\mathbf{x})$, where the latter term is constant if we assume the noise has constant variance. One can show that we can approximate the former term as follows

$$\mathbb{H}(\mathbf{y}) = \sum_{j=1}^L \mathbb{E} [\log \phi'(\mathbf{v}_j^T \mathbf{x})] + \log |\det(\mathbf{V})| \quad (12.124)$$

where, as usual, we can drop the last term if $\mathbf{V}$ is orthogonal. If we define $\phi(z)$ to be a cdf, then $\phi'(z)$ is its pdf, and the above expression is equivalent to the log likelihood. In particular, if we use a logistic nonlinearity, $\phi(z) = \text{sigm}(z)$, then the corresponding pdf is the logistic distribution, and $\log \phi'(z) = \log \cosh(z)$ (ignoring irrelevant constants). Thus we see that infomax is equivalent to maximum likelihood.

Exercises

Exercise 12.1 M step for FA

For the FA model, show that the MLE in the M step for $\mathbf{W}$ is given by Equation 12.23.

Exercise 12.2 MAP estimation for the FA model

Derive the M step for the FA model using conjugate priors for the parameters.

Exercise 12.3 Heuristic for assessing applicability of PCA

(Source: (Press 2005, Q9.8)). Let the empirical covariance matrix Σ have eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d > 0$. Explain why the variance of the evalues, $\sigma^2 = \frac{1}{d} \sum_{i=1}^d (\lambda_i - \bar{\lambda})^2$ is a good measure of whether or not PCA would be useful for analysing the data (the higher the value of σ^2 the more useful PCA).

Exercise 12.4 Deriving the second principal component

a. Let

$$J(\mathbf{v}_2, \mathbf{z}_2) = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - z_{i1}\mathbf{v}_1 - z_{i2}\mathbf{v}_2)^T (\mathbf{x}_i - z_{i1}\mathbf{v}_1 - z_{i2}\mathbf{v}_2) \quad (12.125)$$

Show that $\frac{\partial J}{\partial \mathbf{z}_2} = 0$ yields $z_{i2} = \mathbf{v}_2^T \mathbf{x}_i$.

b. Show that the value of $\mathbf{v}_2$ that minimizes

$$\tilde{J}(\mathbf{v}_2) = -\mathbf{v}_2^T \mathbf{C} \mathbf{v}_2 + \lambda_2 (\mathbf{v}_2^T \mathbf{v}_2 - 1) + \lambda_{12} (\mathbf{v}_2^T \mathbf{v}_1 - 0) \quad (12.126)$$

is given by the eigenvector of $\mathbf{C}$ with the second largest eigenvalue. Hint: recall that $\mathbf{C} \mathbf{v}_1 = \lambda_1 \mathbf{v}_1$ and $\frac{\partial \mathbf{x}^T \mathbf{A} \mathbf{x}}{\partial \mathbf{x}} = (\mathbf{A} + \mathbf{A}^T) \mathbf{x}$.

Exercise 12.5 Deriving the residual error for PCA

a. Prove that

$$\|\mathbf{x}_i - \sum_{j=1}^K z_{ij} \mathbf{v}_j\|^2 = \mathbf{x}_i^T \mathbf{x}_i - \sum_{j=1}^K \mathbf{v}_j^T \mathbf{x}_i \mathbf{x}_i^T \mathbf{v}_j \quad (12.127)$$

Hint: first consider the case $K = 2$. Use the fact that $\mathbf{v}_j^T \mathbf{v}_j = 1$ and $\mathbf{v}_j^T \mathbf{v}_k = 0$ for $k \neq j$. Also, recall $z_{ij} = \mathbf{x}_i^T \mathbf{v}_j$.

b. Now show that

$$J_K \triangleq \frac{1}{n} \sum_{i=1}^n \left(\mathbf{x}_i^T \mathbf{x}_i - \sum_{j=1}^K \mathbf{v}_j^T \mathbf{x}_i \mathbf{x}_i^T \mathbf{v}_j \right) = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i^T \mathbf{x}_i - \sum_{j=1}^K \lambda_j \quad (12.128)$$

Hint: recall $\mathbf{v}_j^T \mathbf{C} \mathbf{v}_j = \lambda_j \mathbf{v}_j^T \mathbf{v}_j = \lambda_j$.

c. If $K = d$ there is no truncation, so $J_d = 0$. Use this to show that the error from only using $K < d$ terms is given by

$$J_K = \sum_{j=K+1}^d \lambda_j \quad (12.129)$$

Hint: partition the sum $\sum_{j=1}^d \lambda_j$ into $\sum_{j=1}^K \lambda_j$ and $\sum_{j=K+1}^d \lambda_j$.

Exercise 12.6 Derivation of Fisher's linear discriminant

Show that the maximum of $J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$ is given by $\mathbf{S}_B \mathbf{w} = \lambda \mathbf{S}_W \mathbf{w}$

where $\lambda = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$. Hint: recall that the derivative of a ratio of two scalars is given by $\frac{d}{dx} \frac{f(x)}{g(x)} = \frac{f'g - fg'}{g^2}$, where $f' = \frac{d}{dx} f(x)$ and $g' = \frac{d}{dx} g(x)$. Also, recall that $\frac{d}{d\mathbf{x}} \mathbf{x}^T \mathbf{A} \mathbf{x} = (\mathbf{A} + \mathbf{A}^T) \mathbf{x}$.

Exercise 12.7 PCA via successive deflation

Let $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k$ be the first k eigenvectors with largest eigenvalues of $\mathbf{C} = \frac{1}{n} \mathbf{X}^T \mathbf{X}$, i.e., the principal basis vectors. These satisfy

$$\mathbf{v}_j^T \mathbf{v}_k = \begin{cases} 0 & \text{if } j \neq k \\ 1 & \text{if } j = k \end{cases} \quad (12.130)$$

We will construct a method for finding the $\mathbf{v}_j$ sequentially.

As we showed in class, $\mathbf{v}_1$ is the first principal eigenvector of $\mathbf{C}$, and satisfies $\mathbf{C}\mathbf{v}_1 = \lambda_1\mathbf{v}_1$. Now define $\tilde{\mathbf{x}}_i$ as the orthogonal projection of $\mathbf{x}_i$ onto the space orthogonal to $\mathbf{v}_1$:

$$\tilde{\mathbf{x}}_i = \mathbf{P}_{\perp \mathbf{v}_1} \mathbf{x}_i = (\mathbf{I} - \mathbf{v}_1\mathbf{v}_1^T)\mathbf{x}_i \quad (12.131)$$

Define $\tilde{\mathbf{X}} = [\tilde{\mathbf{x}}_1; \dots; \tilde{\mathbf{x}}_n]$ as the **deflated matrix** of rank $d - 1$, which is obtained by removing from the d dimensional data the component that lies in the direction of the first principal direction:

$$\tilde{\mathbf{X}} = (\mathbf{I} - \mathbf{v}_1\mathbf{v}_1^T)^T \mathbf{X} = (\mathbf{I} - \mathbf{v}_1\mathbf{v}_1^T)\mathbf{X} \quad (12.132)$$

- a. Using the facts that $\mathbf{X}^T \mathbf{X} \mathbf{v}_1 = n\lambda_1 \mathbf{v}_1$ (and hence $\mathbf{v}_1^T \mathbf{X}^T \mathbf{X} = n\lambda_1 \mathbf{v}_1^T$) and $\mathbf{v}_1^T \mathbf{v}_1 = 1$, show that the covariance of the deflated matrix is given by

$$\tilde{\mathbf{C}} \triangleq \frac{1}{n} \tilde{\mathbf{X}}^T \tilde{\mathbf{X}} = \frac{1}{n} \mathbf{X}^T \mathbf{X} - \lambda_1 \mathbf{v}_1 \mathbf{v}_1^T \quad (12.133)$$

- b. Let $\mathbf{u}$ be the principal eigenvector of $\tilde{\mathbf{C}}$. Explain why $\mathbf{u} = \mathbf{v}_2$. (You may assume $\mathbf{u}$ is unit norm.)
- c. Suppose we have a simple method for finding the leading eigenvector and eigenvalue of a pd matrix, denoted by $[\lambda, \mathbf{u}] = f(\mathbf{C})$. Write some pseudo code for finding the first K principal basis vectors of $\mathbf{X}$ that only uses the special f function and simple vector arithmetic, i.e., your code should not use SVD or the `eig` function. Hint: this should be a simple iterative routine that takes 2-3 lines to write. The input is $\mathbf{C}$, K and the function f , the output should be $\mathbf{v}_j$ and λ_j for $j = 1 : K$. Do not worry about being syntactically correct.

Exercise 12.8 Latent semantic indexing

(Source: de Freitas.). In this exercise, we study a technique called **latent semantic indexing**, which applies SVD to a document by term matrix, to create a low-dimensional embedding of the data that is designed to capture semantic similarity of words.

The file `lsiDocuments.pdf` contains 9 documents on various topics. A list of all the 460 unique words/terms that occur in these documents is in `lsiWords.txt`. A document by term matrix is in `lsiMatrix.txt`.

- a. Let X be the transpose of `lsiMatrix`, so each column represents a document. Compute the SVD of X and make an approximation to it $\hat{X}$ using the first 2 singular values/ vectors. Plot the low dimensional representation of the 9 documents in 2D. You should get something like Figure 12.24.
- b. Consider finding documents that are about alien abductions. If you look at `lsiWords.txt`, there are 3 versions of this word, term 23 (“abducted”), term 24 (“abduction”) and term 25 (“abductions”). Suppose we want to find documents containing the word “abducted”. Documents 2 and 3 contain it, but document 1 does not. However, document 1 is clearly related to this topic. Thus LSI should also find document 1. Create a test document q containing the one word “abducted”, and project it into the 2D subspace to make $\hat{q}$. Now compute the cosine similarity between $\hat{q}$ and the low dimensional representation of all the documents. What are the top 3 closest matches?

Exercise 12.9 Imputation in a FA model

Derive an expression for $p(\mathbf{x}_h | \mathbf{x}_v, \boldsymbol{\theta})$ for a FA model.

Exercise 12.10 Efficiently evaluating the PPCA density

Derive an expression for $p(\mathbf{x} | \hat{\mathbf{W}}, \hat{\sigma}^2)$ for the PPCA model based on plugging in the MLEs and using the matrix inversion lemma.

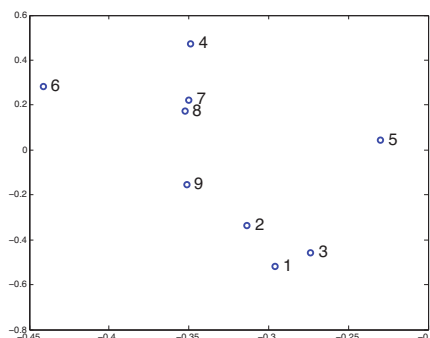


Figure 12.24 Projection of 9 documents into 2 dimensions. Figure generated by `lsiCode`.

Exercise 12.11 PPCA vs FA

(Source: Exercise 14.15 of (Hastie et al. 2009), due to Hinton.). Generate 200 observations from the following model, where $\mathbf{z}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$: $x_{i1} = z_{i1}$, $x_{i2} = z_{i1} + 0.001z_{i2}$, $x_{i3} = 10z_{i3}$. Fit a FA and PCA model with 1 latent factor. Hence show that the corresponding weight vector $\mathbf{w}$ aligns with the maximal variance direction (dimension 3) in the PCA case, but with the maximal correlation direction (dimensions 1+2) in the case of FA.

13

Sparse linear models

13.1 Introduction

We introduced the topic of feature selection in Section 3.5.4, where we discussed methods for finding input variables which had high mutual information with the output. The trouble with this approach is that it is based on a myopic strategy that only looks at one variable at a time. This can fail if there are interaction effects. For example, if $y = \text{xor}(x_1, x_2)$, then neither x_1 nor x_2 on its own can predict the response, but together they perfectly predict the response. For a real-world example of this, consider genetic association studies: sometimes two genes on their own may be harmless, but when present together they cause a recessive disease (Balding 2006).

In this chapter, we focus on selecting sets of variables at a time using a model-based approach. If the model is a generalized linear model, of the form $p(y|\mathbf{x}) = p(y|f(\mathbf{w}^T \mathbf{x}))$ for some link function f , then we can perform feature selection by encouraging the weight vector $\mathbf{w}$ to be **sparse**, i.e., to have lots of zeros. This approach turns out to offer significant computational advantages, as we will see below.

Here are some applications where feature selection/ sparsity is useful:

- In many problems, we have many more dimensions D than training cases N . The corresponding design matrix is short and fat, rather than tall and skinny. This is called the **small N , large D** problem. This is becoming increasingly prevalent as we develop more high throughput measurement devices. For example, with gene microarrays, it is common to measure the expression levels of $D \sim 10,000$ genes, but to only get $N \sim 100$ such examples. (It is perhaps a sign of the times that even our data seems to be getting fatter...) We may want to find the smallest set of features that can accurately predict the response (e.g., growth rate of the cell) in order to prevent overfitting, to reduce the cost of building a diagnostic device, or to help with scientific insight into the problem.
- In Chapter 14, we will use basis functions centered on the training examples, so $\phi(\mathbf{x}) = [\kappa(\mathbf{x}, \mathbf{x}_1), \dots, \kappa(\mathbf{x}, \mathbf{x}_N)]$, where κ is a kernel function. The resulting design matrix has size $N \times N$. Feature selection in this context is equivalent to selecting a subset of the training examples, which can help reduce overfitting and computational cost. This is known as a sparse kernel machine.
- In signal processing, it is common to represent signals (images, speech, etc.) in terms of wavelet basis functions. To save time and space, it is useful to find a sparse representation

of the signals, in terms of a small number of such basis functions. This allows us to estimate signals from a small number of measurements, as well as to compress the signal. See Section 13.8.3 for more information.

Note that the topic of feature selection and sparsity is currently one of the most active areas of machine learning/ statistics. In this chapter, we only have space to give an overview of the main results.

13.2 Bayesian variable selection

A natural way to pose the variable selection problem is as follows. Let $\gamma_j = 1$ if feature j is “relevant”, and let $\gamma_j = 0$ otherwise. Our goal is to compute the posterior over models

$$p(\gamma|\mathcal{D}) = \frac{e^{-f(\gamma)}}{\sum_{\gamma'} e^{-f(\gamma')}} \quad (13.1)$$

where $f(\gamma)$ is the cost function:

$$f(\gamma) \triangleq -[\log p(\mathcal{D}|\gamma) + \log p(\gamma)] \quad (13.2)$$

For example, suppose we generate $N = 20$ samples from a $D = 10$ dimensional linear regression model, $y_i \sim \mathcal{N}(\mathbf{w}^T \mathbf{x}_i, \sigma^2)$, in which $K = 5$ elements of $\mathbf{w}$ are non-zero. In particular, we use $\mathbf{w} = (0.00, -1.67, 0.13, 0.00, 0.00, 1.19, 0.00, -0.04, 0.33, 0.00)$ and $\sigma^2 = 1$. We enumerate all $2^{10} = 1024$ models and compute $p(\gamma|\mathcal{D})$ for each one (we give the equations for this below). We order the models in **Gray code** order, which ensures consecutive vectors differ by exactly 1 bit (the reasons for this are computational, and are discussed in Section 13.2.3).

The resulting set of bit patterns is shown in Figure 13.1(a). The cost of each model, $f(\gamma)$, is shown in Figure 13.1(b). We see that this objective function is extremely “bumpy”. The results are easier to interpret if we compute the posterior distribution over models, $p(\gamma|\mathcal{D})$. This is shown in Figure 13.1(c). The top 8 models are listed below:

model	prob	members
4	0.447	2,
61	0.241	2, 6,
452	0.103	2, 6, 9,
60	0.091	2, 3, 6,
29	0.041	2, 5,
68	0.021	2, 6, 7,
36	0.015	2, 5, 6,
5	0.010	2, 3,

The “true” model is $\{2, 3, 6, 8, 9\}$. However, the coefficients associated with features 3 and 8 are very small (relative to σ^2), so these variables are harder to detect. Given enough data, the method will converge on the true model (assuming the data is generated from a linear model), but for finite data sets, there will usually be considerable posterior uncertainty.

Interpreting the posterior over a large number of models is quite difficult, so we will seek various summary statistics. A natural one is the posterior mode, or MAP estimate

$$\hat{\gamma} = \operatorname{argmax}_{\gamma} p(\gamma|\mathcal{D}) = \operatorname{argmin}_{\gamma} f(\gamma) \quad (13.3)$$

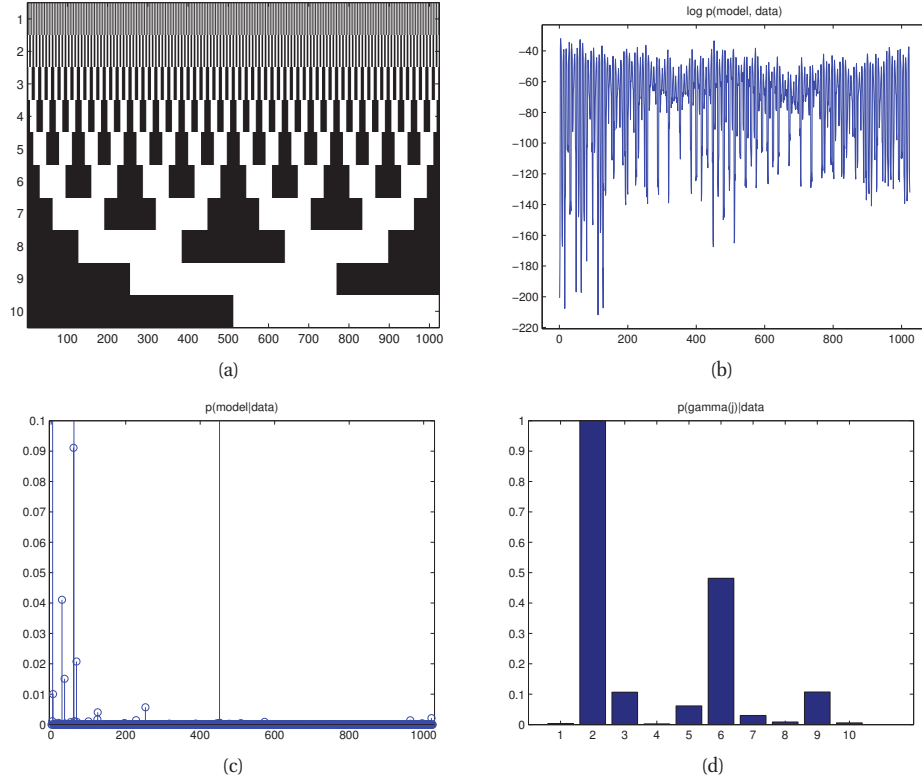


Figure 13.1 (a) All possible bit vectors of length 10 enumerated in Gray code order. (b) Score function for all possible models. (c) Posterior over all 1024 models. Vertical scale has been truncated at 0.1 for clarity. (d) Marginal inclusion probabilities. Figure generated by `linregAllsubsetsGraycodeDemo`.

However, the mode is often not representative of the full posterior mass (see Section 5.2.1.3). A better summary is the **median model** (Barbieri and Berger 2004; Carvahlo and Lawrence 2007), computed using

$$\hat{\gamma} = \{j : p(\gamma_j = 1|\mathcal{D}) > 0.5\} \quad (13.4)$$

This requires computing the posterior marginal **inclusion probabilities**, $p(\gamma_j = 1|\mathcal{D})$. These are shown in Figure 13.1(d). We see that the model is confident that variables 2 and 6 are included; if we lower the decision threshold to 0.1, we would add 3 and 9 as well. However, if we wanted to “capture” variable 8, we would incur two false positives (5 and 7). This tradeoff between false positives and false negatives is discussed in more detail in Section 5.7.2.1.

The above example illustrates the “gold standard” for variable selection: the problem was sufficiently small (only 10 variables) that we were able to compute the full posterior exactly. Of course, variable selection is most useful in the cases where the number of dimensions is large. Since there are 2^D possible models (bit vectors), it will be impossible to compute the full posterior in general, and even finding summaries, such as the MAP estimate or marginal

inclusion probabilities, will be intractable. We will therefore spend most of this chapter focussing on algorithmic speedups. But before we do that, we will explain how we computed $p(\gamma|\mathcal{D})$ in the above example.

13.2.1 The spike and slab model

The posterior is given by

$$p(\gamma|\mathcal{D}) \propto p(\gamma)p(\mathcal{D}|\gamma) \quad (13.5)$$

We first consider the prior, then the likelihood.

It is common to use the following prior on the bit vector:

$$p(\gamma) = \prod_{j=1}^D \text{Ber}(\gamma_j|\pi_0) = \pi_0^{||\gamma||_0} (1 - \pi_0)^{D - ||\gamma||_0} \quad (13.6)$$

where π_0 is the probability a feature is relevant, and $||\gamma||_0 = \sum_{j=1}^D \gamma_j$ is the ℓ_0 **pseudo-norm**, that is, the number of non-zero elements of the vector. For comparison with later models, it is useful to write the log prior as follows:

$$\log p(\gamma|\pi_0) = ||\gamma||_0 \log \pi_0 + (D - ||\gamma||_0) \log(1 - \pi_0) \quad (13.7)$$

$$= ||\gamma||_0 (\log \pi_0 - \log(1 - \pi_0)) + \text{const} \quad (13.8)$$

$$= -\lambda ||\gamma||_0 + \text{const} \quad (13.9)$$

where $\lambda \triangleq \log \frac{1-\pi_0}{\pi_0}$ controls the sparsity of the model.

We can write the likelihood as follows:

$$p(\mathcal{D}|\gamma) = p(\mathbf{y}|\mathbf{X}, \gamma) = \int \int p(\mathbf{y}|\mathbf{X}, \mathbf{w}, \gamma) p(\mathbf{w}|\gamma, \sigma^2) p(\sigma^2) d\mathbf{w} d\sigma^2 \quad (13.10)$$

For notational simplicity, we have assumed the response is centered, (i.e., $\bar{y} = 0$), so we can ignore any offset term μ .

We now discuss the prior $p(\mathbf{w}|\gamma, \sigma^2)$. If $\gamma_j = 0$, feature j is irrelevant, so we expect $w_j = 0$. If $\gamma_j = 1$, we expect w_j to be non-zero. If we standardize the inputs, a reasonable prior is $\mathcal{N}(0, \sigma_w^2 \sigma^2)$, where σ_w^2 controls how big we expect the coefficients associated with the relevant variables to be (which is scaled by the overall noise level σ^2). We can summarize this prior as follows:

$$p(w_j|\sigma^2, \gamma_j) = \begin{cases} \delta_0(w_j) & \text{if } \gamma_j = 0 \\ \mathcal{N}(w_j|0, \sigma_w^2 \sigma^2) & \text{if } \gamma_j = 1 \end{cases} \quad (13.11)$$

The first term is a “spike” at the origin. As $\sigma_w^2 \rightarrow \infty$, the distribution $p(w_j|\gamma_j = 1)$ approaches a uniform distribution, which can be thought of as a “slab” of constant height. Hence this is called the **spike and slab** model (Mitchell and Beauchamp 1988).

We can drop the coefficients w_j for which $w_j = 0$ from the model, since they are clamped to zero under the prior. Hence Equation 13.10 becomes the following (assuming a Gaussian likelihood):

$$p(\mathcal{D}|\gamma) = \int \int \mathcal{N}(\mathbf{y}|\mathbf{X}_\gamma \mathbf{w}_\gamma, \sigma^2 \mathbf{I}_N) \mathcal{N}(\mathbf{w}_\gamma|\mathbf{0}_{D_\gamma}, \sigma_w^2 \sigma^2 \mathbf{I}_{D_\gamma}) p(\sigma^2) d\mathbf{w}_\gamma d\sigma^2 \quad (13.12)$$

where $D_\gamma = \|\gamma\|_0$ is the number of non-zero elements in γ . In what follows, we will generalize this slightly by defining a prior of the form $p(\mathbf{w}|\gamma, \sigma^2) = \mathcal{N}(\mathbf{w}_\gamma|\mathbf{0}_{D_\gamma}, \sigma^2 \Sigma_\gamma)$ for any positive definite matrix Σ_γ .¹

Given these priors, we can now compute the marginal likelihood. If the noise variance is known, we can write down the marginal likelihood (using Equation 13.151) as follows:

$$p(\mathcal{D}|\gamma, \sigma^2) = \int \mathcal{N}(\mathbf{y}|\mathbf{X}_\gamma \mathbf{w}_\gamma, \sigma^2 \mathbf{I}) \mathcal{N}(\mathbf{w}_\gamma|\mathbf{0}, \sigma^2 \Sigma_\gamma) d\mathbf{w}_\gamma = \mathcal{N}(\mathbf{y}|\mathbf{0}, \mathbf{C}_\gamma) \quad (13.13)$$

$$\mathbf{C}_\gamma \triangleq \sigma^2 \mathbf{X}_\gamma \Sigma_\gamma \mathbf{X}_\gamma^T + \sigma^2 \mathbf{I}_N \quad (13.14)$$

If the noise is unknown, we can put a prior on it and integrate it out. It is common to use $p(\sigma^2) = \text{IG}(\sigma^2|a_\sigma, b_\sigma)$. Some guidelines on setting a, b can be found in (Kohn et al. 2001). If we use $a = b = 0$, we recover the Jeffrey's prior, $p(\sigma^2) \propto \sigma^{-2}$. When we integrate out the noise, we get the following more complicated expression for the marginal likelihood (Brown et al. 1998):

$$p(\mathcal{D}|\gamma) = \int \int p(\mathbf{y}|\gamma, \mathbf{w}_\gamma, \sigma^2) p(\mathbf{w}_\gamma|\gamma, \sigma^2) p(\sigma^2) d\mathbf{w}_\gamma d\sigma^2 \quad (13.15)$$

$$\propto |\mathbf{X}_\gamma^T \mathbf{X}_\gamma + \Sigma_\gamma^{-1}|^{-\frac{1}{2}} |\Sigma_\gamma|^{-\frac{1}{2}} (2b_\sigma + S(\gamma))^{-(2a_\sigma + N - 1)/2} \quad (13.16)$$

where $S(\gamma)$ is the RSS:

$$S(\gamma) \triangleq \mathbf{y}^T \mathbf{y} - \mathbf{y}^T \mathbf{X}_\gamma (\mathbf{X}_\gamma^T \mathbf{X}_\gamma + \Sigma_\gamma^{-1})^{-1} \mathbf{X}_\gamma^T \mathbf{y} \quad (13.17)$$

See also Exercise 13.4.

When the marginal likelihood cannot be computed in closed form (e.g., if we are using logistic regression or a nonlinear model), we can approximate it using BIC, which has the form

$$\log p(\mathcal{D}|\gamma) \approx \log p(\mathbf{y}|\mathbf{X}, \hat{\mathbf{w}}_\gamma, \hat{\sigma}^2) - \frac{\|\gamma\|_0}{2} \log N \quad (13.18)$$

where $\hat{\mathbf{w}}_\gamma$ is the ML or MAP estimate based on $\mathbf{X}_\gamma$, and $\|\gamma\|_0$ is the “degrees of freedom” of the model (Zou et al. 2007). Adding the log prior, the overall objective becomes

$$\log p(\gamma|\mathcal{D}) \approx \log p(\mathbf{y}|\mathbf{X}, \hat{\mathbf{w}}_\gamma, \hat{\sigma}^2) - \frac{\|\gamma\|_0}{2} \log N - \lambda \|\gamma\|_0 + \text{const} \quad (13.19)$$

We see that there are two complexity penalties: one arising from the BIC approximation to the marginal likelihood, and the other arising from the prior on $p(\gamma)$. Obviously these can be combined into one overall complexity parameter, which we will denote by λ .

13.2.2 From the Bernoulli-Gaussian model to ℓ_0 regularization

Another model that is sometimes used (e.g., (Kuo and Mallick 1998; Zhou et al. 2009; Soussen et al. 2010)) is the following:

$$y_i|\mathbf{x}_i, \mathbf{w}, \gamma, \sigma^2 \sim \mathcal{N}\left(\sum_j \gamma_j w_j x_{ij}, \sigma^2\right) \quad (13.20)$$

$$\gamma_j \sim \text{Ber}(\pi_0) \quad (13.21)$$

$$w_j \sim \mathcal{N}(0, \sigma_w^2) \quad (13.22)$$

1. It is common to use a g-prior of the form $\Sigma_\gamma = g(\mathbf{X}_\gamma^T \mathbf{X}_\gamma)^{-1}$ for reasons explained in Section 7.6.3.1 (see also Exercise 13.4). Various approaches have been proposed for setting g , including cross validation, empirical Bayes (Minka 2000b; George and Foster 2000), hierarchical Bayes (Liang et al. 2008), etc.

In the signal processing literature (e.g., (Soussen et al. 2010)), this is called the **Bernoulli-Gaussian** model, although we could also call it the **binary mask** model, since we can think of the γ_j variables as “masking out” the weights w_j .

Unlike the spike and slab model, we do not integrate out the “irrelevant” coefficients; they always exist. In addition, the binary mask model has the form $\gamma_j \rightarrow \mathbf{y} \leftarrow w_j$, whereas the spike and slab model has the form $\gamma_j \rightarrow w_j \rightarrow \mathbf{y}$. In the binary mask model, only the product $\gamma_j w_j$ can be identified from the likelihood.

One interesting aspect of this model is that it can be used to derive an objective function that is widely used in the (non-Bayesian) subset selection literature. First, note that the joint prior has the form

$$p(\boldsymbol{\gamma}, \mathbf{w}) \propto \mathcal{N}(\mathbf{w} | \mathbf{0}, \sigma_w^2 \mathbf{I}) \pi_0^{||\boldsymbol{\gamma}||_0} (1 - \pi_0)^{D - ||\boldsymbol{\gamma}||_0} \quad (13.23)$$

Hence the scaled unnormalized negative log posterior has the form

$$\begin{aligned} f(\boldsymbol{\gamma}, \mathbf{w}) &\triangleq -2\sigma^2 \log p(\boldsymbol{\gamma}, \mathbf{w}, \mathbf{y} | \mathbf{X}) = ||\mathbf{y} - \mathbf{X}(\boldsymbol{\gamma} * \mathbf{w})||^2 \\ &\quad + \frac{\sigma^2}{\sigma_w^2} ||\mathbf{w}||^2 + \lambda ||\boldsymbol{\gamma}||_0 + \text{const} \end{aligned} \quad (13.24)$$

where

$$\lambda \triangleq 2\sigma^2 \log\left(\frac{1 - \pi_0}{\pi_0}\right) \quad (13.25)$$

Let us split $\mathbf{w}$ into two subvectors, $\mathbf{w}_{-\gamma}$ and $\mathbf{w}_{\gamma}$, indexed by the zero and non-zero entries of $\boldsymbol{\gamma}$ respectively. Since $\mathbf{X}(\boldsymbol{\gamma} * \mathbf{w}) = \mathbf{X}_{\gamma} \mathbf{w}_{\gamma}$, we can just set $\mathbf{w}_{-\gamma} = \mathbf{0}$.

Now consider the case where $\sigma_w^2 \rightarrow \infty$, so we do not regularize the non-zero weights (so there is no complexity penalty coming from the marginal likelihood or its BIC approximation). In this case, the objective becomes

$$f(\boldsymbol{\gamma}, \mathbf{w}) = ||\mathbf{y} - \mathbf{X}_{\gamma} \mathbf{w}_{\gamma}||_2^2 + \lambda ||\boldsymbol{\gamma}||_0 \quad (13.26)$$

This is similar to the BIC objective above.

Instead of keeping track of the bit vector $\boldsymbol{\gamma}$, we can define the set of relevant variables to be the **support**, or set of non-zero entries, of $\mathbf{w}$. Then we can rewrite the above equation as follows:

$$f(\mathbf{w}) = ||\mathbf{y} - \mathbf{X}\mathbf{w}||_2^2 + \lambda ||\mathbf{w}||_0 \quad (13.27)$$

This is called ℓ_0 **regularization**. We have converted the discrete optimization problem (over $\boldsymbol{\gamma} \in \{0, 1\}^D$) into a continuous one (over $\mathbf{w} \in \mathbb{R}^D$); however, the ℓ_0 pseudo-norm makes the objective very non smooth, so this is still hard to optimize. We will discuss different solutions to this in the rest of this chapter.

13.2.3 Algorithms

Since there are 2^D models, we cannot explore the full posterior, or find the globally optimal model. Instead we will have to resort to heuristics of one form or another. All of the methods we will discuss involve searching through the space of models, and evaluating the cost $f(\boldsymbol{\gamma})$ at

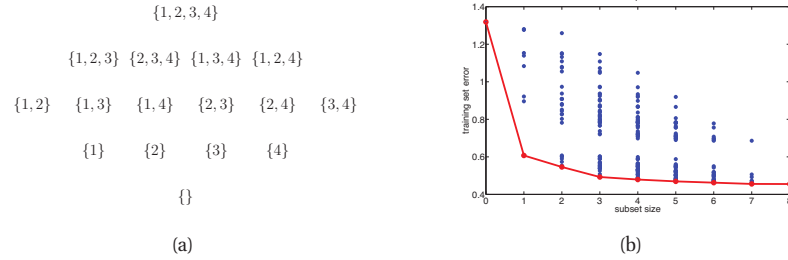


Figure 13.2 (a) A lattice of subsets of $\{1, 2, 3, 4\}$. (b) Residual sum of squares versus subset size, on the prostate cancer data set. The lower envelope is the best RSS achievable for any set of a given size. Based on Figure 3.5 of (Hastie et al. 2001). Figure generated by `prostateSubsets`.

each point. This requires fitting the model (i.e., computing $\arg\max p(\mathcal{D}|\mathbf{w})$), or evaluating its marginal likelihood (i.e., computing $\int p(\mathcal{D}|\mathbf{w})p(\mathbf{w})d\mathbf{w}$) at each step. This is sometimes called the **wrapper method**, since we “wrap” our search for the best model (or set of good models) around a generic model-fitting procedure.

In order to make wrapper methods efficient, it is important that we can quickly evaluate the score function for some new model, γ' , given the score of a previous model, γ . This can be done provided we can efficiently update the sufficient statistics needed to compute $f(\gamma)$. This is possible provided γ' only differs from γ in one bit (corresponding to adding or removing a single variable), and provided $f(\gamma)$ only depends on the data via $\mathbf{X}_\gamma$. In this case, we can use rank-one matrix updates/ downdates to efficiently compute $\mathbf{X}_{\gamma'}^T \mathbf{X}_{\gamma'}$ from $\mathbf{X}_\gamma^T \mathbf{X}_\gamma$. These updates are usually applied to the QR decomposition of $\mathbf{X}$. See e.g., (Miller 2002; Schniter et al. 2008) for details.

13.2.3.1 Greedy search

Suppose we want to find the MAP model. If we use the ℓ_0 -regularized objective in Equation 13.27, we can exploit properties of least squares to derive various efficient greedy forwards search methods, some of which we summarize below. For further details, see (Miller 2002; Soussen et al. 2010).

- **Single best replacement** The simplest method is to use greedy hill climbing, where at each step, we define the neighborhood of the current model to be all models than can be reached by flipping a single bit of γ , i.e., for each variable, if it is currently out of the model, we consider adding it, and if it is currently in the model, we consider removing it. In (Soussen et al. 2010), they call this the **single best replacement** (SBR). Since we are expecting a sparse solution, we can start with the empty set, $\gamma = \mathbf{0}$. We are essentially moving through the lattice of subsets, shown in Figure 13.2(a). We continue adding or removing until no improvement is possible.
- **Orthogonal least squares** If we set $\lambda = 0$ in Equation 13.27, so there is no complexity penalty, there will be no reason to perform deletion steps. In this case, the SBR algorithm is equivalent to **orthogonal least squares** (Chen and Wigger 1995), which in turn is equivalent

to greedy **forwards selection**. In this algorithm, we start with the empty set and add the best feature at each step. The error will go down monotonically with $||\gamma||_0$, as shown in Figure 13.2(b). We can pick the next best feature j^* to add to the current set γ_t by solving

$$j^* = \arg \min_{j \notin \gamma_t} \min_{\mathbf{w}} ||\mathbf{y} - (\mathbf{X}_{\gamma_t \cup j})\mathbf{w}||^2 \quad (13.28)$$

We then update the active set by setting $\gamma^{(t+1)} = \gamma^{(t)} \cup \{j^*\}$. To choose the next feature to add at step t , we need to solve $D - D_t$ least squares problems at step t , where $D_t = |\gamma_t|$ is the cardinality of the current active set. Having chosen the best feature to add, we need to solve an additional least squares problem to compute $\mathbf{w}_{t+1}$.

- **Orthogonal matching pursuits** Orthogonal least squares is somewhat expensive. A simplification is to “freeze” the current weights at their current value, and then to pick the next feature to add by solving

$$j^* = \arg \min_{j \notin \gamma_t} \min_{\beta} ||\mathbf{y} - \mathbf{X}\mathbf{w}_t - \beta \mathbf{x}_{:,j}||^2 \quad (13.29)$$

This inner optimization is easy to solve: we simply set $\beta = \mathbf{x}_{:,j}^T \mathbf{r}_t / ||\mathbf{x}_{:,j}||^2$, where $\mathbf{r}_t = \mathbf{y} - \mathbf{X}\mathbf{w}_t$ is the current residual vector. If the columns are unit norm, we have

$$j^* = \arg \max \mathbf{x}_{:,j}^T \mathbf{r}_t \quad (13.30)$$

so we are just looking for the column that is most correlated with the current residual. We then update the active set, and compute the new least squares estimate $\mathbf{w}_{t+1}$ using $\mathbf{X}_{\gamma_{t+1}}$. This method is called **orthogonal matching pursuits** or **OMP** (Mallat et al. 1994). This only requires one least squares calculation per iteration and so is faster than orthogonal least squares, but is not quite as accurate (Blumensath and Davies 2007).

- **Matching pursuits** An even more aggressive approximation is to just greedily add the feature that is most correlated with the current residual. This is called **matching pursuits** (Mallat and Zhang 1993). This is also equivalent to a method known as least squares boosting (Section 16.4.6).
- **Backwards selection** **Backwards selection** starts with all variables in the model (the so-called **saturated model**), and then deletes the worst one at each step. This is equivalent to performing a greedy search from the top of the lattice downwards. This can give better results than a bottom-up search, since the decision about whether to keep a variable or not is made in the context of all the other variables that might depend on it. However, this method is typically infeasible for large problems, since the saturated model will be too expensive to fit.
- **FoBa** The **forwards-backwards algorithm** of (Zhang 2008) is similar to the single best replacement algorithm presented above, except it uses an OMP-like approximation when choosing the next move to make. A similar “dual-pass” algorithm was described in (Moghadam et al. 2008).
- **Bayesian Matching pursuit** The algorithm of (Schniter et al. 2008) is similar to OMP except it uses a Bayesian marginal likelihood scoring criterion (under a spike and slab model) instead of a least squares objective. In addition, it uses a form of beam search to explore multiple paths through the lattice at once.

13.2.3.2 Stochastic search

If we want to approximate the posterior, rather than just computing a mode (e.g. because we want to compute marginal inclusion probabilities), one option is to use MCMC. The standard approach is to use Metropolis Hastings, where the proposal distribution just flips single bits. This enables us to efficiently compute $p(\gamma'|\mathcal{D})$ given $p(\gamma|\mathcal{D})$. The probability of a state (bit configuration) is estimated by counting how many times the random walk visits this state. See (O'Hara and Sillanpaa 2009) for a review of such methods, and (Bottolo and Richardson 2010) for a very recent method based on evolutionary MCMC.

However, in a discrete state space, MCMC is needlessly inefficient, since we can compute the (unnormalized) probability of a state directly using $p(\gamma, \mathcal{D}) = \exp(-f(\gamma))$; thus *there is no need to ever revisit a state*. A much more efficient alternative is to use some kind of stochastic search algorithm, to generate a set $\mathcal{S}$ of high scoring models, and then to make the following approximation

$$p(\gamma|\mathcal{D}) \approx \frac{e^{-f(\gamma)}}{\sum_{\gamma' \in \mathcal{S}} e^{-f(\gamma')}} \quad (13.31)$$

See (Heaton and Scott 2009) for a review of recent methods of this kind.

13.2.3.3 EM and variational inference *

It is tempting to apply EM to the spike and slab model, which has the form $\gamma_j \rightarrow w_j \rightarrow \mathbf{y}$. We can compute $p(\gamma_j = 1|w_j)$ in the E step, and optimize $\mathbf{w}$ in the M step. However, this will not work, because when we compute $p(\gamma_j = 1|w_j)$, we are comparing a delta-function, $\delta_0(w_j)$, with a Gaussian pdf, $\mathcal{N}(w_j|0, \sigma_w^2)$. We can replace the delta function with a narrow Gaussian, and then the E step amounts to classifying w_j under the two possible Gaussian models. However, this is likely to suffer from severe local minima.

An alternative is to apply EM to the Bernoulli-Gaussian model, which has the form $\gamma_j \rightarrow \mathbf{y} \leftarrow w_j$. In this case, the posterior $p(\gamma|\mathcal{D}, \mathbf{w})$ is intractable to compute because all the bits become correlated due to explaining away. However, it is possible to derive a mean field approximation of the form $\prod_j q(\gamma_j)q(w_j)$ (Huang et al. 2007; Rattray et al. 2009).

13.3 ℓ_1 regularization: basics

When we have many variables, it is computationally difficult to find the posterior mode of $p(\gamma|\mathcal{D})$. And although greedy algorithms often work well (see e.g., (Zhang 2008) for a theoretical analysis), they can of course get stuck in local optima.

Part of the problem is due to the fact that the γ_j variables are discrete, $\gamma_j \in \{0, 1\}$. In the optimization community, it is common to relax hard constraints of this form by replacing discrete variables with continuous variables. We can do this by replacing the spike-and-slab style prior, that assigns finite probability mass to the event that $w_j = 0$, to continuous priors that “encourage” $w_j = 0$ by putting a lot of probability density near the origin, such as a zero-mean Laplace distribution. This was first introduced in Section 7.4 in the context of robust linear regression. There we exploited the fact that the Laplace has heavy tails. Here we exploit the fact

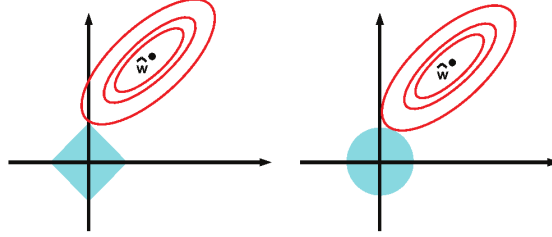


Figure 13.3 Illustration of ℓ_1 (left) vs ℓ_2 (right) regularization of a least squares problem. Based on Figure 3.12 of (Hastie et al. 2001).

that it has a spike near $\mu = 0$. More precisely, consider a prior of the form

$$p(\mathbf{w}|\lambda) = \prod_{j=1}^D \text{Lap}(w_j|0, 1/\lambda) \propto \prod_{j=1}^D e^{-\lambda|w_j|} \quad (13.32)$$

We will use a uniform prior on the offset term, $p(w_0) \propto 1$. Let us perform MAP estimation with this prior. The penalized negative log likelihood has the form

$$f(\mathbf{w}) = -\log p(\mathcal{D}|\mathbf{w}) - \log p(\mathbf{w}|\lambda) = \text{NLL}(\mathbf{w}) + \lambda \|\mathbf{w}\|_1 \quad (13.33)$$

where $\|\mathbf{w}\|_1 = \sum_{j=1}^D |w_j|$ is the ℓ_1 norm of $\mathbf{w}$. For suitably large λ , the estimate $\hat{\mathbf{w}}$ will be sparse, for reasons we explain below. Indeed, this can be thought of as a convex approximation to the non-convex ℓ_0 objective

$$\underset{\mathbf{w}}{\text{argmin}} \text{NLL}(\mathbf{w}) + \lambda \|\mathbf{w}\|_0 \quad (13.34)$$

In the case of linear regression, the ℓ_1 objective becomes

$$f(\mathbf{w}) = \sum_{i=1}^N -\frac{1}{2\sigma^2} (y_i - (w_0 + \mathbf{w}^T \mathbf{x}_i))^2 + \lambda \|\mathbf{w}\|_1 \quad (13.35)$$

$$= \text{RSS}(\mathbf{w}) + \lambda' \|\mathbf{w}\|_1 \quad (13.36)$$

where $\lambda' = 2\lambda\sigma^2$. This method is known as **basis pursuit denoising** or **BPDN** (Chen et al. 1998). The reason for this term will become clear later. In general, the technique of putting a zero-mean Laplace prior on the parameters and performing MAP estimation is called ℓ_1 **regularization**. It can be combined with any convex or non-convex NLL term. Many different algorithms have been devised for solving such problems, some of which we review in Section 13.4.

13.3.1 Why does ℓ_1 regularization yield sparse solutions?

We now explain why ℓ_1 regularization results in sparse solutions, whereas ℓ_2 regularization does not. We focus on the case of linear regression, although similar arguments hold for logistic regression and other GLMs.

The objective is the following non-smooth objective function:

$$\min_{\mathbf{w}} \text{RSS}(\mathbf{w}) + \lambda \|\mathbf{w}\|_1 \quad (13.37)$$

We can rewrite this as a constrained but smooth objective (a quadratic function with linear constraints):

$$\min_{\mathbf{w}} \text{RSS}(\mathbf{w}) \quad \text{s.t.} \quad \|\mathbf{w}\|_1 \leq B \quad (13.38)$$

where B is an upper bound on the ℓ_1 -norm of the weights: a small (tight) bound B corresponds to a large penalty λ , and vice versa.² Equation 13.38 is known as **lasso**, which stands for “least absolute shrinkage and selection operator” (Tibshirani 1996). We will see why it has this name later.

Similarly, we can write ridge regression

$$\min_{\mathbf{w}} \text{RSS}(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2 \quad (13.39)$$

or as a bound constrained form:

$$\min_{\mathbf{w}} \text{RSS}(\mathbf{w}) \quad \text{s.t.} \quad \|\mathbf{w}\|_2^2 \leq B \quad (13.40)$$

In Figure 13.3, we plot the contours of the RSS objective function, as well as the contours of the ℓ_2 and ℓ_1 constraint surfaces. From the theory of constrained optimization, we know that the optimal solution occurs at the point where the lowest level set of the objective function intersects the constraint surface (assuming the constraint is active). It should be geometrically clear that as we relax the constraint B , we “grow” the ℓ_1 “ball” until it meets the objective; the corners of the ball are more likely to intersect the ellipse than one of the sides, especially in high dimensions, because the corners “stick out” more. The corners correspond to sparse solutions, which lie on the coordinate axes. By contrast, when we grow the ℓ_2 ball, it can intersect the objective at any point; there are no “corners”, so there is no preference for sparsity.

To see this another way, notice that, with ridge regression, the prior cost of a sparse solution, such as $\mathbf{w} = (1, 0)$, is the same as the cost of a dense solution, such as $\mathbf{w} = (1/\sqrt{2}, 1/\sqrt{2})$, as long as they have the same ℓ_2 norm:

$$\|(1, 0)\|_2 = \|(1/\sqrt{2}, 1/\sqrt{2})\|_2 = 1 \quad (13.41)$$

However, for lasso, setting $\mathbf{w} = (1, 0)$ is cheaper than setting $\mathbf{w} = (1/\sqrt{2}, 1/\sqrt{2})$, since

$$\|(1, 0)\|_1 = 1 < \|(1/\sqrt{2}, 1/\sqrt{2})\|_1 = \sqrt{2} \quad (13.42)$$

The most rigorous way to see that ℓ_1 regularization results in sparse solutions is to examine conditions that hold at the optimum. We do this in Section 13.3.2.

13.3.2 Optimality conditions for lasso

The lasso objective has the form

$$f(\boldsymbol{\theta}) = \text{RSS}(\boldsymbol{\theta}) + \lambda \|\mathbf{w}\|_1 \quad (13.43)$$

2. Equation 13.38 is an example of a **quadratic program** or **QP**, since we have a quadratic objective subject to linear inequality constraints. Its Lagrangian is given by Equation 13.37.

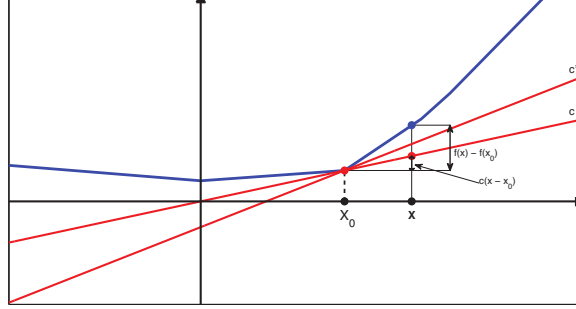


Figure 13.4 Illustration of some sub-derivatives of a function at point x_0 . Based on a figure at <http://en.wikipedia.org/wiki/Subderivative>. Figure generated by `subgradientPlot`.

Unfortunately, the $\|\mathbf{w}\|_1$ term is not differentiable whenever $w_j = 0$. This is an example of a **non-smooth** optimization problem.

To handle non-smooth functions, we need to extend the notion of a derivative. We define a **subderivative** or **subgradient** of a (convex) function $f : \mathcal{I} \rightarrow \mathbb{R}$ at a point θ_0 to be a scalar g such that

$$f(\theta) - f(\theta_0) \geq g(\theta - \theta_0) \quad \forall \theta \in \mathcal{I} \quad (13.44)$$

where $\mathcal{I}$ is some interval containing θ_0 . See Figure 13.4 for an illustration.³ We define the *set* of subderivatives as the interval $[a, b]$ where a and b are the one-sided limits

$$a = \lim_{\theta \rightarrow \theta_0^-} \frac{f(\theta) - f(\theta_0)}{\theta - \theta_0}, \quad b = \lim_{\theta \rightarrow \theta_0^+} \frac{f(\theta) - f(\theta_0)}{\theta - \theta_0} \quad (13.46)$$

The set $[a, b]$ of all subderivatives is called the **subdifferential** of the function f at θ_0 and is denoted $\partial f(\theta)|_{\theta_0}$. For example, in the case of the absolute value function $f(\theta) = |\theta|$, the subderivative is given by

$$\partial f(\theta) = \begin{cases} \{-1\} & \text{if } \theta < 0 \\ [-1, 1] & \text{if } \theta = 0 \\ \{+1\} & \text{if } \theta > 0 \end{cases} \quad (13.47)$$

If the function is everywhere differentiable, then $\partial f(\theta) = \{\frac{df(\theta)}{d\theta}\}$. By analogy to the standard calculus result, one can show that the point $\hat{\theta}$ is a local minimum of f iff $0 \in \partial f(\theta)|_{\hat{\theta}}$.

3. In general, for a vector valued function, we say that $\mathbf{g}$ is a subgradient of f at θ_0 if for all vectors θ ,

$$f(\theta) - f(\theta_0) \geq (\theta - \theta_0)^T \mathbf{g} \quad (13.45)$$

so $\mathbf{g}$ is a linear lower bound to the function at θ_0 .

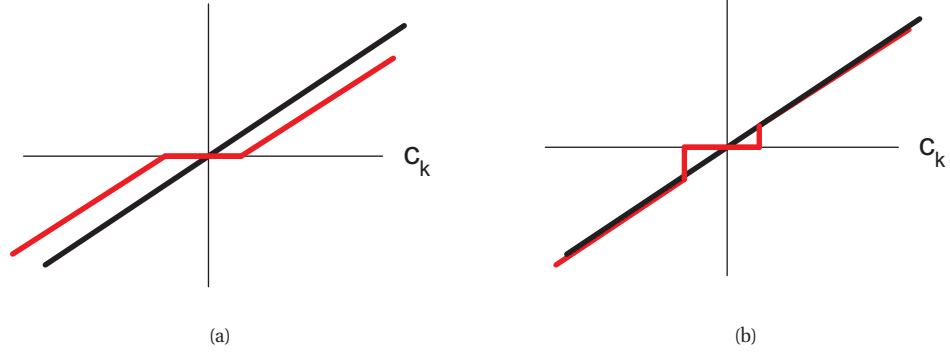


Figure 13.5 Left: soft thresholding. The flat region is the interval $[-\lambda, +\lambda]$. Right: hard thresholding.

Let us apply these concepts to the lasso problem. Let us initially ignore the non-smooth penalty term. One can show (Exercise 13.1) that

$$\frac{\partial}{\partial w_j} \text{RSS}(\mathbf{w}) = a_j w_j - c_j \quad (13.48)$$

$$a_j = 2 \sum_{i=1}^n x_{ij}^2 \quad (13.49)$$

$$c_j = 2 \sum_{i=1}^n x_{ij} (y_i - \mathbf{w}_{-j}^T \mathbf{x}_{i,-j}) \quad (13.50)$$

where $\mathbf{w}_{-j}$ is $\mathbf{w}$ without component j , and similarly for $\mathbf{x}_{i,-j}$. We see that c_j is (proportional to) the correlation between the j 'th feature $\mathbf{x}_{:,j}$ and the residual due to the other features, $\mathbf{r}_{-j} = \mathbf{y} - \mathbf{X}_{:, -j} \mathbf{w}_{-j}$. Hence the magnitude of c_j is an indication of how relevant feature j is for predicting $\mathbf{y}$ (relative to the other features and the current parameters).

Adding in the penalty term, we find that the subderivative is given by

$$\partial_{w_j} f(\mathbf{w}) = (a_j w_j - c_j) + \lambda \partial_{w_j} \|\mathbf{w}\|_1 \quad (13.51)$$

$$= \begin{cases} \{a_j w_j - c_j - \lambda\} & \text{if } w_j < 0 \\ [-c_j - \lambda, -c_j + \lambda] & \text{if } w_j = 0 \\ \{a_j w_j - c_j + \lambda\} & \text{if } w_j > 0 \end{cases} \quad (13.52)$$

We can write this in a more compact fashion as follows:

$$\mathbf{X}^T (\mathbf{X} \mathbf{w} - \mathbf{y})_j \in \begin{cases} \{-\lambda\} & \text{if } w_j < 0 \\ [-\lambda, \lambda] & \text{if } w_j = 0 \\ \{\lambda\} & \text{if } w_j > 0 \end{cases} \quad (13.53)$$

Depending on the value of c_j , the solution to $\partial_{w_j} f(\mathbf{w}) = 0$ can occur at 3 different values of w_j , as follows:

1. If $c_j < -\lambda$, so the feature is strongly negatively correlated with the residual, then the subgradient is zero at $\hat{w}_j = \frac{c_j + \lambda}{a_j} < 0$.
2. If $c_j \in [-\lambda, \lambda]$, so the feature is only weakly correlated with the residual, then the subgradient is zero at $\hat{w}_j = 0$.
3. If $c_j > \lambda$, so the feature is strongly positively correlated with the residual, then the subgradient is zero at $\hat{w}_j = \frac{c_j - \lambda}{a_j} > 0$.

In summary, we have

$$\hat{w}_j(c_j) = \begin{cases} (c_j + \lambda)/a_j & \text{if } c_j < -\lambda \\ 0 & \text{if } c_j \in [-\lambda, \lambda] \\ (c_j - \lambda)/a_j & \text{if } c_j > \lambda \end{cases} \quad (13.54)$$

We can write this as follows:

$$\hat{w}_j = \text{soft}\left(\frac{c_j}{a_j}; \frac{\lambda}{a_j}\right) \quad (13.55)$$

where

$$\text{soft}(a; \delta) \triangleq \text{sign}(a) (|a| - \delta)_+ \quad (13.56)$$

and $x_+ = \max(x, 0)$ is the positive part of x . This is called **soft thresholding**. This is illustrated in Figure 13.5(a), where we plot $\hat{w}_j$ vs c_j . The dotted line is the line $w_j = c_j/a_j$ corresponding to the least squares fit. The solid line, which represents the regularized estimate $\hat{w}_j(c_j)$, shifts the dotted line down (or up) by λ , except when $-\lambda \leq c_j \leq \lambda$, in which case it sets $w_j = 0$.

By contrast, in Figure 13.5(b), we illustrate **hard thresholding**. This sets values of w_j to 0 if $-\lambda \leq c_j \leq \lambda$, but it does not shrink the values of w_j outside of this interval. The slope of the soft thresholding line does not coincide with the diagonal, which means that even large coefficients are shrunk towards zero; consequently lasso is a biased estimator. This is undesirable, since if the likelihood indicates (via c_j) that the coefficient w_j should be large, we do not want to shrink it. We will discuss this issue in more detail in Section 13.6.2.

Now we finally can understand why Tibshirani invented the term “lasso” in (Tibshirani 1996): it stands for “least absolute selection and shrinkage operator”, since it selects a subset of the variables, and shrinks all the coefficients by penalizing the absolute values. If $\lambda = 0$, we get the OLS solution (of minimal ℓ_1 norm). If $\lambda \geq \lambda_{max}$, we get $\hat{\mathbf{w}} = \mathbf{0}$, where

$$\lambda_{max} = \|\mathbf{X}^T \mathbf{y}\|_\infty = \max_j |\mathbf{y}^T \mathbf{x}_{:,j}| \quad (13.57)$$

This value is computed using the fact that $\mathbf{0}$ is optimal if $(\mathbf{X}^T \mathbf{y})_j \in [-\lambda, \lambda]$ for all j . In general, the maximum penalty for an ℓ_1 regularized objective is

$$\lambda_{max} = \max_j |\nabla_j NLL(\mathbf{0})| \quad (13.58)$$

13.3.3 Comparison of least squares, lasso, ridge and subset selection

We can gain further insight into ℓ_1 regularization by comparing it to least squares, and ℓ_2 and ℓ_0 regularized least squares. For simplicity, assume all the features of $\mathbf{X}$ are orthonormal, so $\mathbf{X}^T \mathbf{X} = \mathbf{I}$. In this case, the RSS is given by

$$\text{RSS}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 = \mathbf{y}^T \mathbf{y} + \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w} - 2\mathbf{w}^T \mathbf{X}^T \mathbf{y} \quad (13.59)$$

$$= \text{const} + \sum_k w_k^2 - 2 \sum_k \sum_i w_k x_{ik} y_i \quad (13.60)$$

so we see this factorizes into a sum of terms, one per dimension. Hence we can write down the MAP and ML estimates analytically, as follows:

- **MLE** The OLS solution is given by

$$\hat{w}_k^{OLS} = \mathbf{x}_{:,k}^T \mathbf{y} \quad (13.61)$$

where $\mathbf{x}_{:,k}$ is the k 'th column of $\mathbf{X}$. This follows trivially from Equation 13.60. We see that $\hat{w}_k^{OLS}$ is just the orthogonal projection of feature k onto the response vector (see Section 7.3.2).

- **Ridge** One can show that the ridge estimate is given by

$$\hat{w}_k^{ridge} = \frac{\hat{w}_k^{OLS}}{1 + \lambda} \quad (13.62)$$

- **Lasso** From Equation 13.55, and using the fact that $a_k = 2$ and $\hat{w}_k^{OLS} = c_k/2$, we have

$$\hat{w}_k^{lasso} = \text{sign}(\hat{w}_k^{OLS}) \left(|\hat{w}_k^{OLS}| - \frac{\lambda}{2} \right)_+ \quad (13.63)$$

This corresponds to soft thresholding, shown in Figure 13.5(a).

- **Subset selection** If we pick the best K features using subset selection, the parameter estimate is as follows

$$\hat{w}_k^{SS} = \begin{cases} \hat{w}_k^{OLS} & \text{if } \text{rank}(|w_k^{OLS}|) \leq K \\ 0 & \text{otherwise} \end{cases} \quad (13.64)$$

where rank refers to the location in the sorted list of weight magnitudes. This corresponds to hard thresholding, shown in Figure 13.5(b).

Figure 13.6(a) plots the MSE vs λ for lasso for a degree 14 polynomial, and Figure 13.6(b) plots the MSE vs polynomial order. We see that lasso gives similar results to the subset selection method.

As another example, consider a data set concerning prostate cancer. We have $D = 8$ features and $N = 67$ training cases; the goal is to predict the log prostate-specific antigen levels (see (Hastie et al. 2009, p4) for more biological details). Table 13.1 shows that lasso gives better prediction accuracy (at least on this particular data set) than least squares, ridge, and best subset regression. (In each case, the strength of the regularizer was chosen by cross validation.) Lasso also gives rise to a sparse solution. Of course, for other problems, ridge may give better predictive accuracy. In practice, a combination of lasso and ridge, known as the elastic net, often performs best, since it provides a good combination of sparsity and regularization (see Section 13.5.3).

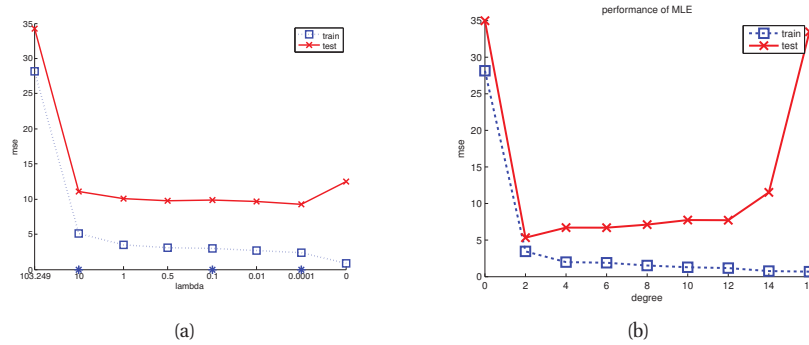


Figure 13.6 (a) MSE vs λ for lasso for a degree 14 polynomial. Note that λ decreases as we move to the right. Figure generated by `linregPolyLassoDemo`. (b) MSE versus polynomial degree. Note that the model order increases as we move to the right. See Figure 1.18 for a plot of some of these polynomial regression models. Figure generated by `linregPolyVsDegree`.

Term	LS	Best Subset	Ridge	Lasso
Intercept	2.452	2.481	2.479	2.480
lcavol	0.716	0.651	0.656	0.653
lweight	0.293	0.380	0.300	0.297
age	-0.143	-0.000	-0.129	-0.119
lbph	0.212	-0.000	0.208	0.200
svi	0.310	-0.000	0.301	0.289
lcp	-0.289	-0.000	-0.260	-0.236
gleason	-0.021	-0.000	-0.019	0.000
pgg45	0.277	0.178	0.256	0.226
Test Error	0.586	0.572	0.580	0.564

Table 13.1 Results of different methods on the prostate cancer data, which has 8 features and 67 training cases. Methods are: LS = least squares, Subset = best subset regression, Ridge, Lasso. Rows represent the coefficients; we see that subset regression and lasso give sparse solutions. Bottom row is the mean squared error on the test set (30 cases). Based on Table 3.3. of (Hastie et al. 2009). Figure generated by `prostateComparison`.

13.3.4 Regularization path

As we increase λ , the solution vector $\hat{\mathbf{w}}(\lambda)$ will tend to get sparser, although not necessarily monotonically. We can plot the values $\hat{w}_j(\lambda)$ vs λ for each feature j ; this is known as the **regularization path**.

This is illustrated for ridge regression in Figure 13.7(a), where we plot $\hat{w}_j(\lambda)$ as the regularizer λ decreases. We see that when $\lambda = \infty$, all the coefficients are zero. But for any finite value of λ , all coefficients are non-zero; furthermore, they increase in magnitude as λ is decreased.

In Figure 13.7(b), we plot the analogous result for lasso. As we move to the right, the upper bound on the ℓ_1 penalty, B , increases. When $B = 0$, all the coefficients are zero. As we increase

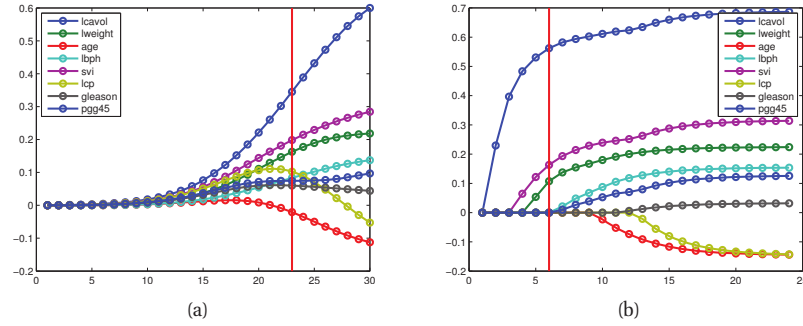


Figure 13.7 (a) Profiles of ridge coefficients for the prostate cancer example vs bound on ℓ_2 norm of $\mathbf{w}$, so small t (large λ) is on the left. The vertical line is the value chosen by 5-fold CV using the ISE rule. Based on Figure 3.8 of (Hastie et al. 2009). Figure generated by `ridgePathProstate`. (b) Profiles of lasso coefficients for the prostate cancer example vs bound on ℓ_1 norm of $\mathbf{w}$, so small t (large λ) is on the left. Based on Figure 3.10 of (Hastie et al. 2009). Figure generated by `lassoPathProstate`.

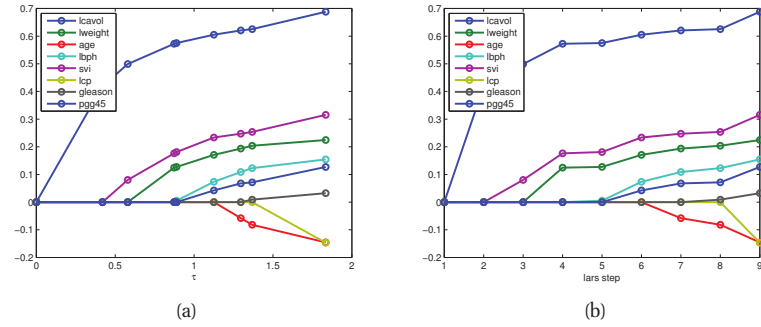


Figure 13.8 Illustration of piecewise linearity of regularization path for lasso on the prostate cancer example. (a) We plot $\hat{w}_j(B)$ vs B for the critical values of B . (b) We plot vs steps of the LARS algorithm. Figure generated by `lassoPathProstate`.

B , the coefficients gradually “turn on”. But for any value between 0 and $B_{max} = \|\hat{\mathbf{w}}_{OLS}\|_1$, the solution is sparse.⁴

Remarkably, it can be shown that the solution path is a piecewise linear function of B (Efron et al. 2004). That is, there are a set of critical values of B where the active set of non-zero coefficients changes. For values of B between these critical values, each non-zero coefficient increases or decreases in a linear fashion. This is illustrated in Figure 13.8(a). Furthermore, one can solve for these critical values analytically. This is the basis of the **LARS** algorithm (Efron et al. 2004), which stands for “least angle regression and shrinkage” (see Section 13.4.2 for details). Remarkably, LARS can compute the entire regularization path for roughly the same

4. It is common to plot the solution versus the **shrinkage factor**, defined as $s(B) = B/B_{max}$, rather than against B . This merely affects the scale of the horizontal axis, not the shape of the curves.

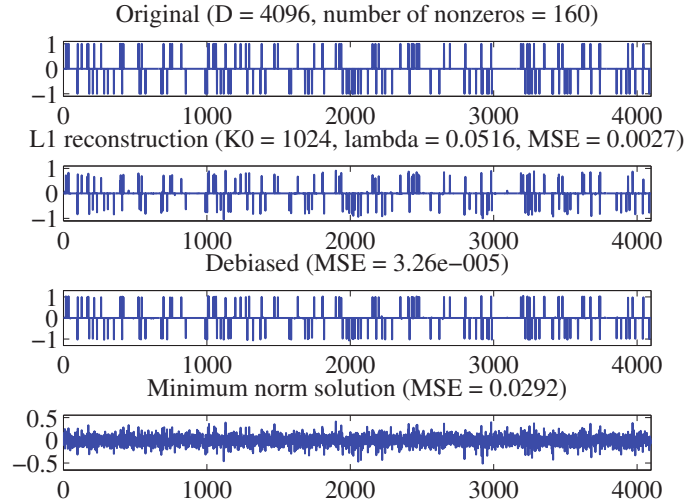


Figure 13.9 Example of recovering a sparse signal using lasso. See text for details. Based on Figure 1 of (Figueiredo et al. 2007). Figure generated by `sparseSensingDemo`, written by Mario Figueiredo.

computational cost as a single least squares fit (namely $O(\min(ND^2, DN^2))$).

In Figure 13.8(b), we plot the coefficients computed at each critical value of B . Now the piecewise linearity is more evident. Below we display the actual coefficient values at each step along the regularization path (the last line is the least squares solution):

Listing 13.1 Output of `lassoPathProstate`

0	0	0	0	0	0	0	0
0.4279	0	0	0	0	0	0	0
0.5015	0.0735	0	0	0	0	0	0
0.5610	0.1878	0	0	0.0930	0	0	0
0.5622	0.1890	0	0.0036	0.0963	0	0	0
0.5797	0.2456	0	0.1435	0.2003	0	0	0.0901
0.5864	0.2572	-0.0321	0.1639	0.2082	0	0	0.1066
0.6994	0.2910	-0.1337	0.2062	0.3003	-0.2565	0	0.2452
0.7164	0.2926	-0.1425	0.2120	0.3096	-0.2890	-0.0209	0.2773

By changing B from 0 to B_{max} , we can go from a solution in which all the weights are zero to a solution in which all weights are non-zero. Unfortunately, not all subset sizes are achievable using lasso. One can show that, if $D > N$, the optimal solution can have at most N variables in it, before reaching the complete set corresponding to the OLS solution of minimal ℓ_1 norm. In Section 13.5.3, we will see that by using an ℓ_2 regularizer as well as an ℓ_1 regularizer (a method known as the elastic net), we can achieve sparse solutions which contain more variables than training cases. This lets us explore model sizes between N and D .

13.3.5 Model selection

It is tempting to use ℓ_1 regularization to estimate the set of relevant variables. In some cases, we can recover the true sparsity pattern of $\mathbf{w}^*$, the parameter vector that generated the data. A method that can recover the true model in the $N \rightarrow \infty$ limit is called **model selection consistent**. The details on which methods enjoy this property, and when, are beyond the scope of this book; see e.g., (Bühlmann and van de Geer 2011) for details.

Instead of going into a theoretical discussion, we will just show a small example. We first generate a sparse signal $\mathbf{w}^*$ of size $D = 4096$, consisting of 160 randomly placed ± 1 spikes. Next we generate a random design matrix $\mathbf{X}$ of size $N \times D$, where $N = 1024$. Finally we generate a noisy observation $\mathbf{y} = \mathbf{X}\mathbf{w}^* + \boldsymbol{\epsilon}$, where $\epsilon_i \sim \mathcal{N}(0, 0.01^2)$. We then estimate $\mathbf{w}$ from $\mathbf{y}$ and $\mathbf{X}$.

The original $\mathbf{w}^*$ is shown in the first row of Figure 13.9. The second row is the ℓ_1 estimate $\hat{\mathbf{w}}_{L1}$ using $\lambda = 0.1\lambda_{max}$. We see that this has “spikes” in the right places, but they are too small. The third row is the least squares estimate of the coefficients which are estimated to be non-zero based on $\text{supp}(\hat{\mathbf{w}}_{L1})$. This is called **debiasing**, and is necessary because lasso shrinks the relevant coefficients as well as the irrelevant ones. The last row is the least squares estimate for all the coefficients jointly, ignoring sparsity. We see that the (debiased) sparse estimate is an excellent estimate of the original signal. By contrast, least squares without the sparsity assumption performs very poorly.

Of course, to perform model selection, we have to pick λ . It is common to use cross validation. However, it is important to note that cross validation is picking a value of λ that results in good predictive accuracy. This is not usually the same value as the one that is likely to recover the “true” model. To see why, recall that ℓ_1 regularization performs selection *and* shrinkage, that is, the chosen coefficients are brought closer to 0. In order to prevent relevant coefficients from being shrunk in this way, cross validation will tend to pick a value of λ that is not too large. Of course, this will result in a less sparse model which contains irrelevant variables (false positives). Indeed, it was proved in (Meinshausen and Bühlmann 2006) that the prediction-optimal value of λ does not result in model selection consistency. In Section 13.6.2, we will discuss some adaptive mechanisms for automatically tuning λ on a per-dimension basis that does result in model selection consistency.

A downside of using ℓ_1 regularization to select variables is that it can give quite different results if the data is perturbed slightly. The Bayesian approach, which estimates posterior marginal inclusion probabilities, $p(\gamma_j = 1|\mathcal{D})$, is much more robust. A frequentist solution to this is to use bootstrap resampling (see Section 6.2.1), and to rerun the estimator on different versions of the data. By computing how often each variable is selected across different trials, we can approximate the posterior inclusion probabilities. This method is known as **stability selection** (Meinshausen and Bühlmann 2010).

We can threshold the stability selection (bootstrap) inclusion probabilities at some level, say 90%, and thus derive a sparse estimator. This is known as **bootstrap lasso** or **bolasso** (Bach 2008). It will include a variable if it occurs in at least 90% of sets returned by lasso (for a fixed λ). This process of intersecting the sets is a way of eliminating the false positives that vanilla lasso produces. The theoretical results in (Bach 2008) prove that bolasso is model selection consistent under a wider range of conditions than vanilla lasso.

As an illustration, we reproduced the experiments in (Bach 2008). In particular, we created

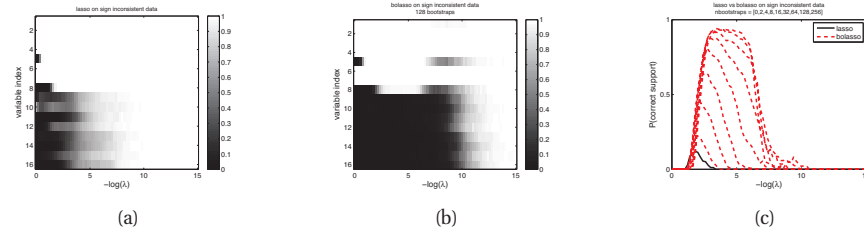


Figure 13.10 (a) Probability of selection of each variable (white = large probabilities, black = small probabilities) vs. regularization parameter for Lasso. As we move from left to right, we decrease the amount of regularization, and therefore select more variables. (b) Same as (a) but for bolasso. (c) Probability of correct sign estimation vs. regularization parameter. Bolasso (red, dashed) and Lasso (black, plain): The number of bootstrap replications is in $\{2, 4, 8, 16, 32, 64, 128, 256\}$. Based on Figures 1-3 of (Bach 2008). Figure generated by `bolassoDemo`.

256 datasets of size $N = 1000$ with $D = 16$ variables, of which 8 are relevant. See (Bach 2008) for more detail on the experimental setup. For dataset n , variable j , and sparsity level k , define $S(j, k, n) = \mathbb{I}(\hat{w}_j(\lambda_k, \mathcal{D}_n) \neq 0)$. Now define $P(j, k)$ be the average of $S(j, k, n)$ over the 256 datasets. In Figure 13.10(a-b), we plot P vs $-\log(\lambda)$ for lasso and bolasso. We see that for bolasso, there is a large range of λ where the true variables are selected, but this is not the case for lasso. This is emphasized in Figure 13.10(c), where we plot the empirical probability that the correct set of variables is recovered, for lasso and for bolasso with an increasing number of bootstrap samples. Of course, using more samples takes longer. In practice, 32 bootstraps seems to be a good compromise between speed and accuracy.

With bolasso, there is the usual issue of picking λ . Obviously we could use cross validation, but plots such as Figure 13.10(b) suggest another heuristic: shuffle the rows to create a large black block, and then pick λ to be in the middle of this region. Of course, operationalizing this intuition may be tricky, and will require various ad-hoc thresholds (it is reminiscent of the “find the knee in the curve” heuristic discussed in Section 11.5.2 when discussing how to pick K for mixture models). A Bayesian approach provides a more principled method for selecting λ .

13.3.6 Bayesian inference for linear models with Laplace priors

We have been focusing on MAP estimation in sparse linear models. It is also possible to perform Bayesian inference (see e.g., (Park and Casella 2008; Seeger 2008)). However, the posterior mean and median, as well as samples from the posterior, are not sparse; only the mode is sparse. This is another example of the phenomenon discussed in Section 5.2.1, where we said that the MAP estimate is often untypical of the bulk of the posterior.

Another argument in favor of using the posterior mean comes from Equation 5.108, which showed that plugging in the posterior mean, rather than the posterior mode, is the optimal thing to do if we want to minimize squared prediction error. (Schniter et al. 2008) shows experimentally, and (Elad and Yavneh 2009) shows theoretically, that using the posterior mean with a spike-and-slab prior results in better prediction accuracy than using the posterior mode with a Laplace prior, albeit at slightly higher computational cost.

13.4 ℓ_1 regularization: algorithms

In this section, we give a brief review of some algorithms that can be used to solve ℓ_1 regularized estimation problems. We focus on the lasso case, where we have a quadratic loss. However, most of the algorithms can be extended to more general settings, such as logistic regression (see (Yaun et al. 2010) for a comprehensive review of ℓ_1 regularized logistic regression). Note that this area of machine learning is advancing very rapidly, so the methods below may not be state of the art by the time you read this chapter. (See (Schmidt et al. 2009; Yaun et al. 2010; Yang et al. 2010) for some recent surveys.)

13.4.1 Coordinate descent

Sometimes it is hard to optimize all the variables simultaneously, but it is easy to optimize them one by one. In particular, we can solve for the j 'th coefficient with all the others held fixed:

$$w_j^* = \underset{z}{\operatorname{argmin}} f(\mathbf{w} + z\mathbf{e}_j) - f(\mathbf{w}) \quad (13.65)$$

where $\mathbf{e}_j$ is the j 'th unit vector. We can either cycle through the coordinates in a deterministic fashion, or we can sample them at random, or we can choose to update the coordinate for which the gradient is steepest.

The coordinate descent method is particularly appealing if each one-dimensional optimization problem can be solved analytically. For example, the **shooting** algorithm (Fu 1998; Wu and Lange 2008) for lasso uses Equation 13.54 to compute the optimal value of w_j given all the other coefficients. See Algorithm 7 for the pseudo code (and `LassoShooting` for some Matlab code).

See (Yaun et al. 2010) for some extensions of this method to the logistic regression case. The resulting algorithm was the fastest method in their experimental comparison, which concerned document classification with large sparse feature vectors (representing bags of words). Other types of data (e.g., dense features and/or regression problems) might call for different algorithms.

Algorithm 13.1: Coordinate descent for lasso (aka shooting algorithm)

```

1 Initialize  $\mathbf{w} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$ ;
2 repeat
3   for  $j = 1, \dots, D$  do
4      $a_j = 2 \sum_{i=1}^n x_{ij}^2$ ;
5      $c_j = 2 \sum_{i=1}^n x_{ij} (y_i - \mathbf{w}^T \mathbf{x}_i + w_j x_{ij})$ ;
6      $w_j = \operatorname{soft}(\frac{c_j}{a_j}, \frac{\lambda}{a_j})$ ;
7 until converged;
```

13.4.2 LARS and other homotopy methods

The problem with coordinate descent is that it only updates one variable at a time, so can be slow to converge. **Active set** methods update many variables at a time. Unfortunately, they are

more complicated, because of the need to identify which variables are constrained to be zero, and which are free to be updated.

Active set methods typically only add or remove a few variables at a time, so they can take a long if they are started far from the solution. But they are ideally suited for generating a set of solutions for different values of λ , starting with the empty set, i.e., for generating regularization path. These algorithms exploit the fact that one can quickly compute $\hat{\mathbf{w}}(\lambda_k)$ from $\hat{\mathbf{w}}(\lambda_{k-1})$ if $\lambda_k \approx \lambda_{k-1}$; this is known as **warm starting**. In fact, even if we only want the solution for a single value of λ , call it λ_* , it can sometimes be computationally more efficient to compute a set of solutions, from λ_{max} down to λ_* , using warm-starting; this is called a **continuation method** or **homotopy method**. This is often much faster than directly “cold-starting” at λ_* ; this is particularly true if λ_* is small.

Perhaps the most well-known example of a homotopy method in machine learning is the **LARS** algorithm, which stands for “least angle regression and shrinkage” (Efron et al. 2004) (a similar algorithm was independently invented in (Osborne et al. 2000b,a)). This can compute $\hat{\mathbf{w}}(\lambda)$ for all possible values of λ in an efficient manner.

LARS works as follows. It starts with a large value of λ , such that only the variable that is most correlated with the response vector $\mathbf{y}$ is chosen. Then λ is decreased until a second variable is found which has the same correlation (in terms of magnitude) with the current residual as the first variable, where the residual at step k is defined as $\mathbf{r}_k = \mathbf{y} - \mathbf{X}_{:,F_k} \mathbf{w}_k$, where F_k is the current **active set** (c.f., Equation 13.50). Remarkably, one can solve for this new value of λ analytically, by using a geometric argument (hence the term “least angle”). This allows the algorithm to quickly “jump” to the next point on the regularization path where the active set changes. This repeats until all the variables are added.

It is necessary to allow variables to be removed from the active set if we want the sequence of solutions to correspond to the regularization path of lasso. If we disallow variable removal, we get a slightly different algorithm called **LAR**, which tends to be faster. In particular, LAR costs the same as a single ordinary least squares fit, namely $O(ND \min(N, D))$, which is $O(ND^2)$ if $N > D$, and $O(N^2D)$ if $D > N$. LAR is very similar to greedy forward selection, and a method known as least squares boosting (see Section 16.4.6).

There have been many attempts to extend the LARS algorithm to compute the full regularization path for ℓ_1 regularized GLMs, such as logistic regression. In general, one cannot analytically solve for the critical values of λ . Instead, the standard approach is to start at λ_{max} , and then slowly decrease λ , tracking the solution as we go; this is called a **continuation method** or **homotopy method**. These methods exploit the fact that we can quickly compute $\hat{\mathbf{w}}(\lambda_k)$ from $\hat{\mathbf{w}}(\lambda_{k-1})$ if $\lambda_k \approx \lambda_{k-1}$; this is known as **warm starting**. Even if we don't want the full path, this method is often much faster than directly “cold-starting” at the desired value of λ (this is particularly true if λ is small).

The method described in (Friedman et al. 2010) combines coordinate descent with this warm-starting strategy, and computes the full regularization path for any ℓ_1 regularized GLM. This has been implemented in the glmnet package, which is bundled with PMTK.

13.4.3 Proximal and gradient projection methods

In this section, we consider some methods that are suitable for very large scale problems, where homotopy methods made be too slow. These methods will also be easy to extend to other kinds

of regularizers, beyond ℓ_1 , as we will see later. Our presentation in this section is based on (Vandenberghe 2011; Yang et al. 2010).

Consider a convex objective of the form

$$f(\boldsymbol{\theta}) = L(\boldsymbol{\theta}) + R(\boldsymbol{\theta}) \quad (13.66)$$

where $L(\boldsymbol{\theta})$ (representing the loss) is convex and differentiable, and $R(\boldsymbol{\theta})$ (representing the regularizer) is convex but not necessarily differentiable. For example, $L(\boldsymbol{\theta}) = \text{RSS}(\boldsymbol{\theta})$ and $R(\boldsymbol{\theta}) = \lambda \|\boldsymbol{\theta}\|_1$ corresponds to the BPDN problem. As another example, the lasso problem can be formulated as follows: $L(\boldsymbol{\theta}) = \text{RSS}(\boldsymbol{\theta})$ and $R(\boldsymbol{\theta}) = I_C(\boldsymbol{\theta})$, where $C = \{\boldsymbol{\theta} : \|\boldsymbol{\theta}\|_1 \leq B\}$, and $I_C(\boldsymbol{\theta})$ is the indicator function of a convex set C , defined as

$$I_C(\boldsymbol{\theta}) \triangleq \begin{cases} 0 & \boldsymbol{\theta} \in C \\ +\infty & \text{otherwise} \end{cases} \quad (13.67)$$

In some cases, it is easy to optimize functions of the form in Equation 13.66. For example, suppose $L(\boldsymbol{\theta}) = \text{RSS}(\boldsymbol{\theta})$, and the design matrix is simply $\mathbf{X} = \mathbf{I}$. Then the objective becomes $f(\boldsymbol{\theta}) = R(\boldsymbol{\theta}) + \frac{1}{2} \|\boldsymbol{\theta} - \mathbf{y}\|_2^2$. The minimizer of this is given by $\text{prox}_R(\mathbf{y})$, which is the **proximal operator** for the convex function R , defined by

$$\text{prox}_R(\mathbf{y}) = \underset{\mathbf{z}}{\text{argmin}} \left(R(\mathbf{z}) + \frac{1}{2} \|\mathbf{z} - \mathbf{y}\|_2^2 \right) \quad (13.68)$$

Intuitively, we are returning a point that minimizes R but which is also close (proximal) to $\mathbf{y}$. In general, we will use this operator inside an iterative optimizer, in which case we want to stay close to the previous iterate. In this case, we use

$$\text{prox}_R(\boldsymbol{\theta}_k) = \underset{\mathbf{z}}{\text{argmin}} \left(R(\mathbf{z}) + \frac{1}{2} \|\mathbf{z} - \boldsymbol{\theta}_k\|_2^2 \right) \quad (13.69)$$

The key issues are: how do we efficiently compute the proximal operator for different regularizers R , and how do we extend this technique to more general loss functions L ? We discuss these issues below.

13.4.3.1 Proximal operators

If $R(\boldsymbol{\theta}) = \lambda \|\boldsymbol{\theta}\|_1$, the proximal operator is given by componentwise soft-thresholding:

$$\text{prox}_R(\boldsymbol{\theta}) = \text{soft}(\boldsymbol{\theta}, \lambda) \quad (13.70)$$

as we showed in Section 13.3.2. If $R(\boldsymbol{\theta}) = \lambda \|\boldsymbol{\theta}\|_0$, the proximal operator is given by componentwise hard-thresholding:

$$\text{prox}_R(\boldsymbol{\theta}) = \text{hard}(\boldsymbol{\theta}, \sqrt{2\lambda}) \quad (13.71)$$

where $\text{hard}(u, a) \triangleq u \mathbb{I}(|u| > a)$.

If $R(\boldsymbol{\theta}) = I_C(\boldsymbol{\theta})$, the proximal operator is given by the projection onto the set C :

$$\text{prox}_R(\boldsymbol{\theta}) = \underset{\mathbf{z} \in C}{\text{argmin}} \|\mathbf{z} - \boldsymbol{\theta}\|_2^2 = \text{proj}_C(\boldsymbol{\theta}) \quad (13.72)$$

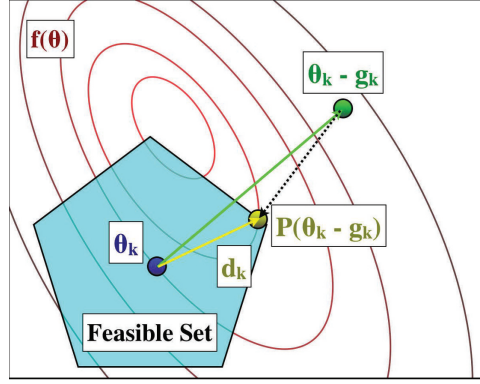


Figure 13.11 Illustration of projected gradient descent. The step along the negative gradient, to $\theta_k - \mathbf{g}_k$, takes us outside the feasible set. If we project that point onto the closest point in the set we get $\theta_{k+1} = \text{proj}_{\Theta}(\theta_k - \mathbf{g}_k)$. We can then derive the implicit update direction using $\mathbf{d}_k = \theta_{k+1} - \theta_k$. Used with kind permission of Mark Schmidt.

For some convex sets, it is easy to compute the projection operator. For example, to project onto the rectangular set defined by the box constraints $C = \{\theta : \ell_j \leq \theta_j \leq u_j\}$ we can use

$$\text{proj}_C(\theta)_j = \begin{cases} \ell_j & \theta_j \leq \ell_j \\ \theta_j & \ell_j \leq \theta_j \leq u_j \\ u_j & \theta_j \geq u_j \end{cases} \quad (13.73)$$

To project onto the Euclidean ball $C = \{\theta : \|\theta\|_2 \leq 1\}$ we can use

$$\text{proj}_C(\theta) = \begin{cases} \frac{\theta}{\|\theta\|_2} & \|\theta\|_2 > 1 \\ \theta & \|\theta\|_2 \leq 1 \end{cases} \quad (13.74)$$

To project onto the 1-norm ball $C = \{\theta : \|\theta\|_1 \leq 1\}$ we can use

$$\text{proj}_C(\theta) = \text{soft}(\theta, \lambda) \quad (13.75)$$

where $\lambda = 0$ if $\|\theta\|_1 \leq 1$, and otherwise λ is the solution to the equation

$$\sum_{j=1}^D \max(|\theta_j| - \lambda, 0) = 1 \quad (13.76)$$

We can implement the whole procedure in $O(D)$ time, as explained in (Duchi et al. 2008).

We will see an application of these different projection methods in Section 13.5.1.2.

13.4.3.2 Proximal gradient method

We now discuss how to use the proximal operator inside of a gradient descent routine. The basic idea is to minimize a simple quadratic approximation to the loss function, centered on the

θ_k :

$$\theta_{k+1} = \underset{\mathbf{z}}{\operatorname{argmin}} R(\mathbf{z}) + L(\theta_k) + \mathbf{g}_k^T(\mathbf{z} - \theta_k) + \frac{1}{2t_k} \|\mathbf{z} - \theta_k\|_2^2 \quad (13.77)$$

where $\mathbf{g}_k = \nabla L(\theta_k)$ is the gradient of the loss, t_k is a constant discussed below, and the last term arises from a simple approximation to the Hessian of the loss of the form $\nabla^2 L(\theta_k) \approx \frac{1}{t_k} \mathbf{I}$.

Dropping terms that are independent of $\mathbf{z}$, and multiplying by t_k , we can rewrite the above expression in terms of a proximal operator as follows:

$$\theta_{k+1} = \underset{\mathbf{z}}{\operatorname{argmin}} \left[t_k R(\mathbf{z}) + \frac{1}{2} \|\mathbf{z} - \mathbf{u}_k\|_2^2 \right] = \operatorname{prox}_{t_k R}(\mathbf{u}_k) \quad (13.78)$$

$$\mathbf{u}_k = \theta_k - t_k \mathbf{g}_k \quad (13.79)$$

$$\mathbf{g}_k = \nabla L(\theta_k) \quad (13.80)$$

If $R(\theta) = 0$, this is equivalent to gradient descent. If $R(\theta) = I_C(\theta)$, the method is equivalent to **projected gradient descent**, sketched in Figure 13.11. If $R(\theta) = \lambda \|\theta\|_1$, the method is known as **iterative soft thresholding**.

There are several ways to pick t_k , or equivalently, $\alpha_k = 1/t_k$. Given that $\alpha_k \mathbf{I}$ is an approximation to the Hessian $\nabla^2 L$, we require that

$$\alpha_k (\theta_k - \theta_{k-1}) \approx \mathbf{g}_k - \mathbf{g}_{k-1} \quad (13.81)$$

in the least squares sense. Hence

$$\alpha_k = \underset{\alpha}{\operatorname{argmin}} \|\alpha(\theta_k - \theta_{k-1}) - (\mathbf{g}_k - \mathbf{g}_{k-1})\|_2^2 = \frac{(\theta_k - \theta_{k-1})^T (\mathbf{g}_k - \mathbf{g}_{k-1})}{(\theta_k - \theta_{k-1})^T (\theta_k - \theta_{k-1})} \quad (13.82)$$

This is known as the **Barzilai-Borwein** (BB) or **spectral** stepsize (Barzilai and Borwein 1988; Fletcher 2005; Raydan 1997). This stepsize can be used with any gradient method, whether proximal or not. It does not lead to monotonic decrease of the objective, but it is much faster than standard line search techniques. (To ensure convergence, we require that the objective decrease “on average”, where the average is computed over a sliding window of size $M + 1$.)

When we combine the BB stepsize with the iterative soft thresholding technique (for $R(\theta) = \lambda \|\theta\|_1$), plus a continuation method that gradually reduces λ , we get a fast method for the BPDN problem known as the SpARSA algorithm, which stands for “sparse reconstruction by separable approximation” (Wright et al. 2009). However, we will call it the iterative shrinkage and thresholding algorithm. See Algorithm 12 for some pseudocode, and SpARSA for some Matlab code. See also Exercise 13.11 for a related approach based on projected gradient descent.

13.4.3.3 Nesterov’s method

A faster version of proximal gradient descent can be obtained by expanding the quadratic approximation around a point other than the most recent parameter value. In particular, consider performing updates of the form

$$\theta_{k+1} = \operatorname{prox}_{t_k R}(\phi_k - t_k \mathbf{g}_k) \quad (13.83)$$

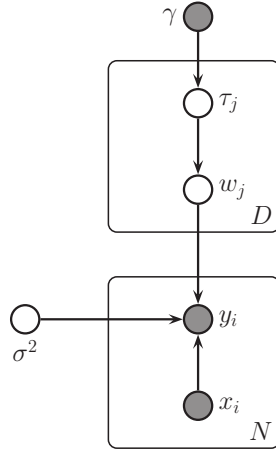
$$\mathbf{g}_k = \nabla L(\phi_k) \quad (13.84)$$

$$\phi_k = \theta_k + \frac{k-1}{k+2} (\theta_k - \theta_{k-1}) \quad (13.85)$$

Algorithm 13.2: Iterative Shrinkage-Thresholding Algorithm (ISTA)

```

1 Input:  $\mathbf{X} \in \mathbb{R}^{N \times D}$ ,  $\mathbf{y} \in \mathbb{R}^N$ , parameters  $\lambda \geq 0$ ,  $M \geq 1$ ,  $0 < s < 1$  ;
2 Initialize  $\boldsymbol{\theta}_0 = \mathbf{0}$ ,  $\alpha = 1$ ,  $\mathbf{r} = \mathbf{y}$ ,  $\lambda_0 = \infty$ ;
3 repeat
4    $\lambda_t = \max(s\|\mathbf{X}^T \mathbf{r}\|_\infty, \lambda)$  // Adapt the regularizer ;
5   repeat
6      $\mathbf{g} = \nabla L(\boldsymbol{\theta})$ ;
7      $\mathbf{u} = \boldsymbol{\theta} - \frac{1}{\alpha} \mathbf{g}$ ;
8      $\boldsymbol{\theta} = \text{soft}(\mathbf{u}, \frac{\lambda_t}{\alpha})$ ;
9     Update  $\alpha$  using BB stepsize in Equation 13.82 ;
10  until  $f(\boldsymbol{\theta})$  increased too much within the past  $M$  steps;
11   $\mathbf{r} = \mathbf{y} - \mathbf{X}\boldsymbol{\theta}$  // Update residual ;
12 until  $\lambda_t = \lambda$ ;
```

**Figure 13.12** Representing lasso using a Gaussian scale mixture prior.

This is known as **Nesterov's method** (Nesterov 2004; Tseng 2008). As before, there are a variety of ways of setting t_k ; typically one uses line search.

When this method is combined with the iterative soft thresholding technique (for $R(\boldsymbol{\theta}) = \lambda \|\boldsymbol{\theta}\|_1$), plus a continuation method that gradually reduces λ , we get a fast method for the BPDN problem known as the **fast iterative shrinkage thresholding algorithm** or **FISTA** (Beck and Teboulle 2009).

13.4.4 EM for lasso

In this section, we show how to solve the lasso problem using lasso. At first sight, this might seem odd, since there are no hidden variables. The key insight is that we can represent the Laplace distribution as a **Gaussian scale mixture** (GSM) (Andrews and Mallows 1974; West 1987) as follows:

$$\text{Lap}(w_j|0, 1/\gamma) = \frac{\gamma}{2} e^{-\gamma|w_j|} = \int \mathcal{N}(w_j|0, \tau_j^2) \text{Ga}(\tau_j^2|1, \frac{\gamma^2}{2}) d\tau_j^2 \quad (13.86)$$

Thus the Laplace is a GSM where the mixing distribution on the variances is the exponential distribution, $\text{Expon}(\tau_j^2|\frac{\gamma^2}{2}) = \text{Ga}(\tau_j^2|1, \frac{\gamma^2}{2})$. Using this decomposition, we can represent the lasso model as shown in Figure 13.12. The corresponding joint distribution has the form

$$p(\mathbf{y}, \mathbf{w}, \boldsymbol{\tau}, \sigma^2|\mathbf{X}) = \mathcal{N}(\mathbf{y}|\mathbf{X}\mathbf{w}, \sigma^2\mathbf{I}_N) \mathcal{N}(\mathbf{w}|\mathbf{0}, \mathbf{D}_\tau) \text{IG}(\sigma^2|a_\sigma, b_\sigma) \left[\prod_j \text{Ga}(\tau_j^2|1, \gamma^2/2) \right] \quad (13.87)$$

where $\mathbf{D}_\tau = \text{diag}(\tau_j^2)$, and where we have assumed for notational simplicity that $\mathbf{X}$ is standardized and that $\mathbf{y}$ is centered (so we can ignore the offset term μ). Expanding out, we get

$$\begin{aligned} p(\mathbf{y}, \mathbf{w}, \boldsymbol{\tau}, \sigma^2|\mathbf{X}) &\propto (\sigma^2)^{-N/2} \exp\left(-\frac{1}{2\sigma^2} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2\right) |\mathbf{D}_\tau|^{-\frac{1}{2}} \\ &\quad \exp\left(-\frac{1}{2} \mathbf{w}^T \mathbf{D}_\tau \mathbf{w}\right) (\sigma^2)^{-(a_\sigma+1)} \\ &\quad \exp(-b_\sigma/\sigma^2) \prod_j \exp\left(-\frac{\gamma^2}{2} \tau_j^2\right) \end{aligned} \quad (13.88)$$

Below we describe how to apply the EM algorithm to the model in Figure 13.12.⁵ In brief, in the E step we infer τ_j^2 and σ^2 , and in the M step we estimate $\mathbf{w}$. The resulting estimate $\hat{\mathbf{w}}$ is the same as the lasso estimator. This approach was first proposed in (Figueiredo 2003) (see also (Griffin and Brown 2007; Caron and Doucet 2008; Ding and Harrison 2010) for some extensions).

13.4.4.1 Why EM?

Before going into the details of EM, it is worthwhile asking why we are presenting this approach at all, given that there are a variety of other (often much faster) algorithms that directly solve the ℓ_1 MAP estimation problem (see `linregFitL1Test` for an empirical comparison). The reason is that the latent variable perspective brings several advantages, such as the following:

- It provides an easy way to derive an algorithm to find ℓ_1 -regularized parameter estimates for a variety of other models, such as robust linear regression (Exercise 11.12) or probit regression (Exercise 13.9).

5. To ensure the posterior is unimodal, one can follow (Park and Casella 2008) and slightly modify the model by making the prior variance for the weights depend on the observation noise: $p(w_j|\tau_j^2, \sigma^2) = \mathcal{N}(w_j|0, \sigma^2\tau_j^2)$. The EM algorithm is easy to modify.